

Layer Zero समझाना

2 СЕНТЯБРЯ 2026 Г.

YOU 23/08/2026, 20:51:57

iss file ko read kro
ek example lo aur uss example se mujhe iss file me diya gya layer wise concept ko smjhao
mujhe ek layer ek time pr smjhao
start kro tum layer zero se
har layer me uska input kya hoga uska output kya hoga uss layer me kya kya ho rha hai aur kese ho rha hai kya technology use
ho rhi hai ye sb btao

DEEP RESEARCH 23/08/2026, 20:52:09

Make sure to include `filecite\turn0file0\5-L8` in your response to cite this file, or to surface it as a link. Replace the placeholder line range with the actual line range from the result that best fits the snippet you want to surface with the citation.

Use `ref_id "turn0file0"` to reference this file. For broad questions or unknown locations, retrieve relevant chunks with `files.search` scoped to this file. Use `files.find` only for exact text, and use `files.read` for a known range or to expand relevant results. Full file size: 2067 file lines.

[L1] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L3] > **Project Type:** Minor Project (M.Tech)

[L4] > **Date:** August 2026

[L5] > **Author:** [Your Name]

[L9] ## Abstract

[L11] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L13] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L15] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L17] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L19] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

- [L23] ## Table of Contents
- [L25] 1. Introduction
- [L26] 2. Literature Review
- [L27] 3. System Architecture
- [L28] 4. Data Model — Layer 0
- [L29] 5. Offline Pipeline — Layers 1-5
- [L30] 6. Online Pipeline — Layers 6-7
- [L31] 7. Operations Layer — Layer 8
- [L32] 8. Novel Contributions
- [L33] 9. Mathematical Framework
- [L34] 10. Implementation
- [L35] 11. Evaluation Framework
- [L36] 12. Risk Analysis
- [L37] 13. Conclusion & Future Work

[L41] ## 1. Introduction

[L43] ### 1.1 Problem Statement

[L45] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

- [L47] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)
- [L48] 2. Embed each chunk into a vector space
- [L49] 3. At query time, retrieve the top-k most similar chunks
- [L50] 4. Feed these chunks as context to an LLM for answer generation

[L52] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L54] | Problem | Description | Impact |

[L55] |-----|-----|-----|

[L56] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L57] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L58] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L59] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |

[L60] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L62] ### 1.2 Proposed Solution: SHIA-RAG

[L64] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L67] Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L68] SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM

[L71] The system:

- [L72] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents
- [L73] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees
- [L74] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L75] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L76] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L77] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L79] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L81] ### 1.3 Objectives

[L83] 1. Design a complete 8-layer architecture for hierarchical RAG

[L84] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L85] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L86] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L87] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L88] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L90] ### 1.4 Contributions

[L92] This work makes five novel contributions to the RAG literature:

[L94] | # | Contribution | Novelty Level |

[L95] |---|-----|-----|

[L96] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L97] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L98] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L99] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L100] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L104] ## 2. Literature Review

[L106] ### 2.1 Related Work

[L108] ##### 2.1.1 Flat-Chunk RAG Systems

[L110] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L112] ##### 2.1.2 Structured RAG Systems

[L114] Recent work has begun exploring structured retrieval:

[L116] | System | Approach | Key Limitation |

[L117] |-----|-----|-----|

[L118] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L119] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L120] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L121] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L123] ##### 2.1.3 Knowledge Graph Construction

[L125] Ontology learning and taxonomy induction have a rich history in NLP:

[L126] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L127] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L128] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L130] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L132] ### 2.2 Research Gap

[L135] [L136] System	Structured Retrieval?	Concept-Level Extraction?	Self- Evolving?	Hierarchy-Aware Context Selection?
[L137]				
[L138] Traditional RAG	x	x	x	x
[L139] RAPTOR	✓ (summaries)	x	x	x
[L140] GraphRAG	✓ (subgraph)	✓ (partial)	x	x
[L141] TreeRAG	✓ (tree)	x	x	x
[L142] HiChunk	✓ (chunks)	x	x	x
[L143]				
[L144] SHIA-RAG (Ours)	✓ (forest)	✓ (full)	✓	✓

[L147] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L149] ### 2.3 Literature Search Methodology

[L151] | Parameter | Details |

[L152] |-----|-----|

[L153] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L154] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L155] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L156] | **Time Range** | 2020–2026 |

[L157] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L161] ## 3. System Architecture

[L163] ### 3.1 Overview

[L165] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L167] | Phase | Layers | Mode | Purpose |

[L168] |-----|-----|-----|-----|

[L169] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L170] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L171] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L173] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L175]


```
[L176] graph TD
[L177]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L178]         L1["Layer 1: Input Processing<br>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br>Text normalization, tokenization"]
[L179]         L2 --> L3["Layer 3: Knowledge Extraction<br>Concept detection, alias resolution"]
[L180]         L3 --> L4["Layer 4: Relation Management<br>Triple extraction, confidence scoring"]
[L181]         L4 --> L5["Layer 5: SHIA Core<br>CPG++ → PRE → HV → FI → CLD"]
[L182]         L5 --> KF["Knowledge Forest Store<br>(Graph DB + Vector DB)"]
[L182]     end

[L185]     subgraph "Phase 2: Online – Retrieval & Generation"
[L186]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br>Query analysis, forest traversal, RB0"]
[L187]         KF --> L6
[L188]         L6 --> L7["Layer 7: Generation Engine<br>Prompt assembly, LLM, verification"]
[L189]         L7 --> A["Answer + Citations"]
[L189]     end

[L192]     subgraph "Phase 3: Continuous – Operations"
[L193]         L8["Layer 8: Operations<br>Logging, monitoring, feedback loop"]
[L194]         A --> L8
[L195]         L8 -->|"Self-Evolution<br>Feedback"| KF
[L195]     end
```

[L199] ### 3.2 Data Flow

```
[L202] Raw Document (PDF/DOCX/HTML)
[L203]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L204] Normalized Text + Metadata
[L205]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L206] [Page → Region → Block] hierarchy
[L207]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L208] KnowledgeUnit objects (candidate concepts)
[L209]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L210] KnowledgeNode objects (canonical, deduplicated)
[L211]     ↓ Layer 4: Validate relations against ontology, score confidence
[L212] KnowledgeEdge objects (typed, scored)
[L213]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L214] Knowledge Forest (rooted trees + cross-links)
[L215]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L216] Selected context nodes (within token budget)
[L217]     ↓ Layer 7: Build prompt, invoke LLM
[L218]     ↓ Layer 7: Verify claims against evidence, bind citations
[L219] Answer with citations + confidence score
[L220]     ↓ Layer 8: Log, collect feedback, update forest weights
```

[L223] ### 3.3 Layer Summary

[L225] | Layer | Name | Input | Output | Key Algorithms |

[L226] |-----|-----|-----|-----|-----|

[L227] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L228] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L229] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L230] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L231] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L232] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L233] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L234] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L235] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L239] ## 4. Data Model — Layer 0

[L241] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` (concepts) and `KnowledgeEdge` (relationships).

[L243] ### 4.1 KnowledgeNode Schema

[L245] Each concept in the Knowledge Forest is represented as a `KnowledgeNode` :

[L247] ```json

{

[L249] "node_id": "NODE_000789",

[L250] "uuid": "550e8400-e29b-41d4-a716-446655440000",

[L251] "canonical_name": "Round Robin",

[L252] "aliases": ["RR", "Round-Robin Scheduling"],

[L253] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],

[L254] "summaries": ["CPU scheduling method using time slices"],

[L255] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],

[L256] "domain": "Operating Systems",

[L257] "properties": {

[L258] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},

[L259] "preemptive": {"type": "boolean", "value": true}

},

[L261] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],

[L262] "parent": "NODE_000456",

[L263] "children": ["NODE_001001", "NODE_001002"],

[L264] "related": ["NODE_000890"],

[L265] "incoming_edges": ["EDGE_001"],

[L266] "outgoing_edges": ["EDGE_002", "EDGE_003"],

[L

CHATGPT 23/08/2026, 20:52:09

from pathlib import Path

p=Path('/mnt/data/SHIA_RAG_Final_Report(4)(1).md')

text=p.read_text(encoding='utf-8')

len(text.splitlines()), text[:500]

DEEP RESEARCH 23/08/2026, 20:52:26

(2066,

'# SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation\n\n> **Project Type:** Minor Project (M.Tech)\n\n> **Date:** August 2026\n\n> **Author:** [Your Name]\n\n---\n\n\n## Abstract\n\nRetrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar')

CHATGPT 23/08/2026, 20:52:31

Bilkul. Main **isi uploaded file ko basis bana kar** tumhe poora SHIA-RAG architecture **layer by layer** samjhaunga — aur **har baar sirf ek layer**. Hum ek **same running example** use karenge, taaki Layer 0 se Layer 8 tak data ka flow clearly samajh aaye.

File ke according, **Layer 0 koi processing layer nahi hai**. Ye poore system ka **foundation/data model** define karta hai: har layer kis format ka data use karegi, concept kya hoga, relation kya hoga, aur system ka Knowledge Forest internally kaisa dikhega.

📄filecite🔗turn0file0🔗L225-L234🔗 📄filecite🔗turn0file0🔗L239-L243🔗

Layer 0 — Data Model Foundation

1. Sabse pehle ek bada example fix karte hain

Maan lo hum SHIA-RAG me ye **Operating Systems ka document** upload karte hain.

Document me ye information hai:

Operating System manages computer resources.

CPU Scheduling is a mechanism used by an operating system to decide which process gets CPU time.

Round Robin is a preemptive CPU scheduling algorithm.

*Round Robin gives each process a fixed **time quantum**, for example **20 ms**.*

Round Robin is a type of CPU Scheduling Algorithm.

Round Robin uses Time Quantum.

Preemptive Scheduling contrasts with Non-Preemptive Scheduling.

Abhi dhyar do:

Raw document me kya hai?

Sirf text:

```
Operating System
↓
CPU Scheduling
↓
Round Robin
↓
Time Quantum = 20 ms
```

Lekin computer ko sirf text de dena enough nahi hai.

System ko internally pata hona chahiye:

- "Round Robin" kya hai?
- Iske alternate names kya hain?
- Iski definition kya hai?
- Iski properties kya hain?
- Iska parent kaun hai?
- Iske children kaun hain?
- Iske related concepts kaun hain?
- Ye information kis document aur kis page se aayi?
- Is information par kitna trust/confidence hai?

Ye sab define karna Layer 0 ka kaam hai.

2. Layer 0 ka main purpose kya hai?

Simple language me:

Layer 0 poore SHIA-RAG system ka database contract/schema hai.

Analogy:

Agar tum Hospital Management System bana rahe ho, to pehle tum decide karte ho:

```
Patient object me:
- patient_id
- name
- age
- disease
```

Similarly, SHIA-RAG me pehle decide karna padta hai:

```
KnowledgeNode me:
- concept ka naam
- definition
- aliases
- properties
- evidence
- embedding
- confidence
...
```

Aur relation ke liye:

```
KnowledgeEdge me:
- source concept
- target concept
- relation type
- confidence
- evidence
...
```

File ke according Layer 0 ke do core data structures hain:

1. **KnowledgeNode** → Knowledge/concept ko represent karta hai.
2. **KnowledgeEdge** → Do concepts ke beech relationship represent karta hai. `filecite\turn0file0\L241-L243`

3. Layer 0 ka Input kya hai?

Yahan ek important point hai:

Layer 0 ka traditional input nahi hota ×

File ke layer summary me bhi Layer 0 ka input — diya gaya hai aur output **Schemas** hai. `filecite\turn0file0\L225-L228`

Matlab:

```
Layer 1:
Raw File input lega
      ↓
Layer 2:
NormalizedDocument input lega
```

Lekin Layer 0:

```
Actual data process nahi karta

Layer 0
      ↓
Define karta hai ki data KIS FORMAT me
Layer 1 → Layer 2 → Layer 3 → ... me flow karega
```

Isliye Layer 0 ka conceptual input:

System requirements + data design requirements

Output:

KnowledgeNode Schema
KnowledgeEdge Schema
Relation Ontology
Confidence Rules
Validation Rules

4. KnowledgeNode kya hai?

Ab hamare example par aate hain.

Document se ek concept mila:

Round Robin

Layer 3 future me isse extract karegi.

Lekin extract hone ke baad system ise kis structure me store karega?

Layer 0 pehle se define karta hai:

KnowledgeNode

File me example roughly is concept ko represent karta hai:

```
KnowledgeNode
|
├─ node_id
├─ uuid
├─ canonical_name
├─ aliases
├─ definitions
├─ summaries
├─ keywords
├─ domain
├─ properties
├─ examples
├─ parent
├─ children
├─ related
├─ evidence
├─ embeddings
├─ confidence
└─ abstraction_level
```

Ye structure directly file ke KnowledgeNode schema se aata hai. filecite.com/turn0file0L247-L285/

5. Hamare example me Round Robin ka KnowledgeNode

Maan lo future layers document process karke ye information nikalti hain.

Then internally:

```
{
  "node_id": "NODE_RR_001",

  "canonical_name": "Round Robin",

  "aliases": [
    "RR",
    "Round-Robin Scheduling"
  ],

  "definitions": [
    "A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order."
  ],

  "keywords": [
    "CPU scheduling",
    "time quantum",
    "preemptive"
  ],

  "domain": "Operating Systems",

  "properties": {
    "time_quantum": {
      "type": "integer",
      "value": 20,
      "unit": "ms"
    },

    "preemptive": {
      "type": "boolean",
      "value": true
    }
  }
}
```

Ab ek-ek field samajhte hain.

6. `canonical_name` — Actual standard naam

Round Robin

Problem ye hai ki same concept ko document me different ways me likha ja sakta hai:

Round Robin
RR
Round-Robin
Round Robin Scheduling

Agar system in sabko alag-alag concept samajh le:

Node 1 → Round Robin
Node 2 → RR
Node 3 → Round Robin Scheduling

to duplicate knowledge create ho jayega.

Isliye:

`canonical_name` = "Round Robin"

Ye concept ka **main official identity** hai.

7. `aliases` — Same concept ke dusre naam

```
aliases = [
    "RR",
    "Round-Robin Scheduling"
]
```

Example:

User query:

What is RR scheduling?

System ko ideally samajhna chahiye:

```
RR
↓
Alias resolution
↓
Round Robin
```

To retrieval fail nahi hogi.

Behind the scenes idea:

```
User Query: "RR algorithm kya hai?"

"RR"
↓
Alias match
↓
Canonical Concept:
"Round Robin"
↓
Retrieve knowledge
```

8. definitions — Concept actually kya hai?

```
Round Robin =
A preemptive CPU scheduling algorithm...
```

Ye bahut important hai.

Kyunki naam:

```
Round Robin
```

khud enough information nahi deta.

Definition semantic meaning store karti hai.

Later jab query aayegi:

What is Round Robin?

System directly node ki definition use kar sakta hai.

```
KnowledgeNode
├── Name → Round Robin
└── Definition → actual meaning
```

9. keywords — Retrieval ko help karne wale terms

```
[
  "scheduling",
  "time quantum",
  "preemptive",
  "CPU"
]
```

Maan lo user directly "Round Robin" nahi bolta.

User poochta hai:

Which scheduling algorithm gives each process a fixed time slice?

Ab query me:

```
fixed time slice
process
algorithm
```

ho sakta hai.

Keywords semantic matching ko additional support de sakte hain.

10. **domain** — Ye knowledge kis subject se related hai?

```
domain = "Operating Systems"
```

Kyun zaroori hai?

Same word alag domains me different meaning rakh sakta hai.

Example:

```
Process
```

Operating Systems me:

```
Running program
```

Business domain me:

```
Business workflow
```

Manufacturing me:

```
Production process
```

Isliye node ko domain assign karna important hai.

Hamare example me:

```
Round Robin
  ↓
Operating Systems domain
```

Later hierarchy construction me bhi domain routing use hoti hai, jisse unrelated domains ke concepts parent candidates nahi banenge. File ka architecture domain-aware knowledge organization par based hai. filecite.org/turn0file0/L1017-L1028/

11. **properties** — Concept ki structured attributes

Round Robin ke liye:


```

"properties": {
  "time_quantum": {
    "type": "integer",
    "value": 20,
    "unit": "ms"
  },
  "preemptive": {
    "type": "boolean",
    "value": true
  }
}

```

Iska benefit kya hai?

Normal text:

Round Robin usually uses a fixed time quantum.

Structured data:

```

Property:
time_quantum

Value:
20

Unit:
ms

```

Ab system sirf paragraph retrieve nahi karta; structured knowledge bhi rakhta hai.

Example query:

Is Round Robin preemptive?

Directly:

```

node.properties.preemptive
      ↓
true

```

12. examples — Concept ko practical context dena

Example:

```

P1 → 20 ms
P2 → 20 ms
P3 → 20 ms
P1 → again

```

Node me:

```

examples = [
  "P1 gets 20ms, then P2 gets 20ms..."
]

```

Definition aur example alag cheezein hain:

| Field | Purpose |

|---|---|

| Definition | Concept kya hai |

| Property | Uski attribute/value |

| Example | Real situation me kaise kaam karta hai |

13. evidence — Ye knowledge aaya kahan se?

Ye SHIA-RAG ka bahut important part hai.

Maan lo system ke paas:

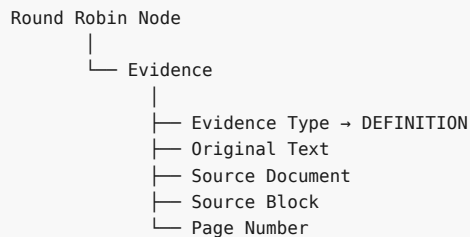
Concept:
Round Robin

Lekin koi pooche:

Tumne ye definition kahan se li?

System ko source trace karna chahiye.

Isliye evidence store hota hai:



File ke schema me evidence original text ke saath `source_doc` , `source_block` , aur `page` jaisi provenance information rakhta hai. `filecite\turn0file0\L267-L274`

Example:

```

Evidence:

Text:
"Round Robin is a preemptive scheduling algorithm..."

Source:
OperatingSystems.pdf

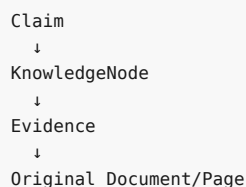
Page:
42
  
```

Why important?

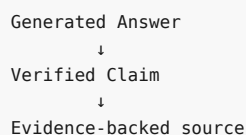
Later Layer 7 answer generate karegi:

Round Robin is a preemptive CPU scheduling algorithm.

Phir system citation/evidence binding kar sakta hai:



So:



14. embedding — Node ko vector me convert karna

Computer directly concept ka meaning mathematical form me represent karne ke liye:

```
Round Robin
+
Definition
```

ko embedding model me bhejega.

Output:

```
[
  0.123,
 -0.456,
  0.782,
  ...
]
```

File ke example me embedding model aur vector dimension bhi node metadata ka part hain, jaise `bge-base-en-v1.5` aur 768-dimensional vector. `filecite\turn0file0\L276-L283`

Conceptually:

```
"Round Robin is a CPU scheduling algorithm"
      ↓
    Embedding Model
      ↓
  [0.12, -0.45, 0.78, ...]
      ↓
    Vector Index
```

Later user query:

```
| How does time slice scheduling work?
```

```
Query
  ↓
Query Embedding
  ↓
Vector similarity
  ↓
Round Robin node may be found
```

Important: Layer 0 embedding create nahi kar rahi.

Layer 0 sirf ye contract define karti hai:

```
| Har KnowledgeNode ke paas embedding ko store karne ka standard format kya hoga.
```

15. confidence — System ko knowledge par kitna trust hai?

Example:

```
confidence = 0.92
```

Meaning:

```
92% confidence
```

Lekin confidence randomly nahi aata.

File ke according confidence me factors include hote hain:

```

Node's extraction confidence
  +
Parent confidence
  +
Evidence confidence
  ↓
Final propagated confidence

```

Specified weighting:

```

40% → Node extraction confidence
40% → Parent confidence
20% → Evidence confidence

```

Aur confidence propagation **topological ordering / Kahn's algorithm** se roots se children ki taraf hoti hai, taaki child stale parent confidence use na kare. `filecite\turn0file0\L370-L410`

Hamare example:

```

CPU Scheduling
Confidence = 0.95
  |
  | Parent influence
  ▼
Round Robin
Extraction confidence = 0.90
Evidence confidence = 0.95
  ↓
Final confidence calculated

```

So confidence ek fixed static value nahi hai; hierarchy ke context se bhi influence ho sakta hai.

16. `abstraction_level` — Concept kitna general ya specific hai?

File me:

```

0.0 = very specific
1.0 = very abstract

```

Example:

```

Operating System
  abstraction ≈ 0.95

CPU Scheduling
  abstraction ≈ 0.70

Round Robin
  abstraction ≈ 0.35

P1 gets 20 ms
  abstraction ≈ 0.10

```

Hierarchy:

```

MORE ABSTRACT
  ↑
Operating System
  |
CPU Scheduling
  |
Round Robin
  |
20 ms time quantum example
  ↓
MORE SPECIFIC

```

Why needed?

Later Layer 5 jab hierarchy banayegi, system ko decide karna hoga:

"Round Robin" ka parent kaun ho sakta hai?

Correct:

CPU Scheduling

Incorrect:

Time Quantum

Abstraction level ek signal provide karta hai:

```

Parent should generally be
MORE ABSTRACT

Child should generally be
MORE SPECIFIC

```

17. Ab aate hain doosre main object par: KnowledgeEdge

KnowledgeNode = "Thing kya hai?"

KnowledgeEdge = "Do things ka relation kya hai?"

Hamare concepts:

```

[Operating System]
[CPU Scheduling]
[Round Robin]
[Time Quantum]
[Preemptive Scheduling]

```

Inke beech connections hain.

Example:

```

Round Robin
  IS_A
CPU Scheduling Algorithm

```

Is relationship ko KnowledgeEdge represent karega.

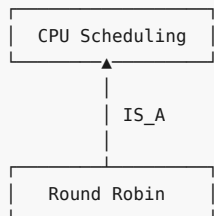
File me edge schema me source node, target node, edge class, relation type, confidence, weight aur evidence jaise fields defined hain. <https://filecite.io/turn0file0/L296-L315/>

18. Hamare example ka KnowledgeEdge

Conceptually:

```
{
  "edge_id": "EDGE_001",
  "source_node": "Round Robin",
  "target_node": "CPU Scheduling",
  "edge_class": "HIERARCHICAL",
  "relation_type": "IS_A",
  "confidence": 0.95,
  "weight": 1.0
}
```

Visual form:



19. Layer 0 me do types ke Edges hain

Type 1: HIERARCHICAL Edge 🌳

Ye actual tree structure banati hai.

Examples:

```
IS_A
PART_OF
SUBCLASS_OF
INSTANCE_OF
```

Hamare example:

```
Round Robin
  IS_A
CPU Scheduling Algorithm
```

File ke according hierarchical edges ko **acyclic forest** maintain karna hota hai aur node ke liye single-parent constraint enforce hota hai. `filecite[turn0file0]L298-L303`

Matlab:

Ye allowed hai:

```
Operating System
  |
CPU Scheduling
  |
Round Robin
```

Ye dangerous hai:

```

Round Robin
  ↓
CPU Scheduling
  ↓
Round Robin
  
```

Ye **cycle** ban gayi ✕

20. Type 2: SEMANTIC Edge

Ye tree hierarchy nahi banati.

Ye extra semantic relationships hain.

Examples:

```

USES
CAUSES
DEPENDS_ON
CONTRASTS_WITH
SIMILAR_TO
HAS_PROPERTY
PRODUCES
PREREQUISITE
  
```

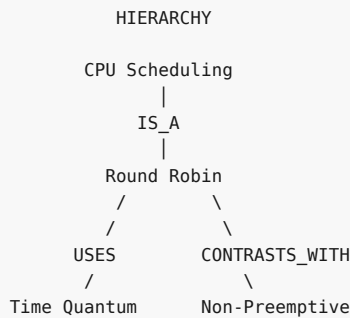
File ki relation ontology exactly hierarchical aur semantic relation types ko separate karti hai. [filecite0turn0file0L348-L367](#)

Hamare example:

```

Round Robin
  |
  ├── USES → Time Quantum
  |
  └── CONTRASTS_WITH → Some Non-Preemptive Approach
  
```

Visualize:



Yahan:

```

Vertical relation = hierarchy
Side relation = semantic relationship
  
```

Ye distinction bahut important hai.

21. HIERARCHICAL edge me strict validation kyun?

Maan lo galti se system ne banaya:

```

Round Robin
  ↓ IS_A
CPU Scheduling

CPU Scheduling
  ↓ IS_A
Round Robin

```

Ab cycle ban gayi.

Traversal kabhi-kabhi infinite loop me ja sakti hai:

```

Round Robin
  ↓
CPU Scheduling
  ↓
Round Robin
  ↓
CPU Scheduling
  ↓
∞

```

Isliye Layer 0 level par rules define kiye gaye hain:

Validation rules:

1. Source node exist karna chahiye.
2. Target node exist karna chahiye.
3. Hierarchical edge cycle create nahi kar sakti.
4. Single-parent constraint respect hona chahiye.
5. Semantic edges me cycles allowed ho sakti hain.

File ke edge validation logic me ye checks explicitly defined hain. `filecite\turn0File0\L319-L345`

22. `parent` aur `children` fields ka ek important design point

Tum soch sakte ho:

Agar edges already relation store kar rahe hain, to `parent` aur `children` alag se kyu?

Example:

```

KnowledgeEdge:
CPU Scheduling → Round Robin

```

Aur node ke andar:

```

Round Robin.parent = CPU Scheduling

```

Ye duplicate information hai.

File ka important design decision:

`parent` aur `children` convenience-only fields hain; production me source of truth actual edges honi chahiye.

Kyunki otherwise inconsistency ho sakti hai:

```

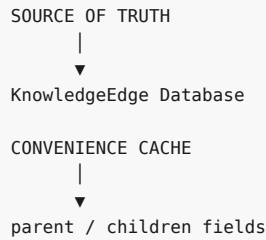
Node.parent = CPU Scheduling

But actual Edge says:
Round Robin → Operating System

```

Ab system confused ×

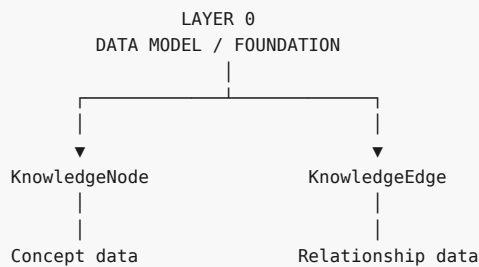
Isliye:



Ye file me explicitly stated hai. `filecite\turn0file0\L290-L294`

23. Ab poora Layer 0 ek saath visualize karo

Hamare document ke liye:



Nodes:

NODE 1
Operating System

 NODE 2
CPU Scheduling

 NODE 3
Round Robin

 NODE 4
Time Quantum

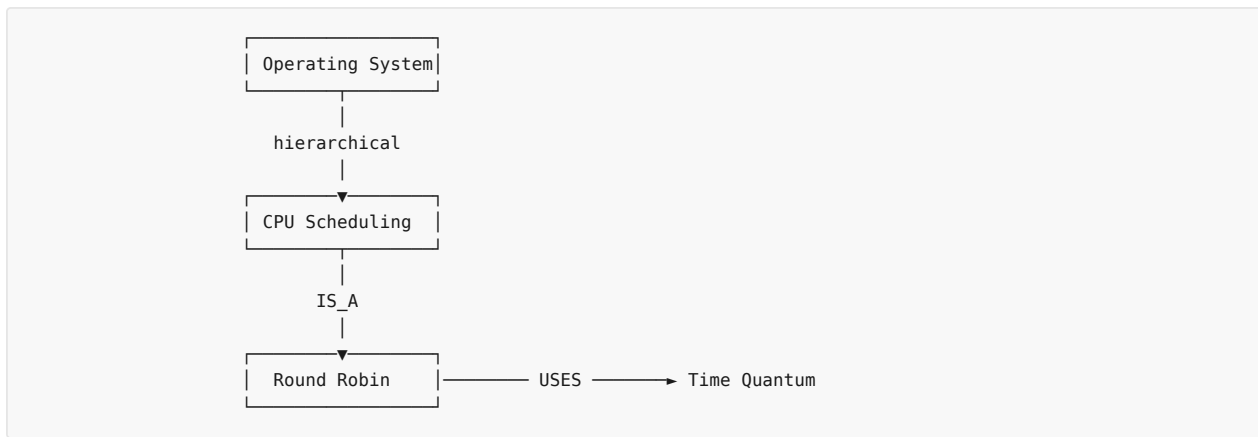
Hierarchical Edges:

Round Robin
| IS_A
▼
CPU Scheduling

Semantic Edges:

Round Robin —USES→ Time Quantum

Finally:



24. Layer 0 me internally exactly kya ho raha hai?

Technically:

Step 1 — Schema define hota hai

```

KnowledgeNode ka structure define
KnowledgeEdge ka structure define
  
```

Step 2 — Allowed relation types define hote hain

```

IS_A
PART_OF
USES
CAUSES
...
  
```

Step 3 — Structural rules define hote hain

```

Hierarchical:
✓ No cycle
✓ Single parent
✓ Forest structure

Semantic:
✓ Graph structure
✓ Cycles allowed
  
```

Step 4 — Confidence rules define hote hain

```

Extraction confidence
+
Parent confidence
+
Evidence confidence
  
```

Step 5 — Data contract ready hota hai

Ab next layers ko clear hai:

```
Layer 3:
Mujhe KnowledgeNode banana hai

Layer 4:
Mujhe KnowledgeEdge banana hai

Layer 5:
Mujhe Nodes + Edges se Knowledge Forest banana hai

Layer 6:
Mujhe Knowledge Forest traverse karna hai

Layer 7:
Mujhe Evidence use karke answer verify karna hai

Layer 8:
Mujhe edges/feedback update karna hai
```

25. Layer 0 ki Technology kya hai?

Layer 0 ek single technology nahi hai.

Ye **data architecture** hai.

Implementation me file ke concepts ke hisaab se likely components:

- | Requirement | Role |
- |---|---|
- | Data classes / schemas | KnowledgeNode , KnowledgeEdge structure |
- | Graph database | Nodes aur relationships store/traverse |
- | Vector index/database | Embeddings ke basis par semantic search |
- | Topological processing | Confidence propagation |
- | Kahn's Algorithm | Parent-first confidence ordering |
- | UUID / IDs | Unique node identity |
- | Validation layer | Cycle aur node existence checks |

File specifically confidence propagation ke liye **Kahn's algorithm / topological sort** define karti hai. [filecite turn0file0L370-L390](#)

26. Layer 0 ka Output kya hai?

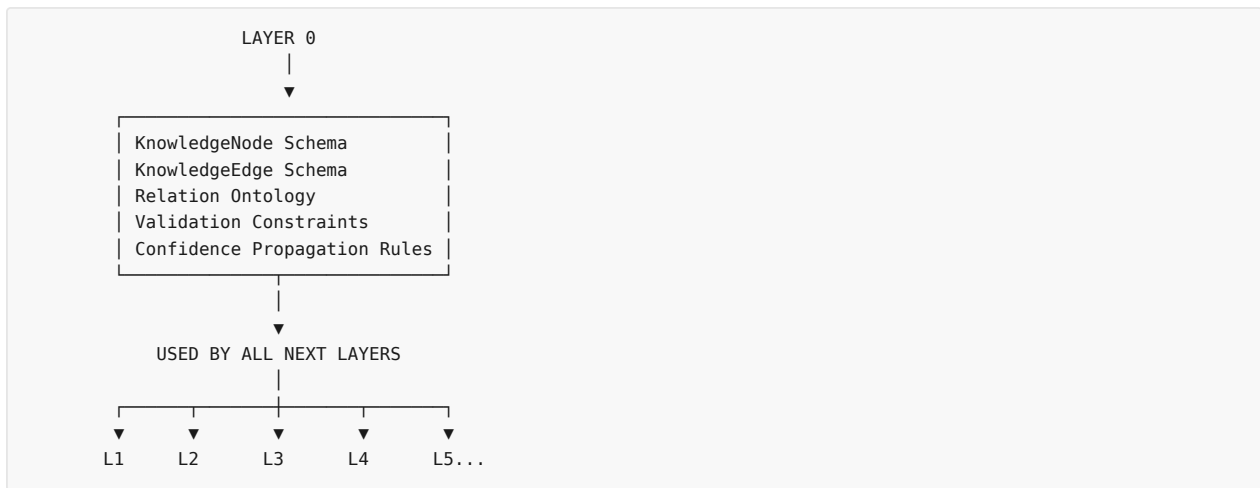
Final output:

```
SCHEMA CONTRACT
```

Specifically:

1. KnowledgeNode Schema
2. KnowledgeEdge Schema
3. Relation Ontology
4. Edge Validation Rules
5. Confidence Propagation Rules

Visual:



27. Sabse important: Layer 0 ke baad actual example ka state kya hai?

Abhi actual document process nahi hua hai.

Isliye:

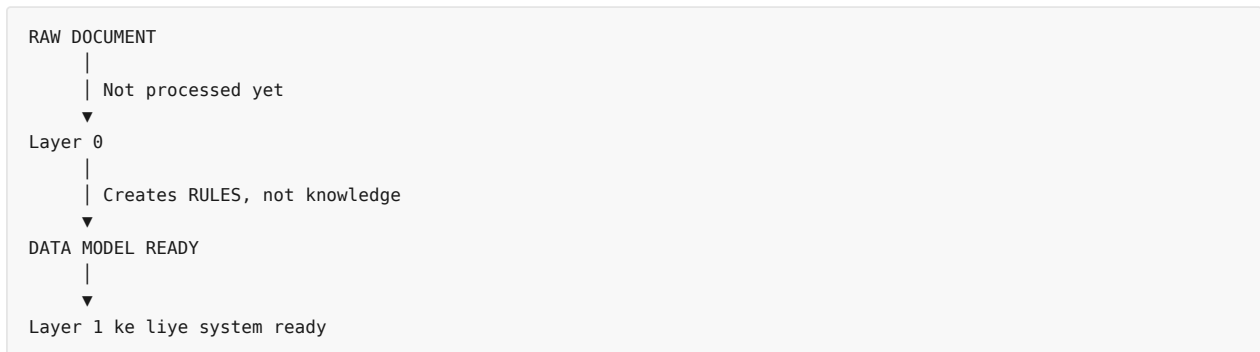
Round Robin Node

abhi actual database me create hona zaroori nahi.

Layer 0 sirf ye bol rahi hai:

"Jab future me Layer 3 Round Robin extract karegi, to usse is exact type ke structure me represent karna hoga."

So current flow:



Layer 0 — Final Summary

| Point | Answer |

|---|---|

| **Layer ka naam** | Data Model |

| **Input** | Direct runtime input nahi |

| **Main purpose** | Poore SHIA-RAG ke data structures aur contracts define karna |

| **Main output** | KnowledgeNode + KnowledgeEdge schemas |

| **KnowledgeNode** | Ek concept ko represent karta hai |

| **KnowledgeEdge** | Do concepts ka relation represent karta hai |

| **Hierarchy** | Tree/Forest structure |

| **Semantic relations** | Cross-links/graph relationships |

| **Main algorithm** | Confidence propagation ke liye Kahn's topological sort |

| **Main safeguards** | Cycle prevention, node existence, single-parent constraint |

Hamare example me node	Round Robin
Parent relation	Round Robin → IS_A → CPU Scheduling
Semantic relation	Round Robin → USES → Time Quantum
Evidence	Original document/page/block se traceability
Next layer	Layer 1 actual raw document ko process karegi

Ek line me yaad rakho:

Layer 0 knowledge ko extract nahi karti — Layer 0 pehle se define karti hai ki extract hone wala knowledge computer ke andar kis structure aur rules ke saath represent hoga.

Next step logically Layer 1 hoga, jahan hum isi **Operating Systems / Round Robin document** ko actual input file maan kar dekhenge: file system me enter hoti hai, validation kaise hota hai, PDF/DOCX detect kaise hota hai, OCR kab use hota hai, duplicate check kaise hota hai, aur output **NormalizedDocument** exactly kaisa banta hai.

YOU 23/08/2026, 21:08:29

mujhe layer zero mere guide ko smjhana hai
mujhe basic understanding deni hai mere sir ko
toh tum mujhe layer ko english me batao lekin brife me
sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 23/08/2026, 21:08:31

```
{"size":"1024x1024","n":1,"transparent_background":false,"is_style_transfer":false,"prompt":null}
```

CHATGPT 23/08/2026, 21:09:11

Layer 0 – Data Model Foundation

You can explain it to your guide like this:

“Layer 0 is the foundation of the SHIA-RAG architecture. It does not process documents or retrieve information. Instead, it defines how knowledge will be represented, connected, and validated throughout the entire system.”

The main idea

Layer 0 defines two core structures:

1. **KnowledgeNode** – represents a concept or piece of knowledge.

For example: **Round Robin** can be stored as a node with its name, definition, properties, evidence, confidence, and embedding.

1. **KnowledgeEdge** – represents the relationship between two concepts.

For example:

Round Robin → IS_A → CPU Scheduling

Round Robin → USES → Time Quantum

It also defines

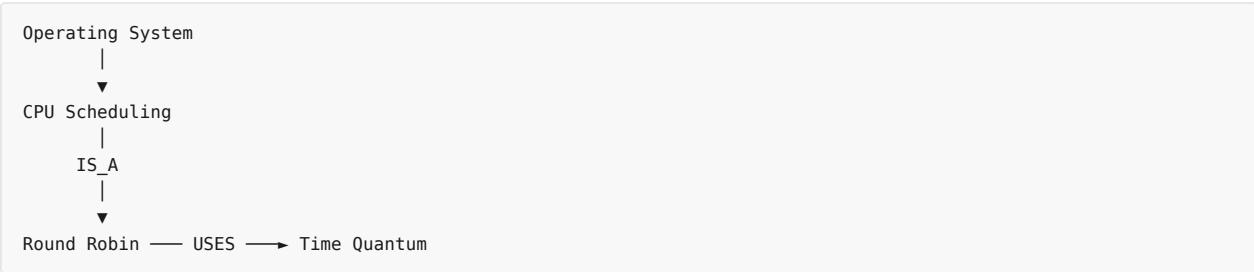
- Types of relationships between concepts.
- Rules to prevent invalid hierarchical structures, such as cycles.
- Evidence and source tracking.
- Confidence propagation rules.

- A common data structure that will be used by all subsequent layers.

Simple conclusion

“In short, Layer 0 creates the common data model and rules for the entire system. All the next layers use this foundation to extract, organize, retrieve, and update knowledge.”

Simple example



Here, **Operating System**, **CPU Scheduling**, **Round Robin**, and **Time Quantum** are **KnowledgeNodes**, while **IS_A** and **USES** are **KnowledgeEdges**.

The explanation above is based on the file's Layer 0 description of KnowledgeNode, KnowledgeEdge, relation ontology, validation constraints, evidence, and confidence propagation. `filecite[turn0file0L225-L234]` `filecite[turn0file0L241-L315]`

YOU 23/08/2026, 21:37:49

ab mujhe layer 1 smjhao jese layer 0 smjhaya in depth

CHATGPT 23/08/2026, 21:37:50

`{"paths":["files"]}`

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite[turn1file0]`

```
[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.0315136476426799
[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--12
[L10] snippet:
[L11] parents(n)) for n in forest.nodes()
[L12] queue = deque([n for n in forest.nodes() if in_degree[n.id] == 0]) # Roots first
[L13]
[L14] while queue:
[L15] node = queue.popleft()
[L16] if forest.parents(node):
[L17] parent_confs = [p.confidence for p in forest.parents(node)]
[L18] avg_parent_conf = sum(parent_confs) / len(parent_confs)
```

```

[L19] evidence_conf = (
[L20] sum(e.confidence for e in node.evidence) / len(node.evidence)
[L21] if node.evidence else 0.5
[L22] )
[L23] node.confidence = (
[L24] ALPHA node.extraction_confidence +
[L25] BETA avg_parent_conf +
[L26] GAMMA evidence_conf
[L27] ) DECAF
[L28] else:
[L29] node.confidence = node.extraction_confidence or BASE_CONFIDENCE
[L30]
[L31] for child in forest.children(node):
[L32] in_degree[child.id] -= 1
[L33] if in_degree[child.id] == 0:
[L34] queue.append(child)
[L35]

```

```

[L36]
[L37] ---
[L38]
[L39] ## 5. Offline Pipeline – Layers 1-5
[L40]
[L41] ### 5.1 Layer 1: Input Processing
[L42]
[L43] Responsibility: Accept raw documents (PDF, DOCX, HTML, TXT), validate format, deduplicate, extract text, a
[L44]
[L45] Input: Raw document file → Output: `NormalizedDocument`
[L46]
[L47]

```

python

```

[L48] @dataclass
[L49] class NormalizedDocument:
[L50]     """Output of Layer 1 — ready for Layer 2 parsing."""
[L51]     doc_id: str
[L52]     original_filename: str
[L53]     format: DocumentFormat # PDF, DOCX, HTML, TXT
[L54]     text: str # Full extracted text
[L55]     pages: list # List of page texts (for PDFs)
[L56]     metadata: dict # Author, title, date, etc.
[L57]     is_ocr: bool = False # True if OCR was used
[L58]     ocr_confidence: float = 1.0 # Average OCR confidence score
[L59]     char_count: int = 0
[L60]     page_count: int = 0
[L61]     content_hash: str = "" # SHA-256 hash for deduplication
[L62]     language: str = "en" # Detected language code
[L63]     partial_extraction: bool = False # True if some pages failed
[L64]     failed_pages: list = None # List of page numbers that failed
[L65]

```

```

[L66]
[L67] **Technology choices:**
[L68]
[L69] | Component | Technology | Purpose |
[L70] |-----|-----|-----|
[L71] | PDF extraction | PyMuPDF (fitz) | Primary text extraction |
[L72] | PDF fallback | pdfplumber | Complex layouts |
[L73] | OCR | Tesseract OCR | Scanned document pages |
[L74] | DOCX | python-docx | Word documents |
[L75] | HTML | BeautifulSoup4 | Web pages |
[L76] | Language detection | langdetect | Auto-detect document language |
[L77] | Deduplication | hashlib (SHA-256) | Prevent duplicate ingestion |
[L78]
[L79] ##### Document Deduplication
[L80]
[L81] Before processing, compute a SHA-256 hash of the file content. If the hash already exists in the document regi
[L82]
[L83]

```

python

```

[L84] import hashlib
[L85]
[L86] def check_duplicate(file_path: str, doc_registry: dict) -> bool:
[L87] """Return True if document is a duplicate."""
[L88] with open(file_path, 'rb') as f:
[L89] content_hash = hashlib.sha256(f.read()).hexdigest()
[L90] if content_hash in doc_registry:
[L91] raise DuplicateDocumentError(
[L92] f"Document already ingested as {doc_registry[content_hash]}")
[L93] return content_hash
[L94] ```
[L95]
[L96] ##### OCR Detection (Multi-Signal)

```

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite`turn1file1

```

[L97] metadata:
[L98] query_result_index: 1
[L99] file_id: file_00000000dcc082119f935683a5e13eed
[L100] version_id: 1
[L101] name: SHIA_RAG_Final_Report(4)(1).md
[L102] mime_type: text/markdown
[L103] surface: conversation
[L104] score: 0.03131881575727918
[L105] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6
[L106] snippet:
[L107]

```



```
[L108] graph TD
[L109]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L110]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L111]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L112]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L113]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L114]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L115]     end
[L116]
[L117]     subgraph "Phase 2: Online – Retrieval & Generation"
[L118]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L119]         KF --> L6
[L120]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L121]         L7 --> A["Answer + Citations"]
[L122]     end
[L123]
[L124]     subgraph "Phase 3: Continuous – Operations"
[L125]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L126]         A --> L8
[L127]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L128]     end
[L129] end
```

[L130]

[L131] ### 3.2 Data Flow

[L132]

[L133]

```
[L134] Raw Document (PDF/DOCX/HTML)
[L135]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L136] Normalized Text + Metadata
[L137]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L138] [Page → Region → Block] hierarchy
[L139]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L140] KnowledgeUnit objects (candidate concepts)
[L141]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L142] KnowledgeNode objects (canonical, deduplicated)
[L143]     ↓ Layer 4: Validate relations against ontology, score confidence
[L144] KnowledgeEdge objects (typed, scored)
[L145]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L146] Knowledge Forest (rooted trees + cross-links)
[L147]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L148] Selected context nodes (within token budget)
[L149]     ↓ Layer 7: Build prompt, invoke LLM
[L150]     ↓ Layer 7: Verify claims against evidence, bind citations
[L151] Answer with citations + confidence score
[L152]     ↓ Layer 8: Log, collect feedback, update forest weights
[L153]
```

[L154]

[L155] ### 3.3 Layer Summary

[L156]

[L157] | Layer | Name | Input | Output | Key Algorithms |

[L158] |-----|-----|-----|-----|-----|

[L159] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L160] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L161] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L162] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L163] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L164] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L165] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L166] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L167] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L168]
[L169] ---
[L170]
[L171] ## 4. Data Model — Layer 0

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `[[filecite]]turn1file2[[`

[L172] metadata:
[L173] query_result_index: 2
[L174] file_id: file_00000000dcc082119f935683a5e13eed
[L175] version_id: 1
[L176] name: SHIA_RAG_Final_Report(4)(1).md
[L177] mime_type: text/markdown
[L178] surface: conversation
[L179] score: 0.030330882352941176
[L180] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--11
[L181] snippet:
[L182] semantic_relations:
[L183] - USES # "TCP USES Congestion Control"
[L184] - CAUSES # "Deadlock CAUSES System Hang"
[L185] - DEPENDS_ON # "Virtual Memory DEPENDS_ON Paging"
[L186] - CONTRASTS_WITH # "Preemptive CONTRASTS_WITH Non-Preemptive"
[L187] - SIMILAR_TO # "Round Robin SIMILAR_TO Fair-Share"
[L188] - HAS_PROPERTY # "Round Robin HAS_PROPERTY Time Quantum"
[L189] - PRODUCES # "Compiler PRODUCES Machine Code"
[L190] - PREREQUISITE # "Data Structures PREREQUISITE Algorithms"
[L191]

```
[L192]
[L193] ### 4.4 Confidence Propagation
[L194]
[L195] Node confidence is propagated from roots to leaves using topological sort (Kahn's algorithm) to guarantee
[L196]
[L197]
```

python
[L198] from collections import deque
[L199]
[L200] # Confidence weighting factors (must sum to 1.0)
[L201] ALPHA = 0.4 # Node's own extraction confidence weight
[L202] BETA = 0.4 # Parent confidence weight
[L203] GAMMA = 0.2 # Evidence confidence weight
[L204] DECAY = 0.95 # Per-level decay factor
[L205] BASE_CONFIDENCE = 0.5 # Default for root nodes (configurable)
[L206]
[L207] def propagate_confidence(forest):
[L208] """Propagate confidence scores from roots to leaves.
[L209]
[L210] Uses Kahn's algorithm (topological ordering) to ensure

```

[L211] parents are always processed before their children.
[L212] Processing in arbitrary order would give children stale
[L213] parent confidence values. """
[L214] in_degree = {n.id: len(forest.parents(n)) for n in forest.nodes()}
[L215] queue = deque([n for n in forest.nodes() if in_degree[n.id] == 0]) # Roots first
[L216]
[L217] while queue:
[L218]     node = queue.popleft()
[L219]     if forest.parents(node):
[L220]         parent_confs = [p.confidence for p in forest.parents(node)]
[L221]         avg_parent_conf = sum(parent_confs) / len(parent_confs)
[L222]         evidence_conf = (
[L223]             sum(e.confidence for e in node.evidence) / len(node.evidence)
[L224]             if node.evidence else 0.5
[L225]         )
[L226]         node.confidence = (
[L227]             ALPHA node.extraction_confidence +
[L228]             BETA avg_parent_conf +
[L229]             GAMMA evidence_conf
[L230]         ) DECAF
[L231]     else:
[L232]         node.confidence = node.extraction_confidence or BASE_CONFIDENCE
[L233]
[L234]     for child in forest.children(node):
[L235]         in_degree[child.id] -= 1
[L236]         if in_degree[child.id] == 0:
[L237]             queue.append(child)
[L238]

```

```

[L239]
[L240] ---
[L241]
[L242] ## 5. Offline Pipeline – Layers 1-5
[L243]
[L244] ### 5.1 Layer 1: Input Processing
[L245]
[L246] Responsibility: Accept raw documents (PDF, DOCX, HTML, TXT), validate format, deduplicate, extract text,
[L247]
[L248] Input: Raw document file → Output: `NormalizedDocument`
[L249]
[L250]

```

python

```

[L251] @dataclass
[L252] class NormalizedDocument:
[L253]     """Output of Layer 1 — ready for Layer 2 parsing."""
[L254]     doc_id: str
[L255]     original_filename: str
[L256]     format: DocumentFormat # PDF, DOCX, HTML, TXT
[L257]     text: str # Full extracted text
[L258]     pages: list # List of page texts (for PDFs)
[L259]     metadata: dict # Author, title, date, etc.

```

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite`turn1file3

[L260] metadata:

[L261] query_result_index: 3

[L262] file_id: file_00000000dcc082119f935683a5e13eed

[L263] version_id: 1

[L264] name: SHIA_RAG_Final_Report(4)(1).md

[L265] mime_type: text/markdown

[L266] surface: conversation

[L267] score: 0.02964426877470356

[L268] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--17

[L269] snippet:

[L270] 1)

[L271] if success_rate < 0.8:

[L272] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")

[L273] elif fmt == 'docx':

[L274] text, pages = extract_docx(file_path), []

[L275] is_ocr, ocr_conf = False, 1.0

[L276] elif fmt == 'html':

[L277] text, pages = extract_html(file_path), []

[L278] is_ocr, ocr_conf = False, 1.0

[L279] else: # txt

[L280] text = open(file_path, 'r', encoding='utf-8').read()

[L281] pages, is_ocr, ocr_conf = [], False, 1.0

[L282]

[L283] language = detect_language(text)

[L284] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal

[L285] metadata = extract_metadata(file_path, fmt)

[L286]

[L287] doc = NormalizedDocument(

[L288] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),

[L289] format=DocumentFormat(fmt), text=text, pages=pages,

[L290] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,

[L291] char_count=len(text), page_count=len(pages) or 1,

[L292] content_hash=content_hash, language=language,

[L293] partial_extraction=bool(failed_pages), failed_pages=failed_pages

[L294])

[L295] doc_registry[content_hash] = doc.doc_id # Register for dedup

[L296] return doc

[L297]

[L298]

[L299] ---

[L300]

[L301] ### 5.2 Layer 2: Document Parsing

[L302]

[L303] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with block cl

[L304]

[L305] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`

[L306]

[L307]

```
python
[L308] class BlockType(Enum):
[L309] HEADING = "heading"
[L310] PARAGRAPH = "paragraph"
[L311] LIST = "list"
[L312] TABLE = "table"
[L313] CODE = "code"
[L314] ALGORITHM = "algorithm"
[L315] FIGURE_CAPTION = "figure_caption"
[L316] EQUATION = "equation"
[L317] DEFINITION = "definition"
[L318]
[L319] @dataclass
[L320] class DocumentBlock:
[L321] """Atomic unit of document structure."""
[L322] block_id: str
[L323] doc_id: str
[L324] page_number: int
[L325] region_index: int # Which region on the page
[L326] block_type: BlockType
[L327] text: str
[L328] reading_order: int # Global ordering across the document
[L329] confidence: float = 1.0 # Classification confidence
[L330] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L331]
```

```
[L332]
[L333] The parser uses rule-based heuristics for block classification:
[L334] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase
[L335] - **List:** Starts with ` - `, `* `, `• `, or `1.`
[L336] - **Table:** Contains 3+ pipe characters (`|`) - uses regex `re.match(r'^(def |class |import |from )', line)`
[L337] - **Code:** Uses regex pattern matching (NOT `def ` in text, which falsely matches "definitely")
[L338] - **Definition:** Contains phrases like "is defined as", "refers to"
[L339] - **Default:** Paragraph
[L340]
[L341] ##### Table Cell-Level Parsing
[L342]
[L343] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level st
[L344]
[L345]
```

```
python
[L346] def parse_table_cells(page, table_region) -> list:
[L347] """Extract table structure with cell-level detail.
[L348]
[L349] Uses pdfplumber for text-based tables, Camelot for image-based.
[L350] Returns list of rows, each row is a list of cell strings."""
```

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: turn1file4

```
[L351] metadata:
[L352] query_result_index: 4
[L353] file_id: file_00000000dcc082119f935683a5e13eed
```

[L354] version_id: 1
 [L355] name: SHIA_RAG_Final_Report(4)(1).md
 [L356] mime_type: text/markdown
 [L357] surface: conversation
 [L358] score: 0.02628900949796472
 [L359] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--15
 [L360] snippet:
 [L361] ##### Text Cleaning (Safe)
 [L362]
 [L363]

```
[L364] def clean_text(text: str, pages: list) -> str:
[L365]     """Clean extracted text while preserving legitimate content.
[L366]
[L367]     Safeguards against false header/footer removal:
[L368]     1. Identify candidate headers/footers by frequency across pages
[L369]     2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L370]     3. Never remove lines that appear on fewer than 3 pages
[L371]     4. Normalize whitespace but preserve paragraph boundaries
[L372]     """
[L373]     if len(pages) >= 3:
[L374]         # Frequency-based header/footer detection
[L375]         first_lines = [p.split('\n')[0].strip() for p in pages if p.strip()]
[L376]         last_lines = [p.split('\n')[-1].strip() for p in pages if p.strip()]
[L377]
[L378]         from collections import Counter
[L379]         header_counts = Counter(first_lines)
[L380]         footer_counts = Counter(last_lines)
[L381]
[L382]         threshold = max(3, int(len(pages) * 0.6)) # 60% of pages OR at least 3
[L383]         headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L384]         footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L385]
[L386]         lines = text.split('\n')
[L387]         lines = [l for l in lines
[L388]                 if l.strip() not in headers_to_remove
[L389]                 and l.strip() not in footers_to_remove]
[L390]         text = '\n'.join(lines)
[L391]
[L392]         # Normalize whitespace (preserve paragraph breaks)
[L393]         import re
[L394]         text = re.sub(r'[\t]+', ' ', text) # Collapse horizontal space
[L395]         text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L396]         return text.strip()
[L397]
```

[L398]
 [L399] ##### Language Detection
 [L400]
 [L401]

```
[L402] from langdetect import detect
[L403]
[L404] def detect_language(text: str) -> str:
[L405]     """Detect document language for OCR configuration.
[L406]
[L407]     Used to configure Tesseract's language parameter.
[L408]     Falls back to 'eng' if detection fails."""
[L409]     try:
[L410]         lang = detect(text[:5000]) # Sample first 5000 chars
[L411]         # Map langdetect codes to Tesseract codes
[L412]         lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L413]         return lang_map.get(lang, 'eng')
[L414]     except Exception:
[L415]         return 'eng'
[L416]
```

[L417]

[L418] ##### Partial Extraction Policy

[L419]

[L420] If some pages fail during extraction, the system uses a **best-effort policy**:

[L421]

[L422] | Condition | Action |

[L423] |-----|-----|

[L424] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |

[L425] | <80% pages succeed | Reject entire document, queue for manual review |

[L426] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |

[L427]

[L428] ```python

[L429] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:

[L430] """Layer 1: Input Processing pipeline with all safeguards."""

[L431] # Step 0: Duplicate check

[L432] content_hash = check_duplicate(file_path, doc_registry)

[L433]

[L434] fmt = detect_format(file_path)

[L435] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)

[L436]

[L437] failed_pages = []

[L438] if fmt == 'pdf':

[L439] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)

[L440] # Partial extraction policy

[L441] success_rate = (len(pages) - len(failed_pages)) / max(len(pages),

 DEEP RESEARCH 23/08/2026, 21:37:55
Citation Marker: `filecite`turn1file5

[L442] metadata:

[L443] query_result_index: 5

[L444] file_id: file_00000000dcc082119f935683a5e13eed

[L445] version_id: 1

[L446] name: SHIA_RAG_Final_Report(4)(1).md

[L447] mime_type: text/markdown

[L448] surface: conversation

[L449] score: 0.025837770122142185

[L450] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--7

[L451] snippet:
[L452] deduplicated)
[L453] ↓ Layer 4: Validate relations against ontology, score confidence
[L454] KnowledgeEdge objects (typed, scored)
[L455] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L456] Knowledge Forest (rooted trees + cross-links)
[L457] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L458] Selected context nodes (within token budget)
[L459] ↓ Layer 7: Build prompt, invoke LLM
[L460] ↓ Layer 7: Verify claims against evidence, bind citations
[L461] Answer with citations + confidence score
[L462] ↓ Layer 8: Log, collect feedback, update forest weights
[L463]

```
[L464]
[L465] ### 3.3 Layer Summary
[L466]
[L467] | Layer | Name | Input | Output | Key Algorithms |
[L468] |-----|-----|-----|-----|-----|
[L469] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L470] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L471] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L472] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L473] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L474] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L475] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L476] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L477] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L478]
[L479] ---
[L480]
[L481] ## 4. Data Model – Layer 0
[L482]
[L483] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L484]
[L485] ### 4.1 KnowledgeNode Schema
[L486]
[L487] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L488]
[L489]
```

json

```
[L490] {
[L491] "node_id": "NODE_000789",
[L492] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L493] "canonical_name": "Round Robin",
[L494] "aliases": ["RR", "Round-Robin Scheduling"],
[L495] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L496] "summaries": ["CPU scheduling method using time slices"],
[L497] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L498] "domain": "Operating Systems",
[L499] "properties": {
[L500] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L501] "preemptive": {"type": "boolean", "value": true}
[L502] },
[L503] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L504] "parent": "NODE_000456",
```



```
[L505] "children": ["NODE_001001", "NODE_001002"],
[L506] "related": ["NODE_000890"],
[L507] "incoming_edges": ["EDGE_001"],
[L508] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L509] "evidence": [
[L510] {
[L511] "evidence_type": "DEFINITION",
[L512] "text": "Round Robin is a preemptive scheduling algorithm...",
[L513] "source_doc": "DOC_000001",
[L514] "source_block": "BLK_000234",
[L515] "page": 42
[L516] }
[L517] ],
```

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite`turn1file6

```
[L518] metadata:
[L519] query_result_index: 6
[L520] file_id: file_00000000dcc082119f935683a5e13eed
[L521] version_id: 1
[L522] name: SHIA_RAG_Final_Report(4)(1).md
[L523] mime_type: text/markdown
[L524] surface: conversation
[L525] score: 0.025218810265539238
[L526] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--16
[L527] snippet:
[L528] strip()
[L529]
```

```
[L530]
[L531] #### Language Detection
[L532]
[L533]
```

```
python
[L534] from langdetect import detect
[L535]
[L536] def detect_language(text: str) -> str:
[L537] """Detect document language for OCR configuration.
[L538]
[L539] Used to configure Tesseract's language parameter.
[L540] Falls back to 'eng' if detection fails."""
[L541] try:
[L542] lang = detect(text[:5000]) # Sample first 5000 chars
[L543] # Map langdetect codes to Tesseract codes
[L544] lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L545] return lang_map.get(lang, 'eng')
[L546] except Exception:
[L547] return 'eng'
[L548]
```

```

[L549]
[L550] ##### Partial Extraction Policy
[L551]
[L552] If some pages fail during extraction, the system uses a **best-effort policy**:
[L553]
[L554] | Condition | Action |
[L555] |-----|-----|
[L556] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |
[L557] | <80% pages succeed | Reject entire document, queue for manual review |
[L558] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L559]
[L560]

```

python

```

[L561] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L562] """Layer 1: Input Processing pipeline with all safeguards."""
[L563] # Step 0: Duplicate check
[L564] content_hash = check_duplicate(file_path, doc_registry)
[L565]
[L566] fmt = detect_format(file_path)
[L567] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L568]
[L569] failed_pages = []
[L570] if fmt == 'pdf':
[L571] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L572] # Partial extraction policy
[L573] success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L574] if success_rate < 0.8:
[L575] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L576] elif fmt == 'docx':
[L577] text, pages = extract_docx(file_path), []
[L578] is_ocr, ocr_conf = False, 1.0
[L579] elif fmt == 'html':
[L580] text, pages = extract_html(file_path), []
[L581] is_ocr, ocr_conf = False, 1.0
[L582] else: # txt
[L583] text = open(file_path, 'r', encoding='utf-8').read()
[L584] pages, is_ocr, ocr_conf = [], False, 1.0
[L585]
[L586] language = detect_language(text)
[L587] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L588] metadata = extract_metadata(file_path, fmt)
[L589]
[L590] doc = NormalizedDocument(
[L591] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L592] format=DocumentFormat(fmt), text=text, pages=pages,
[L593] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L594] char_count=len(text), page_count=len(pages) or 1,
[L595] content_hash=content_hash, language=language,
[L596] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L597] )
[L598] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L599] return doc

```

[L600] ``
[L601]
[L602] ---
[L603]
[L604] ### 5.2 Layer 2: Document Parsing
[L605]
[L606] **Responsibility:** Parse normalized text into a structured [Page → Region → Block] hierarchy with block classification and reading order.
[L607]
[L608] **Input:** NormalizedDocument → **Output:** List of DocumentBlock`

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite`turn1file7

[L609] metadata:
[L610] query_result_index: 7
[L611] file_id: file_00000000dcc082119f935683a5e13eed
[L612] version_id: 1
[L613] name: SHIA_RAG_Final_Report(4)(1).md
[L614] mime_type: text/markdown
[L615] surface: conversation
[L616] score: 0.030536130536130537
[L617] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
[L618] snippet:
[L619] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L620]
[L621] ### 2.3 Literature Search Methodology
[L622]
[L623] | Parameter | Details |
[L624] |-----|-----|
[L625] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L626] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L627] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L628] | **Time Range** | 2020–2026 |
[L629] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L630]
[L631] ---
[L632]
[L633] ## 3. System Architecture
[L634]
[L635] ### 3.1 Overview
[L636]
[L637] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:
[L638]
[L639] | Phase | Layers | Mode | Purpose |
[L640] |-----|-----|-----|-----|
[L641] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest

|

[L642] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L643] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L644]

[L645] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L646]

[L647]

```
[L648] graph TD
[L649]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L650]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L651]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L652]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L653]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L654]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L655]     end
[L656]
[L657]     subgraph "Phase 2: Online – Retrieval & Generation"
[L658]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L659]         KF --> L6
[L660]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L661]         L7 --> A["Answer + Citations"]
[L662]     end
[L663]
[L664]     subgraph "Phase 3: Continuous – Operations"
[L665]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L666]         A --> L8
[L667]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L668]     end
[L669]
```

[L670]

[L671] ### 3.2 Data Flow

[L672]

[L673] ```

[L674] Raw Document (PDF/DOCX/HTML)

[L675] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L676] Normalized Text + Metadata

[L677] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L678] [Page → Region → Block] hierarchy

[L679] ↓ Layer 3: Split sentences,

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite[turn1file8]`

[L680] metadata:

[L681] query_result_index: 8

[L682] file_id: file_00000000dcc082119f935683a5e13eed

[L683] version_id: 1

[L684] name: SHIA_RAG_Final_Report(4)(1).md

[L685] mime_type: text/markdown

[L686] surface: conversation

[L687] score: 0.0288981288981289

[L688] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--0

[L689] snippet:

[L690] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L691]

[L692] > **Project Type:** Minor Project (M.Tech)

[L693] > **Date:** August 2026

[L694] > **Author:** [Your Name]

[L695]

[L696] ---

[L697]

[L698] ## Abstract

[L699]

[L700] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L701]

[L702] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L703]

[L704] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L705]

[L706] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L707]

[L708] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L709]

[L710] ---

[L711]

[L712] ## Table of Contents

[L713]

[L714] 1. Introduction

[L715] 2. Literature Review

[L716] 3. System Architecture

[L717] 4. Data Model — Layer 0

[L718] 5. Offline Pipeline — Layers 1-5

[L719] 6. Online Pipeline — Layers 6-7

[L720] 7. Operations Layer — Layer 8

[L721] 8. Novel Contributions

[L722] 9. Mathematical Framework

[L723] 10. Implementation

[L724] 11. Evaluation Framework

[L725] 12. Risk Analysis

[L726] 13. Conclusion & Future Work

[L727]

[L728] ---

[L729]

[L730] ## 1. Introduction

[L731]

[L732] ### 1.1 Problem Statement

[L733]

[L734] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L735]

[L736] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L737] 2. Embed each chunk into a vector space

[L738] 3. At query time, retrieve the top-k most similar chunks

[L739] 4. Feed these chunks as context to an LLM for answer generation

[L740]

[L741] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

DEEP RESEARCH 23/08/2026, 21:37:55

Citation Marker: `filecite[turn1file9]`

[L742] metadata:

[L743] query_result_index: 9

[L744] file_id: file_00000000dcc082119f935683a5e13eed

[L745] version_id: 1

[L746] name: SHIA_RAG_Final_Report(4)(1).md

[L747] mime_type: text/markdown

[L748] surface: conversation

[L749] score: 0.02889344262295082

[L750] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--32

[L751] snippet:

[L752]

```

[L753] @dataclass
[L754] class RetrievalResult:
[L755]     """Output of Layer 6 – input to Layer 7."""
[L756]     selected_nodes: list          # Nodes selected by Tree Knapsack
[L757]     context_text: str             # Assembled context string
[L758]     traversal_path_edges: list    # Edges traversed (for Layer 8 feedback)
[L759]     query_complexity: str         # SIMPLE, MODERATE, COMPLEX
[L760]     query_intent: str             # DEFINITION, COMPARISON, etc.
[L761]     seed_node_ids: list           # Initial seed nodes from vector search
[L762]     token_count: int              # Estimated tokens in context
[L763]
[L764] class RetrievalEngine:
[L765]     def __init__(self, forest, vector_index, cache_manager, config):
[L766]         self.forest = forest
[L767]         self.vector_index = vector_index
[L768]         self.cache = cache_manager
[L769]         self.config = config
[L770]
[L771]     def retrieve(self, query) -> RetrievalResult:
[L772]         # Step 0: Cache check
[L773]         cached = self.cache.get_query_result(hash_query(query))
[L774]         if cached:
[L775]             return cached
[L776]
[L777]         # Step 1: Classify query intent and complexity
[L778]         intent = detect_intent(query)
[L779]         complexity = classify_complexity(intent)
[L780]
[L781]         # Step 2: Embed query and find seed nodes
[L782]         query_emb = embed_query(query) # Asymmetric embedding model
[L783]         seed_nodes = self.vector_index.search(query_emb, k=5)
[L784]
[L785]         # Step 3: Adaptive depth traversal
[L786]         depth_config = {
[L787]             'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L788]             'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L789]             'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L790]         }
[L791]         traversal = depth_config[complexity]
[L792]
[L793]         # Step 4: BFS traversal from all seeds
[L794]         candidate_nodes = set()
[L795]         for seed in seed_nodes:
[L796]             candidate_nodes |= self.traverse(seed, traversal)
[L797]
[L798]         # Step 5: Tree Knapsack selection (per-tree, then merge)
[L799]         selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L800]
[L801]         # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L802]         context_text = self.assemble_context(selected, intent)
[L803]
[L804]         return RetrievalResult(
[L805]             selected_nodes=selected, context_text=context_text,
[L806]             traversal_path_edges=self._get_path_edges(selected),
[L807]             query_complexity=complexity, query_intent=intent,
[L808]             seed_node_ids=[s.id for s in seed_nodes],
[L809]             token_count=estimate_tokens(context_text)
[L810]         )
[L811]

```

[L812]

[L813] ##### Multi-Tree Knapsack Selection

[L814]

[L815] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L816]

[L817] 1. **Group** candidates by tree root

[L818] 2. **Allocate** token budget proportionally to each tree

[L819] 3. **Run** Tree Knapsack on each tree independently

```
[L820] 4. Merge results and trim to total budget
[L821]
[L822] ```python
[L823] def multi_tree_knapsack(self, candidate_nodes, budget):
[L824]     tree_groups = {}
[L825]     for node in candidate_nodes:
[L826]         root = self.forest.find_root(node)
[L827]         tree_groups.setdefault(root.id, []).append(node)
[L828]
[L829]     all_selected = []
[L830]     for root_id, nodes in tree_groups.items():
[L831]         tree_budget = int(budget * len(nodes) / max(len(candidate_nodes), 1))
[L832]         _, selected = tree_knapsack(self.forest.get_node(root_id), tree_budget, self.
```

DEEP RESEARCH 23/08/2026, 21:38:00

Citation Marker: `filecite`turn2file

```
[L1] metadata:
[L2] ref_id: turn1file0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mode: chunk_context
[L7] lines_shown: 1-120
[L8] total_file_lines: 201
[L9] has_more: True
[L10] next_start_line: 121
[L11] next_read:
{"ref_id":"turn1file0","mode":"chunk_context","start_line":121,"context_before_lines":20,"context_after_lines":180}
[L12] content:
[L13] node.confidence = (
[L14] ALPHA node.extraction_confidence +
[L15] BETA avg_parent_conf +
[L16] GAMMA evidence_conf
[L17] ) DECAY
[L18] else:
[L19] node.confidence = node.extraction_confidence or BASE_CONFIDENCE
[L20]
[L21] for child in forest.children(node):
[L22]     in_degree[child.id] -= 1
[L23]     if in_degree[child.id] == 0:
[L24]         queue.append(child)
[L25]
```



```

[L26]
[L27] ---
[L28]
[L29] ## 5. Offline Pipeline – Layers 1-5
[L30]
[L31] ### 5.1 Layer 1: Input Processing
[L32]
[L33] **Responsibility:** Accept raw documents (PDF, DOCX, HTML, TXT), validate format, deduplicate, extract text, a
[L34]
[L35] **Input:** Raw document file → **Output:** `NormalizedDocument`
[L36]
[L37]

```

python

```

[L38] @dataclass
[L39] class NormalizedDocument:
[L40] """Output of Layer 1 — ready for Layer 2 parsing."""
[L41] doc_id: str
[L42] original_filename: str
[L43] format: DocumentFormat # PDF, DOCX, HTML, TXT
[L44] text: str # Full extracted text
[L45] pages: list # List of page texts (for PDFs)
[L46] metadata: dict # Author, title, date, etc.
[L47] is_ocr: bool = False # True if OCR was used
[L48] ocr_confidence: float = 1.0 # Average OCR confidence score
[L49] char_count: int = 0
[L50] page_count: int = 0
[L51] content_hash: str = "" # SHA-256 hash for deduplication
[L52] language: str = "en" # Detected language code
[L53] partial_extraction: bool = False # True if some pages failed
[L54] failed_pages: list = None # List of page numbers that failed
[L55]

```

```

[L56]
[L57] **Technology choices:**
[L58]
[L59] | Component | Technology | Purpose |
[L60] |-----|-----|-----|
[L61] | PDF extraction | PyMuPDF (fitz) | Primary text extraction |
[L62] | PDF fallback | pdfplumber | Complex layouts |
[L63] | OCR | Tesseract OCR | Scanned document pages |
[L64] | DOCX | python-docx | Word documents |
[L65] | HTML | BeautifulSoup4 | Web pages |
[L66] | Language detection | langdetect | Auto-detect document language |
[L67] | Deduplication | hashlib (SHA-256) | Prevent duplicate ingestion |
[L68]
[L69] ### Document Deduplication
[L70]
[L71] Before processing, compute a SHA-256 hash of the file content. If the hash already exists in the document regi
[L72]
[L73]

```

python

```

[L74] import hashlib
[L75]
[L76] def check_duplicate(file_path: str, doc_registry: dict) -> bool:
[L77] """Return True if document is a duplicate."""
[L78] with open(file_path, 'rb') as f:

```

```
[L79] content_hash = hashlib.sha256(f.read()).hexdigest()
[L80] if content_hash in doc_registry:
[L81] raise DuplicateDocumentError(
[L82] f"Document already ingested as {doc_registry[content_hash]}")
[L83] return content_hash
[L84]
```

```
[L85]
[L86] #### OCR Detection (Multi-Signal)
[L87]
[L88] A single heuristic (`text_chars / page_area < 0.1`) is unreliable. For example, a page with only a large headi
[L89]
[L90]
```

python

```
[L91] def needs_ocr(page) -> bool:
[L92] """Multi-signal OCR detection — more robust than text/area ratio alone.
[L93]
[L94] Signal 1: Text density (text chars / page area)
[L95] Signal 2: Font metadata availability (scanned pages lack font info)
[L96] Signal 3: Image coverage ratio (scanned pages are mostly images)
[L97] """
[L98] text = page.get_text()
[L99] area = page.rect.width * page.rect.height
[L100]
[L101] # Signal 1: Text density
[L102] text_density = len(text.strip()) / max(area, 1)
[L103]
[L104] # Signal 2: Font metadata (scanned PDFs have no embedded fonts)
[L105] has_fonts = len(page.get_fonts()) > 0
[L106]
[L107] # Signal 3: Image coverage (scanned pages are large images)
[L108] images = page.get_images()
[L109] image_area = sum(img[2] * img[3] for img in images) if images else 0
[L110] image_coverage = image_area / max(area, 1)
[L111]
[L112] # Decision: OCR needed if text is sparse AND (no fonts OR high image coverage)
[L113] if text_density < 0.1 and (not has_fonts or image_coverage > 0.5):
[L114] return True
[L115] return False
[L116]
```

```
[L117]
[L118] #### OCR Confidence Calculation
[L119]
[L120]
```

python

```
[L121] def calculate_ocr_confidence(ocr_result) -> float:
[L122] """Calculate OCR confidence from Tesseract per-word confidence scores.
[L123]
[L124] Tesseract returns per-word confidence (0-100). We:
[L125] 1. Get per-word confidences
```

```
[L126] 2. Filter out words with confidence < 10 (noise)
[L127] 3. Compute weighted average by word length (longer words = more reliable signal)
[L128] 4. Normalize to [0.0, 1.0]
[L129] """
[L130] word_data = ocr_result.get('words', [])
[L131] if not word_data:
[L132] return 0.0
```

DEEP RESEARCH 23/08/2026, 21:38:00

Citation Marker: `filecite`turn2file1

```
[L133] metadata:
[L134] read_index: 1
[L135] ref_id: turn1file4
[L136] file_id: file_00000000dcc082119f935683a5e13eed
[L137] version_id: 1
[L138] name: SHIA_RAG_Final_Report(4)(1).md
[L139] mode: chunk_context
[L140] lines_shown: 1-120
[L141] total_file_lines: 210
[L142] has_more: True
[L143] next_start_line: 121
[L144] next_read:
{"ref_id":"turn1file4","mode":"chunk_context","start_line":121,"context_before_lines":40,"context_after_lines":170}
[L145] content:
[L146] ##### OCR Confidence Calculation
[L147]
[L148]
```

```
[L149] def calculate_ocr_confidence(ocr_result) -> float:
[L150]     """Calculate OCR confidence from Tesseract per-word confidence scores.
[L151]
[L152]     Tesseract returns per-word confidence (0-100). We:
[L153]     1. Get per-word confidences
[L154]     2. Filter out words with confidence < 10 (noise)
[L155]     3. Compute weighted average by word length (longer words = more reliable signal)
[L156]     4. Normalize to [0.0, 1.0]
[L157]     """
[L158]     word_data = ocr_result.get('words', [])
[L159]     if not word_data:
[L160]         return 0.0
[L161]
[L162]     total_weight, weighted_sum = 0, 0
[L163]     for word in word_data:
[L164]         if word['confidence'] >= 10: # Filter noise
[L165]             weight = len(word['text']) # Weight by word length
[L166]             weighted_sum += word['confidence'] * weight
[L167]             total_weight += weight
[L168]
[L169]     return (weighted_sum / max(total_weight, 1)) / 100.0
[L170]
```

```
[L171]
[L172] ##### Text Cleaning (Safe)
[L173]
[L174]
```

```

[L175] def clean_text(text: str, pages: list) -> str:
[L176]     """Clean extracted text while preserving legitimate content.
[L177]
[L178]     Safeguards against false header/footer removal:
[L179]     1. Identify candidate headers/footers by frequency across pages
[L180]     2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L181]     3. Never remove lines that appear on fewer than 3 pages
[L182]     4. Normalize whitespace but preserve paragraph boundaries
[L183]     """
[L184]     if len(pages) >= 3:
[L185]         # Frequency-based header/footer detection
[L186]         first_lines = [p.split('\n')[0].strip() for p in pages if p.strip()]
[L187]         last_lines = [p.split('\n')[-1].strip() for p in pages if p.strip()]
[L188]
[L189]         from collections import Counter
[L190]         header_counts = Counter(first_lines)
[L191]         footer_counts = Counter(last_lines)
[L192]
[L193]         threshold = max(3, int(len(pages) * 0.6)) # 60% of pages OR at least 3
[L194]         headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L195]         footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L196]
[L197]         lines = text.split('\n')
[L198]         lines = [l for l in lines
[L199]                     if l.strip() not in headers_to_remove
[L200]                     and l.strip() not in footers_to_remove]
[L201]         text = '\n'.join(lines)
[L202]
[L203]         # Normalize whitespace (preserve paragraph breaks)
[L204]         import re
[L205]         text = re.sub(r'[\t]+' , ' ', text) # Collapse horizontal space
[L206]         text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L207]         return text.strip()
[L208]

```

[L209]

[L210] ##### Language Detection

[L211]

[L212]

```

[L213] from langdetect import detect
[L214]
[L215] def detect_language(text: str) -> str:
[L216]     """Detect document language for OCR configuration.
[L217]
[L218]     Used to configure Tesseract's language parameter.
[L219]     Falls back to 'eng' if detection fails."""
[L220]     try:
[L221]         lang = detect(text[:5000]) # Sample first 5000 chars
[L222]         # Map langdetect codes to Tesseract codes
[L223]         lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L224]         return lang_map.get(lang, 'eng')
[L225]     except Exception:
[L226]         return 'eng'
[L227]

```

[L228]

[L229] ##### Partial Extraction Policy

[L230]

[L231] If some pages fail during extraction, the system uses a **best-effort policy**:

[L232]

[L233] | Condition | Action |

[L234] |-----|-----|

[L235] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |

```
[L236] | <80% pages succeed | Reject entire document, queue for manual review |
[L237] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L238]
[L239] ```python
[L240] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L241] """Layer 1: Input Processing pipeline with all safeguards."""
[L242] # Step 0: Duplicate check
[L243] content_hash = check_duplicate(file_path, doc_registry)
[L244]
[L245] fmt = detect_format(file_path)
[L246] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L247]
[L248] failed_pages = []
[L249] if fmt == 'pdf':
[L250] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L251] # Partial extraction policy
[L252] success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L253] if success_rate < 0.8:
[L254] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L255] elif fmt == 'docx':
[L256] text, pages = extract_docx(file_path), []
[L257] is_ocr, ocr_conf = False, 1.0
[L258] elif fmt == 'html':
[L259] text, pages = extract_html(file_path), []
[L260] is_ocr, ocr_conf = False, 1.0
[L261] else: # txt
[L262] text = open(file_path, 'r', encoding='utf-8').read()
[L263] pages, is_ocr, ocr_conf = [], False, 1.0
[L264]
[L265] language = detect_language(text)
```

DEEP RESEARCH 23/08/2026, 21:38:04

Citation Marker: `filecite`turn3file0

```
[L1] metadata:
[L2] ref_id: turn1file0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mode: chunk_context
[L7] lines_shown: 121-121
[L8] total_file_lines: 121
[L9] has_more: False
[L10] content:
[L11] 3. Never remove lines that appear on fewer than 3 pages
```

DEEP RESEARCH 23/08/2026, 21:38:04

Citation Marker: `filecite`turn3file1

[L12] metadata:
 [L13] read_index: 1
 [L14] ref_id: turn1file4
 [L15] file_id: file_00000000dcc082119f935683a5e13eed
 [L16] version_id: 1
 [L17] name: SHIA_RAG_Final_Report(4)(1).md
 [L18] mode: chunk_context
 [L19] lines_shown: 121-121
 [L20] total_file_lines: 121
 [L21] has_more: False
 [L22] content:
 [L23] page_number: int

DEEP RESEARCH 23/08/2026, 21:38:08

Citation Marker: `[]filecite[]turn4file0[]`

[L1] metadata:
 [L2] query_result_index: 0
 [L3] file_id: file_00000000dcc082119f935683a5e13eed
 [L4] version_id: 1
 [L5] name: SHIA_RAG_Final_Report(4)(1).md
 [L6] mime_type: text/markdown
 [L7] surface: conversation
 [L8] score: 0.03057889822595705
 [L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--12
 [L10] snippet:
 [L11] parents(n)) for n in forest.nodes()}
 [L12] queue = deque([n for n in forest.nodes() if in_degree[n.id] == 0]) # Roots first
 [L13]
 [L14] while queue:
 [L15] node = queue.popleft()
 [L16] if forest.parents(node):
 [L17] parent_confs = [p.confidence for p in forest.parents(node)]
 [L18] avg_parent_conf = sum(parent_confs) / len(parent_confs)
 [L19] evidence_conf = (
 [L20] sum(e.confidence for e in node.evidence) / len(node.evidence)
 [L21] if node.evidence else 0.5
 [L22])
 [L23] node.confidence = (
 [L24] ALPHA *node.extraction_confidence* +
 [L25] BETA avg_parent_conf +
 [L26] GAMMA *evidence_conf*
 [L27]) DECAY
 [L28] else:
 [L29] node.confidence = node.extraction_confidence or BASE_CONFIDENCE
 [L30]
 [L31] for child in forest.children(node):
 [L32] in_degree[child.id] -= 1

```
[L33] if in_degree[child.id] == 0:
```

```
[L34] queue.append(child)
```

```
[L35]
```

```
[L36]
[L37] ---
[L38]
[L39] ## 5. Offline Pipeline – Layers 1-5
[L40]
[L41] ### 5.1 Layer 1: Input Processing
[L42]
[L43] **Responsibility:** Accept raw documents (PDF, DOCX, HTML, TXT), validate format, deduplicate, extract text, a
[L44]
[L45] **Input:** Raw document file → **Output:** `NormalizedDocument`
[L46]
[L47]
```

```
python
```

```
[L48] @dataclass
```

```
[L49] class NormalizedDocument:
```

```
[L50] """Output of Layer 1 — ready for Layer 2 parsing."""
```

```
[L51] doc_id: str
```

```
[L52] original_filename: str
```

```
[L53] format: DocumentFormat # PDF, DOCX, HTML, TXT
```

```
[L54] text: str # Full extracted text
```

```
[L55] pages: list # List of page texts (for PDFs)
```

```
[L56] metadata: dict # Author, title, date, etc.
```

```
[L57] is_ocr: bool = False # True if OCR was used
```

```
[L58] ocr_confidence: float = 1.0 # Average OCR confidence score
```

```
[L59] char_count: int = 0
```

```
[L60] page_count: int = 0
```

```
[L61] content_hash: str = "" # SHA-256 hash for deduplication
```

```
[L62] language: str = "en" # Detected language code
```

```
[L63] partial_extraction: bool = False # True if some pages failed
```

```
[L64] failed_pages: list = None # List of page numbers that failed
```

```
[L65]
```

```
[L66]
[L67] **Technology choices:**
[L68]
[L69] | Component | Technology | Purpose |
[L70] |-----|-----|-----|
[L71] | PDF extraction | PyMuPDF (fitz) | Primary text extraction |
[L72] | PDF fallback | pdfplumber | Complex layouts |
[L73] | OCR | Tesseract OCR | Scanned document pages |
[L74] | DOCX | python-docx | Word documents |
[L75] | HTML | BeautifulSoup4 | Web pages |
[L76] | Language detection | langdetect | Auto-detect document language |
[L77] | Deduplication | hashlib (SHA-256) | Prevent duplicate ingestion |
[L78]
[L79] #### Document Deduplication
[L80]
[L81] Before processing, compute a SHA-256 hash of the file content. If the hash already exists in the document regi
[L82]
[L83]
```

```
python
```

```
[L84] import hashlib
```

```
[L85]
```

```
[L86] def check_duplicate(file_path: str, doc_registry: dict) -> bool:
[L87] """Return True if document is a duplicate."""
[L88] with open(file_path, 'rb') as f:
[L89] content_hash = hashlib.sha256(f.read()).hexdigest()
[L90] if content_hash in doc_registry:
[L91] raise DuplicateDocumentError(
[L92] f"Document already ingested as {doc_registry[content_hash]}")
[L93] return content_hash
[L94] ```
[L95]
[L96] ##### OCR Detection (Multi-Signal)
```

DEEP RESEARCH 23/08/2026, 21:38:08

Citation Marker: `[]filecite[]turn4file1[]`

```
[L97] metadata:
[L98] query_result_index: 1
[L99] file_id: file_00000000dcc082119f935683a5e13eed
[L100] version_id: 1
[L101] name: SHIA_RAG_Final_Report(4)(1).md
[L102] mime_type: text/markdown
[L103] surface: conversation
[L104] score: 0.0304147465437788
[L105] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--15
[L106] snippet:
[L107] ##### Text Cleaning (Safe)
[L108]
[L109]
```



```

[L110] def clean_text(text: str, pages: list) -> str:
[L111]     """Clean extracted text while preserving legitimate content.
[L112]
[L113]     Safeguards against false header/footer removal:
[L114]     1. Identify candidate headers/footers by frequency across pages
[L115]     2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L116]     3. Never remove lines that appear on fewer than 3 pages
[L117]     4. Normalize whitespace but preserve paragraph boundaries
[L118]     """
[L119]     if len(pages) >= 3:
[L120]         # Frequency-based header/footer detection
[L121]         first_lines = [p.split('\n')[0].strip() for p in pages if p.strip()]
[L122]         last_lines = [p.split('\n')[-1].strip() for p in pages if p.strip()]
[L123]
[L124]         from collections import Counter
[L125]         header_counts = Counter(first_lines)
[L126]         footer_counts = Counter(last_lines)
[L127]
[L128]         threshold = max(3, int(len(pages) * 0.6)) # 60% of pages OR at least 3
[L129]         headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L130]         footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L131]
[L132]         lines = text.split('\n')
[L133]         lines = [l for l in lines
[L134]                     if l.strip() not in headers_to_remove
[L135]                     and l.strip() not in footers_to_remove]
[L136]         text = '\n'.join(lines)
[L137]
[L138]         # Normalize whitespace (preserve paragraph breaks)
[L139]         import re
[L140]         text = re.sub(r'[\t]+' , ' ', text) # Collapse horizontal space
[L141]         text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L142]         return text.strip()
[L143]

```

[L144]

[L145] ##### Language Detection

[L146]

[L147]

```

[L148] from langdetect import detect
[L149]
[L150] def detect_language(text: str) -> str:
[L151]     """Detect document language for OCR configuration.
[L152]
[L153]     Used to configure Tesseract's language parameter.
[L154]     Falls back to 'eng' if detection fails."""
[L155]     try:
[L156]         lang = detect(text[:5000]) # Sample first 5000 chars
[L157]         # Map langdetect codes to Tesseract codes
[L158]         lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L159]         return lang_map.get(lang, 'eng')
[L160]     except Exception:
[L161]         return 'eng'
[L162]

```

[L163]

[L164] ##### Partial Extraction Policy

[L165]

[L166] If some pages fail during extraction, the system uses a **best-effort policy**:

[L167]

[L168] | Condition | Action |

[L169] |-----|-----|

[L170] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |

```
[L171] | <80% pages succeed | Reject entire document, queue for manual review |
[L172] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L173]
[L174] ```python
[L175] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L176] """Layer 1: Input Processing pipeline with all safeguards."""
[L177] # Step 0: Duplicate check
[L178] content_hash = check_duplicate(file_path, doc_registry)
[L179]
[L180] fmt = detect_format(file_path)
[L181] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L182]
[L183] failed_pages = []
[L184] if fmt == 'pdf':
[L185] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L186] # Partial extraction policy
[L187] success_rate = (len(pages) - len(failed_pages)) / max(len(pages),
```

DEEP RESEARCH 23/08/2026, 21:38:08

Citation Marker: `filecite`turn4file2

```
[L188] metadata:
[L189] query_result_index: 2
[L190] file_id: file_00000000dcc082119f935683a5e13eed
[L191] version_id: 1
[L192] name: SHIA_RAG_Final_Report(4)(1).md
[L193] mime_type: text/markdown
[L194] surface: conversation
[L195] score: 0.029513888888888888
[L196] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--13
[L197] snippet:
[L198] is_ocr: bool = False # True if OCR was used
[L199] ocr_confidence: float = 1.0 # Average OCR confidence score
[L200] char_count: int = 0
[L201] page_count: int = 0
[L202] content_hash: str = "" # SHA-256 hash for deduplication
[L203] language: str = "en" # Detected language code
[L204] partial_extraction: bool = False # True if some pages failed
[L205] failed_pages: list = None # List of page numbers that failed
[L206]
```

```

[L207]
[L208] **Technology choices:**
[L209]
[L210] | Component | Technology | Purpose |
[L211] |-----|-----|-----|
[L212] | PDF extraction | PyMuPDF (fitz) | Primary text extraction |
[L213] | PDF fallback | pdfplumber | Complex layouts |
[L214] | OCR | Tesseract OCR | Scanned document pages |
[L215] | DOCX | python-docx | Word documents |
[L216] | HTML | BeautifulSoup4 | Web pages |
[L217] | Language detection | langdetect | Auto-detect document language |
[L218] | Deduplication | hashlib (SHA-256) | Prevent duplicate ingestion |
[L219]
[L220] #### Document Deduplication
[L221]
[L222] Before processing, compute a SHA-256 hash of the file content. If the hash already exists in the document reg
[L223]
[L224]

```

python

```

[L225] import hashlib
[L226]
[L227] def check_duplicate(file_path: str, doc_registry: dict) -> bool:
[L228] """Return True if document is a duplicate."""
[L229] with open(file_path, 'rb') as f:
[L230] content_hash = hashlib.sha256(f.read()).hexdigest()
[L231] if content_hash in doc_registry:
[L232] raise DuplicateDocumentError(
[L233] f"Document already ingested as {doc_registry[content_hash]}")
[L234] return content_hash
[L235]

```

```

[L236]
[L237] #### OCR Detection (Multi-Signal)
[L238]
[L239] A single heuristic (`text_chars / page_area < 0.1`) is unreliable. For example, a page with only a large head
[L240]
[L241]

```

python

```

[L242] def needs_ocr(page) -> bool:
[L243] """Multi-signal OCR detection — more robust than text/area ratio alone.
[L244]
[L245] Signal 1: Text density (text chars / page area)
[L246] Signal 2: Font metadata availability (scanned pages lack font info)
[L247] Signal 3: Image coverage ratio (scanned pages are mostly images)
[L248] """
[L249] text = page.get_text()
[L250] area = page.rect.width * page.rect.height
[L251]
[L252] # Signal 1: Text density
[L253] text_density = len(text.strip()) / max(area, 1)
[L254]
[L255] # Signal 2: Font metadata (scanned PDFs have no embedded fonts)
[L256] has_fonts = len(page.get_fonts()) > 0
[L257]

```

```
[L258] # Signal 3: Image coverage (scanned pages are large images)
[L259] images = page.get_images()
[L260] image_area = sum(img[2] img[3] for img in images) if images else 0
[L261] image_coverage = image_area / max(area, 1)
[L262]
[L263] # Decision: OCR needed if text is sparse AND (no fonts OR high image coverage)
[L264] if text_density < 0.1 and (not has_fonts or image_coverage > 0.5):
[L265] return True
[L266] return False
[L267]
```

```
[L268]
[L269] ##### OCR Confidence Calculation
[L270]
[L271]
```

python

```
[L272] def calculate_ocr_confidence(ocr_result) -> float:
[L273] """Calculate OCR confidence from Tesseract per-word confidence scores.
[L274]
[L275] Tesseract returns per-word confidence (0-100). We:
[L276] 1. Get per-word confidences
[L277] 2. Filter out words with confidence < 10 (noise)
[L278] 3. Compute weighted average by word length (longer words = more reliable signal)
[L279] 4.
```

DEEP RESEARCH 23/08/2026, 21:38:08

Citation Marker: `filecite`turn4file3

```
[L280] metadata:
[L281] query_result_index: 3
[L282] file_id: file_00000000dcc082119f935683a5e13eed
[L283] version_id: 1
[L284] name: SHIA_RAG_Final_Report(4)(1).md
[L285] mime_type: text/markdown
[L286] surface: conversation
[L287] score: 0.02921395544346364
[L288] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--35
[L289] snippet:
[L290] ##### Prompt Template
[L291]
[L292]
```

```
[L293] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L294] RULES:
[L295] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L296] 2. For each claim, cite the source using [NODE_ID] format.
[L297] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L298] 4. If evidence conflicts, present both views with citations.
[L299]
[L300] REQUIRED OUTPUT FORMAT:
[L301] {
[L302]   "answer": "Your answer with [NODE_ID] citations inline.",
[L303]   "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L304]   "overall_confidence": 0.0-1.0,
[L305]   "insufficient_evidence": false
[L306] }"""
[L307]
```

[L308]

[L309] ##### Claim Verification

[L310]

[L311] After the LLM generates an answer, each claim is verified against its cited evidence:

[L312]

[L313]

```
[L314] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L315]     """Returns (status, confidence).
[L316]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L317]     - PARTIAL:       evidence partially supports (0.4 < confidence ≤ 0.7)
[L318]     - UNSUPPORTED:   no evidence supports (confidence ≤ 0.4)
[L319]     - CONFLICTING:   evidence items contradict each other
[L320]     """
[L321]     scores = []
[L322]     for eid in evidence_ids:
[L323]         node = node_map.get(eid)
[L324]         if node and node.evidence:
[L325]             evidence_text = ' '.join(e.text for e in node.evidence)
[L326]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L327]
[L328]     if not scores:
[L329]         return 'UNSUPPORTED', 0.0
[L330]
[L331]     avg_score = sum(scores) / len(scores)
[L332]
[L333]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L334]         return 'CONFLICTING', avg_score
[L335]     if avg_score > 0.7:
[L336]         return 'SUPPORTED', avg_score
[L337]     elif avg_score > 0.4:
[L338]         return 'PARTIAL', avg_score
[L339]     else:
[L340]         return 'UNSUPPORTED', avg_score
[L341]
```

[L342]

[L343] ##### Unsupported Claim Policy

[L344]

[L345] | Status | Action |

[L346] |-----|-----|

[L347] | SUPPORTED | Keep as-is with citation |

[L348] | PARTIAL | Keep but mark as partially supported |

[L349] | UNSUPPORTED | Remove from answer, note at end |

[L350] | CONFLICTING | Present both views with note |

[L351]

[L352] ##### Answer Confidence Formula

[L353]

[L354]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{node_conf}}{\text{cited nodes}} + 0.1 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L355]

[L356] ##### Output Schema

[L357]

[L358]

```
[L359] @dataclass
[L360] class GenerationResult:
[L361]     """Output of Layer 7 – ready for user and Layer 8."""
[L362]     answer: str                # Answer text with citations
[L363]     claims: list              # List of VerifiedClaim
[L364]     citations: list           # Formatted citation list
[L365]     overall_confidence: float # 0.0-1.0
[L366]     retrieval_result: object  # Preserved for Layer 8
[L367]     generation_metadata: dict # Model used, claim counts, etc.
[L368]
```

[L369]

[L370] ---

[L371]

[L372] ## 7. Operations Layer — Layer 8

[L373]

[L374] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L375]

[L376] ### 7.1 Logging

[L377]

[L378] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 23/08/2026, 21:38:08

Citation Marker: `filecite[turn4file4]`

[L379] metadata:

[L380] query_result_index: 4

[L381] file_id: file_00000000dcc082119f935683a5e13eed

[L382] version_id: 1

[L383] name: SHIA_RAG_Final_Report(4)(1).md

[L384] mime_type: text/markdown

[L385] surface: conversation

[L386] score: 0.028283227848101264

[L387] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--54

[L388] snippet:

[L389] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L390] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L391] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L392] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L393] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L394]

[L395] The complete system specification covers:

[L396] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L397] - **12 defined functions** with complete implementations

[L398] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L399] - **8 schemas** covering all data objects from document blocks to query logs

[L400] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L401] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L402]

[L403] ### 13.2 Key Novelties

[L404]

[L405] | # | Novelty | Level | Why It's New |

[L406] |---|-----|-----|-----|

[L407] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L408] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L409] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L410] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L411] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L412]

[L413] ### 13.3 Next Steps

[L414]

[L415] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L416] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L417] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L418] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L419] 5. **Build Layer 4** (Relation management with ontology validation)

[L420] 6. **Build Layer 5** (SHIA Core with algorithms)

[L421] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L422] 8. **Build Layer 7** (Generation with verification)

[L423] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L424] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L425] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L426] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L427]

[L428] ### 13.4 Future Work

CHATGPT 23/08/2026, 21:38:08

Bilkul. Ab hum **Layer 1** ko exactly **Layer 0** ki tarah, same running example ke saath, ek-ek step me deeply samjhenge.

Hum wahi example use karenge:

| *Operating Systems ka ek PDF document, jisme CPU Scheduling, Round Robin aur Time Quantum ki information hai.*

Layer 1 — Input Processing

Sabse pehle ek line me samjho

*Layer 1 ka job hai: raw document ko system ke andar safely accept karna, check karna, duplicate detect karna, text extract karna, zarurat pade to OCR use karna, aur finally ek standardized **NormalizedDocument** banana.*

File ke according, Layer 1 **PDF, DOCX, HTML, aur TXT** documents accept karta hai aur validation, deduplication, text extraction, normalization, aur metadata creation karta hai. इसका output **NormalizedDocument** hota hai.

filecite\turn2file0\L29-L67

1. Layer 1 se pehle actual problem kya hai?

User system me koi bhi document upload kar sakta hai:

```
Book.pdf
Notes.docx
ResearchPaper.html
Data.txt
```

Ab system ke saamne problem hai:

Different files, different structures

```
PDF → pages aur complex layout
DOCX → paragraphs aur formatting
HTML → tags
TXT → plain text
```

Agar Layer 2 ko directly ye different files de diye jaayen, to Layer 2 ko har format separately samajhna padega.

For example:

```
Layer 2:
"Ye PDF hai kya?"
"Isme OCR chahiye?"
"Ye duplicate hai?"
"HTML tags remove karne hain?"
```

× Ye bad design hoga.

Isliye Layer 1 ek **standardization gate** ki tarah kaam karti hai.

```
graph TD
    A[Different Input Formats] --> B[LAYER 1  
Input Processing]
    B --> C[One Standard Format  
= NormalizedDocument]
```

2. Hamare example me Layer 1 ka Input kya hai?

Maan lo hum ye file upload karte hain:

```
Operating_Systems_Notes.pdf
```

Is PDF me:


```
Page 1:
Operating Systems Introduction

Page 10:
CPU Scheduling

Page 15:
Round Robin is a preemptive scheduling algorithm...

Page 16:
Time Quantum = 20 ms
```

To Layer 1 ko actual input milega:

```
Raw File
```

Conceptually:

```
{
  file_name: "Operating_Systems_Notes.pdf",
  file_type: "PDF",
  file_content: Binary Data
}
```

Important point

Layer 1 ko abhi ye **nahi pata**:

- Round Robin ek concept hai ya nahi
- CPU Scheduling ka Round Robin se relation kya hai
- Time Quantum property hai ya concept

⚠ **Layer 1 knowledge understand nahi karti.**

Ye sirf document ko machine-readable aur standardized banati hai.

Ye distinction bahut important hai.

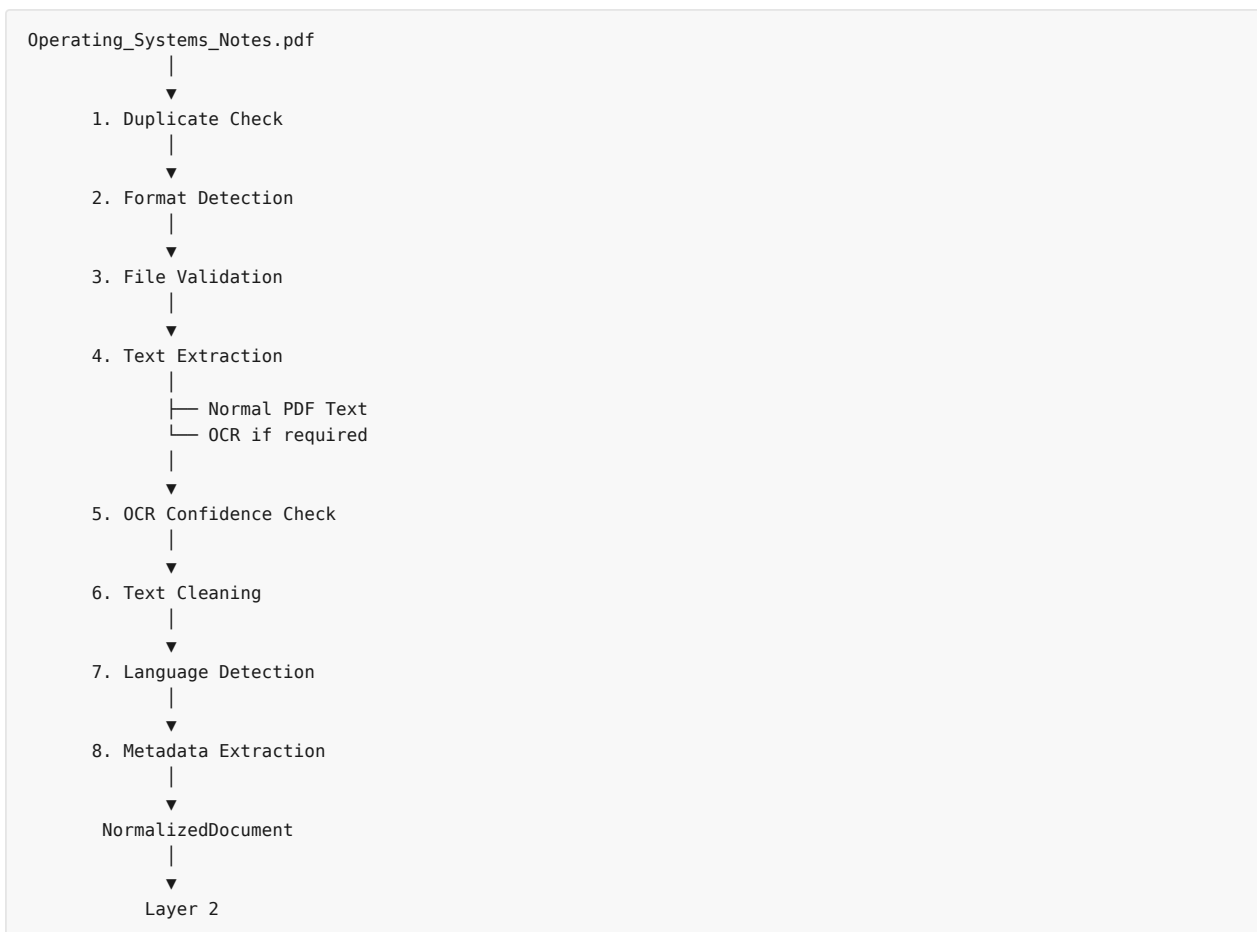
```
Layer 1 = "Document ko read karo"

Layer 2 = "Document ka structure samjho"

Layer 3 = "Knowledge/concepts extract karo"
```

3. Layer 1 ka complete flow

Hamari PDF system me enter hoti hai:



File me Layer 1 ka implementation flow duplicate check → format detection/validation → format-specific extraction → language detection → safe text cleaning → metadata → `NormalizedDocument` ke form me defined hai.

📄filecite🔁turn2file1📄L229-L265📄📄filecite🔁turn4file1📄L174-L187📄

Ab hum **har step ko detail me** samajhte hain.

4. Step 1 — Duplicate Document Check

Problem

Maan lo tumne same PDF do baar upload kar di:

Upload 1:
Operating_Systems_Notes.pdf

Upload 2:
Operating_Systems_Notes.pdf

Agar system blindly dono process kare:

```

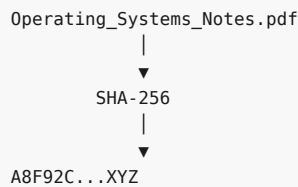
Same document
  ↓
Same concepts extract honge
  ↓
Same Round Robin knowledge dubara
  ↓
Duplicate nodes/knowledge
  
```

Aage jaake Knowledge Forest dirty ho sakta hai.

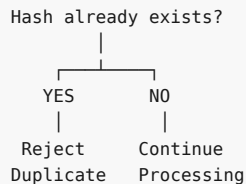
Solution: SHA-256 Hash

Layer 1 file ke actual binary content ka **SHA-256 hash** calculate karti hai.

Conceptually:



Ab system registry check karta hai:



File ke according, agar content hash pehle se registry me ho, upload ko `DuplicateDocumentError` ke through reject kiya jata hai. `FileCite` turn `file0L69-L84`

Example

First Upload:
 File → Hash H123
 Registry → H123 doesn't exist
 Result → Process document

Second Upload:
 File → Hash H123
 Registry → H123 already exists
 Result → Reject duplicate

Why filename check nahi?

Kyunki:

OS_Notes.pdf

aur

Operating_System_Book.pdf

dono alag names ho sakte hain, lekin actual content same ho sakta hai.

Isliye:

Layer 1 filename nahi, content ko identify karti hai.

5. Step 2 — Format Detection

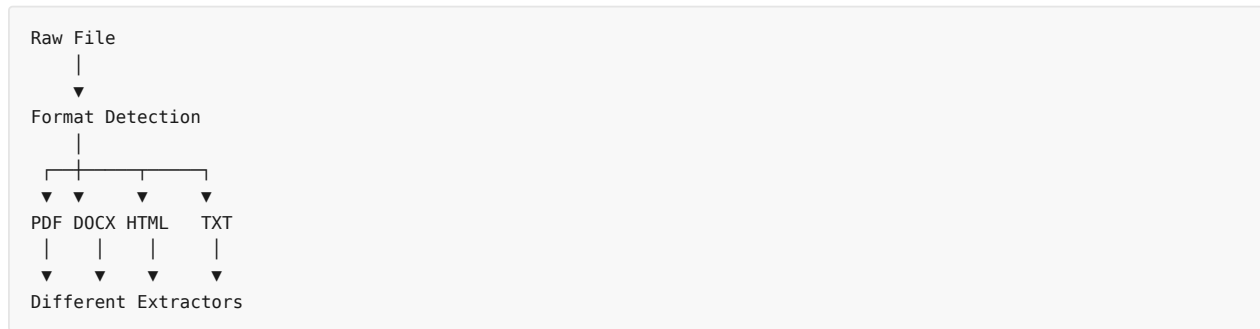
Duplicate nahi hai, ab system dekhta hai:

Ye file actually kis format ki hai?

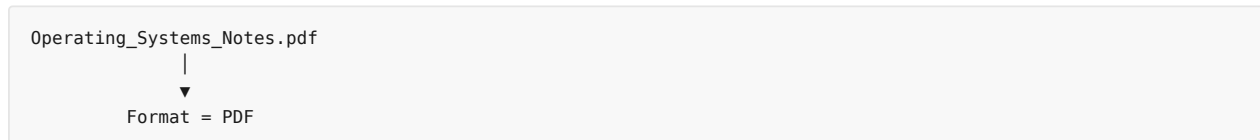
Possible formats:

PDF
 DOCX
 HTML
 TXT

Phir system extraction strategy choose karega.



Hamare example me



To system PDF extraction pipeline choose karega.

File ke technology choices me PDF, DOCX, HTML aur TXT ke liye different extraction paths define hain.

☐filecite☐turn2file0☐L57-L67☐

6. Step 3 — File Validation

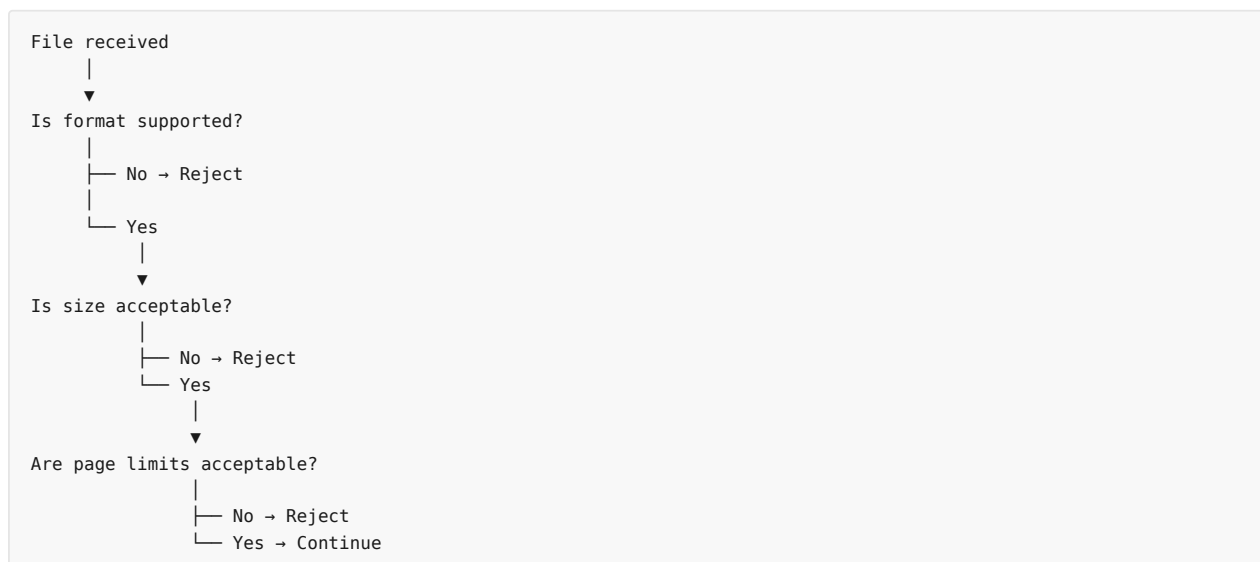
Ab system file ko blindly process nahi karega.

Usko validate karega.

File ke implementation me format check ke saath **maximum 100 MB size** aur **5000 pages** ki validation limit specified hai.

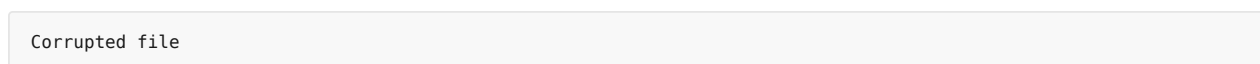
☐filecite☐turn4file1☐L174-L187☐

Conceptually:



Why needed?

Imagine:



ya:

Extremely large PDF

Agar validation nahi hai, system:

- crash ho sakta hai
- unnecessary resources consume kar sakta hai
- extraction pipeline fail kar sakti hai

So Layer 1 ka first role hai:

"Bad input ko aage jaane se rokna."

7. Step 4 — Text Extraction

Ab hamari file valid hai.

Operating_Systems_Notes.pdf
↓
Extract the text

Lekin PDF ke case me do situations ho sakti hain.

Case A — Normal Digital PDF

Example:

Round Robin is a preemptive CPU scheduling algorithm.

Ye actual digital text hai.

PDF extraction library directly isko read kar sakti hai.

File ke according primary PDF extraction ke liye:

PyMuPDF (fitz)

aur complex layouts ke fallback ke liye:

pdfplumber

use kiya gaya hai. `filecite[turn2file0][L57-L67]`

Flow:

PDF
↓
PyMuPDF
↓
Extracted Text

Example:

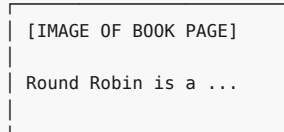
"CPU Scheduling"

"Round Robin is a preemptive scheduling algorithm."

"Time Quantum is 20 ms."

Case B — Scanned PDF

Maan lo PDF ke pages actual image hain:



Yahan visually human text dekh sakta hai.

Lekin computer ke liye:

Text Extraction Result = almost empty

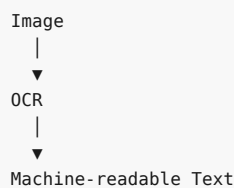
To normal extraction kaam nahi karega.

Yahan OCR chahiye.

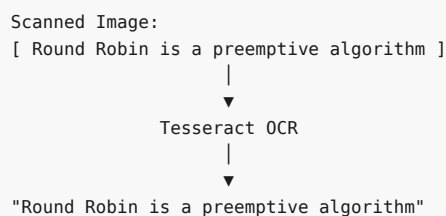
8. OCR kya hota hai?

OCR = Optical Character Recognition

Simple:



Example:



File ke according scanned document pages ke liye **Tesseract OCR** use hota hai. filecite.io/turn2file0/L61-L67/

9. System ko kaise pata chalega ki OCR chahiye?

Ye Layer 1 ka interesting part hai.

System sirf ye nahi dekhta:

"Text kam hai, therefore OCR."

Kyunki ek page par sirf ek bada heading bhi ho sakta hai:

CPU SCHEDULING

Text kam hai, lekin page scanned nahi hai.

Isliye file me **multi-signal OCR detection** use ki gayi hai.

Signal 1 — Text Density

System check karta hai:

```
Text characters
-----
Page area
```

Agar text bahut kam hai → OCR ka possibility signal.

Signal 2 — Font Metadata

Normal digital PDF me embedded fonts ki information hoti hai.

Scanned PDF me generally:

```
Actual text = absent
Font information = absent
```

So system check karta hai:

```
Does the page have font metadata?
```

Signal 3 — Image Coverage

Scanned page usually ek large image hota hai.

System dekhta hai:

```
Page ka kitna area images cover kar rahi hain?
```

Decision roughly:

```
Text sparse?
  AND
(No fonts OR high image coverage?)
  |
  ▼
Use OCR
```

File me exactly ye three signals—text density, font metadata, aur image coverage—OCR decision ke liye use kiye gaye hain.

filecite[turn2file0][L86-L116]

Why is this better?

```
Single rule
  ↓
False detection ho sakti hai ✖

Multiple signals
  ↓
Better OCR decision ✔
```

10. OCR ke baad confidence bhi calculate hota hai

OCR perfect nahi hota.

Example:

Actual text:

Round Robin

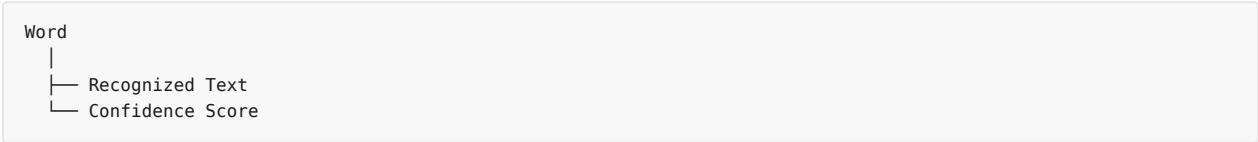
Poor scan ke baad OCR galat read kar sakta hai:

Round Robln

Isliye system blindly OCR output trust nahi karta.

Tesseract word-level confidence scores deta hai.

Layer 1:



Low-confidence noise words filter kiye jaate hain, phir weighted average calculate karke confidence 0.0–1.0 range me normalize kiya jata hai. `filecite\turn2file0\L118-L132` `filecite\turn2file1\L146-L170`

Example

Round

→ 98%

Robin

→ 96%

Scheduling

→ 94%

Overall:

ocr_confidence ≈ 0.96

Lekin:

Round

→ 45%

Robln

→ 32%

To confidence low hogi.

Important safeguard

Agar kisi page ki OCR confidence:

< 0.6

ho, to page manual review ke liye flag hota hai, lekin document me include reh sakta hai. `filecite\turn2file1\L229-L237`

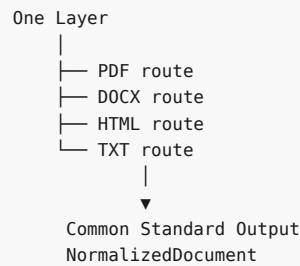
11. Step 5 — Different formats ke liye different technology

Layer 1 ek single extractor use nahi karti.

- | Input | Technology | Kaam |
- |---|---
- | PDF | PyMuPDF | Primary text extraction |
- | Complex PDF | pdfplumber | Fallback extraction |
- | Scanned PDF | Tesseract OCR | Image se text |
- | DOCX | python-docx | Word document extraction |
- | HTML | BeautifulSoup4 | Web page content extraction |
- | Language | langdetect | Language detection |
- | Duplicate check | SHA-256 / hashlib | Same content detect |

Ye technology stack file me explicitly defined hai. `filecite\turn2file0\L57-L67`

So:



12. Step 6 — Text Cleaning

Ab hamari PDF se text extract ho gaya.

Maan lo raw output:

```

NITK Operating Systems Notes

CPU Scheduling is important.

Page 10

CPU Scheduling decides which process gets CPU.

NITK Operating Systems Notes
  
```

Ab problem:

- Repeated headers
- Repeated footers
- Extra spaces
- Extra newlines

Layer 1 text clean karti hai.

Lekin yahan ek **important safeguard** hai.

System blindly header remove nahi karta

Kyunki koi actual important line accidentally remove ho sakti hai.

File ke according:

- Candidate header/footer ki frequency check hoti hai.
- Sirf woh lines remove hoti hain jo **60%+ pages** me repeat hoti hain.
- 3 pages se kam me appearing lines remove nahi ki jaati.
- Whitespace normalize hota hai, lekin paragraph boundaries preserve ki jaati hain. `filecite[turn2file1][L172-L207]`

Example

100-page document:

```
"NITK Operating Systems Notes"
```

Agar 80 pages par repeat ho raha hai:

80%

→ likely actual header → remove ✓

Lekin:

"Round Robin"

sirf 1 page par hai:

1%

→ actual content ho sakta hai → remove nahi karna ✓

Ye false deletion se bachata hai.

13. Step 7 — Language Detection

Ab system detect karta hai:

Document kis language me hai?

Example:

English
Hindi
German
French
Spanish

File ke according `langdetect` document ke text sample se language detect karta hai aur detection fail hone par English (`eng`) fallback use karta hai. Ye OCR configuration ke liye bhi use hota hai. `filecite\turn2file1\L210-L227`

Hamare example me

```

Operating Systems Notes
|
▼
Language Detection
|
▼
English → eng

```

Why useful?

Agar scanned Hindi document ho:

यह ऑपरेटिंग सिस्टम है

to ideally OCR ko correct language configuration chahiye.

14. Step 8 — Metadata Extraction

Text alone enough nahi hai.

System document ke baare me bhi information store karta hai.

Example:

```
Original Filename:  
Operating_Systems_Notes.pdf  
  
Format:  
PDF  
  
Author:  
John Smith  
  
Title:  
Operating Systems  
  
Date:  
2026
```

Ye `metadata` field me ja sakta hai.

File ke `NormalizedDocument` schema me metadata ke examples **author, title, date** diye gaye hain. `filecite\turn2file0\L37-L54`

15. Ab sabse important: Layer 1 ka Output kya hai?

`NormalizedDocument`

Ye Layer 1 ka final output hai.

Hamare example me roughly:

```
NormalizedDocument(  
  doc_id = "DOC_001",  
  
  original_filename =  
    "Operating_Systems_Notes.pdf",  
  
  format = "PDF",  
  
  text = ""  
    Operating System manages computer resources.  
  
    CPU Scheduling decides which process  
    gets CPU time.  
  
    Round Robin is a preemptive CPU  
    scheduling algorithm.  
  
    Round Robin uses a fixed time quantum.  
    "",  
  
  pages = [  
    "Page 1 extracted text...",  
    "Page 2 extracted text...",  
    ...  
  ],  
  
  metadata = {  
    "title": "Operating Systems Notes",  
    "author": "..."  
  },  
  
  is_ocr = False,  
  
  ocr_confidence = 1.0,  
  
  char_count = 250000,  
  
  page_count = 200,  
  
  content_hash = "SHA256_HASH...",  
  
  language = "eng",  
  
  partial_extraction = False,  
  
  failed_pages = []  
)
```

File me `NormalizedDocument` ke andar exactly document identity, original filename, format, full text, pages, metadata, OCR status/confidence, counts, content hash, language, aur partial extraction information defined hai. `filecite\turn2file0\L37-L54`

16. Har field ko simple language me samjho

`doc_id`

DOC_001

System ke andar document ki unique identity.

`original_filename`

Operating_Systems_Notes.pdf

Original uploaded file ka naam.

format

PDF

Document originally kis format me tha.

text

Ye sabse important output hai.

Entire extracted and cleaned document text

pages

PDF ke page-wise text:

pages[0] → Page 1 text
pages[1] → Page 2 text
pages[2] → Page 3 text

metadata

Extra document information:

Author
Title
Date
etc.

is_ocr

True → OCR use hua
False → Normal extraction hui

ocr_confidence

0.96

OCR result kitna reliable tha.

content_hash

SHA-256 Hash

Duplicate detection aur document identity ke liye.

partial_extraction

Agar kuch pages fail hue:

True

failed_pages

Example:

```
[15, 28, 29]
```

System ko exactly pata rahega ki kaunse pages properly extract nahi hue.

17. Partial Extraction — Bahut important error handling

Real world me har PDF perfectly extract nahi hoti.

Maan lo:

```
100 pages total
95 pages → successfully extracted
5 pages → failed
```

System kya kare?

File ki policy:

Case 1: $\geq 80\%$ pages successfully extract hue

```
Accept document ✓
partial_extraction = True
failed_pages = [...]
```

Case 2: $< 80\%$ pages successfully extract hue

```
Reject document ✗
Queue for manual review
```

Case 3: OCR confidence < 0.6

```
Document/page flag for manual review
```

File me ye best-effort policy explicitly defined hai. `filecite\turn2file1\L229-L237`

Why 80% policy?

System do extreme approaches avoid karta hai:

Approach 1

```
1 page fail
↓
Reject entire 500-page document
```

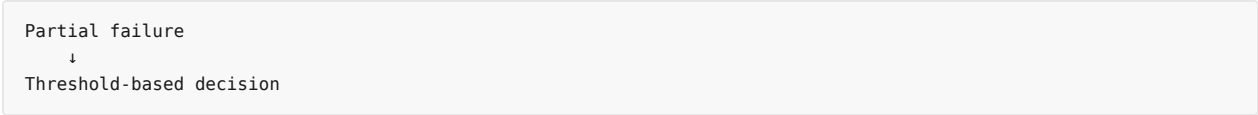
Too strict ✗

Approach 2

```
90% pages fail
↓
Still accept
```

Too risky ✗

So:



18. Layer 1 me kya nahi ho raha?

Ye bahut important hai, especially agar tum guide ko explain karoge.

Layer 1:

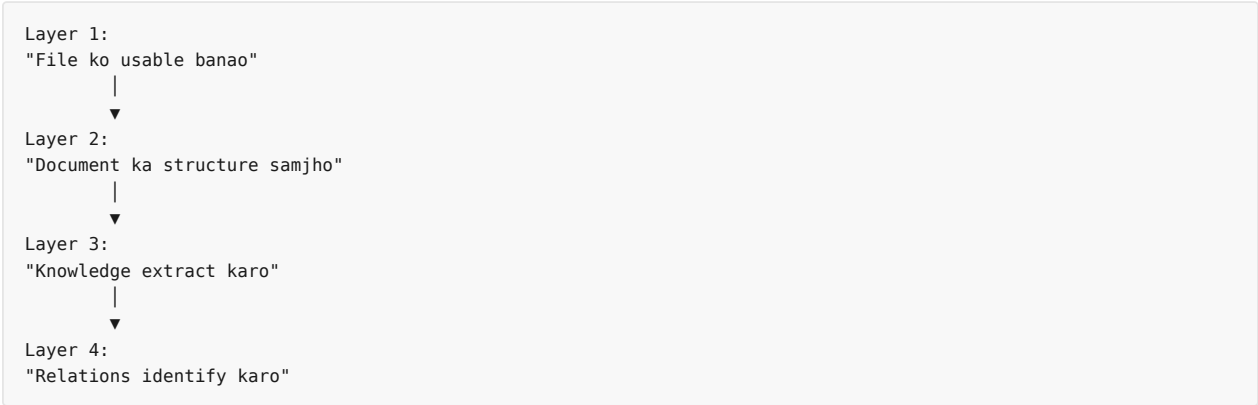
- × Round Robin ko concept identify nahi karti
- × Round Robin aur CPU Scheduling ka relation nahi banati
- × Document ko chunks me split nahi karti
- × Heading aur paragraph ka deep structural classification nahi karti
- × KnowledgeNode nahi banati

Layer 1 sirf:

Raw document → reliable standardized document representation

banati hai.

Actual flow:



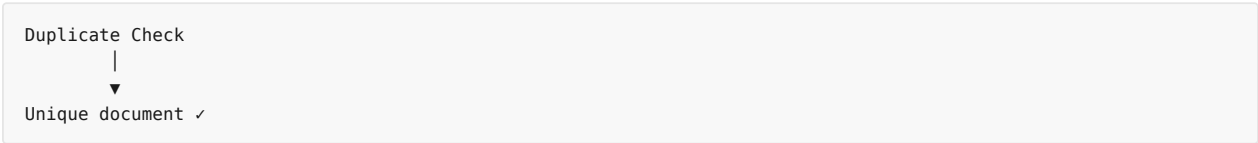
Architecture ke overall data flow me Layer 1 ke baad Normalized Text + Metadata Layer 2 ko diya jata hai, jahan document structure parse hota hai. fileciteurn1file1L131-L167

19. Hamare running example ka complete Layer 1 journey

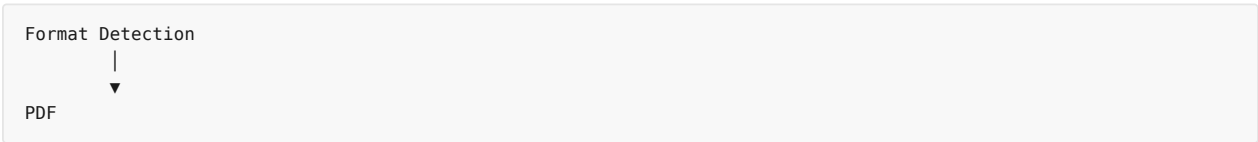
Input

Operating_Systems_Notes.pdf

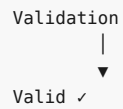
Step 1



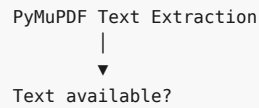
Step 2



Step 3



Step 4



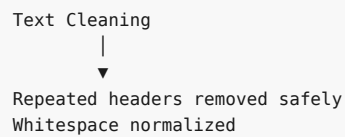
Suppose:

Yes

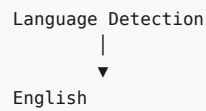
Then:

No OCR required

Step 5



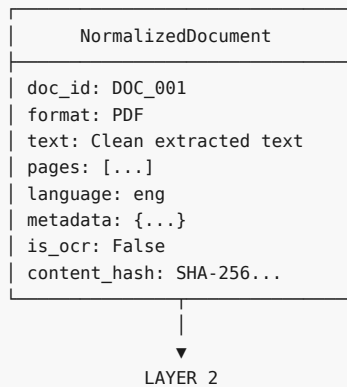
Step 6



Step 7

Metadata Extraction

Final



20. Layer 1 ki technologies ek line me


```

PDF      → PyMuPDF
Fallback → pdfplumber
Scanned  → Tesseract OCR
DOCX     → python-docx
HTML     → BeautifulSoup4
Language → langdetect
Duplicate → SHA-256 / hashlib

```

Ye Layer 1 ke file-specified technology choices hain. `filecite\turn2file0\L57-L67`

21. Layer 1 ke possible errors aur safeguards

```

| Problem | Layer 1 ka response |
|---|---|
| Duplicate file | SHA-256 se detect, reject |
| Unsupported format | Validation me reject |
| Very large input | Size/page limits validate |
| Complex PDF | Fallback extraction path |
| Scanned PDF | OCR |
| Low text but not scanned | Multi-signal OCR detection |
| OCR mistakes | OCR confidence calculate |
| Low OCR confidence | Manual review flag |
| Some pages fail | Partial extraction policy |
| Too many pages fail | Reject + manual review |
| Repeated header/footer | Frequency-based safe removal |
| Language detection fails | English fallback |

```

Ye safeguards report ke Layer 1 implementation aur policies me defined hain. `filecite\turn2file0\L69-L116`
`filecite\turn2file1\L172-L237`

22. Layer 1 ko ek simple real-world analogy se samjho

Maan lo ek **factory** hai.

Raw materials aate hain:

```

Different shapes
Different sizes
Different quality

```

Factory ka first gate:

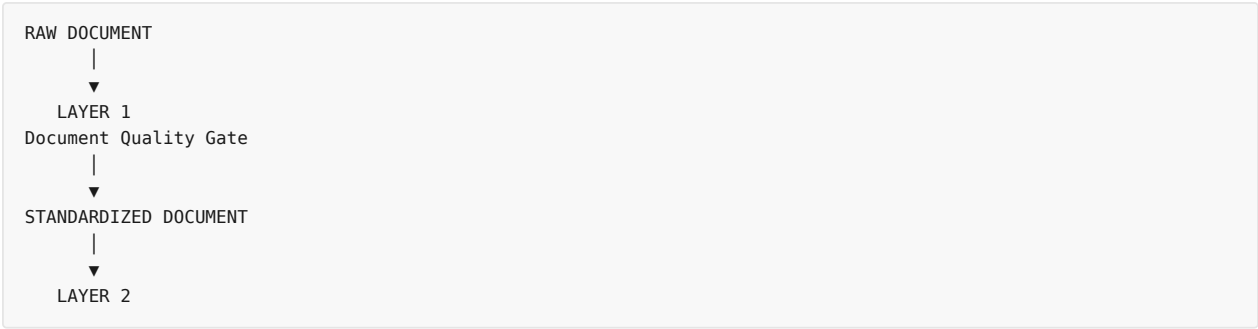
```

Check karo
  ↓
Valid hai?
  ↓
Duplicate nahi?
  ↓
Clean karo
  ↓
Standard form me convert karo

```

Uske baad hi actual manufacturing hoti hai.

Exactly same:



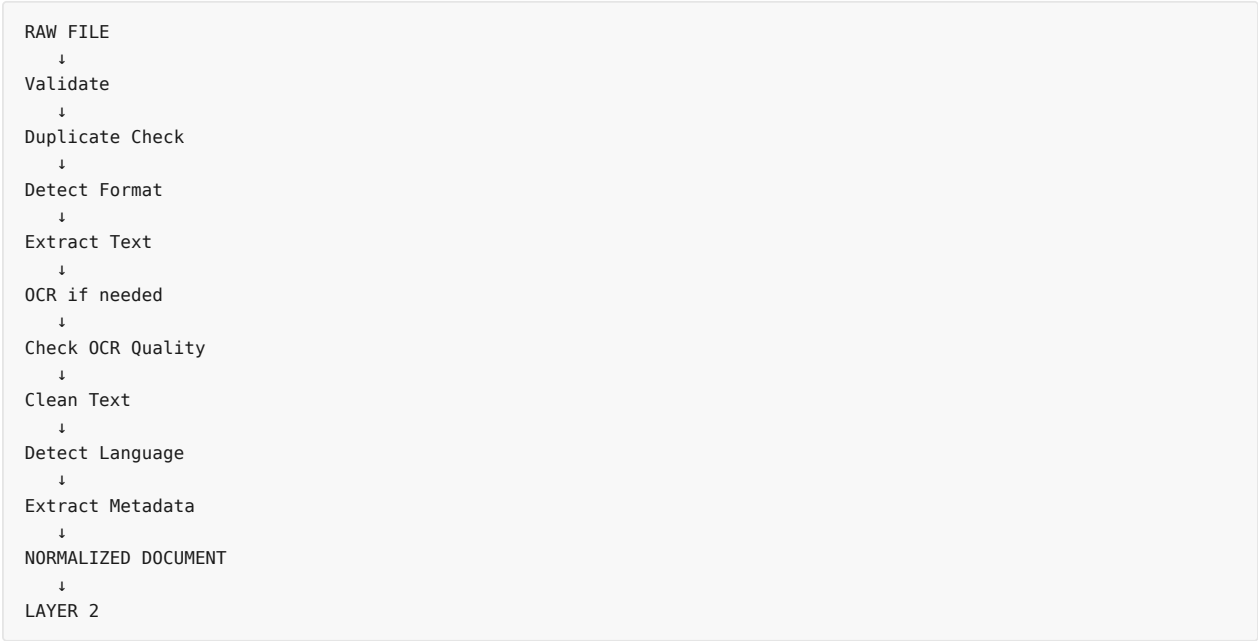
23. Layer 1 ka final summary

- | Point | Answer |
- |---|---|
- | **Layer Name** | Input Processing |
- | **Input** | Raw files |
- | **Supported formats** | PDF, DOCX, HTML, TXT |
- | **Main job** | Validate, deduplicate, extract, OCR, clean, standardize |
- | **Output** | NormalizedDocument |
- | **Duplicate algorithm** | SHA-256 hashing |
- | **PDF extraction** | PyMuPDF |
- | **Complex PDF fallback** | pdfplumber |
- | **Scanned documents** | Tesseract OCR |
- | **OCR detection** | Multi-signal detection |
- | **Language detection** | langdetect |
- | **Cleaning** | Frequency-based header/footer removal |
- | **Next Layer** | Layer 2 — Document Parsing |

Sabse important one-line understanding

*“Layer 1 converts a raw document into a clean, validated, deduplicated, machine-readable, and standardized **NormalizedDocument** , so that all later layers can process one common document format instead of handling different file types separately.”*

Complete flow yaad rakhna:



Layer 1 clear ho gaya to next hum Layer 2 me isi `NormalizedDocument` ko lenge aur dekhenge ki system usko `Page → Region → Block` structure me kaise convert karta hai — headings, paragraphs, tables, code, definitions aur reading order kaise identify hote hain.

YOU 23/08/2026, 22:18:58

metadata ki jarurat kyu padi??

CHATGPT 23/08/2026, 22:19:00

Haan, **bahut important question hai**. Tumhara doubt valid hai:

“Jab document ka actual text already store ho raha hai, to metadata alag se kyu chahiye?”

Simple answer

Text batata hai document ke andar kya likha hai.

Metadata batata hai document kya hai aur kahan se aaya hai.

Hamara example

Maan lo document ka text hai:

```
Round Robin is a preemptive CPU scheduling algorithm.
```

Sirf text store karne se system ko ye pata chalega:

KYA likha hai? ☒

Lekin ye nahi pata chalega:

```
Kis document se aaya? ✗
Author kaun hai? ✗
Document ka title kya hai? ✗
Kab publish hua? ✗
Original file kaun si hai? ✗
```

Yahan **metadata** kaam aata hai.

Example

```
NormalizedDocument
├── Text
│   └── "Round Robin is a preemptive..."
└── Metadata
    ├── Title: Operating Systems
    ├── Author: ABC
    ├── Date: 2026
    └── Source/File: OS_Notes.pdf
```

File ke `NormalizedDocument` structure me metadata document-related information, such as **author, title, and date**, store karne ke liye defined hai. `filecite{turn2file0-L37-L54}`

Metadata ki 3 main zaruratein

1. Source identify karne ke liye

Baad me system bole:

Ye information kis document se aayi?

Metadata help karega:

```
Round Robin information
    ↓
Source Document
    ↓
Operating Systems Notes
    ↓
Author / Date
```

2. Documents ko filter karne ke liye

Maan lo future me system ke paas 10,000 documents hain.

User query:

Give me information only from Operating Systems documents.

Metadata se filter:

```
All Documents
    ↓
Filter by metadata
    ↓
Domain/Title/Author/Date
    ↓
Relevant documents
```

3. Provenance and trust ke liye

Maan lo same concept do documents me hai:

```
Document A
Author: Unknown

Document B
Author: Official University Textbook
```

Future layers source information use karke answer ki **traceability** maintain kar sakti hain.

Sabse simple example jo tum Sir ko de sakte ho

"The text tells us what the document says, while metadata provides information about the document itself. For example, text may contain information about Round Robin, whereas metadata tells us the document title, author, date, and source. This helps in document identification, filtering, and maintaining source traceability."

Ek line me:

Content = What is written inside the document.

Metadata = Information about the document itself.

Aur yaad rakho: Layer 1 metadata ko knowledge samajhkar use nahi kar rahi; wo document ki identity aur context preserve kar rahi hai.

mujhe layer 1 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 1 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 23/08/2026, 22:20:40

{ "size": "1024x1024", "n": 1, "is_style_transfer": false, "transparent_background": false }

CHATGPT 23/08/2026, 22:21:24

Layer 1 – Input Processing

You can explain it to your guide like this:

“Layer 1 is the entry point of the SHIA-RAG system. Its main purpose is to accept raw documents in different formats and convert them into one clean and standardized format for the next layer.”

What happens in Layer 1?

- Accepts documents such as **PDF, DOCX, HTML, and TXT**.
- Checks whether the document is **valid and not a duplicate** using content hashing.
- Extracts the text using format-specific tools.
- If the document is scanned, it uses **OCR** to convert the image into text.
- Checks the **OCR confidence** to identify unreliable extraction.
- Cleans the extracted text and detects the document language.
- Extracts basic **metadata**, such as title, author, and date.

Output

The final output is a **NormalizedDocument**, which contains:

Clean Text + Pages + Metadata + Language + OCR Information + Document Identity

This standardized document is then passed to **Layer 2 for Document Parsing and Structure Identification**.

Simple conclusion

“In short, Layer 1 acts as a quality and standardization gate. It converts different types of raw documents into a single, reliable, machine-readable format so that all the next layers can process the document consistently.”

The supported formats, duplicate checking, extraction/OCR, confidence checks, cleaning, language detection, metadata, and **NormalizedDocument** output are based on the Layer 1 specification in the uploaded SHIA-RAG report.

filecite turn2file0L29-L67 filecite turn2file1L229-L265

YOU 23/08/2026, 22:34:10

ab mujhe layer 2 smjhao jese baki do layer smjhayi thi detailed me

CHATGPT 23/08/2026, 22:34:12

{ "paths": ["files"], "query": "search" }

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: filecite turn5file0

[L1] metadata:
 [L2] query_result_index: 0
 [L3] file_id: file_00000000dcc082119f935683a5e13eed
 [L4] version_id: 1
 [L5] name: SHIA_RAG_Final_Report(4)(1).md
 [L6] mime_type: text/markdown
 [L7] surface: conversation
 [L8] score: 0.03131881575727918
 [L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6
 [L10] snippet:
 [L11]

```
[L12] graph TD
[L13]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L14]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L15]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L16]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L17]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L18]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L19]     end
[L20]
[L21]     subgraph "Phase 2: Online – Retrieval & Generation"
[L22]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L23]         KF --> L6
[L24]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L25]         L7 --> A["Answer + Citations"]
[L26]     end
[L27]
[L28]     subgraph "Phase 3: Continuous – Operations"
[L29]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L30]         A --> L8
[L31]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L32]     end
[L33]
```

[L34]
 [L35] ### 3.2 Data Flow
 [L36]
 [L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L44] KnowledgeUnit objects (candidate concepts)
[L45]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47]     ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53]     ↓ Layer 7: Build prompt, invoke LLM
[L54]     ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56]     ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]
[L59] ### 3.3 Layer Summary
[L60]
[L61] | Layer | Name | Input | Output | Key Algorithms |
[L62] |-----|-----|-----|-----|-----|
[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L72]
[L73] ---
[L74]
[L75] ## 4. Data Model — Layer 0

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `filecite`turn5file1

[L76] metadata:
[L77] query_result_index: 1
[L78] file_id: file_00000000dcc082119f935683a5e13eed
[L79] version_id: 1
[L80] name: SHIA_RAG_Final_Report(4)(1).md
[L81] mime_type: text/markdown
[L82] surface: conversation
[L83] score: 0.03125763125763126
[L84] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--17
[L85] snippet:
[L86] 1)
[L87] if success_rate < 0.8:
[L88] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L89] elif fmt == 'docx':
[L90] text, pages = extract_docx(file_path), []
[L91] is_ocr, ocr_conf = False, 1.0
[L92] elif fmt == 'html':
[L93] text, pages = extract_html(file_path), []
[L94] is_ocr, ocr_conf = False, 1.0
[L95] else: # txt
[L96] text = open(file_path, 'r', encoding='utf-8').read()
[L97] pages, is_ocr, ocr_conf = [], False, 1.0
[L98]
[L99] language = detect_language(text)
[L100] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L101] metadata = extract_metadata(file_path, fmt)
[L102]

```

[L103] doc = NormalizedDocument(
[L104] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L105] format=DocumentFormat(fmt), text=text, pages=pages,
[L106] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L107] char_count=len(text), page_count=len(pages) or 1,
[L108] content_hash=content_hash, language=language,
[L109] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L110] )
[L111] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L112] return doc
[L113]

```

```

[L114]
[L115] ---
[L116]
[L117] ### 5.2 Layer 2: Document Parsing
[L118]
[L119] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with block cl
[L120]
[L121] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L122]
[L123]

```

python

```

[L124] class BlockType(Enum):
[L125] HEADING = "heading"
[L126] PARAGRAPH = "paragraph"
[L127] LIST = "list"
[L128] TABLE = "table"
[L129] CODE = "code"
[L130] ALGORITHM = "algorithm"
[L131] FIGURE_CAPTION = "figure_caption"
[L132] EQUATION = "equation"
[L133] DEFINITION = "definition"
[L134]
[L135] @dataclass
[L136] class DocumentBlock:
[L137] """Atomic unit of document structure."""
[L138] block_id: str
[L139] doc_id: str
[L140] page_number: int
[L141] region_index: int # Which region on the page
[L142] block_type: BlockType
[L143] text: str
[L144] reading_order: int # Global ordering across the document
[L145] confidence: float = 1.0 # Classification confidence
[L146] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L147]

```



```
[L148]
[L149] The parser uses rule-based heuristics for block classification:
[L150] - Heading: Short line (< 100 chars), no ending punctuation, starts with uppercase
[L151] - List: Starts with '- ', '* ', '• ', or '1.'
[L152] - Table: Contains 3+ pipe characters ('|') – uses regex `re.match(r'^(def |class |import |from )', line)`
[L153] - Code: Uses regex pattern matching (NOT `def " in text`, which falsely matches "definitely")
[L154] - Definition: Contains phrases like "is defined as", "refers to"
[L155] - Default: Paragraph
[L156]
[L157] ##### Table Cell-Level Parsing
[L158]
[L159] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level st
[L160]
[L161]
```

python

```
[L162] def parse_table_cells(page, table_region) -> list:
[L163] """Extract table structure with cell-level detail.
[L164]
[L165] Uses pdfplumber for text-based tables, Camelot for image-based.
[L166] Returns list of rows, each row is a list of cell strings."""
```

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `filecite`turn5file

```
[L167] metadata:
[L168] query_result_index: 2
[L169] file_id: file_00000000dcc082119f935683a5e13eed
[L170] version_id: 1
[L171] name: SHIA_RAG_Final_Report(4)(1).md
[L172] mime_type: text/markdown
[L173] surface: conversation
[L174] score: 0.028782894736842105
[L175] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--16
[L176] snippet:
[L177] strip()
[L178]
```

```
[L179]
[L180] ##### Language Detection
[L181]
[L182]
```

python

```
[L183] from langdetect import detect
[L184]
[L185] def detect_language(text: str) -> str:
[L186] """Detect document language for OCR configuration.
[L187]
[L188] Used to configure Tesseract's language parameter.
[L189] Falls back to 'eng' if detection fails."""
[L190] try:
[L191] lang = detect(text[:5000]) # Sample first 5000 chars
[L192] # Map langdetect codes to Tesseract codes
[L193] lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
```

```
[L194] return lang_map.get(lang, 'eng')
[L195] except Exception:
[L196] return 'eng'
[L197]
```

```
[L198]
[L199] ##### Partial Extraction Policy
[L200]
[L201] If some pages fail during extraction, the system uses a best-effort policy:
[L202]
[L203] | Condition | Action |
[L204] |-----|-----|
[L205] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |
[L206] | <80% pages succeed | Reject entire document, queue for manual review |
[L207] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L208]
[L209]
```

python

```
[L210] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L211] """Layer 1: Input Processing pipeline with all safeguards."""
[L212] # Step 0: Duplicate check
[L213] content_hash = check_duplicate(file_path, doc_registry)
[L214]
[L215] fmt = detect_format(file_path)
[L216] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L217]
[L218] failed_pages = []
[L219] if fmt == 'pdf':
[L220] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L221] # Partial extraction policy
[L222] success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L223] if success_rate < 0.8:
[L224] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L225] elif fmt == 'docx':
[L226] text, pages = extract_docx(file_path), []
[L227] is_ocr, ocr_conf = False, 1.0
[L228] elif fmt == 'html':
[L229] text, pages = extract_html(file_path), []
[L230] is_ocr, ocr_conf = False, 1.0
[L231] else: # txt
[L232] text = open(file_path, 'r', encoding='utf-8').read()
[L233] pages, is_ocr, ocr_conf = [], False, 1.0
[L234]
[L235] language = detect_language(text)
[L236] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L237] metadata = extract_metadata(file_path, fmt)
[L238]
[L239] doc = NormalizedDocument(
[L240] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L241] format=DocumentFormat(fmt), text=text, pages=pages,
[L242] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L243] char_count=len(text), page_count=len(pages) or 1,
[L244] content_hash=content_hash, language=language,
```

```
[L245] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L246] )
[L247] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L248] return doc
[L249] ``
[L250]
[L251] ---
[L252]
[L253] ### 5.2 Layer 2: Document Parsing
[L254]
[L255] Responsibility: Parse normalized text into a structured [Page → Region → Block] hierarchy with block
classification and reading order.
[L256]
[L257] Input: NormalizedDocument → Output: List of DocumentBlock`
```

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `filecite`turn5file3

```
[L258] metadata:
[L259] query_result_index: 3
[L260] file_id: file_00000000dcc082119f935683a5e13eed
[L261] version_id: 1
[L262] name: SHIA_RAG_Final_Report(4)(1).md
[L263] mime_type: text/markdown
[L264] surface: conversation
[L265] score: 0.02623004235907462
[L266] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--7
[L267] snippet:
[L268] deduplicated)
[L269] ↓ Layer 4: Validate relations against ontology, score confidence
[L270] KnowledgeEdge objects (typed, scored)
[L271] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L272] Knowledge Forest (rooted trees + cross-links)
[L273] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L274] Selected context nodes (within token budget)
[L275] ↓ Layer 7: Build prompt, invoke LLM
[L276] ↓ Layer 7: Verify claims against evidence, bind citations
[L277] Answer with citations + confidence score
[L278] ↓ Layer 8: Log, collect feedback, update forest weights
[L279]
```

```
[L280]
[L281] ### 3.3 Layer Summary
[L282]
[L283] | Layer | Name | Input | Output | Key Algorithms |
[L284] |-----|-----|-----|-----|-----|
[L285] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L286] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L287] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L288] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L289] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L290] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L291] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L292] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L293] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L294]
[L295] ---
[L296]
[L297] ## 4. Data Model – Layer 0
[L298]
[L299] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode`
[L300]
[L301] ### 4.1 KnowledgeNode Schema
[L302]
[L303] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L304]
[L305]
```

json

```
[L306] {
[L307] "node_id": "NODE_000789",
[L308] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L309] "canonical_name": "Round Robin",
[L310] "aliases": ["RR", "Round-Robin Scheduling"],
[L311] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L312] "summaries": ["CPU scheduling method using time slices"],
[L313] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L314] "domain": "Operating Systems",
[L315] "properties": {
[L316] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L317] "preemptive": {"type": "boolean", "value": true}
[L318] },
[L319] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L320] "parent": "NODE_000456",
[L321] "children": ["NODE_001001", "NODE_001002"],
[L322] "related": ["NODE_000890"],
[L323] "incoming_edges": ["EDGE_001"],
[L324] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L325] "evidence": [
[L326] {
[L327] "evidence_type": "DEFINITION",
[L328] "text": "Round Robin is a preemptive scheduling algorithm...",
[L329] "source_doc": "DOC_000001",
[L330] "source_block": "BLK_000234",
[L331] "page": 42
[L332] }
[L333] ],
```

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `filecite`turn5file4

[L334] metadata:
[L335] query_result_index: 4
[L336] file_id: file_00000000dcc082119f935683a5e13eed
[L337] version_id: 1
[L338] name: SHIA_RAG_Final_Report(4)(1).md
[L339] mime_type: text/markdown
[L340] surface: conversation
[L341] score: 0.030834914611005692
[L342] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
[L343] snippet:
[L344] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L345]
[L346] **### 2.3 Literature Search Methodology**
[L347]
[L348] | Parameter | Details |
[L349] |-----|-----|
[L350] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L351] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L352] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L353] | **Time Range** | 2020–2026 |
[L354] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L355]
[L356] ---
[L357]
[L358] **## 3. System Architecture**
[L359]
[L360] **### 3.1 Overview**
[L361]
[L362] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:
[L363]
[L364] | Phase | Layers | Mode | Purpose |
[L365] |-----|-----|-----|-----|
[L366] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |
[L367] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |
[L368] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |
[L369]
[L370] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.
[L371]
[L372]

```
[L373] graph TD
[L374]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L375]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L376]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L377]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L378]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L379]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L380]     end
[L381]
[L382]     subgraph "Phase 2: Online – Retrieval & Generation"
[L383]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L384]         KF --> L6
[L385]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L386]         L7 --> A["Answer + Citations"]
[L387]     end
[L388]
[L389]     subgraph "Phase 3: Continuous – Operations"
[L390]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L391]         A --> L8
[L392]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L393]     end
[L394]
```

[L395]

[L396] ### 3.2 Data Flow

[L397]

[L398] ```

[L399] Raw Document (PDF/DOCX/HTML)

[L400] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L401] Normalized Text + Metadata

[L402] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L403] [Page → Region → Block] hierarchy

[L404] ↓ Layer 3: Split sentences,

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `[[filecite]]turn5file5`

[L405] metadata:

[L406] query_result_index: 5

[L407] file_id: file_00000000dcc082119f935683a5e13eed

[L408] version_id: 1

[L409] name: SHIA_RAG_Final_Report(4)(1).md

[L410] mime_type: text/markdown

[L411] surface: conversation

[L412] score: 0.03028233151183971

[L413] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--18

[L414] snippet:

[L415] @dataclass

[L416] class DocumentBlock:

[L417] """Atomic unit of document structure."""

[L418] block_id: str

[L419] doc_id: str

[L420] page_number: int

[L421] region_index: int # Which region on the page

[L422] block_type: BlockType

```
[L423] text: str
[L424] reading_order: int # Global ordering across the document
[L425] confidence: float = 1.0 # Classification confidence
[L426] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L427]
```

```
[L428]
[L429] The parser uses rule-based heuristics for block classification:
[L430] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase
[L431] - **List:** Starts with '- ', '* ', '• ', or '1.'
[L432] - **Table:** Contains 3+ pipe characters (|) - uses regex `re.match(r'^(def |class |import |from )', line)`
[L433] - **Code:** Uses regex pattern matching (NOT `"def " in text`, which falsely matches "definitely")
[L434] - **Definition:** Contains phrases like "is defined as", "refers to"
[L435] - **Default:** Paragraph
[L436]
[L437] ##### Table Cell-Level Parsing
[L438]
[L439] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level st
[L440]
[L441]
```

python

```
[L442] def parse_table_cells(page, table_region) -> list:
[L443] """Extract table structure with cell-level detail.
[L444]
[L445] Uses pdfplumber for text-based tables, Camelot for image-based.
[L446] Returns list of rows, each row is a list of cell strings."""
[L447] try:
[L448] # Primary: pdfplumber table extraction
[L449] table = page.extract_table(table_region)
[L450] if table and len(table) > 1: # At least header + 1 data row
[L451] return table
[L452] except Exception:
[L453] pass
[L454]
[L455] # Fallback: Camelot for image-based tables
[L456] try:
[L457] import camelot
[L458] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L459] if tables:
[L460] return tables[0].df.values.tolist()
[L461] except Exception:
[L462] pass
[L463]
[L464] return None # Table structure could not be parsed
[L465]
```

```
[L466]
[L467] ##### Table Detection Fallback
[L468]
[L469] If table detection confidence is low, the block is flagged:
[L470]
[L471]
```

python

```
[L472] def classify_block_with_fallback(text, page) -> tuple:
```

[L473] """Returns (BlockType, confidence). Low-confidence tables get fallback."""

[L474] block_type, confidence = classify_block(text)

[L475]

[L476] if block_type == BlockType.TABLE and confidence < 0.6:

[L477] # Attempt cell-level parsing to confirm

[L478] cells = parse_table_cells(page, text)

[L479] if cells is None:

[L480] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph

[L481] else:

[L482] return BlockType.TABLE, 0.8 # Confirmed by parser

[L483]

[L484] return block_type, confidence

[L485]

[L486]

[L487] ##### Reading Order Validation

[L488]

[L489] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit

[L490]

[L491]

python

[L492] def assign_reading_order(blocks: list) -> list:

[L493] """Assign global reading order with confidence.

[L494]

[L495] Multi-column layouts can produce incorrect reading order.

[L496] We detect potential issues and flag low confidence."""

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `filecite`turn5file6

[L497] metadata:

[L498] query_result_index: 6

[L499] file_id: file_00000000dcc082119f935683a5e13eed

[L500] version_id: 1

[L501] name: SHIA_RAG_Final_Report(4)(1).md

[L502] mime_type: text/markdown

[L503] surface: conversation

[L504] score: 0.029631255487269532

[L505] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--11

[L506] snippet:

[L507] semantic_relations:

[L508] - USES # "TCP USES Congestion Control"

[L509] - CAUSES # "Deadlock CAUSES System Hang"

[L510] - DEPENDS_ON # "Virtual Memory DEPENDS_ON Paging"

[L511] - CONTRASTS_WITH # "Preemptive CONTRASTS_WITH Non-Preemptive"

[L512] - SIMILAR_TO # "Round Robin SIMILAR_TO Fair-Share"

[L513] - HAS_PROPERTY # "Round Robin HAS_PROPERTY Time Quantum"

[L514] - PRODUCES # "Compiler PRODUCES Machine Code"

[L515] - PREREQUISITE # "Data Structures PREREQUISITE Algorithms"

[L516]


```
[L517]
[L518] ### 4.4 Confidence Propagation
[L519]
[L520] Node confidence is propagated from roots to leaves using topological sort (Kahn's algorithm) to guarantee
[L521]
[L522]
```

python

```
[L523] from collections import deque
[L524]
[L525] # Confidence weighting factors (must sum to 1.0)
[L526] ALPHA = 0.4 # Node's own extraction confidence weight
[L527] BETA = 0.4 # Parent confidence weight
[L528] GAMMA = 0.2 # Evidence confidence weight
[L529] DECAY = 0.95 # Per-level decay factor
[L530] BASE_CONFIDENCE = 0.5 # Default for root nodes (configurable)
[L531]
[L532] def propagate_confidence(forest):
[L533] """Propagate confidence scores from roots to leaves.
[L534]
[L535] Uses Kahn's algorithm (topological ordering) to ensure
[L536] parents are always processed before their children.
[L537] Processing in arbitrary order would give children stale
[L538] parent confidence values."""
[L539] in_degree = {n.id: len(forest.parents(n)) for n in forest.nodes()}
[L540] queue = deque([n for n in forest.nodes() if in_degree[n.id] == 0]) # Roots first
[L541]
[L542] while queue:
[L543] node = queue.popleft()
[L544] if forest.parents(node):
[L545] parent_confs = [p.confidence for p in forest.parents(node)]
[L546] avg_parent_conf = sum(parent_confs) / len(parent_confs)
[L547] evidence_conf = (
[L548] sum(e.confidence for e in node.evidence) / len(node.evidence)
[L549] if node.evidence else 0.5
[L550] )
[L551] node.confidence = (
[L552] ALPHA node.extraction_confidence +
[L553] BETA avg_parent_conf +
[L554] GAMMA evidence_conf
[L555] ) DECAY
[L556] else:
[L557] node.confidence = node.extraction_confidence or BASE_CONFIDENCE
[L558]
[L559] for child in forest.children(node):
[L560] in_degree[child.id] -= 1
[L561] if in_degree[child.id] == 0:
[L562] queue.append(child)
[L563]
```

```
[L564]
[L565] ---
[L566]
[L567] ## 5. Offline Pipeline – Layers 1-5
[L568]
[L569] ### 5.1 Layer 1: Input Processing
[L570]
[L571] **Responsibility:** Accept raw documents (PDF, DOCX, HTML, TXT), validate format, deduplicate, extract text,
[L572]
[L573] **Input:** Raw document file → **Output:** `NormalizedDocument`
[L574]
[L575]
```

python

```
[L576] @dataclass
[L577] class NormalizedDocument:
[L578]     """Output of Layer 1 — ready for Layer 2 parsing."""
[L579]     doc_id: str
[L580]     original_filename: str
[L581]     format: DocumentFormat # PDF, DOCX, HTML, TXT
[L582]     text: str # Full extracted text
[L583]     pages: list # List of page texts (for PDFs)
[L584]     metadata: dict # Author, title, date, etc.
```

DEEP RESEARCH 23/08/2026, 22:34:20

Citation Marker: `filecite`turn5file7

```
[L585] metadata:
[L586] query_result_index: 7
[L587] file_id: file_00000000dcc082119f935683a5e13eed
[L588] version_id: 1
[L589] name: SHIA_RAG_Final_Report(4)(1).md
[L590] mime_type: text/markdown
[L591] surface: conversation
[L592] score: 0.029273504273504274
[L593] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--19
[L594] snippet:
[L595] Uses pdfplumber for text-based tables, Camelot for image-based.
[L596] Returns list of rows, each row is a list of cell strings."""
[L597] try:
[L598]     # Primary: pdfplumber table extraction
[L599]     table = page.extract_table(table_region)
[L600]     if table and len(table) > 1: # At least header + 1 data row
[L601]         return table
[L602]     except Exception:
[L603]         pass
[L604]
[L605]     # Fallback: Camelot for image-based tables
[L606]     try:
[L607]         import camelot
[L608]         tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L609]         if tables:
[L610]             return tables[0].df.values.tolist()
```

```
[L611] except Exception:
[L612] pass
[L613]
[L614] return None # Table structure could not be parsed
[L615]
```

```
[L616]
[L617] ##### Table Detection Fallback
[L618]
[L619] If table detection confidence is low, the block is flagged:
[L620]
[L621]
```

```
python
[L622] def classify_block_with_fallback(text, page) -> tuple:
[L623] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L624] block_type, confidence = classify_block(text)
[L625]
[L626] if block_type == BlockType.TABLE and confidence < 0.6:
[L627] # Attempt cell-level parsing to confirm
[L628] cells = parse_table_cells(page, text)
[L629] if cells is None:
[L630] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L631] else:
[L632] return BlockType.TABLE, 0.8 # Confirmed by parser
[L633]
[L634] return block_type, confidence
[L635]
```

```
[L636]
[L637] ##### Reading Order Validation
[L638]
[L639] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L640]
[L641]
```

```
python
[L642] def assign_reading_order(blocks: list) -> list:
[L643] """Assign global reading order with confidence.
[L644]
[L645] Multi-column layouts can produce incorrect reading order.
[L646] We detect potential issues and flag low confidence."""
[L647] # Sort by page, then y-coordinate, then x-coordinate
[L648] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L649]
[L650] for i, block in enumerate(sorted_blocks):
[L651] block.reading_order = i
[L652]
[L653] # Detect potential multi-column issues
[L654] if i > 0:
[L655] prev = sorted_blocks[i - 1]
[L656] if (prev.page_number == block.page_number and
[L657] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L658] prev.x_pos > block.x_pos): # But previous is to the right
```

[L659] block.confidence *= 0.7 # Reduce confidence — possible column issue

[L660]

[L661] return sorted_blocks

[L662]

```
[L663]
[L664] ---
[L665]
[L666] ### 5.3 Layer 3: Knowledge Extraction
[L667]
[L668] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L669]
[L670] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L671]
[L672] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L673]
[L674] #### Extraction Prompt
[L675]
[L676]
```

[L677] You are a knowledge extraction system. Given a text block, extract:

[L678] 1. Concepts: Named entities, technical terms

[L679] 2. Definitions: If the text defines any concept, extract the definition

[L680] 3. Properties: Key attributes with types and values

[L681] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L682] 5. Domain: Subject area (e.g., "Operating Systems")

DEEP RESEARCH 23/08/2026, 22:34:24

Citation Marker: `filecite`turn6file0

[L1] metadata:

[L2] query_result_index: 0

[L3] file_id: file_00000000dcc082119f935683a5e13eed

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4)(1).md

[L6] mime_type: text/markdown

[L7] surface: conversation

[L8] score: 0.03252247488101534

[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--17

[L10] snippet:

[L11] 1)

[L12] if success_rate < 0.8:

[L13] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")

[L14] elif fmt == 'docx':

[L15] text, pages = extract_docx(file_path), []

[L16] is_ocr, ocr_conf = False, 1.0

[L17] elif fmt == 'html':

[L18] text, pages = extract_html(file_path), []

[L19] is_ocr, ocr_conf = False, 1.0

[L20] else: # txt

[L21] text = open(file_path, 'r', encoding='utf-8').read()

[L22] pages, is_ocr, ocr_conf = [], False, 1.0

[L23]

```

[L24] language = detect_language(text)
[L25] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L26] metadata = extract_metadata(file_path, fmt)
[L27]
[L28] doc = NormalizedDocument(
[L29] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L30] format=DocumentFormat(fmt), text=text, pages=pages,
[L31] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L32] char_count=len(text), page_count=len(pages) or 1,
[L33] content_hash=content_hash, language=language,
[L34] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L35] )
[L36] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L37] return doc
[L38]

```

```

[L39]
[L40] ---
[L41]
[L42] ### 5.2 Layer 2: Document Parsing
[L43]
[L44] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with block cla
[L45]
[L46] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L47]
[L48]

```

python

```

[L49] class BlockType(Enum):
[L50] HEADING = "heading"
[L51] PARAGRAPH = "paragraph"
[L52] LIST = "list"
[L53] TABLE = "table"
[L54] CODE = "code"
[L55] ALGORITHM = "algorithm"
[L56] FIGURE_CAPTION = "figure_caption"
[L57] EQUATION = "equation"
[L58] DEFINITION = "definition"
[L59]
[L60] @dataclass
[L61] class DocumentBlock:
[L62] """Atomic unit of document structure."""
[L63] block_id: str
[L64] doc_id: str
[L65] page_number: int
[L66] region_index: int # Which region on the page
[L67] block_type: BlockType
[L68] text: str
[L69] reading_order: int # Global ordering across the document
[L70] confidence: float = 1.0 # Classification confidence
[L71] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L72]

```

```

[L73]
[L74] The parser uses rule-based heuristics for block classification:
[L75] - Heading: Short line (< 100 chars), no ending punctuation, starts with uppercase
[L76] - List: Starts with '-', '*', or '1.'
[L77] - Table: Contains 3+ pipe characters (|) - uses regex `re.match(r'^(def |class |import |from )', line)`
[L78] - Code: Uses regex pattern matching (NOT `def " in text`, which falsely matches "definitely")
[L79] - Definition: Contains phrases like "is defined as", "refers to"
[L80] - Default: Paragraph
[L81]
[L82] Table Cell-Level Parsing
[L83]
[L84] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level str
[L85]
[L86]

```

python

[L87] def parse_table_cells(page, table_region) -> list:

[L88] """Extract table structure with cell-level detail.

[L89]

[L90] Uses pdfplumber for text-based tables, Camelot for image-based.

[L91] Returns list of rows, each row is a list of cell strings."""

DEEP RESEARCH 23/08/2026, 22:34:24

Citation Marker: `filecite`turn6file1

[L92] metadata:

[L93] query_result_index: 1

[L94] file_id: file_00000000dcc082119f935683a5e13eed

[L95] version_id: 1

[L96] name: SHIA_RAG_Final_Report(4)(1).md

[L97] mime_type: text/markdown

[L98] surface: conversation

[L99] score: 0.02803379416282642

[L100] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--18

[L101] snippet:

[L102] @dataclass

[L103] class DocumentBlock:

[L104] """Atomic unit of document structure."""

[L105] block_id: str

[L106] doc_id: str

[L107] page_number: int

[L108] region_index: int # Which region on the page

[L109] block_type: BlockType

[L110] text: str

[L111] reading_order: int # Global ordering across the document

[L112] confidence: float = 1.0 # Classification confidence

[L113] table_cells: list = None # Parsed table cells (for TABLE blocks only)

[L114]

```

[L115]
[L116] The parser uses rule-based heuristics for block classification:
[L117] - Heading: Short line (< 100 chars), no ending punctuation, starts with uppercase
[L118] - List: Starts with '- ', '* ', '• ', or '1.'
[L119] - Table: Contains 3+ pipe characters ('|') – uses regex `re.match(r'^(def |class |import |from )', line)`
[L120] - Code: Uses regex pattern matching (NOT `"def " in text`, which falsely matches "definitely")
[L121] - Definition: Contains phrases like "is defined as", "refers to"
[L122] - Default: Paragraph
[L123]
[L124] ### Table Cell-Level Parsing
[L125]
[L126] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level st
[L127]
[L128]

```

python

```

[L129] def parse_table_cells(page, table_region) -> list:
[L130] """Extract table structure with cell-level detail.
[L131]
[L132] Uses pdfplumber for text-based tables, Camelot for image-based.
[L133] Returns list of rows, each row is a list of cell strings."""
[L134] try:
[L135] # Primary: pdfplumber table extraction
[L136] table = page.extract_table(table_region)
[L137] if table and len(table) > 1: # At least header + 1 data row
[L138] return table
[L139] except Exception:
[L140] pass
[L141]
[L142] # Fallback: Camelot for image-based tables
[L143] try:
[L144] import camelot
[L145] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L146] if tables:
[L147] return tables[0].df.values.tolist()
[L148] except Exception:
[L149] pass
[L150]
[L151] return None # Table structure could not be parsed
[L152]

```

```

[L153]
[L154] ### Table Detection Fallback
[L155]
[L156] If table detection confidence is low, the block is flagged:
[L157]
[L158]

```

python

```

[L159] def classify_block_with_fallback(text, page) -> tuple:
[L160] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L161] block_type, confidence = classify_block(text)
[L162]
[L163] if block_type == BlockType.TABLE and confidence < 0.6:
[L164] # Attempt cell-level parsing to confirm
[L165] cells = parse_table_cells(page, text)

```

```
[L166] if cells is None:
[L167] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L168] else:
[L169] return BlockType.TABLE, 0.8 # Confirmed by parser
[L170]
[L171] return block_type, confidence
[L172]
```

```
[L173]
[L174] ##### Reading Order Validation
[L175]
[L176] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L177]
[L178]
```

```
python
[L179] def assign_reading_order(blocks: list) -> list:
[L180] """Assign global reading order with confidence.
[L181]
[L182] Multi-column layouts can produce incorrect reading order.
[L183] We detect potential issues and flag low confidence."""
```

DEEP RESEARCH 23/08/2026, 22:34:24

Citation Marker: `filecite`turn6file2

```
[L184] metadata:
[L185] query_result_index: 2
[L186] file_id: file_00000000dcc082119f935683a5e13eed
[L187] version_id: 1
[L188] name: SHIA_RAG_Final_Report(4)(1).md
[L189] mime_type: text/markdown
[L190] surface: conversation
[L191] score: 0.032266458495966696
[L192] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
[L193] snippet:
[L194] Gap: No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3)
hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L195]
[L196] ### 2.3 Literature Search Methodology
[L197]
[L198] | Parameter | Details |
[L199] |-----|-----|
[L200] | Databases Searched | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L201] | Conferences/Venues | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L202] | Search Keywords | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval
augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L203] | Time Range | 2020–2026 |
[L204] | Key Papers Reviewed | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk
(LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L205]
[L206] ---
[L207]
```


[L208] ## 3. System Architecture

[L209]

[L210] ### 3.1 Overview

[L211]

[L212] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L213]

[L214] | Phase | Layers | Mode | Purpose |

[L215] |-----|-----|-----|-----|

[L216] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest

|

[L217] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L218] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L219]

[L220] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L221]

[L222]

```
[L223] graph TD
[L224]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L225]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L226]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L227]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L228]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L229]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L230]     end
[L231]
[L232]     subgraph "Phase 2: Online – Retrieval & Generation"
[L233]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L234]         KF --> L6
[L235]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L236]         L7 --> A["Answer + Citations"]
[L237]     end
[L238]
[L239]     subgraph "Phase 3: Continuous – Operations"
[L240]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L241]         A --> L8
[L242]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L243]     end
[L244]
```

[L245]

[L246] ### 3.2 Data Flow

[L247]

[L248] ```

[L249] Raw Document (PDF/DOCX/HTML)

[L250] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L251] Normalized Text + Metadata

[L252] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L253] [Page → Region → Block] hierarchy

[L254] ↓ Layer 3: Split sentences,

[L255] metadata:

[L256] query_result_index: 3

[L257] file_id: file_00000000dcc082119f935683a5e13eed

[L258] version_id: 1

[L259] name: SHIA_RAG_Final_Report(4)(1).md

[L260] mime_type: text/markdown

[L261] surface: conversation

[L262] score: 0.03028233151183971

[L263] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--35

[L264] snippet:

[L265] ##### Prompt Template

[L266]

[L267]

[L268] SYSTEM_PROMPT = ""You are a knowledge-grounded question answering system.

[L269] RULES:

[L270] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.

[L271] 2. For each claim, cite the source using [NODE_ID] format.

[L272] 3. If evidence is insufficient, say "Insufficient evidence to answer."

[L273] 4. If evidence conflicts, present both views with citations.

[L274]

[L275] REQUIRED OUTPUT FORMAT:

[L276] {

[L277] "answer": "Your answer with [NODE_ID] citations inline.",

[L278] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],

[L279] "overall_confidence": 0.0-1.0,

[L280] "insufficient_evidence": false

[L281] }""

[L282]

[L283]

[L284] ##### Claim Verification

[L285]

[L286] After the LLM generates an answer, each claim is verified against its cited evidence:

[L287]

[L288]

```

[L289] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L290]     """Returns (status, confidence).
[L291]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L292]     - PARTIAL:      evidence partially supports (0.4 < confidence ≤ 0.7)
[L293]     - UNSUPPORTED:  no evidence supports (confidence ≤ 0.4)
[L294]     - CONFLICTING:  evidence items contradict each other
[L295]     """
[L296]     scores = []
[L297]     for eid in evidence_ids:
[L298]         node = node_map.get(eid)
[L299]         if node and node.evidence:
[L300]             evidence_text = ' '.join(e.text for e in node.evidence)
[L301]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L302]
[L303]     if not scores:
[L304]         return 'UNSUPPORTED', 0.0
[L305]
[L306]     avg_score = sum(scores) / len(scores)
[L307]
[L308]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L309]         return 'CONFLICTING', avg_score
[L310]     if avg_score > 0.7:
[L311]         return 'SUPPORTED', avg_score
[L312]     elif avg_score > 0.4:
[L313]         return 'PARTIAL', avg_score
[L314]     else:
[L315]         return 'UNSUPPORTED', avg_score
[L316]

```

[L317]

[L318] ##### Unsupported Claim Policy

[L319]

[L320] | Status | Action |

[L321] |-----|-----|

[L322] | SUPPORTED | Keep as-is with citation |

[L323] | PARTIAL | Keep but mark as partially supported |

[L324] | UNSUPPORTED | Remove from answer, note at end |

[L325] | CONFLICTING | Present both views with note |

[L326]

[L327] ##### Answer Confidence Formula

[L328]

[L329]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{selected nodes}}{\text{cited nodes}}$$

[L330]

[L331] ##### Output Schema

[L332]

[L333]

```

[L334] @dataclass
[L335] class GenerationResult:
[L336]     """Output of Layer 7 – ready for user and Layer 8."""
[L337]     answer: str                # Answer text with citations
[L338]     claims: list              # List of VerifiedClaim
[L339]     citations: list           # Formatted citation list
[L340]     overall_confidence: float # 0.0-1.0
[L341]     retrieval_result: object  # Preserved for Layer 8
[L342]     generation_metadata: dict # Model used, claim counts, etc.
[L343]

```

[L344]

[L345] ---

[L346]

[L347] ## 7. Operations Layer — Layer 8

[L348]

[L349] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L350]

[L351] ### 7.1 Logging

[L352]

[L353] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 23/08/2026, 22:34:24

Citation Marker: `filecite`turn6file4

[L354] metadata:

[L355] query_result_index: 4

[L356] file_id: file_00000000dcc082119f935683a5e13eed

[L357] version_id: 1

[L358] name: SHIA_RAG_Final_Report(4)(1).md

[L359] mime_type: text/markdown

[L360] surface: conversation

[L361] score: 0.029726775956284153

[L362] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--19

[L363] snippet:

[L364] Uses pdfplumber for text-based tables, Camelot for image-based.

[L365] Returns list of rows, each row is a list of cell strings. """

[L366] try:

[L367] # Primary: pdfplumber table extraction

[L368] table = page.extract_table(table_region)

[L369] if table and len(table) > 1: # At least header + 1 data row

[L370] return table

[L371] except Exception:

[L372] pass

[L373]

[L374] # Fallback: Camelot for image-based tables

[L375] try:

[L376] import camelot

[L377] tables = camelot.read_pdf(page.source_path, pages=str(page.number))

[L378] if tables:

[L379] return tables[0].df.values.tolist()

[L380] except Exception:

[L381] pass

[L382]

[L383] return None # Table structure could not be parsed

[L384]

```
[L385]
[L386] ##### Table Detection Fallback
[L387]
[L388] If table detection confidence is low, the block is flagged:
[L389]
[L390]
```

python

```
[L391] def classify_block_with_fallback(text, page) -> tuple:
[L392] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L393] block_type, confidence = classify_block(text)
[L394]
[L395] if block_type == BlockType.TABLE and confidence < 0.6:
[L396] # Attempt cell-level parsing to confirm
[L397] cells = parse_table_cells(page, text)
[L398] if cells is None:
[L399] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L400] else:
[L401] return BlockType.TABLE, 0.8 # Confirmed by parser
[L402]
[L403] return block_type, confidence
[L404]
```

```
[L405]
[L406] ##### Reading Order Validation
[L407]
[L408] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L409]
[L410]
```

python

```
[L411] def assign_reading_order(blocks: list) -> list:
[L412] """Assign global reading order with confidence.
[L413]
[L414] Multi-column layouts can produce incorrect reading order.
[L415] We detect potential issues and flag low confidence."""
[L416] # Sort by page, then y-coordinate, then x-coordinate
[L417] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L418]
[L419] for i, block in enumerate(sorted_blocks):
[L420] block.reading_order = i
[L421]
[L422] # Detect potential multi-column issues
[L423] if i > 0:
[L424] prev = sorted_blocks[i - 1]
[L425] if (prev.page_number == block.page_number and
[L426] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L427] prev.x_pos > block.x_pos): # But previous is to the right
[L428] block.confidence *= 0.7 # Reduce confidence — possible column issue
[L429]
[L430] return sorted_blocks
[L431]
```

```
[L432]
[L433] ---
[L434]
[L435] ### 5.3 Layer 3: Knowledge Extraction
[L436]
[L437] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L438]
[L439] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L440]
[L441] **Layer 3 / Layer 4 boundary:** Layer 3 performs candidate extraction – it identifies entities, definitio
[L442]
[L443] #### Extraction Prompt
[L444]
[L445]
```

[L446] You are a knowledge extraction system. Given a text block, extract:

- [L447] 1. Concepts: Named entities, technical terms
- [L448] 2. Definitions: If the text defines any concept, extract the definition
- [L449] 3. Properties: Key attributes with types and values
- [L450] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
- [L451] 5. Domain: Subject area (e.g., "Operating Systems")

CHATGPT 23/08/2026, 22:34:24

Bilkul. Ab hum **Layer 2 ko same style me, detail se aur same running example ke saath** samjhenge.

Hum wahi example use karenge:

Operating_Systems_Notes.pdf, jisme topics hain: *CPU Scheduling, Round Robin, Time Quantum, etc.*

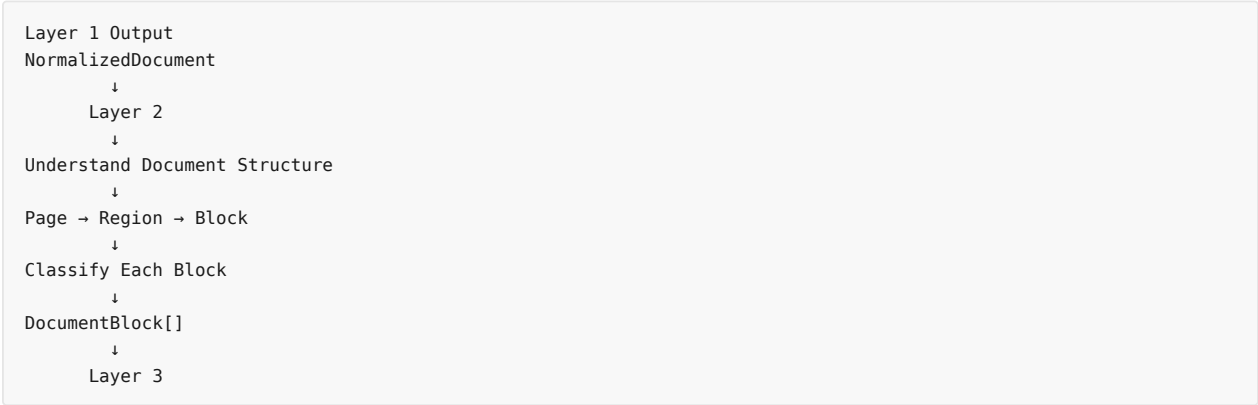
Layer 1 ne is raw PDF ko already process karke ek **NormalizedDocument** bana diya hai. Ab **Layer 2 ka kaam start** hota hai.

Layer 2 — Document Parsing

Sabse pehle ek line me samjho

Layer 2 takes the clean document from Layer 1 and understands its structure. It divides the document into structured blocks, identifies the type of each block, and assigns the correct reading order.

Simple flow:



File ke according, Layer 2 ki responsibility normalized text ko **[Page → Region → Block] hierarchy** me parse karna hai, with **block classification and reading order**. Input **NormalizedDocument** hai aur output **DocumentBlock** objects ki list hai.

fileciteurn6file0L42-L71

1. Layer 2 ki zarurat kyu padi?

Layer 1 se hume clean text mil gaya.

Maan lo Layer 1 output:

```
CPU SCHEDULING

CPU scheduling decides which process gets the CPU.

Round Robin

Round Robin is a preemptive CPU scheduling algorithm.

Properties of Round Robin:
- Uses a fixed time quantum
- Preemptive

Algorithm:
for each process:
    execute for time quantum
```

Ab agar Layer 3 ko directly ye pura text de diya:

```
"CPU SCHEDULING CPU scheduling decides..."
```

to problem kya hai?

System ko pata nahi chalega:

- CPU SCHEDULING heading hai?
- Uske neeche wala text paragraph hai?
- Round Robin ek new heading hai?
- Bullet points list hain?
- for each process: code/algorithm hai?
- Kis content ko kis order me read karna hai?

Isliye Layer 2 document ko **sirf text nahi, structured content** me convert karti hai.

2. Layer 1 vs Layer 2 — Sabse important difference

Layer 1

Question:

"Document ke andar kya text hai?"

Example output:

```
CPU Scheduling
Round Robin
Time Quantum
...
```

Layer 2

Question:

"Ye text document ke andar kis structure me organized hai?"

Example output:

```

CPU Scheduling → HEADING

CPU scheduling decides... → PARAGRAPH

Round Robin → HEADING

Round Robin is a... → DEFINITION/PARAGRAPH

- Uses fixed time quantum → LIST

for each process: → CODE/ALGORITHM

```

Short difference:

Layer 1 extracts the content. Layer 2 understands the structure of that content.

3. Layer 2 ka Input kya hai?

Layer 2 ka input hai:

```
NormalizedDocument
```

Jo Layer 1 ne create kiya tha.

Conceptually:

```

NormalizedDocument(
  doc_id = "DOC_001",

  text = "CPU Scheduling...\nRound Robin...",

  pages = [
    "Page 1 text...",
    "Page 2 text..."
  ],

  metadata = {...},

  language = "eng",

  ...
)

```

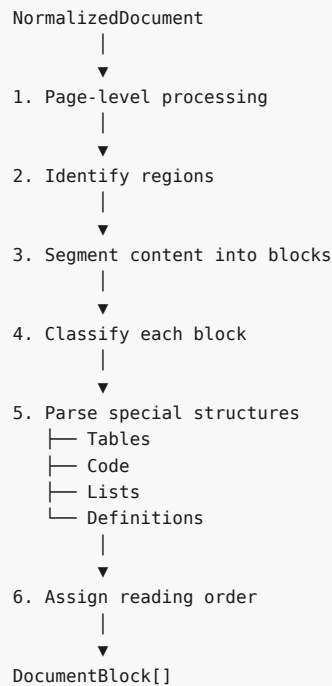
Ab Layer 2 primarily dekhti hai:

1. Document ka text
2. Page information
3. Content ki position/layout information where available

Source architecture ke data flow ke hisaab se Layer 1 ka normalized text + metadata Layer 2 me aata hai, jahan layout detection, block segmentation, classification aur reading order hota hai. [filecite\[turn5file0L35-L57\]](https://arxiv.org/abs/2505.15701)

4. Layer 2 internally kya karti hai?

Layer 2 ka main flow:

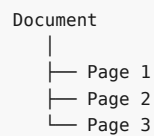


5. Sabse pehle — Page → Region → Block kya hai?

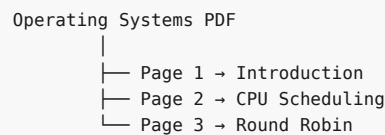
Ye Layer 2 ka core concept hai.

Level 1: Page

Document:



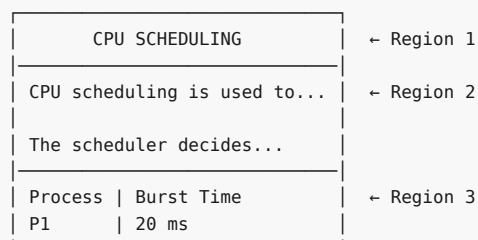
Example:



Level 2: Region

Ek page ke andar alag-alag content areas ho sakte hain.

Example page:



So:

```

Page 2
|
├─ Region 1 → Heading Area
├─ Region 2 → Main Text Area
└─ Region 3 → Table Area
  
```

`DocumentBlock` me `region_index` specifically indicate karta hai ki block page ke kis region se aaya hai.

☐filecite☐turn6file0☐L60-L71☐

Important note

File **Page** → **Region** → **Block hierarchy** clearly define karti hai, lekin source snippet exact region-detection algorithm specify nahi karta. Isliye is report ke basis par hum confidently ye keh sakte hain ki Layer 2 regions ko structural organization ke liye use karti hai, but exact region segmentation method yahan defined nahi hai. ☐filecite☐turn6file0☐L42-L46☐

6. Level 3 — Block kya hota hai?

Block = document ka atomic structural unit

Example:

```
CPU SCHEDULING
```

Ek block.

```
CPU scheduling decides which process gets CPU time.
```

Ek alag block.

```

- Round Robin
- FCFS
- Priority Scheduling
  
```

Ek list block.

```

for each process:
    execute()
  
```

Ek code/algorithm block.

File `DocumentBlock` ko "**Atomic unit of document structure**" define karti hai. ☐filecite☐turn5file5☐L415-L426☐

7. Ab hamare example ko Layer 2 process karegi

Layer 1 se input aaya:

```
CPU SCHEDULING

CPU scheduling decides which process gets CPU time.

ROUND ROBIN

Round Robin is defined as a preemptive CPU scheduling algorithm.

Properties:
- Uses a fixed time quantum
- Each process gets equal CPU time

for each process:
    execute for time_quantum
```

Layer 2 ise blocks me divide karegi:

```
Block 1:
CPU SCHEDULING

Block 2:
CPU scheduling decides which process gets CPU time.

Block 3:
ROUND ROBIN

Block 4:
Round Robin is defined as a preemptive CPU scheduling algorithm.

Block 5:
- Uses a fixed time quantum
- Each process gets equal CPU time

Block 6:
for each process:
    execute for time_quantum
```

Lekin sirf divide karna enough nahi hai.

Ab har block ka **type identify** karna hai.

8. Block Classification — Ye Layer 2 ka major kaam hai

File ke according possible block types hain:

```
HEADING
PARAGRAPH
LIST
TABLE
CODE
ALGORITHM
FIGURE_CAPTION
EQUATION
DEFINITION
```

filecite\turn6file0\48-L71

Ab hamare example me:

Block	Content	Type
1	CPU SCHEDULING	HEADING
2	CPU scheduling decides...	PARAGRAPH
3	ROUND ROBIN	HEADING

| 4 | Round Robin is defined as... | DEFINITION |

| 5 | - Uses fixed time quantum... | LIST |

| 6 | for each process... | CODE / ALGORITHM |

9. System kaise decide karta hai ki Heading hai?

File me rule-based heuristics use kiye gaye hain.

Heading ke signals:

- ✓ Short line (< 100 characters)
- ✓ Ending punctuation nahi
- ✓ Uppercase se start hoti hai

Example:

```
ROUND ROBIN
```

Check:

```
Length < 100?      ✓
Ends with " ." ?   No ✓
Starts uppercase?  ✓
```

Result:

```
Block Type = HEADING
```

Source me ye exact heading heuristics diye gaye hain. `filecite\turn6file0\L74-L80`

10. List kaise detect hoti hai?

Example:

- Uses time quantum
- Preemptive
- Fair CPU allocation

System patterns check karta hai:

```
-
*
•
1.
```

Agar line in patterns se start hoti hai:

```
Block Type = LIST
```

File me list detection ke liye ye exact markers defined hain. `filecite\turn6file0\L74-L80`

11. Definition kaise detect hoti hai?

Example:

```
Round Robin is defined as a preemptive scheduling algorithm.
```

System phrases detect karta hai:

```
"is defined as"
"refers to"
```

Agar definition-like phrase mili:

```
Block Type = DEFINITION
```

Why important?

Because:

```
Round Robin
```

aur:

```
Round Robin is defined as...
```

dono equally important nahi hain.

Second block directly Layer 3 ko indicate karta hai:

```
"Yahan concept ki definition mil sakti hai."
```

Source me definition detection ke liye phrases like **"is defined as"** aur **"refers to"** use kiye gaye hain. [filecite\turn6file0\L74-L80](#)

12. Code block kaise detect hota hai?

Example:

```
def round_robin(processes):
    for process in processes:
        execute(process)
```

Layer 2 code-like patterns detect karti hai.

File me regex use hota hai, for example:

```
def
class
import
from
```

Aur report ek important point mention karti hai:

× Sirf **"def"** in text check nahi karna chahiye.

Kyunki:

```
"definitely"
```

me bhi **"def"** hai.

Isliye:

```
Regex pattern matching
```

use hoti hai.

Source specifically regex-based code detection mention karti hai. [filecite\turn5file5\L429-L435](#)

13. Table detection kaise hoti hai?

Maan lo extracted text:

Algorithm		Type
Round Robin		Preemptive
FCFS		Non-Preemptive
Priority		Depends

Layer 2 heuristic check kar sakti hai:

3 or more | characters

to identify a possible table.

But yahan ek problem hai.

Table ko plain text treat karne se structure lose ho sakta hai.

For example:

Round Robin | Preemptive

Agar simply sentence ban gaya:

Round Robin Preemptive

to:

Column relationship lost ❌

Isliye table blocks ke liye special processing hoti hai.

14. Table Cell-Level Parsing

File ke according table detect hone ke baad Layer 2 cell-level structure reconstruct karne ki koshish karti hai.

Example original table:

Algorithm	Type
Round Robin	Preemptive
FCFS	Non-Preemptive

Internally:

```
[
  ["Algorithm", "Type"],
  ["Round Robin", "Preemptive"],
  ["FCFS", "Non-Preemptive"]
]
```

Ye `table_cells` me store ho sakta hai.

Technology:

Primary

pdfplumber

Fallback

Camelot

File ke according pdfplumber text-based tables ke liye aur Camelot fallback table reconstruction ke liye use hota hai.
fileciteurlturn6file1L124-L151

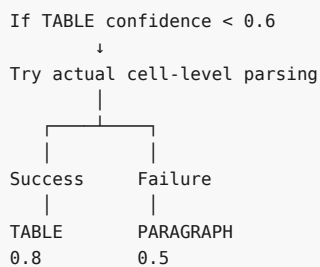
15. Table ke liye fallback kyu chahiye?

Maan lo classifier bolta hai:

"This is probably a table"
Confidence = 0.50

Lekin confidence low hai.

File me rule:



So system blindly galat table label ko accept nahi karta.

Agar actual table structure parse nahi ho pati:

Downgrade → PARAGRAPH

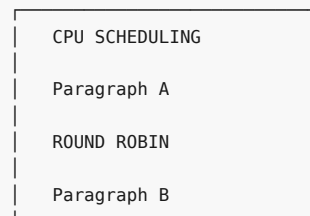
Agar successfully parse ho jati hai:

Confirm → TABLE

Ye fallback mechanism source me explicitly defined hai. fileciteurlturn6file1L154-L171

16. Ab Reading Order — Bahut important concept

Maan lo page par ye blocks hain:



Correct reading order:

- 1 → CPU SCHEDULING
- 2 → Paragraph A
- 3 → ROUND ROBIN
- 4 → Paragraph B

Agar Layer 2 galat order de:

- 1 → CPU SCHEDULING
- 2 → ROUND ROBIN
- 3 → Paragraph A
- 4 → Paragraph B

to context break ho jayega.

17. Reading Order assign kaise hota hai?

File ke according blocks ko sort kiya jata hai:

- 1. Page Number
- 2. Y-coordinate → Top to Bottom
- 3. X-coordinate → Left to Right

Conceptually:

```
sorted(
    blocks,
    key = (
        page_number,
        y_position,
        x_position
    )
)
```

Then:

```
First Block → reading_order = 0
Second Block → reading_order = 1
Third Block → reading_order = 2
```

File me exactly ye sorting strategy define hai. `filecite\turn6file4\L406-L430`

18. Multi-column document problem

Ab imagine karo research paper:

Column 1	Column 2
A1	B1
A2	B2
A3	B3

Correct reading order usually:

```
A1
A2
A3
B1
B2
B3
```

Lekin simple coordinate-based algorithm kabhi galat interleave kar sakta hai:

```
A1
B1
A2
B2
A3
B3
```


× Context break.

File specifically mention karti hai ki multi-column layouts reading order errors create kar sakte hain.

Isliye system suspicious patterns detect karke confidence reduce karta hai.

For example, same vertical band me:

Previous block right side par tha
Current block suddenly left side par hai

To:

`block.confidence × 0.7`

File me multi-column issue detection aur confidence reduction explicitly defined hai. `filecite\turn6file4\L406-L430`

19. DocumentBlock actual output kaisa dikhta hai?

Hamare example ka heading:

ROUND ROBIN

roughly:

```
DocumentBlock(
  block_id = "BLK_001",
  doc_id = "DOC_001",
  page_number = 15,
  region_index = 0,
  block_type = "HEADING",
  text = "ROUND ROBIN",
  reading_order = 42,
  confidence = 0.98,
  table_cells = None
)
```

Definition:

```
DocumentBlock(
  block_id = "BLK_002",
  doc_id = "DOC_001",
  page_number = 15,
  region_index = 0,
  block_type = "DEFINITION",
  text = "Round Robin is defined as a preemptive CPU scheduling algorithm.",
  reading_order = 43,
  confidence = 0.95,
  table_cells = None
)
```

Table:

```
DocumentBlock(  
  block_id = "BLK_003",  
  doc_id = "DOC_001",  
  page_number = 16,  
  region_index = 2,  
  block_type = "TABLE",  
  text = "Algorithm | Type ...",  
  reading_order = 44,  
  confidence = 0.80,  
  
  table_cells = [  
    ["Algorithm", "Type"],  
    ["Round Robin", "Preemptive"],  
    ["FCFS", "Non-Preemptive"]  
  ]  
)
```

Ye fields source ke `DocumentBlock` schema se directly supported hain. `filecite\turn5file5\L415-L426`

20. Har field ki zarurat kyu hai?

block_id

BLK_001

Har block ki unique identity.

doc_id

DOC_001

Block kis document se aaya?

Block → Document

Traceability.

page_number

15

Original document ke kis page se aaya?

Later evidence ke liye useful.

region_index

2

Page ke kis structural region se block aaya.

block_type

HEADING

Ya:

PARAGRAPH
LIST
TABLE
CODE
DEFINITION
...

Ye Layer 3 ko context provide karta hai.

text

Actual content.

Round Robin is defined as...

reading_order

Document me logical position.

42 → pehle
43 → baad me
44 → uske baad

confidence

Classification kitni reliable hai?

0.95 → High confidence
0.50 → Less certain

table_cells

Sirf table ke case me structured cells.

21. Ab hamare complete example ka flow dekho

Layer 1 Output

```
NormalizedDocument

Text:
CPU SCHEDULING

CPU scheduling decides which process gets CPU time.

ROUND ROBIN

Round Robin is defined as a preemptive CPU scheduling algorithm.

Properties:
- Uses fixed time quantum
- Preemptive

Algorithm | Type
Round Robin | Preemptive
FCFS | Non-Preemptive
```

Step 1 — Segment

```
B1 → CPU SCHEDULING

B2 → CPU scheduling decides which process gets CPU time.

B3 → ROUND ROBIN

B4 → Round Robin is defined as...

B5 → Properties:
    - Uses fixed time quantum
    - Preemptive

B6 → Algorithm | Type
    Round Robin | Preemptive
    FCFS       | Non-Preemptive
```

Step 2 — Classification

```
B1 → HEADING
B2 → PARAGRAPH
B3 → HEADING
B4 → DEFINITION
B5 → LIST / structural list content
B6 → TABLE
```

Step 3 — Reading Order

```
B1 → Order 1
B2 → Order 2
B3 → Order 3
B4 → Order 4
B5 → Order 5
B6 → Order 6
```

Step 4 — Final Output

```
[
  DocumentBlock(B1),
  DocumentBlock(B2),
  DocumentBlock(B3),
  DocumentBlock(B4),
  DocumentBlock(B5),
  DocumentBlock(B6)
]
```

This is:

```
DocumentBlock[]
```

22. Layer 2 ke baad Layer 3 ko kya milta hai?

Layer 3 ko ab ye nahi milega:

```
Ek bada raw text ❌
```

Usko milega:

```
Structured blocks ✓
```

Example:

Block 1
Type: HEADING
Text: "CPU SCHEDULING"

↓

Block 2
Type: PARAGRAPH
Text: "CPU scheduling decides..."

↓

Block 3
Type: HEADING
Text: "ROUND ROBIN"

↓

Block 4
Type: DEFINITION
Text: "Round Robin is defined as..."

Ab Layer 3 easily understand kar sakti hai:

"Round Robin ek important concept hai aur is block me iski definition di gayi hai."

Isliye Layer 2 aur Layer 3 ka connection important hai.

Source ke data flow me Layer 2 ka [Page → Region → Block] output next Layer 3 me knowledge extraction ke liye jata hai.

23. Layer 2 me kya technology use ho rahi hai?

Source ke according Layer 2 ke relevant tools:

- | Technology / Method | Use |
- | ---| ---|
- | Rule-based heuristics | Block classification |
- | Regular Expressions (Regex) | Code aur structural patterns |
- | pdfplumber | Table extraction |
- | Camelot | Table extraction fallback |
- | Coordinate sorting | Reading order |
- | Confidence scoring | Classification/reliability tracking |

24. Layer 2 ke possible errors

Error 1: Heading ko Paragraph samajh lena

ROUND ROBIN

could be classified incorrectly.

Safeguard

Short length
+ no punctuation
+ uppercase

Multiple heuristic signals use hote hain. `filecite\turn6file0\L74-L80`

Error 2: Code ko normal text samajhna

Example:

```
def execute():
```

Safeguard

Regex-based detection.

Error 3: False Code Detection

Word:

```
definitely
```

Agar "def" in text use kiya:

False CODE detection ❌

Safeguard

Proper regex matching. `filecite\turn5file5\L429-L435`

Error 4: Table galat detect hona

Text me `|` characters aa sakte hain, lekin actual table na ho.

Safeguard

```
Low confidence
  ↓
Cell-level parsing
  ↓
Confirmed?
 /      \
Yes      No
TABLE   PARAGRAPH
```

`filecite\turn6file1\L154-L171`

Error 5: Multi-column reading order galat hona

Safeguard

Suspicious column transitions detect karke confidence reduce kiya jata hai. `filecite\turn6file4\L406-L430`

25. Layer 2 me kya nahi ho raha?

Ye bahut important hai.

Layer 2:

- × Concept extraction nahi karti
- × "Round Robin" ko KnowledgeNode nahi banati
- × Semantic relationship create nahi karti

- × Embeddings generate nahi karti
- × Knowledge Forest nahi banati

Layer 2 sirf bolti hai:

"Document ka ye part heading hai, ye paragraph hai, ye definition hai, ye table hai — aur inhe kis order me read karna hai."

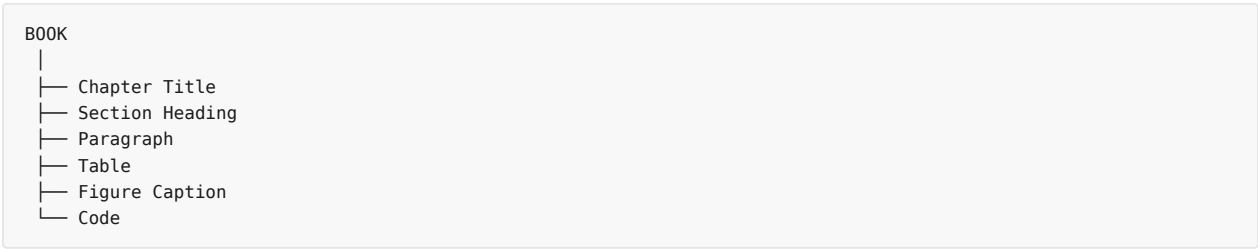
Actual concept extraction:

Layer 3

me hoga.

26. Ek real-world analogy

Imagine ek book:



Agar tum sirf OCR se book ka pura text extract kar lo:

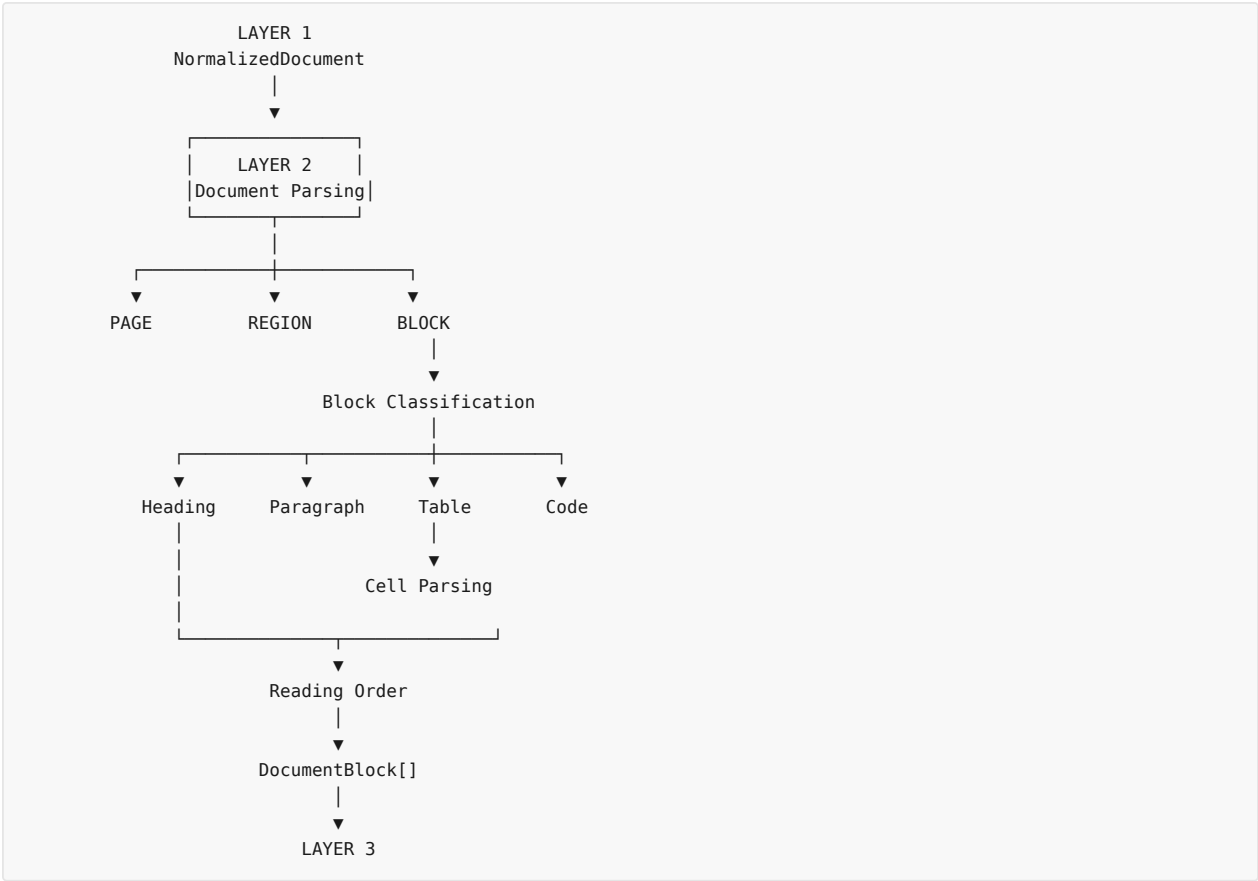
Sab text ek jagah

To structure lost ho sakta hai.

Layer 2 ek **librarian/editor** ki tarah kaam karti hai:

"Ye heading hai, ye paragraph hai, ye table hai, ye code hai, aur ye content iske baad read hoga."

27. Layer 2 ka complete architecture



28. Final Summary

- | Point | Layer 2 |
- |---|---|
- | **Name** | Document Parsing |
- | **Input** | NormalizedDocument |
- | **Main purpose** | Document ka structure understand karna |
- | **Hierarchy** | Page → Region → Block |
- | **Main output** | DocumentBlock[] |
- | **Block types** | Heading, Paragraph, List, Table, Code, Algorithm, Definition, etc. |
- | **Main classification method** | Rule-based heuristics |
- | **Code detection** | Regex |
- | **Table parsing** | pdfplumber + Camelot fallback |
- | **Reading order** | Page → Top-to-Bottom → Left-to-Right |
- | **Quality check** | Confidence scores |
- | **Next layer** | Layer 3 — Knowledge Extraction |

Sabse important one-line understanding

“Layer 2 does not extract knowledge yet. It first converts the normalized document into structured blocks, identifies what each block represents, and preserves the correct reading order. This structured output helps Layer 3 extract knowledge more accurately.”

Layer 1 → Layer 2 difference ek baar aur



Next logically Layer 3 hai, aur wahan se system first time actually **concepts, definitions, properties aur candidate relations** nikalna start karega—jaise hamare example me Round Robin ko actual knowledge concept ke roop me identify karna.

YOU 23/08/2026, 22:44:21

mujhe layer 2 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 2 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 23/08/2026, 22:44:24

{\"size\":\"1024x1024\", \"n\":1, \"transparent_background\":false, \"is_style_transfer\":false, \"referenced_image_ids\":null}

CHATGPT 23/08/2026, 22:48:15

Layer 2 – Document Parsing

You can explain it to your guide like this:

“Layer 2 receives the **NormalizedDocument** from Layer 1 and understands the structure of the document. It does not extract knowledge yet; instead, it identifies how the content is organized.”

What happens in Layer 2?

- The document is organized into a **Page → Region → Block** structure.
- Each region is divided into meaningful **content blocks**.
- Every block is classified as a **Heading, Paragraph, List, Table, Code, Algorithm, Definition**, etc.
- The system determines the correct **reading order** of the blocks.
- Special structures such as **tables and code** are parsed separately when required.

Output

The final output is a list of **DocumentBlock[]** , where each block contains information such as:

Block Type + Text + Page Number + Region + Reading Order + Confidence

This structured output is then passed to **Layer 3 for Knowledge Extraction**.

Simple conclusion

“In short, Layer 2 converts a clean document into a structured representation. It tells the system what each part of the document is and in what order it should be read, so that the next layer can extract knowledge more accurately.”

The explanation follows the file's Layer 2 structure of **Page → Region → Block**, block classification, reading order, special structure parsing, and **DocumentBlock[]** output. [filecite\turn6file0\42-L80](https://chatgpt.com/22bc3c39-b594-46a2-bdf0-c46ad675ec43)

YOU 23/08/2026, 22:57:36

ab mujhe layer 3 smjhao jese baki layer smjhayi detailed me

CHATGPT 23/08/2026, 22:57:40

{"paths":["files"],"query":"search"}

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite`turn7file0

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_00000000dcc082119f935683a5e13eed
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report(4)(1).md
- [L6] mime_type: text/markdown
- [L7] surface: conversation
- [L8] score: 0.03252247488101534
- [L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
- [L10] snippet:
- [L11] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
- [L12]
- [L13] **### 2.3 Literature Search Methodology**
- [L14]
- [L15] | Parameter | Details |
- [L16] |-----|-----|
- [L17] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
- [L18] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
- [L19] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
- [L20] | **Time Range** | 2020–2026 |
- [L21] | **Key Papers Reviewed** | RAPTOR (Sarthi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
- [L22]
- [L23] ---
- [L24]
- [L25] **## 3. System Architecture**
- [L26]
- [L27] **### 3.1 Overview**
- [L28]
- [L29] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:
- [L30]
- [L31] | Phase | Layers | Mode | Purpose |
- [L32] |-----|-----|-----|-----|
- [L33] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |
- [L34] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |
- [L35] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |
- [L36]

[L37] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L38]

[L39]

```
[L40] graph TD
[L41]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L42]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L43]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L44]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L45]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L46]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L47]     end
[L48]
[L49]     subgraph "Phase 2: Online – Retrieval & Generation"
[L50]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L51]         KF --> L6
[L52]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L53]         L7 --> A["Answer + Citations"]
[L54]     end
[L55]
[L56]     subgraph "Phase 3: Continuous – Operations"
[L57]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L58]         A --> L8
[L59]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L60]     end
[L61] end
```

[L62]

[L63] ### 3.2 Data Flow

[L64]

[L65] ```

[L66] Raw Document (PDF/DOCX/HTML)

[L67] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L68] Normalized Text + Metadata

[L69] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L70] [Page → Region → Block] hierarchy

[L71] ↓ Layer 3: Split sentences,

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `[[filecite]]turn7file1`

[L72] metadata:

[L73] query_result_index: 1

[L74] file_id: file_00000000dcc082119f935683a5e13eed

[L75] version_id: 1

[L76] name: SHIA_RAG_Final_Report(4)(1).md

[L77] mime_type: text/markdown

[L78] surface: conversation

[L79] score: 0.03128054740957967

[L80] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6

[L81] snippet:

[L82]

```
[L83] graph TD
[L84]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L85]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L86]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L87]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L88]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L89]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L90]     end
[L91]
[L92]     subgraph "Phase 2: Online – Retrieval & Generation"
[L93]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L94]         KF --> L6
[L95]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L96]         L7 --> A["Answer + Citations"]
[L97]     end
[L98]
[L99]     subgraph "Phase 3: Continuous – Operations"
[L100]        L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L101]        A --> L8
[L102]        L8 -->| "Self-Evolution<br/>Feedback" | KF
[L103]     end
[L104]
```

[L105]

[L106] ### 3.2 Data Flow

[L107]

[L108]

```
[L109] Raw Document (PDF/DOCX/HTML)
[L110]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L111] Normalized Text + Metadata
[L112]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L113] [Page → Region → Block] hierarchy
[L114]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L115] KnowledgeUnit objects (candidate concepts)
[L116]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L117] KnowledgeNode objects (canonical, deduplicated)
[L118]     ↓ Layer 4: Validate relations against ontology, score confidence
[L119] KnowledgeEdge objects (typed, scored)
[L120]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L121] Knowledge Forest (rooted trees + cross-links)
[L122]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L123] Selected context nodes (within token budget)
[L124]     ↓ Layer 7: Build prompt, invoke LLM
[L125]     ↓ Layer 7: Verify claims against evidence, bind citations
[L126] Answer with citations + confidence score
[L127]     ↓ Layer 8: Log, collect feedback, update forest weights
[L128]
```

[L129]

[L130] ### 3.3 Layer Summary

[L131]

[L132] | Layer | Name | Input | Output | Key Algorithms |

[L133] |-----|-----|-----|-----|-----|

[L134] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L135] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L136] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L137] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L138] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L139] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L140] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L141] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L142] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L143]

[L144] ---

[L145]

[L146] ## 4. Data Model — Layer 0

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `[[filecite]]turn7file2[[`

[L147] metadata:

[L148] query_result_index: 2

[L149] file_id: file_00000000dcc082119f935683a5e13eed

[L150] version_id: 1

[L151] name: SHIA_RAG_Final_Report(4)(1).md

[L152] mime_type: text/markdown

[L153] surface: conversation

[L154] score: 0.030309988518943745

[L155] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--54

[L156] snippet:

[L157] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L158] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L159] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L160] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L161] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L162]

[L163] The complete system specification covers:

[L164] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L165] - **12 defined functions** with complete implementations

[L166] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L167] - **8 schemas** covering all data objects from document blocks to query logs

[L168] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L169] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L170]

[L171] ### 13.2 Key Novelties

[L172]

[L173] | # | Novelty | Level | Why It's New |

[L174] |---|-----|-----|-----|

[L175] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L176] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L177] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L178] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L179] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically

designed for hierarchical RAG quality |

[L180]

[L181] ### 13.3 Next Steps

[L182]

[L183] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L184] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L185] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L186] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L187] 5. **Build Layer 4** (Relation management with ontology validation)

[L188] 6. **Build Layer 5** (SHIA Core with algorithms)

[L189] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L190] 8. **Build Layer 7** (Generation with verification)

[L191] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L192] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L193] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L194] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L195]

[L196] ### 13.4 Future Work

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite[turn7file3]`

[L197] metadata:

[L198] query_result_index: 3

[L199] file_id: file_00000000dcc082119f935683a5e13eed

[L200] version_id: 1

[L201] name: SHIA_RAG_Final_Report(4)(1).md

[L202] mime_type: text/markdown

[L203] surface: conversation

[L204] score: 0.028577260665441927

[L205] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--53

[L206] snippet:

[L207] Real knowledge often has multiple valid parents (e.g., "Machine Learning" → both "AI" and "Statistics") | Deliberate design choice | Limits expressiveness | Future work: DAG support (Section 13.4) |

[L208] | L2 | **Greedy Tree Knapsack** — uses greedy approximation instead of exact DP. Exact DP is $O(nB)$ where B = token budget, which is computationally prohibitive | Deliberate trade-off | Near-optimal, not optimal | Future work: Exact DP for small budgets |

[L209] | L3 | **No incremental updates** — adding new documents requires re-running the full offline pipeline (Layers 1-5) | Engineering gap | Limits production scalability | Future work: Incremental forest updates |

[L210] | L4 | **Abstraction-level heuristic** — the multi-factor heuristic for estimating abstraction level has not been empirically validated | Validation needed | May produce incorrect hierarchy | Planned: Empirical validation on domain datasets |

[L211] | L5 | **LLM extraction quality** — knowledge extraction relies on LLM accuracy. Extraction errors propagate through the entire pipeline | Inherent limitation | Garbage-in-garbage-out risk | Mitigated by confidence thresholds and human review |

[L212] | L6 | **Self-evolution bias** — positive feedback on wrong paths can reinforce errors. ϵ -greedy exploration mitigates but does not eliminate this risk | Research challenge | May degrade over time | Mitigated by ϵ -greedy; future: Thompson sampling |

[L213] | L7 | **Novelty claims require verification** — "first" claims are based on systematic literature review but require exhaustive verification before publication | Research maturity | Affects publication readiness | All claims qualified with "to the best of our knowledge" |

[L214]

[L215] ---
[L216]
[L217] ## 13. Conclusion & Future Work
[L218]
[L219] ### 13.1 Summary
[L220]
[L221] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:
[L222]
[L223] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem
[L224] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
[L225] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence
[L226] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG
[L227] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality
[L228]
[L229] The complete system specification covers:
[L230] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
[L231] - **12 defined functions** with complete implementations
[L232] - **12 relation types** across 4 hierarchical and 8 semantic categories
[L233] - **8 schemas** covering all data objects from document blocks to query logs
[L234] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline
[L235] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing
[L236]
[L237] ### 13.2 Key Novelties
[L238]
[L239] | # | Novelty | Level | Why It's New |
[L240] |---|-----|-----|-----|
[L241] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |
[L242] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge,

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite`turn7file4

[L243] metadata:
[L244] query_result_index: 4
[L245] file_id: file_00000000dcc082119f935683a5e13eed
[L246] version_id: 1
[L247] name: SHIA_RAG_Final_Report(4)(1).md
[L248] mime_type: text/markdown
[L249] surface: conversation
[L250] score: 0.028371628371628373
[L251] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--7
[L252] snippet:
[L253] deduplicated)
[L254] ↓ Layer 4: Validate relations against ontology, score confidence

[L255] KnowledgeEdge objects (typed, scored)
[L256] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L257] Knowledge Forest (rooted trees + cross-links)
[L258] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L259] Selected context nodes (within token budget)
[L260] ↓ Layer 7: Build prompt, invoke LLM
[L261] ↓ Layer 7: Verify claims against evidence, bind citations
[L262] Answer with citations + confidence score
[L263] ↓ Layer 8: Log, collect feedback, update forest weights
[L264]

```
[L265]
[L266] ### 3.3 Layer Summary
[L267]
[L268] | Layer | Name | Input | Output | Key Algorithms |
[L269] |-----|-----|-----|-----|-----|
[L270] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L271] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L272] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L273] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L274] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L275] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L276] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L277] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L278] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L279]
[L280] ---
[L281]
[L282] ## 4. Data Model – Layer 0
[L283]
[L284] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode`
[L285]
[L286] ### 4.1 KnowledgeNode Schema
[L287]
[L288] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L289]
[L290]
```

json

```
[L291] {
[L292] "node_id": "NODE_000789",
[L293] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L294] "canonical_name": "Round Robin",
[L295] "aliases": ["RR", "Round-Robin Scheduling"],
[L296] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L297] "summaries": ["CPU scheduling method using time slices"],
[L298] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L299] "domain": "Operating Systems",
[L300] "properties": {
[L301] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L302] "preemptive": {"type": "boolean", "value": true}
[L303] },
[L304] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L305] "parent": "NODE_000456",
[L306] "children": ["NODE_001001", "NODE_001002"],
[L307] "related": ["NODE_000890"],
[L308] "incoming_edges": ["EDGE_001"],
```



```
[L309] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L310] "evidence": [
[L311] {
[L312] "evidence_type": "DEFINITION",
[L313] "text": "Round Robin is a preemptive scheduling algorithm...",
[L314] "source_doc": "DOC_000001",
[L315] "source_block": "BLK_000234",
[L316] "page": 42
[L317] }
[L318] ],
```

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite`turn7file5

```
[L319] metadata:
[L320] query_result_index: 5
[L321] file_id: file_00000000dcc082119f935683a5e13eed
[L322] version_id: 1
[L323] name: SHIA_RAG_Final_Report(4)(1).md
[L324] mime_type: text/markdown
[L325] surface: conversation
[L326] score: 0.027119252873563218
[L327] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--19
[L328] snippet:
[L329] Uses pdfplumber for text-based tables, Camelot for image-based.
[L330] Returns list of rows, each row is a list of cell strings. ""
[L331] try:
[L332] # Primary: pdfplumber table extraction
[L333] table = page.extract_table(table_region)
[L334] if table and len(table) > 1: # At least header + 1 data row
[L335] return table
[L336] except Exception:
[L337] pass
[L338]
[L339] # Fallback: Camelot for image-based tables
[L340] try:
[L341] import camelot
[L342] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L343] if tables:
[L344] return tables[0].df.values.tolist()
[L345] except Exception:
[L346] pass
[L347]
[L348] return None # Table structure could not be parsed
[L349]
```

```
[L350]
[L351] ##### Table Detection Fallback
[L352]
[L353] If table detection confidence is low, the block is flagged:
[L354]
[L355]
```

```
python
[L356] def classify_block_with_fallback(text, page) -> tuple:
[L357] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L358] block_type, confidence = classify_block(text)
[L359]
[L360] if block_type == BlockType.TABLE and confidence < 0.6:
[L361] # Attempt cell-level parsing to confirm
[L362] cells = parse_table_cells(page, text)
[L363] if cells is None:
[L364] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L365] else:
[L366] return BlockType.TABLE, 0.8 # Confirmed by parser
[L367]
[L368] return block_type, confidence
[L369]
```

```
[L370]
[L371] ##### Reading Order Validation
[L372]
[L373] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L374]
[L375]
```

```
python
[L376] def assign_reading_order(blocks: list) -> list:
[L377] """Assign global reading order with confidence.
[L378]
[L379] Multi-column layouts can produce incorrect reading order.
[L380] We detect potential issues and flag low confidence."""
[L381] # Sort by page, then y-coordinate, then x-coordinate
[L382] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L383]
[L384] for i, block in enumerate(sorted_blocks):
[L385] block.reading_order = i
[L386]
[L387] # Detect potential multi-column issues
[L388] if i > 0:
[L389] prev = sorted_blocks[i - 1]
[L390] if (prev.page_number == block.page_number and
[L391] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L392] prev.x_pos > block.x_pos): # But previous is to the right
[L393] block.confidence *= 0.7 # Reduce confidence — possible column issue
[L394]
[L395] return sorted_blocks
[L396]
```

```

[L397]
[L398] ---
[L399]
[L400] ### 5.3 Layer 3: Knowledge Extraction
[L401]
[L402] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L403]
[L404] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L405]
[L406] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L407]
[L408] #### Extraction Prompt
[L409]
[L410]

```

[L411] You are a knowledge extraction system. Given a text block, extract:

[L412] 1. Concepts: Named entities, technical terms

[L413] 2. Definitions: If the text defines any concept, extract the definition

[L414] 3. Properties: Key attributes with types and values

[L415] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L416] 5. Domain: Subject area (e.g., "Operating Systems")

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite[turn7file6]`

[L417] metadata:

[L418] query_result_index: 6

[L419] file_id: file_00000000dcc082119f935683a5e13eed

[L420] version_id: 1

[L421] name: SHIA_RAG_Final_Report(4)(1).md

[L422] mime_type: text/markdown

[L423] surface: conversation

[L424] score: 0.025052505250525055

[L425] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--47

[L426] snippet:

[L427] py # LLM-based concept extraction

[L428] | | | alias_resolver.py # LSH-based deduplication

[L429] | | | fingerprint.py

[L430] | | | layer4_relations/

[L431] | | | relation_extractor.py

[L432] | | | validator.py

[L433] | | | confidence_scorer.py

[L434] | | | layer5_shia/

[L435] | | | cpq.py # Candidate Parent Generator

[L436] | | | pre.py # Parent Ranking Engine

[L437] | | | hv.py # Hierarchy Validator

[L438] | | | fi.py # Forest Integrator

[L439] | | | cld.py # Cross-Link Discovery

[L440] | | | forest.py # Knowledge Forest data structure

[L441] | | | layer6_retrieval/

[L442] | | | query_analyzer.py

[L443] | | | complexity_classifier.py

[L444] | | | traversal.py # Adaptive depth traversal

```
[L445] | | |— tree_knapsack.py # Precedence-constrained optimizer
[L446] | | |— context_assembler.py
[L447] | |— layer7_generation/
[L448] | | |— prompt_builder.py
[L449] | | |— llm_client.py
[L450] | | |— verifier.py
[L451] | |— layer8_ops/
[L452] | | |— logger.py
[L453] | | |— metrics.py
[L454] | |— feedback.py # Self-evolution engine
[L455] |— evaluation/
[L456] | |— shef.py # SHEF framework
[L457] | |— datasets/ # Test corpora
[L458] | |— baselines/ # Flat RAG, TreeRAG comparisons
[L459] |— api/
[L460] | |— main.py # FastAPI endpoints
[L461] |— tests/
[L462] | |— test_hv.py # Cycle detection tests
[L463] | |— test_pre.py
[L464] | |— test_tree_knapsack.py
[L465] | |— test_retrieval.py
[L466] |— docker-compose.yaml
[L467] |— requirements.txt
[L468] |— README.md
[L469]
```

```
[L470]
[L471] ### 10.3 Environment Configuration
[L472]
[L473]
```

text

```
[L474] # .env.example
[L475] OPENAI_API_KEY=sk-...
[L476] LLM_MODEL=gpt-4o-mini
[L477] NEO4J_URI=bolt://localhost:7687
[L478] NEO4J_USER=neo4j
[L479] NEO4J_PASSWORD=
[L480] POSTGRES_URI=postgresql://localhost:5432/shia_rag
[L481] REDIS_URL=redis://localhost:6379
[L482] EMBEDDING_MODEL=bge-base-en-v1.5
[L483] EMBEDDING_DIMENSION=768
[L484] LOG_LEVEL=INFO
[L485] TOKEN_BUDGET=4096
[L486] MAX_DEPTH=8
[L487]
```

```
[L488]
[L489] ### 10.4 Implementation Timeline (6 Months)
[L490]
[L491]
```

mermaid
[L492] gantt
[L493] title SHIA-RAG Implementation Roadmap
[L494] dateFormat YYYY-MM-DD
[L495]
[L496] section Phase 1: Foundation
[L497] Core Data Model + Schemas :a1, 2026-09-01, 2w
[L498] Input Processing (Layer 1) :a2, after a1, 1w
[L499] Document Parser (Layer 2) :a3, after a2, 2w
[L500]
[L501] section Phase 2: Knowledge Extraction
[L502] Knowledge Extractor (Layer 3) :b1, after a3, 3w
[L503] Alias Resolution + LSH :b2, after b1, 2w
[L504] Relation Management (Layer 4) :b3, after b2,

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite`turn7file

[L505] metadata:
[L506] query_result_index: 7
[L507] file_id: file_00000000dcc082119f935683a5e13eed
[L508] version_id: 1
[L509] name: SHIA_RAG_Final_Report(4)(1).md
[L510] mime_type: text/markdown
[L511] surface: conversation
[L512] score: 0.0250384024577573
[L513] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--48
[L514] snippet:
[L515] py # FastAPI endpoints
[L516] |— tests/
[L517] | |— test_hv.py # Cycle detection tests
[L518] | |— test_pre.py
[L519] | |— test_tree_knapsack.py
[L520] | |— test_retrieval.py
[L521] |— docker-compose.yaml
[L522] |— requirements.txt
[L523] |— README.md
[L524]

```
[L525]  
[L526] ### 10.3 Environment Configuration  
[L527]  
[L528]
```

text
[L529] # .env.example
[L530] OPENAI_API_KEY=sk-...
[L531] LLM_MODEL=gpt-4o-mini
[L532] NEO4J_URI=bolt://localhost:7687
[L533] NEO4J_USER=neo4j
[L534] NEO4J_PASSWORD=
[L535] POSTGRES_URI=postgresql://localhost:5432/shia_rag

[L536] REDIS_URL=redis://localhost:6379
[L537] EMBEDDING_MODEL=bge-base-en-v1.5
[L538] EMBEDDING_DIMENSION=768
[L539] LOG_LEVEL=INFO
[L540] TOKEN_BUDGET=4096
[L541] MAX_DEPTH=8
[L542]

[L543]
[L544] ### 10.4 Implementation Timeline (6 Months)
[L545]
[L546]

mermaid

[L547] gantt
[L548] title SHIA-RAG Implementation Roadmap
[L549] dateFormat YYYY-MM-DD
[L550]
[L551] section Phase 1: Foundation
[L552] Core Data Model + Schemas :a1, 2026-09-01, 2w
[L553] Input Processing (Layer 1) :a2, after a1, 1w
[L554] Document Parser (Layer 2) :a3, after a2, 2w
[L555]
[L556] section Phase 2: Knowledge Extraction
[L557] Knowledge Extractor (Layer 3) :b1, after a3, 3w
[L558] Alias Resolution + LSH :b2, after b1, 2w
[L559] Relation Management (Layer 4) :b3, after b2, 2w
[L560]
[L561] section Phase 3: SHIA Core
[L562] CPG++ Implementation :c1, after b3, 2w
[L563] PRE + HV (with feedback loop) :c2, after c1, 2w
[L564] FI + CLD :c3, after c2, 1w
[L565] Forest Construction Pipeline :c4, after c3, 1w
[L566]
[L567] section Phase 4: Retrieval & Generation
[L568] Query Analyzer + Classifier :d1, after c4, 2w
[L569] Forest Traversal + Tree Knapsack :d2, after d1, 2w
[L570] LLM Generation + Verification :d3, after d2, 2w
[L571]
[L572] section Phase 5: Evaluation & Polish
[L573] SHEF Metrics Implementation :e1, after d3, 2w
[L574] Self-Evolution Feedback Loop :e2, after e1, 1w
[L575] Benchmarking + Paper Writing :e3, after e2, 3w
[L576]

[L577]
[L578] ### 10.5 CI/CD Pipeline
[L579]
[L580]

yaml

[L581] # .github/workflows/ci.yml
[L582] name: CI

```

[L583] on: [push, pull_request]
[L584] jobs:
[L585] test:
[L586] runs-on: ubuntu-latest
[L587] steps:
[L588] - uses: actions/checkout@v4
[L589] - uses: actions/setup-python@v5
[L590] with:
[L591] python-version: '3.11'
[L592] - run: pip install -r requirements.txt
[L593] - run: pytest tests/ -v --tb=short
[L594] - run: mypy src/ --ignore-missing-imports
[L595] ```
[L596]
[L597] ---
[L598]
[L599] ## 11. Evaluation Framework
[L600]
[L601] ### 11.1 Datasets

```

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: `filecite`turn7file8

```

[L602] metadata:
[L603] query_result_index: 8
[L604] file_id: file_00000000dcc082119f935683a5e13eed
[L605] version_id: 1
[L606] name: SHIA_RAG_Final_Report(4)(1).md
[L607] mime_type: text/markdown
[L608] surface: conversation
[L609] score: 0.0248015873015873
[L610] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--20
[L611] snippet:
[L612] Multi-column layouts can produce incorrect reading order.
[L613] We detect potential issues and flag low confidence.
[L614] # Sort by page, then y-coordinate, then x-coordinate
[L615] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L616]
[L617] for i, block in enumerate(sorted_blocks):
[L618]     block.reading_order = i
[L619]
[L620] # Detect potential multi-column issues
[L621] if i > 0:
[L622]     prev = sorted_blocks[i - 1]
[L623]     if (prev.page_number == block.page_number and
[L624]         abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L625]         prev.x_pos > block.x_pos): # But previous is to the right
[L626]         block.confidence *= 0.7 # Reduce confidence — possible column issue

```

[L627]

[L628] return sorted_blocks

[L629]

```
[L630]
[L631] ---
[L632]
[L633] ### 5.3 Layer 3: Knowledge Extraction
[L634]
[L635] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L636]
[L637] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L638]
[L639] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L640]
[L641] #### Extraction Prompt
[L642]
[L643]
```

[L644] You are a knowledge extraction system. Given a text block, extract:

[L645] 1. Concepts: Named entities, technical terms

[L646] 2. Definitions: If the text defines any concept, extract the definition

[L647] 3. Properties: Key attributes with types and values

[L648] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L649] 5. Domain: Subject area (e.g., "Operating Systems")

[L650]

[L651] Respond in JSON format:

[L652] {

[L653] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."]},

[L654] "properties": {"key": {"type": "...", "value": "..."}},

[L655] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",

[L656] "evidence": "..."}],

[L657] "domain": "..."

[L658] }

[L659]

```
[L660]
[L661] #### Definition Extraction (Dependency-Parse Based)
[L662]
[L663] Simple regex patterns like `^(.*) is (a|an|the)? (.*)` are too broad – they match non-definitions like "The s
[L664]
[L665]
```

python

[L666] def is_definition(text: str) -> tuple:

[L667] """Detect definitions using dependency parse + pattern matching.

[L668]

[L669] Returns (is_def: bool, concept: str, definition: str).

[L670]

[L671] Strategy:

[L672] 1. Check for explicit definitional phrases (high precision)

[L673] 2. Use dependency parse to find copular definitions (medium precision)

[L674] 3. Fallback to LLM classification (expensive, high recall)

[L675] """

[L676] import re

[L677]
[L678] # Strategy 1: Explicit definitional phrases (high precision)
[L679] explicit_patterns = [
[L680] r'^(.+?)\s+is defined as\s+(.+)',
[L681] r'^(.+?)\s+refers to\s+(.+)',
[L682] r'^(.+?):\s+A-Z', # "Concept: A description..."
[L683] r'^(.+?

DEEP RESEARCH 23/08/2026, 22:57:47

Citation Marker: □filecite□turn7file9□

[L684] metadata:
[L685] query_result_index: 9
[L686] file_id: file_00000000dcc082119f935683a5e13eed
[L687] version_id: 1
[L688] name: SHIA_RAG_Final_Report(4)(1).md
[L689] mime_type: text/markdown
[L690] surface: conversation
[L691] score: 0.023667065549793913
[L692] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--55
[L693] snippet:
[L694] first RAG system to formulate context selection as a precedence-constrained optimization |
[L695] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |
[L696] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |
[L697] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |
[L698] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |
[L699]
[L700] ### 13.3 Next Steps
[L701]
[L702] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)
[L703] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)
[L704] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)
[L705] 4. **Build Layer 3** (Knowledge extraction with LLM)
[L706] 5. **Build Layer 4** (Relation management with ontology validation)
[L707] 6. **Build Layer 5** (SHIA Core with algorithms)
[L708] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
[L709] 8. **Build Layer 7** (Generation with verification)
[L710] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
[L711] 10. **Run first evaluation** on OS textbook dataset using SHEF
[L712] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
[L713] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
[L714]
[L715] ### 13.4 Future Work
[L716]
[L717] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)
[L718] - **Multi-Tenancy**: Implement access control and document-level permissions
[L719] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets

[L720] - **Cross-Lingual Support:** Extend to multilingual documents and concept hierarchies

[L721] - **Incremental Updates:** Support adding new documents without full forest reconstruction

[L722] - **Advanced Verification:** Replace semantic similarity with NLI (Natural Language Inference) model for claim verification

[L723] - **Exploration in Self-Evolution:** Upgrade from ϵ -greedy ($\epsilon=0.1$, currently implemented) to Thompson sampling for more principled exploration-exploitation trade-off

[L724]

[L725] ---

[L726]

[L727] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0-8) are fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

[L728]

[L729] ---

[L730]

[L731] **## Appendix: PDF to Markdown Conversion**

[L732]

[L733] To convert this report to PDF, use Pandoc:

[L734]

[L735]

```
[L736] # Install pandoc (https://pandoc.org/installing.html)
[L737] # Basic conversion
[L738] pandoc SHIA_RAG_Final_Report.md -o SHIA_RAG_Final_Report.pdf --pdf-engine=xelatex
[L739]
[L740] # With better formatting (recommended)
[L741] pandoc SHIA_RAG_Final_Report.md \
[L742]   -o SHIA_RAG_Final_Report.pdf \
[L743]   --pdf-engine=xelatex \
[L744]   --variable geometry:margin=1in \
[L745]   --variable fontsize=11pt \
[L746]   --variable mainfont="Times New Roman" \
[L747]   --toc \
[L748]   --toc-depth=3 \
[L749]   --highlight-style=tango \
[L750]   --number-sections
[L751]
[L752] # Alternative: use weasyprint for HTML-based PDF (better Mermaid support)
[L753] pandoc SHIA_RAG_Final_Report.md -o report.html --standalone --css=style.css
[L754] # Then use a browser to print to PDF
[L755]
```

DEEP RESEARCH 23/08/2026, 22:57:50

Citation Marker: `filecite turn8file0`

[L1] metadata:

[L2] ref_id: turn7file8

[L3] file_id: file_00000000dcc082119f935683a5e13eed

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4)(1).md

[L6] mode: chunk_context

[L7] lines_shown: 1-120

[L8] total_file_lines: 266

[L9] has_more: True

[L10] next_start_line: 121

[L11] next_read:

`{"ref_id":"turn7file8","mode":"chunk_context","start_line":121,"context_before_lines":5,"context_after_lines":260}`

[L12] content:

[L13]

[L14] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge synonyms to create canonical `KnowledgeNode` objects.

[L15]

[L16] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)

[L17]

[L18] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology and assigns confidence scores.

[L19]

[L20] ##### Extraction Prompt

[L21]

[L22]

```
[L23] You are a knowledge extraction system. Given a text block, extract:
[L24] 1. Concepts: Named entities, technical terms
[L25] 2. Definitions: If the text defines any concept, extract the definition
[L26] 3. Properties: Key attributes with types and values
[L27] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L28] 5. Domain: Subject area (e.g., "Operating Systems")
[L29]
[L30] Respond in JSON format:
[L31] {
[L32]   "concepts": [{ "name": "...", "aliases": [...], "definitions": ["..."],
[L33]                  "properties": { "key": { "type": "...", "value": "..." } } },
[L34]   "relations": [{ "source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L35]                  "evidence": "..." } ],
[L36]   "domain": "..."
[L37] }
[L38]
```

[L39]

[L40] ##### Definition Extraction (Dependency-Parse Based)

[L41]

[L42] Simple regex patterns like `^(.) is (a|an|the)? (.)` are too broad — they match non-definitions like "The sky is blue today." Instead, we use **dependency parsing** to detect definitional structures:

[L43]

[L44]

```

[L45] def is_definition(text: str) -> tuple:
[L46]     """Detect definitions using dependency parse + pattern matching.
[L47]
[L48]     Returns (is_def: bool, concept: str, definition: str).
[L49]
[L50]     Strategy:
[L51]     1. Check for explicit definitional phrases (high precision)
[L52]     2. Use dependency parse to find copular definitions (medium precision)
[L53]     3. Fallback to LLM classification (expensive, high recall)
[L54]     """
[L55]     import re
[L56]
[L57]     # Strategy 1: Explicit definitional phrases (high precision)
[L58]     explicit_patterns = [
[L59]         r'^(.+?)\s+is defined as\s+(.+)',
[L60]         r'^(.+?)\s+refers to\s+(.+)',
[L61]         r'^(.+?):\s+[A-Z](.+)', # "Concept: A description..."
[L62]         r'^(.+?)\s+is a\s+(type|kind|form|category|class)\s+of\s+(.+)',
[L63]     ]
[L64]     for pattern in explicit_patterns:
[L65]         match = re.match(pattern, text, re.IGNORECASE)
[L66]         if match:
[L67]             return True, match.group(1).strip(), match.group(2).strip()
[L68]
[L69]     # Strategy 2: Copular structure via dependency parse
[L70]     # (requires spaCy or similar – use LLM as fallback if not available)
[L71]
[L72]     # Strategy 3: LLM fallback (only for DEFINITION-classified blocks)
[L73]     return False, None, None
[L74]

```

[L75]

[L76] ##### Alias Resolution (with Antonym Protection)

[L77]

[L78] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them — but must **avoid merging antonyms** (e.g., "preemptive" and "non-preemptive" have high similarity but are distinct concepts).

[L79]

[L80] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L81] 2. **Antonym check:** Before merging, verify candidates do not have a CONTRASTS_WITH relation

[L82] 3. Find connected components of near-duplicate units

[L83] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L84]

[L85]

```

[L86] def alias_resolution(units: list, relation_cache: dict) -> list:
[L87]     """Merge KnowledgeUnits referring to the same concept.
[L88]
[L89]     Merge strategy:
[L90]     - Canonical name: most frequent name across units
[L91]     - Aliases: union of all names and aliases
[L92]     - Definitions: union of all definitions
[L93]     - Confidence: average of unit extraction confidences
[L94]     - Properties: merged (latest value wins, conflicts logged)
[L95]
[L96]     Antonym protection:
[L97]     - Before merging, check if candidates have CONTRASTS_WITH relation
[L98]     - Also check for negation prefixes (non-, un-, in-, dis-)
[L99]     """
[L100]     import re
[L101]     NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L102]
[L103]     lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L104]     # ... build LSH index from concept names + aliases ...
[L105]
[L106]     raw_groups = find_connected_components(units, lsh)
[L107]
[L108]     # Antonym protection: split groups containing antonym pairs
[L109]     safe_groups = []
[L110]     for group in raw_groups:
[L111]         names = [u.name.lower() for u in group]
[L112]         has_antonym = False
[L113]         for i, n1 in enumerate(names):
[L114]             for n2 in names[i+1:]:
[L115]                 # Check 1: Negation prefix
[L116]                 if NEGATION_PREFIXES.sub('', n1) == NEGATION_PREFIXES.sub('', n2):
[L117]                     has_antonym = True
[L118]                 # Check 2: Known CONTRASTS_WITH relation
[L119]                 if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L120]                     has_antonym = True
[L121]
[L122]         if has_antonym:
[L123]             safe_groups.extend([u for u in group]) # Don't merge
[L124]         else:
[L125]             safe_groups.append(group)
[L126]
[L127]     return [create_canonical_node(group) for group in safe_groups]
[L128]

```

[L129]

[L130] ##### Embedding Strategy on Merge

[L131]

[L132] When nodes are merged during alias resolution, embeddings are handled as follows:

DEEP RESEARCH 23/08/2026, 22:57:53

Citation Marker: turn9file0

[L1] metadata:

[L2] ref_id: turn7file8

[L3] file_id: file_00000000dcc082119f935683a5e13eed

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4)(1).md

[L6] mode: chunk_context

[L7] lines_shown: 121-240

[L8] total_file_lines: 266

[L9] has_more: True

[L10] next_start_line: 241

[L11] next_read:

{ "ref_id": "turn7file8", "mode": "chunk_context", "start_line": 241, "context_before_lines": 5, "context_after_lines": 260 }

[L12] content:

[L13] - **Do NOT average embeddings** — averaging collapses distinct semantic information

[L14] - **Re-embed the merged node** using the canonical name + merged definitions

[L15] - This ensures the embedding accurately represents the merged concept

[L16]

[L17]

```
[L18] def merge_node_embeddings(merged_node, embed_fn):
[L19]     """Re-compute embedding for merged node instead of averaging.
[L20]
[L21]     Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L22]     produce a vector that represents neither concept well.
[L23]     Instead, embed the canonical representation."""
[L24]     text_to_embed = merged_node.canonical_name
[L25]     if merged_node.definitions:
[L26]         text_to_embed += " " + merged_node.definitions[0]
[L27]     merged_node.embedding = embed_fn(text_to_embed)
[L28]
```

[L29]

[L30] ---

[L31]

[L32] ### 5.4 Layer 4: Relation Management

[L33]

[L34] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve cross-document conflicts, and produce final `KnowledgeEdge` objects.

[L35]

[L36] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

[L37]

[L38] **Pipeline:**

[L39] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L40] 2. **Resolve target node** — map target concept name to a `KnowledgeNode`

[L41] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L42] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L43] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L44] 6. **Resolve cross-document conflicts** — when multiple documents disagree on a relation

[L45]

[L46]

```
[L47] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L48]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L49]     conf = 0.5
[L50]     conf += 0.15 * source.confidence
[L51]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L52]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L53]     return min(conf, 1.0)
[L54]
```

[L55]

[L56] ##### Cross-Document Conflict Resolution

[L57]

[L58] When multiple documents provide **conflicting relations** for the same concept pair (e.g., Document A says "Round Robin IS_A Scheduling" but Document B says "Round Robin PART_OF OS"), the system resolves conflicts using evidence-weighted voting:

[L59]

[L60]

```

[L61] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:
[L62]     """Resolve conflicting relations from different documents.
[L63]
[L64]     Strategy: Evidence-weighted voting
[L65]     1. Group edges by (source_node, target_node) pair
[L66]     2. For each group, tally votes per relation_type weighted by evidence confidence
[L67]     3. Select the relation_type with highest weighted vote
[L68]     4. Log conflicts for manual review
[L69]     """
[L70]     from collections import defaultdict
[L71]
[L72]     # Group by node pair
[L73]     pair_groups = defaultdict(list)
[L74]     for edge in candidate_edges:
[L75]         key = (edge.source_node, edge.target_node)
[L76]         pair_groups[key].append(edge)
[L77]
[L78]     resolved = []
[L79]     for (src, tgt), edges in pair_groups.items():
[L80]         if len(edges) == 1:
[L81]             resolved.append(edges[0])
[L82]             continue
[L83]
[L84]         # Tally weighted votes per relation type
[L85]         votes = defaultdict(float)
[L86]         evidence_pool = defaultdict(list)
[L87]         for edge in edges:
[L88]             weight = edge.confidence * len(edge.evidence)
[L89]             votes[edge.relation_type] += weight
[L90]             evidence_pool[edge.relation_type].extend(edge.evidence)
[L91]
[L92]         # Select winning relation type
[L93]         winner = max(votes, key=votes.get)
[L94]
[L95]         # Log conflict if close contest
[L96]         sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L97]         if len(sorted_votes) > 1:
[L98]             margin = sorted_votes[0][1] - sorted_votes[1][1]
[L99]             if margin < 0.2: # Close contest
[L100]                 log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L101]
[L102]             winning_edge = edges[0]
[L103]             winning_edge.relation_type = winner
[L104]             winning_edge.evidence = evidence_pool[winner]
[L105]             winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L106]             winning_edge.metadata['conflict_resolved'] = True
[L107]             winning_edge.metadata['alternative_relations'] = [
[L108]                 (rt, score) for rt, score in sorted_votes if rt != winner
[L109]             ]
[L110]             resolved.append(winning_edge)
[L111]
[L112]     return resolved
[L113]

```

[L114]

[L115] ---

[L116]

[L117] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L118]

[L119] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L120]

[L121] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L122]

[L123] ##### Pipeline Overview
[L124]
[L125] ```mermaid
[L126] graph LR
[L127] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L128] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L129] PRE --> HV["HV
Validate #1?"]
[L130] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L131] HV -->|"Fail"| HV2["HV
Validate #2?"]
[L132] HV2 -->|"Pass"| FI

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: turn10file0

[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.03252247488101534
[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6
[L10] snippet:
[L11]

```
[L12] graph TD
[L13]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L14]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L15]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L16]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L17]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L18]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L19]     end
[L20]
[L21]     subgraph "Phase 2: Online – Retrieval & Generation"
[L22]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L23]         KF --> L6
[L24]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L25]         L7 --> A["Answer + Citations"]
[L26]     end
[L27]
[L28]     subgraph "Phase 3: Continuous – Operations"
[L29]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L30]         A --> L8
[L31]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L32]     end
[L33]
```

[L34]
[L35] ### 3.2 Data Flow
[L36]
[L37]


```
[L38] Raw Document (PDF/DOCX/HTML)
[L39]     ↳ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41]     ↳ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43]     ↳ Layer 3: Split sentences, extract concepts/definitions/relations
[L44] KnowledgeUnit objects (candidate concepts)
[L45]     ↳ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47]     ↳ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49]     ↳ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51]     ↳ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53]     ↳ Layer 7: Build prompt, invoke LLM
[L54]     ↳ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56]     ↳ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]

[L59] ### 3.3 Layer Summary

[L60]

[L61] | Layer | Name | Input | Output | Key Algorithms |

[L62] |-----|-----|-----|-----|-----|

[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L72]

[L73] ---

[L74]

[L75] ## 4. Data Model — Layer 0

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: `filecite`turn10file1

[L76] metadata:

[L77] query_result_index: 1

[L78] file_id: file_00000000dcc082119f935683a5e13eed

[L79] version_id: 1

[L80] name: SHIA_RAG_Final_Report(4)(1).md

[L81] mime_type: text/markdown

[L82] surface: conversation

[L83] score: 0.03252247488101534

[L84] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5

[L85] snippet:

[L86] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L87]

[L88] ### 2.3 Literature Search Methodology

[L89]

[L90] | Parameter | Details |

[L91] |-----|-----|

[L92] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L93] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L94] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L95] | **Time Range** | 2020–2026 |

[L96] | **Key Papers Reviewed** | RAPTOR (Sarothi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L97]

[L98] ---

[L99]

[L100] ## 3. System Architecture

[L101]

[L102] ### 3.1 Overview

[L103]

[L104] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L105]

[L106] | Phase | Layers | Mode | Purpose |

[L107] |-----|-----|-----|-----|

[L108] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

|

[L109] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L110] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L111]

[L112] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L113]

[L114]

```
[L115] graph TD
[L116]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L117]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Tokenization, chunking"]
[L118]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L119]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L120]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L121]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L122]     end
[L123]
[L124]     subgraph "Phase 2: Online – Retrieval & Generation"
[L125]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RBO"]
[L126]         KF --> L6
[L127]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L128]         L7 --> A["Answer + Citations"]
[L129]     end
[L130]
[L131]     subgraph "Phase 3: Continuous – Operations"
[L132]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L133]         A --> L8
[L134]         L8 --> SE["Self-Evolution<br/>Feedback"] | KF
[L135]     end
[L136]
```

[L137]

[L138] ### 3.2 Data Flow

[L139]

[L140] ```

[L141] Raw Document (PDF/DOCX/HTML)

[L142] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L143] Normalized Text + Metadata

[L144] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L145] [Page → Region → Block] hierarchy

[L146] ↓ Layer 3: Split sentences,

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: `[]filecite[]turn10file2[]`

[L147] metadata:

[L148] query_result_index: 2

[L149] file_id: file_00000000dcc082119f935683a5e13eed

[L150] version_id: 1

[L151] name: SHIA_RAG_Final_Report(4)(1).md

[L152] mime_type: text/markdown

[L153] surface: conversation

[L154] score: 0.03177805800756621

[L155] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--35

[L156] snippet:

[L157] ##### Prompt Template

[L158]

[L159]

```
[L160] SYSTEM_PROMPT = ""You are a knowledge-grounded question answering system.
[L161] RULES:
[L162] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L163] 2. For each claim, cite the source using [NODE_ID] format.
[L164] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L165] 4. If evidence conflicts, present both views with citations.
[L166]
[L167] REQUIRED OUTPUT FORMAT:
[L168] {
[L169]   "answer": "Your answer with [NODE_ID] citations inline.",
[L170]   "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L171]   "overall_confidence": 0.0-1.0,
[L172]   "insufficient_evidence": false
[L173] }""
[L174]
```

[L175]

[L176] ##### Claim Verification

[L177]

[L178] After the LLM generates an answer, each claim is verified against its cited evidence:

[L179]

[L180]

```

[L181] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L182]     """Returns (status, confidence).
[L183]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L184]     - PARTIAL:      evidence partially supports (0.4 < confidence ≤ 0.7)
[L185]     - UNSUPPORTED:  no evidence supports (confidence ≤ 0.4)
[L186]     - CONFLICTING: evidence items contradict each other
[L187]     """
[L188]     scores = []
[L189]     for eid in evidence_ids:
[L190]         node = node_map.get(eid)
[L191]         if node and node.evidence:
[L192]             evidence_text = ' '.join(e.text for e in node.evidence)
[L193]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L194]
[L195]     if not scores:
[L196]         return 'UNSUPPORTED', 0.0
[L197]
[L198]     avg_score = sum(scores) / len(scores)
[L199]
[L200]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L201]         return 'CONFLICTING', avg_score
[L202]     if avg_score > 0.7:
[L203]         return 'SUPPORTED', avg_score
[L204]     elif avg_score > 0.4:
[L205]         return 'PARTIAL', avg_score
[L206]     else:
[L207]         return 'UNSUPPORTED', avg_score
[L208]

```

[L209]

[L210] ##### Unsupported Claim Policy

[L211]

[L212] | Status | Action |

[L213] |-----|-----|

[L214] | SUPPORTED | Keep as-is with citation |

[L215] | PARTIAL | Keep but mark as partially supported |

[L216] | UNSUPPORTED | Remove from answer, note at end |

[L217] | CONFLICTING | Present both views with note |

[L218]

[L219] ##### Answer Confidence Formula

[L220]

[L221]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{selected nodes}}{\text{cited nodes}}$$

[L222]

[L223] ##### Output Schema

[L224]

[L225]

```

[L226] @dataclass
[L227] class GenerationResult:
[L228]     """Output of Layer 7 – ready for user and Layer 8."""
[L229]     answer: str                # Answer text with citations
[L230]     claims: list              # List of VerifiedClaim
[L231]     citations: list           # Formatted citation list
[L232]     overall_confidence: float # 0.0-1.0
[L233]     retrieval_result: object  # Preserved for Layer 8
[L234]     generation_metadata: dict  # Model used, claim counts, etc.
[L235]

```

[L236]
[L237] ---
[L238]
[L239] ## 7. Operations Layer — Layer 8
[L240]
[L241] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.
[L242]
[L243] ### 7.1 Logging
[L244]
[L245] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: `filecite`turn10file3

[L246] metadata:
[L247] query_result_index: 3
[L248] file_id: file_00000000dcc082119f935683a5e13eed
[L249] version_id: 1
[L250] name: SHIA_RAG_Final_Report(4)(1).md
[L251] mime_type: text/markdown
[L252] surface: conversation
[L253] score: 0.030536130536130537
[L254] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--48
[L255] snippet:
[L256] py # FastAPI endpoints
[L257] |— tests/
[L258] | |— test_hv.py # Cycle detection tests
[L259] | |— test_pre.py
[L260] | |— test_tree_knapsack.py
[L261] | |— test_retrieval.py
[L262] |— docker-compose.yaml
[L263] |— requirements.txt
[L264] |— README.md
[L265]

```
[L266]  
[L267] ### 10.3 Environment Configuration  
[L268]  
[L269]
```

text
[L270] # .env.example
[L271] OPENAI_API_KEY=sk-...
[L272] LLM_MODEL=gpt-4o-mini
[L273] NEO4J_URI=bolt://localhost:7687
[L274] NEO4J_USER=neo4j
[L275] NEO4J_PASSWORD=
[L276] POSTGRES_URI=postgresql://localhost:5432/shia_rag
[L277] REDIS_URL=redis://localhost:6379
[L278] EMBEDDING_MODEL=bge-base-en-v1.5

[L279] EMBEDDING_DIMENSION=768

[L280] LOG_LEVEL=INFO

[L281] TOKEN_BUDGET=4096

[L282] MAX_DEPTH=8

[L283]

[L284]

[L285] ### 10.4 Implementation Timeline (6 Months)

[L286]

[L287]

mermaid

[L288] gantt

[L289] title SHIA-RAG Implementation Roadmap

[L290] dateFormat YYYY-MM-DD

[L291]

[L292] section Phase 1: Foundation

[L293] Core Data Model + Schemas :a1, 2026-09-01, 2w

[L294] Input Processing (Layer 1) :a2, after a1, 1w

[L295] Document Parser (Layer 2) :a3, after a2, 2w

[L296]

[L297] section Phase 2: Knowledge Extraction

[L298] Knowledge Extractor (Layer 3) :b1, after a3, 3w

[L299] Alias Resolution + LSH :b2, after b1, 2w

[L300] Relation Management (Layer 4) :b3, after b2, 2w

[L301]

[L302] section Phase 3: SHIA Core

[L303] CPG++ Implementation :c1, after b3, 2w

[L304] PRE + HV (with feedback loop) :c2, after c1, 2w

[L305] FI + CLD :c3, after c2, 1w

[L306] Forest Construction Pipeline :c4, after c3, 1w

[L307]

[L308] section Phase 4: Retrieval & Generation

[L309] Query Analyzer + Classifier :d1, after c4, 2w

[L310] Forest Traversal + Tree Knapsack :d2, after d1, 2w

[L311] LLM Generation + Verification :d3, after d2, 2w

[L312]

[L313] section Phase 5: Evaluation & Polish

[L314] SHEF Metrics Implementation :e1, after d3, 2w

[L315] Self-Evolution Feedback Loop :e2, after e1, 1w

[L316] Benchmarking + Paper Writing :e3, after e2, 3w

[L317]

[L318]

[L319] ### 10.5 CI/CD Pipeline

[L320]

[L321]

yaml

[L322] # .github/workflows/ci.yml

[L323] name: CI

[L324] on: [push, pull_request]

[L325] jobs:

```

[L326] test:
[L327] runs-on: ubuntu-latest
[L328] steps:
[L329] - uses: actions/checkout@v4
[L330] - uses: actions/setup-python@v5
[L331] with:
[L332] python-version: '3.11'
[L333] - run: pip install -r requirements.txt
[L334] - run: pytest tests/ -v --tb=short
[L335] - run: mypy src/ --ignore-missing-imports
[L336] ```
[L337]
[L338] ---
[L339]
[L340] ## 11. Evaluation Framework
[L341]
[L342] ### 11.1 Datasets

```

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: `filecite[turn10file4]`

```

[L343] metadata:
[L344] query_result_index: 4
[L345] file_id: file_00000000dcc082119f935683a5e13eed
[L346] version_id: 1
[L347] name: SHIA_RAG_Final_Report(4)(1).md
[L348] mime_type: text/markdown
[L349] surface: conversation
[L350] score: 0.030330882352941176
[L351] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--22
[L352] snippet:
[L353] ##### Alias Resolution (with Antonym Protection)
[L354]
[L355] After extraction, multiple KnowledgeUnit objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-
Robin Scheduling"). Alias resolution merges them — but must avoid merging antonyms (e.g., "preemptive" and "non-
preemptive" have high similarity but are distinct concepts).
[L356]
[L357] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases
[L358] 2. Antonym check: Before merging, verify candidates do not have a CONTRASTS_WITH relation
[L359] 3. Find connected components of near-duplicate units
[L360] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence
[L361]
[L362]

```

```

[L363] def alias_resolution(units: list, relation_cache: dict) -> list:
[L364]     """Merge KnowledgeUnits referring to the same concept.
[L365]
[L366]     Merge strategy:
[L367]     - Canonical name: most frequent name across units
[L368]     - Aliases: union of all names and aliases
[L369]     - Definitions: union of all definitions
[L370]     - Confidence: average of unit extraction confidences
[L371]     - Properties: merged (latest value wins, conflicts logged)
[L372]
[L373]     Antonym protection:
[L374]     - Before merging, check if candidates have CONTRASTS_WITH relation
[L375]     - Also check for negation prefixes (non-, un-, in-, dis-)
[L376]     """
[L377]     import re
[L378]     NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L379]
[L380]     lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L381]     # ... build LSH index from concept names + aliases ...
[L382]
[L383]     raw_groups = find_connected_components(units, lsh)
[L384]
[L385]     # Antonym protection: split groups containing antonym pairs
[L386]     safe_groups = []
[L387]     for group in raw_groups:
[L388]         names = [u.name.lower() for u in group]
[L389]         has_antonym = False
[L390]         for i, n1 in enumerate(names):
[L391]             for n2 in names[i+1:]:
[L392]                 # Check 1: Negation prefix
[L393]                 if NEGATION_PREFIXES.sub('', n1) == NEGATION_PREFIXES.sub('', n2):
[L394]                     has_antonym = True
[L395]                 # Check 2: Known CONTRASTS_WITH relation
[L396]                 if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L397]                     has_antonym = True
[L398]
[L399]         if has_antonym:
[L400]             safe_groups.extend([u for u in group]) # Don't merge
[L401]         else:
[L402]             safe_groups.append(group)
[L403]
[L404]     return [create_canonical_node(group) for group in safe_groups]
[L405]

```

[L406]

[L407] ##### Embedding Strategy on Merge

[L408]

[L409] When nodes are merged during alias resolution, embeddings are handled as follows:

[L410] - **Do NOT average embeddings** — averaging collapses distinct semantic information

[L411] - **Re-embed the merged node** using the canonical name + merged definitions

[L412] - This ensures the embedding accurately represents the merged concept

[L413]

[L414]

```

[L415] def merge_node_embeddings(merged_node, embed_fn):
[L416]     """Re-compute embedding for merged node instead of averaging.
[L417]
[L418]     Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L419]     produce a vector that represents neither concept well.
[L420]     Instead, embed the canonical representation."""
[L421]     text_to_embed = merged_node.canonical_name
[L422]     if merged_node.definitions:
[L423]         text_to_embed += " " + merged_node.definitions[0]
[L424]     merged_node.embedding = embed_fn(text_to_embed)
[L425]

```


[L426]

[L427] ---

[L428]

[L429] ### 5.4 Layer 4: Relation Management

[L430]

[L431] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve cross-document conflicts, and produce final `KnowledgeEdge` objects.

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: `filecite`turn10file5

[L432] metadata:

[L433] query_result_index: 5

[L434] file_id: file_00000000dcc082119f935683a5e13eed

[L435] version_id: 1

[L436] name: SHIA_RAG_Final_Report(4)(1).md

[L437] mime_type: text/markdown

[L438] surface: conversation

[L439] score: 0.030330882352941176

[L440] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--17

[L441] snippet:

[L442] 1)

[L443] if success_rate < 0.8:

[L444] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")

[L445] elif fmt == 'docx':

[L446] text, pages = extract_docx(file_path), []

[L447] is_ocr, ocr_conf = False, 1.0

[L448] elif fmt == 'html':

[L449] text, pages = extract_html(file_path), []

[L450] is_ocr, ocr_conf = False, 1.0

[L451] else: # txt

[L452] text = open(file_path, 'r', encoding='utf-8').read()

[L453] pages, is_ocr, ocr_conf = [], False, 1.0

[L454]

[L455] language = detect_language(text)

[L456] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal

[L457] metadata = extract_metadata(file_path, fmt)

[L458]

[L459] doc = NormalizedDocument(

[L460] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),

[L461] format=DocumentFormat(fmt), text=text, pages=pages,

[L462] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,

[L463] char_count=len(text), page_count=len(pages) or 1,

[L464] content_hash=content_hash, language=language,

[L465] partial_extraction=bool(failed_pages), failed_pages=failed_pages

[L466])

[L467] doc_registry[content_hash] = doc.doc_id # Register for dedup

[L468] return doc

[L469]

```
[L470]
[L471] ---
[L472]
[L473] ### 5.2 Layer 2: Document Parsing
[L474]
[L475] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with block cl
[L476]
[L477] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L478]
[L479]
```

python

```
[L480] class BlockType(Enum):
[L481] HEADING = "heading"
[L482] PARAGRAPH = "paragraph"
[L483] LIST = "list"
[L484] TABLE = "table"
[L485] CODE = "code"
[L486] ALGORITHM = "algorithm"
[L487] FIGURE_CAPTION = "figure_caption"
[L488] EQUATION = "equation"
[L489] DEFINITION = "definition"
[L490]
[L491] @dataclass
[L492] class DocumentBlock:
[L493] """Atomic unit of document structure."""
[L494] block_id: str
[L495] doc_id: str
[L496] page_number: int
[L497] region_index: int # Which region on the page
[L498] block_type: BlockType
[L499] text: str
[L500] reading_order: int # Global ordering across the document
[L501] confidence: float = 1.0 # Classification confidence
[L502] table_cells: list = None # Parsed table cells (For TABLE blocks only)
[L503]
```

```
[L504]
[L505] The parser uses rule-based heuristics for block classification:
[L506] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase
[L507] - **List:** Starts with `- `, `* `, `• `, or `1.`
[L508] - **Table:** Contains 3+ pipe characters (`|`) – uses regex `re.match(r'^(def |class |import |from )', line)`
[L509] - **Code:** Uses regex pattern matching (NOT `def ` in text, which falsely matches "definitely")
[L510] - **Definition:** Contains phrases like "is defined as", "refers to"
[L511] - **Default:** Paragraph
[L512]
[L513] #### Table Cell-Level Parsing
[L514]
[L515] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level st
[L516]
[L517]
```

python

```
[L518] def parse_table_cells(page, table_region) -> list:
[L519] """Extract table structure with cell-level detail.
[L520]
```

[L521] Uses pdfplumber for text-based tables, Camelot for image-based.

[L522] Returns list of rows, each row is a list of cell strings. """

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: [filecite]turn10file6[

[L523] metadata:

[L524] query_result_index: 6

[L525] file_id: file_00000000dcc082119f935683a5e13eed

[L526] version_id: 1

[L527] name: SHIA_RAG_Final_Report(4)(1).md

[L528] mime_type: text/markdown

[L529] surface: conversation

[L530] score: 0.030090497737556562

[L531] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--1

[L532] snippet:

[L533] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L534]

[L535] ---

[L536]

[L537] ## Table of Contents

[L538]

[L539] 1. Introduction

[L540] 2. Literature Review

[L541] 3. System Architecture

[L542] 4. Data Model — Layer 0

[L543] 5. Offline Pipeline — Layers 1-5

[L544] 6. Online Pipeline — Layers 6-7

[L545] 7. Operations Layer — Layer 8

[L546] 8. Novel Contributions

[L547] 9. Mathematical Framework

[L548] 10. Implementation

[L549] 11. Evaluation Framework

[L550] 12. Risk Analysis

[L551] 13. Conclusion & Future Work

[L552]

[L553] ---

[L554]

[L555] ## 1. Introduction

[L556]

[L557] ### 1.1 Problem Statement

[L558]

[L559] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L560]

[L561] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L562] 2. Embed each chunk into a vector space

[L563] 3. At query time, retrieve the top-k most similar chunks

[L564] 4. Feed these chunks as context to an LLM for answer generation

[L565]

[L566] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L567]

[L568] | Problem | Description | Impact |

[L569] |-----|-----|-----|

[L570] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L571] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L572] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L573] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |

[L574] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L575]

[L576] ### 1.2 Proposed Solution: SHIA-RAG

[L577]

[L578] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L579]

[L580]

[L581] Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L582] SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L583]	

[L584]

[L585] The system:

[L586] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L587] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L588] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L589] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L590] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L591] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

DEEP RESEARCH 23/08/2026, 22:57:57

Citation Marker: fileciteturn10file7

[L592] metadata:

[L593] query_result_index: 7

[L594] file_id: file_00000000dcc082119f935683a5e13eed

[L595] version_id: 1

[L596] name: SHIA_RAG_Final_Report(4)(1).md

[L597] mime_type: text/markdown

[L598] surface: conversation

[L599] score: 0.029910714285714284

[L600] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--0

[L601] snippet:

[L602] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L603]

[L604] > **Project Type:** Minor Project (M.Tech)

[L605] > **Date:** August 2026

[L606] > **Author:** [Your Name]

[L607]

[L608] ---

[L609]

[L610] ## Abstract

[L611]

[L612] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L613]

[L614] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L615]

[L616] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L617]

[L618] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L619]

[L620] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L621]

[L622] ---

[L623]

[L624] ## Table of Contents

[L625]

[L626] 1. Introduction

[L627] 2. Literature Review

[L628] 3. System Architecture

[L629] 4. Data Model — Layer 0

[L630] 5. Offline Pipeline — Layers 1-5

[L631] 6. Online Pipeline — Layers 6-7

[L632] 7. Operations Layer — Layer 8

[L633] 8. Novel Contributions

[L634] 9. Mathematical Framework

[L635] 10. Implementation

[L636] 11. Evaluation Framework

[L637] 12. Risk Analysis

[L638] 13. Conclusion & Future Work

[L639]

[L640] ---

[L641]

[L642] ## 1. Introduction

[L643]

[L644] ### 1.1 Problem Statement

[L645]

[L646] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L647]

[L648] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L649] 2. Embed each chunk into a vector space

[L650] 3. At query time, retrieve the top-k most similar chunks

[L651] 4. Feed these chunks as context to an LLM for answer generation

[L652]

[L653] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

CHATGPT 23/08/2026, 22:57:57

Bilkul. Ab hum **Layer 3 ko exactly Layer 1 aur Layer 2 ki tarah detail mein**, same running example ke saath samjhenge.

Hum wahi example continue karenge:

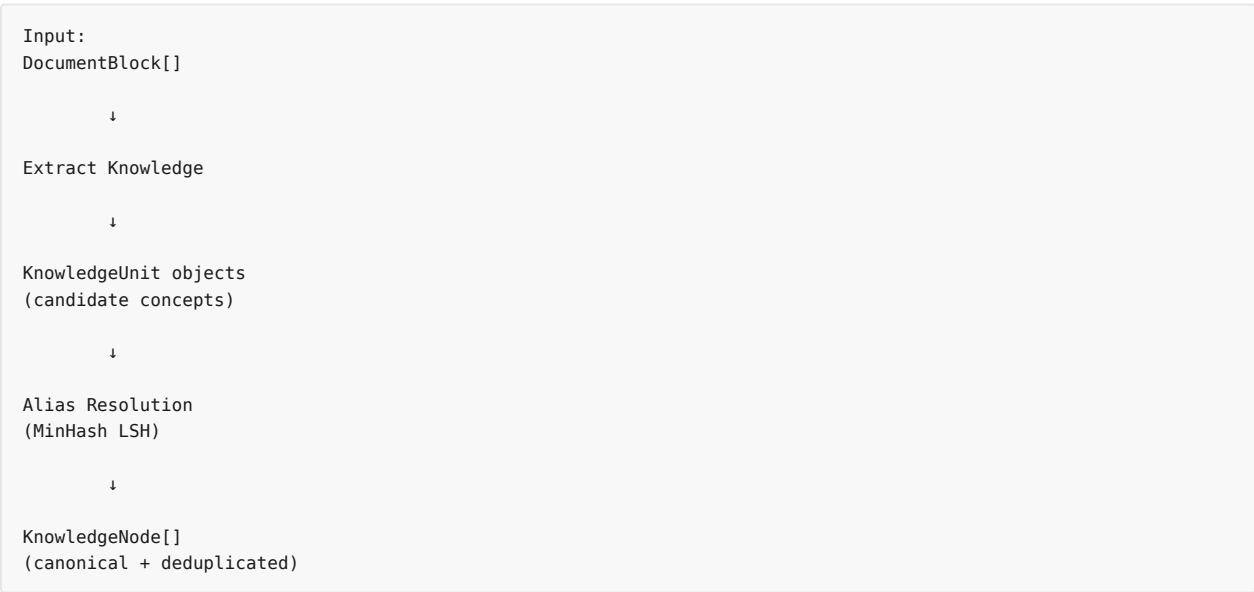
Operating Systems document → CPU Scheduling → Round Robin → Time Quantum

Layer 3 — Knowledge Extraction

Sabse pehle ek line mein samjho

Layer 3 takes the structured `DocumentBlock[]` from Layer 2, extracts actual knowledge such as concepts, definitions, properties, and candidate relations, and then merges different names of the same concept to create canonical and deduplicated `KnowledgeNode` objects.

Source ke according:



Layer 3 ki responsibility concepts, definitions aur candidate relations extract karna, phir synonyms/aliases merge karke canonical `KnowledgeNode` banana hai. [filecite turn8file0L14-L18](#) [filecite turn10file0L37-L46](#)

1. Layer 2 aur Layer 3 mein main difference kya hai?

Ye sabse important point hai.

Layer 2 kya karti thi?

Layer 2 bolti thi:

"Ye text kaunsa structural block hai?"

Example:

```
ROUND ROBIN
  ↓
HEADING

Round Robin is defined as a preemptive CPU scheduling algorithm.
  ↓
DEFINITION

- Uses a fixed time quantum
  ↓
LIST
```

Layer 2 structure samajhti hai.

Layer 3 kya karti hai?

Layer 3 poochti hai:

"Is structured content ke andar actual knowledge kya hai?"

Example:

```
Concept:
Round Robin

Definition:
A preemptive CPU scheduling algorithm...

Property:
preemptive = true

Property:
time_quantum = 20 ms

Candidate Relation:
Round Robin → USES → Time Quantum
```

Short difference:

```
Layer 2
"What type of content is this?"

Layer 3
"What knowledge does this content contain?"
```

2. Layer 3 ki zarurat kyu padi?

Layer 2 ke baad hamare paas structured blocks hain:

```

Block 1
Type: HEADING
Text: CPU SCHEDULING

Block 2
Type: DEFINITION
Text: Round Robin is defined as a preemptive CPU scheduling algorithm.

Block 3
Type: LIST
Text: Uses a fixed time quantum.

```

Lekin abhi system ke paas actual **knowledge objects** nahi hain.

For example, system ke paas abhi:

```
"Round Robin is a preemptive CPU scheduling algorithm."
```

sirf **text** ke form mein hai.

Lekin future layers ko chahiye:

```

KnowledgeNode:
  Round Robin

Definition:
  A preemptive CPU scheduling algorithm

Property:
  preemptive = true

Evidence:
  Original source sentence

```

Isliye Layer 3:

```

Structured Text
  ↓
Extract Meaningful Knowledge
  ↓
Structured Knowledge Objects

```

banati hai.

3. Layer 3 ka Input kya hai?

Layer 3 ka input:

```
DocumentBlock[]
```

Example:


```

DocumentBlock(
    block_id="BLK_001",
    block_type="HEADING",
    text="ROUND ROBIN"
)

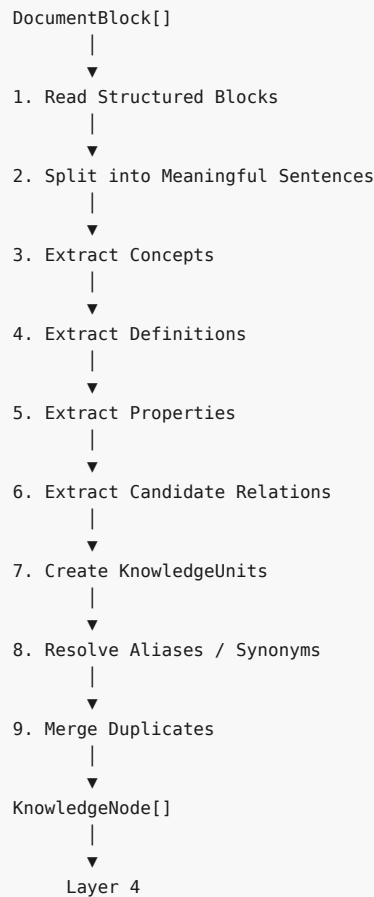
DocumentBlock(
    block_id="BLK_002",
    block_type="DEFINITION",
    text="Round Robin is defined as a preemptive CPU scheduling algorithm."
)

DocumentBlock(
    block_id="BLK_003",
    block_type="LIST",
    text="Uses a fixed time quantum."
)

```

Layer 3 in blocks ko process karke actual knowledge extract karti hai. System architecture ke data flow mein Layer 2 ke [Page → Region → Block] hierarchy ke baad Layer 3 sentence-level processing aur concepts/definitions/relations extraction karti hai. [filecite\turn10file0\L35-L46](#)

4. Layer 3 ka complete flow



Important: Layer 3 relations ko extract karti hai as **candidates**. Un relations ki ontology validation aur confidence scoring Layer 4 mein hoti hai. [filecite\turn8file0\L14-L18](#)

5. Hamara running example

Maan lo Layer 2 ne ye structured blocks diye:

Block 1

CPU SCHEDULING

Type:

HEADING

Block 2

Round Robin is defined as a preemptive CPU scheduling algorithm.

Type:

DEFINITION

Block 3

Each process receives a fixed time quantum of 20 ms.

Type:

PARAGRAPH

Block 4

Round Robin is also called RR.

Type:

PARAGRAPH

Ab Layer 3 actual processing start karegi.

6. Step 1 — Concept Extraction

Concept kya hota hai?

Simple language mein:

Concept is a meaningful entity or technical term that represents knowledge.

Hamare example mein:

CPU Scheduling
Round Robin
Time Quantum
Process

Ye possible concepts hain.

Source extraction prompt specifically extract karta hai:

Named entities
+
Technical terms

as concepts. 0filecite0turn8file00L20-L370

Example

Input:

Round Robin is defined as a preemptive CPU scheduling algorithm.

Layer 3 LLM ko structured extraction instruction deti hai.

Possible output:

Concept 1:
Round Robin

Concept 2:
CPU Scheduling

Ab Layer 3 inhe candidate knowledge objects ke form mein store kar sakti hai.

```
KnowledgeUnit
├──
├── Name: Round Robin
├── Source Block: BLK_002
└── Evidence: Original text
```

7. Layer 3 mein LLM ka use kyu?

Kyunki sirf keywords extract karna enough nahi hai.

Example:

Round Robin is a CPU scheduling algorithm.

Agar keyword extraction use karein:

Round
Robin
CPU
Scheduling
Algorithm

× Ye meaningful concepts ko tod sakta hai.

Lekin LLM semantic meaning samajhkar identify kar sakta hai:

Round Robin → One complete concept
CPU Scheduling → Another concept

Report ke according Layer 3 ka key extraction approach **LLM-based concept extraction** hai, with a structured JSON output for concepts, definitions, properties, relations, and domain. [filecite\turn7file1\130-L142\filecite\turn8file0\120-L37](#)

8. Step 2 — Definition Extraction

Ab Layer 3 dekhti hai:

Kya ye sentence kisi concept ko define kar raha hai?

Example:

Round Robin is defined as a preemptive CPU scheduling algorithm.

Yahan:

Concept:
Round Robin

Definition:
a preemptive CPU scheduling algorithm

Clearly extract ho sakta hai.

Lekin simple "x is y" rule kyu nahi?

Maan lo sentence:

The sky is blue today.

Agar system blindly bole:

X is Y
=
Definition

To ye galat hoga.

× Har "is" sentence definition nahi hota.

Source specifically ye problem mention karta hai.

Isliye Layer 3 definition detection ke liye:

Strategy 1 — Explicit phrases

"is defined as"
"refers to"
"Concept: Description"
"is a type/kind/form/category/class of"

Strategy 2 — Dependency Parsing

Sentence ka grammatical structure analyze kiya jata hai.

Strategy 3 — LLM Fallback

Agar previous methods sure nahi hain, to LLM classification use ho sakti hai, particularly definition-classified blocks ke liye.

📄filecite📄turn8file0📄L40-L74📄

9. Dependency Parsing yahan kya karta hai?

Simple example:

Round Robin is a preemptive scheduling algorithm.

Dependency parser sentence ki grammar samajhta hai:

Round Robin
|
└ is
|
└ a preemptive scheduling algorithm

Yahan system dekh sakta hai:

Subject:
Round Robin

Copular verb:
is

Description:
a preemptive scheduling algorithm

Isse system distinguish karne ki koshish karta hai:

Actual definition
vs
Normal statement

10. Step 3 — Property Extraction

Ab input:

Each process receives a fixed time quantum of 20 ms.

Layer 3 information extract kar sakti hai:

Concept:
Round Robin

Property:
time_quantum

Type:
integer

Value:
20

Unit:
ms

Conceptually:

```
{
  "time_quantum": {
    "type": "integer",
    "value": 20,
    "unit": "ms"
  }
}
```

Source extraction schema explicitly key attributes with their **types and values** extract karne ke liye properties include karta hai. `filecite\turn8file0\L23-L37`

11. Property aur Concept mein difference

Ye important hai.

Concept

Round Robin

Ek knowledge entity hai.

Property

```
Time Quantum = 20 ms
```

Concept ke baare mein specific information hai.

Simple analogy:

```
Person
|
├─ Name → Rahul
├─ Age → 25
└─ City → Mumbai
```

Yahan:

```
Person = Entity/Concept
Age = Property
25 = Property Value
```

Similarly:

```
Round Robin = Concept

preemptive = Property → true

time_quantum = Property → 20 ms
```

12. Step 4 — Candidate Relation Extraction

Ab Layer 3 concepts ke beech possible relationship identify karti hai.

Example:

```
Round Robin uses a fixed time quantum.
```

Possible extraction:

```
Source:
Round Robin

Relation:
USES

Target:
Time Quantum
```

So:

```
Round Robin
|
USES
▼
Time Quantum
```

Another example:

```
Round Robin is a type of CPU Scheduling.
```

Possible candidate relation:

```

Round Robin
  |
  IS_A
  ▼
CPU Scheduling

```

Source extraction prompt relation types जैसे `IS_A`, `PART_OF`, `USES`, `CAUSES` extract करता है। [filecite\turn8file0\L23-L37](#)

13. Lekin ye relation final kyu nahi hai?

Bahut important.

Layer 3 bolti hai:

"Mujhe text se ye relation mila hai."

But it does **not yet guarantee**:

"Ye relation valid ontology relation hai aur fully trusted hai."

Example:

LLM extract kare:

```

Round Robin
  PART_OF
  CPU Scheduling

```

But maybe correct relation is:

```

Round Robin
  IS_A
  CPU Scheduling Algorithm

```

Isliye Layer 3:

```

Candidate Relation

```

banati hai.

Aur Layer 4:

```

Validate
+
Ontology Check
+
Confidence Scoring

```

karti hai.

Source explicitly Layer 3/Layer 4 boundary define करता है: **Layer 3 candidate extraction**, while **Layer 4 validation and scoring** करता है। [filecite\turn8file0\L14-L18](#)

14. Step 5 — Domain Extraction

System ko concept ka subject area bhi identify करना है।

Example:

```

Round Robin

```

Ye kis domain ka concept है?

```
Operating Systems
```

So:

```
Domain:  
Operating Systems
```

Source extraction prompt explicitly domain extract karta hai, for example **"Operating Systems."** `filecite\turn8file0\L23-L37`

15. Step 6 — KnowledgeUnit kya hai?

Layer 3 ke beech mein ek intermediate object hota hai:

```
KnowledgeUnit
```

Isko simple language mein samjho:

A KnowledgeUnit is a temporary extracted piece of knowledge before duplicate and alias resolution.

Example:

```
KnowledgeUnit 1  
Name: Round Robin  
Source: BLK_002  
Definition: Preemptive CPU scheduling algorithm
```

Dusre block se:

```
KnowledgeUnit 2  
Name: RR  
Source: BLK_010  
Definition: CPU scheduling using time slices
```

Aur teesre document se:

```
KnowledgeUnit 3  
Name: Round-Robin Scheduling  
Source: BLK_050
```

Ab ye teen alag objects hain.

Lekin actual mein:

```
RR  
Round Robin  
Round-Robin Scheduling  
  
↓  
  
Same concept?
```

Yahi next step solve karta hai.

16. Step 7 — Alias Resolution

Problem

Same concept different names se aa sakta hai:

Round Robin
RR
Round-Robin Scheduling

Agar hum inhe separate nodes bana dein:

Node 1 → Round Robin
Node 2 → RR
Node 3 → Round-Robin Scheduling

to Knowledge Forest mein unnecessary duplicates honge.

- × Duplicate knowledge
- × Fragmented evidence
- × Retrieval confusion

Isliye:

Alias Resolution

use hota hai.

Source specifically ye three example names ko same concept ke aliases ke form mein use karta hai. `filecite` `turn8file0` `L76-L84`

17. Alias Resolution kaise hota hai?

Source ke according:

Step 1

Concept names aur aliases se:

MinHash LSH Index

build hota hai.

Threshold:

0.7

Step 2

Similar concepts find kiye jaate hain.

Step 3

Connected groups banaye jaate hain.

Step 4

Har group ke liye:

Canonical Name

select hota hai.

Generally source ke according:

Most Frequent Name

canonical name ban sakta hai.

Step 5

Baaki names:

Aliases

ban jaate hain.

filecite\turn8file0\L76-L84

18. MinHash LSH ko simple language mein samjho

Suppose 1 million concepts hain.

Har concept ko har doosre concept se compare karna:

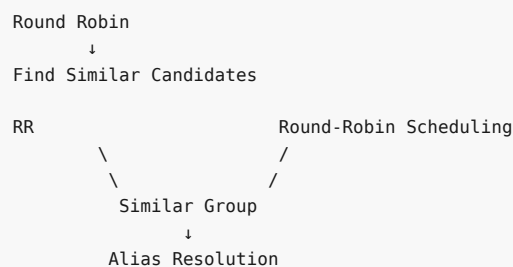
$N \times N$ comparisons

bahut expensive ho sakta hai.

MinHash LSH ka basic purpose:

Pehle quickly similar-looking candidates find karo, phir relevant candidates ko merge decision ke liye process karo.

Example:



Report ke according Layer 3 alias resolution **MinHash LSH** use karta hai. filecite\turn8file0\L76-L84

19. Lekin similarity dangerous bhi ho sakti hai

Very important problem.

Example:

Preemptive
Non-Preemptive

Dono kaafi similar hain.

Agar system sirf similarity dekhe:

Similarity High
↓
Merge

To:

× Galat!

Dono opposite concepts hain.

Source specifically is problem ko **antonym protection** ke through handle karta hai. filecite\turn8file0\L76-L84

20. Antonym Protection

Merge karne se pehle system check karta hai:

Check 1 — Known opposite relation

```
CONTRASTS_WITH
```

Agar:

```
Preemptive
↕
CONTRASTS_WITH
↕
Non-Preemptive
```

to merge nahi karega.

Check 2 — Negation prefixes

System prefixes check karta hai:

```
non-
un-
in-
dis-
```

Example:

```
preemptive
non-preemptive
```

Similar hone ke bawajood:

```
DO NOT MERGE
```

Source alias-resolution strategy mein both `CONTRASTS_WITH` check aur negation prefixes specified hain.

filecite[turn8file0L96-L127]

21. Alias Resolution ke baad kya merge hota hai?

Suppose:

```
Unit 1:
Round Robin

Unit 2:
RR

Unit 3:
Round-Robin Scheduling
```

After merge:

```

KnowledgeNode
├── Canonical Name:
│   Round Robin
├── Aliases:
│   RR
│   Round-Robin Scheduling
├── Definitions:
│   Definition 1
│   Definition 2
├── Properties:
│   time_quantum = 20 ms
│   preemptive = true
└── Evidence:
    Source Block 1
    Source Block 2
    Source Block 3

```

Source merge strategy definitions, aliases, properties aur evidence ko combine karta hai; extraction confidence ka average use karta hai, aur property conflicts log kiye jaate hain. `filecite\turn8file0\L85-L99`

22. Evidence preserve karna kyu important hai?

Maan lo KnowledgeNode bana:

```
Round Robin
```

Baad mein user poochta hai:

What is Round Robin?

System ko answer ke source tak jaana hoga.

Isliye node ke saath evidence preserve hota hai:

```

KnowledgeNode
├── Knowledge
└── Evidence
    ├── Original Text
    ├── Source Document
    ├── Source Block
    └── Page Number

```

Layer 0 ke KnowledgeNode schema mein evidence structure original text ke saath `source_doc`, `source_block`, aur `page` preserve karta hai. `filecite\turn7file4\L288-L317`

Iska benefit

```

Knowledge
↓
Trace back
↓
Original evidence

```

Ye future verification aur citations ke liye important hai.

23. Embedding ka kya hota hai jab aliases merge hote hain?

Suppose:

```
RR
Round Robin
Round-Robin Scheduling
```

ke separate embeddings already hain.

Ek option hota:

```
Sab embeddings ka average
```

Lekin report ke according:

Do NOT average embeddings.

Kyunki averaging merged concept ki semantic representation ko distort kar sakti hai.

Instead:

```
Canonical Name
+
Merged Definition
      ↓
Generate New Embedding
```

Example:

```
Round Robin
+
"A preemptive CPU scheduling algorithm..."
      ↓
Embedding Model
      ↓
Fresh Embedding
```

Source explicitly merged nodes ko re-embed karne ka approach specify karta hai. [filecite turn9file0L13-L28](#)

24. Hamare example ka complete Layer 3 processing

Input from Layer 2

```
Block 1:
CPU SCHEDULING
Type: HEADING

Block 2:
Round Robin is defined as a preemptive CPU scheduling algorithm.
Type: DEFINITION

Block 3:
Each process receives a fixed time quantum of 20 ms.
Type: PARAGRAPH

Block 4:
Round Robin is also called RR.
Type: PARAGRAPH
```

Step 1 — Concept Extraction

Extracted:

CPU Scheduling
Round Robin
Time Quantum
RR

Step 2 — Definition Extraction

Concept:
Round Robin

Definition:
A preemptive CPU scheduling algorithm

Step 3 — Property Extraction

Property:
time_quantum

Value:
20

Unit:
ms

Possible additional property:

preemptive = true

Step 4 — Candidate Relation Extraction

Round Robin
IS_A
↓
CPU Scheduling Algorithm

and:

Round Robin
USES
↓
Time Quantum

Remember:

These are candidate relations.

Final validation Layer 4 mein hogi.

Step 5 — Alias Extraction

Round Robin
RR

System detect karta hai ki dono same concept ho sakte hain.

Step 6 — Alias Resolution

```

Round Robin + RR
    ↓
MERGE
    ↓
Canonical KnowledgeNode

```

25. Final Output — KnowledgeNode

Finally Layer 3 output:

```

KnowledgeNode(
  canonical_name = "Round Robin",

  aliases = [
    "RR",
    "Round-Robin Scheduling"
  ],

  definitions = [
    "A preemptive CPU scheduling algorithm."
  ],

  domain = "Operating Systems",

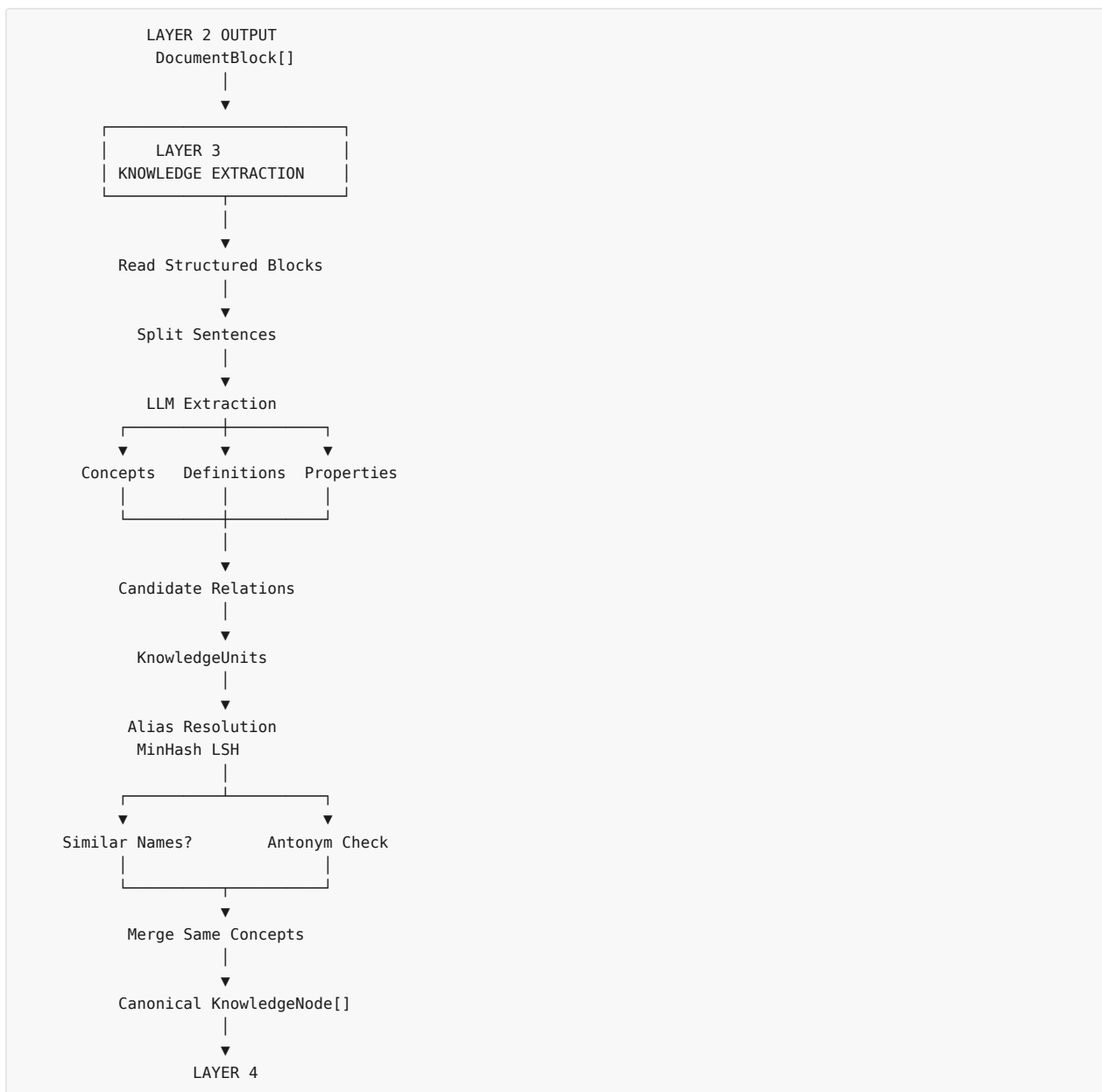
  properties = {
    "time_quantum": {
      "type": "integer",
      "value": 20,
      "unit": "ms"
    },
    "preemptive": {
      "type": "boolean",
      "value": True
    }
  },

  evidence = [
    {
      "text": "Round Robin is defined as...",
      "source_doc": "DOC_001",
      "source_block": "BLK_002",
      "page": 15
    }
  ]
)

```

Layer 3 ka final output canonical aur deduplicated KnowledgeNode[] hai. [filecite\turn8file0\L14-L18](#)

26. Layer 3 ka actual complete flow ek diagram mein



27. Layer 3 mein kaunsi technologies/algorithms use ho rahi hain?

| Technology / Algorithm | Kaam |

|---|---|

| **LLM** | Concepts, definitions, properties, candidate relations extract karna |

| **Structured JSON output** | Extracted knowledge ko standard format mein lena |

| **Regex** | Explicit definition patterns detect karna |

| **Dependency Parsing** | Grammatical definition structure detect karna |

| **LLM fallback** | Difficult definition classification |

| **MinHash LSH** | Similar aliases/duplicates efficiently find karna |

| **Connected Components** | Similar KnowledgeUnits ko groups mein organize karna |

| **Antonym Protection** | Opposite concepts ko galti se merge hone se bachana |

| **Re-embedding** | Merged canonical node ki fresh semantic representation banana |

Ye Layer 3 specification mein defined extraction, dependency-based definition detection, alias resolution, antonym protection, aur re-embedding strategy se derived hai. [filecite\turn8file0\L20-L37](#) [filecite\turn8file0\L40-L74](#) [filecite\turn8file0\L76-L127](#) [filecite\turn9file0\L13-L28](#)

28. Layer 3 mein possible errors aur safeguards

Error 1 — Har noun ko concept samajh lena

Example:

```
The process runs for 20 ms.
```

System har word ko concept bana sakta hai:

```
Process
Runs
20
ms
```

× Noise.

Safeguard

Structured LLM extraction + technical/named concepts extraction.

Error 2 — Normal statement ko definition samajhna

```
The sky is blue today.
```

Simple "X is Y" rule se:

```
False Definition ❌
```

Safeguard

```
Explicit patterns
  ↓
Dependency Parsing
  ↓
LLM Fallback
```

filecite turn8file0 L40-L74

Error 3 — Same concept ke multiple nodes

```
RR
Round Robin
Round-Robin Scheduling
```

Safeguard

```
MinHash LSH
+
Alias Resolution
```

filecite turn8file0 L76-L84

Error 4 — Opposite concepts ko merge kar dena

```
Preemptive
Non-Preemptive
```

Safeguard

```

CONTRASTS_WITH check
+
Negation prefix check

```

filecite\turn8file0\L96-L127

Error 5 — Relation galat extract hona

Example:

```

Round Robin → PART_OF → CPU Scheduling

```

instead of some other valid ontology relation.

Safeguard

Layer 3 is relation ko **final nahi maanta**.

It passes it as:

```

Candidate Relation

```

Then:

```

Layer 4
→ Ontology Validation
→ Confidence Scoring

```

filecite\turn8file0\L14-L18

29. Layer 3 kya nahi karti?

Ye bhi bahut important hai.

Layer 3:

- × Final relation validation nahi karti
- × Final `KnowledgeEdge` produce nahi karti
- × Hierarchy nahi banati
- × Parent-child structure decide nahi karti
- × Knowledge Forest construct nahi karti

Layer 3 ka kaam:

Document se knowledge extract karna aur duplicate/synonym concepts ko merge karke clean canonical knowledge nodes banana.

30. Layer 2 → Layer 3 → Layer 4 ka difference

LAYER 2

Document Structure

"Ye heading hai"
"Ye paragraph hai"
"Ye definition hai"

↓

LAYER 3

Knowledge Extraction

"Round Robin ek concept hai"
"Ye uski definition hai"
"Ye uski property hai"
"RR iska alias hai"
"Ye possible relation hai"

↓

LAYER 4

Relation Management

"Kya relation valid hai?"
"Ontology mein allowed hai?"
"Confidence kitna hai?"
"Final KnowledgeEdge banao"

31. Layer 3 ka real-world analogy

Imagine tumhare paas ek textbook hai.

Layer 2 ka editor bolta hai:

Ye Chapter Heading hai.
Ye Definition hai.
Ye Table hai.
Ye Paragraph hai.

Layer 3 ka knowledge analyst bolta hai:

Is definition mein main concept Round Robin hai.

Iska alias RR hai.

Iski property:
Time Quantum = 20 ms.

Ye CPU Scheduling se related hai.

Layer 4 ka validator bolta hai:

Kya relation exactly correct hai?
Kya ontology mein allowed hai?
Evidence strong hai?
Confidence kitna hai?

32. Final Summary

- | Point | Layer 3 |
- | ---| ---|
- | Name | Knowledge Extraction |
- | Input | DocumentBlock[] |
- | Main purpose | Structured blocks se actual knowledge extract karna |
- | Extracts | Concepts, definitions, properties, candidate relations, domain |
- | Intermediate output | KnowledgeUnit objects |
- | Deduplication | Alias Resolution |
- | Main algorithm | MinHash LSH |
- | Special safeguard | Antonym Protection |
- | Definition detection | Explicit patterns + Dependency Parsing + LLM fallback |
- | Embedding after merge | Re-embed canonical node; don't average |
- | Final output | Canonical, deduplicated KnowledgeNode[] |
- | Next layer | Layer 4 — Relation Management |

Sabse important one-line understanding

“Layer 3 transforms structured document blocks into structured knowledge. It extracts concepts, definitions, properties, and candidate relations, resolves different names of the same concept, and produces canonical, deduplicated KnowledgeNodes for the next layer.”

Complete flow yaad rakhna:



Sabse important conceptual shift yahan hota hai: Layer 1 aur Layer 2 tak system document ko process aur structure kar raha tha. Layer 3 se system first time actual knowledge objects banana start karta hai.

YOU 24/08/2026, 00:12:10

mujhe layer 3 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 3 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 24/08/2026, 00:12:18

{"size":"1024x1024","n":1,"is_style_transfer":false,"transparent_background":false}

CHATGPT 24/08/2026, 00:13:08

Layer 3 – Knowledge Extraction

You can explain it to your guide like this:

“Layer 3 receives the structured DocumentBlock[] from Layer 2 and extracts actual knowledge from the document. It identifies concepts, definitions, properties, and candidate relationships.”

What happens in Layer 3?

- It extracts important **concepts and technical terms**.
- It identifies **definitions and properties** related to those concepts.
- It extracts possible **relationships between concepts**.
- Different names of the same concept, such as **“Round Robin” and “RR,”** are identified and merged.
- Finally, it creates clean and canonical **KnowledgeNode[]** objects while preserving the supporting evidence.

Output

Canonical KnowledgeNode[]

Each KnowledgeNode can contain:

Concept Name + Aliases + Definition + Properties + Evidence

This output is then sent to **Layer 4**, where the candidate relationships are validated and scored.

Simple conclusion

“In short, Layer 3 converts structured document content into actual, clean, and deduplicated knowledge that can be used to build reliable relationships in the next layers.”

The explanation follows the Layer 3 source specification for concept, definition, property, candidate-relation extraction, alias resolution, and canonical KnowledgeNode[] creation. [filecite\turn8file0\L14-L37](#) [filecite\turn8file0\L76-L99](#)

YOU 24/08/2026, 00:22:36

ab mujhe layer 4 smjhao detailed me jese baki alyer smjhayi thi

CHATGPT 24/08/2026, 00:22:37

{"paths":["files"],"query":"search"}

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: [filecite\turn11file0](#)

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_00000000dcc082119f935683a5e13eed
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report(4)(1).md
- [L6] mime_type: text/markdown
- [L7] surface: conversation
- [L8] score: 0.030158730158730156
- [L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--23
- [L10] snippet:

```

[L11] name.lower() for u in group]
[L12] has_antonym = False
[L13] for i, n1 in enumerate(names):
[L14] for n2 in names[i+1:]:
[L15] # Check 1: Negation prefix
[L16] if NEGATION_PREFIXES.sub("", n1) == NEGATION_PREFIXES.sub("", n2):
[L17] has_antonym = True
[L18] # Check 2: Known CONTRASTS_WITH relation
[L19] if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L20] has_antonym = True
[L21]
[L22] if has_antonym:
[L23] safe_groups.extend([[u] for u in group]) # Don't merge
[L24] else:
[L25] safe_groups.append(group)
[L26]
[L27] return [create_canonical_node(group) for group in safe_groups]
[L28]

```

```

[L29]
[L30] ##### Embedding Strategy on Merge
[L31]
[L32] When nodes are merged during alias resolution, embeddings are handled as follows:
[L33] - **Do NOT average embeddings** – averaging collapses distinct semantic information
[L34] - **Re-embed the merged node** using the canonical name + merged definitions
[L35] - This ensures the embedding accurately represents the merged concept
[L36]
[L37]

```

python

```

[L38] def merge_node_embeddings(merged_node, embed_fn):
[L39] """Re-compute embedding for merged node instead of averaging.
[L40]
[L41] Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L42] produce a vector that represents neither concept well.
[L43] Instead, embed the canonical representation."""
[L44] text_to_embed = merged_node.canonical_name
[L45] if merged_node.definitions:
[L46] text_to_embed += " " + merged_node.definitions[0]
[L47] merged_node.embedding = embed_fn(text_to_embed)
[L48]

```

```

[L49]
[L50] ---
[L51]
[L52] ### 5.4 Layer 4: Relation Management
[L53]
[L54] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence,
[L55]
[L56] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)
[L57]
[L58] **Pipeline:**
[L59] 1. **Validate relation type** – check against ontology; if unknown type, attempt inference via LLM
[L60] 2. **Resolve target node** – map target concept name to a `KnowledgeNode`
[L61] 3. **Check for duplicates** – merge evidence into existing edge if duplicate
[L62] 4. **Determine edge class** – hierarchical or semantic based on ontology mapping
[L63] 5. **Calculate confidence** – weighted combination of source confidence, evidence length, and semantic similar
[L64] 6. **Resolve cross-document conflicts** – when multiple documents disagree on a relation
[L65]
[L66]

```

python

```

[L67] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L68] """Confidence = 0.5 (base) + 0.15 source_conf + 0.10 evidence_quality + 0.10 sim"""
[L69] conf = 0.5
[L70] conf += 0.15 source.confidence
[L71] conf += 0.10 min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L72] conf += 0.10 cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L73] return min(conf, 1.0)
[L74]

```

```

[L75]
[L76] #### Cross-Document Conflict Resolution
[L77]
[L78] When multiple documents provide **conflicting relations** for the same concept pair (e.g., Document A says "Ro
[L79]
[L80]

```

python

```

[L81] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:
[L82] """Resolve conflicting relations from different documents.
[L83]
[L84] Strategy: Evidence-weighted voting
[L85] 1. Group edges by (source_node, target_node) pair
[L86] 2.

```

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: □filecite□turn11file1□

```

[L87] metadata:
[L88] query_result_index: 1
[L89] file_id: file_00000000dcc082119f935683a5e13eed
[L90] version_id: 1
[L91] name: SHIA_RAG_Final_Report(4)(1).md
[L92] mime_type: text/markdown
[L93] surface: conversation
[L94] score: 0.030117753623188408
[L95] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--22

```

[L96] snippet:

[L97] ##### Alias Resolution (with Antonym Protection)

[L98]

[L99] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them — but must **avoid merging antonyms** (e.g., "preemptive" and "non-preemptive" have high similarity but are distinct concepts).

[L100]

[L101] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L102] 2. **Antonym check**: Before merging, verify candidates do not have a `CONTRASTS_WITH` relation

[L103] 3. Find connected components of near-duplicate units

[L104] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L105]

[L106]

```
[L107] def alias_resolution(units: list, relation_cache: dict) -> list:
[L108]     """Merge KnowledgeUnits referring to the same concept.
[L109]
[L110]     Merge strategy:
[L111]     - Canonical name: most frequent name across units
[L112]     - Aliases: union of all names and aliases
[L113]     - Definitions: union of all definitions
[L114]     - Confidence: average of unit extraction confidences
[L115]     - Properties: merged (latest value wins, conflicts logged)
[L116]
[L117]     Antonym protection:
[L118]     - Before merging, check if candidates have CONTRASTS_WITH relation
[L119]     - Also check for negation prefixes (non-, un-, in-, dis-)
[L120]     """
[L121]     import re
[L122]     NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L123]
[L124]     lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L125]     # ... build LSH index from concept names + aliases ...
[L126]
[L127]     raw_groups = find_connected_components(units, lsh)
[L128]
[L129]     # Antonym protection: split groups containing antonym pairs
[L130]     safe_groups = []
[L131]     for group in raw_groups:
[L132]         names = [u.name.lower() for u in group]
[L133]         has_antonym = False
[L134]         for i, n1 in enumerate(names):
[L135]             for n2 in names[i+1:]:
[L136]                 # Check 1: Negation prefix
[L137]                 if NEGATION_PREFIXES.sub('', n1) == NEGATION_PREFIXES.sub('', n2):
[L138]                     has_antonym = True
[L139]                 # Check 2: Known CONTRASTS_WITH relation
[L140]                 if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L141]                     has_antonym = True
[L142]
[L143]         if has_antonym:
[L144]             safe_groups.extend([u for u in group]) # Don't merge
[L145]         else:
[L146]             safe_groups.append(group)
[L147]
[L148]     return [create_canonical_node(group) for group in safe_groups]
[L149]
```

[L150]

[L151] ##### Embedding Strategy on Merge

[L152]

[L153] When nodes are merged during alias resolution, embeddings are handled as follows:

[L154] - **Do NOT average embeddings** — averaging collapses distinct semantic information

[L155] - **Re-embed the merged node** using the canonical name + merged definitions

[L156] - This ensures the embedding accurately represents the merged concept

[L157]

[L158]

```
[L159] def merge_node_embeddings(merged_node, embed_fn):
[L160]     """Re-compute embedding for merged node instead of averaging.
[L161]
[L162]     Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L163]     produce a vector that represents neither concept well.
[L164]     Instead, embed the canonical representation."""
[L165]     text_to_embed = merged_node.canonical_name
[L166]     if merged_node.definitions:
[L167]         text_to_embed += " " + merged_node.definitions[0]
[L168]     merged_node.embedding = embed_fn(text_to_embed)
[L169]
```

[L170]

[L171] ---

[L172]

[L173] ### 5.4 Layer 4: Relation Management

[L174]

[L175] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve cross-document conflicts, and produce final `KnowledgeEdge` objects.

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: `filecite`turn11file2

[L176] metadata:

[L177] query_result_index: 2

[L178] file_id: file_00000000dcc082119f935683a5e13eed

[L179] version_id: 1

[L180] name: SHIA_RAG_Final_Report(4)(1).md

[L181] mime_type: text/markdown

[L182] surface: conversation

[L183] score: 0.028588564574170333

[L184] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6

[L185] snippet:

[L186]

```
[L187] graph TD
[L188]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L189]         L1["Layer 1: Input Processing<br>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br>Text normalization, tokenization"]
[L190]         L2 --> L3["Layer 3: Knowledge Extraction<br>Concept detection, alias resolution"]
[L191]         L3 --> L4["Layer 4: Relation Management<br>Triple extraction, confidence scoring"]
[L192]         L4 --> L5["Layer 5: SHIA Core<br>CPG++ → PRE → HV → FI → CLD"]
[L193]         L5 --> KF["Knowledge Forest Store<br>(Graph DB + Vector DB)"]
[L194]     end
[L195]
[L196]     subgraph "Phase 2: Online – Retrieval & Generation"
[L197]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br>Query analysis, forest traversal, RB0"]
[L198]         KF --> L6
[L199]         L6 --> L7["Layer 7: Generation Engine<br>Prompt assembly, LLM, verification"]
[L200]         L7 --> A["Answer + Citations"]
[L201]     end
[L202]
[L203]     subgraph "Phase 3: Continuous – Operations"
[L204]         L8["Layer 8: Operations<br>Logging, monitoring, feedback loop"]
[L205]         A --> L8
[L206]         L8 -->|"Self-Evolution<br>Feedback"| KF
[L207]     end
[L208]
```

[L209]

[L210] ### 3.2 Data Flow

[L211]

[L212]

```
[L213] Raw Document (PDF/DOCX/HTML)
[L214]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L215] Normalized Text + Metadata
[L216]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L217] [Page → Region → Block] hierarchy
[L218]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L219] KnowledgeUnit objects (candidate concepts)
[L220]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L221] KnowledgeNode objects (canonical, deduplicated)
[L222]     ↓ Layer 4: Validate relations against ontology, score confidence
[L223] KnowledgeEdge objects (typed, scored)
[L224]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L225] Knowledge Forest (rooted trees + cross-links)
[L226]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L227] Selected context nodes (within token budget)
[L228]     ↓ Layer 7: Build prompt, invoke LLM
[L229]     ↓ Layer 7: Verify claims against evidence, bind citations
[L230] Answer with citations + confidence score
[L231]     ↓ Layer 8: Log, collect feedback, update forest weights
[L232]
```

[L233]

[L234] ### 3.3 Layer Summary

[L235]

[L236] | Layer | Name | Input | Output | Key Algorithms |

[L237] |-----|-----|-----|-----|-----|

[L238] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L239] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L240] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L241] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L242] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L243] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L244] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L245] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L246] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L247]
[L248] ---
[L249]
[L250] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: `[[filecite]]turn11file3`

[L251] metadata:
[L252] query_result_index: 3
[L253] file_id: file_00000000dcc082119f935683a5e13eed
[L254] version_id: 1
[L255] name: SHIA_RAG_Final_Report(4)(1).md
[L256] mime_type: text/markdown
[L257] surface: conversation
[L258] score: 0.026702720973466286
[L259] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--7
[L260] snippet:
[L261] deduplicated)
[L262] ↓ Layer 4: Validate relations against ontology, score confidence
[L263] KnowledgeEdge objects (typed, scored)
[L264] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L265] Knowledge Forest (rooted trees + cross-links)
[L266] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L267] Selected context nodes (within token budget)
[L268] ↓ Layer 7: Build prompt, invoke LLM
[L269] ↓ Layer 7: Verify claims against evidence, bind citations
[L270] Answer with citations + confidence score
[L271] ↓ Layer 8: Log, collect feedback, update forest weights
[L272]

```
[L273]
[L274] ### 3.3 Layer Summary
[L275]
[L276] | Layer | Name | Input | Output | Key Algorithms |
[L277] |-----|-----|-----|-----|-----|
[L278] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L279] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L280] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L281] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L282] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L283] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L284] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L285] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L286] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L287]
[L288] ---
[L289]
[L290] ## 4. Data Model – Layer 0
[L291]
[L292] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode`
[L293]
[L294] ### 4.1 KnowledgeNode Schema
[L295]
[L296] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L297]
[L298]
```

json

```
[L299] {
[L300] "node_id": "NODE_000789",
[L301] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L302] "canonical_name": "Round Robin",
[L303] "aliases": ["RR", "Round-Robin Scheduling"],
[L304] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L305] "summaries": ["CPU scheduling method using time slices"],
[L306] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L307] "domain": "Operating Systems",
[L308] "properties": {
[L309] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L310] "preemptive": {"type": "boolean", "value": true}
[L311] },
[L312] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L313] "parent": "NODE_000456",
[L314] "children": ["NODE_001001", "NODE_001002"],
[L315] "related": ["NODE_000890"],
[L316] "incoming_edges": ["EDGE_001"],
[L317] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L318] "evidence": [
[L319] {
[L320] "evidence_type": "DEFINITION",
[L321] "text": "Round Robin is a preemptive scheduling algorithm...",
[L322] "source_doc": "DOC_000001",
[L323] "source_block": "BLK_000234",
[L324] "page": 42
[L325] }
[L326] ],
```

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: `filecite`turn11file4

[L327] metadata:
[L328] query_result_index: 4
[L329] file_id: file_00000000dcc082119f935683a5e13eed
[L330] version_id: 1
[L331] name: SHIA_RAG_Final_Report(4)(1).md
[L332] mime_type: text/markdown
[L333] surface: conversation
[L334] score: 0.025353016688061617
[L335] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--26
[L336] snippet:
[L337] ---
[L338]
[L339] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction
[L340]
[L341] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.
[L342]
[L343] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest
[L344]
[L345] #### Pipeline Overview
[L346]
[L347]

```
[L348] graph LR
[L349]   N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L350]   CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L351]   PRE --> HV{"HV<br/>Validate #1?"}
[L352]   HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L353]   HV -->|"Fail"| HV2{"HV<br/>Validate #2?"}
[L354]   HV2 -->|"Pass"| FI
[L355]   HV2 -->|"Fail"| HV3{"HV<br/>Validate #3?"}
[L356]   HV3 -->|"Pass"| FI
[L357]   HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L358]   FI --> CLD["CLD<br/>Discover Cross-Links"]
[L359]   CLD --> DONE["Node Placed ✓"]
[L360]
```

[L361]
[L362] #### 5.5.1 CPG++ — Candidate Parent Generator
[L363]
[L364] Generates up to 5 candidate parents for a new node using a multi-stage filter:
[L365]
[L366]

```

[L367] def cpg_plus_plus(node_q, forest, config):
[L368]     """Generate ≤k candidate parents for node q."""
[L369]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L370]     domain = classify_domain(node_q)
[L371]     domain_nodes = forest.get_nodes_by_domain(domain)
[L372]
[L373]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L374]     if len(domain_nodes) > config.ann_k:
[L375]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L376]         candidates = [r for r in ann_results if r in domain_nodes]
[L377]     else:
[L378]         candidates = domain_nodes
[L379]
[L380]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L381]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L382]
[L383]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L384]     # estimate_abstraction_level uses multi-factor heuristic + optional LLM fallback
[L385]     q_level = estimate_abstraction_level(node_q)
[L386]     candidates = [c for c in candidates
[L387]                   if c.abstraction_level > q_level
[L388]                   and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L389]
[L390]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L391]     candidates = sorted(candidates,
[L392]                         key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L393]
[L394]     return candidates[:config.max_candidates] # Return top 5
[L395]

```

[L396]

[L397] **Overall complexity:** $O(\log N)$ dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L398]

[L399] **Abstraction Level Estimation:**

[L400]

[L401] ```python

[L402] def estimate_abstraction_level(node) -> float:

[L403] """Multi-factor heuristic for estimating abstraction level of unplaced nodes.

[L404] Returns a score in [0.0, 1.0] where higher = more abstract.

[L405]

[L406] NOTE: Simple name-length alone is insufficient (e.g., 'CPU' is short but

[L407] context-dependent). We combine multiple signals for robustness."""

[L408] score = 0.0

[L409]

[L410] # Factor 1: Name word count (shorter multi-word names tend to be more abstract)

[L411] word_count = len(node.name.split())

[L412] if word_count <= 2:

[L413] score += 0.15

[L414] elif word_count >= 4:

[L415] score -= 0.

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: turn11file5

[L416] metadata:

[L417] query_result_index: 5

[L418] file_id: file_00000000dcc082119f935683a5e13eed

```

[L419] version_id: 1
[L420] name: SHIA_RAG_Final_Report(4)(1).md
[L421] mime_type: text/markdown
[L422] surface: conversation
[L423] score: 0.02209219011225221
[L424] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--25
[L425] snippet:
[L426] Select the relation_type with highest weighted vote
[L427] 4. Log conflicts for manual review
[L428] """
[L429] from collections import defaultdict
[L430]
[L431] # Group by node pair
[L432] pair_groups = defaultdict(list)
[L433] for edge in candidate_edges:
[L434]     key = (edge.source_node, edge.target_node)
[L435]     pair_groups[key].append(edge)
[L436]
[L437] resolved = []
[L438] for (src, tgt), edges in pair_groups.items():
[L439]     if len(edges) == 1:
[L440]         resolved.append(edges[0])
[L441]         continue
[L442]
[L443] # Tally weighted votes per relation type
[L444] votes = defaultdict(float)
[L445] evidence_pool = defaultdict(list)
[L446] for edge in edges:
[L447]     weight = edge.confidence * len(edge.evidence)
[L448]     votes[edge.relation_type] += weight
[L449]     evidence_pool[edge.relation_type].extend(edge.evidence)
[L450]
[L451] # Select winning relation type
[L452] winner = max(votes, key=votes.get)
[L453]
[L454] # Log conflict if close contest
[L455] sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L456] if len(sorted_votes) > 1:
[L457]     margin = sorted_votes[0][1] - sorted_votes[1][1]
[L458]     if margin < 0.2: # Close contest
[L459]         log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L460]
[L461] winning_edge = edges[0]
[L462] winning_edge.relation_type = winner
[L463] winning_edge.evidence = evidence_pool[winner]
[L464] winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L465] winning_edge.metadata['conflict_resolved'] = True
[L466] winning_edge.metadata['alternative_relations'] = [
[L467]     (rt, score) for rt, score in sorted_votes if rt != winner
[L468] ]

```

```
[L469] resolved.append(winning_edge)
[L470]
[L471] return resolved
[L472]
```

```
[L473]
[L474] ---
[L475]
[L476] ### 5.5 Layer 5: SHIA Core – Hierarchy Construction
[L477]
[L478] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline
[L479]
[L480] **Input:** `KnowledgeNode[]` + `KnowledgeEdge[]` → **Output:** Knowledge Forest
[L481]
[L482] #### Pipeline Overview
[L483]
[L484]
```

mermaid

```
[L485] graph LR
[L486] N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L487] CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L488] PRE --> HV{"HV<br/>Validate #1?"}
[L489] HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L490] HV -->|"Fail"| HV2{"HV<br/>Validate #2?"}
[L491] HV2 -->|"Pass"| FI
[L492] HV2 -->|"Fail"| HV3{"HV<br/>Validate #3?"}
[L493] HV3 -->|"Pass"| FI
[L494] HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L495] FI --> CLD["CLD<br/>Discover Cross-Links"]
[L496] CLD --> DONE["Node Placed ✓"]
[L497]
```

```
[L498]
[L499] #### 5.5.1 CPG++ – Candidate Parent Generator
[L500]
[L501] Generates up to 5 candidate parents for a new node using a multi-stage filter:
[L502]
[L503]
```

python

```
[L504] def cpg_plus_plus(node_q, forest, config):
[L505] """Generate ≤k candidate parents for node q."""
[L506] # Stage 1: Domain Routing (O(1)) — filter to same-domain nodes
[L507] domain = classify_domain(node_q)
[L508] domain_nodes = forest.get_nodes_by_domain(domain)
[L509]
[L510] # Stage 2: ANN Retrieval (O(log N)) — find semantically similar nodes
[L511] if len(domain_nodes) > config.ann_k:
[L512] ann_results = vector_index.search(node_q.embedding, k=config.
```


[L513] metadata:
[L514] query_result_index: 6
[L515] file_id: file_00000000dcc082119f935683a5e13eed
[L516] version_id: 1
[L517] name: SHIA_RAG_Final_Report(4)(1).md
[L518] mime_type: text/markdown
[L519] surface: conversation
[L520] score: 0.03128054740957967
[L521] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
[L522] snippet:
[L523] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L524]
[L525] ### 2.3 Literature Search Methodology
[L526]
[L527] | Parameter | Details |
[L528] |-----|-----|
[L529] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L530] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L531] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L532] | **Time Range** | 2020–2026 |
[L533] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L534]
[L535] ---
[L536]
[L537] ## 3. System Architecture
[L538]
[L539] ### 3.1 Overview
[L540]
[L541] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:
[L542]
[L543] | Phase | Layers | Mode | Purpose |
[L544] |-----|-----|-----|-----|
[L545] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |
[L546] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |
[L547] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |
[L548]
[L549] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.
[L550]
[L551]

```
[L552] graph TD
[L553]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L554]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L555]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L556]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L557]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L558]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L559]     end
[L560]
[L561]     subgraph "Phase 2: Online – Retrieval & Generation"
[L562]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L563]         KF --> L6
[L564]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L565]         L7 --> A["Answer + Citations"]
[L566]     end
[L567]
[L568]     subgraph "Phase 3: Continuous – Operations"
[L569]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L570]         A --> L8
[L571]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L572]     end
[L573]
```

[L574]

[L575] ### 3.2 Data Flow

[L576]

[L577] ```

[L578] Raw Document (PDF/DOCX/HTML)

[L579] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L580] Normalized Text + Metadata

[L581] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L582] [Page → Region → Block] hierarchy

[L583] ↓ Layer 3: Split sentences,

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: `filecite`turn11file7

[L584] metadata:

[L585] query_result_index: 7

[L586] file_id: file_00000000dcc082119f935683a5e13eed

[L587] version_id: 1

[L588] name: SHIA_RAG_Final_Report(4)(1).md

[L589] mime_type: text/markdown

[L590] surface: conversation

[L591] score: 0.03021353930031804

[L592] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--24

[L593] snippet:

[L594] **Input:** KnowledgeNode[] (with raw relations) → **Output:** KnowledgeEdge[] (validated, scored)

[L595]

[L596] **Pipeline:**

[L597] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L598] 2. **Resolve target node** — map target concept name to a KnowledgeNode

[L599] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L600] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L601] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L602] 6. **Resolve cross-document conflicts** — when multiple documents disagree on a relation

[L603]

[L604]

```
[L605] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L606]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L607]     conf = 0.5
[L608]     conf += 0.15 * source.confidence
[L609]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L610]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L611]     return min(conf, 1.0)
[L612]
```

[L613]

[L614] ##### Cross-Document Conflict Resolution

[L615]

[L616] When multiple documents provide **conflicting relations** for the same concept pair (e.g., Document A says "Round Robin IS_A Scheduling" but Document B says "Round Robin PART_OF OS"), the system resolves conflicts using evidence-weighted voting:

[L617]

[L618] ```python

[L619] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:

[L620] """Resolve conflicting relations from different documents.

[L621]

[L622] Strategy: Evidence-weighted voting

[L623] 1. Group edges by (source_node, target_node) pair

[L624] 2. For each group, tally votes per relation_type weighted by evidence confidence

[L625] 3. Select the relation_type with highest weighted vote

[L626] 4. Log conflicts for manual review

[L627] """

[L628] from collections import defaultdict

[L629]

[L630] # Group by node pair

[L631] pair_groups = defaultdict(list)

[L632] for edge in candidate_edges:

[L633] key = (edge.source_node, edge.target_node)

[L634] pair_groups[key].append(edge)

[L635]

[L636] resolved = []

[L637] for (src, tgt), edges in pair_groups.items():

[L638] if len(edges) == 1:

[L639] resolved.append(edges[0])

[L640] continue

[L641]

[L642] # Tally weighted votes per relation type

[L643] votes = defaultdict(float)

[L644] evidence_pool = defaultdict(list)

[L645] for edge in edges:

[L646] weight = edge.confidence * len(edge.evidence)

[L647] votes[edge.relation_type] += weight

[L648] evidence_pool[edge.relation_type].extend(edge.evidence)

[L649]

```

[L650] # Select winning relation type
[L651] winner = max(votes, key=votes.get)
[L652]
[L653] # Log conflict if close contest
[L654] sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L655] if len(sorted_votes) > 1:
[L656] margin = sorted_votes[0][1] - sorted_votes[1][1]
[L657] if margin < 0.2: # Close contest
[L658] log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L659]
[L660] winning_edge = edges[0]
[L661] winning_edge.relation_type = winner
[L662] winning_edge.evidence = evidence_pool[winner]
[L663] winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L664] winning_edge.metadata['conflict_resolved'] = True
[L665] winning_edge.metadata['alternative_relations'] = [
[L666] (rt, score) for rt, score in sorted_votes if rt != winner
[L667] ]
[L668] resolved.

```

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: `filecite`turn11file8

```

[L669] metadata:
[L670] query_result_index: 8
[L671] file_id: file_00000000dcc082119f935683a5e13eed
[L672] version_id: 1
[L673] name: SHIA_RAG_Final_Report(4)(1).md
[L674] mime_type: text/markdown
[L675] surface: conversation
[L676] score: 0.029910714285714284
[L677] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--19
[L678] snippet:
[L679] Uses pdfplumber for text-based tables, Camelot for image-based.
[L680] Returns list of rows, each row is a list of cell strings.
[L681] try:
[L682] # Primary: pdfplumber table extraction
[L683] table = page.extract_table(table_region)
[L684] if table and len(table) > 1: # At least header + 1 data row
[L685] return table
[L686] except Exception:
[L687] pass
[L688]
[L689] # Fallback: Camelot for image-based tables
[L690] try:
[L691] import camelot
[L692] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L693] if tables:
[L694] return tables[0].df.values.tolist()
[L695] except Exception:

```

```
[L696] pass
[L697]
[L698] return None # Table structure could not be parsed
[L699]
```

```
[L700]
[L701] ##### Table Detection Fallback
[L702]
[L703] If table detection confidence is low, the block is flagged:
[L704]
[L705]
```

```
python
[L706] def classify_block_with_fallback(text, page) -> tuple:
[L707] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L708] block_type, confidence = classify_block(text)
[L709]
[L710] if block_type == BlockType.TABLE and confidence < 0.6:
[L711] # Attempt cell-level parsing to confirm
[L712] cells = parse_table_cells(page, text)
[L713] if cells is None:
[L714] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L715] else:
[L716] return BlockType.TABLE, 0.8 # Confirmed by parser
[L717]
[L718] return block_type, confidence
[L719]
```

```
[L720]
[L721] ##### Reading Order Validation
[L722]
[L723] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L724]
[L725]
```

```
python
[L726] def assign_reading_order(blocks: list) -> list:
[L727] """Assign global reading order with confidence.
[L728]
[L729] Multi-column layouts can produce incorrect reading order.
[L730] We detect potential issues and flag low confidence."""
[L731] # Sort by page, then y-coordinate, then x-coordinate
[L732] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L733]
[L734] for i, block in enumerate(sorted_blocks):
[L735] block.reading_order = i
[L736]
[L737] # Detect potential multi-column issues
[L738] if i > 0:
[L739] prev = sorted_blocks[i - 1]
[L740] if (prev.page_number == block.page_number and
[L741] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L742] prev.x_pos > block.x_pos): # But previous is to the right
[L743] block.confidence *= 0.7 # Reduce confidence — possible column issue
```

[L744]

[L745] return sorted_blocks

[L746]

```
[L747]
[L748] ---
[L749]
[L750] ### 5.3 Layer 3: Knowledge Extraction
[L751]
[L752] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L753]
[L754] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L755]
[L756] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L757]
[L758] ##### Extraction Prompt
[L759]
[L760]
```

[L761] You are a knowledge extraction system. Given a text block, extract:

[L762] 1. Concepts: Named entities, technical terms

[L763] 2. Definitions: If the text defines any concept, extract the definition

[L764] 3. Properties: Key attributes with types and values

[L765] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L766] 5. Domain: Subject area (e.g., "Operating Systems")

DEEP RESEARCH 24/08/2026, 00:22:44

Citation Marker: `filecite`turn11file9

[L767] metadata:

[L768] query_result_index: 9

[L769] file_id: file_00000000dcc082119f935683a5e13eed

[L770] version_id: 1

[L771] name: SHIA_RAG_Final_Report(4)(1).md

[L772] mime_type: text/markdown

[L773] surface: conversation

[L774] score: 0.029551337359792925

[L775] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--35

[L776] snippet:

[L777] ##### Prompt Template

[L778]

[L779]

```
[L780] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L781] RULES:
[L782] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L783] 2. For each claim, cite the source using [NODE_ID] format.
[L784] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L785] 4. If evidence conflicts, present both views with citations.
[L786]
[L787] REQUIRED OUTPUT FORMAT:
[L788] {
[L789]   "answer": "Your answer with [NODE_ID] citations inline.",
[L790]   "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L791]   "overall_confidence": 0.0-1.0,
[L792]   "insufficient_evidence": false
[L793] }"""
[L794]
```

[L795]

[L796] ##### Claim Verification

[L797]

[L798] After the LLM generates an answer, each claim is verified against its cited evidence:

[L799]

[L800]

```
[L801] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L802]     """Returns (status, confidence).
[L803]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L804]     - PARTIAL:       evidence partially supports (0.4 < confidence ≤ 0.7)
[L805]     - UNSUPPORTED:   no evidence supports (confidence ≤ 0.4)
[L806]     - CONFLICTING:   evidence items contradict each other
[L807]     """
[L808]     scores = []
[L809]     for eid in evidence_ids:
[L810]         node = node_map.get(eid)
[L811]         if node and node.evidence:
[L812]             evidence_text = ' '.join(e.text for e in node.evidence)
[L813]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L814]
[L815]     if not scores:
[L816]         return 'UNSUPPORTED', 0.0
[L817]
[L818]     avg_score = sum(scores) / len(scores)
[L819]
[L820]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L821]         return 'CONFLICTING', avg_score
[L822]     if avg_score > 0.7:
[L823]         return 'SUPPORTED', avg_score
[L824]     elif avg_score > 0.4:
[L825]         return 'PARTIAL', avg_score
[L826]     else:
[L827]         return 'UNSUPPORTED', avg_score
[L828]
```

[L829]

[L830] ##### Unsupported Claim Policy

[L831]

[L832] | Status | Action |

[L833] |-----|-----|

[L834] | SUPPORTED | Keep as-is with citation |

[L835] | PARTIAL | Keep but mark as partially supported |

[L836] | UNSUPPORTED | Remove from answer, note at end |

[L837] | CONFLICTING | Present both views with note |

[L838]

[L839] ##### Answer Confidence Formula

[L840]

[L841]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{node_conf}}{\text{selected nodes}} + 0.1 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L842]

[L843] ##### Output Schema

[L844]

[L845]

```
[L846] @dataclass
[L847] class GenerationResult:
[L848]     """Output of Layer 7 – ready for user and Layer 8."""
[L849]     answer: str                # Answer text with citations
[L850]     claims: list              # List of VerifiedClaim
[L851]     citations: list           # Formatted citation list
[L852]     overall_confidence: float # 0.0-1.0
[L853]     retrieval_result: object  # Preserved for Layer 8
[L854]     generation_metadata: dict # Model used, claim counts, etc.
[L855]
```

[L856]

[L857] ---

[L858]

[L859] ## 7. Operations Layer — Layer 8

[L860]

[L861] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L862]

[L863] ### 7.1 Logging

[L864]

[L865] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite`turn12file0

[L1] metadata:

[L2] query_result_index: 0

[L3] file_id: file_00000000dcc082119f935683a5e13eed

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4)(1).md

[L6] mime_type: text/markdown

[L7] surface: conversation

[L8] score: 0.030798389007344232

[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--0

[L10] snippet:

[L11] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L12]

[L13] > **Project Type:** Minor Project (M.Tech)

[L14] > **Date:** August 2026

[L15] > **Author:** [Your Name]

[L16]

[L17] ---

[L18]

[L19] ## Abstract

[L20]

[L21] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L22]

[L23] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L24]

[L25] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L26]

[L27] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L28]

[L29] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L30]

[L31] ---

[L32]

[L33] ## Table of Contents

[L34]

[L35] 1. Introduction

[L36] 2. Literature Review

[L37] 3. System Architecture

[L38] 4. Data Model — Layer 0

[L39] 5. Offline Pipeline — Layers 1-5

[L40] 6. Online Pipeline — Layers 6-7

[L41] 7. Operations Layer — Layer 8

[L42] 8. Novel Contributions

[L43] 9. Mathematical Framework

[L44] 10. Implementation

[L45] 11. Evaluation Framework

[L46] 12. Risk Analysis

[L47] 13. Conclusion & Future Work

[L48]

[L49] ---

[L50]

[L51] ## 1. Introduction

[L52]

[L53] ### 1.1 Problem Statement

[L54]

[L55] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L56]

[L57] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L58] 2. Embed each chunk into a vector space

[L59] 3. At query time, retrieve the top-k most similar chunks

[L60] 4. Feed these chunks as context to an LLM for answer generation

[L61]

[L62] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite[turn12file1]`

[L63] metadata:

[L64] query_result_index: 1

[L65] file_id: file_00000000dcc082119f935683a5e13eed

[L66] version_id: 1

[L67] name: SHIA_RAG_Final_Report(4)(1).md

[L68] mime_type: text/markdown

[L69] surface: conversation

[L70] score: 0.03055037313432836

[L71] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--23

[L72] snippet:

[L73] name.lower() for u in group]

[L74] has_antonym = False

[L75] for i, n1 in enumerate(names):

[L76] for n2 in names[i+1:]:

[L77] # Check 1: Negation prefix

[L78] if NEGATION_PREFIXES.sub(", n1) == NEGATION_PREFIXES.sub(", n2):

[L79] has_antonym = True

[L80] # Check 2: Known CONTRASTS_WITH relation

[L81] if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':

[L82] has_antonym = True

[L83]

[L84] if has_antonym:

[L85] safe_groups.extend([[u] for u in group]) # Don't merge

[L86] else:

[L87] safe_groups.append(group)

[L88]

[L89] return [create_canonical_node(group) for group in safe_groups]

[L90]

[L91]

[L92] ### Embedding Strategy on Merge

[L93]

[L94] When nodes are merged during alias resolution, embeddings are handled as follows:

[L95] - **Do NOT average embeddings** – averaging collapses distinct semantic information

[L96] - **Re-embed the merged node** using the canonical name + merged definitions

[L97] - This ensures the embedding accurately represents the merged concept

[L98]

[L99]

python

```
[L100] def merge_node_embeddings(merged_node, embed_fn):
[L101] """Re-compute embedding for merged node instead of averaging.
[L102]
[L103] Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L104] produce a vector that represents neither concept well.
[L105] Instead, embed the canonical representation."""
[L106] text_to_embed = merged_node.canonical_name
[L107] if merged_node.definitions:
[L108] text_to_embed += " " + merged_node.definitions[0]
[L109] merged_node.embedding = embed_fn(text_to_embed)
[L110]
```

```
[L111]
[L112] ---
[L113]
[L114] ### 5.4 Layer 4: Relation Management
[L115]
[L116] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence
[L117]
[L118] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)
[L119]
[L120] **Pipeline:**
[L121] 1. **Validate relation type** – check against ontology; if unknown type, attempt inference via LLM
[L122] 2. **Resolve target node** – map target concept name to a `KnowledgeNode`
[L123] 3. **Check for duplicates** – merge evidence into existing edge if duplicate
[L124] 4. **Determine edge class** – hierarchical or semantic based on ontology mapping
[L125] 5. **Calculate confidence** – weighted combination of source confidence, evidence length, and semantic simila
[L126] 6. **Resolve cross-document conflicts** – when multiple documents disagree on a relation
[L127]
[L128]
```

python

```
[L129] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L130] """Confidence = 0.5 (base) + 0.15 source_conf + 0.10 evidence_quality + 0.10 sim"""
[L131] conf = 0.5
[L132] conf += 0.15 source.confidence
[L133] conf += 0.10 min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L134] conf += 0.10 cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L135] return min(conf, 1.0)
[L136]
```

```
[L137]
[L138] ### Cross-Document Conflict Resolution
[L139]
[L140] When multiple documents provide conflicting relations for the same concept pair (e.g., Document A says "R
[L141]
[L142]
```

python

```
[L143] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:
[L144] """Resolve conflicting relations from different documents.
[L145]
[L146] Strategy: Evidence-weighted voting
[L147] 1. Group edges by (source_node, target_node) pair
[L148] 2.
```

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite`turn12file2

[L149] metadata:

[L150] query_result_index: 2

[L151] file_id: file_00000000dcc082119f935683a5e13eed

[L152] version_id: 1

[L153] name: SHIA_RAG_Final_Report(4)(1).md

[L154] mime_type: text/markdown

[L155] surface: conversation

[L156] score: 0.03047794966520434

[L157] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--22

[L158] snippet:

[L159] ##### Alias Resolution (with Antonym Protection)

[L160]

[L161] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them — but must **avoid merging antonyms** (e.g., "preemptive" and "non-preemptive" have high similarity but are distinct concepts).

[L162]

[L163] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L164] 2. **Antonym check:** Before merging, verify candidates do not have a `CONTRASTS_WITH` relation

[L165] 3. Find connected components of near-duplicate units

[L166] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L167]

[L168]

```

[L169] def alias_resolution(units: list, relation_cache: dict) -> list:
[L170]     """Merge KnowledgeUnits referring to the same concept.
[L171]
[L172]     Merge strategy:
[L173]     - Canonical name: most frequent name across units
[L174]     - Aliases: union of all names and aliases
[L175]     - Definitions: union of all definitions
[L176]     - Confidence: average of unit extraction confidences
[L177]     - Properties: merged (latest value wins, conflicts logged)
[L178]
[L179]     Antonym protection:
[L180]     - Before merging, check if candidates have CONTRASTS_WITH relation
[L181]     - Also check for negation prefixes (non-, un-, in-, dis-)
[L182]     """
[L183]     import re
[L184]     NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L185]
[L186]     lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L187]     # ... build LSH index from concept names + aliases ...
[L188]
[L189]     raw_groups = find_connected_components(units, lsh)
[L190]
[L191]     # Antonym protection: split groups containing antonym pairs
[L192]     safe_groups = []
[L193]     for group in raw_groups:
[L194]         names = [u.name.lower() for u in group]
[L195]         has_antonym = False
[L196]         for i, n1 in enumerate(names):
[L197]             for n2 in names[i+1:]:
[L198]                 # Check 1: Negation prefix
[L199]                 if NEGATION_PREFIXES.sub('', n1) == NEGATION_PREFIXES.sub('', n2):
[L200]                     has_antonym = True
[L201]                 # Check 2: Known CONTRASTS_WITH relation
[L202]                 if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L203]                     has_antonym = True
[L204]
[L205]         if has_antonym:
[L206]             safe_groups.extend([u for u in group]) # Don't merge
[L207]         else:
[L208]             safe_groups.append(group)
[L209]
[L210]     return [create_canonical_node(group) for group in safe_groups]
[L211]

```

[L212]

[L213] ##### Embedding Strategy on Merge

[L214]

[L215] When nodes are merged during alias resolution, embeddings are handled as follows:

[L216] - **Do NOT average embeddings** — averaging collapses distinct semantic information

[L217] - **Re-embed the merged node** using the canonical name + merged definitions

[L218] - This ensures the embedding accurately represents the merged concept

[L219]

[L220]

```

[L221] def merge_node_embeddings(merged_node, embed_fn):
[L222]     """Re-compute embedding for merged node instead of averaging.
[L223]
[L224]     Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L225]     produce a vector that represents neither concept well.
[L226]     Instead, embed the canonical representation."""
[L227]     text_to_embed = merged_node.canonical_name
[L228]     if merged_node.definitions:
[L229]         text_to_embed += " " + merged_node.definitions[0]
[L230]     merged_node.embedding = embed_fn(text_to_embed)
[L231]

```

[L232]

[L233] ---

[L234]

[L235] ### 5.4 Layer 4: Relation Management

[L236]

[L237] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve cross-document conflicts, and produce final `KnowledgeEdge` objects.

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `[]filecite[]turn12file3[]`

[L238] metadata:

[L239] query_result_index: 3

[L240] file_id: file_00000000dcc082119f935683a5e13eed

[L241] version_id: 1

[L242] name: SHIA_RAG_Final_Report(4)(1).md

[L243] mime_type: text/markdown

[L244] surface: conversation

[L245] score: 0.0304147465437788

[L246] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--35

[L247] snippet:

[L248] ##### Prompt Template

[L249]

[L250]

[L251] SYSTEM_PROMPT = ""You are a knowledge-grounded question answering system.

[L252] RULES:

[L253] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.

[L254] 2. For each claim, cite the source using [NODE_ID] format.

[L255] 3. If evidence is insufficient, say "Insufficient evidence to answer."

[L256] 4. If evidence conflicts, present both views with citations.

[L257]

[L258] REQUIRED OUTPUT FORMAT:

[L259] {

[L260] "answer": "Your answer with [NODE_ID] citations inline.",

[L261] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],

[L262] "overall_confidence": 0.0-1.0,

[L263] "insufficient_evidence": false

[L264] }""

[L265]

[L266]

[L267] ##### Claim Verification

[L268]

[L269] After the LLM generates an answer, each claim is verified against its cited evidence:

[L270]

[L271]

```

[L272] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L273]     """Returns (status, confidence).
[L274]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L275]     - PARTIAL:      evidence partially supports (0.4 < confidence ≤ 0.7)
[L276]     - UNSUPPORTED:  no evidence supports (confidence ≤ 0.4)
[L277]     - CONFLICTING:  evidence items contradict each other
[L278]     """
[L279]     scores = []
[L280]     for eid in evidence_ids:
[L281]         node = node_map.get(eid)
[L282]         if node and node.evidence:
[L283]             evidence_text = ' '.join(e.text for e in node.evidence)
[L284]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L285]
[L286]     if not scores:
[L287]         return 'UNSUPPORTED', 0.0
[L288]
[L289]     avg_score = sum(scores) / len(scores)
[L290]
[L291]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L292]         return 'CONFLICTING', avg_score
[L293]     if avg_score > 0.7:
[L294]         return 'SUPPORTED', avg_score
[L295]     elif avg_score > 0.4:
[L296]         return 'PARTIAL', avg_score
[L297]     else:
[L298]         return 'UNSUPPORTED', avg_score
[L299]

```

[L300]

[L301] ##### Unsupported Claim Policy

[L302]

[L303] | Status | Action |

[L304] |-----|-----|

[L305] | SUPPORTED | Keep as-is with citation |

[L306] | PARTIAL | Keep but mark as partially supported |

[L307] | UNSUPPORTED | Remove from answer, note at end |

[L308] | CONFLICTING | Present both views with note |

[L309]

[L310] ##### Answer Confidence Formula

[L311]

[L312]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{selected nodes}}{\text{cited nodes}}$$

[L313]

[L314] ##### Output Schema

[L315]

[L316]

```

[L317] @dataclass
[L318] class GenerationResult:
[L319]     """Output of Layer 7 – ready for user and Layer 8."""
[L320]     answer: str                # Answer text with citations
[L321]     claims: list              # List of VerifiedClaim
[L322]     citations: list           # Formatted citation list
[L323]     overall_confidence: float # 0.0-1.0
[L324]     retrieval_result: object  # Preserved for Layer 8
[L325]     generation_metadata: dict # Model used, claim counts, etc.
[L326]

```

[L327]

[L328] ---

[L329]

[L330] ## 7. Operations Layer — Layer 8

[L331]

[L332] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L333]

[L334] ### 7.1 Logging

[L335]

[L336] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite`turn12file4

[L337] metadata:

[L338] query_result_index: 4

[L339] file_id: file_00000000dcc082119f935683a5e13eed

[L340] version_id: 1

[L341] name: SHIA_RAG_Final_Report(4)(1).md

[L342] mime_type: text/markdown

[L343] surface: conversation

[L344] score: 0.03007688828584351

[L345] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--31

[L346] snippet:

[L347] ##### 5.5.5 CLD — Cross-Link Discoverer

[L348]

[L349] After hierarchical placement, discover semantic (non-hierarchical) links:

[L350]

[L351]

```
[L352] def cld_discover(node, forest, config):
[L353]     """Find semantic cross-links for a newly placed node."""
[L354]     cross_links = []
[L355]     neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L356]
[L357]     for neighbor in neighbors:
[L358]         if neighbor.id == node.id or neighbor.id == node.parent:
[L359]             continue
[L360]
[L361]         sim = cosine_similarity(node.embedding, neighbor.embedding)
[L362]         if sim > config.cross_link_threshold:
[L363]             relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L364]             if relation_type:
[L365]                 forest.add_edge(node.id, neighbor.id,
[L366]                                 edge_class='SEMANTIC', relation_type=relation_type,
[L367]                                 confidence=sim, weight=1.0)
[L368]                 cross_links.append((neighbor.id, relation_type, sim))
[L369]
[L370]     return cross_links
[L371]
```

[L372]

[L373] ---

[L374]

[L375] ## 6. Online Pipeline — Layers 6-7

[L376]

[L377] ### 6.1 Layer 6: Retrieval Engine

[L378]

[L379] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L380]

[L381] **Input:** User query string → **Output:** RetrievalResult

[L382]

[L383] #### Query Complexity Classification

[L384]

[L385] | Intent | Example | Complexity | Traversal Depth |

[L386] |-----|-----|-----|-----|

[L387] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |

[L388] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |

[L389] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |

[L390] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |

[L391] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |

[L392]

[L393] #### Retrieval Pipeline

[L394]

[L395] ```python

[L396] @dataclass

[L397] class RetrievalResult:

[L398] """Output of Layer 6 — input to Layer 7."""

[L399] selected_nodes: list # Nodes selected by Tree Knapsack

[L400] context_text: str # Assembled context string

[L401] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)

[L402] query_complexity: str # SIMPLE, MODERATE, COMPLEX

[L403] query_intent: str # DEFINITION, COMPARISON, etc.

[L404] seed_node_ids: list # Initial seed nodes from vector search

[L405] token_count: int # Estimated tokens in context

[L406]

[L407] class RetrievalEngine:

[L408] def __init__(self, forest, vector_index, cache_manager, config):

[L409] self.forest = forest

[L410] self.vector_index = vector_index

[L411] self.cache = cache_manager

[L412] self.config = config

[L413]

[L414] def retrieve(self, query) -> RetrievalResult:

[L415] # Step 0: Cache check

[L416] cached = self.cache.get_query_result(hash_query(query))

[L417] if cached:

[L418] return cached

[L419]

[L420] # Step 1: Classify query intent and complexity

[L421] intent = detect_intent(query)

[L422] complexity = classify_complexity(intent)

[L423]

```
[L424] # Step 2: Embed query and find seed nodes
[L425] query_emb = embed_query(query) # Asymmetric embedding model
[L426] seed_nodes = self.vector_index.search(query_emb, k=5)
[L427]
[L428] # Step 3: Adaptive depth traversal
[L429] depth_config = {
[L430] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L431] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L432] 'COMPLEX': {'max_depth': 3, 'parents': True,
```

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `[[filecite]]turn12file5[[`

```
[L433] metadata:
[L434] query_result_index: 5
[L435] file_id: file_00000000dcc082119f935683a5e13eed
[L436] version_id: 1
[L437] name: SHIA_RAG_Final_Report(4)(1).md
[L438] mime_type: text/markdown
[L439] surface: conversation
[L440] score: 0.02946236559139785
[L441] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--3
[L442] snippet:
[L443] > Key Insight: Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.
[L444]
[L445] ### 1.3 Objectives
[L446]
[L447] 1. Design a complete 8-layer architecture for hierarchical RAG
[L448] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)
[L449] 3. Implement a self-evolving feedback loop that improves retrieval quality over time
[L450] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)
[L451] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality
[L452] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans
[L453]
[L454] ### 1.4 Contributions
[L455]
[L456] This work makes five novel contributions to the RAG literature:
[L457]
[L458] | # | Contribution | Novelty Level |
[L459] |---|-----|-----|
[L460] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |
[L461] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |
[L462] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |
[L463] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |
[L464] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |
[L465]
[L466] ---
[L467]
[L468] ## 2. Literature Review
```

[L469]

[L470] ### 2.1 Related Work

[L471]

[L472] ##### 2.1.1 Flat-Chunk RAG Systems

[L473]

[L474] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L475]

[L476] ##### 2.1.2 Structured RAG Systems

[L477]

[L478] Recent work has begun exploring structured retrieval:

[L479]

[L480] | System | Approach | Key Limitation |

[L481] |-----|-----|-----|

[L482] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L483] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L484] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L485] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L486]

[L487] ##### 2.1.3 Knowledge Graph Construction

[L488]

[L489] Ontology learning and taxonomy induction have a rich history in NLP:

[L490] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L491] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L492] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L493]

[L494] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L495]

[L496] ### 2.2 Research Gap

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite[turn12file6]`

[L497] metadata:

[L498] query_result_index: 6

[L499] file_id: file_00000000dcc082119f935683a5e13eed

[L500] version_id: 1

[L501] name: SHIA_RAG_Final_Report(4)(1).md

[L502] mime_type: text/markdown

[L503] surface: conversation

[L504] score: 0.02919863597612958

[L505] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5

[L506] snippet:

[L507] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3)

hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L508]

[L509] ### 2.3 Literature Search Methodology

[L510]

[L511] | Parameter | Details |

[L512] |-----|-----|

[L513] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L514] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L515] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L516] | **Time Range** | 2020–2026 |

[L517] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L518]

[L519] ---

[L520]

[L521] ## 3. System Architecture

[L522]

[L523] ### 3.1 Overview

[L524]

[L525] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L526]

[L527] | Phase | Layers | Mode | Purpose |

[L528] |-----|-----|-----|-----|

[L529] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L530] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L531] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L532]

[L533] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L534]

[L535]

[L536] graph TD

[L537] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"

[L538] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Parsing
Text chunking, embedding"]

[L539] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]

[L540] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]

[L541] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]

[L542] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]

[L543] end

[L544]

[L545] subgraph "Phase 2: Online – Retrieval & Generation"

[L546] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]

[L547] KF --> L6

[L548] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]

[L549] L7 --> A["Answer + Citations"]

[L550] end

[L551]

[L552] subgraph "Phase 3: Continuous – Operations"

[L553] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]

[L554] A --> L8

[L555] L8 -->| "Self-Evolution
Feedback" | KF

[L556] end

[L557]

[L558]

[L559] ### 3.2 Data Flow

[L560]

[L561] ```

[L562] Raw Document (PDF/DOCX/HTML)

[L563] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L564] Normalized Text + Metadata

[L565] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L566] [Page → Region → Block] hierarchy

[L567] ↓ Layer 3: Split sentences,

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: [filecite]turn12file7[

[L568] metadata:

[L569] query_result_index: 7

[L570] file_id: file_00000000dcc082119f935683a5e13eed

[L571] version_id: 1

[L572] name: SHIA_RAG_Final_Report(4)(1).md

[L573] mime_type: text/markdown

[L574] surface: conversation

[L575] score: 0.028790389395194696

[L576] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--19

[L577] snippet:

[L578] Uses pdfplumber for text-based tables, Camelot for image-based.

[L579] Returns list of rows, each row is a list of cell strings. """

[L580] try:

[L581] # Primary: pdfplumber table extraction

[L582] table = page.extract_table(table_region)

[L583] if table and len(table) > 1: # At least header + 1 data row

[L584] return table

[L585] except Exception:

[L586] pass

[L587]

[L588] # Fallback: Camelot for image-based tables

[L589] try:

[L590] import camelot

[L591] tables = camelot.read_pdf(page.source_path, pages=str(page.number))

[L592] if tables:

[L593] return tables[0].df.values.tolist()

[L594] except Exception:

[L595] pass

[L596]

[L597] return None # Table structure could not be parsed

[L598]

```
[L599]
[L600] ##### Table Detection Fallback
[L601]
[L602] If table detection confidence is low, the block is flagged:
[L603]
[L604]
```

python

```
[L605] def classify_block_with_fallback(text, page) -> tuple:
[L606] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L607] block_type, confidence = classify_block(text)
[L608]
[L609] if block_type == BlockType.TABLE and confidence < 0.6:
[L610] # Attempt cell-level parsing to confirm
[L611] cells = parse_table_cells(page, text)
[L612] if cells is None:
[L613] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L614] else:
[L615] return BlockType.TABLE, 0.8 # Confirmed by parser
[L616]
[L617] return block_type, confidence
[L618]
```

```
[L619]
[L620] ##### Reading Order Validation
[L621]
[L622] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L623]
[L624]
```

python

```
[L625] def assign_reading_order(blocks: list) -> list:
[L626] """Assign global reading order with confidence.
[L627]
[L628] Multi-column layouts can produce incorrect reading order.
[L629] We detect potential issues and flag low confidence."""
[L630] # Sort by page, then y-coordinate, then x-coordinate
[L631] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L632]
[L633] for i, block in enumerate(sorted_blocks):
[L634] block.reading_order = i
[L635]
[L636] # Detect potential multi-column issues
[L637] if i > 0:
[L638] prev = sorted_blocks[i - 1]
[L639] if (prev.page_number == block.page_number and
[L640] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L641] prev.x_pos > block.x_pos): # But previous is to the right
[L642] block.confidence *= 0.7 # Reduce confidence — possible column issue
[L643]
[L644] return sorted_blocks
[L645]
```

```
[L646]
[L647] ---
[L648]
[L649] ### 5.3 Layer 3: Knowledge Extraction
[L650]
[L651] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L652]
[L653] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L654]
[L655] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L656]
[L657] #### Extraction Prompt
[L658]
[L659]
```

[L660] You are a knowledge extraction system. Given a text block, extract:

[L661] 1. Concepts: Named entities, technical terms

[L662] 2. Definitions: If the text defines any concept, extract the definition

[L663] 3. Properties: Key attributes with types and values

[L664] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L665] 5. Domain: Subject area (e.g., "Operating Systems")

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite`turn12file8

[L666] metadata:

[L667] query_result_index: 8

[L668] file_id: file_00000000dcc082119f935683a5e13eed

[L669] version_id: 1

[L670] name: SHIA_RAG_Final_Report(4)(1).md

[L671] mime_type: text/markdown

[L672] surface: conversation

[L673] score: 0.028782894736842105

[L674] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--24

[L675] snippet:

[L676] **Input:** KnowledgeNode[] (with raw relations) → **Output:** KnowledgeEdge[] (validated, scored)

[L677]

[L678] **Pipeline:**

[L679] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L680] 2. **Resolve target node** — map target concept name to a KnowledgeNode

[L681] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L682] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L683] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L684] 6. **Resolve cross-document conflicts** — when multiple documents disagree on a relation

[L685]

[L686]

```
[L687] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L688]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L689]     conf = 0.5
[L690]     conf += 0.15 * source.confidence
[L691]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L692]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L693]     return min(conf, 1.0)
[L694]
```

[L695]

[L696] ##### Cross-Document Conflict Resolution

[L697]

[L698] When multiple documents provide **conflicting relations** for the same concept pair (e.g., Document A says "Round Robin IS_A Scheduling" but Document B says "Round Robin PART_OF OS"), the system resolves conflicts using evidence-weighted voting:

[L699]

[L700] ```python

[L701] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:

[L702] """Resolve conflicting relations from different documents.

[L703]

[L704] Strategy: Evidence-weighted voting

[L705] 1. Group edges by (source_node, target_node) pair

[L706] 2. For each group, tally votes per relation_type weighted by evidence confidence

[L707] 3. Select the relation_type with highest weighted vote

[L708] 4. Log conflicts for manual review

[L709] """

[L710] from collections import defaultdict

[L711]

[L712] # Group by node pair

[L713] pair_groups = defaultdict(list)

[L714] for edge in candidate_edges:

[L715] key = (edge.source_node, edge.target_node)

[L716] pair_groups[key].append(edge)

[L717]

[L718] resolved = []

[L719] for (src, tgt), edges in pair_groups.items():

[L720] if len(edges) == 1:

[L721] resolved.append(edges[0])

[L722] continue

[L723]

[L724] # Tally weighted votes per relation type

[L725] votes = defaultdict(float)

[L726] evidence_pool = defaultdict(list)

[L727] for edge in edges:

[L728] weight = edge.confidence * len(edge.evidence)

[L729] votes[edge.relation_type] += weight

[L730] evidence_pool[edge.relation_type].extend(edge.evidence)

[L731]

[L732] # Select winning relation type

[L733] winner = max(votes, key=votes.get)

[L734]


```
[L735] # Log conflict if close contest
[L736] sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L737] if len(sorted_votes) > 1:
[L738]     margin = sorted_votes[0][1] - sorted_votes[1][1]
[L739]     if margin < 0.2: # Close contest
[L740]         log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L741]
[L742]     winning_edge = edges[0]
[L743]     winning_edge.relation_type = winner
[L744]     winning_edge.evidence = evidence_pool[winner]
[L745]     winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L746]     winning_edge.metadata['conflict_resolved'] = True
[L747]     winning_edge.metadata['alternative_relations'] = [
[L748]         (rt, score) for rt, score in sorted_votes if rt != winner
[L749]     ]
[L750]     resolved.
```

DEEP RESEARCH 24/08/2026, 00:22:48

Citation Marker: `filecite`turn12file9

```
[L751] metadata:
[L752] query_result_index: 9
[L753] file_id: file_00000000dcc082119f935683a5e13eed
[L754] version_id: 1
[L755] name: SHIA_RAG_Final_Report(4)(1).md
[L756] mime_type: text/markdown
[L757] surface: conversation
[L758] score: 0.028782894736842105
[L759] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--54
[L760] snippet:
[L761] 1. Precedence-Constrained Tree Knapsack — to the best of our knowledge, the first formulation of RAG context
selection as a precedence-constrained optimization problem
[L762] 2. Self-Evolving Knowledge Forest — to the best of our knowledge, the first closed-loop feedback mechanism that
modifies RAG knowledge structures based on retrieval success/failure
[L763] 3. Multi-Objective Energy Minimization — a formal optimization framework for automated hierarchy induction
combining semantic similarity, depth, confidence, and type coherence
[L764] 4. Adaptive Depth Traversal — query-complexity-aware retrieval depth adaptation for hierarchical RAG
[L765] 5. SHEF Evaluation Framework — to the best of our knowledge, the first benchmark specifically designed for
hierarchical RAG quality
[L766]
[L767] The complete system specification covers:
[L768] - 9 layers (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
[L769] - 12 defined functions with complete implementations
[L770] - 12 relation types across 4 hierarchical and 8 semantic categories
[L771] - 8 schemas covering all data objects from document blocks to query logs
[L772] - A 6-month implementation roadmap with technology stack, folder structure, and CI/CD pipeline
[L773] - A comprehensive evaluation plan with datasets, baselines, ablation studies, and statistical significance testing
[L774]
[L775] ### 13.2 Key Novelties
[L776]
```

[L777] | # | Novelty | Level | Why It's New |
[L778] | ---|-----|-----|-----|
[L779] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |
[L780] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |
[L781] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |
[L782] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |
[L783] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |
[L784]
[L785] ### 13.3 Next Steps
[L786]
[L787] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)
[L788] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)
[L789] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)
[L790] 4. **Build Layer 3** (Knowledge extraction with LLM)
[L791] 5. **Build Layer 4** (Relation management with ontology validation)
[L792] 6. **Build Layer 5** (SHIA Core with algorithms)
[L793] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
[L794] 8. **Build Layer 7** (Generation with verification)
[L795] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
[L796] 10. **Run first evaluation** on OS textbook dataset using SHEF
[L797] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
[L798] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
[L799]
[L800] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file0

[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.029726775956284153
[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--54
[L10] snippet:
[L11] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem
[L12] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
[L13] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence
[L14] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG
[L15] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for

hierarchical RAG quality

[L16]

[L17] The complete system specification covers:

[L18] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L19] - **12 defined functions** with complete implementations

[L20] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L21] - **8 schemas** covering all data objects from document blocks to query logs

[L22] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L23] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L24]

[L25] ### 13.2 Key Novelties

[L26]

[L27] | # | Novelty | Level | Why It's New |

[L28] |---|-----|-----|-----|

[L29] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L30] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L31] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L32] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L33] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L34]

[L35] ### 13.3 Next Steps

[L36]

[L37] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L38] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L39] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L40] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L41] 5. **Build Layer 4** (Relation management with ontology validation)

[L42] 6. **Build Layer 5** (SHIA Core with algorithms)

[L43] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L44] 8. **Build Layer 7** (Generation with verification)

[L45] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L46] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L47] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L48] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L49]

[L50] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file1

[L51] metadata:

[L52] query_result_index: 1

[L53] file_id: file_00000000dcc082119f935683a5e13eed

[L54] version_id: 1

[L55] name: SHIA_RAG_Final_Report(4)(1).md

[L56] mime_type: text/markdown

```
[L57] surface: conversation
[L58] score: 0.027500922849760058
[L59] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--9
[L60] snippet:
[L61] 5",
[L62] "dimension": 768,
[L63] "vector": [0.123, -0.456, "..."],
[L64] "created_at": "2026-08-17T10:00:00Z"
[L65] }
[L66] ],
[L67] "confidence": 0.92,
[L68] "abstraction_level": 0.35,
[L69] "statistics": {"access_count": 0, "update_count": 1, "merge_count": 0}
[L70] }
[L71]
```

```
[L72]
[L73] **Key design decisions:**
[L74] - `parent` / `children` are convenience-only fields derived from hierarchical edges. In production, al
[L75] - confidence is propagated from extraction confidence, parent confidence, and evidence confidence using
[L76] - abstraction_level (0.0 = very specific, 1.0 = very abstract) is used by CPG++ for parent filtering.
[L77] - extraction_confidence (not shown above) stores the original LLM extraction confidence before propagati
[L78]
[L79] ### 4.2 KnowledgeEdge Schema
[L80]
[L81] Relationships between nodes are represented as KnowledgeEdge objects. There are two edge classes:
[L82]
[L83] | Edge Class | Structural Rule | Example |
[L84] |-----|-----|-----|
[L85] | HIERARCHICAL | Must form an acyclic forest (single parent per node) | IS_A, PART_OF, SUBCLASS_OF, INST
[L86] | SEMANTIC | Unrestricted graph (cycles allowed) | USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIMILAR_TO
[L87]
[L88]
```

json

```
[L89] {
[L90] "edge_id": "EDGE_001",
[L91] "source_node": "NODE_000789",
[L92] "target_node": "NODE_000456",
[L93] "edge_class": "HIERARCHICAL",
[L94] "relation_type": "IS_A",
[L95] "confidence": 0.95,
[L96] "weight": 1.0,
[L97] "evidence": [{"text": "Round Robin is a type of CPU scheduling algorithm.", "source_doc": "DOC_001", "page": 42}],
[L98] "metadata": {"source_docs": ["DOC_001"]}
[L99] }
[L100]
```

```
[L101]
[L102] **Edge class enforcement:**
[L103]
[L104]
```

python

```
[L105] class NodeNotFoundError(Exception):
[L106] """Raised when a referenced node does not exist in the forest."""
[L107] pass
```

```

[L108]
[L109] def validate_edge(edge, forest):
[L110] """Validate edge based on its class.
[L111]
[L112] NOTE: We use explicit exceptions instead of assert statements because
[L113] Python's -O flag disables assertions in production."""
[L114] # Both edge classes require valid nodes
[L115] if not forest.has_node(edge.source_node):
[L116] raise NodeNotFoundError(f"Source node {edge.source_node} not found")
[L117] if not forest.has_node(edge.target_node):
[L118] raise NodeNotFoundError(f"Target node {edge.target_node} not found")
[L119]
[L120] if edge.edge_class == "HIERARCHICAL":
[L121] # MUST be acyclic — check for cycles
[L122] if creates_cycle(edge.source_node, edge.target_node, forest):
[L123] raise CycleError(f"HIERARCHICAL edge would create a cycle")
[L124] # MUST respect single-parent constraint
[L125] validate_single_parent(forest.get_node(edge.target_node), edge.source_node, forest)
[L126] elif edge.edge_class == "SEMANTIC":
[L127] # Cycles are ALLOWED — node existence already validated above
[L128] pass
[L129] ```
[L130]
[L131] ### 4.3 Relation Ontology
[L132]
[L133] The complete set of supported relation types:

```

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file2

```

[L134] metadata:
[L135] query_result_index: 2
[L136] file_id: file_00000000dcc082119f935683a5e13eed
[L137] version_id: 1
[L138] name: SHIA_RAG_Final_Report(4)(1).md
[L139] mime_type: text/markdown
[L140] surface: conversation
[L141] score: 0.025165372447512224
[L142] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--10
[L143] snippet:
[L144]

```

```
[L145] {
[L146]     "edge_id": "EDGE_001",
[L147]     "source_node": "NODE_000789",
[L148]     "target_node": "NODE_000456",
[L149]     "edge_class": "HIERARCHICAL",
[L150]     "relation_type": "IS_A",
[L151]     "confidence": 0.95,
[L152]     "weight": 1.0,
[L153]     "evidence": [{"text": "Round Robin is a type of CPU scheduling algorithm.", "source_doc": "DOC_001", "pag
[L154]     "metadata": {"source_docs": ["DOC_001"]}
[L155] }
[L156]
```

[L157]

[L158] **Edge class enforcement:**

[L159]

[L160]

```
[L161] class NodeNotFoundError(Exception):
[L162]     """Raised when a referenced node does not exist in the forest."""
[L163]     pass
[L164]
[L165] def validate_edge(edge, forest):
[L166]     """Validate edge based on its class.
[L167]
[L168]     NOTE: We use explicit exceptions instead of assert statements because
[L169]     Python's -O flag disables assertions in production."""
[L170]     # Both edge classes require valid nodes
[L171]     if not forest.has_node(edge.source_node):
[L172]         raise NodeNotFoundError(f"Source node {edge.source_node} not found")
[L173]     if not forest.has_node(edge.target_node):
[L174]         raise NodeNotFoundError(f"Target node {edge.target_node} not found")
[L175]
[L176]     if edge.edge_class == "HIERARCHICAL":
[L177]         # MUST be acyclic – check for cycles
[L178]         if creates_cycle(edge.source_node, edge.target_node, forest):
[L179]             raise CycleError(f"HIERARCHICAL edge would create a cycle")
[L180]         # MUST respect single-parent constraint
[L181]         validate_single_parent(forest.get_node(edge.target_node), edge.source_node, forest)
[L182]     elif edge.edge_class == "SEMANTIC":
[L183]         # Cycles are ALLOWED – node existence already validated above
[L184]         pass
[L185]
```

[L186]

[L187] **### 4.3 Relation Ontology**

[L188]

[L189] The complete set of supported relation types:

[L190]

[L191]

```
[L192] hierarchical_relations:
[L193]   - IS_A          # "Round Robin IS_A CPU Scheduling Algorithm"
[L194]   - PART_OF     # "ALU PART_OF CPU"
[L195]   - SUBCLASS_OF # "OSPF SUBCLASS_OF Routing Protocol"
[L196]   - INSTANCE_OF # "Linux INSTANCE_OF Operating System"
[L197]
[L198] semantic_relations:
[L199]   - USES          # "TCP USES Congestion Control"
[L200]   - CAUSES       # "Deadlock CAUSES System Hang"
[L201]   - DEPENDS_ON   # "Virtual Memory DEPENDS_ON Paging"
[L202]   - CONTRASTS_WITH # "Preemptive CONTRASTS_WITH Non-Preemptive"
[L203]   - SIMILAR_TO   # "Round Robin SIMILAR_TO Fair-Share"
[L204]   - HAS_PROPERTY # "Round Robin HAS_PROPERTY Time Quantum"
[L205]   - PRODUCES     # "Compiler PRODUCES Machine Code"
[L206]   - PREREQUISITE # "Data Structures PREREQUISITE Algorithms"
[L207]
```

[L208]

[L209] ### 4.4 Confidence Propagation

[L210]

[L211] Node confidence is propagated from roots to leaves using **topological sort** (Kahn's algorithm) to guarantee parents are always processed before children:

[L212]

[L213] ```python

[L214] from collections import deque

[L215]

[L216] # Confidence weighting factors (must sum to 1.0)

[L217] ALPHA = 0.4 # Node's own extraction confidence weight

[L218] BETA = 0.4 # Parent confidence weight

[L219] GAMMA = 0.2 # Evidence confidence weight

[L220] DECAY = 0.95 # Per-level decay factor

[L221] BASE_CONFIDENCE = 0.5 # Default for root nodes (configurable)

[L222]

[L223] def propagate_confidence(forest):

[L224] """Propagate confidence scores from roots to leaves.

[L225]

[L226] Uses Kahn's algorithm (topological ordering) to ensure

[L227] parents are always processed before their children.

[L228] Processing in arbitrary order would give children stale

[L229] parent confidence values."""

[L230] in_degree = {n.id: len(forest.

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file3

[L231] metadata:

[L232] query_result_index: 3

[L233] file_id: file_00000000dcc082119f935683a5e13eed

[L234] version_id: 1

[L235] name: SHIA_RAG_Final_Report(4)(1).md

[L236] mime_type: text/markdown

[L237] surface: conversation

[L238] score: 0.02492401215805471

[L239] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--53

[L240] snippet:

[L241] Real knowledge often has multiple valid parents (e.g., "Machine Learning" → both "AI" and "Statistics") | Deliberate design choice | Limits expressiveness | Future work: DAG support (Section 13.4) |

[L242] | L2 | **Greedy Tree Knapsack** — uses greedy approximation instead of exact DP. Exact DP is $O(nB)$ where B = token budget, which is computationally prohibitive | Deliberate trade-off | Near-optimal, not optimal | Future work: Exact DP for small budgets |

[L243] | L3 | **No incremental updates** — adding new documents requires re-running the full offline pipeline (Layers 1-5) | Engineering gap | Limits production scalability | Future work: Incremental forest updates |

[L244] | L4 | **Abstraction-level heuristic** — the multi-factor heuristic for estimating abstraction level has not been empirically validated | Validation needed | May produce incorrect hierarchy | Planned: Empirical validation on domain datasets |

[L245] | L5 | **LLM extraction quality** — knowledge extraction relies on LLM accuracy. Extraction errors propagate through the entire pipeline | Inherent limitation | Garbage-in-garbage-out risk | Mitigated by confidence thresholds and human review |

[L246] | L6 | **Self-evolution bias** — positive feedback on wrong paths can reinforce errors. ϵ -greedy exploration mitigates but does not eliminate this risk | Research challenge | May degrade over time | Mitigated by ϵ -greedy; future: Thompson sampling |

[L247] | L7 | **Novelty claims require verification** — "first" claims are based on systematic literature review but require exhaustive verification before publication | Research maturity | Affects publication readiness | All claims qualified with "to the best of our knowledge" |

[L248]

[L249] ---

[L250]

[L251] ## 13. Conclusion & Future Work

[L252]

[L253] ### 13.1 Summary

[L254]

[L255] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L256]

[L257] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L258] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L259] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L260] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L261] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L262]

[L263] The complete system specification covers:

[L264] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L265] - **12 defined functions** with complete implementations

[L266] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L267] - **8 schemas** covering all data objects from document blocks to query logs

[L268] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L269] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L270]

[L271] ### 13.2 Key Novelties

[L272]

[L273] | # | Novelty | Level | Why It's New |

[L274] |---|-----|-----|-----|

[L275] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L276] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge,

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite[turn13file4]`

[L277] metadata:

[L278] query_result_index: 4

[L279] file_id: file_00000000dcc082119f935683a5e13eed

[L280] version_id: 1

[L281] name: SHIA_RAG_Final_Report(4)(1).md

[L282] mime_type: text/markdown

[L283] surface: conversation

[L284] score: 0.031024531024531024

[L285] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--0

[L286] snippet:

[L287] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L288]

[L289] > **Project Type:** Minor Project (M.Tech)

[L290] > **Date:** August 2026

[L291] > **Author:** [Your Name]

[L292]

[L293] ---

[L294]

[L295] ## Abstract

[L296]

[L297] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L298]

[L299] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L300]

[L301] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L302]

[L303] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L304]

[L305] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving

Knowledge Systems, Natural Language Processing

[L306]

[L307] ---

[L308]

[L309] ## Table of Contents

[L310]

[L311] 1. Introduction

[L312] 2. Literature Review

[L313] 3. System Architecture

[L314] 4. Data Model — Layer 0

[L315] 5. Offline Pipeline — Layers 1-5

[L316] 6. Online Pipeline — Layers 6-7

[L317] 7. Operations Layer — Layer 8

[L318] 8. Novel Contributions

[L319] 9. Mathematical Framework

[L320] 10. Implementation

[L321] 11. Evaluation Framework

[L322] 12. Risk Analysis

[L323] 13. Conclusion & Future Work

[L324]

[L325] ---

[L326]

[L327] ## 1. Introduction

[L328]

[L329] ### 1.1 Problem Statement

[L330]

[L331] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L332]

[L333] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L334] 2. Embed each chunk into a vector space

[L335] 3. At query time, retrieve the top-k most similar chunks

[L336] 4. Feed these chunks as context to an LLM for answer generation

[L337]

[L338] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file5

[L339] metadata:

[L340] query_result_index: 5

[L341] file_id: file_00000000dcc082119f935683a5e13eed

[L342] version_id: 1

[L343] name: SHIA_RAG_Final_Report(4)(1).md

[L344] mime_type: text/markdown

[L345] surface: conversation

[L346] score: 0.03007688828584351

[L347] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6

[L348] snippet:

[L349]

```
[L350] graph TD
[L351]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L352]         L1["Layer 1: Input Processing<br>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br>Text normalization, tokenization"]
[L353]         L2 --> L3["Layer 3: Knowledge Extraction<br>Concept detection, alias resolution"]
[L354]         L3 --> L4["Layer 4: Relation Management<br>Triple extraction, confidence scoring"]
[L355]         L4 --> L5["Layer 5: SHIA Core<br>CPG++ → PRE → HV → FI → CLD"]
[L356]         L5 --> KF["Knowledge Forest Store<br>(Graph DB + Vector DB)"]
[L357]     end
[L358]
[L359]     subgraph "Phase 2: Online – Retrieval & Generation"
[L360]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br>Query analysis, forest traversal, RB0"]
[L361]         KF --> L6
[L362]         L6 --> L7["Layer 7: Generation Engine<br>Prompt assembly, LLM, verification"]
[L363]         L7 --> A["Answer + Citations"]
[L364]     end
[L365]
[L366]     subgraph "Phase 3: Continuous – Operations"
[L367]         L8["Layer 8: Operations<br>Logging, monitoring, feedback loop"]
[L368]         A --> L8
[L369]         L8 -->|"Self-Evolution<br>Feedback"| KF
[L370]     end
[L371] end
```

[L372]

[L373] ### 3.2 Data Flow

[L374]

[L375]

```
[L376] Raw Document (PDF/DOCX/HTML)
[L377]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L378] Normalized Text + Metadata
[L379]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L380] [Page → Region → Block] hierarchy
[L381]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L382] KnowledgeUnit objects (candidate concepts)
[L383]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L384] KnowledgeNode objects (canonical, deduplicated)
[L385]     ↓ Layer 4: Validate relations against ontology, score confidence
[L386] KnowledgeEdge objects (typed, scored)
[L387]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L388] Knowledge Forest (rooted trees + cross-links)
[L389]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L390] Selected context nodes (within token budget)
[L391]     ↓ Layer 7: Build prompt, invoke LLM
[L392]     ↓ Layer 7: Verify claims against evidence, bind citations
[L393] Answer with citations + confidence score
[L394]     ↓ Layer 8: Log, collect feedback, update forest weights
[L395]
```

[L396]

[L397] ### 3.3 Layer Summary

[L398]

[L399] | Layer | Name | Input | Output | Key Algorithms |

[L400] |-----|-----|-----|-----|-----|

[L401] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L402] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L403] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L404] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L405] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L406] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L407] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L408] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L409] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L410]
[L411] ---
[L412]
[L413] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file6

[L414] metadata:
[L415] query_result_index: 6
[L416] file_id: file_00000000dcc082119f935683a5e13eed
[L417] version_id: 1
[L418] name: SHIA_RAG_Final_Report(4)(1).md
[L419] mime_type: text/markdown
[L420] surface: conversation
[L421] score: 0.029957522915269395
[L422] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--8
[L423] snippet:
[L424] ### 4.1 KnowledgeNode Schema
[L425]
[L426] Each concept in the Knowledge Forest is represented as a `KnowledgeNode` :
[L427]
[L428]

```

[L429] {
[L430]   "node_id": "NODE_000789",
[L431]   "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L432]   "canonical_name": "Round Robin",
[L433]   "aliases": ["RR", "Round-Robin Scheduling"],
[L434]   "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process"],
[L435]   "summaries": ["CPU scheduling method using time slices"],
[L436]   "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L437]   "domain": "Operating Systems",
[L438]   "properties": {
[L439]     "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L440]     "preemptive": {"type": "boolean", "value": true}
[L441]   },
[L442]   "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L443]   "parent": "NODE_000456",
[L444]   "children": ["NODE_001001", "NODE_001002"],
[L445]   "related": ["NODE_000890"],
[L446]   "incoming_edges": ["EDGE_001"],
[L447]   "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L448]   "evidence": [
[L449]     {
[L450]       "evidence_type": "DEFINITION",
[L451]       "text": "Round Robin is a preemptive scheduling algorithm...",
[L452]       "source_doc": "DOC_000001",
[L453]       "source_block": "BLK_000234",
[L454]       "page": 42
[L455]     }
[L456]   ],
[L457]   "embeddings": [
[L458]     {
[L459]       "model": "bge-base-en-v1.5",
[L460]       "dimension": 768,
[L461]       "vector": [0.123, -0.456, "..."],
[L462]       "created_at": "2026-08-17T10:00:00Z"
[L463]     }
[L464]   ],
[L465]   "confidence": 0.92,
[L466]   "abstraction_level": 0.35,
[L467]   "statistics": {"access_count": 0, "update_count": 1, "merge_count": 0}
[L468] }
[L469]

```

[L470]

[L471] **Key design decisions:**

[L472] - **parent / children** are **convenience-only** fields derived from hierarchical edges. In production, always derive these from edge queries to avoid update anomalies (e.g., **parent** field says **NODE_100** but the actual edge points to **NODE_200**). These fields MUST NOT be treated as the source of truth.

[L473] - **confidence** is propagated from extraction confidence, parent confidence, and evidence confidence using topological sort (see Section 4.4).

[L474] - **abstraction_level** (0.0 = very specific, 1.0 = very abstract) is used by CPG++ for parent filtering.

[L475] - **extraction_confidence** (not shown above) stores the original LLM extraction confidence before propagation. This is the immutable baseline.

[L476]

[L477] **### 4.2 KnowledgeEdge Schema**

[L478]

[L479] Relationships between nodes are represented as **KnowledgeEdge** objects. There are **two edge classes**:

[L480]

[L481] | Edge Class | Structural Rule | Example |

[L482] |-----|-----|-----|

[L483] | **HIERARCHICAL** | Must form an **acyclic forest** (single parent per node) | IS_A, PART_OF, SUBCLASS_OF, INSTANCE_OF |[L484] | **SEMANTIC** | Unrestricted **graph** (cycles allowed) | USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIMILAR_TO |

DEEP RESEARCH 24/08/2026, 00:22:52

Citation Marker: `filecite`turn13file7

[L485] metadata:

[L486] query_result_index: 7

[L487] file_id: file_00000000dcc082119f935683a5e13eed

[L488] version_id: 1

[L489] name: SHIA_RAG_Final_Report(4)(1).md

[L490] mime_type: text/markdown

[L491] surface: conversation

[L492] score: 0.029957522915269395

[L493] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--31

[L494] snippet:

[L495] ##### 5.5.5 CLD — Cross-Link Discoverer

[L496]

[L497] After hierarchical placement, discover semantic (non-hierarchical) links:

[L498]

[L499]

```
[L500] def cld_discover(node, forest, config):
[L501]     """Find semantic cross-links for a newly placed node."""
[L502]     cross_links = []
[L503]     neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L504]
[L505]     for neighbor in neighbors:
[L506]         if neighbor.id == node.id or neighbor.id == node.parent:
[L507]             continue
[L508]
[L509]         sim = cosine_similarity(node.embedding, neighbor.embedding)
[L510]         if sim > config.cross_link_threshold:
[L511]             relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L512]             if relation_type:
[L513]                 forest.add_edge(node.id, neighbor.id,
[L514]                                 edge_class='SEMANTIC', relation_type=relation_type,
[L515]                                 confidence=sim, weight=1.0)
[L516]                 cross_links.append((neighbor.id, relation_type, sim))
[L517]
[L518]     return cross_links
[L519]
```

[L520]

[L521] ---

[L522]

[L523] ## 6. Online Pipeline — Layers 6-7

[L524]

[L525] ### 6.1 Layer 6: Retrieval Engine

[L526]

[L527] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L528]

[L529] **Input:** User query string → **Output:** RetrievalResult

[L530]

[L531] ##### Query Complexity Classification

[L532]

[L533] | Intent | Example | Complexity | Traversal Depth |

```

[L534] |-----|-----|-----|-----|
[L535] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |
[L536] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |
[L537] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |
[L538] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |
[L539] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |
[L540]
[L541] ##### Retrieval Pipeline
[L542]
[L543] ```python
[L544] @dataclass
[L545] class RetrievalResult:
[L546] """Output of Layer 6 — input to Layer 7."""
[L547] selected_nodes: list # Nodes selected by Tree Knapsack
[L548] context_text: str # Assembled context string
[L549] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L550] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L551] query_intent: str # DEFINITION, COMPARISON, etc.
[L552] seed_node_ids: list # Initial seed nodes from vector search
[L553] token_count: int # Estimated tokens in context
[L554]
[L555] class RetrievalEngine:
[L556] def __init__(self, forest, vector_index, cache_manager, config):
[L557] self.forest = forest
[L558] self.vector_index = vector_index
[L559] self.cache = cache_manager
[L560] self.config = config
[L561]
[L562] def retrieve(self, query) -> RetrievalResult:
[L563] # Step 0: Cache check
[L564] cached = self.cache.get_query_result(hash_query(query))
[L565] if cached:
[L566] return cached
[L567]
[L568] # Step 1: Classify query intent and complexity
[L569] intent = detect_intent(query)
[L570] complexity = classify_complexity(intent)
[L571]
[L572] # Step 2: Embed query and find seed nodes
[L573] query_emb = embed_query(query) # Asymmetric embedding model
[L574] seed_nodes = self.vector_index.search(query_emb, k=5)
[L575]
[L576] # Step 3: Adaptive depth traversal
[L577] depth_config = {
[L578] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L579] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L580] 'COMPLEX': {'max_depth': 3, 'parents': True,

```

Bilkul. Ab hum **Layer 4** ko bhi same running example ke saath, step-by-step aur depth mein samjhenge.

Hum wahi example continue karte hain:

Operating Systems → CPU Scheduling → Round Robin → Time Quantum

Ab tak:

Layer 1 → Raw document ko clean kiya
Layer 2 → Document structure samjha
Layer 3 → Concepts aur candidate relations nikale
Layer 4 → Candidate relations ko validate karke reliable edges banayega

Layer 4 — Relation Management

Sabse pehle ek line mein samjho

Layer 4 receives KnowledgeNodes and the raw candidate relations extracted in Layer 3. It checks whether each relation is valid according to the relation ontology, resolves the target concept, removes duplicates, calculates confidence, resolves conflicts between documents, and creates final KnowledgeEdges.

Simple flow:

```
KnowledgeNode[]  
(with candidate relations)  
    ↓  
  Layer 4  
    ↓  
Validate Relation  
    ↓  
Resolve Source & Target  
    ↓  
Check Duplicate  
    ↓  
Determine Edge Class  
    ↓  
Calculate Confidence  
    ↓  
Resolve Document Conflicts  
    ↓  
Validated KnowledgeEdge[]  
    ↓  
  Layer 5
```

Layer 4 ki exact responsibility hai **candidate relations** ko ontology ke against validate karna, **confidence score** karna, **cross-document conflicts** resolve karna, aur final **KnowledgeEdge** objects banana. [filecite]turn11file0[L52-L74]

1. Sabse pehle — Layer 3 aur Layer 4 mein difference

Ye bahut important boundary hai.

Layer 3

Layer 3 kehti hai:

“Mujhe text mein ek possible relation mila.”

Example:

Round Robin USES Time Quantum

Lekin abhi ye sirf:

Candidate Relation

hai.

System ko abhi guarantee nahi hai ki:

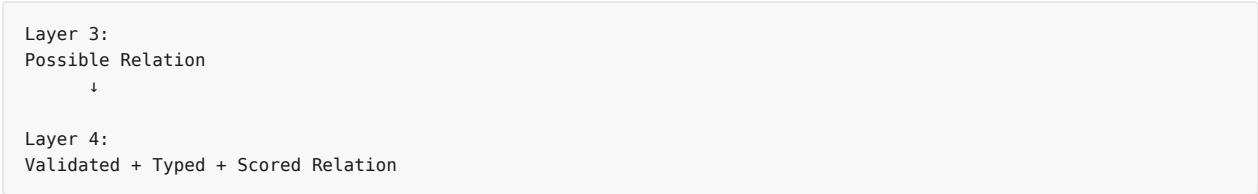
- **USES** correct relation hai
- source aur target correct hain
- relation duplicate toh nahi hai
- relation ka evidence strong hai ya weak
- doosre document mein contradictory relation toh nahi mila

Layer 4

Layer 4 kehti hai:

“Ab main verify karungi ki ye relation system mein accept kiya ja sakta hai ya nahi.”

So:

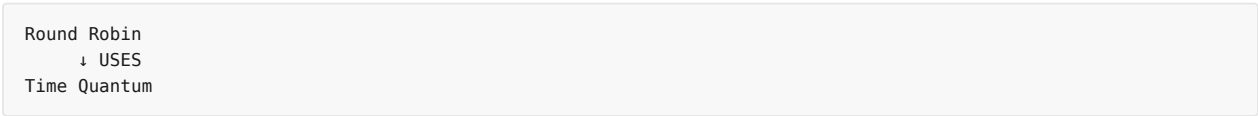


Report clearly Layer 3 ko **candidate extraction** aur Layer 4 ko **validation and scoring** layer define karti hai.

filecite[turn12file7L649-L665]

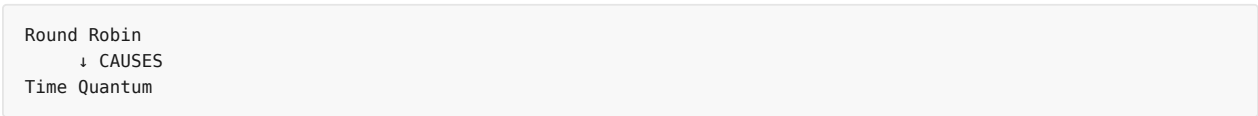
2. Layer 4 ki zarurat kyu padi?

Maan lo Layer 3 ne ye relation nikala:

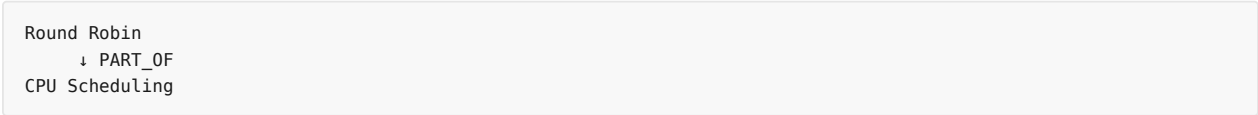


Ye correct lag raha hai.

Lekin LLM kabhi galat relation bhi extract kar sakta hai:



Ya:



Ab har extracted relation ko directly Knowledge Forest mein daal diya toh:

```

Wrong Relation
  ↓
Wrong Graph
  ↓
Wrong Hierarchy
  ↓
Wrong Retrieval
  ↓
Wrong Answer

```

Isliye Layer 4 ek **quality-control gate** ki tarah kaam karti hai.

Layer 3 finds relationships. Layer 4 decides which relationships are valid and how reliable they are.

3. Layer 4 ka Input kya hai?

Official input:

```

KnowledgeNode[]
(with raw relations)

```

Har `KnowledgeNode` ke paas Layer 3 se extracted concepts, definitions, properties aur possible raw relations aa sakte hain.

Example:

```

KnowledgeNode 1
Name: Round Robin

Raw Relations:
1. Round Robin USES Time Quantum
2. Round Robin IS_A CPU Scheduling Algorithm

```

Aur:

```

KnowledgeNode 2
Name: Time Quantum

```

```

KnowledgeNode 3
Name: CPU Scheduling Algorithm

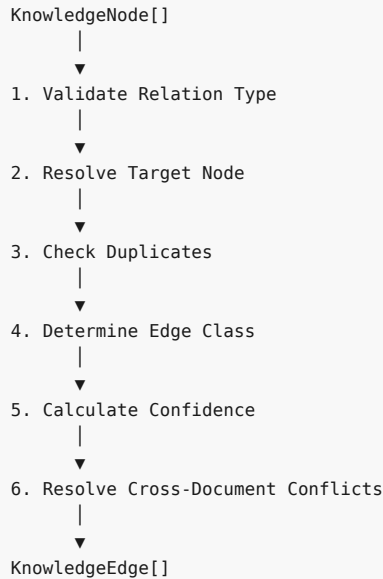
```

Ab Layer 4 in raw relations ko process karegi.

Source ke according input `KnowledgeNode[]` with raw relations hai aur output validated/scored `KnowledgeEdge[]` hai.

4. Layer 4 ka complete pipeline

File ke according pipeline:



filecite[turn11file7L594-L611]

Ab ek-ek step detail mein samajhte hain.

5. Step 1 — Relation Type Validation

Problem kya hai?

Layer 3 LLM se koi relation extract hua:

Round Robin → USES → Time Quantum

Ab Layer 4 check karegi:

*Kya **USES** hamari predefined relation ontology mein allowed hai?*

Agar haan:

VALID ✓

Agar relation kuch unknown hai:

Round Robin
↓ UTILIZES
Time Quantum

Lekin ontology mein **UTILIZES** nahi hai.

Toh Layer 4 direct accept nahi karegi.

Source ke according:

Check relation against ontology
↓
Unknown?
↓
Attempt inference via LLM

filecite[turn12file1L114-L135]

6. Relation Ontology kya hoti hai?

Simple language mein:

Relation Ontology is a predefined set of allowed relationship types.

System kisi bhi random relation name ko graph mein add nahi karta.

Example:

```
✗ CONNECTS_TO_SOMETHING
✗ RELATED_IN_SOME_WAY
✗ UTILIZES_SOMEHOW
```

Instead, predefined relations use honge.

7. SHIA-RAG mein relation ke 2 main classes hain

A. Hierarchical Relations

Ye hierarchy banane mein help karti hain.

Supported examples:

```
IS_A
PART_OF
SUBCLASS_OF
INSTANCE_OF
```

Example:

```
Round Robin
  IS_A
    ↓
CPU Scheduling Algorithm
```

Aur:

```
ALU
  PART_OF
    ↓
CPU
```

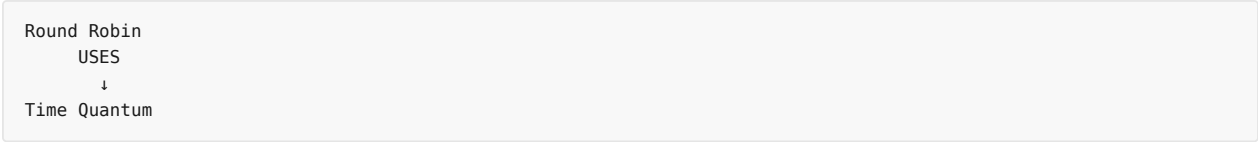
B. Semantic Relations

Ye concepts ko connect karti hain, lekin parent-child hierarchy zaroori nahi banati.

Examples:

```
USES
CAUSES
DEPENDS_ON
CONTRASTS_WITH
SIMILAR_TO
HAS_PROPERTY
PRODUCES
PREREQUISITE
```

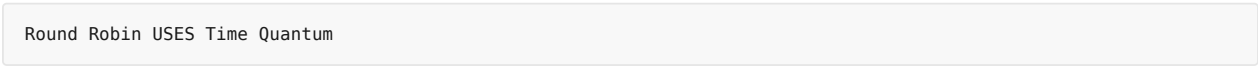
Example:



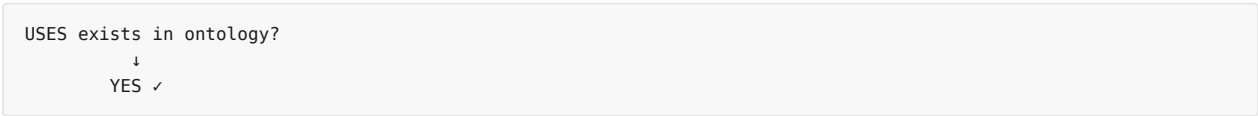
Report mein exactly 4 hierarchical aur 8 semantic relation types define kiye gaye hain. [filecite]turn13file2[187-L207]

8. Relation Type Validation hamare example mein

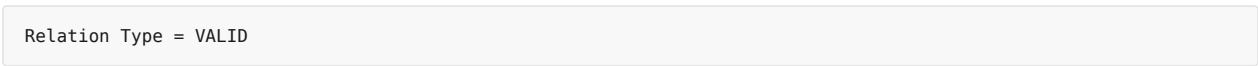
Layer 3 output:



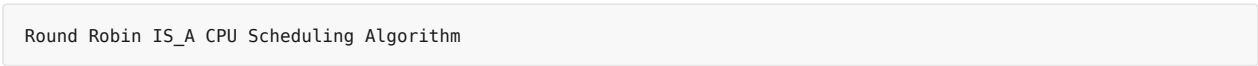
Layer 4 ontology check:



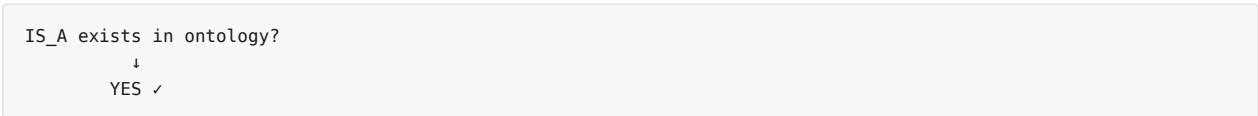
Therefore:



Second relation:



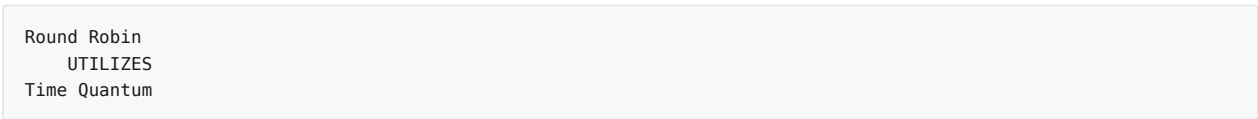
Check:



Valid.

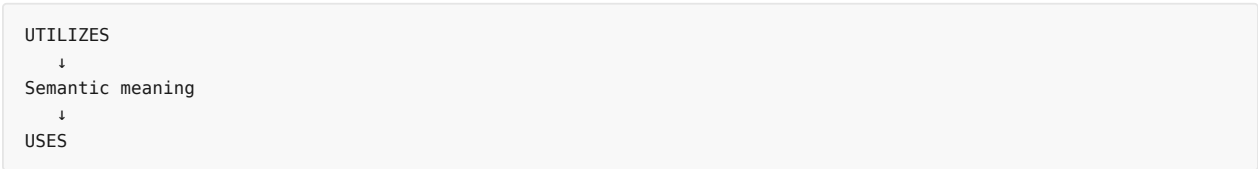
9. Agar relation ontology mein nahi mila toh?

Example:

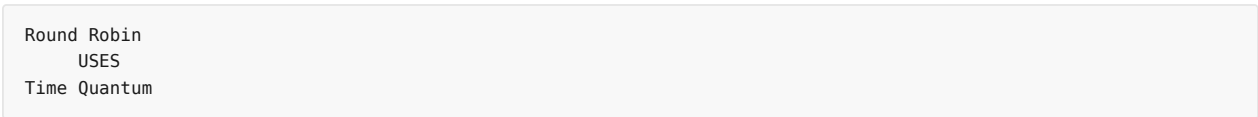


Suppose UTILIZES predefined ontology mein nahi hai.

Layer 4 LLM se infer karne ki koshish kar sakti hai:



Then:



File specifically unknown relation type ke liye **LLM inference attempt** define karti hai. Exact mapping rules beyond that source section mein specify nahi kiye gaye hain. `filecite\turn11file0\L58-L64`

10. Step 2 — Target Node Resolution

Ye bahut important step hai.

Layer 3 ne relation nikala:

```
Round Robin USES Time Quantum
```

Lekin relation text mein target:

```
"Time Quantum"
```

sirf ek **name/string** ho sakta hai.

Layer 4 ko actual `KnowledgeNode` find karna hai.

So:

```
Raw Target Name
"Time Quantum"
  ↓
KnowledgeNode Search / Resolution
  ↓
NODE_000456
```

Final relation:

```
Source Node: NODE_RR
Relation: USES
Target Node: NODE_TIME_QUANTUM
```

Source pipeline mein ye step **"map target concept name to a KnowledgeNode"** hai. `filecite\turn12file8\L676-L684`

11. Is step ki zarurat kyu hai?

Maan lo lag documents mein likha hai:

```
Time Quantum
```

Aur:

```
Time Slice
```

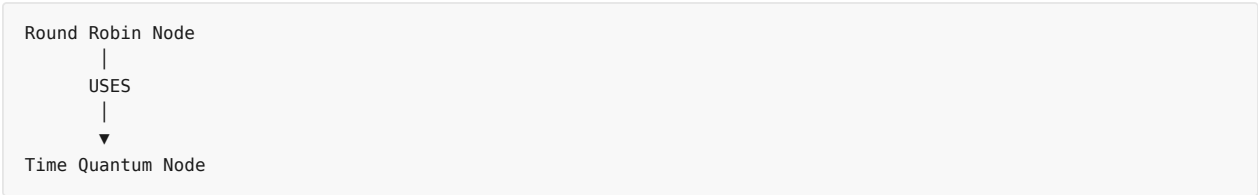
Agar Layer 3 alias resolution ne already in concepts ko canonical form mein merge kiya hai, Layer 4 ko relation correct node se connect karna hoga.

Otherwise:

```
Round Robin
  USES
    ↓
  "Time Quantum" ← only text
```

Graph properly structured nahi hoga.

Correct:



Important idea:

Layer 4 text-to-text relation ko actual node-to-node relation mein convert karti hai.

12. Step 3 — Duplicate Relation Check

Ab maan lo same relation 3 jagah mili:

Document A

Round Robin USES Time Quantum

Document B

Round Robin USES Time Quantum

Document C

Round Robin USES Time Quantum

Agar har baar nayi edge bana dein:

Round Robin —USES→ Time Quantum
Round Robin —USES→ Time Quantum
Round Robin —USES→ Time Quantum

× Unnecessary duplicates.

Layer 4 check karti hai:

Same Source?
Same Target?
Same Relation Type?
↓
Duplicate?

Agar duplicate:

New Edge create mat karo
↓
Existing Edge mein Evidence merge karo

Source pipeline exactly duplicate edge ke case mein evidence ko existing edge mein merge karne ka rule define karti hai.
filecite_turn11file0L58-L64

13. Example — Evidence Merge

Initially:

```
EDGE_001

Round Robin
  USES
    Time Quantum

Evidence:
Document A, Page 10
```

Phir same relation Document B mein mili:

```
Round Robin
  USES
    Time Quantum
```

New edge banane ke bajaye:

```
EDGE_001

Round Robin
  USES
    Time Quantum

Evidence:
✓ Document A, Page 10
✓ Document B, Page 25
```

Isse edge stronger evidence collect kar sakti hai.

14. Step 4 — Edge Class Determine karna

Ab relation type valid hai.

Layer 4 decide karegi:

Ye hierarchical edge hai ya semantic edge?

Example 1

```
Round Robin
  IS_A
    CPU Scheduling Algorithm
```

IS_A belongs to:

```
HIERARCHICAL
```

So:

```
edge_class = HIERARCHICAL
```

Example 2

```
Round Robin
  USES
    Time Quantum
```

USES belongs to:

```
SEMANTIC
```


So:

```
edge_class = SEMANTIC
```

15. Hierarchical aur Semantic edge mein difference

HIERARCHICAL

Example:

```
CPU Scheduling
  |
IS_A / hierarchy
  |
Round Robin
```

System ke report design mein hierarchical edges ko **acyclic forest** constraints follow karne hote hain, aur single-parent structure enforce kiya jata hai. [filecite\turn13file1\L79-L129](#)

Simple meaning:

```
A → B → C
```

Allowed.

But:

```
A → B
↑      ↓
└────┘
```

Hierarchy cycle:

```
Not Allowed ❌
```

SEMANTIC

Example:

```
Round Robin
  USES
Time Quantum
```

Ye parent-child relation nahi hai.

Ye simply semantic connection hai.

Semantic graph mein cycles allowed hain. [filecite\turn13file1\L81-L86](#)

16. Step 5 — Confidence Calculation

Ab relation valid hai.

Lekin system poochta hai:

“Mujhe is relation par kitna trust hai?”

Iske liye:

```
Confidence Score
```

calculate hota hai.

File mein formula ka concept:

```
Final Confidence
=
Base Confidence
+
Source Confidence Contribution
+
Evidence Quality Contribution
+
Semantic Similarity Contribution
```

Source pseudocode:

```
conf = 0.5

+ 0.15 × source confidence
+ 0.10 × evidence quality
+ 0.10 × semantic similarity
```

Then score ko maximum 1.0 tak cap kiya jata hai. `filecite\turn11file7\L604-L611`

17. Confidence ke har component ko samjho

A. Base Confidence

System ek initial base se start karta hai:

```
0.5
```

Matlab relation bilkul zero se start nahi hota.

B. Source Confidence

Source `KnowledgeNode` ki extraction confidence.

Maan lo:

```
Round Robin Node Confidence = 0.90
```

To contribution:

```
0.15 × 0.90
```

C. Evidence Quality

Evidence length ko source pseudocode mein proxy ke roop mein use kiya gaya hai:

```
Evidence longer / sufficient
↓
Potentially higher evidence-quality contribution
```

Formula evidence length ko 200 characters tak normalize karta hai. `filecite\turn12file8\L686-L693`

D. Semantic Similarity

System check karta hai:

Relation ka evidence source concept ke semantic context se kitna match karta hai?

Conceptually:

```

Source Embedding
  ↔
Evidence Embedding
  ↓
Cosine Similarity
  
```

Higher semantic match:

```
Higher confidence contribution
```

18. Simple confidence example

Maan lo:

```

Source Confidence = 0.90
Evidence Quality = 0.80
Semantic Similarity = 0.85
  
```

Conceptually:

```

Base
+ Source contribution
+ Evidence contribution
+ Similarity contribution
  
```

Aur final score ek confidence value ban jayega.

Important: Exact numerical formula report mein pseudocode ke form mein diya gaya hai; implementation mein usi formula ke according score compute hota hai. [filecite\turn11file0\L66-L74](#)

19. Step 6 — Cross-Document Conflict Resolution

Ye Layer 4 ka bahut important part hai.

Maan lo do documents relation ke baare mein disagree karte hain.

Document A

```

Round Robin
  IS_A
Scheduling
  
```

Document B

```

Round Robin
  PART_OF
Operating System
  
```

Ab system blindly dono ko same final meaning ke saath accept nahi karta.

File ke according:

```

Same Source-Target Pair
    ↓
Group All Candidate Edges
    ↓
Different Relation Types?
    ↓
Evidence-Weighted Voting
    ↓
Select Stronger Relation
    ↓
Close Conflict?
    ↓
Log for Manual Review

```

filecite\turn11file7\L614-L668

20. Evidence-Weighted Voting kya hai?

Suppose:

```
Round Robin → CPU Scheduling Algorithm
```

ke liye multiple documents relations suggest karte hain.

IS_A

Evidence:

```
Confidence = High
Evidence = Strong
```

PART_OF

Evidence:

```
Confidence = Low
Evidence = Weak
```

System simple vote counting nahi karti:

```
IS_A = 1
PART_OF = 1
```

Instead weight consider karti hai.

Source pseudocode mein weight:

```
weight =
edge.confidence × evidence length
```

Then:

```
Har relation type ka total weighted vote
    ↓
Highest vote wins
```

filecite\turn11file5\L426-L471

21. Agar conflict bahut close ho?

Maan lo:

```
IS_A Vote = 0.82
PART_OF Vote = 0.78
```

Difference bahut small hai.

System ke source logic mein close margin hone par conflict:

```
Manual Review ke liye log
```

kiya jata hai.

Matlab system blindly overconfident nahi hota. `filecite{turn11file7}{L650-L667}`

22. Hamara complete running example

Ab start se Layer 4 process dekhte hain.

Input from Layer 3

```
KnowledgeNode:
Round Robin

Candidate Relations:

1. Round Robin IS_A CPU Scheduling Algorithm

2. Round Robin USES Time Quantum
```

Step 1 — Validate relation types

```
IS_A
↓
Ontology check
↓
Valid ✓
```

```
USES
↓
Ontology check
↓
Valid ✓
```

Step 2 — Resolve target nodes

```
"CPU Scheduling Algorithm"
↓
NODE_100
```

```
"Time Quantum"
↓
NODE_200
```

Now relations become:

```
NODE_RR
  IS_A
  NODE_100
```

and:

```
NODE_RR
  USES
  NODE_200
```

Step 3 — Check duplicates

Suppose:

```
Round Robin USES Time Quantum
```

already exists.

Then:

```
Don't create new edge ❌
```

Instead:

```
Add new evidence ✓
```

Step 4 — Determine edge class

Relation 1

```
IS_A
↓
HIERARCHICAL
```

Relation 2

```
USES
↓
SEMANTIC
```

Step 5 — Calculate confidence

For both edges:

```
Source Confidence
+
Evidence Quality
+
Semantic Similarity
↓
Final Confidence
```

Step 6 — Resolve conflict

Agar doosre documents different relation suggest karte hain:

```
Group
↓
Weighted Voting
↓
Select strongest supported relation
↓
Close conflict → Manual Review Log
```

23. Final Output — KnowledgeEdge[]

Finally Layer 4 output:

```
KnowledgeEdge[]
```

Example 1:

```
KnowledgeEdge(  
    edge_id = "EDGE_001",  
  
    source_node = "Round Robin",  
  
    target_node = "CPU Scheduling Algorithm",  
  
    edge_class = "HIERARCHICAL",  
  
    relation_type = "IS_A",  
  
    confidence = 0.95,  
  
    evidence = [  
        "Round Robin is a type of CPU scheduling algorithm."  
    ]  
)
```

Example 2:

```
KnowledgeEdge(  
    edge_id = "EDGE_002",  
  
    source_node = "Round Robin",  
  
    target_node = "Time Quantum",  
  
    edge_class = "SEMANTIC",  
  
    relation_type = "USES",  
  
    confidence = 0.88,  
  
    evidence = [  
        "Round Robin assigns a fixed time quantum to each process."  
    ]  
)
```

`KnowledgeEdge` schema source node, target node, edge class, relation type, confidence, weight, evidence aur metadata maintain karta hai. <https://filecite.com/turn13file1/L79-L100>

24. KnowledgeNode vs KnowledgeEdge

Ye bahut important difference hai.

KnowledgeNode = Concept

Example:

```
Round Robin
```

```
Time Quantum
```

```
CPU Scheduling Algorithm
```

KnowledgeEdge = Relationship

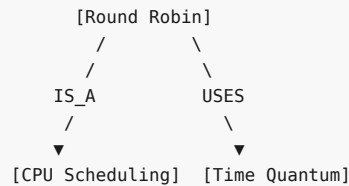
Example:

Round Robin
IS_A
CPU Scheduling Algorithm

Aur:

Round Robin
USES
Time Quantum

Simple diagram:



Nodes tell us what knowledge exists. Edges tell us how that knowledge is connected.

25. Layer 4 mein kaunsi technologies/algorithms use ho rahe hain?

| Technology / Method | Purpose |

|---|---|

| **Relation Ontology** | Allowed relation types validate karna |

| **LLM Inference** | Unknown relation type infer karna |

| **Node Resolution** | Target concept ko actual KnowledgeNode se map karna |

| **Duplicate Detection** | Same relation ki multiple edges avoid karna |

| **Ontology Mapping** | Hierarchical vs Semantic class decide karna |

| **Cosine Similarity** | Semantic evidence similarity measure karna |

| **Confidence Scoring** | Relation reliability score karna |

| **Evidence-Weighted Voting** | Cross-document conflicts resolve karna |

| **Conflict Logging** | Close conflicts ko manual review ke liye record karna |

Ye sab Layer 4 specification aur edge/ontology schema mein defined hain. [filecite\turn12file8\L676-L749](#)

[filecite\turn13file1\L79-L129](#)

26. Layer 4 mein possible errors aur safeguards

Error 1 — Invalid relation

Round Robin → RANDOM_RELATION → Time Quantum

Safeguard

Ontology Validation

Unknown relation ko direct accept nahi kiya jata; source design mein LLM-based inference attempt hota hai.

[filecite\turn11file0\L58-L64](#)

Error 2 — Target concept exist nahi karta

Round Robin
USES
Unknown Concept

Safeguard

Target Node Resolution

Relation ko actual KnowledgeNode se map karna zaroori hai.

Layer 0 edge validation mein missing source/target node ke liye explicit node-not-found errors bhi defined hain.

`filecite\turn13file1\L102-L128`

Error 3 — Same relation baar-baar create hona

Round Robin → USES → Time Quantum

100 documents mein mil sakta hai.

Safeguard

Duplicate Check
+
Evidence Merge

Error 4 — Weak evidence par high trust

Safeguard

Confidence Scoring

Multiple signals combine kiye jaate hain:

Source Confidence
+
Evidence Quality
+
Semantic Similarity

`filecite\turn11file7\L604-L611`

Error 5 — Documents conflict karte hain

Document A → IS_A

Document B → PART_OF

Safeguard

Evidence-Weighted Voting

Aur close conflict:

Manual Review Log

fileciteurn11file5L443-L469

Error 6 — Hierarchy mein cycle ban jana

Example:

```
A → B
B → C
C → A
```

Safeguard

Hierarchical edges ke liye cycle detection aur single-parent constraints defined hain. Semantic edges ke liye cycles allowed hain. fileciteurn13file1L102-L129

27. Layer 4 kya nahi karti?

Ye samajhna bahut important hai.

Layer 4:

- × Naye concepts primarily extract nahi karti
- × Synonym/alias resolution nahi karti — ye Layer 3 ka main kaam tha
- × Final hierarchy construct nahi karti
- × Parent-child placement decide nahi karti
- × Knowledge Forest nahi banati

Layer 4 ka main focus hai:

“Concepts ke beech extracted relationships ko validate, clean, score, and resolve karna.”

Actual hierarchy construction:

Layer 5

mein start hogi.

28. Layer 3 → Layer 4 → Layer 5 difference

LAYER 3**Knowledge Extraction**

"Round Robin is a concept."
"RR is its alias."
"This is a possible relation."

↓

LAYER 4**Relation Management**

"Is this relation valid?"
"Is it allowed by ontology?"
"Which actual nodes does it connect?"
"How confident are we?"
"Do documents disagree?"

↓

LAYER 5**Hierarchy Construction**

"Where should this node be placed?"
"Who should be its parent?"
"Does the placement make sense?"
"Build the Knowledge Forest."

29. Real-world analogy

Imagine **Layer 3** ek **detective** hai.

Detective kehta hai:

Mujhe evidence mila hai ki:

Round Robin
USES
Time Quantum

Lekin detective ka finding directly final court decision nahi hota.

Layer 4 ek verifier/judge ki tarah hai

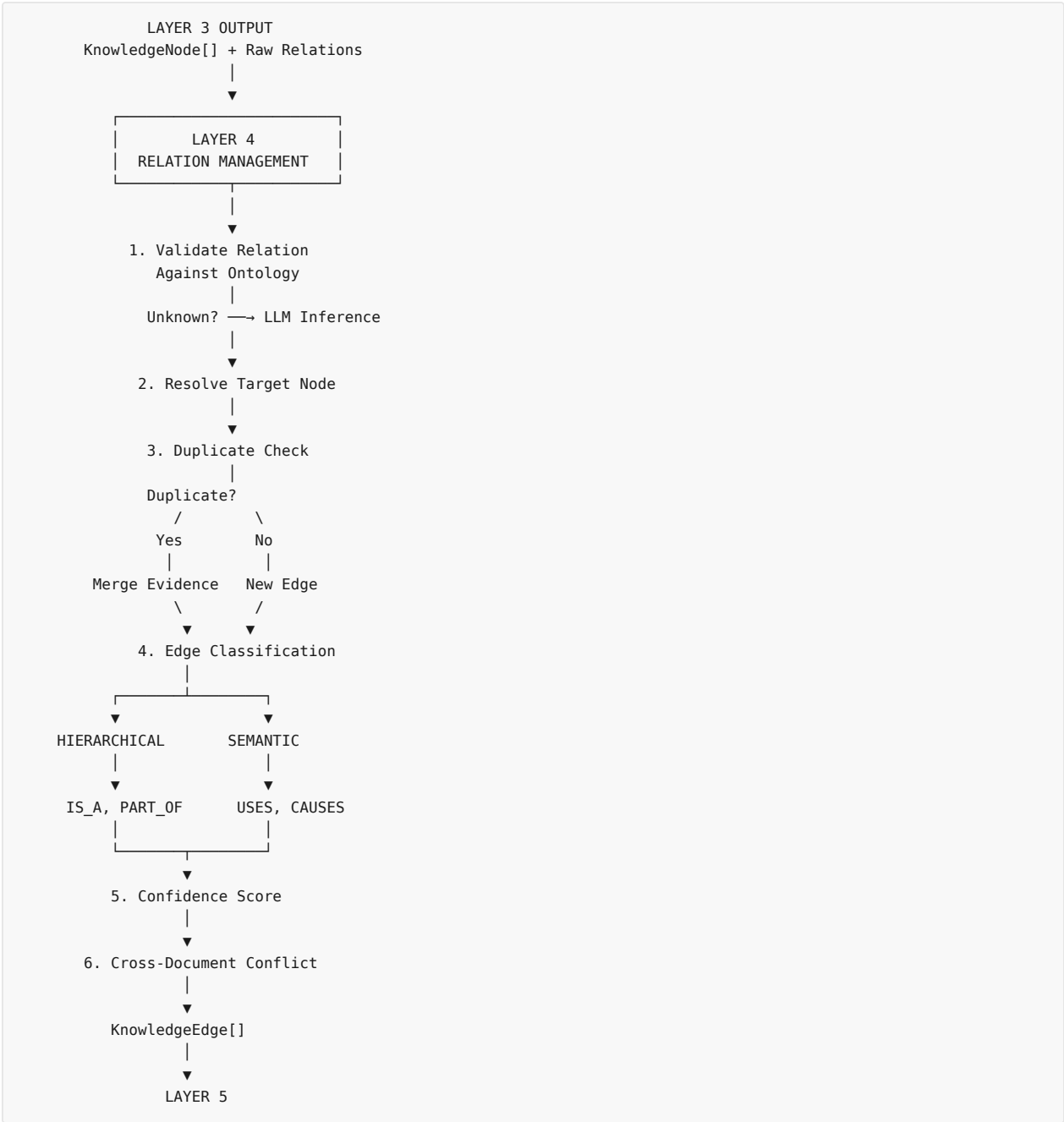
Wo check karta hai:

- ✓ Is relation type valid?
- ✓ Kya dono concepts actually exist karte hain?
- ✓ Kya relation already known hai?
- ✓ Evidence kitna strong hai?
- ✓ Doosre documents kya bolte hain?
- ✓ Kya conflict hai?

Then final decision:

VALIDATED EDGE

30. Complete Layer 4 architecture



31. Final Summary

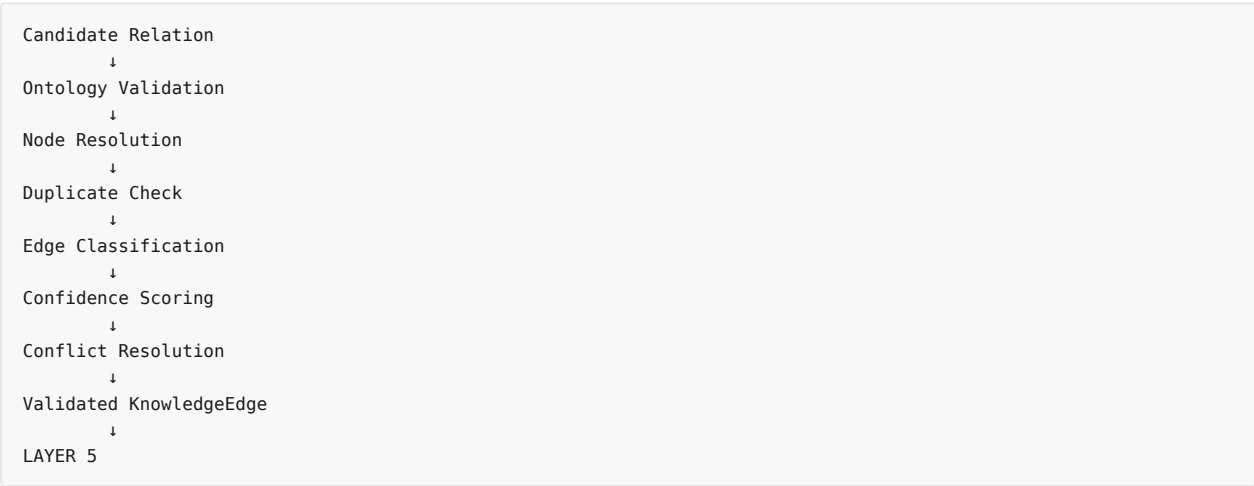
- | Point | Layer 4 |
- |---|---|
- | **Name** | Relation Management |
- | **Input** | KnowledgeNode[] with raw relations |
- | **Main Purpose** | Relations ko validate aur manage karna |
- | **Relation Check** | Relation Ontology |
- | **Unknown Relation** | LLM inference attempt |
- | **Target Resolution** | Concept name → actual KnowledgeNode |
- | **Duplicate Handling** | Existing edge mein evidence merge |
- | **Edge Classes** | Hierarchical / Semantic |
- | **Confidence** | Source confidence + evidence quality + semantic similarity |
- | **Conflict Handling** | Evidence-weighted voting |

Close Conflict	Manual review log
Final Output	KnowledgeEdge[]
Next Layer	Layer 5 — SHIA Core / Hierarchy Construction

Sabse important one-line understanding

“Layer 4 transforms the candidate relations extracted in Layer 3 into validated and reliable KnowledgeEdges by checking them against a relation ontology, resolving the correct nodes, removing duplicates, calculating confidence, and resolving conflicts between multiple documents.”

Complete flow yaad rakhne ke liye:



Sabse important conceptual shift:
Layer 3 says: “I found a possible connection.”
Layer 4 says: “I have validated, scored, and structured this connection so the system can safely use it.”

YOU 24/08/2026, 00:33:44

mujhe layer 4 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 4 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 24/08/2026, 00:33:51

{“size”:“1024x1024”,“n”:1,“transparent_background”:false,“is_style_transfer”:false}

YOU 24/08/2026, 00:36:23

mujhe layer 4 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 4 ko english me btao lekin brife me

CHATGPT 24/08/2026, 00:36:28

Layer 4 – Relation Management

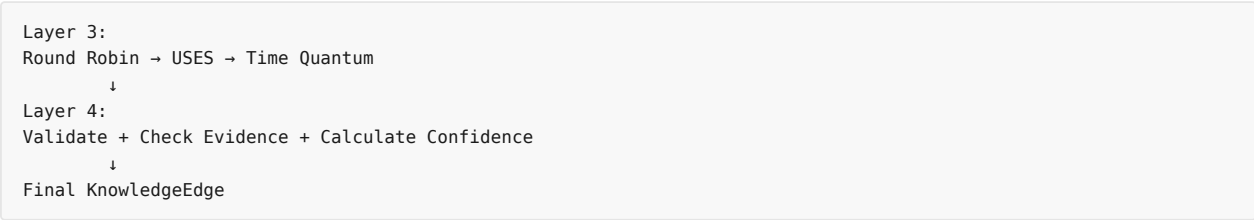
You can explain it to your guide like this:

“Layer 4 receives the candidate relationships extracted in Layer 3 and verifies whether those relationships are correct and reliable. It checks each relation against a predefined relation ontology, connects it to the correct KnowledgeNodes, removes duplicates, calculates a confidence score, and handles conflicts between different documents.”

What happens in Layer 4?

- It **validates the relation type**, such as `IS_A` , `PART_OF` , or `USES` .
- It maps the relation to the correct **source and target KnowledgeNodes**.
- It **removes duplicate relationships** and combines their evidence.
- It classifies relations as **Hierarchical** or **Semantic**.
- It calculates a **confidence score** based on the source and evidence quality.
- If multiple documents provide conflicting relations, it **resolves the conflict** using the available evidence.

Example



Output

```
KnowledgeEdge[]
```

Simple conclusion

“In short, Layer 4 acts as a quality-control layer for relationships. It ensures that only valid, reliable, and well-supported connections between knowledge concepts are passed to the next layer for hierarchy construction.”

YOU 24/08/2026, 00:40:59

ab mujhe layer 5 smjhao detailed me jese baki layer smjhayi thi

CHATGPT 24/08/2026, 00:41:01

```
{"paths":["files"],"query":"search"}
```

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `filecite`turn14file0

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_000000003218820b918ee3d86ec345ae
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report(1).md
- [L6] mime_type: text/markdown
- [L7] surface: library
- [L8] score: 0.03131881575727918
- [L9] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--6
- [L10] snippet:
- [L11]

```
[L12] graph TD
[L13]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L14]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L15]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L16]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L17]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L18]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L19]     end
[L20]
[L21]     subgraph "Phase 2: Online – Retrieval & Generation"
[L22]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L23]         KF --> L6
[L24]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L25]         L7 --> A["Answer + Citations"]
[L26]     end
[L27]
[L28]     subgraph "Phase 3: Continuous – Operations"
[L29]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L30]         A --> L8
[L31]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L32]     end
[L33]
```

[L34]

[L35] ### 3.2 Data Flow

[L36]

[L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43]     ↓ Layer 3: Split sentences, extract concepts/definitions/rerelations
[L44] KnowledgeUnit objects (candidate concepts)
[L45]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47]     ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53]     ↓ Layer 7: Build prompt, invoke LLM
[L54]     ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56]     ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]

[L59] ### 3.3 Layer Summary

[L60]

[L61] | Layer | Name | Input | Output | Key Algorithms |

[L62] |-----|-----|-----|-----|-----|

[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L72]
[L73] ---
[L74]
[L75] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `[[filecite]]turn14file1`

[L76] metadata:
[L77] query_result_index: 1
[L78] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L79] version_id: 1
[L80] name: SHIA_RAG_Final_Report(2).md
[L81] mime_type: text/markdown
[L82] surface: library
[L83] score: 0.031099324975891997
[L84] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--6
[L85] snippet:
[L86]

```
[L87] graph TD
[L88]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L89]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L90]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L91]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L92]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L93]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L94]     end
[L95]
[L96]     subgraph "Phase 2: Online – Retrieval & Generation"
[L97]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L98]         KF --> L6
[L99]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L100]        L7 --> A["Answer + Citations"]
[L101]     end
[L102]
[L103]     subgraph "Phase 3: Continuous – Operations"
[L104]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L105]         A --> L8
[L106]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L107]     end
[L108]
```

[L109]
[L110] ### 3.2 Data Flow
[L111]
[L112]


```
[L113] Raw Document (PDF/DOCX/HTML)
[L114]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L115] Normalized Text + Metadata
[L116]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L117] [Page → Region → Block] hierarchy
[L118]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L119] KnowledgeUnit objects (candidate concepts)
[L120]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L121] KnowledgeNode objects (canonical, deduplicated)
[L122]   ↓ Layer 4: Validate relations against ontology, score confidence
[L123] KnowledgeEdge objects (typed, scored)
[L124]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L125] Knowledge Forest (rooted trees + cross-links)
[L126]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L127] Selected context nodes (within token budget)
[L128]   ↓ Layer 7: Build prompt, invoke LLM
[L129]   ↓ Layer 7: Verify claims against evidence, bind citations
[L130] Answer with citations + confidence score
[L131]   ↓ Layer 8: Log, collect feedback, update forest weights
[L132]
```

[L133]

[L134] ### 3.3 Layer Summary

[L135]

[L136] | Layer | Name | Input | Output | Key Algorithms |

[L137] |-----|-----|-----|-----|-----|

[L138] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L139] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L140] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L141] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L142] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L143] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L144] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L145] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L146] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L147]

[L148] ---

[L149]

[L150] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `[[filecite]]turn14file2`

[L151] metadata:

[L152] query_result_index: 2

[L153] file_id: file_00000000fb2c81fd82f14a7a48993554

[L154] version_id: 1

[L155] name: SHIA_RAG_Final_Report.md

[L156] mime_type: text/markdown

[L157] surface: library

[L158] score: 0.030886196246139225

[L159] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--6

[L160] snippet:

[L161]

```
[L162] graph TD
[L163]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L164]         L1["Layer 1: Input Processing<br>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br>Text normalization, tokenization"]
[L165]         L2 --> L3["Layer 3: Knowledge Extraction<br>Concept detection, alias resolution"]
[L166]         L3 --> L4["Layer 4: Relation Management<br>Triple extraction, confidence scoring"]
[L167]         L4 --> L5["Layer 5: SHIA Core<br>CPG++ → PRE → HV → FI → CLD"]
[L168]         L5 --> KF["Knowledge Forest Store<br>(Graph DB + Vector DB)"]
[L169]     end
[L170]
[L171]     subgraph "Phase 2: Online – Retrieval & Generation"
[L172]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br>Query analysis, forest traversal, RB0"]
[L173]         KF --> L6
[L174]         L6 --> L7["Layer 7: Generation Engine<br>Prompt assembly, LLM, verification"]
[L175]         L7 --> A["Answer + Citations"]
[L176]     end
[L177]
[L178]     subgraph "Phase 3: Continuous – Operations"
[L179]         L8["Layer 8: Operations<br>Logging, monitoring, feedback loop"]
[L180]         A --> L8
[L181]         L8 -->|"Self-Evolution<br>Feedback"| KF
[L182]     end
[L183]
```

[L184]

[L185] ### 3.2 Data Flow

[L186]

[L187]

```
[L188] Raw Document (PDF/DOCX/HTML)
[L189]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L190] Normalized Text + Metadata
[L191]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L192] [Page → Region → Block] hierarchy
[L193]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L194] KnowledgeUnit objects (candidate concepts)
[L195]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L196] KnowledgeNode objects (canonical, deduplicated)
[L197]     ↓ Layer 4: Validate relations against ontology, score confidence
[L198] KnowledgeEdge objects (typed, scored)
[L199]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L200] Knowledge Forest (rooted trees + cross-links)
[L201]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L202] Selected context nodes (within token budget)
[L203]     ↓ Layer 7: Build prompt, invoke LLM
[L204]     ↓ Layer 7: Verify claims against evidence, bind citations
[L205] Answer with citations + confidence score
[L206]     ↓ Layer 8: Log, collect feedback, update forest weights
[L207]
```

[L208]

[L209] ### 3.3 Layer Summary

[L210]

[L211] | Layer | Name | Input | Output | Key Algorithms |

[L212] |-----|-----|-----|-----|-----|

[L213] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L214] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L215] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L216] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L217] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L218] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L219] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L220] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L221] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L222]
[L223] ---
[L224]
[L225] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `[[filecite]]turn14file3[[`

[L226] metadata:
[L227] query_result_index: 3
[L228] file_id: file_00000000f620821186e54e582d59731e
[L229] version_id: 1
[L230] name: SHIA_RAG_Final_Report(3).md
[L231] mime_type: text/markdown
[L232] surface: library
[L233] score: 0.030679156908665108
[L234] document_chunk_id: file_00000000f620821186e54e582d59731e--1--6
[L235] snippet:
[L236]

```
[L237] graph TD
[L238]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L239]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L240]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L241]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L242]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L243]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L244]     end
[L245]
[L246]     subgraph "Phase 2: Online – Retrieval & Generation"
[L247]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L248]         KF --> L6
[L249]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L250]         L7 --> A["Answer + Citations"]
[L251]     end
[L252]
[L253]     subgraph "Phase 3: Continuous – Operations"
[L254]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L255]         A --> L8
[L256]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L257]     end
[L258]
```

[L259]
[L260] ### 3.2 Data Flow
[L261]
[L262]

```
[L263] Raw Document (PDF/DOCX/HTML)
[L264]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L265] Normalized Text + Metadata
[L266]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L267] [Page → Region → Block] hierarchy
[L268]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L269] KnowledgeUnit objects (candidate concepts)
[L270]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L271] KnowledgeNode objects (canonical, deduplicated)
[L272]   ↓ Layer 4: Validate relations against ontology, score confidence
[L273] KnowledgeEdge objects (typed, scored)
[L274]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L275] Knowledge Forest (rooted trees + cross-links)
[L276]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L277] Selected context nodes (within token budget)
[L278]   ↓ Layer 7: Build prompt, invoke LLM
[L279]   ↓ Layer 7: Verify claims against evidence, bind citations
[L280] Answer with citations + confidence score
[L281]   ↓ Layer 8: Log, collect feedback, update forest weights
[L282]
```

[L283]

[L284] ### 3.3 Layer Summary

[L285]

[L286] | Layer | Name | Input | Output | Key Algorithms |

[L287] |-----|-----|-----|-----|-----|

[L288] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L289] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L290] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L291] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L292] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L293] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L294] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L295] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L296] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L297]

[L298] ---

[L299]

[L300] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `[[filecite]]turn14file4`

[L301] metadata:

[L302] query_result_index: 4

[L303] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L304] version_id: 1

[L305] name: SHIA_RAG_Final_Report(4).md

[L306] mime_type: text/markdown

[L307] surface: library

[L308] score: 0.03047794966520434

[L309] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--6

[L310] snippet:

[L311]

```
[L312] graph TD
[L313]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L314]         L1["Layer 1: Input Processing<br>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br>Text normalization, tokenization"]
[L315]         L2 --> L3["Layer 3: Knowledge Extraction<br>Concept detection, alias resolution"]
[L316]         L3 --> L4["Layer 4: Relation Management<br>Triple extraction, confidence scoring"]
[L317]         L4 --> L5["Layer 5: SHIA Core<br>CPG++ → PRE → HV → FI → CLD"]
[L318]         L5 --> KF["Knowledge Forest Store<br>(Graph DB + Vector DB)"]
[L319]     end
[L320]
[L321]     subgraph "Phase 2: Online – Retrieval & Generation"
[L322]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br>Query analysis, forest traversal, RB0"]
[L323]         KF --> L6
[L324]         L6 --> L7["Layer 7: Generation Engine<br>Prompt assembly, LLM, verification"]
[L325]         L7 --> A["Answer + Citations"]
[L326]     end
[L327]
[L328]     subgraph "Phase 3: Continuous – Operations"
[L329]         L8["Layer 8: Operations<br>Logging, monitoring, feedback loop"]
[L330]         A --> L8
[L331]         L8 -->|"Self-Evolution<br>Feedback"| KF
[L332]     end
[L333]
```

[L334]

[L335] ### 3.2 Data Flow

[L336]

[L337]

```
[L338] Raw Document (PDF/DOCX/HTML)
[L339]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L340] Normalized Text + Metadata
[L341]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L342] [Page → Region → Block] hierarchy
[L343]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L344] KnowledgeUnit objects (candidate concepts)
[L345]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L346] KnowledgeNode objects (canonical, deduplicated)
[L347]     ↓ Layer 4: Validate relations against ontology, score confidence
[L348] KnowledgeEdge objects (typed, scored)
[L349]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L350] Knowledge Forest (rooted trees + cross-links)
[L351]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L352] Selected context nodes (within token budget)
[L353]     ↓ Layer 7: Build prompt, invoke LLM
[L354]     ↓ Layer 7: Verify claims against evidence, bind citations
[L355] Answer with citations + confidence score
[L356]     ↓ Layer 8: Log, collect feedback, update forest weights
[L357]
```

[L358]

[L359] ### 3.3 Layer Summary

[L360]

[L361] | Layer | Name | Input | Output | Key Algorithms |

[L362] |-----|-----|-----|-----|-----|

[L363] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L364] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L365] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L366] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L367] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L368] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L369] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L370] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L371] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L372]
[L373] ---
[L374]
[L375] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `[[filecite]]turn14file5[[`

[L376] metadata:
[L377] query_result_index: 5
[L378] file_id: file_00000000dcc082119f935683a5e13eed
[L379] version_id: 1
[L380] name: SHIA_RAG_Final_Report(4)(1).md
[L381] mime_type: text/markdown
[L382] surface: conversation
[L383] score: 0.028169014084507043
[L384] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6
[L385] snippet:
[L386]

```
[L387] graph TD
[L388]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L389]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L390]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L391]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L392]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L393]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L394]     end
[L395]
[L396]     subgraph "Phase 2: Online – Retrieval & Generation"
[L397]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L398]         KF --> L6
[L399]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L400]         L7 --> A["Answer + Citations"]
[L401]     end
[L402]
[L403]     subgraph "Phase 3: Continuous – Operations"
[L404]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L405]         A --> L8
[L406]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L407]     end
[L408]
```

[L409]
[L410] ### 3.2 Data Flow
[L411]
[L412]

```
[L413] Raw Document (PDF/DOCX/HTML)
[L414]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L415] Normalized Text + Metadata
[L416]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L417] [Page → Region → Block] hierarchy
[L418]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L419] KnowledgeUnit objects (candidate concepts)
[L420]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L421] KnowledgeNode objects (canonical, deduplicated)
[L422]   ↓ Layer 4: Validate relations against ontology, score confidence
[L423] KnowledgeEdge objects (typed, scored)
[L424]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L425] Knowledge Forest (rooted trees + cross-links)
[L426]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L427] Selected context nodes (within token budget)
[L428]   ↓ Layer 7: Build prompt, invoke LLM
[L429]   ↓ Layer 7: Verify claims against evidence, bind citations
[L430] Answer with citations + confidence score
[L431]   ↓ Layer 8: Log, collect feedback, update forest weights
[L432]
```

[L433]

[L434] ### 3.3 Layer Summary

[L435]

[L436] | Layer | Name | Input | Output | Key Algorithms |

[L437] |-----|-----|-----|-----|-----|

[L438] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L439] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L440] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L441] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L442] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L443] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L444] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L445] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L446] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L447]

[L448] ---

[L449]

[L450] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `[[filecite]]turn14file6`

[L451] metadata:

[L452] query_result_index: 6

[L453] file_id: file_00000000f620821186e54e582d59731e

[L454] version_id: 1

[L455] name: SHIA_RAG_Final_Report(3).md

[L456] mime_type: text/markdown

[L457] surface: library

[L458] score: 0.018294583307361466

[L459] document_chunk_id: file_00000000f620821186e54e582d59731e--1--7

[L460] snippet:

[L461] deduplicated)

[L462] ↓ Layer 4: Validate relations against ontology, score confidence

[L463] KnowledgeEdge objects (typed, scored)
[L464] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L465] Knowledge Forest (rooted trees + cross-links)
[L466] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L467] Selected context nodes (within token budget)
[L468] ↓ Layer 7: Build prompt, invoke LLM
[L469] ↓ Layer 7: Verify claims against evidence, bind citations
[L470] Answer with citations + confidence score
[L471] ↓ Layer 8: Log, collect feedback, update forest weights
[L472]

```
[L473]
[L474] ### 3.3 Layer Summary
[L475]
[L476] | Layer | Name | Input | Output | Key Algorithms |
[L477] |-----|-----|-----|-----|-----|
[L478] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L479] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L480] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L481] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L482] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L483] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L484] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L485] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L486] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L487]
[L488] ---
[L489]
[L490] ## 4. Data Model – Layer 0
[L491]
[L492] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode`
[L493]
[L494] ### 4.1 KnowledgeNode Schema
[L495]
[L496] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L497]
[L498]
```

json

```
[L499] {
[L500] "node_id": "NODE_000789",
[L501] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L502] "canonical_name": "Round Robin",
[L503] "aliases": ["RR", "Round-Robin Scheduling"],
[L504] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L505] "summaries": ["CPU scheduling method using time slices"],
[L506] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L507] "domain": "Operating Systems",
[L508] "properties": {
[L509] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L510] "preemptive": {"type": "boolean", "value": true}
[L511] },
[L512] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L513] "parent": "NODE_000456",
[L514] "children": ["NODE_001001", "NODE_001002"],
[L515] "related": ["NODE_000890"],
[L516] "incoming_edges": ["EDGE_001"],
```



```
[L517] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L518] "evidence": [
[L519] {
[L520] "evidence_type": "DEFINITION",
[L521] "text": "Round Robin is a preemptive scheduling algorithm...",
[L522] "source_doc": "DOC_000001",
[L523] "source_block": "BLK_000234",
[L524] "page": 42
[L525] }
[L526] ],
```

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `filecite`turn14file7

```
[L527] metadata:
[L528] query_result_index: 7
[L529] file_id: file_0000000002708208b0d69fecf8bcdadd
[L530] version_id: 1
[L531] name: SHIA_RAG_Complete_Report(4).md
[L532] mime_type: text/markdown
[L533] surface: library
[L534] score: 0.018294583307361466
[L535] document_chunk_id: file_0000000002708208b0d69fecf8bcdadd--1--13
[L536] snippet:
[L537] ### 4.3 Tree vs DAG vs Graph
[L538]
[L539] | PDF | Claim |
[L540] |----|-----|
[L541] | All PDFs | "Knowledge Forest is acyclic (no loops)" |
[L542] | All PDFs | "Cross-Link Discovery (CLD) adds semantic associations" |
[L543]
[L544] Contradiction: Cross-links create graph cycles. The forest is only acyclic in its hierarchical edges (parent-child). Cross-
links form a separate semantic graph layer.
[L545]
[L546] Resolution: Define two distinct edge types:
[L547] 1. Hierarchical edges (IS_A, PART_OF) — must be acyclic, form a forest
[L548] 2. Semantic edges (USES, RELATED_TO, CAUSES) — may form cycles, separate graph layer
[L549]
[L550] > [!IMPORTANT]
[L551] > Single-Parent Constraint: SHIA-RAG enforces strict single-parent trees for hierarchical edges. Each node has at
most one parent in the hierarchy. DAG support (multiple parents) is deferred to future work. This simplifies:
[L552] > - Precedence constraints in Tree Knapsack:  $x_i \leq x_{\text{parent}(i)}$  is unambiguous
[L553] > - Confidence propagation: single parent path, no need for multi-parent averaging
[L554] > - Cycle detection: simple ancestor traversal
[L555] >
[L556] > Multiple semantic associations are handled via SEMANTIC edges (cross-links), which are unrestricted.
[L557]
[L558]
```

```
[L559] # Explicit enforcement of single-parent constraint
[L560] def validate_single_parent(node, new_parent, forest):
[L561]     """Ensure node has at most one hierarchical parent."""
[L562]     existing_parents = [
[L563]         e for e in forest.get_incoming_edges(node)
[L564]         if e.edge_class == "HIERARCHICAL"
[L565]     ]
[L566]     if existing_parents:
[L567]         raise ValueError(
[L568]             f"Node {node.id} already has parent via edge {existing_parents[0].edge_id}. "
[L569]             f"SHIA-RAG enforces single-parent trees. Use SEMANTIC edges for additional associations."
[L570]         )
[L571]
```

[L572]

[L573] ---

[L574]

[L575] ## 5. Novelty Contributions

[L576]

[L577] > [!IMPORTANT]

[L578] > These are **5 novel contributions** that make SHIA-RAG publishable and distinguish it from all existing RAG systems (TreeRAG, HiChunk, RAPTOR, GraphRAG, etc.).

[L579]

[L580] ### Novelty 1: Precedence-Constrained Tree Knapsack for Context Selection

[L581]

[L582] **What exists:** All current RAG systems use flat top-k selection or simple greedy knapsack for context packing.

[L583]

[L584] **What SHIA-RAG proposes:** A **greedy approximation for the Tree Knapsack** formulation where selecting a child node forces inclusion of its parent (precedence constraint). This ensures hierarchical coherence in the assembled context.

[L585]

[L586]
$$\max \sum_{i=1}^n U_i \text{ s.t. } \sum_{i=1}^n C_i \leq B, \quad x_i \in \{0,1\} \text{ for all } i$$

[L587]

[L588] > [!NOTE]

[L589] > The implementation uses a **greedy utility-density heuristic** rather than exact DP, as exact solutions are computationally prohibitive for large token budgets. The precedence-constrained formulation itself is the novelty.

[L590]

[L591] **Why novel:** No RAG paper has formulated context selection as a precedence-constrained optimization problem. The adaptation of the Tree Knapsack formulation from operations research to RAG context selection is a direct, citable contribution (incremental but novel application).

[L592]

[L593] ---

[L594]

[L595] ### Novelty 2: Self-Evolving Knowledge Forest via Retrieval Feedback

[L596]

[L597] **What exists:** All current Knowledge Graphs and hierarchies are static after construction.

DEEP RESEARCH 24/08/2026, 00:41:09

Citation Marker: `filecite[turn14file8]`

[L598] metadata:

[L599] query_result_index: 8

[L600] file_id: file_00000000473c72069a723b3187461cb6

[L601] version_id: 1
[L602] name: 2601.04742v1.pdf
[L603] mime_type: application/pdf
[L604] surface: library
[L605] score: 0.018057336133766127
[L606] document_chunk_id: file_00000000473c72069a723b3187461cb6:4--1--2
[L607] snippet:
[L608] For agent $A \in \{AR, AS\}$,
[L609] the process is formally defined as:
[L610] q
[L611] 1
[L612] $A = \text{Query}(A, c)$ (1)
[L613] $D1$
[L614] $A = \text{Retrieve}(A, q1$
[L615] $A)$ (2)
[L616] a
[L617] 1
[L618] $A = \text{Respond}(A, D1$
[L619] $A, c)$ (3)
[L620] where q
[L621] 1
[L622] A is the retrieval query, $D1$
[L623] A is the set of retrieved
[L624] documents, and a
[L625] 1
[L626] A is the generated response for round 1.
[L627] Debate Rounds ($r > 1$). In each subsequent round,
[L628] agents refine their queries and responses by incorporating the
[L629] opponent's previous answer. This enables dynamic evidence
[L630] updates and encourages more robust reasoning. Each round
[L631] consists of three steps: query formulation, evidence retrieval,
[L632] and response generation. Specifically:
[L633] TABLE I
[L634] DATASETS USED IN TOOL-MAD, CATEGORIZED BY TASK TYPE AND
[L635] CORRESPONDING REFERENCES.
[L636] Dataset Task
[L637] FEVER [27] Fact Verification
[L638] FEVEROUS [28] Fact Verification
[L639] FAVIQ [29] Fact Verification
[L640] AVERITEC [30] Fact Verification
[L641] MEDQA [31] Medical
[L642] PUBMEDQA [32] Medical
[L643] q
[L644] r
[L645] $A = \text{Query}(A, c, ar-1$
[L646] A^r
[L647]) (4)
[L648] Dr
[L649] $A = \text{Retrieve}(A, qr$
[L650] $A)$ (5)

[L651] a

[L652] r

[L653] $A = \text{Respond}(A, D_r$

[L654] A, c, a_{r-1}

[L655] A^-

[L656]) (6)

[L657] where a

[L658] $r-1$

[L659] A^-

[L660] is the previous response from the opposing agent

[L661] A^- .

[L662] If both agents produce the same response in any round,

[L663] the debate terminates early with a consensus, considering

[L664] predefined thresholds of the stability score. Otherwise, the

[L665] current round is appended to the debate history. If consensus

[L666] is not reached or the responses fall below the thresholds

[L667] for faithfulness or answer relevance, the debate continues

[L668] until round T. The Judge agent AJ then determines the final

[L669] outcome.

[L670] Judge Decision(triggered only when the debate reaches

[L671] the final round) If the agents fail to reach a consensus by the

[L672] final round T, the Judge agent AJ makes the final decision

[L673] based on the original claim c, the complete debate history

[L674] H, and the stability score S. Here, S is composed of the

[L675] average faithfulness and answer relevance scores across all

[L676] rounds, capturing the overall quality of the agents' responses.

[L677] This decision process is formalized as:

[L678] $\text{Final answer} = \text{Judge}(AJ, H, S, c)$ (7)

[L679] The LLM-based Judge agent receives the stability score S

[L680] and debate history H as part of its prompt and makes the final

[L681] decision by jointly considering both.

[L682] For reproducibility, we provide the full prompts used for

[L683] the debate round, query selection, and judge components in

[L684] Figures 4, 5, and 6. Note that the prompt for the initial

[L685] round corresponds to the same template with the opponent's

[L686] response removed.

[L687] IV. EXPERIMENTS

[L688] We evaluate the effectiveness of a Tool-MAD framework

[L689] that leverages external tools in performing the fact verification task. Unless specified otherwise, all experiments were

[L690] conducted on 200 randomly sampled instances per dataset,

[L691] evaluated using an Exact Match (EM) criterion, where a

[L692] prediction is considered correct if it exactly matches the

[L693] ground truth labels.⁷

[L694] TABLE II

[L695] STRUCTURAL COMPARISON OF BASELINE REASONING FRAMEWORKS.

[L696] metadata:
[L697] query_result_index: 9
[L698] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L699] version_id: 1
[L700] name: SHIA_RAG_Final_Report(2).md
[L701] mime_type: text/markdown
[L702] surface: library
[L703] score: 0.01803017294247516
[L704] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--7
[L705] snippet:
[L706] deduplicated)
[L707] ↓ Layer 4: Validate relations against ontology, score confidence
[L708] KnowledgeEdge objects (typed, scored)
[L709] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L710] Knowledge Forest (rooted trees + cross-links)
[L711] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L712] Selected context nodes (within token budget)
[L713] ↓ Layer 7: Build prompt, invoke LLM
[L714] ↓ Layer 7: Verify claims against evidence, bind citations
[L715] Answer with citations + confidence score
[L716] ↓ Layer 8: Log, collect feedback, update forest weights
[L717]

```
[L718]
[L719] ### 3.3 Layer Summary
[L720]
[L721] | Layer | Name | Input | Output | Key Algorithms |
[L722] |-----|-----|-----|-----|-----|
[L723] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L724] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L725] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L726] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L727] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L728] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L729] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L730] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L731] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L732]
[L733] ---
[L734]
[L735] ## 4. Data Model – Layer 0
[L736]
[L737] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L738]
[L739] ### 4.1 KnowledgeNode Schema
[L740]
[L741] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L742]
[L743]
```

json

```
[L744] {
[L745] "node_id": "NODE_000789",
[L746] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L747] "canonical_name": "Round Robin",
[L748] "aliases": ["RR", "Round-Robin Scheduling"],
[L749] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
}
```

```
[L750] "summaries": ["CPU scheduling method using time slices"],
[L751] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L752] "domain": "Operating Systems",
[L753] "properties": {
[L754] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L755] "preemptive": {"type": "boolean", "value": true}
[L756] },
[L757] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L758] "parent": "NODE_000456",
[L759] "children": ["NODE_001001", "NODE_001002"],
[L760] "related": ["NODE_000890"],
[L761] "incoming_edges": ["EDGE_001"],
[L762] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L763] "evidence": [
[L764] {
[L765] "evidence_type": "DEFINITION",
[L766] "text": "Round Robin is a preemptive scheduling algorithm...",
[L767] "source_doc": "DOC_000001",
[L768] "source_block": "BLK_000234",
[L769] "page": 42
[L770] }
[L771] ],
```

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file0

```
[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_00000000f620821186e54e582d59731e
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(3).md
[L6] mime_type: text/markdown
[L7] surface: library
[L8] score: 0.018294583307361466
[L9] document_chunk_id: file_00000000f620821186e54e582d59731e--1--15
[L10] snippet:
[L11]
```

```
[L12] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L13]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L14]     conf = 0.5
[L15]     conf += 0.15 * source.confidence
[L16]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L17]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L18]     return min(conf, 1.0)
[L19]
```

```
[L20]
[L21] ---
[L22]
[L23] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction
[L24]
```

[L25] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L26]

[L27] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L28]

[L29] ##### Pipeline Overview

[L30]

[L31]

```
[L32] graph LR
[L33]     N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L34]     CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L35]     PRE --> HV["HV<br/>Validate #1?"]
[L36]     HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L37]     HV -->|"Fail"| HV2["HV<br/>Validate #2?"]
[L38]     HV2 -->|"Pass"| FI
[L39]     HV2 -->|"Fail"| HV3["HV<br/>Validate #3?"]
[L40]     HV3 -->|"Pass"| FI
[L41]     HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L42]     FI --> CLD["CLD<br/>Discover Cross-Links"]
[L43]     CLD --> DONE["Node Placed ✓"]
[L44]
```

[L45]

[L46] ##### 5.5.1 CPG++ — Candidate Parent Generator

[L47]

[L48] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L49]

[L50]

```
[L51] def cpq_plus_plus(node_q, forest, config):
[L52]     """Generate ≤k candidate parents for node q."""
[L53]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L54]     domain = classify_domain(node_q)
[L55]     domain_nodes = forest.get_nodes_by_domain(domain)
[L56]
[L57]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L58]     if len(domain_nodes) > config.ann_k:
[L59]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L60]         candidates = [r for r in ann_results if r in domain_nodes]
[L61]     else:
[L62]         candidates = domain_nodes
[L63]
[L64]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L65]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L66]
[L67]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L68]     q_level = estimate_abstraction_level(node_q)
[L69]     candidates = [c for c in candidates
[L70]         if c.abstraction_level > q_level
[L71]         and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L72]
[L73]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L74]     candidates = sorted(candidates,
[L75]         key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L76]
[L77]     return candidates[:config.max_candidates] # Return top 5
[L78]
```

[L79]

[L80] **Overall complexity:** O(log N) dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L81]

[L82] ##### 5.5.2 PRE — Parent Ranking Engine

[L83]

[L84] Ranks all candidates using a weighted scoring formula:

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file1

[L85] metadata:

[L86] query_result_index: 1

[L87] file_id: file_000000003218820b918ee3d86ec345ae

[L88] version_id: 1

[L89] name: SHIA_RAG_Final_Report(1).md

[L90] mime_type: text/markdown

[L91] surface: library

[L92] score: 0.018294583307361466

[L93] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--15

[L94] snippet:

[L95]

```
[L96] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L97]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L98]     conf = 0.5
[L99]     conf += 0.15 * source.confidence
[L100]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L101]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L102]     return min(conf, 1.0)
[L103]
```

[L104]

[L105] ---

[L106]

[L107] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L108]

[L109] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L110]

[L111] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L112]

[L113] ##### Pipeline Overview

[L114]

[L115]

```
[L116] graph LR
[L117]     N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L118]     CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L119]     PRE --> HV["HV<br/>Validate #1?"]
[L120]     HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L121]     HV -->|"Fail"| HV2["HV<br/>Validate #2?"]
[L122]     HV2 -->|"Pass"| FI
[L123]     HV2 -->|"Fail"| HV3["HV<br/>Validate #3?"]
[L124]     HV3 -->|"Pass"| FI
[L125]     HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L126]     FI --> CLD["CLD<br/>Discover Cross-Links"]
[L127]     CLD --> DONE["Node Placed ✓"]
[L128]
```


[L129]

[L130] ##### 5.5.1 CPG++ — Candidate Parent Generator

[L131]

[L132] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L133]

[L134]

```

[L135] def cpg_plus_plus(node_q, forest, config):
[L136]     """Generate ≤k candidate parents for node q."""
[L137]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L138]     domain = classify_domain(node_q)
[L139]     domain_nodes = forest.get_nodes_by_domain(domain)
[L140]
[L141]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L142]     if len(domain_nodes) > config.ann_k:
[L143]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L144]         candidates = [r for r in ann_results if r in domain_nodes]
[L145]     else:
[L146]         candidates = domain_nodes
[L147]
[L148]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L149]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L150]
[L151]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L152]     q_level = estimate_abstraction_level(node_q)
[L153]     candidates = [c for c in candidates
[L154]                   if c.abstraction_level > q_level
[L155]                   and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L156]
[L157]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L158]     candidates = sorted(candidates,
[L159]                         key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L160]
[L161]     return candidates[:config.max_candidates] # Return top 5
[L162]

```

[L163]

[L164] **Overall complexity:** $O(\log N)$ dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L165]

[L166] ##### 5.5.2 PRE — Parent Ranking Engine

[L167]

[L168] Ranks all candidates using a weighted scoring formula:

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file2

[L169] metadata:

[L170] query_result_index: 2

[L171] file_id: file_000000004bc481fdbf9e949ec3b7544b

[L172] version_id: 1

[L173] name: SHIA_RAG_Final_Report(2).md

[L174] mime_type: text/markdown

[L175] surface: library

[L176] score: 0.018294583307361466

[L177] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--15

[L178] snippet:

[L179]

```
[L180] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L181]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L182]     conf = 0.5
[L183]     conf += 0.15 * source.confidence
[L184]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L185]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L186]     return min(conf, 1.0)
[L187]
```

[L188]

[L189] ---

[L190]

[L191] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L192]

[L193] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L194]

[L195] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L196]

[L197] #### Pipeline Overview

[L198]

[L199]

```
[L200] graph LR
[L201]     N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L202]     CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L203]     PRE --> HV{"HV<br/>Validate #1?"}
[L204]     HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L205]     HV -->|"Fail"| HV2{"HV<br/>Validate #2?"}
[L206]     HV2 -->|"Pass"| FI
[L207]     HV2 -->|"Fail"| HV3{"HV<br/>Validate #3?"}
[L208]     HV3 -->|"Pass"| FI
[L209]     HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L210]     FI --> CLD["CLD<br/>Discover Cross-Links"]
[L211]     CLD --> DONE["Node Placed ✓"]
[L212]
```

[L213]

[L214] #### 5.5.1 CPG++ — Candidate Parent Generator

[L215]

[L216] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L217]

[L218]

```

[L219] def cpq_plus_plus(node_q, forest, config):
[L220]     """Generate ≤k candidate parents for node q."""
[L221]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L222]     domain = classify_domain(node_q)
[L223]     domain_nodes = forest.get_nodes_by_domain(domain)
[L224]
[L225]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L226]     if len(domain_nodes) > config.ann_k:
[L227]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L228]         candidates = [r for r in ann_results if r in domain_nodes]
[L229]     else:
[L230]         candidates = domain_nodes
[L231]
[L232]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L233]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L234]
[L235]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L236]     q_level = estimate_abstraction_level(node_q)
[L237]     candidates = [c for c in candidates
[L238]                   if c.abstraction_level > q_level
[L239]                   and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L240]
[L241]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L242]     candidates = sorted(candidates,
[L243]                         key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L244]
[L245]     return candidates[:config.max_candidates] # Return top 5
[L246]

```

[L247]

[L248] **Overall complexity:** $O(\log N)$ dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L249]

[L250] ##### 5.5.2 PRE — Parent Ranking Engine

[L251]

[L252] Ranks all candidates using a weighted scoring formula:

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file3

[L253] metadata:

[L254] query_result_index: 3

[L255] file_id: file_00000000f54081fa942542456259658a

[L256] version_id: 1

[L257] name: Executive Summary(1)

[L258] library_artifact_type: deep_research_report

[L259] mime_type: application/json

[L260] surface: library

[L261] score: 0.018294583307361466

[L262] document_chunk_id: file_00000000f54081fa942542456259658a--1--14

[L263] snippet:

[L264] At runtime, pending relations can be confirmed by consensus (e.g. multiple docs) or via user review.

[L265]

[L266] ### 7. Hierarchy Construction (SHIA Core)

[L267] **Input:** Canonical KnowledgeNodes and accepted relations (from KCE).

[L268] **Output:** A **Knowledge Forest**: one or more concept trees (plus sibling links).

[L269]

[L270] **Goal:** For each node, determine the best parent(s) and place it in the forest. This is done incrementally for all nodes.

We formalize SHIA as the following sub-algorithms:

[L271]

[L272] - **Candidate Parent Generation (CPG++)**: Given a node n , quickly identify a small set of plausible parent nodes from the forest.

[L273] - **Parent Ranking Engine (PRE)**: Score each candidate parent using multiple signals (semantic similarity, hypernym patterns, evidence support, generality) and pick the best.

[L274] - **Hierarchy Validation (HV)**: Verify that adding n under the chosen parent does not create cycles, exceed depth limits, or violate sibling coherence.

[L275] - **Forest Integration (FI)**: Attach n to the forest under its parent (or create new tree if no parent).

[L276] - **Cross-Link Discovery (CLD)**: Add additional “soft links” between nodes (e.g. semantic associations) that do not form the main parent-child tree but could aid retrieval.

[L277]

[L278] We describe each in detail below.

[L279]

[L280] ##### 7.1 Candidate Parent Generation (CPG++)

[L281] **Challenge**: The forest may contain tens of thousands of nodes (all concepts from the corpus). A naive linear search for a parent is infeasible.

[L282]

[L283] **Solution**: CPG++ is a *multi-stage hypothesis generator* that dramatically prunes the search space. For a new or existing node q , it performs:

[L284] 1. **Domain Routing**: Use document metadata or concept taxonomy (if available) to narrow candidate domains. For example, if q came from an OS textbook, restrict to OS-related nodes. Domain labels (from ontology or section context) filter the forest.

[L285] 2. **ANN Retrieval**: Use a vector index (e.g. HNSW) over node embeddings. Query q 's embedding to retrieve top- k nearest existing nodes ($k \approx 20-50$). This runs in $\approx O(\log N)$ time.

[L286] 3. **Relation Filter**: From the top- k neighbors, keep only those with some plausible relation. For instance, if q is a “Scheduling” algorithm, neighbors like “Process” or “Operating System” might pass (if relations exist), while unrelated ones are dropped. We may use the relation set or co-occurrence stats: e.g. keep neighbor p only if LLM suggests a relation $q \rightarrow p$ (is-a/part-of) or vice versa.

[L287] 4. **Abstraction Filter**: Estimate abstraction levels. Compute a genericity score for each candidate (e.g. children count or learned metric). Reject candidates that are too abstract or too specific. For example, if q =“Round Robin”, reject “Operating System” (too high) and “Time Quantum” (too low) at this stage.

[L288] 5. **Evidence Filter**: Optionally, verify candidates against cross-document evidence. If multiple sources have already related q to c , boost score; if none, deprioritize.

[L289]

[L290] The output is a small **candidate set** (e.g. ≤ 5 hypotheses), each with initial scores or confidence. CPG++ never declares the single parent; it only proposes plausible parents for PRE to rank.

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite[turn15file4]`

[L291] metadata:

[L292] query_result_index: 4

[L293] file_id: file_00000000f54081fa942542456259658a

[L294] version_id: 1

[L295] name: Executive Summary(1)

[L296] library_artifact_type: deep_research_report

[L297] mime_type: application/json

[L298] surface: library

[L299] score: 0.01803017294247516

[L300] document_chunk_id: file_00000000f54081fa942542456259658a--1--13

[L301] snippet:

[L302] Using either rule-based extraction (Hearst patterns for hypernymy) or LLM prompts, we refine relations:

[L303] - **Hypernym (is-a)**: Identify "X is a type of Y". Mark $X \rightarrow Y$ "is-a".

[L304] - **Part-of / Contains**: "X consists of Y".

[L305] - **Cause-effect, uses, synonyms, etc.** The main goal is to detect parent-child or sibling-type edges.

[L306] - Relations may be **directed** with types. Each edge gets a confidence (LLM-provided probability or similarity score).

[L307]

[L308] We aggregate relation mentions from all documents. If multiple sources assert the same relation, confidence increases.

[L309]

[L310] *API*: `detect_relations(concept) -> list of (relation, target_concept, confidence)`.

[L311]

[L312] **Data**: We update nodes: e.g. node "Round Robin" gains a "is-a $\rightarrow$ CPU Scheduling" relation.

[L313]

[L314] ### 6. Knowledge Confidence Engine (KCE)

[L315] **Input**: All extracted relations and definitions with confidence estimates.

[L316] **Output**: Filtered set of reliable relations for hierarchy building; uncertain cases flagged or queued for review.

[L317]

[L318] To prevent "garbage in" from corrupting the structure, we apply thresholds:

[L319] - If confidence $\geq$ *high_threshold* (e.g. 0.9), accept relation automatically.

[L320] - If confidence in [low, high), mark as *pending* for later re-evaluation.

[L321] - If below *low_threshold* (e.g. <0.5), reject or set aside.

[L322]

[L323] We also check consistency: conflicting high-confidence relations (e.g. A is-a B and A is-a C where $B \neq C$) are flagged.

[L324]

[L325] *Pseudocode (KCE)*:

[L326]

```
[L327] for each relation r in all_relations:
[L328]     if r.confidence >= HIGH:
[L329]         accepted_relations.add(r)
[L330]     else if r.confidence >= LOW:
[L331]         pending_relations.add(r)
[L332]     else:
[L333]         reject r
[L334]
```

[L335] Nodes carry a confidence score based on their sources. Relations are stored with these labels.

[L336]

[L337] At runtime, pending relations can be confirmed by consensus (e.g. multiple docs) or via user review.

[L338]

[L339] ### 7. Hierarchy Construction (SHIA Core)

[L340] **Input**: Canonical KnowledgeNodes and accepted relations (from KCE).

[L341] **Output**: A **Knowledge Forest**: one or more concept trees (plus sibling links).

[L342]

[L343] **Goal**: For each node, determine the best parent(s) and place it in the forest. This is done incrementally for all nodes. We formalize SHIA as the following sub-algorithms:

[L344]

[L345] - **Candidate Parent Generation (CPG++)**: Given a node n , quickly identify a small set of plausible parent nodes from the forest.

[L346] - **Parent Ranking Engine (PRE)**: Score each candidate parent using multiple signals (semantic similarity, hypernym

patterns, evidence support, generality) and pick the best.

[L347] - **Hierarchy Validation (HV):** Verify that adding n under the chosen parent does not create cycles, exceed depth limits, or violate sibling coherence.

[L348] - **Forest Integration (FI):** Attach n to the forest under its parent (or create new tree if no parent).

[L349] - **Cross-Link Discovery (CLD):** Add additional "soft links" between nodes (e.g. semantic associations) that do not form the main parent-child tree but could aid retrieval.

[L350]

[L351] We describe each in detail below.

[L352]

[L353] ##### 7.1 Candidate Parent Generation (CPG++)

[L354] **Challenge:** The forest may contain tens of thousands of nodes (all concepts from the corpus). A naive linear search for a parent is infeasible.

[L355]

[L356] **Solution:** CPG++ is a *multi-stage hypothesis generator* that dramatically prunes the search space.

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file5

[L357] metadata:

[L358] query_result_index: 5

[L359] file_id: file_00000000a0a481fab4c739c73d87551b

[L360] version_id: 1

[L361] name: SHIA_RAG_Complete_Report(7).md

[L362] mime_type: text/markdown

[L363] surface: library

[L364] score: 0.017526140684410645

[L365] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--40

[L366] snippet:

[L367] }},

[L368])

[L369] edges.append(edge)

[L370]

[L371] return edges

[L372]

[L373] def calculate_edge_confidence(raw_rel: dict, source, target) -> float:

[L374] """Calculate confidence for a relation edge based on multiple signals."""

[L375] conf = 0.5 # Base

[L376]

[L377] # Signal 1: Source node confidence

[L378] conf += 0.15 source.confidence

[L379]

[L380] # Signal 2: Evidence text length (longer = more detailed = more reliable)

[L381] evidence_len = len(raw_rel.get('evidence', ''))

[L382] conf += 0.10 min(evidence_len / 200, 1.0)

[L383]

[L384] # Signal 3: Semantic similarity between source and evidence

[L385] # (Higher similarity = evidence is more relevant)

[L386] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))

```
[L387]
[L388] return min(conf, 1.0)
[L389]
```

```
[L390]
[L391] ---
[L392]
[L393] ### 7.6 Layer 5: SHIA Core (Corrected Pipeline)
[L394]
[L395]
```

mermaid

```
[L396] graph LR
[L397] N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L398] CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L399] PRE --> HV{"HV<br/>Validate #1?"}
[L400] HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L401] HV -->|"Fail"| HV2{"HV<br/>Validate #2?"}
[L402] HV2 -->|"Pass"| FI
[L403] HV2 -->|"Fail"| HV3{"HV<br/>Validate #3?"}
[L404] HV3 -->|"Pass"| FI
[L405] HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L406] FI --> CLD["CLD<br/>Discover Cross-Links"]
[L407] CLD --> DONE["Node Placed ✓"]
[L408]
```

```
[L409]
[L410] #### CPG++ (Corrected)
[L411]
[L412]
```

python

```
[L413] def cpg_plus_plus(node_q, forest, config):
[L414] """Generate ≤k candidate parents for node q."""
[L415] candidates = []
[L416]
[L417] # Stage 1: Domain Routing (O(1))
[L418] domain = classify_domain(node_q) # See implementation below
[L419] domain_nodes = forest.get_nodes_by_domain(domain)
[L420]
[L421] # Stage 2: ANN Retrieval (O(log N))
[L422] if len(domain_nodes) > config.ann_k:
[L423] ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L424] candidates = [r for r in ann_results if r in domain_nodes]
[L425] else:
[L426] candidates = domain_nodes
[L427]
[L428] # Stage 3: Relation Filter
[L429] candidates = [c for c in candidates
[L430] if has_plausible_relation(node_q, c)]
[L431]
[L432] # Stage 4: Abstraction Filter (FIXED: use estimated level)
[L433] q_level = estimate_abstraction_level(node_q) # Heuristic, not tree depth
[L434] candidates = [c for c in candidates
```

```
[L435] if abs(q_level - c.abstraction_level) < config.max_level_gap
[L436] and c.abstraction_level > q_level] # Parent must be MORE abstract
[L437]
[L438] # Stage 5: Evidence Filter
[L439] candidates = sorted(candidates,
[L440] key=lambda c: cross_doc_evidence_count(node_q, c),
[L441] reverse=True)
[L442]
[L443] return candidates[:config.max_candidates] # Return top 5
[L444]
```

```
[L445]
[L446] ##### `classify_domain(node)` Implementation
[L447]
[L448]
```

python

```
[L449] def classify_domain(node):
[L450] """Classify a node into a domain for CPG++ Stage 1 routing.
[L451] Uses keyword matching first (fast), falls back to LLM if ambiguous."""
[L452]
[L453] DOMAIN_KEYWORDS = {
[L454] "Operating Systems": ["cpu", "scheduling", "process", "thread", "memory",
[L455] "kernel", "deadlock",
```

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file6

```
[L456] metadata:
[L457] query_result_index: 6
[L458] file_id: file_000000004a3881fab7e320a5ee7ca1ad
[L459] version_id: 1
[L460] name: SHIA_RAG_Complete_Report.md
[L461] mime_type: text/markdown
[L462] surface: library
[L463] score: 0.01728575606902603
[L464] document_chunk_id: file_000000004a3881fab7e320a5ee7ca1ad--1--17
[L465] snippet:
[L466] "incoming_edges": ["EDGE_001"],
[L467] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L468] "evidence": [
[L469] {
[L470] "text": "Round Robin is a preemptive scheduling algorithm.",
[L471] "source_doc": "DOC_000001",
[L472] "source_block": "BLK_000234",
[L473] "page": 42
[L474] }
[L475] ],
[L476] "embeddings": [0.123, -0.456, .],
[L477] "confidence": 0.92,
[L478] "abstraction_level": 0.35,
[L479] "statistics": {
```



```
[L480] "access_count": 0,  
[L481] "update_count": 1,  
[L482] "merge_count": 0,  
[L483] "last_access": null  
[L484] },  
[L485] "version": {  
[L486] "current": 1,  
[L487] "revisions": []  
[L488] }  
[L489] }  
[L490]
```

```
[L491]  
[L492] ##### KnowledgeEdge (Two Types)  
[L493]  
[L494]
```

json

```
[L495] {  
[L496] "edge_id": "EDGE_001",  
[L497] "source_node": "NODE_000789",  
[L498] "target_node": "NODE_000456",  
[L499] "edge_class": "HIERARCHICAL",  
[L500] "relation_type": "IS_A",  
[L501] "confidence": 0.95,  
[L502] "weight": 1.0,  
[L503] "evidence": [...],  
[L504] "metadata": {"source_docs": ["DOC_001"]}  
[L505] }  
[L506]
```

```
[L507]  
[L508]
```

json

```
[L509] {  
[L510] "edge_id": "EDGE_002",  
[L511] "source_node": "NODE_000789",  
[L512] "target_node": "NODE_000890",  
[L513] "edge_class": "SEMANTIC",  
[L514] "relation_type": "USES",  
[L515] "confidence": 0.87,  
[L516] "weight": 1.0,  
[L517] "evidence": [...],  
[L518] "metadata": {"source_docs": ["DOC_001", "DOC_003"]}  
[L519] }  
[L520]
```

```
[L521]
[L522] > [!IMPORTANT]
[L523] > **Two edge classes solve the Tree-vs-Graph contradiction:**
[L524] > - `HIERARCHICAL` edges form an acyclic forest (enforced by HV)
[L525] > - `SEMANTIC` edges form an unrestricted graph (cycles allowed)
[L526]
[L527] ---
[L528]
[L529] ### 7.2 Layer 5: SHIA Core (Corrected Pipeline)
[L530]
[L531]
```

mermaid

```
[L532] graph LR
[L533] N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L534] CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L535] PRE --> HV["HV<br/>Validate #1?"]
[L536] HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L537] HV -->|"Fail"| HV2["HV<br/>Validate #2?"]
[L538] HV2 -->|"Pass"| FI
[L539] HV2 -->|"Fail"| HV3["HV<br/>Validate #3?"]
[L540] HV3 -->|"Pass"| FI
[L541] HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L542] FI --> CLD["CLD<br/>Discover Cross-Links"]
[L543] CLD --> DONE["Node Placed ✓"]
[L544]
```

```
[L545]
[L546] #### CPG++ (Corrected)
[L547]
[L548]
```

python

```
[L549] def cpg_plus_plus(node_q, forest, config):
[L550] """Generate ≤k candidate parents for node q."""
[L551] candidates = []
[L552]
[L553] # Stage 1: Domain Routing (O(1))
[L554] domain = classify_domain(node_q) # LLM or keyword-based
[L555] domain_nodes = forest.get_nodes_by_domain(domain)
[L556]
[L557] # Stage 2: ANN Retrieval (O(log N))
[L558] if len(domain_nodes) > config.ann_k:
[L559] ann_results = vector_index.search(node_q.embedding, k=config.
```

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: `filecite`turn15file7

```
[L560] metadata:
[L561] query_result_index: 7
[L562] file_id: file_0000000033088207b45d2ae58cbd406c
[L563] version_id: 1
[L564] name: SHIA_RAG_Complete_Report(8).md
[L565] mime_type: text/markdown
[L566] surface: library
```

```

[L567] score: 0.017052655835925797
[L568] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--39
[L569] snippet:
[L570] raw_relations:
[L571] rel_type = raw_rel['type'].upper()
[L572] target_name = raw_rel['target'].lower()
[L573]
[L574] # Step 1: Validate relation type against ontology
[L575] if rel_type not in all_valid_types:
[L576] rel_type = infer_relation_type(raw_rel['evidence'], ontology)
[L577] if rel_type is None:
[L578] continue # Skip unrecognizable relation
[L579]
[L580] # Step 2: Resolve target node
[L581] target_node = node_map.get(target_name)
[L582] if target_node is None:
[L583] continue # Target concept not in our nodes (skip or create placeholder)
[L584]
[L585] # Step 3: Check for duplicate edges
[L586] if edge_exists(edges, node.node_id, target_node.node_id, rel_type):
[L587] # Merge evidence into existing edge
[L588] merge_edge_evidence(edges, node.node_id, target_node.node_id, raw_rel)
[L589] continue
[L590]
[L591] # Step 4: Determine edge class from ontology
[L592] edge_class = 'HIERARCHICAL' if rel_type in ontology.get('edge_class_mapping', {}).get('hierarchical', []) else
'SEMANTIC'
[L593]
[L594] # Step 5: Calculate confidence
[L595] confidence = calculate_edge_confidence(raw_rel, node, target_node)
[L596]
[L597] # Step 6: Create edge
[L598] edge = KnowledgeEdge(
[L599] edge_id=f"EDGE_{len(edges):06d}",
[L600] source_node=node.node_id,
[L601] target_node=target_node.node_id,
[L602] edge_class=edge_class,
[L603] relation_type=rel_type,
[L604] confidence=confidence,
[L605] weight=1.0, # Initial neutral weight
[L606] evidence=[{
[L607] 'text': raw_rel['evidence'],
[L608] 'source_doc': node.evidence[0].source_doc if node.evidence else None,
[L609] }],
[L610] )
[L611] edges.append(edge)
[L612]
[L613] return edges
[L614]
[L615] def calculate_edge_confidence(raw_rel: dict, source, target) -> float:

```

```
[L616] """Calculate confidence for a relation edge based on multiple signals."""
[L617] conf = 0.5 # Base
[L618]
[L619] # Signal 1: Source node confidence
[L620] conf += 0.15 source.confidence
[L621]
[L622] # Signal 2: Evidence text length (longer = more detailed = more reliable)
[L623] evidence_len = len(raw_rel.get('evidence', ''))
[L624] conf += 0.10 min(evidence_len / 200, 1.0)
[L625]
[L626] # Signal 3: Semantic similarity between source and evidence
[L627] # (Higher similarity = evidence is more relevant)
[L628] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L629]
[L630] return min(conf, 1.0)
[L631]
```

```
[L632]
[L633] ---
[L634]
[L635] ### 7.6 Layer 5: SHIA Core (Corrected Pipeline)
[L636]
[L637]
```

mermaid

```
[L638] graph LR
[L639] N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L640] CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L641] PRE --> HV["HV<br/>Validate #1?"]
[L642] HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L643] HV -->|"Fail"| HV2["HV<br/>Validate #2?"]
[L644] HV2 -->|"Pass"| FI
[L645] HV2 -->|"Fail"| HV3["HV<br/>Validate #3?"]
[L646] HV3 -->|"Pass"| FI
[L647] HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L648] FI --> CLD["CLD<br/>Discover Cross-Links"]
[L649] CLD --> DONE["Node Placed ✓"]
[L650] ``
[L651]
[L652] #### CPG++ (Corrected)
```

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: fileciteturn15file8

```
[L653] metadata:
[L654] query_result_index: 8
[L655] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L656] version_id: 1
[L657] name: SHIA_RAG_Final_Report(4).md
[L658] mime_type: text/markdown
[L659] surface: library
[L660] score: 0.016826513818739005
```

[L661] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--26

[L662] snippet:

[L663] ---

[L664]

[L665] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L666]

[L667] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L668]

[L669] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L670]

[L671] #### Pipeline Overview

[L672]

[L673]

```
[L674] graph LR
[L675]   N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L676]   CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L677]   PRE --> HV{"HV<br/>Validate #1?"}
[L678]   HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L679]   HV -->|"Fail"| HV2{"HV<br/>Validate #2?"}
[L680]   HV2 -->|"Pass"| FI
[L681]   HV2 -->|"Fail"| HV3{"HV<br/>Validate #3?"}
[L682]   HV3 -->|"Pass"| FI
[L683]   HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L684]   FI --> CLD["CLD<br/>Discover Cross-Links"]
[L685]   CLD --> DONE["Node Placed ✓"]
[L686]
```

[L687]

[L688] #### 5.5.1 CPG++ — Candidate Parent Generator

[L689]

[L690] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L691]

[L692]

```

[L693] def cpg_plus_plus(node_q, forest, config):
[L694]     """Generate ≤k candidate parents for node q."""
[L695]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L696]     domain = classify_domain(node_q)
[L697]     domain_nodes = forest.get_nodes_by_domain(domain)
[L698]
[L699]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L700]     if len(domain_nodes) > config.ann_k:
[L701]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L702]         candidates = [r for r in ann_results if r in domain_nodes]
[L703]     else:
[L704]         candidates = domain_nodes
[L705]
[L706]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L707]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L708]
[L709]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L710]     # estimate_abstraction_level uses multi-factor heuristic + optional LLM fallback
[L711]     q_level = estimate_abstraction_level(node_q)
[L712]     candidates = [c for c in candidates
[L713]                   if c.abstraction_level > q_level
[L714]                   and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L715]
[L716]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L717]     candidates = sorted(candidates,
[L718]                         key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L719]
[L720]     return candidates[:config.max_candidates] # Return top 5
[L721]

```

[L722]

[L723] **Overall complexity:** $O(\log N)$ dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L724]

[L725] **Abstraction Level Estimation:**

[L726]

[L727] ```python

[L728] def estimate_abstraction_level(node) -> float:

[L729] """Multi-factor heuristic for estimating abstraction level of unplaced nodes.

[L730] Returns a score in [0.0, 1.0] where higher = more abstract.

[L731]

[L732] NOTE: Simple name-length alone is insufficient (e.g., 'CPU' is short but

[L733] context-dependent). We combine multiple signals for robustness."""

[L734] score = 0.0

[L735]

[L736] # Factor 1: Name word count (shorter multi-word names tend to be more abstract)

[L737] word_count = len(node.name.split())

[L738] if word_count <= 2:

[L739] score += 0.15

[L740] elif word_count >= 4:

[L741] score -= 0.

DEEP RESEARCH 24/08/2026, 00:41:14

Citation Marker: turn15file9

[L742] metadata:

[L743] query_result_index: 9

[L744] file_id: file_00000000fb2c81fd82f14a7a48993554

[L745] version_id: 1
 [L746] name: SHIA_RAG_Final_Report.md
 [L747] mime_type: text/markdown
 [L748] surface: library
 [L749] score: 0.01660702303735182
 [L750] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--15
 [L751] snippet:
 [L752]

```
[L753] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L754]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L755]     conf = 0.5
[L756]     conf += 0.15 * source.confidence
[L757]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L758]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L759]     return min(conf, 1.0)
[L760]
```

[L761]
 [L762] ---
 [L763]
 [L764] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction
 [L765]
 [L766] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.
 [L767]
 [L768] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest
 [L769]
 [L770] #### Pipeline Overview
 [L771]
 [L772]

```
[L773] graph LR
[L774]     N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L775]     CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L776]     PRE --> HV["HV<br/>Validate #1?"]
[L777]     HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L778]     HV -->|"Fail"| HV2["HV<br/>Validate #2?"]
[L779]     HV2 -->|"Pass"| FI
[L780]     HV2 -->|"Fail"| HV3["HV<br/>Validate #3?"]
[L781]     HV3 -->|"Pass"| FI
[L782]     HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L783]     FI --> CLD["CLD<br/>Discover Cross-Links"]
[L784]     CLD --> DONE["Node Placed ✓"]
[L785]
```

[L786]
 [L787] #### 5.5.1 CPG++ — Candidate Parent Generator
 [L788]
 [L789] Generates up to 5 candidate parents for a new node using a multi-stage filter:
 [L790]
 [L791]

```

[L792] def cpg_plus_plus(node_q, forest, config):
[L793]     """Generate ≤k candidate parents for node q."""
[L794]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L795]     domain = classify_domain(node_q)
[L796]     domain_nodes = forest.get_nodes_by_domain(domain)
[L797]
[L798]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L799]     if len(domain_nodes) > config.ann_k:
[L800]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L801]         candidates = [r for r in ann_results if r in domain_nodes]
[L802]     else:
[L803]         candidates = domain_nodes
[L804]
[L805]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L806]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L807]
[L808]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L809]     q_level = estimate_abstraction_level(node_q)
[L810]     candidates = [c for c in candidates
[L811]                    if c.abstraction_level > q_level
[L812]                    and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L813]
[L814]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L815]     candidates = sorted(candidates,
[L816]                          key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L817]
[L818]     return candidates[:config.max_candidates] # Return top 5
[L819]

```

[L820]

[L821] **Overall complexity:** $O(\log N)$ dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L822]

[L823] ##### 5.5.2 PRE — Parent Ranking Engine

[L824]

[L825] Ranks all candidates using a weighted scoring formula:

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: `filecite`turn16file0

[L1] metadata:

[L2] query_result_index: 0

[L3] file_id: file_00000000c5a481faa8d8675f97f2f7b5

[L4] version_id: 1

[L5] name: Executive Summary (1)(2)(1).pdf

[L6] mime_type: application/pdf

[L7] surface: library

[L8] score: 0.019955849889624726

[L9] document_chunk_id: file_00000000c5a481faa8d8675f97f2f7b5:0--1--17

[L10] snippet:

[L11] •

[L12] •

[L13] $\text{Score}(q, p) = \alpha \text{Sim}(q, p) + \beta \text{IsA}(q, p) + \gamma \text{Evidence}(q, p) + \delta \text{General}(p) - \zeta \text{Conflict}(q, p)$,

[L14] $\alpha, \beta, \gamma, \delta, \zeta$

[L15] •

[L16] •

[L17] 11Sibling Consistency: Optionally, check that q's topic matches its new siblings. For example, a

[L18] "Scheduling Algorithm" node should not be placed with unrelated siblings. If incongruent, we could

- [L19] lower confidence or reconsider.
- [L20] Abstraction Check: Ensure p remains reasonably general and q not too broad. This overlaps with
- [L21] PRE but can enforce global depth ratios.
- [L22] If any check fails, HV asks PRE for the next-best parent. If none remain, q is placed as a new root.
- [L23] 7.4 Forest Integration (FI)
- [L24] Action: Attach q under its approved parent p in the Knowledge Forest.
- [L25] Update p.children.
- [L26] Update q.ancestors to include p and all p's ancestors.
- [L27] Merge any duplicate nodes: if q was already partially in the forest (e.g. from previous doc), unify
- [L28] structures.
- [L29] Update embeddings or store q's embedding in the index.
- [L30] This step may also add q to the vector database for future retrieval. It is essentially tree-insertion (or new root creation).
- [L31] 7.5 Cross-Link Discovery (CLD)
- [L32] Objective: After the main parent-child tree is built, we add secondary semantic links (forming a DAG) to
- [L33] capture associations not covered by the tree. For example, connect related nodes ("Queue" to "Round
- [L34] Robin" even if not parent/child).
- [L35] Approaches:
- [L36] For each node n, use ANN search again to find a few high-similarity nodes that are not its ancestor
- [L37] or descendant. Add bidirectional "related-to" edges if similarity exceeds a threshold.
- [L38] Use relation extraction: if n appears in a relation with m outside the tree, link them.
- [L39] Use knowledge bases: e.g. Wikipedia links.
- [L40] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.
- [L41] Context Assembler may include one-hop neighbors via CLD).
- [L42] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and
- [L43] additional edges are "related-to."
- [L44] Data Structures and Indexing
- [L45] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent
- [L46] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),
- [L47] embedding vector, and a version/timestamp.
- [L48] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)
- [L49] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.
- [L50] NetworkX, Neo4j).
- [L51] •
- [L52] •
- [L53] •
- [L54] •
- [L55] •
- [L56] •
- [L57] •
- [L58] •
- [L59] •
- [L60] •
- [L61] •
- [L62] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node
- [L63] embeddings. This supports CPG++ search and semantic neighbor queries.
- [L64] Ancillary Indexes:
- [L65] Entity-to-Node map: For quick lookups by name/alias.
- [L66] Relation Index: If needed, a mapping from relation type to node pairs.
- [L67] Metadata store: (document IDs, source text) for evidence retrieval.

[L68] All indices are updated incrementally as nodes are added.
 [L69] Retrieval and Context Assembly
 [L70] 10. Query Analyzer
 [L71] Input: Natural language user query.
 [L72] Output: Query intent and candidate target concepts.
 [L73] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g.

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: `[[filecite]]turn16file1`

[L74] metadata:
 [L75] query_result_index: 1
 [L76] file_id: file_00000000383081fa8435ff2a7b4a6f5f
 [L77] version_id: 1
 [L78] name: Executive Summary (1)(5)(1).pdf
 [L79] mime_type: application/pdf
 [L80] surface: library
 [L81] score: 0.019912985274431055
 [L82] document_chunk_id: file_00000000383081fa8435ff2a7b4a6f5f:0--1--17
 [L83] snippet:
 [L84] •
 [L85] •
 [L86] $\text{Score}(q, p) = \alpha \text{Sim}(q, p) + \beta \text{ISA}(q, p) + \gamma \text{Evidence}(q, p) + \delta \text{General}(p) - \zeta \text{Conflict}(q, p)$,
 [L87] $\alpha, \beta, \gamma, \delta, \zeta$
 [L88] •
 [L89] •
 [L90] 11Sibling Consistency: Optionally, check that q's topic matches its new siblings. For example, a
 [L91] "Scheduling Algorithm" node should not be placed with unrelated siblings. If incongruent, we could
 [L92] lower confidence or reconsider.
 [L93] Abstraction Check: Ensure p remains reasonably general and q not too broad. This overlaps with
 [L94] PRE but can enforce global depth ratios.
 [L95] If any check fails, HV asks PRE for the next-best parent. If none remain, q is placed as a new root.
 [L96] 7.4 Forest Integration (FI)
 [L97] Action: Attach q under its approved parent p in the Knowledge Forest.
 [L98] Update p.children.
 [L99] Update q.ancestors to include p and all p's ancestors.
 [L100] Merge any duplicate nodes: if q was already partially in the forest (e.g. from previous doc), unify
 [L101] structures.
 [L102] Update embeddings or store q's embedding in the index.
 [L103] This step may also add q to the vector database for future retrieval. It is essentially tree-insertion (or new-root creation).
 [L104] 7.5 Cross-Link Discovery (CLD)
 [L105] Objective: After the main parent-child tree is built, we add secondary semantic links (forming a DAG) to
 [L106] capture associations not covered by the tree. For example, connect related nodes ("Queue" to "Round
 [L107] Robin" even if not parent/child).
 [L108] Approaches:
 [L109] For each node n, use ANN search again to find a few high-similarity nodes that are not its ancestor
 [L110] or descendant. Add bidirectional "related-to" edges if similarity exceeds a threshold.
 [L111] Use relation extraction: if n appears in a relation with m outside the tree, link them.
 [L112] Use knowledge bases: e.g. Wikipedia links.

[L113] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g. [L114] Context Assembler may include one-hop neighbors via CLD).

[L115] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and [L116] additional edges are "related-to."

[L117] Data Structures and Indexing

[L118] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent [L119] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences), [L120] embedding vector, and a version/timestamp.

[L121] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges) [L122] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g. [L123] NetworkX, Neo4j).

[L124] •

[L125] •

[L126] •

[L127] •

[L128] •

[L129] •

[L130] •

[L131] •

[L132] •

[L133] •

[L134] •

[L135] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node [L136] embeddings. This supports CPG++ search and semantic neighbor queries.

[L137] Ancillary Indexes:

[L138] Entity-to-Node map: For quick lookups by name/alias.

[L139] Relation Index: If needed, a mapping from relation type to node pairs.

[L140] Metadata store: (document IDs, source text) for evidence retrieval.

[L141] All indices are updated incrementally as nodes are added.

[L142] Retrieval and Context Assembly

[L143] 10. Query Analyzer

[L144] Input: Natural language user query.

[L145] Output: Query intent and candidate target concepts.

[L146] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g.

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: □filecite□turn16file2□

[L147] metadata:

[L148] query_result_index: 2

[L149] file_id: file_00000000488481fa988d49d745190c4d

[L150] version_id: 1

[L151] name: first Pdf.pdf

[L152] mime_type: application/pdf

[L153] surface: library

[L154] score: 0.019512583625358394

[L155] document_chunk_id: file_00000000488481fa988d49d745190c4d:0--1--17

[L156] snippet:

[L157] •

[L158] •

- [L159] $\text{Score}(q, p) = \alpha \text{Sim}(q, p) + \beta \text{IsA}(q, p) + \gamma \text{Evidence}(q, p) + \delta \text{General}(p) - \zeta \text{Conflict}(q, p)$,
- [L160] $\alpha, \beta, \gamma, \delta, \zeta$
- [L161] •
- [L162] •
- [L163] 11Sibling Consistency: Optionally, check that q's topic matches its new siblings. For example, a
- [L164] "Scheduling Algorithm" node should not be placed with unrelated siblings. If incongruent, we could
- [L165] lower confidence or reconsider.
- [L166] Abstraction Check: Ensure p remains reasonably general and q not too broad. This overlaps with
- [L167] PRE but can enforce global depth ratios.
- [L168] If any check fails, HV asks PRE for the next-best parent. If none remain, q is placed as a new root.
- [L169] 7.4 Forest Integration (FI)
- [L170] Action: Attach q under its approved parent p in the Knowledge Forest.
- [L171] Update p.children.
- [L172] Update q.ancestors to include p and all p's ancestors.
- [L173] Merge any duplicate nodes: if q was already partially in the forest (e.g. from previous doc), unify
- [L174] structures.
- [L175] Update embeddings or store q's embedding in the index.
- [L176] This step may also add q to the vector database for future retrieval. It is essentially tree-insertion (or new-root creation).
- [L177] 7.5 Cross-Link Discovery (CLD)
- [L178] Objective: After the main parent-child tree is built, we add secondary semantic links (forming a DAG) to
- [L179] capture associations not covered by the tree. For example, connect related nodes ("Queue" to "Round
- [L180] Robin" even if not parent/child).
- [L181] Approaches:
- [L182] For each node n, use ANN search again to find a few high-similarity nodes that are not its ancestor
- [L183] or descendant. Add bidirectional "related-to" edges if similarity exceeds a threshold.
- [L184] Use relation extraction: if n appears in a relation with m outside the tree, link them.
- [L185] Use knowledge bases: e.g. Wikipedia links.
- [L186] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.
- [L187] Context Assembler may include one-hop neighbors via CLD).
- [L188] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and
- [L189] additional edges are "related-to."
- [L190] Data Structures and Indexing
- [L191] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent
- [L192] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),
- [L193] embedding vector, and a version/timestamp.
- [L194] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)
- [L195] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.
- [L196] NetworkX, Neo4j).
- [L197] •
- [L198] •
- [L199] •
- [L200] •
- [L201] •
- [L202] •
- [L203] •
- [L204] •
- [L205] •
- [L206] •
- [L207] •

[L208] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node
 [L209] embeddings. This supports CPG++ search and semantic neighbor queries.
 [L210] Ancillary Indexes:
 [L211] Entity-to-Node map: For quick lookups by name/alias.
 [L212] Relation Index: If needed, a mapping from relation type to node pairs.
 [L213] Metadata store: (document IDs, source text) for evidence retrieval.
 [L214] All indices are updated incrementally as nodes are added.
 [L215] Retrieval and Context Assembly
 [L216] 10. Query Analyzer
 [L217] Input: Natural language user query.
 [L218] Output: Query intent and candidate target concepts.
 [L219] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g.

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: `filecite`turn16file3

[L220] metadata:
 [L221] query_result_index: 3
 [L222] file_id: file_00000000f54081fa942542456259658a
 [L223] version_id: 1
 [L224] name: Executive Summary(1)
 [L225] library_artifact_type: deep_research_report
 [L226] mime_type: application/json
 [L227] surface: library
 [L228] score: 0.018452380952380953
 [L229] document_chunk_id: file_00000000f54081fa942542456259658a--1--18
 [L230] snippet:
 [L231] - **Cycle Check:** Do a DFS from p upwards; if we encounter q , adding $p \rightarrow q$ would create a loop. Reject if so.
 [L232] - **Depth Limit:** If p is already at maximum allowed depth or q 's subtree would exceed a depth budget, we might restrict the depth (to avoid extremely deep chains).
 [L233] - **Sibling Consistency:** Optionally, check that q 's topic matches its new siblings. For example, a "Scheduling Algorithm" node should not be placed with unrelated siblings. If incongruent, we could lower confidence or reconsider.
 [L234] - **Abstraction Check:** Ensure p remains reasonably general and q not too broad. This overlaps with PRE but can enforce global depth ratios.
 [L235]
 [L236] If any check fails, HV asks PRE for the next-best parent. If none remain, q is placed as a new root.
 [L237]
 [L238] ##### 7.4 Forest Integration (FI)
 [L239] **Action:** Attach q under its approved parent p in the Knowledge Forest.
 [L240] - Update $p.children$.
 [L241] - Update $q.ancestors$ to include p and all p 's ancestors.
 [L242] - Merge any duplicate nodes: if q was already partially in the forest (e.g. from previous doc), unify structures.
 [L243] - Update embeddings or store q 's embedding in the index.
 [L244]
 [L245] This step may also add q to the vector database for future retrieval. It is essentially tree-insertion (or new-root creation).
 [L246]
 [L247] ##### 7.5 Cross-Link Discovery (CLD)
 [L248] **Objective:** After the main parent-child tree is built, we add **secondary semantic links** (forming a DAG) to capture associations not covered by the tree. For example, connect related nodes ("Queue" to "Round Robin" even if not

parent/child).

[L249]

[L250] Approaches:

[L251] - For each node n , use ANN search again to find a few high-similarity nodes that are **not** its ancestor or descendant. Add bidirectional "related-to" edges if similarity exceeds a threshold.

[L252] - Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L253] - Use knowledge bases: e.g. Wikipedia links.

[L254]

[L255] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g. Context Assembler may include one-hop neighbors via CLD).

[L256]

[L257] **Data Structure:** The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and additional edges are "related-to."

[L258]

[L259] **## Data Structures and Indexing**

[L260]

[L261] - **Knowledge Node Object:** Stores concept name, aliases, definition text, children list, parent pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences), embedding vector, and a version/timestamp.

[L262] - **Knowledge Forest:** A collection of nodes. Represented as either an adjacency list (for tree edges) plus extra edges for related links. We can use a graph database or an in-memory graph (e.g. NetworkX, Neo4j).

[L263] - **Vector Index:** We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the **node embeddings**. This supports CPG++ search and semantic neighbor queries.

[L264] - **Ancillary Indexes:**

[L265] - *Entity-to-Node map:* For quick lookups by name/alias.

[L266] - *Relation Index:* If needed, a mapping from relation type to node pairs.

[L267] - *Metadata store:* (document IDs, source text) for evidence retrieval.

[L268]

[L269] All indices are updated incrementally as nodes are added.

[L270]

[L271] **## Retrieval and Context Assembly**

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: □filecite□turn16file4□

[L272] metadata:

[L273] query_result_index: 4

[L274] file_id: file_00000000fb2c81fd82f14a7a48993554

[L275] version_id: 1

[L276] name: SHIA_RAG_Final_Report.md

[L277] mime_type: text/markdown

[L278] surface: library

[L279] score: 0.018294583307361466

[L280] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--15

[L281] snippet:

[L282]

```
[L283] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L284]     """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L285]     conf = 0.5
[L286]     conf += 0.15 * source.confidence
[L287]     conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L288]     conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L289]     return min(conf, 1.0)
[L290]
```

[L291]

[L292] ---

[L293]

[L294] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L295]

[L296] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L297]

[L298] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L299]

[L300] #### Pipeline Overview

[L301]

[L302]

```
[L303] graph LR
[L304]     N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L305]     CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L306]     PRE --> HV{"HV<br/>Validate #1?"}
[L307]     HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L308]     HV -->|"Fail"| HV2{"HV<br/>Validate #2?"}
[L309]     HV2 -->|"Pass"| FI
[L310]     HV2 -->|"Fail"| HV3{"HV<br/>Validate #3?"}
[L311]     HV3 -->|"Pass"| FI
[L312]     HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L313]     FI --> CLD["CLD<br/>Discover Cross-Links"]
[L314]     CLD --> DONE["Node Placed ✓"]
[L315]
```

[L316]

[L317] #### 5.5.1 CPG++ — Candidate Parent Generator

[L318]

[L319] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L320]

[L321]

```

[L322] def cpq_plus_plus(node_q, forest, config):
[L323]     """Generate ≤k candidate parents for node q."""
[L324]     # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L325]     domain = classify_domain(node_q)
[L326]     domain_nodes = forest.get_nodes_by_domain(domain)
[L327]
[L328]     # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L329]     if len(domain_nodes) > config.ann_k:
[L330]         ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L331]         candidates = [r for r in ann_results if r in domain_nodes]
[L332]     else:
[L333]         candidates = domain_nodes
[L334]
[L335]     # Stage 3: Relation Filter – keep only nodes with plausible relations
[L336]     candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L337]
[L338]     # Stage 4: Abstraction Filter – parent must be MORE abstract
[L339]     q_level = estimate_abstraction_level(node_q)
[L340]     candidates = [c for c in candidates
[L341]                   if c.abstraction_level > q_level
[L342]                   and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L343]
[L344]     # Stage 5: Evidence Filter – prioritize cross-document evidence
[L345]     candidates = sorted(candidates,
[L346]                         key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L347]
[L348]     return candidates[:config.max_candidates] # Return top 5
[L349]

```

[L350]

[L351] **Overall complexity:** $O(\log N)$ dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L352]

[L353] ##### 5.5.2 PRE — Parent Ranking Engine

[L354]

[L355] Ranks all candidates using a weighted scoring formula:

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: □filecite□turn16file5□

[L356] metadata:

[L357] query_result_index: 5

[L358] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L359] version_id: 1

[L360] name: SHIA_RAG_Final_Report(4).md

[L361] mime_type: text/markdown

[L362] surface: library

[L363] score: 0.018294583307361466

[L364] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--29

[L365] snippet:

[L366] 3, 0.6] range)

[L367] if 0.3 <= score <= 0.6 and config.use_llm_abstraction:

[L368] llm_score = llm_classify_abstraction(node.name, node.definitions)

[L369] score = 0.5 score + 0.5 llm_score # Blend heuristic + LLM

[L370]

[L371] return max(0.0, min(score, 1.0)) # Clamp to [0, 1]

[L372]


```
[L373]
[L374] > Note: This heuristic requires empirical validation on domain-specific datasets. The LLM fallback (Facto
[L375]
[L376] ##### 5.5.2 PRE – Parent Ranking Engine
[L377]
[L378] Ranks all candidates using a weighted scoring formula:
[L379]
[L380] 
$$\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}}{D_{\max}}$$

[L381]
[L382] Where:
[L383] -  $w_1=0.30$  – Embedding cosine similarity
[L384] -  $w_2=0.25$  – Hypernym score (Hearst patterns + LLM fallback)
[L385] -  $w_3=0.15$  – Depth penalty (shallower parents preferred;  $D_{\max}=8$ )
[L386] -  $w_4=0.20$  – Evidence support
[L387] -  $w_5=0.10$  – Parent confidence
[L388]
[L389]
```

python

```
[L390] def pre_rank_all(node_q, candidates, config):
[L391] """Rank ALL candidates, return sorted list."""
[L392] scored = []
[L393] for p in candidates:
[L394] sim = cosine_similarity(node_q.embedding, p.embedding)
[L395] hyp = hypernym_score(node_q, p)
[L396] depth_score = (config.d_max - p.depth) / config.d_max
[L397] evi = evidence_support(node_q, p)
[L398] conf = p.confidence
[L399]
[L400] score = config.w1sim + config.w2hyp + config.w3depth_score + config.w4evi + config.w5*conf
[L401] scored.append(CandidateResult(parent=p, pre_score=score,
[L402] components={'sim': sim, 'hyp': hyp, 'depth': depth_score, 'evi': evi, 'conf': conf}))
[L403]
[L404] scored.sort(key=lambda x: x.pre_score, reverse=True)
[L405]
[L406] if scored and scored[0].pre_score < config.new_tree_threshold:
[L407] return [] # No acceptable candidate → create new root
[L408] return scored
[L409] ``
[L410]
[L411] ##### 5.5.3 HV – Hierarchy Validator
[L412]
[L413] Validates that adding a parent→child edge maintains forest invariants:
[L414]
[L415] 1. Self-loop check – parent ≠ child
[L416] 2. Cycle detection – traverse UP from parent looking for child; if found, reject (would create cycle)
[L417] 3. Depth limit – parent.depth + 1 < config.max_depth`
[L418] 4. Sibling coherence — soft penalty (not a hard gate); dissimilar siblings are allowed but penalized
[L419]
[L420] ##### 5.5.4 FI — Forest Inserter
[L421]
[L422] Inserts a node into the forest by trying HV-validated candidates in ranked order:
```

Citation Marker: `filecite[turn16file6]`

[L423] metadata:

[L424] query_result_index: 6

[L425] file_id: file_000000003218820b918ee3d86ec345ae

[L426] version_id: 1

[L427] name: SHIA_RAG_Final_Report(1).md

[L428] mime_type: text/markdown

[L429] surface: library

[L430] score: 0.018294583307361466

[L431] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--17

[L432] snippet:

[L433] abstraction_level > q_level

[L434] and abs(q_level - c.abstraction_level) < config.max_level_gap]

[L435]

[L436] # Stage 5: Evidence Filter — prioritize cross-document evidence

[L437] candidates = sorted(candidates,

[L438] key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)

[L439]

[L440] return candidates[:config.max_candidates] # Return top 5

[L441]

```
[L442]
[L443] **Overall complexity:**  $O(\log N)$  dominated by the ANN search (Stage 2). All other stages are linear filters o
[L444]
[L445] ##### 5.5.2 PRE – Parent Ranking Engine
[L446]
[L447] Ranks all candidates using a weighted scoring formula:
[L448]
[L449]  $\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max}}{D_{\max} - \text{depth}}$ 
[L450]
[L451] Where:
[L452] -  $w_1=0.30$  – Embedding cosine similarity
[L453] -  $w_2=0.25$  – Hypernym score (Hearst patterns + LLM fallback)
[L454] -  $w_3=0.15$  – Depth penalty (shallower parents preferred;  $D_{\max}=8$ )
[L455] -  $w_4=0.20$  – Evidence support
[L456] -  $w_5=0.10$  – Parent confidence
[L457]
[L458]
```

python

[L459] def pre_rank_all(node_q, candidates, config):

[L460] """Rank ALL candidates, return sorted list."""

[L461] scored = []

[L462] for p in candidates:

[L463] sim = cosine_similarity(node_q.embedding, p.embedding)

[L464] hyp = hypnym_score(node_q, p)

[L465] depth_score = (config.d_max - p.depth) / config.d_max

[L466] evi = evidence_support(node_q, p)

[L467] conf = p.confidence

[L468]

[L469] score = config.w1sim + config.w2hyp + config.w3depth_score + config.w4evi + config.w5*conf

[L470] scored.append(CandidateResult(parent=p, pre_score=score,

[L471] components={'sim': sim, 'hyp': hyp, 'depth': depth_score, 'evi': evi, 'conf': conf}))

[L472]

[L473] scored.sort(key=lambda x: x.pre_score, reverse=True)

```

[L474]
[L475] if scored and scored[0].pre_score < config.new_tree_threshold:
[L476] return [] # No acceptable candidate → create new root
[L477] return scored
[L478] ``
[L479]
[L480] ##### 5.5.3 HV – Hierarchy Validator
[L481]
[L482] Validates that adding a parent-child edge maintains forest invariants:
[L483]
[L484] 1. Self-loop check – parent ≠ child
[L485] 2. Cycle detection – traverse UP from parent looking for child; if found, reject (would create cycle)
[L486] 3. Depth limit – parent.depth + 1 < config.max_depth`
[L487] 4. Sibling coherence — soft penalty (not a hard gate); dissimilar siblings are allowed but penalized
[L488]
[L489] ##### 5.5.4 FI — Forest Inserter
[L490]
[L491] Inserts a node into the forest by trying HV-validated candidates in ranked order:

```

DEEP RESEARCH 24/08/2026, 00:41:19

Citation Marker: `Filecite`turn16file7

```

[L492] metadata:
[L493] query_result_index: 7
[L494] file_id: file_00000000f620821186e54e582d59731e
[L495] version_id: 1
[L496] name: SHIA_RAG_Final_Report(3).md
[L497] mime_type: text/markdown
[L498] surface: library
[L499] score: 0.018294583307361466
[L500] document_chunk_id: file_00000000f620821186e54e582d59731e--1--17
[L501] snippet:
[L502] abstraction_level > q_level
[L503] and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L504]
[L505] # Stage 5: Evidence Filter — prioritize cross-document evidence
[L506] candidates = sorted(candidates,
[L507] key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L508]
[L509] return candidates[:config.max_candidates] # Return top 5
[L510]

```

```

[L511]
[L512] **Overall complexity:**  $O(\log N)$  dominated by the ANN search (Stage 2). All other stages are linear filters o
[L513]
[L514] #### 5.5.2 PRE – Parent Ranking Engine
[L515]
[L516] Ranks all candidates using a weighted scoring formula:
[L517]
[L518]  $\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}}{D_{\max}}$ 
[L519]
[L520] Where:
[L521] -  $w_1=0.30$  – Embedding cosine similarity
[L522] -  $w_2=0.25$  – Hypernym score (Hearst patterns + LLM fallback)
[L523] -  $w_3=0.15$  – Depth penalty (shallower parents preferred;  $D_{\max}=8$ )
[L524] -  $w_4=0.20$  – Evidence support
[L525] -  $w_5=0.10$  – Parent confidence
[L526]
[L527]

```

python

```

[L528] def pre_rank_all(node_q, candidates, config):
[L529] """Rank ALL candidates, return sorted list."""
[L530] scored = []
[L531] for p in candidates:
[L532] sim = cosine_similarity(node_q.embedding, p.embedding)
[L533] hyp = hypernym_score(node_q, p)
[L534] depth_score = (config.d_max - p.depth) / config.d_max
[L535] evi = evidence_support(node_q, p)
[L536] conf = p.confidence
[L537]
[L538] score = config.w1sim + config.w2hyp + config.w3depth_score + config.w4evi + config.w5*conf
[L539] scored.append(CandidateResult(parent=p, pre_score=score,
[L540] components={'sim': sim, 'hyp': hyp, 'depth': depth_score, 'evi': evi, 'conf': conf}))
[L541]
[L542] scored.sort(key=lambda x: x.pre_score, reverse=True)
[L543]
[L544] if scored and scored[0].pre_score < config.new_tree_threshold:
[L545] return [] # No acceptable candidate → create new root
[L546] return scored
[L547] ``
[L548]
[L549] #### 5.5.3 HV – Hierarchy Validator
[L550]
[L551] Validates that adding a parent→child edge maintains forest invariants:
[L552]
[L553] 1. Self-loop check – parent ≠ child
[L554] 2. Cycle detection – traverse UP from parent looking for child; if found, reject (would create cycle)
[L555] 3. Depth limit – parent.depth + 1 < config.max_depth`
[L556] 4. Sibling coherence — soft penalty (not a hard gate); dissimilar siblings are allowed but penalized
[L557]
[L558] #### 5.5.4 FI — Forest Inserter
[L559]
[L560] Inserts a node into the forest by trying HV-validated candidates in ranked order:

```

Citation Marker: `filecite`turn16file8

[L561] metadata:

[L562] query_result_index: 8

[L563] file_id: file_000000004bc481fdbf9e949ec3b7544b

[L564] version_id: 1

[L565] name: SHIA_RAG_Final_Report(2).md

[L566] mime_type: text/markdown

[L567] surface: library

[L568] score: 0.018294583307361466

[L569] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--18

[L570] snippet:

[L571]

```
[L572] def pre_rank_all(node_q, candidates, config):
[L573]     """Rank ALL candidates, return sorted list."""
[L574]     scored = []
[L575]     for p in candidates:
[L576]         sim = cosine_similarity(node_q.embedding, p.embedding)
[L577]         hyp = hypernym_score(node_q, p)
[L578]         depth_score = (config.d_max - p.depth) / config.d_max
[L579]         evi = evidence_support(node_q, p)
[L580]         conf = p.confidence
[L581]
[L582]         score = config.w1*sim + config.w2*hyp + config.w3*depth_score + config.w4*evi + config.w5*conf
[L583]         scored.append(CandidateResult(parent=p, pre_score=score,
[L584]             components={'sim': sim, 'hyp': hyp, 'depth': depth_score, 'evi': evi, 'conf': conf}))
[L585]
[L586]     scored.sort(key=lambda x: x.pre_score, reverse=True)
[L587]
[L588]     if scored and scored[0].pre_score < config.new_tree_threshold:
[L589]         return [] # No acceptable candidate → create new root
[L590]     return scored
[L591]
```

[L592]

[L593] ##### 5.5.3 HV — Hierarchy Validator

[L594]

[L595] Validates that adding a parent → child edge maintains forest invariants:

[L596]

[L597] 1. **Self-loop check** — parent ≠ child

[L598] 2. **Cycle detection** — traverse UP from parent looking for child; if found, reject (would create cycle)

[L599] 3. **Depth limit** — `parent.depth + 1 < config.max_depth`

[L600] 4. **Sibling coherence** — soft penalty (not a hard gate); dissimilar siblings are allowed but penalized

[L601]

[L602] ##### 5.5.4 FI — Forest Inserter

[L603]

[L604] Inserts a node into the forest by trying HV-validated candidates in ranked order:

[L605]

[L606]

```
[L607] def fi_insert(node, ranked_candidates, forest, config):
[L608]     """Insert node into forest. Try candidates in order; fall back to new root."""
[L609]     for candidate in ranked_candidates:
[L610]         if hv_validate(candidate.parent, node, forest, config):
[L611]             forest.add_edge(candidate.parent.id, node.id, edge_class='HIERARCHICAL')
[L612]             node.parent = candidate.parent.id
[L613]             node.depth = candidate.parent.depth + 1
[L614]             propagate_confidence(forest) # Re-propagate confidence scores
[L615]             return InsertResult(status='PLACED', parent=candidate.parent.id)
[L616]
[L617]     # All candidates failed validation – create new root
[L618]     forest.add_root(node)
[L619]     return InsertResult(status='NEW_ROOT', reason='NO_VALID_CANDIDATE')
[L620]
```

[L621]

[L622] ##### 5.5.5 CLD — Cross-Link Discoverer

[L623]

[L624] After hierarchical placement, discover semantic (non-hierarchical) links:

[L625]

[L626]

```
[L627] def cld_discover(node, forest, config):
[L628]     """Find semantic cross-links for a newly placed node."""
[L629]     cross_links = []
[L630]     neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L631]
[L632]     for neighbor in neighbors:
[L633]         if neighbor.id == node.id or neighbor.id == node.parent:
[L634]             continue
[L635]
[L636]         sim = cosine_similarity(node.embedding, neighbor.embedding)
[L637]         if sim > config.cross_link_threshold:
[L638]             relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L639]             if relation_type:
[L640]                 forest.add_edge(node.id, neighbor.id,
[L641]                     edge_class='SEMANTIC', relation_type=relation_type,
[L642]                     confidence=sim, weight=1.0)
[L643]                 cross_links.append((neighbor.id, relation_type, sim))
[L644]
[L645]     return cross_links
[L646]
```

[L647]

[L648] ---

[L649]

[L650] ## 6. Online Pipeline — Layers 6-7

[L651]

[L652] ### 6.1 Layer 6: Retrieval Engine

[L653]

[L654] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L655]

[L656] **Input:** User query string → **Output:** `RetrievalResult`

[L657]

[L658] ##### Query Complexity Classification

Citation Marker: [filecite]turn16file9[

[L659] metadata:

[L660] query_result_index: 9

[L661] file_id: file_00000000fb2c81fd82f14a7a48993554

[L662] version_id: 1

[L663] name: SHIA_RAG_Final_Report.md

[L664] mime_type: text/markdown

[L665] surface: library

[L666] score: 0.018051958245035714

[L667] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--35

[L668] snippet:

[L669] py # Candidate Parent Generator

[L670] | | |— pre.py # Parent Ranking Engine

[L671] | | |— hv.py # Hierarchy Validator

[L672] | | |— fi.py # Forest Integrator

[L673] | | |— cld.py # Cross-Link Discovery

[L674] | | |— forest.py # Knowledge Forest data structure

[L675] | |— layer6_retrieval/

[L676] | | |— query_analyzer.py

[L677] | | |— complexity_classifier.py

[L678] | | |— traversal.py # Adaptive depth traversal

[L679] | | |— tree_knapsack.py # Precedence-constrained optimizer

[L680] | | |— context_assembler.py

[L681] | |— layer7_generation/

[L682] | | |— prompt_builder.py

[L683] | | |— llm_client.py

[L684] | | |— verifier.py

[L685] | |— layer8_ops/

[L686] | |— logger.py

[L687] | |— metrics.py

[L688] | |— feedback.py # Self-evolution engine

[L689] |— evaluation/

[L690] | |— shef.py # SHEF framework

[L691] | |— datasets/ # Test corpora

[L692] | |— baselines/ # Flat RAG, TreeRAG comparisons

[L693] |— api/

[L694] | |— main.py # FastAPI endpoints

[L695] |— tests/

[L696] | |— test_hv.py # Cycle detection tests

[L697] | |— test_pre.py

[L698] | |— test_tree_knapsack.py

[L699] | |— test_retrieval.py

[L700] |— docker-compose.yaml

[L701] |— requirements.txt

[L702] |— README.md

[L703]

```
[L704]
[L705] ### 10.3 Environment Configuration
[L706]
[L707]
```

text

```
[L708] # .env.example
[L709] OPENAI_API_KEY=sk-...
[L710] LLM_MODEL=gpt-4o-mini
[L711] NEO4J_URI=bolt://localhost:7687
[L712] NEO4J_USER=neo4j
[L713] NEO4J_PASSWORD=
[L714] POSTGRES_URI=postgresql://localhost:5432/shia_rag
[L715] REDIS_URL=redis://localhost:6379
[L716] EMBEDDING_MODEL=bge-base-en-v1.5
[L717] EMBEDDING_DIMENSION=768
[L718] LOG_LEVEL=INFO
[L719] TOKEN_BUDGET=4096
[L720] MAX_DEPTH=8
[L721]
```

```
[L722]
[L723] ### 10.4 Implementation Timeline (6 Months)
[L724]
[L725]
```

mermaid

```
[L726] gantt
[L727] title SHIA-RAG Implementation Roadmap
[L728] dateFormat YYYY-MM-DD
[L729]
[L730] section Phase 1: Foundation
[L731] Core Data Model + Schemas :a1, 2026-09-01, 2w
[L732] Input Processing (Layer 1) :a2, after a1, 1w
[L733] Document Parser (Layer 2) :a3, after a2, 2w
[L734]
[L735] section Phase 2: Knowledge Extraction
[L736] Knowledge Extractor (Layer 3) :b1, after a3, 3w
[L737] Alias Resolution + LSH :b2, after b1, 2w
[L738] Relation Management (Layer 4) :b3, after b2, 2w
[L739]
[L740] section Phase 3: SHIA Core
[L741] CPG++ Implementation :c1, after b3, 2w
[L742] PRE + HV (with feedback loop) :c2, after c1, 2w
[L743] FI + CLD :c3, after c2, 1w
[L744] Forest Construction Pipeline :c4, after c3, 1w
[L745]
[L746] section Phase 4: Retrieval & Generation
[L747] Query Analyzer + Classifier :d1, after c4,
```

CHATGPT 24/08/2026, 00:41:19

Bilkul. Ab hum Layer 5 ko same style mein, ek hi running example ke saath, step-by-step aur detail mein samjhenge.

Hum wahi example continue karenge:

Operating Systems → CPU Scheduling → CPU Scheduling Algorithm → Round Robin → Time Quantum

Layer 5 — SHIA Core / Hierarchy Construction

Sabse pehle ek line mein samjho

Layer 5 takes the validated KnowledgeNodes and KnowledgeEdges from the previous layers and decides where each concept should be placed inside the Knowledge Forest.

Simple words mein:

Layer 3 ne concepts banaye.

Layer 4 ne unke relations validate kiye.

Layer 5 decide karta hai: "Kaunsa concept kis concept ke under jaana chahiye?"

File ke according Layer 5 ka input `KnowledgeNode[] + KnowledgeEdge[]` hai aur output **Knowledge Forest** hai. Iske five main components hain: `CPG++ → PRE → HV → FI → CLD`. [filecite\turn15file0\L23-L44](#)

1. Layer 5 ki zarurat kyu padi?

Layer 4 ke baad maan lo hamare paas ye nodes hain:

```
Operating Systems
CPU Scheduling
CPU Scheduling Algorithm
Round Robin
Time Quantum
```

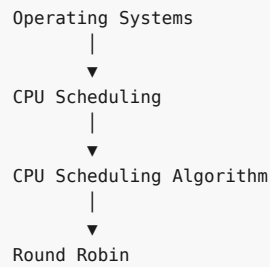
Aur relations hain:

```
Round Robin
  IS_A
CPU Scheduling Algorithm
Round Robin
  USES
Time Quantum
```

Ab problem:

System ko abhi ye nahi pata ki final hierarchy kya hogi

For example:



Ye structure automatically sirf concepts aur relations se final form mein ready nahi hota.

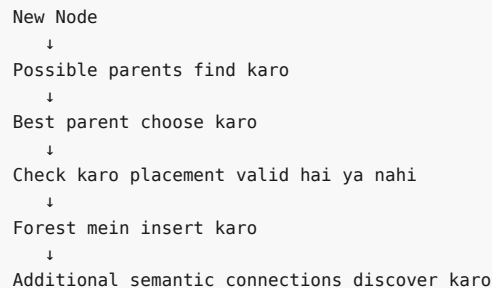
System ko decide karna padega:

- Round Robin ka parent kaun hoga?
- CPU Scheduling?
- CPU Scheduling Algorithm?
- Operating Systems?
- Kya Round Robin ek new root hona chahiye?
- Kya hierarchy mein cycle ban rahi hai?
- Kya Time Quantum ko parent banana chahiye? x
- Kya **USES** relation ko hierarchy mein convert karna chahiye? x

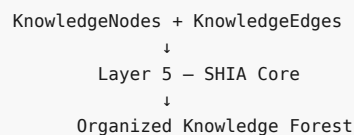
Layer 5 ye sab decisions leta hai.

2. Layer 5 ka main goal

Har new concept ke liye:



So overall:



The file describes Layer 5 as the hierarchy-construction stage that builds a **Knowledge Forest of rooted trees plus cross-links**. `filecite[turn14file0[L35-L50]`

3. Sabse pehle — Knowledge Forest kya hai?

Normal list

Round Robin
CPU Scheduling
Time Quantum
Operating System
Process

Ye bas concepts ki list hai.

× Relationship ka proper structure clear nahi hai.

Knowledge Forest

```

Operating Systems
├── CPU Scheduling
│   ├── CPU Scheduling Algorithm
│   │   ├── Round Robin
│   │   │   └── [Semantic Link]
│   │   │       USES → Time Quantum

```

Yahan do important structures hain:

1. Hierarchical structure

```

Parent
  ↓
Child

```

Example:

```

CPU Scheduling Algorithm
  ↓
Round Robin

```

2. Semantic connections

Example:

```

Round Robin
  USES
  ↓
Time Quantum

```

Time Quantum Round Robin ka parent nahi hai. Dono ke beech ek **semantic relation** hai.

The final report distinguishes **hierarchical edges**, which form the acyclic forest, from **semantic edges**, which are separate associations and may form cycles. <https://filecite.com/turn14file7/L537-L556/>

4. Layer 5 ka complete pipeline

Layer 5 mein 5 major components hain:

```
New Node
  |
  ▼
CPG++
Candidate Parent Generation
  |
  ▼
PRE
Parent Ranking Engine
  |
  ▼
HV
Hierarchy Validator
  |
  ▼
FI
Forest Integration
  |
  ▼
CLD
Cross-Link Discovery
  |
  ▼
Knowledge Forest
```

File ka exact pipeline ye flow define karta hai:

```
New Node
  ↓
Generate ≤5 Parent Candidates
  ↓
Rank All Candidates
  ↓
Validate Best Candidate
  ↓
If invalid → Try Next Candidate
  ↓
If all fail → Create New Root
  ↓
Discover Semantic Cross-Links
```

filecite turn15file0 L29-L44

Ab hum **ek-ek component deeply** samjhenge.

5. Running Example

Maan lo Layer 5 mein ek new node aaya:

Round Robin

Uski information:

Concept:
Round Robin

Domain:
Operating Systems

Definition:
A preemptive CPU scheduling algorithm.

Relations:
IS_A → CPU Scheduling Algorithm

USES → Time Quantum

Embedding:
[vector representation]

Evidence:
Multiple source documents

Ab Layer 5 ka question:

“Round Robin ko Knowledge Forest mein exactly kahan place karna chahiye?”

PART 1 — CPG++

6. CPG++ kya hai?

CPG++ = Candidate Parent Generator

Iska kaam final parent choose karna nahi hai.

Iska kaam hai:

Possible parents ki ek small list generate karna.

Example:

Round Robin ke liye forest mein thousands of nodes ho sakte hain.

CPG++ possible candidates de sakta hai:

Candidate 1:
CPU Scheduling Algorithm

Candidate 2:
CPU Scheduling

Candidate 3:
Operating Systems

Aur finally maximum limited candidates, report configuration mein typically **top 5**, PRE ko diye jaate hain.

filecite{turn15file0}{L46-L80}

7. CPG++ ki zarurat kyu hai?

Maan lo Knowledge Forest mein:

100,000 concepts

Ab Round Robin ke liye har node ko parent candidate maan kar compare karna:

```

Round Robin
  ↓
Compare with Node 1
Compare with Node 2
Compare with Node 3
...
Compare with Node 100,000

```

- × Expensive
- × Slow
- × Unnecessary

CPG++ pehle search space reduce karta hai.

```

100,000 Nodes
  ↓
Relevant Domain
  ↓
Semantic Candidates
  ↓
Relation-Based Candidates
  ↓
Abstraction Filter
  ↓
Evidence Filter
  ↓
Top ≤5 Candidates

```

File ke according CPG++ ka overall complexity ANN search se dominated hai, approximately $O(\log N)$, instead of naively scanning the entire forest. [filecite\turn15file0\L46-L80](#)

8. CPG++ ke 5 stages

Stage 1 — Domain Routing

Sabse pehle system dekhta hai:

Ye new node kis domain ka hai?

Round Robin:

```

Domain:
Operating Systems

```

Toh system directly in domains ko ignore kar sakta hai:

```

Computer Networks ✗
Database Systems ✗
Machine Learning ✗

```

Aur relevant nodes dekhega:

```

Operating Systems ✓

```

Example:

```

Round Robin
  ↓
Domain = Operating Systems
  ↓
Search only OS-related nodes

```

Source pipeline mein first stage same-domain filtering hai. [filecite\turn15file1\L135-L140](#)

9. Stage 2 — ANN Retrieval

Ab bhi OS domain mein bahut nodes ho sakte hain:

```
Process
Thread
Deadlock
Memory Management
CPU Scheduling
Round Robin
FCFS
Priority Scheduling
...
```

System **ANN (Approximate Nearest Neighbor)** search use karta hai.

Input:

```
Round Robin Embedding
```

Vector search:

```
Round Robin
  ↓
ANN / Vector Index
  ↓
Find semantically similar nodes
```

Possible output:

```
1. CPU Scheduling Algorithm
2. CPU Scheduling
3. FCFS
4. Priority Scheduling
5. Time Quantum
```

Yahan technologies/concepts:

- **Vector Embeddings**
- **ANN Search**
- File examples mention an ANN index such as **HNSW**
- Vector index implementations can use systems such as **FAISS** or **Qdrant**

filecite[turn15file3L280-L289] filecite[turn16file3L259-L269]

10. Lekin semantic similarity se parent directly choose kyu nahi kar sakte?

Bahut important.

Suppose ANN result:

```
Round Robin ↔ Time Quantum
```

Highly similar hain.

Lekin:

```
Time Quantum
  ↓
Round Robin
```

parent-child hierarchy galat hogi.

Kyun?

Kyunki:

```
Round Robin USES Time Quantum
```

Ye **semantic relation** hai, hierarchical relation nahi.

Isliye ANN sirf bolta hai:

“*Ye concept semantically related lagta hai.*”

ANN ye nahi bolta:

“*Ye definitely iska parent hai.*”

Isliye next filters chahiye.

11. Stage 3 — Relation Filter

Ab CPG++ check karta hai:

“*Kya new node aur candidate ke beech plausible hierarchical relation exist karta hai?*”

Example candidates:

Candidate: CPU Scheduling Algorithm

```
Round Robin
IS_A
CPU Scheduling Algorithm
```

✓ Strong candidate.

Candidate: Time Quantum

```
Round Robin
USES
Time Quantum
```

Ye relation hierarchical parent relation nahi hai.

So:

```
Time Quantum
❌ Not a suitable parent
```

Relation Filter semantic similarity ke baad irrelevant candidates ko remove karta hai. [filecite]turn15file0[L64-L65]

12. Stage 4 — Abstraction Filter

Ye Layer 5 ka bahut important concept hai.

Parent kaisa hona chahiye?

Generally:

“*Parent should be more abstract/general than the child.*”

Example:

Operating Systems
↑ More General

CPU Scheduling
↑

CPU Scheduling Algorithm
↑

Round Robin
↑ More Specific

So:

Round Robin

ka parent ideally:

CPU Scheduling Algorithm

ho sakta hai.

Lekin:

Operating Systems

bahut zyada broad ho sakta hai.

Aur:

Time Quantum

Round Robin se parent-level abstraction mein appropriate nahi hai.

Isliye system abstraction level estimate karta hai.

File ke according parent candidate ka abstraction level child se **higher/more abstract** hona chahiye, aur level gap configured maximum se zyada nahi hona chahiye. `filecite\turn15file8\L709-L720`

Simple analogy

Vehicle
↓
Car
↓
Electric Car

Agar node:

Electric Car

hai:

Car

good parent hai ✓

Vehicle

possible hai but hierarchy level bahut broad ho sakta hai.

Battery

parent nahi hai x

because:

Electric Car USES Battery

13. Abstraction Level kaise estimate hota hai?

Final report abstraction estimation ko **multi-factor heuristic** ke form mein describe karta hai, aur ambiguous cases mein optional LLM fallback bhi mention karta hai. [filecite\turn15file8\L725-L741](#)

Simple conceptual idea:

```
More general concept
  ↓
Higher abstraction

More specific concept
  ↓
Lower abstraction
```

Example:

Operating Systems	→ High abstraction
CPU Scheduling	→ Medium
Round Robin	→ Lower

Important: File khud note karta hai ki abstraction estimation heuristic ko empirical validation ki zarurat hai; LLM fallback accuracy improve kar sakta hai but cost increase karta hai. [filecite\turn16file5\L366-L374](#)

14. Stage 5 — Evidence Filter

Maan lo do candidate parents hain:

```
Candidate A:
CPU Scheduling Algorithm

Candidate B:
CPU Scheduling
```

Ab system dekhta hai:

Kis candidate ke support mein zyada evidence hai?

Example:

CPU Scheduling Algorithm

```
Document 1:
Round Robin is a CPU scheduling algorithm.

Document 2:
Round Robin is a preemptive scheduling algorithm.
```

CPU Scheduling

Only one weak mention.

Then:

CPU Scheduling Algorithm

gets stronger priority.

CPG++ cross-document evidence count ko prioritize karke candidates sort karta hai. [filecite\turn15file0\L73-L77](#)

15. CPG++ ka final output

After all filters:

```
Round Robin
  ↓
Candidate Parents

1. CPU Scheduling Algorithm
2. CPU Scheduling
3. Operating Systems
```

Maximum few candidates.

Important:

CPG++ final parent select nahi karta.

It only says:

“Ye possible parents hain.”

Final decision next component **PRE** karega.

PART 2 — PRE

16. PRE kya hai?

PRE = Parent Ranking Engine

CPG++ ne candidates diye:

```
1. CPU Scheduling Algorithm
2. CPU Scheduling
3. Operating Systems
```

Ab PRE poochta hai:

“Inmein se best parent kaun hai?”

PRE har candidate ko score deta hai.

17. PRE scoring

File ke according PRE multiple signals combine karta hai:

```
Parent Score
=
Semantic Similarity
+
Hypernym Score
+
Depth Preference
+
Evidence Support
+
Parent Confidence
```

The report gives these weights:

Signal	Weight
Embedding cosine similarity	0.30
Hypernym score	0.25
Depth preference	0.15
Evidence support	0.20
Parent confidence	0.10

□filecite□turn16file5□L376-L408□

Ab ek-ek samajhte hain.

18. Signal 1 — Semantic Similarity

Question:

Round Robin aur candidate concept semantic meaning mein kitne related hain?

Example:

Round Robin
 ↔
 CPU Scheduling Algorithm

High similarity.

Round Robin
 ↔
 Operating Systems

Related, but broader.

Round Robin
 ↔
 Deadlock

Low similarity.

Technology:

Embeddings
 +
 Cosine Similarity

19. Signal 2 — Hypernym Score

Hypernym ka simple meaning:

General category relationship.

Example:

Round Robin is a type of CPU Scheduling Algorithm.

Yahan:

CPU Scheduling Algorithm

is the hypernym/general category.

So:

Round Robin
IS_A
CPU Scheduling Algorithm

Strong hypernym score.

File specifies this signal as **Hearst patterns + LLM fallback**. `filecite\turn16file5\L380-L387`

20. Signal 3 — Depth Preference

System unnecessarily bahut deep hierarchy nahi banana chahta.

Suppose:

Candidate A:
Depth = 2

Candidate B:
Depth = 7

All else being similar, a shallower appropriate parent may be preferred.

Report formula uses a depth-based score and sets `Dmax = 8` in the described configuration. `filecite\turn16file5\L380-L387`

21. Signal 4 — Evidence Support

Question:

Kya documents actually support karte hain ki ye parent-child relation hai?

Example:

Document A:
Round Robin is a CPU Scheduling Algorithm.

Document B:
Round Robin belongs to the CPU Scheduling Algorithm category.

Strong evidence.

Vs:

One vague mention only.

Weak evidence.

22. Signal 5 — Parent Confidence

Har KnowledgeNode ka confidence hota hai.

Example:

CPU Scheduling Algorithm
Confidence = 0.95

vs:

Some uncertain concept
Confidence = 0.55

A reliable parent candidate ko PRE score mein advantage milta hai.

23. PRE ka example

Maan lo:

| Candidate Parent | Final PRE Score |

|---|---:|

| CPU Scheduling Algorithm | 0.92 |

| CPU Scheduling | 0.78 |

| Operating Systems | 0.54 |

PRE output:

```
Rank 1 → CPU Scheduling Algorithm
Rank 2 → CPU Scheduling
Rank 3 → Operating Systems
```

Important point:

PRE only best candidate return karke stop nahi karta.

File ke corrected pipeline mein **all candidates sorted order mein** maintain hote hain. Agar Rank 1 validation mein fail ho jaye, system Rank 2 try kar sakta hai. `filecite\turn16file5\L389-L408`

PART 3 — HV

24. HV kya hai?

HV = Hierarchy Validator

PRE bolta hai:

```
Best Parent:
CPU Scheduling Algorithm
```

Ab HV poochta hai:

"Kya Round Robin ko is parent ke under safely place kar sakte hain?"

25. HV ke validation checks

File ke according main checks:

1. Self-loop check

```
Round Robin
  ↓
Round Robin
```

× Not allowed.

Parent and child same nahi ho sakte.

2. Cycle Detection

Suppose:

```
A → B
B → C
```

Ab agar add karein:

C → A

Then:

A → B → C
 ↑ ↓
 └────────┘

Cycle ban gayi.

× Not allowed for hierarchical edges.

HV parent se ancestors ki taraf traverse karke check karta hai ki proposed child wahan already toh nahi hai.

filecite turn16file5 L411-L419

3. Depth Limit

System hierarchy ko unlimited deep nahi hone deta.

Example:

Level 1
 ↓
 Level 2
 ↓
 Level 3
 ↓
 ...

Bahut deep tree:

× Retrieval difficult

× Context traversal costly

× Structure unnecessarily complex

File's example configuration uses a maximum depth constraint, and the environment example specifies `MAX_DEPTH=8`.

filecite turn16file8 L593-L620 filecite turn16file9 L707-L721

4. Sibling Coherence

Suppose parent:

CPU Scheduling Algorithm

Existing children:

FCFS
 Priority Scheduling
 SJF

Ab new child:

Round Robin

✓ Makes sense.

Lekin:

SQL JOIN

× Incoherent sibling.

Final report treats sibling coherence as a **soft penalty rather than a hard gate**—dissimilar siblings can still be allowed but are penalized. `filecite{turn16file5}{L411-L419}`

26. Agar best parent fail ho gaya toh?

Suppose:

```
Rank 1:
CPU Scheduling Algorithm
```

HV mein fail.

System:

```
Try Rank 2
```

```
CPU Scheduling
```

Agar pass:

```
Insert Round Robin
```

Agar Rank 2 bhi fail:

```
Try Rank 3
```

Agar sab fail:

```
Round Robin becomes a New Root
```

Ye exact fallback behavior Layer 5 pipeline mein defined hai. `filecite{turn15file0}{L31-L43}`

PART 4 — FI

27. FI kya hai?

FI = Forest Integration / Forest Inserter

Ab parent successfully validate ho gaya:

```
CPU Scheduling Algorithm
      ↓
Round Robin
```

FI actual insertion karta hai.

Matlab ab concept officially Knowledge Forest ka part banega.

28. FI internally kya karta hai?

Suppose:

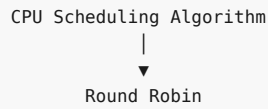
```
Parent:
CPU Scheduling Algorithm
```

```
Child:
Round Robin
```


FI:

Step 1

Hierarchical edge add karta hai:



Step 2

Round Robin ka parent update karta hai:

```

RoundRobin.parent
=
CPU_Scheduling_Algorithm
  
```

Step 3

Depth update karta hai:

```

RoundRobin.depth
=
Parent.depth + 1
  
```

Step 4

Confidence propagation re-run karta hai.

File's `fi_insert` logic tries candidates in ranked order, inserts under the first HV-valid parent, updates parent/depth information, and falls back to a new root if no candidate is valid. [filecite\[turn16file8L602-L620\]](#)

29. Agar koi valid parent nahi mila?

Example:

New concept:

```

Quantum Entanglement
  
```

Aur current forest mein mostly:

```

Operating Systems
Database
Computer Networks
  
```

koi meaningful parent nahi mila.

To system forcefully galat parent assign nahi karega.

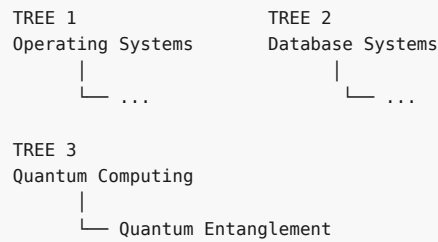
Instead:

```

Quantum Entanglement
  ↓
NEW ROOT
  
```

Yahan se ek new tree start ho sakta hai.

Isliye ise **Knowledge Forest** kehte hain, sirf ek single tree nahi.

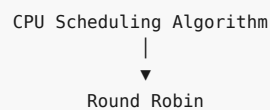


PART 5 — CLD

30. CLD kya hai?

CLD = Cross-Link Discovery

Ab hierarchy place ho chuki:



Lekin real knowledge mein sirf parent-child relation enough nahi hota.

Round Robin ka connection ho sakta hai:

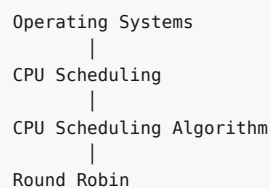
Round Robin
USES
Time Quantum

Ye hierarchy nahi hai.

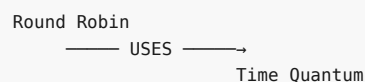
Isliye CLD additional semantic links discover karta hai.

31. Example

Main hierarchy:



Semantic cross-link:



Time Quantum ko Round Robin ka child ya parent banana zaroori nahi hai.

Bas unke beech ek meaningful connection hai.

32. CLD kaise kaam karta hai?

File ke corrected pipeline ke according:

Step 1

Newly placed node ka embedding use hota hai.

```
Round Robin Embedding
```

Step 2

ANN search se semantic neighbors find hote hain.

```
Round Robin
  ↓
Vector Search
  ↓
Time Quantum
FCFS
Process
Queue
```

Step 3

Direct parent aur self ko skip kiya jata hai.

Step 4

Cosine similarity check hoti hai.

```
Similarity > Threshold?
  ↓
Yes
```

Step 5

Relation classify hota hai:

```
USES
CAUSES
DEPENDS_ON
...
```

Step 6

Semantic edge add hoti hai.

```
filecite[turn16file8L622-L645]
```

33. CLD kyu important hai?

Imagine query:

```
“What is the role of Time Quantum in Round Robin?”
```

Agar hierarchy sirf parent-child hoti:

```
Operating Systems
  ↓
CPU Scheduling
  ↓
Round Robin
```

To Time Quantum directly connected nahi hota.

But CLD semantic connection deta hai:

Round Robin
USES
Time Quantum

Ab Layer 6 retrieval ke time:

Round Robin
↓
Semantic Cross-Link
↓
Time Quantum

tak efficiently ja sakta hai.

File explicitly says cross-links do not affect parent selection but **enrich retrieval** and can be used to include one-hop neighbors at query time. `filecite[turn16file3][L247-L257]`

34. Important: Hierarchy aur Semantic Links ko mix mat karna

Hierarchical Edge

CPU Scheduling Algorithm
|
IS_A
▼
Round Robin

Actually direction implementation may store parent → child structurally, while the conceptual relation is that Round Robin IS_A CPU Scheduling Algorithm.

Hierarchy rule:

No cycles
Single parent

Semantic Edge

Round Robin
USES
Time Quantum

Rule:

Additional association
Not used as main parent-child structure

Final report explicitly enforces a **strict single-parent constraint for hierarchical edges**, while multiple semantic associations remain unrestricted. `filecite[turn14file7][L544-L556]`

35. Ab hamara complete running example start se dekhte hain

Input from Layer 4

Knowledge Nodes

N1 = Operating Systems
N2 = CPU Scheduling
N3 = CPU Scheduling Algorithm
N4 = Round Robin
N5 = Time Quantum

Validated Edges

Round Robin
IS_A
CPU Scheduling Algorithm

Round Robin
USES
Time Quantum

Step 1 — CPG++

New node:

Round Robin

Possible candidates:

CPU Scheduling Algorithm
CPU Scheduling
Operating Systems

Step 2 — PRE

Scores:

CPU Scheduling Algorithm → 0.92

CPU Scheduling → 0.78

Operating Systems → 0.54

Sorted output:

Rank 1 → CPU Scheduling Algorithm
Rank 2 → CPU Scheduling
Rank 3 → Operating Systems

Step 3 — HV

Check Rank 1:

Self-loop? No ✓
Cycle? No ✓
Depth valid? Yes ✓
Sibling coherence? Yes ✓

Result:

VALID

Step 4 — FI

Insert:

```

CPU Scheduling Algorithm
  |
  ▼
Round Robin
  
```

Update:

```

Round Robin.parent
Round Robin.depth
Parent.children
Forest structure
  
```

Step 5 — CLD

Find semantic neighbor:

```

Time Quantum
  
```

Existing validated relation:

```

Round Robin USES Time Quantum
  
```

Add semantic cross-link:

```

Round Robin
  — USES —>
Time Quantum
  
```

36. Final Output — Knowledge Forest

Finally:

```

Operating Systems
  |
  ▼
CPU Scheduling
  |
  ▼
CPU Scheduling Algorithm
  |
  ▼
Round Robin
  |
  USES |
  ▼
Time Quantum
  
```

Conceptually:

- **Vertical structure** = Hierarchy
- **USES connection** = Semantic cross-link

This is the type of output Layer 5 prepares for storage and later retrieval. `filecite\turn14file0\L47-L50`

37. Layer 5 ka Input exactly kya hai?

```
KnowledgeNode[]
+
KnowledgeEdge[]
```

Example:

```
KnowledgeNode:
Round Robin
Domain:
Operating Systems
Embedding:
[...]
Confidence:
0.92
Abstraction Level:
...

KnowledgeEdge:
Round Robin
IS_A
CPU Scheduling Algorithm

Confidence:
0.95
```

Ye data Layer 5 ko previous layers se milta hai. [filecite\turn14file0\L43-L50](#)

38. Layer 5 ka Output kya hai?

Output:

Knowledge Forest

Ismein:

```
✓ Parent-child hierarchy
✓ Multiple rooted trees
✓ Node depth
✓ Parent information
✓ Children information
✓ Hierarchical edges
✓ Semantic cross-links
✓ Updated indices
```

Main architecture flow ke according Layer 5 output **Knowledge Forest (rooted trees + cross-links)** hai.

[filecite\turn14file0\L47-L50](#)

39. Layer 5 mein technologies/algorithms kya use ho rahe hain?

- | Component | Technology / Algorithm | Kaam |
- |---|---|---
- | **CPG++** | Domain Routing | Relevant domain filter karna |
- | **CPG++** | ANN / Vector Search | Similar candidate parents find karna |
- | **CPG++** | Relation Filter | Plausible relations check karna |
- | **CPG++** | Abstraction Estimation | Parent more general hai ya nahi |
- | **CPG++** | Evidence Filter | Strong cross-document evidence prioritize karna |
- | **PRE** | Cosine Similarity | Semantic similarity score |
- | **PRE** | Hypernym Detection | Parent-child/general-category relation score |
- | **PRE** | Hearst Patterns + LLM fallback | Hypernym detection |
- | **HV** | Ancestor Traversal / Cycle Detection | Hierarchy cycle avoid karna |

HV	Depth Constraint	Tree excessively deep na ho
HV	Sibling Coherence	Related concepts ko coherent siblings banana
FI	Tree Insertion	Node ko forest mein place karna
CLD	ANN + Similarity Threshold	Semantic cross-links discover karna
Storage	Graph DB + Vector DB	Forest aur embeddings store/retrieve karna

The report describes ANN indexes such as HNSW and examples like FAISS/Qdrant, along with graph/in-memory structures for the forest. [filecite\turn16file3\L259-L269](#)

40. Layer 5 mein possible errors aur safeguards

Error 1 — Wrong parent select hona

Example:

```
Round Robin
  ↓
Operating System
```

Jab better parent:

```
CPU Scheduling Algorithm
```

ho.

Safeguard

```
CPG++
  ↓
PRE Multi-Signal Ranking
```

Error 2 — Semantic relation ko hierarchy bana dena

Example:

```
Round Robin
  USES
Time Quantum
```

Galat:

```
Round Robin
  ↓
Time Quantum
```

as parent-child.

Safeguard

Hierarchical aur semantic edges ko separate classes mein maintain kiya jata hai. [filecite\turn14file7\L544-L556](#)

Error 3 — Cycle ban jana

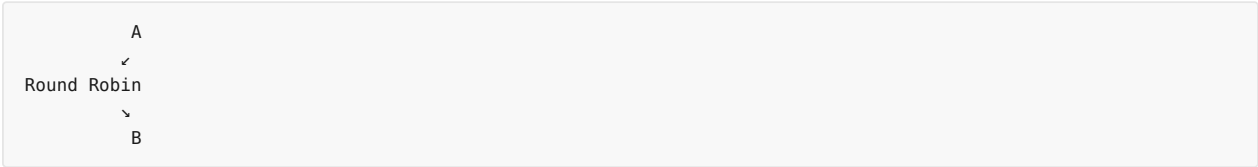
```
A → B → C
↑       |
└──────┘
```

Safeguard

HV Cycle Detection

Error 4 — Multiple parents

Example:



Current SHIA-RAG design:

Strict single-parent hierarchy

Additional associations semantic edges ke through represent kiye jaate hain.

Error 5 — Sab candidates invalid

Safeguard

Forcefully wrong parent nahi choose karte.

Instead:

New Root

banaya jata hai.

Error 6 — Hierarchy unnecessarily deep

Safeguard

Depth Limit

41. Layer 5 kya nahi karta?

Ye bhi important hai.

Layer 5:

- × Raw document read nahi karta
- × OCR nahi karta
- × New concepts extract karna iska primary job nahi hai
- × Raw relation extraction nahi karta
- × Relation confidence calculate karne ka primary Layer 4-level task nahi hai

Layer 5 ka main kaam:

“Existing validated knowledge ko intelligently organize karna.”

42. Layer 4 → Layer 5 → Layer 6 difference

Layer 4 — Relation Management

Question:

“Kya Round Robin aur Time Quantum ka relation valid hai?”

Output:

Validated KnowledgeEdge

Layer 5 — Hierarchy Construction

Question:

“Round Robin ko Knowledge Forest mein kahan place karna chahiye?”

Output:

Knowledge Forest

Layer 6 — Retrieval

Question:

“User ke question ke liye Forest ke kaunse nodes retrieve karne chahiye?”

Output:

Relevant Context

43. Real-world analogy

Imagine ek library hai.

Layer 3

Books ke andar se concepts identify karta hai.

Round Robin
CPU Scheduling
Time Quantum

Layer 4

Check karta hai:

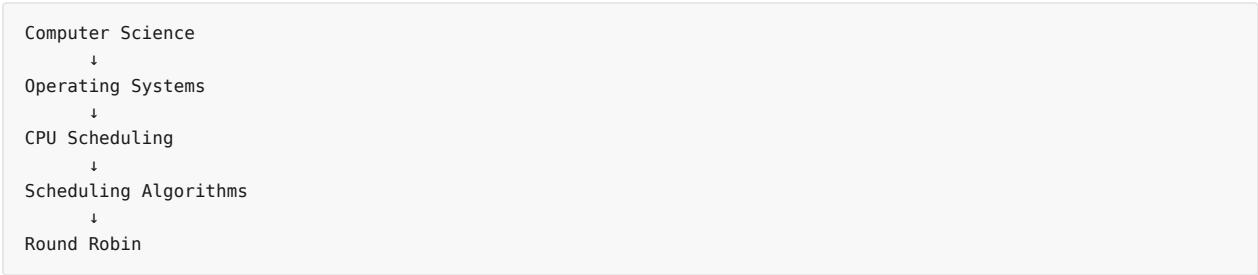
Round Robin → IS_A → CPU Scheduling Algorithm

Kya ye relation valid hai?

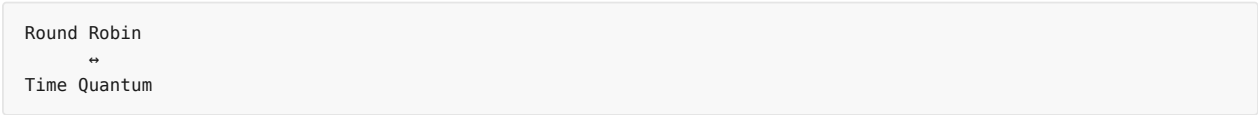
Layer 5

Ab librarian decide karta hai:

“Is book/concept ko library ke kis section aur shelf par rakhna chahiye?”

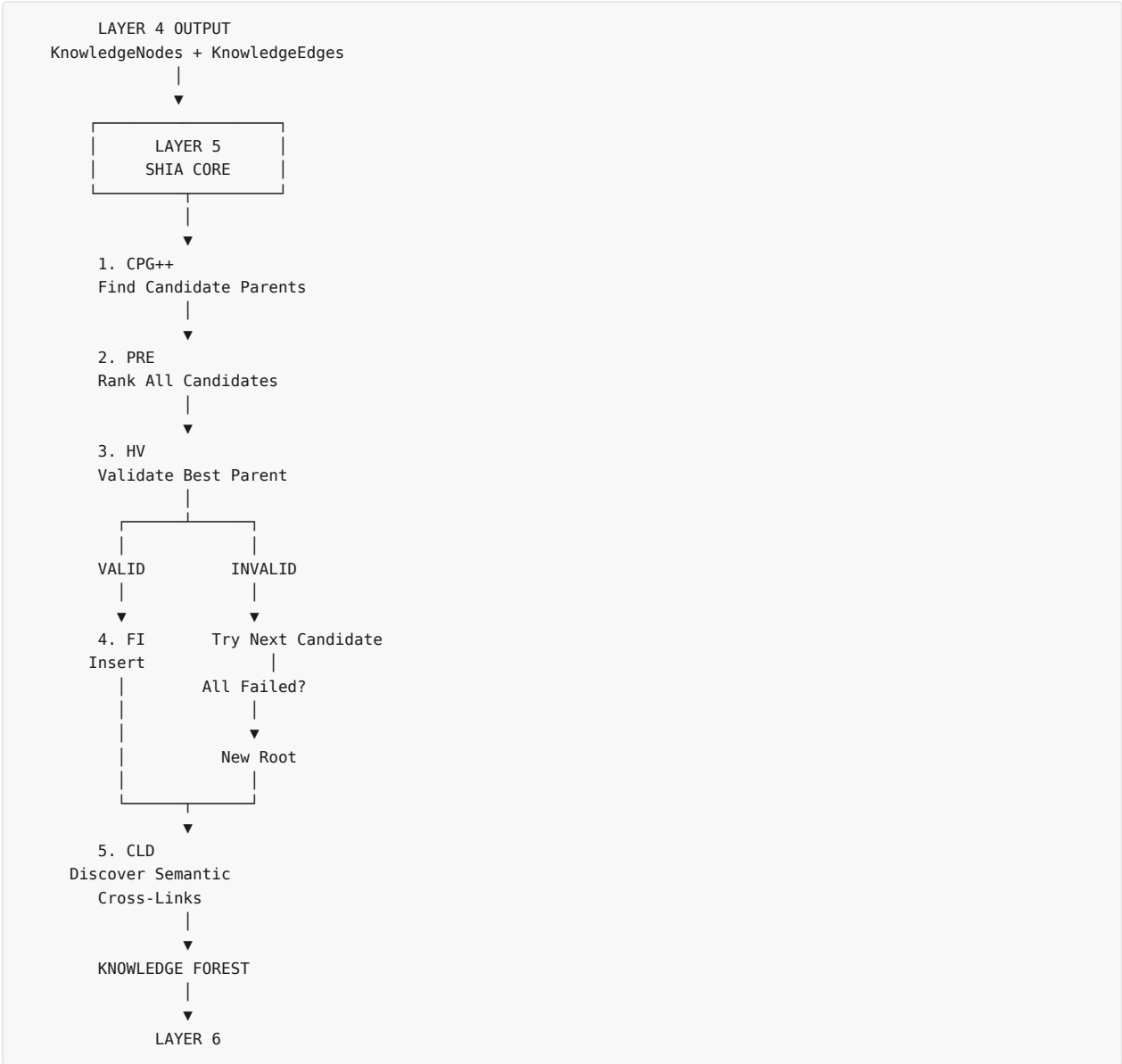


Aur ek extra reference card:



Yahi Layer 5 ka actual role hai.

44. Complete Layer 5 architecture



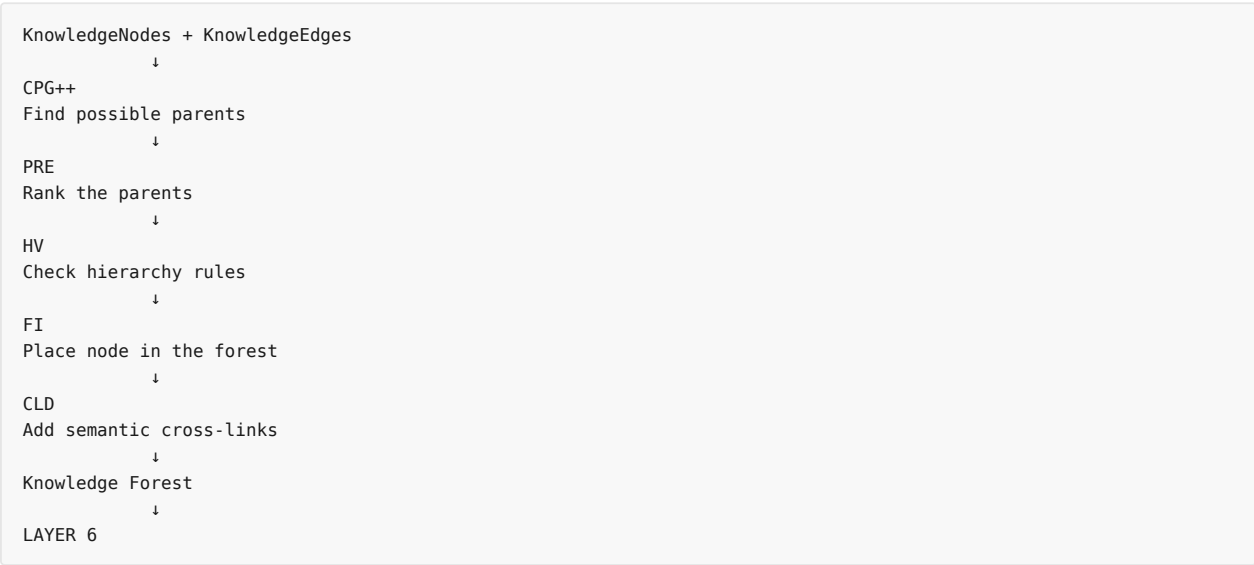
45. Final Summary Table

- | Point | Layer 5 |
- |---|
- | **Name** | SHIA Core / Hierarchy Construction |
- | **Input** | KnowledgeNode[] + KnowledgeEdge[] |
- | **Main Purpose** | Concepts ko meaningful hierarchy mein organize karna |
- | **Step 1** | CPG++ – Generate parent candidates |
- | **Step 2** | PRE – Rank candidates |
- | **Step 3** | HV – Validate hierarchy |
- | **Step 4** | FI – Insert node into forest |
- | **Step 5** | CLD – Discover semantic cross-links |
- | **Hierarchy Rule** | Acyclic |
- | **Parent Rule** | Strict single parent |
- | **No Valid Parent** | Create new root |
- | **Final Output** | Knowledge Forest |
- | **Next Layer** | Layer 6 – Retrieval Engine |

Sabse important one-line understanding

“Layer 5 is the core hierarchy construction layer of SHIA-RAG. It takes validated concepts and relationships, finds the best parent for each concept, validates the placement, inserts the concept into the Knowledge Forest, and adds additional semantic cross-links for better retrieval.”

Complete flow yaad rakhna:



Sabse important conceptual shift:

Layer 4 tells us which concepts are connected. Layer 5 organizes those validated connections into a structured Knowledge Forest so that Layer 6 can retrieve information intelligently.

memcite

YOU 24/08/2026, 01:12:07

mujhe layer 5 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 5 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 24/08/2026, 01:12:16

```
{ "size": "1024x1024", "n": 1, "is_style_transfer": false }
```

CHATGPT 24/08/2026, 01:13:06

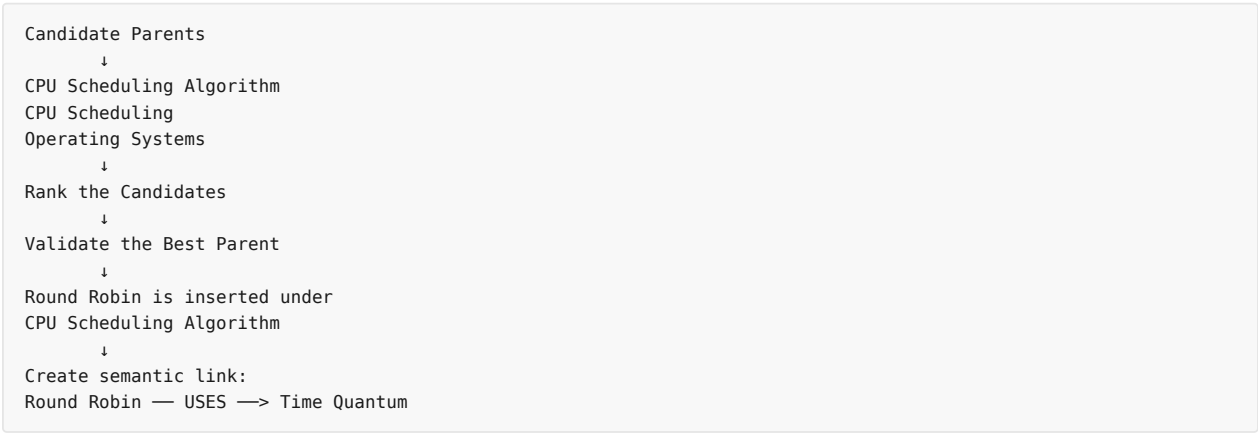
Layer 5 – SHIA Core / Hierarchy Construction

You can explain it to your guide like this:

“Layer 5 is the core organization layer of the system. It receives validated KnowledgeNodes and KnowledgeEdges from Layer 4 and organizes them into a structured Knowledge Forest. For each new concept, it finds possible parent concepts, ranks them, validates the best parent, inserts the concept into the hierarchy, and creates additional semantic cross-links between related concepts.”

Simple Example

Suppose the new concept is **Round Robin**.



Main Components

- **CPG++** – Finds possible parent concepts.
- **PRE** – Ranks the possible parents.
- **HV** – Validates the hierarchy and prevents problems such as cycles.
- **FI** – Inserts the concept into the Knowledge Forest.
- **CLD** – Creates semantic cross-links between related concepts.

Output

A structured Knowledge Forest containing hierarchical relationships and semantic cross-links, which is then passed to Layer 6 for intelligent retrieval.

In one simple line

“Layer 5 converts validated concepts and relationships into an organized Knowledge Forest by deciding where each concept belongs and how related concepts should be connected.”

YOU 24/08/2026, 01:19:32

ab mujhe layer 6 smjhao detailed me jese baki layers smjhayi thi

CHATGPT 24/08/2026, 01:19:34

```
{ "paths": [ "files" ] }
```

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite`turn17file0

```

[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_000000001dd481fa9855f2731dbc53d5
[L4] version_id: 1
[L5] name: SHIA_RAG_Complete_Report(6).md
[L6] mime_type: text/markdown
[L7] surface: library
[L8] score: 0.020176640779655856
[L9] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--45
[L10] snippet:
[L11] @dataclass
[L12] class RetrievalResult:
[L13] """Output of Layer 6 — input to Layer 7."""
[L14] selected_nodes: list # Nodes selected by Tree Knapsack
[L15] context_text: str # Assembled context string
[L16] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L17] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L18] query_intent: str # DEFINITION, COMPARISON, etc.
[L19] seed_node_ids: list # Initial seed nodes from vector search
[L20] token_count: int # Estimated tokens in context
[L21]
[L22] class RetrievalEngine:
[L23] def __init__(self, forest, vector_index, cache_manager, config):
[L24] self.forest = forest
[L25] self.vector_index = vector_index
[L26] self.cache = cache_manager # Redis cache integration
[L27] self.config = config
[L28] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L29]
[L30] def retrieve(self, query) -> RetrievalResult:
[L31] # Step 0: Cache check
[L32] cache_key = hash_query(query)
[L33] cached = self.cache.get_query_result(cache_key)
[L34] if cached:
[L35] return cached
[L36]
[L37] # Step 1: Analyze query
[L38] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L39] complexity = classify_complexity(intent) # Rule-based mapping
[L40]
[L41] # Step 2: Embed and find seed nodes
[L42] query_emb = embed_query(query) # Asymmetric embedding model
[L43] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L44]
[L45] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L46] depth_config = {
[L47] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L48] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},

```

```

[L49] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L50] }
[L51] traversal = depth_config[complexity]
[L52]
[L53] # Step 4: Traverse forest from all seeds
[L54] candidate_nodes = set()
[L55] for seed in seed_nodes:
[L56] candidate_nodes |= self.traverse(seed, traversal)
[L57]
[L58] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L59] selected = self.multi_tree_knapsack(
[L60] candidate_nodes,
[L61] budget=self.config.token_budget
[L62] )
[L63]
[L64] # Step 6: Assemble context in semantic order
[L65] context_text = self.assemble_context(selected, intent)
[L66]
[L67] # Build result with traversal path for Layer 8 feedback
[L68] result = RetrievalResult(
[L69] selected_nodes=selected,
[L70] context_text=context_text,
[L71] traversal_path_edges=self._get_path_edges(selected),
[L72] query_complexity=complexity,
[L73] query_intent=intent,
[L74] seed_node_ids=[s.id for s in seed_nodes],
[L75] token_count=estimate_tokens(context_text)
[L76] )
[L77]
[L78] # Cache result
[L79] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L80]
[L81] return result
[L82]
[L83] def multi_tree_knapsack(self, candidate_nodes, budget):
[L84] """Apply Tree Knapsack per distinct tree, then merge results.
[L85]
[L86] For multi-seed queries, candidates may span multiple trees.
[L87] We optimize each tree independently, then merge within budget.

```

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite[turn17file1]`

```

[L88] metadata:
[L89] query_result_index: 1
[L90] file_id: file_000000003c8081fab658b1752549d64
[L91] version_id: 1
[L92] name: SHIA_RAG_Complete_Report(5).md
[L93] mime_type: text/markdown
[L94] surface: library

```

```

[L95] score: 0.020176640779655856
[L96] document_chunk_id: file_000000003c8081fab658b1752549d64--1--45
[L97] snippet:
[L98] @dataclass
[L99] class RetrievalResult:
[L100] """Output of Layer 6 — input to Layer 7."""
[L101] selected_nodes: list # Nodes selected by Tree Knapsack
[L102] context_text: str # Assembled context string
[L103] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L104] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L105] query_intent: str # DEFINITION, COMPARISON, etc.
[L106] seed_node_ids: list # Initial seed nodes from vector search
[L107] token_count: int # Estimated tokens in context
[L108]
[L109] class RetrievalEngine:
[L110] def __init__(self, forest, vector_index, cache_manager, config):
[L111] self.forest = forest
[L112] self.vector_index = vector_index
[L113] self.cache = cache_manager # Redis cache integration
[L114] self.config = config
[L115] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L116]
[L117] def retrieve(self, query) -> RetrievalResult:
[L118] # Step 0: Cache check
[L119] cache_key = hash_query(query)
[L120] cached = self.cache.get_query_result(cache_key)
[L121] if cached:
[L122] return cached
[L123]
[L124] # Step 1: Analyze query
[L125] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L126] complexity = classify_complexity(intent) # Rule-based mapping
[L127]
[L128] # Step 2: Embed and find seed nodes
[L129] query_emb = embed_query(query) # Asymmetric embedding model
[L130] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L131]
[L132] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L133] depth_config = {
[L134] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L135] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L136] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L137] }
[L138] traversal = depth_config[complexity]
[L139]
[L140] # Step 4: Traverse forest from all seeds
[L141] candidate_nodes = set()
[L142] for seed in seed_nodes:
[L143] candidate_nodes |= self.traverse(seed, traversal)
[L144]

```



```

[L145] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L146] selected = self.multi_tree_knapsack(
[L147] candidate_nodes,
[L148] budget=self.config.token_budget
[L149] )
[L150]
[L151] # Step 6: Assemble context in semantic order
[L152] context_text = self.assemble_context(selected, intent)
[L153]
[L154] # Build result with traversal path for Layer 8 feedback
[L155] result = RetrievalResult(
[L156] selected_nodes=selected,
[L157] context_text=context_text,
[L158] traversal_path_edges=self._get_path_edges(selected),
[L159] query_complexity=complexity,
[L160] query_intent=intent,
[L161] seed_node_ids=[s.id for s in seed_nodes],
[L162] token_count=estimate_tokens(context_text)
[L163] )
[L164]
[L165] # Cache result
[L166] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L167]
[L168] return result
[L169]
[L170] def multi_tree_knapsack(self, candidate_nodes, budget):
[L171] """Apply Tree Knapsack per distinct tree, then merge results.
[L172]
[L173] For multi-seed queries, candidates may span multiple trees.
[L174] We optimize each tree independently, then merge within budget.

```

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite[turn17file2]`

```

[L175] metadata:
[L176] query_result_index: 2
[L177] file_id: file_00000000a0a481fab4c739c73d87551b
[L178] version_id: 1
[L179] name: SHIA_RAG_Complete_Report(7).md
[L180] mime_type: text/markdown
[L181] surface: library
[L182] score: 0.020029563932002958
[L183] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--45
[L184] snippet:
[L185] @dataclass
[L186] class RetrievalResult:
[L187] """Output of Layer 6 — input to Layer 7."""
[L188] selected_nodes: list # Nodes selected by Tree Knapsack
[L189] context_text: str # Assembled context string
[L190] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)

```

```

[L191] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L192] query_intent: str # DEFINITION, COMPARISON, etc.
[L193] seed_node_ids: list # Initial seed nodes from vector search
[L194] token_count: int # Estimated tokens in context
[L195]
[L196] class RetrievalEngine:
[L197] def __init__(self, forest, vector_index, cache_manager, config):
[L198] self.forest = forest
[L199] self.vector_index = vector_index
[L200] self.cache = cache_manager # Redis cache integration
[L201] self.config = config
[L202] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L203]
[L204] def retrieve(self, query) -> RetrievalResult:
[L205] # Step 0: Cache check
[L206] cache_key = hash_query(query)
[L207] cached = self.cache.get_query_result(cache_key)
[L208] if cached:
[L209] return cached
[L210]
[L211] # Step 1: Analyze query
[L212] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L213] complexity = classify_complexity(intent) # Rule-based mapping
[L214]
[L215] # Step 2: Embed and find seed nodes
[L216] query_emb = embed_query(query) # Asymmetric embedding model
[L217] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L218]
[L219] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L220] depth_config = {
[L221] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L222] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L223] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L224] }
[L225] traversal = depth_config[complexity]
[L226]
[L227] # Step 4: Traverse forest from all seeds
[L228] candidate_nodes = set()
[L229] for seed in seed_nodes:
[L230] candidate_nodes |= self.traverse(seed, traversal)
[L231]
[L232] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L233] selected = self.multi_tree_knapsack(
[L234] candidate_nodes,
[L235] budget=self.config.token_budget
[L236] )
[L237]
[L238] # Step 6: Assemble context in semantic order
[L239] context_text = self.assemble_context(selected, intent)
[L240]

```

```
[L241] # Build result with traversal path for Layer 8 feedback
[L242] result = RetrievalResult(
[L243] selected_nodes=selected,
[L244] context_text=context_text,
[L245] traversal_path_edges=self._get_path_edges(selected),
[L246] query_complexity=complexity,
[L247] query_intent=intent,
[L248] seed_node_ids=[s.id for s in seed_nodes],
[L249] token_count=estimate_tokens(context_text)
[L250] )
[L251]
[L252] # Cache result
[L253] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L254]
[L255] return result
[L256]
[L257] def multi_tree_knapsack(self, candidate_nodes, budget):
[L258] """Apply Tree Knapsack per distinct tree, then merge results.
[L259]
[L260] For multi-seed queries, candidates may span multiple trees.
[L261] We optimize each tree independently, then merge within budget.
```

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite`turn17file3

```
[L262] metadata:
[L263] query_result_index: 3
[L264] file_id: file_00000000dcc082119f935683a5e13eed
[L265] version_id: 1
[L266] name: SHIA_RAG_Final_Report(4)(1).md
[L267] mime_type: text/markdown
[L268] surface: conversation
[L269] score: 0.019151138716356108
[L270] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
[L271] snippet:
[L272] Gap: No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3)
hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L273]
[L274] ### 2.3 Literature Search Methodology
[L275]
[L276] | Parameter | Details |
[L277] |-----|-----|
[L278] | Databases Searched | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L279] | Conferences/Venues | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L280] | Search Keywords | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval
augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L281] | Time Range | 2020–2026 |
[L282] | Key Papers Reviewed | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk
(LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L283]
```

[L284] ---

[L285]

[L286] ## 3. System Architecture

[L287]

[L288] ### 3.1 Overview

[L289]

[L290] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L291]

[L292] | Phase | Layers | Mode | Purpose |

[L293] |-----|-----|-----|-----|

[L294] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest

|

[L295] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L296] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L297]

[L298] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L299]

[L300]

```
[L301] graph TD
[L302]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L303]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L304]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L305]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L306]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L307]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L308]     end
[L309]
[L310]     subgraph "Phase 2: Online – Retrieval & Generation"
[L311]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L312]         KF --> L6
[L313]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L314]         L7 --> A["Answer + Citations"]
[L315]     end
[L316]
[L317]     subgraph "Phase 3: Continuous – Operations"
[L318]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L319]         A --> L8
[L320]         L8 --> SE["Self-Evolution<br/>Feedback"] | KF
[L321]     end
[L322]
```

[L323]

[L324] ### 3.2 Data Flow

[L325]

[L326] ```

[L327] Raw Document (PDF/DOCX/HTML)

[L328] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L329] Normalized Text + Metadata

[L330] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L331] [Page → Region → Block] hierarchy

[L332] ↓ Layer 3: Split sentences,

[L333] metadata:
 [L334] query_result_index: 4
 [L335] file_id: file_000000004bc481fdbf9e949ec3b7544b
 [L336] version_id: 1
 [L337] name: SHIA_RAG_Final_Report(2).md
 [L338] mime_type: text/markdown
 [L339] surface: library
 [L340] score: 0.018309151435211356
 [L341] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--6
 [L342] snippet:
 [L343]

```
[L344] graph TD
[L345]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L346]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L347]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L348]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L349]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L350]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L351]     end
[L352]
[L353]     subgraph "Phase 2: Online – Retrieval & Generation"
[L354]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L355]         KF --> L6
[L356]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L357]         L7 --> A["Answer + Citations"]
[L358]     end
[L359]
[L360]     subgraph "Phase 3: Continuous – Operations"
[L361]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L362]         A --> L8
[L363]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L364]     end
[L365]
```

[L366]
 [L367] ### 3.2 Data Flow
 [L368]
 [L369]

```
[L370] Raw Document (PDF/DOCX/HTML)
[L371]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L372] Normalized Text + Metadata
[L373]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L374] [Page → Region → Block] hierarchy
[L375]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L376] KnowledgeUnit objects (candidate concepts)
[L377]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L378] KnowledgeNode objects (canonical, deduplicated)
[L379]     ↓ Layer 4: Validate relations against ontology, score confidence
[L380] KnowledgeEdge objects (typed, scored)
[L381]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L382] Knowledge Forest (rooted trees + cross-links)
[L383]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L384] Selected context nodes (within token budget)
[L385]     ↓ Layer 7: Build prompt, invoke LLM
[L386]     ↓ Layer 7: Verify claims against evidence, bind citations
[L387] Answer with citations + confidence score
[L388]     ↓ Layer 8: Log, collect feedback, update forest weights
[L389]
```

[L390]

[L391] ### 3.3 Layer Summary

[L392]

[L393] | Layer | Name | Input | Output | Key Algorithms |

[L394] |-----|-----|-----|-----|-----|

[L395] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L396] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L397] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L398] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L399] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L400] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L401] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L402] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L403] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L404]

[L405] ---

[L406]

[L407] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: turn17file5

[L408] metadata:

[L409] query_result_index: 5

[L410] file_id: file_000000003218820b918ee3d86ec345ae

[L411] version_id: 1

[L412] name: SHIA_RAG_Final_Report(1).md

[L413] mime_type: text/markdown

[L414] surface: library

[L415] score: 0.018309151435211356

[L416] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--6

[L417] snippet:

[L418]

```
[L419] graph TD
[L420]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L421]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L422]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L423]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L424]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L425]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L426]     end
[L427]
[L428]     subgraph "Phase 2: Online – Retrieval & Generation"
[L429]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L430]         KF --> L6
[L431]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L432]         L7 --> A["Answer + Citations"]
[L433]     end
[L434]
[L435]     subgraph "Phase 3: Continuous – Operations"
[L436]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L437]         A --> L8
[L438]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L439]     end
[L440]
```

[L441]

[L442] ### 3.2 Data Flow

[L443]

[L444]

```
[L445] Raw Document (PDF/DOCX/HTML)
[L446]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L447] Normalized Text + Metadata
[L448]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L449] [Page → Region → Block] hierarchy
[L450]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L451] KnowledgeUnit objects (candidate concepts)
[L452]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L453] KnowledgeNode objects (canonical, deduplicated)
[L454]     ↓ Layer 4: Validate relations against ontology, score confidence
[L455] KnowledgeEdge objects (typed, scored)
[L456]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L457] Knowledge Forest (rooted trees + cross-links)
[L458]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L459] Selected context nodes (within token budget)
[L460]     ↓ Layer 7: Build prompt, invoke LLM
[L461]     ↓ Layer 7: Verify claims against evidence, bind citations
[L462] Answer with citations + confidence score
[L463]     ↓ Layer 8: Log, collect feedback, update forest weights
[L464]
```

[L465]

[L466] ### 3.3 Layer Summary

[L467]

[L468] | Layer | Name | Input | Output | Key Algorithms |

[L469] |-----|-----|-----|-----|-----|

[L470] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L471] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L472] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L473] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L474] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L475] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L476] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L477] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L478] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L479]
[L480] ---
[L481]
[L482] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `[[filecite]]turn17file6`

[L483] metadata:
[L484] query_result_index: 6
[L485] file_id: file_0000000033088207b45d2ae58cbd406c
[L486] version_id: 1
[L487] name: SHIA_RAG_Complete_Report(8).md
[L488] mime_type: text/markdown
[L489] surface: library
[L490] score: 0.018309151435211356
[L491] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--13
[L492] snippet:
[L493] **Resolution:** Unify into a single canonical formula:
[L494]
[L495]
$$\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}(P)}{D_{\max}} + w_4 \cdot S_{\text{evi}}(P, N) + w_5 \cdot \text{Conf}(P)$$

[L496]
[L497] Where $\sum_{i=1}^5 w_i = 1$, defaults: $w_1=0.30, w_2=0.25, w_3=0.15, w_4=0.20, w_5=0.10$, $D_{\max}=8$ (configurable)
[L498]
[L499] ### 4.2 Architecture: 13 Modules vs 8 Layers
[L500]
[L501] | PDF | Architecture Style |
[L502] |----|-----|
[L503] | PDFs 1, 5 | 13 sequential modules (Modules 1-13) |
[L504] | PDFs 4, 6 | 8 architectural layers (Layers 0-8) |
[L505]
[L506] **Resolution:** The 8-layer model is more mature. Map modules into layers:
[L507]
[L508] | Layer | Modules Contained |
[L509] |-----|-----|
[L510] | Layer 0: Data Model | Core schemas |
[L511] | Layer 1: Input Processing | Module 1 (Document Parser) |
[L512] | Layer 2: Document Parsing | Module 2 (Structure Analyzer) |
[L513] | Layer 3: Knowledge Extraction | Modules 3-4 (Extractor + Normalizer) |
[L514] | Layer 4: Relation Management | Module 5 (Relation Discovery) + Module 6 (KCE) |
[L515] | Layer 5: SHIA Core | Module 7 (Hierarchy Builder: CPG++, PRE, HV, FI, CLD) |
[L516] | Layer 6: Retrieval Engine | Modules 8-10 (Query Analyzer + Retrieval + Context Assembly) |
[L517] | Layer 7: Generation Engine | Module 11 (LLM Generator) + Module 12 (Citation Verifier) |
[L518] | Layer 8: Operations | Logging, monitoring, eval + Module 13 (Evaluation/SHEF) |
[L519]
[L520] ### 4.3 Tree vs DAG vs Graph

[L521]
[L522] | PDF | Claim |
[L523] |----|-----|
[L524] | All PDFs | "Knowledge Forest is acyclic (no loops)" |
[L525] | All PDFs | "Cross-Link Discovery (CLD) adds semantic associations" |
[L526]
[L527] **Contradiction:** Cross-links create graph cycles. The forest is only acyclic in its **hierarchical edges** (parent-child). Cross-links form a separate **semantic graph** layer.
[L528]
[L529] **Resolution:** Define two distinct edge types:
[L530] 1. **Hierarchical edges** (IS_A, PART_OF) — must be acyclic, form a forest
[L531] 2. **Semantic edges** (USES, RELATED_TO, CAUSES) — may form cycles, separate graph layer
[L532]
[L533] > [!IMPORTANT]
[L534] > **Single-Parent Constraint:** SHIA-RAG enforces **strict single-parent trees** for hierarchical edges. Each node has at most one parent in the hierarchy. DAG support (multiple parents) is deferred to future work. This simplifies:
[L535] > - Precedence constraints in Tree Knapsack: $x_i \leq x_{\text{parent}(i)}$ is unambiguous
[L536] > - Confidence propagation: single parent path, no need for multi-parent averaging
[L537] > - Cycle detection: simple ancestor traversal
[L538] >
[L539] > Multiple semantic associations are handled via **SEMANTIC** edges (cross-links), which are unrestricted.

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite`turn17file7

[L540] metadata:
[L541] query_result_index: 7
[L542] file_id: file_000000003c8081fab658b1752549d64
[L543] version_id: 1
[L544] name: SHIA_RAG_Complete_Report(5).md
[L545] mime_type: text/markdown
[L546] surface: library
[L547] score: 0.018168360137272056
[L548] document_chunk_id: file_000000003c8081fab658b1752549d64--1--19
[L549] snippet:
[L550] ---
[L551]
[L552] ### Novelty 3: Multi-Objective Hierarchy Energy Minimization
[L553]
[L554] **What exists:** TreeRAG and RAPTOR use simple clustering. GraphRAG uses flat triple extraction.
[L555]
[L556] **What SHIA-RAG proposes:** Hierarchy construction as a formal **energy minimization** problem:
[L557]
[L558]
$$\mathcal{F} = \sum_{(P_i, N_i)} \left[\lambda_1 \cdot L_{\text{sem}}(P_i, N_i) + \lambda_2 \cdot L_{\text{depth}}(N_i) + \lambda_3 \cdot L_{\text{conf}}(P_i, N_i) + \lambda_4 \cdot L_{\text{onto}}(P_i, N_i) \right]$$

[L559]
[L560] Where:
[L561] - L_{sem} = semantic distance loss (1 - cosine similarity)
[L562] - L_{depth} = depth regularization (penalizes too-deep or too-shallow trees)
[L563] - L_{conf} = confidence loss (low-confidence placements increase energy)

[L564] - L_{onto} = ontological consistency loss (parent should be more general)

[L565]

[L566] **Why novel:** No RAG paper has formalized hierarchy construction as a multi-objective optimization problem for hierarchy induction in RAG. The formulation enables principled trade-offs between semantic fit, structural balance, confidence, and ontological consistency.

[L567]

[L568] > [!NOTE]

[L569] > The L_{onto} term uses a **sigmoid relaxation** $\sigma(\text{type_score}(P) - \text{type_score}(N))$ instead of a non-differentiable indicator function. See Section 8.1 for the corrected formulation.

[L570]

[L571] ---

[L572]

[L573] ### Novelty 4: Adaptive Depth Traversal via Query Complexity Classification

[L574]

[L575] **What exists:** All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.

[L576]

[L577] **What SHIA-RAG proposes:** A **query complexity classifier** that dynamically adjusts traversal depth:

[L578]

[L579] | Query Type | Example | Traversal Strategy |

[L580] |-----|-----|-----|

[L581] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |

[L582] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |

[L583] | MULTI-HOP | "How does OSPF convergence affect network reliability?" | Deep: Full path traversal + cross-links (depth=3+) |

[L584] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Retrieve at root/topic level, expand selectively |

[L585]

[L586] ```python

[L587] class QueryComplexityClassifier:

[L588] def classify(self, query):

[L589] features = {

[L590] 'num_entities': count_entities(query),

[L591] 'has_comparison': any(w in query.lower() for w in ['compare', 'vs', 'difference']),

[L592] 'has_causal': any(w in query.lower() for w in ['why', 'how', 'cause', 'effect']),

[L593] 'num_hops_estimated': estimate_reasoning_hops(query),

[L594] }

[L595] # Lightweight classifier (fine-tuned BERT or rule-based)

[L596] return self.model.predict(features) # → SIMPLE | MODERATE | COMPLEX | SYNTHESIS

[L597]

[L598] def estimate_reasoning_hops(query):

[L599] """Rule-based heuristic to estimate the number of reasoning hops

[L600] required to answer a query. Returns an integer in [1, 5]."""

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite`turn17file8

[L601] metadata:

[L602] query_result_index: 8

[L603] file_id: file_00000000a0a481fab4c739c73d87551b

[L604] version_id: 1

[L605] name: SHIA_RAG_Complete_Report(7).md

```

[L606] mime_type: text/markdown
[L607] surface: library
[L608] score: 0.018057336133766127
[L609] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--63
[L610] snippet:
[L611] 5"""
[L612] "down": ""ALTER TABLE knowledge_nodes
[L613] DROP COLUMN abstraction_level,
[L614] DROP COLUMN base_confidence,
[L615] DROP COLUMN extraction_confidence"""
[L616] },
[L617] {
[L618] "version": 3,
[L619] "description": "Migrate properties from string array to JSONB",
[L620] "up": ""ALTER TABLE knowledge_nodes
[L621] ALTER COLUMN data TYPE JSONB USING data::JSONB""",
[L622] "down": "-- Manual rollback required"
[L623] }
[L624] ]
[L625]
[L626] def run_migrations(db, target_version=None):
[L627]     current = db.get_migration_version()
[L628]     for migration in MIGRATIONS:
[L629]         if migration["version"] > current:
[L630]             if target_version and migration["version"] > target_version:
[L631]                 break
[L632]             db.execute(migration["up"])
[L633]             db.set_migration_version(migration["version"])
[L634]             print(f"Applied migration v{migration['version']}: {migration['description']}")
[L635]

```

```

[L636]
[L637] ---
[L638]
[L639] ### 7.13 Error Handling & Retry Logic
[L640]
[L641]

```

python

```

[L642] import time
[L643] from functools import wraps
[L644]
[L645] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L646]     """Decorator for retrying failed operations with exponential backoff."""
[L647]     def decorator(func):
[L648]         @wraps(func)
[L649]         def wrapper(*args, **kwargs):
[L650]             for attempt in range(max_retries):
[L651]                 try:
[L652]                     return func(*args, **kwargs)
[L653]                 except (TimeoutError, ConnectionError) as e:
[L654]                     if attempt == max_retries - 1:

```

```

[L655] raise
[L656] delay = min(base_delay * (2 ** attempt), max_delay)
[L657] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L658] time.sleep(delay)
[L659] return wrapper
[L660] return decorator
[L661]
[L662] class ProcessingPipeline:
[L663] """Error handling for the SHIA-RAG processing pipeline."""
[L664]
[L665] def process_document(self, doc):
[L666] # Layer 1: OCR with fallback
[L667] try:
[L668] text = self.extract_text(doc)
[L669] except OCRFailure:
[L670] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L671]
[L672] # Layer 3: LLM extraction with retry + fallback model
[L673] try:
[L674] concepts = self.llm_extract_concepts(text)
[L675] except LLMLTimeout:
[L676] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L677]
[L678] # Layer 6: Vector DB with cache fallback
[L679] try:
[L680] results = self.vector_search(query_embedding)
[L681] except VectorDBUnavailable:
[L682] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L683]
[L684] return {"status": "SUCCESS", "concepts": concepts}
[L685]

```

```

[L686]
[L687] ---
[L688]
[L689] ### 7.14 Caching Strategy
[L690]
[L691]

```

```

python
[L692] import redis
[L693] import hashlib
[L694] import json
[L695]
[L696] class CacheManager:
[L697] """Multi-layer caching for SHIA-RAG."""
[L698]
[L699] def __init__(self, redis_url="redis://localhost:6379"):
[L700] self.redis = redis.from_url(redis_url)
[L701] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L702] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L703] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings

```

```
[L704]
[L705] def get_query_result(self, query_text):
[L706] """Check if this exact query has been answered recently."""
```

DEEP RESEARCH 24/08/2026, 01:19:42

Citation Marker: `filecite`turn17file9

```
[L707] metadata:
[L708] query_result_index: 9
[L709] file_id: file_000000003c8081fab658b1752549d64
[L710] version_id: 1
[L711] name: SHIA_RAG_Complete_Report(5).md
[L712] mime_type: text/markdown
[L713] surface: library
[L714] score: 0.018057336133766127
[L715] document_chunk_id: file_000000003c8081fab658b1752549d64--1--63
[L716] snippet:
[L717] 5"""
[L718] "down": """ALTER TABLE knowledge_nodes
[L719] DROP COLUMN abstraction_level,
[L720] DROP COLUMN base_confidence,
[L721] DROP COLUMN extraction_confidence"""
[L722] },
[L723] {
[L724] "version": 3,
[L725] "description": "Migrate properties from string array to JSONB",
[L726] "up": """ALTER TABLE knowledge_nodes
[L727] ALTER COLUMN data TYPE JSONB USING data::JSONB"""
[L728] "down": "-- Manual rollback required"
[L729] }
[L730] ]
[L731]
[L732] def run_migrations(db, target_version=None):
[L733] current = db.get_migration_version()
[L734] for migration in MIGRATIONS:
[L735] if migration["version"] > current:
[L736] if target_version and migration["version"] > target_version:
[L737] break
[L738] db.execute(migration["up"])
[L739] db.set_migration_version(migration["version"])
[L740] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L741]
```

```
[L742]
[L743] ---
[L744]
[L745] ### 7.13 Error Handling & Retry Logic
[L746]
[L747]
```

python

```
[L748] import time
```

```
[L749] from functools import wraps
[L750]
[L751] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L752] """Decorator for retrying failed operations with exponential backoff."""
[L753] def decorator(func):
[L754] @wraps(func)
[L755] def wrapper(*args, **kwargs):
[L756] for attempt in range(max_retries):
[L757] try:
[L758] return func(*args, **kwargs)
[L759] except (TimeoutError, ConnectionError) as e:
[L760] if attempt == max_retries - 1:
[L761] raise
[L762] delay = min(base_delay * (2 ** attempt), max_delay)
[L763] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L764] time.sleep(delay)
[L765] return wrapper
[L766] return decorator
[L767]
[L768] class ProcessingPipeline:
[L769] """Error handling for the SHIA-RAG processing pipeline."""
[L770]
[L771] def process_document(self, doc):
[L772] # Layer 1: OCR with fallback
[L773] try:
[L774] text = self.extract_text(doc)
[L775] except OCRFailure:
[L776] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L777]
[L778] # Layer 3: LLM extraction with retry + fallback model
[L779] try:
[L780] concepts = self.llm_extract_concepts(text)
[L781] except LLMLLMTimeout:
[L782] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L783]
[L784] # Layer 6: Vector DB with cache fallback
[L785] try:
[L786] results = self.vector_search(query_embedding)
[L787] except VectorDBUnavailable:
[L788] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L789]
[L790] return {"status": "SUCCESS", "concepts": concepts}
[L791]
```

```
[L792]  
[L793] ---  
[L794]  
[L795] ### 7.14 Caching Strategy  
[L796]  
[L797]
```

```
python
[L798] import redis
[L799] import hashlib
[L800] import json
[L801]
[L802] class CacheManager:
[L803] """Multi-layer caching for SHIA-RAG."""
[L804]
[L805] def __init__(self, redis_url="redis://localhost:6379"):
[L806] self.redis = redis.from_url(redis_url)
[L807] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L808] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L809] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L810]
[L811] def get_query_result(self, query_text):
[L812] """Check if this exact query has been answered recently."""
```

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: `filecite`turn18file0

```
[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4).md
[L6] mime_type: text/markdown
[L7] surface: library
[L8] score: 0.03131881575727918
[L9] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--2
[L10] snippet:
[L11] | Problem | Description | Impact |
[L12] |-----|-----|-----|
[L13] | Broken semantic continuity | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |
[L14] | Duplicate/fragmented information | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |
[L15] | Fixed granularity | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |
[L16] | Weak multi-hop reasoning | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |
[L17] | No cross-document fusion | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |
[L18]
[L19] ### 1.2 Proposed Solution: SHIA-RAG
[L20]
[L21] SHIA-RAG (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured Knowledge Forest:
[L22]
[L23]
```

[L24]	Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L25]	SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L26]		

[L27]

[L28] The system:

[L29] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L30] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L31] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L32] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L33] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L34] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L35]

[L36] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L37]

[L38] ### 1.3 Objectives

[L39]

[L40] 1. Design a complete 8-layer architecture for hierarchical RAG

[L41] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L42] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L43] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L44] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L45] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L46]

[L47] ### 1.4 Contributions

[L48]

[L49] This work makes five novel contributions to the RAG literature:

[L50]

[L51] | # | Contribution | Novelty Level |

[L52] |---|-----|-----|

[L53] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L54] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L55] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L56] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L57] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L58]

[L59] ---

[L60]

[L61] ## 2. Literature Review

[L62]

[L63] ### 2.1 Related Work

[L64]

[L65] #### 2.1.1 Flat-Chunk RAG Systems

[L66] metadata:
[L67] query_result_index: 1
[L68] file_id: file_00000000fb2c81fd82f14a7a48993554
[L69] version_id: 1
[L70] name: SHIA_RAG_Final_Report.md
[L71] mime_type: text/markdown
[L72] surface: library
[L73] score: 0.031099324975891997
[L74] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--2
[L75] snippet:
[L76] | Problem | Description | Impact |
[L77] |-----|-----|-----|
[L78] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |
[L79] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |
[L80] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |
[L81] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |
[L82] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |
[L83]
[L84] ### 1.2 Proposed Solution: SHIA-RAG
[L85]
[L86] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:
[L87]
[L88]

[L89] Traditional RAG: Document → [Chunk₁, Chunk₂, ..., Chunk_n] → Top-k → LLM

[L90] SHIA-RAG: Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM

[L91]

[L92]
[L93] The system:
[L94] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents
[L95] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees
[L96] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth
[L97] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints
[L98] 5. **Generates verified answers** — with claim-level evidence verification and citation binding
[L99] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure
[L100]
[L101] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.
[L102]
[L103] ### 1.3 Objectives
[L104]
[L105] 1. Design a complete 8-layer architecture for hierarchical RAG
[L106] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L107] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L108] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L109] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L110] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L111]

[L112] ### 1.4 Contributions

[L113]

[L114] This work makes five novel contributions to the RAG literature:

[L115]

[L116] | # | Contribution | Novelty Level |

[L117] |---|-----|-----|

[L118] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L119] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L120] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L121] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L122] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L123]

[L124] ---

[L125]

[L126] ## 2. Literature Review

[L127]

[L128] ### 2.1 Related Work

[L129]

[L130] #### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: `filecite`turn18file2

[L131] metadata:

[L132] query_result_index: 2

[L133] file_id: file_000000003218820b918ee3d86ec345ae

[L134] version_id: 1

[L135] name: SHIA_RAG_Final_Report(1).md

[L136] mime_type: text/markdown

[L137] surface: library

[L138] score: 0.03047794966520434

[L139] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--2

[L140] snippet:

[L141] | Problem | Description | Impact |

[L142] |-----|-----|-----|

[L143] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L144] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L145] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L146] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |

[L147] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L148]

[L149] ### 1.2 Proposed Solution: SHIA-RAG

[L150]

[L151] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L152]

[L153]

[L154] Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L155] SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L156]	

[L157]

[L158] The system:

[L159] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L160] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L161] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L162] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L163] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L164] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L165]

[L166] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L167]

[L168] ### 1.3 Objectives

[L169]

[L170] 1. Design a complete 8-layer architecture for hierarchical RAG

[L171] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L172] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L173] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L174] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L175] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L176]

[L177] ### 1.4 Contributions

[L178]

[L179] This work makes five novel contributions to the RAG literature:

[L180]

[L181]	# Contribution Novelty Level
[L182]	--- ----- -----
[L183]	1 Precedence-Constrained Tree Knapsack for context selection Incremental (Novel Application)
[L184]	2 Self-Evolving Knowledge Forest via retrieval feedback Moderately Novel
[L185]	3 Multi-Objective Energy Minimization for hierarchy induction Moderately Novel
[L186]	4 Adaptive Depth Traversal via query complexity classification Incremental
[L187]	5 SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG Incremental (Novel Combination)
[L188]	
[L189]	---
[L190]	

[L191] ## 2. Literature Review

[L192]

[L193] ### 2.1 Related Work

[L194]
[L195] ##### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: [filecite]turn18file3[

[L196] metadata:
[L197] query_result_index: 3
[L198] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L199] version_id: 1
[L200] name: SHIA_RAG_Final_Report(2).md
[L201] mime_type: text/markdown
[L202] surface: library
[L203] score: 0.03047794966520434
[L204] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--2
[L205] snippet:
[L206] | Problem | Description | Impact |
[L207] |-----|-----|-----|
[L208] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |
[L209] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |
[L210] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |
[L211] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |
[L212] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |
[L213]
[L214] ### 1.2 Proposed Solution: SHIA-RAG
[L215]
[L216] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:
[L217]
[L218]

[L219] Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L220] SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L221]	

[L222]
[L223] The system:
[L224] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents
[L225] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees
[L226] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth
[L227] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints
[L228] 5. **Generates verified answers** — with claim-level evidence verification and citation binding
[L229] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure
[L230]
[L231] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems →

Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L232]

[L233] ### 1.3 Objectives

[L234]

[L235] 1. Design a complete 8-layer architecture for hierarchical RAG

[L236] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L237] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L238] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L239] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L240] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L241]

[L242] ### 1.4 Contributions

[L243]

[L244] This work makes five novel contributions to the RAG literature:

[L245]

[L246] | # | Contribution | Novelty Level |

[L247] |---|-----|-----|

[L248] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L249] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L250] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L251] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L252] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L253]

[L254] ---

[L255]

[L256] ## 2. Literature Review

[L257]

[L258] ### 2.1 Related Work

[L259]

[L260] #### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: `filecite{turn18file4}`

[L261] metadata:

[L262] query_result_index: 4

[L263] file_id: file_00000000f620821186e54e582d59731e

[L264] version_id: 1

[L265] name: SHIA_RAG_Final_Report(3).md

[L266] mime_type: text/markdown

[L267] surface: library

[L268] score: 0.03009207275993712

[L269] document_chunk_id: file_00000000f620821186e54e582d59731e--1--2

[L270] snippet:

[L271] | Problem | Description | Impact |

[L272] |-----|-----|-----|

[L273] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L274] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L275] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L276] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L277] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L278]

[L279] ### 1.2 Proposed Solution: SHIA-RAG

[L280]

[L281] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L282]

[L283]

[L284] Traditional RAG:	Document $\rightarrow$ [Chunk ₁ , Chunk ₂ , ..., Chunk _n] $\rightarrow$ Top-k $\rightarrow$ LLM
[L285] SHIA-RAG:	Document $\rightarrow$ Knowledge Forest (concepts + relations) $\rightarrow$ Adaptive Traversal $\rightarrow$ LLM
[L286]	

[L287]

[L288] The system:

[L289] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L290] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L291] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L292] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L293] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L294] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L295]

[L296] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science $\rightarrow$ Operating Systems $\rightarrow$ Scheduling $\rightarrow$ Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L297]

[L298] ### 1.3 Objectives

[L299]

[L300] 1. Design a complete 8-layer architecture for hierarchical RAG

[L301] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L302] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L303] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L304] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L305] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L306]

[L307] ### 1.4 Contributions

[L308]

[L309] This work makes five novel contributions to the RAG literature:

[L310]

[L311] | # | Contribution | Novelty Level |

[L312] |---|-----|-----|

[L313] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L314] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L315] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L316] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L317] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L318]
[L319] ---
[L320]
[L321] ## 2. Literature Review
[L322]
[L323] ### 2.1 Related Work
[L324]
[L325] #### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: `filecite[turn18file5]`

[L326] metadata:
[L327] query_result_index: 5
[L328] file_id: file_00000000dcc082119f935683a5e13eed
[L329] version_id: 1
[L330] name: SHIA_RAG_Final_Report(4)(1).md
[L331] mime_type: text/markdown
[L332] surface: conversation
[L333] score: 0.02761904761904762
[L334] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--2
[L335] snippet:
[L336] | Problem | Description | Impact |
[L337] |-----|-----|-----|
[L338] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |
[L339] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |
[L340] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |
[L341] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |
[L342] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |
[L343]
[L344] ### 1.2 Proposed Solution: SHIA-RAG
[L345]
[L346] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:
[L347]
[L348]

[L349]	Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L350]	SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L351]		

[L352]
[L353] The system:
[L354] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents
[L355] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L356] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L357] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L358] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L359] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L360]

[L361] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L362]

[L363] ### 1.3 Objectives

[L364]

[L365] 1. Design a complete 8-layer architecture for hierarchical RAG

[L366] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L367] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L368] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L369] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L370] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L371]

[L372] ### 1.4 Contributions

[L373]

[L374] This work makes five novel contributions to the RAG literature:

[L375]

[L376] | # | Contribution | Novelty Level |

[L377] | --|-----|-----|

[L378] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L379] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L380] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L381] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L382] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L383]

[L384] ---

[L385]

[L386] ## 2. Literature Review

[L387]

[L388] ### 2.1 Related Work

[L389]

[L390] #### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: fileciteturn18file6

[L391] metadata:

[L392] query_result_index: 6

[L393] file_id: file_00000000c708820bba7bf4726d6b4174

[L394] version_id: 1

[L395] name: Executive Summary (1)(3).pdf

[L396] mime_type: application/pdf

[L397] surface: library

[L398] score: 0.02712049508554787

[L399] document_chunk_id: file_00000000c708820bba7bf4726d6b4174:0--1--18

[L400] snippet:

- [L401] Use relation extraction: if n appears in a relation with m outside the tree, link them.
- [L402] Use knowledge bases: e.g. Wikipedia links.
- [L403] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.
- [L404] Context Assembler may include one-hop neighbors via CLD).
- [L405] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and
- [L406] additional edges are "related-to."
- [L407] Data Structures and Indexing
- [L408] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent
- [L409] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),
- [L410] embedding vector, and a version/timestamp.
- [L411] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)
- [L412] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.
- [L413] NetworkX, Neo4j).
- [L414] •
- [L415] •
- [L416] •
- [L417] •
- [L418] •
- [L419] •
- [L420] •
- [L421] •
- [L422] •
- [L423] •
- [L424] •
- [L425] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node
- [L426] embeddings. This supports CPG++ search and semantic neighbor queries.
- [L427] Ancillary Indexes:
- [L428] Entity-to-Node map: For quick lookups by name/alias.
- [L429] Relation Index: If needed, a mapping from relation type to node pairs.
- [L430] Metadata store: (document IDs, source text) for evidence retrieval.
- [L431] All indices are updated incrementally as nodes are added.
- [L432] Retrieval and Context Assembly
- [L433] 10. Query Analyzer
- [L434] Input: Natural language user query.
- [L435] Output: Query intent and candidate target concepts.
- [L436] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"
- [L437] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).
- [L438] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept
- [L439] matching (e.g. if query mentions "Round Robin", we directly target that node).
- [L440] 11. Semantic Retrieval Engine
- [L441] Input: Query intent, key terms.
- [L442] Output: Set of relevant KnowledgeNodes.
- [L443] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with
- [L444] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).
- [L445] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:
- [L446] Parent Expansion: If the query is general, include some ancestors of the matched nodes.
- [L447] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For
- [L448] example, TagRAG-style retrieval of tag chains 【10†L185-L194】 suggests siblings may hold related
- [L449] knowledge.)
- [L450] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's

[L451] semantic paths [29†L37-L44]).

[L452] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L453] sentences, and optionally a context snippet from source documents.

[L454] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L455] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L456] This process yields a Context Graph of nodes and edges relevant to the query.

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: [filecite]turn18file7[

[L457] metadata:

[L458] query_result_index: 7

[L459] file_id: file_00000000068482079b26e665931694b4

[L460] version_id: 1

[L461] name: Executive Summary (1).pdf

[L462] mime_type: application/pdf

[L463] surface: library

[L464] score: 0.02712049508554787

[L465] document_chunk_id: file_00000000068482079b26e665931694b4:0--1--18

[L466] snippet:

[L467] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L468] Use knowledge bases: e.g. Wikipedia links.

[L469] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L470] Context Assembler may include one-hop neighbors via CLD).

[L471] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L472] additional edges are "related-to."

[L473] Data Structures and Indexing

[L474] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L475] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L476] embedding vector, and a version/timestamp.

[L477] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L478] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L479] NetworkX, Neo4j).

[L480] •

[L481] •

[L482] •

[L483] •

[L484] •

[L485] •

[L486] •

[L487] •

[L488] •

[L489] •

[L490] •

[L491] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L492] embeddings. This supports CPG++ search and semantic neighbor queries.

[L493] Ancillary Indexes:

[L494] Entity-to-Node map: For quick lookups by name/alias.

[L495] Relation Index: If needed, a mapping from relation type to node pairs.

[L496] Metadata store: (document IDs, source text) for evidence retrieval.

[L497] All indices are updated incrementally as nodes are added.

[L498] Retrieval and Context Assembly

[L499] 10. Query Analyzer

[L500] Input: Natural language user query.

[L501] Output: Query intent and candidate target concepts.

[L502] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L503] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L504] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L505] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L506] 11. Semantic Retrieval Engine

[L507] Input: Query intent, key terms.

[L508] Output: Set of relevant KnowledgeNodes.

[L509] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L510] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L511] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L512] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L513] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L514] example, TagRAG-style retrieval of tag chains **【10†L185-L194】** suggests siblings may hold related

[L515] knowledge.)

[L516] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's

[L517] semantic paths **【29†L37-L44】**).

[L518] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L519] sentences, and optionally a context snippet from source documents.

[L520] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L521] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L522] This process yields a Context Graph of nodes and edges relevant to the query.

DEEP RESEARCH 24/08/2026, 01:19:47

Citation Marker: `filecite`turn18file8

[L523] metadata:

[L524] query_result_index: 8

[L525] file_id: file_00000000f7d08207bca62408a5680f95

[L526] version_id: 1

[L527] name: Executive Summary (1)(1).pdf

[L528] mime_type: application/pdf

[L529] surface: library

[L530] score: 0.026553280111072546

[L531] document_chunk_id: file_00000000f7d08207bca62408a5680f95:0--1--18

[L532] snippet:

[L533] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L534] Use knowledge bases: e.g. Wikipedia links.

[L535] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L536] Context Assembler may include one-hop neighbors via CLD).

[L537] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L538] additional edges are "related-to."

[L539] Data Structures and Indexing

[L540] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L541] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L542] embedding vector, and a version/timestamp.

[L543] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L544] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L545] NetworkX, Neo4j).

[L546] •

[L547] •

[L548] •

[L549] •

[L550] •

[L551] •

[L552] •

[L553] •

[L554] •

[L555] •

[L556] •

[L557] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L558] embeddings. This supports CPG++ search and semantic neighbor queries.

[L559] Ancillary Indexes:

[L560] Entity-to-Node map: For quick lookups by name/alias.

[L561] Relation Index: If needed, a mapping from relation type to node pairs.

[L562] Metadata store: (document IDs, source text) for evidence retrieval.

[L563] All indices are updated incrementally as nodes are added.

[L564] Retrieval and Context Assembly

[L565] 10. Query Analyzer

[L566] Input: Natural language user query.

[L567] Output: Query intent and candidate target concepts.

[L568] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L569] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L570] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L571] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L572] 11. Semantic Retrieval Engine

[L573] Input: Query intent, key terms.

[L574] Output: Set of relevant KnowledgeNodes.

[L575] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L576] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L577] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L578] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L579] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L580] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related

[L581] knowledge.)

[L582] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's

[L583] semantic paths【29†L37-L44】).

[L584] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L585] sentences, and optionally a context snippet from source documents.

[L586] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L587] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L588] This process yields a Context Graph of nodes and edges relevant to the query.

[L589] metadata:

[L590] query_result_index: 9

[L591] file_id: file_00000000a31c81fa97d0727ba9c7d681

[L592] version_id: 1

[L593] name: Executive Summary (1)(4).pdf

[L594] mime_type: application/pdf

[L595] surface: library

[L596] score: 0.026553280111072546

[L597] document_chunk_id: file_00000000a31c81fa97d0727ba9c7d681:0--1--18

[L598] snippet:

[L599] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L600] Use knowledge bases: e.g. Wikipedia links.

[L601] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L602] Context Assembler may include one-hop neighbors via CLD).

[L603] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L604] additional edges are "related-to."

[L605] Data Structures and Indexing

[L606] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L607] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L608] embedding vector, and a version/timestamp.

[L609] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L610] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L611] NetworkX, Neo4j).

[L612] •

[L613] •

[L614] •

[L615] •

[L616] •

[L617] •

[L618] •

[L619] •

[L620] •

[L621] •

[L622] •

[L623] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L624] embeddings. This supports CPG++ search and semantic neighbor queries.

[L625] Ancillary Indexes:

[L626] Entity-to-Node map: For quick lookups by name/alias.

[L627] Relation Index: If needed, a mapping from relation type to node pairs.

[L628] Metadata store: (document IDs, source text) for evidence retrieval.

[L629] All indices are updated incrementally as nodes are added.

[L630] Retrieval and Context Assembly

[L631] 10. Query Analyzer

[L632] Input: Natural language user query.

[L633] Output: Query intent and candidate target concepts.

[L634] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L635] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L636] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L637] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L638] 11. Semantic Retrieval Engine

[L639] Input: Query intent, key terms.

[L640] Output: Set of relevant KnowledgeNodes.

[L641] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L642] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L643] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L644] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L645] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L646] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related

[L647] knowledge.)

[L648] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's

[L649] semantic paths【29†L37-L44】).

[L650] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L651] sentences, and optionally a context snippet from source documents.

[L652] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L653] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L654] This process yields a Context Graph of nodes and edges relevant to the query.

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite[turn19file0]`

[L1] metadata:

[L2] query_result_index: 0

[L3] file_id: file_00000000a0a481fab4c739c73d87551b

[L4] version_id: 1

[L5] name: SHIA_RAG_Complete_Report(7).md

[L6] mime_type: text/markdown

[L7] surface: library

[L8] score: 0.02152247667514843

[L9] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--45

[L10] snippet:

[L11] @dataclass

[L12] class RetrievalResult:

[L13] """Output of Layer 6 — input to Layer 7."""

[L14] selected_nodes: list # Nodes selected by Tree Knapsack

[L15] context_text: str # Assembled context string

[L16] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)

[L17] query_complexity: str # SIMPLE, MODERATE, COMPLEX

[L18] query_intent: str # DEFINITION, COMPARISON, etc.

[L19] seed_node_ids: list # Initial seed nodes from vector search

[L20] token_count: int # Estimated tokens in context

[L21]

[L22] class RetrievalEngine:

[L23] def __init__(self, forest, vector_index, cache_manager, config):

[L24] self.forest = forest

[L25] self.vector_index = vector_index

[L26] self.cache = cache_manager # Redis cache integration

[L27] self.config = config

[L28] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes

[L29]

[L30] def retrieve(self, query) -> RetrievalResult:

```

[L31] # Step 0: Cache check
[L32] cache_key = hash_query(query)
[L33] cached = self.cache.get_query_result(cache_key)
[L34] if cached:
[L35] return cached
[L36]
[L37] # Step 1: Analyze query
[L38] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L39] complexity = classify_complexity(intent) # Rule-based mapping
[L40]
[L41] # Step 2: Embed and find seed nodes
[L42] query_emb = embed_query(query) # Asymmetric embedding model
[L43] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L44]
[L45] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L46] depth_config = {
[L47] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L48] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L49] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L50] }
[L51] traversal = depth_config[complexity]
[L52]
[L53] # Step 4: Traverse forest from all seeds
[L54] candidate_nodes = set()
[L55] for seed in seed_nodes:
[L56] candidate_nodes |= self.traverse(seed, traversal)
[L57]
[L58] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L59] selected = self.multi_tree_knapsack(
[L60] candidate_nodes,
[L61] budget=self.config.token_budget
[L62] )
[L63]
[L64] # Step 6: Assemble context in semantic order
[L65] context_text = self.assemble_context(selected, intent)
[L66]
[L67] # Build result with traversal path for Layer 8 feedback
[L68] result = RetrievalResult(
[L69] selected_nodes=selected,
[L70] context_text=context_text,
[L71] traversal_path_edges=self._get_path_edges(selected),
[L72] query_complexity=complexity,
[L73] query_intent=intent,
[L74] seed_node_ids=[s.id for s in seed_nodes],
[L75] token_count=estimate_tokens(context_text)
[L76] )
[L77]
[L78] # Cache result
[L79] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L80]

```

```
[L81] return result
[L82]
[L83] def multi_tree_knapsack(self, candidate_nodes, budget):
[L84] """Apply Tree Knapsack per distinct tree, then merge results.
[L85]
[L86] For multi-seed queries, candidates may span multiple trees.
[L87] We optimize each tree independently, then merge within budget.
```

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file1

```
[L88] metadata:
[L89] query_result_index: 1
[L90] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L91] version_id: 1
[L92] name: SHIA_RAG_Final_Report(4).md
[L93] mime_type: text/markdown
[L94] surface: library
[L95] score: 0.021491228070175436
[L96] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--54
[L97] snippet:
[L98] 1. Precedence-Constrained Tree Knapsack — to the best of our knowledge, the first formulation of RAG context
selection as a precedence-constrained optimization problem
[L99] 2. Self-Evolving Knowledge Forest — to the best of our knowledge, the first closed-loop feedback mechanism that
modifies RAG knowledge structures based on retrieval success/failure
[L100] 3. Multi-Objective Energy Minimization — a formal optimization framework for automated hierarchy induction
combining semantic similarity, depth, confidence, and type coherence
[L101] 4. Adaptive Depth Traversal — query-complexity-aware retrieval depth adaptation for hierarchical RAG
[L102] 5. SHEF Evaluation Framework — to the best of our knowledge, the first benchmark specifically designed for
hierarchical RAG quality
[L103]
[L104] The complete system specification covers:
[L105] - 9 layers (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
[L106] - 12 defined functions with complete implementations
[L107] - 12 relation types across 4 hierarchical and 8 semantic categories
[L108] - 8 schemas covering all data objects from document blocks to query logs
[L109] - A 6-month implementation roadmap with technology stack, folder structure, and CI/CD pipeline
[L110] - A comprehensive evaluation plan with datasets, baselines, ablation studies, and statistical significance testing
[L111]
[L112] ### 13.2 Key Novelties
[L113]
[L114] | # | Novelty | Level | Why It's New |
[L115] |--|-----|-----|-----|
[L116] | 1 | Tree Knapsack for Context Selection | Incremental (Novel Application) | To our knowledge, first RAG system to
formulate context selection as a precedence-constrained optimization |
[L117] | 2 | Self-Evolving Knowledge Forest | Moderately Novel | To our knowledge, first closed-loop feedback mechanism
that modifies RAG knowledge structures |
[L118] | 3 | Multi-Objective Energy Minimization | Moderately Novel | Novel multi-objective optimization combining 4
objectives for hierarchy induction in RAG |
[L119] | 4 | Adaptive Depth Traversal | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |
```


[L120] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L121]

[L122] ### 13.3 Next Steps

[L123]

[L124] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L125] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L126] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L127] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L128] 5. **Build Layer 4** (Relation management with ontology validation)

[L129] 6. **Build Layer 5** (SHIA Core with algorithms)

[L130] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L131] 8. **Build Layer 7** (Generation with verification)

[L132] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L133] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L134] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L135] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L136]

[L137] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file2

[L138] metadata:

[L139] query_result_index: 2

[L140] file_id: file_000000004bc481fdbf9e949ec3b7544b

[L141] version_id: 1

[L142] name: SHIA_RAG_Final_Report(2).md

[L143] mime_type: text/markdown

[L144] surface: library

[L145] score: 0.02146158925819943

[L146] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--40

[L147] snippet:

[L148] temporal versioning |

[L149] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L150] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L151] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L152]

[L153] ---

[L154]

[L155] ## 13. Conclusion & Future Work

[L156]

[L157] ### 13.1 Summary

[L158]

[L159] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L160]

[L161] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem

[L162] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge

structures based on retrieval success/failure

[L163] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction

[L164] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation

[L165] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality

[L166]

[L167] The complete system specification covers:

[L168] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L169] - **12 defined functions** with complete implementations

[L170] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L171] - **8 schemas** covering all data objects from document blocks to query logs

[L172] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L173] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L174]

[L175] ### 13.2 Key Novelties

[L176]

[L177] | # | Novelty | Level | Why It's New |

[L178] |---|-----|-----|-----|

[L179] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L180] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L181] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L182] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L183] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L184]

[L185] ### 13.3 Next Steps

[L186]

[L187] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L188] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L189] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L190] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L191] 5. **Build Layer 4** (Relation management with ontology validation)

[L192] 6. **Build Layer 5** (SHIA Core with algorithms)

[L193] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L194] 8. **Build Layer 7** (Generation with verification)

[L195] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L196] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L197] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L198] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L199]

[L200] ### 13.4 Future Work

[L201] metadata:
[L202] query_result_index: 3
[L203] file_id: file_00000000f620821186e54e582d59731e
[L204] version_id: 1
[L205] name: SHIA_RAG_Final_Report(3).md
[L206] mime_type: text/markdown
[L207] surface: library
[L208] score: 0.02146158925819943
[L209] document_chunk_id: file_00000000f620821186e54e582d59731e--1--40
[L210] snippet:
[L211] temporal versioning |
[L212] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |
[L213] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |
[L214] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |
[L215]
[L216] ---
[L217]
[L218] ## 13. Conclusion & Future Work
[L219]
[L220] ### 13.1 Summary
[L221]
[L222] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:
[L223]
[L224] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem
[L225] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
[L226] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction
[L227] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation
[L228] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality
[L229]
[L230] The complete system specification covers:
[L231] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
[L232] - **12 defined functions** with complete implementations
[L233] - **12 relation types** across 4 hierarchical and 8 semantic categories
[L234] - **8 schemas** covering all data objects from document blocks to query logs
[L235] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline
[L236] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing
[L237]
[L238] ### 13.2 Key Novelties
[L239]
[L240] | # | Novelty | Level | Why It's New |
[L241] |---|-----|-----|-----|
[L242] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |
[L243] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |
[L244] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

- [L245] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |
- [L246] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |
- [L247]
- [L248] ### 13.3 Next Steps
- [L249]
- [L250] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)
- [L251] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)
- [L252] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)
- [L253] 4. **Build Layer 3** (Knowledge extraction with LLM)
- [L254] 5. **Build Layer 4** (Relation management with ontology validation)
- [L255] 6. **Build Layer 5** (SHIA Core with algorithms)
- [L256] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
- [L257] 8. **Build Layer 7** (Generation with verification)
- [L258] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
- [L259] 10. **Run first evaluation** on OS textbook dataset using SHEF
- [L260] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
- [L261] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
- [L262]
- [L263] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file4

- [L264] metadata:
- [L265] query_result_index: 4
- [L266] file_id: file_00000000dcc082119f935683a5e13eed
- [L267] version_id: 1
- [L268] name: SHIA_RAG_Final_Report(4)(1).md
- [L269] mime_type: text/markdown
- [L270] surface: conversation
- [L271] score: 0.02104247104247104
- [L272] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--54
- [L273] snippet:
- [L274] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem
- [L275] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
- [L276] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence
- [L277] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG
- [L278] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality
- [L279]
- [L280] The complete system specification covers:
- [L281] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
- [L282] - **12 defined functions** with complete implementations
- [L283] - **12 relation types** across 4 hierarchical and 8 semantic categories
- [L284] - **8 schemas** covering all data objects from document blocks to query logs

[L285] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L286] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L287]

[L288] ### 13.2 Key Novelties

[L289]

[L290] | # | Novelty | Level | Why It's New |

[L291] | ---|-----|-----|-----|

[L292] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L293] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L294] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L295] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L296] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L297]

[L298] ### 13.3 Next Steps

[L299]

[L300] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L301] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L302] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L303] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L304] 5. **Build Layer 4** (Relation management with ontology validation)

[L305] 6. **Build Layer 5** (SHIA Core with algorithms)

[L306] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L307] 8. **Build Layer 7** (Generation with verification)

[L308] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L309] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L310] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L311] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L312]

[L313] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite[turn19file5]`

[L314] metadata:

[L315] query_result_index: 5

[L316] file_id: file_000000003c8081fab658b1752549d64

[L317] version_id: 1

[L318] name: SHIA_RAG_Complete_Report(5).md

[L319] mime_type: text/markdown

[L320] surface: library

[L321] score: 0.02078544061302682

[L322] document_chunk_id: file_000000003c8081fab658b1752549d64--1--45

[L323] snippet:

[L324] @dataclass

[L325] class RetrievalResult:

[L326] """Output of Layer 6 — input to Layer 7."""

```

[L327] selected_nodes: list # Nodes selected by Tree Knapsack
[L328] context_text: str # Assembled context string
[L329] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L330] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L331] query_intent: str # DEFINITION, COMPARISON, etc.
[L332] seed_node_ids: list # Initial seed nodes from vector search
[L333] token_count: int # Estimated tokens in context
[L334]
[L335] class RetrievalEngine:
[L336] def __init__(self, forest, vector_index, cache_manager, config):
[L337] self.forest = forest
[L338] self.vector_index = vector_index
[L339] self.cache = cache_manager # Redis cache integration
[L340] self.config = config
[L341] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L342]
[L343] def retrieve(self, query) -> RetrievalResult:
[L344] # Step 0: Cache check
[L345] cache_key = hash_query(query)
[L346] cached = self.cache.get_query_result(cache_key)
[L347] if cached:
[L348] return cached
[L349]
[L350] # Step 1: Analyze query
[L351] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L352] complexity = classify_complexity(intent) # Rule-based mapping
[L353]
[L354] # Step 2: Embed and find seed nodes
[L355] query_emb = embed_query(query) # Asymmetric embedding model
[L356] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L357]
[L358] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L359] depth_config = {
[L360] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L361] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L362] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L363] }
[L364] traversal = depth_config[complexity]
[L365]
[L366] # Step 4: Traverse forest from all seeds
[L367] candidate_nodes = set()
[L368] for seed in seed_nodes:
[L369] candidate_nodes |= self.traverse(seed, traversal)
[L370]
[L371] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L372] selected = self.multi_tree_knapsack(
[L373] candidate_nodes,
[L374] budget=self.config.token_budget
[L375] )
[L376]

```

```

[L377] # Step 6: Assemble context in semantic order
[L378] context_text = self.assemble_context(selected, intent)
[L379]
[L380] # Build result with traversal path for Layer 8 feedback
[L381] result = RetrievalResult(
[L382]     selected_nodes=selected,
[L383]     context_text=context_text,
[L384]     traversal_path_edges=self._get_path_edges(selected),
[L385]     query_complexity=complexity,
[L386]     query_intent=intent,
[L387]     seed_node_ids=[s.id for s in seed_nodes],
[L388]     token_count=estimate_tokens(context_text)
[L389] )
[L390]
[L391] # Cache result
[L392] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L393]
[L394] return result
[L395]
[L396] def multi_tree_knapsack(self, candidate_nodes, budget):
[L397]     """Apply Tree Knapsack per distinct tree, then merge results.
[L398]
[L399]     For multi-seed queries, candidates may span multiple trees.
[L400]     We optimize each tree independently, then merge within budget.

```

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file6

```

[L401] metadata:
[L402] query_result_index: 6
[L403] file_id: file_000000001dd481fa9855f2731dbc53d5
[L404] version_id: 1
[L405] name: SHIA_RAG_Complete_Report(6).md
[L406] mime_type: text/markdown
[L407] surface: library
[L408] score: 0.02078544061302682
[L409] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--45
[L410] snippet:
[L411] @dataclass
[L412] class RetrievalResult:
[L413]     """Output of Layer 6 — input to Layer 7."""
[L414]     selected_nodes: list # Nodes selected by Tree Knapsack
[L415]     context_text: str # Assembled context string
[L416]     traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L417]     query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L418]     query_intent: str # DEFINITION, COMPARISON, etc.
[L419]     seed_node_ids: list # Initial seed nodes from vector search
[L420]     token_count: int # Estimated tokens in context
[L421]
[L422] class RetrievalEngine:

```

```

[L423] def __init__(self, forest, vector_index, cache_manager, config):
[L424] self.forest = forest
[L425] self.vector_index = vector_index
[L426] self.cache = cache_manager # Redis cache integration
[L427] self.config = config
[L428] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L429]
[L430] def retrieve(self, query) -> RetrievalResult:
[L431] # Step 0: Cache check
[L432] cache_key = hash_query(query)
[L433] cached = self.cache.get_query_result(cache_key)
[L434] if cached:
[L435] return cached
[L436]
[L437] # Step 1: Analyze query
[L438] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L439] complexity = classify_complexity(intent) # Rule-based mapping
[L440]
[L441] # Step 2: Embed and find seed nodes
[L442] query_emb = embed_query(query) # Asymmetric embedding model
[L443] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L444]
[L445] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L446] depth_config = {
[L447] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L448] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L449] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L450] }
[L451] traversal = depth_config[complexity]
[L452]
[L453] # Step 4: Traverse forest from all seeds
[L454] candidate_nodes = set()
[L455] for seed in seed_nodes:
[L456] candidate_nodes |= self.traverse(seed, traversal)
[L457]
[L458] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L459] selected = self.multi_tree_knapsack(
[L460] candidate_nodes,
[L461] budget=self.config.token_budget
[L462] )
[L463]
[L464] # Step 6: Assemble context in semantic order
[L465] context_text = self.assemble_context(selected, intent)
[L466]
[L467] # Build result with traversal path for Layer 8 feedback
[L468] result = RetrievalResult(
[L469] selected_nodes=selected,
[L470] context_text=context_text,
[L471] traversal_path_edges=self._get_path_edges(selected),
[L472] query_complexity=complexity,

```



```

[L473] query_intent=intent,
[L474] seed_node_ids=[s.id for s in seed_nodes],
[L475] token_count=estimate_tokens(context_text)
[L476] )
[L477]
[L478] # Cache result
[L479] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L480]
[L481] return result
[L482]
[L483] def multi_tree_knapsack(self, candidate_nodes, budget):
[L484] """Apply Tree Knapsack per distinct tree, then merge results.
[L485]
[L486] For multi-seed queries, candidates may span multiple trees.
[L487] We optimize each tree independently, then merge within budget.

```

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file7

```

[L488] metadata:
[L489] query_result_index: 7
[L490] file_id: file_0000000033088207b45d2ae58cbd406c
[L491] version_id: 1
[L492] name: SHIA_RAG_Complete_Report(8).md
[L493] mime_type: text/markdown
[L494] surface: library
[L495] score: 0.020326340326340328
[L496] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--45
[L497] snippet:
[L498] @dataclass
[L499] class RetrievalResult:
[L500] """Output of Layer 6 — input to Layer 7."""
[L501] selected_nodes: list # Nodes selected by Tree Knapsack
[L502] context_text: str # Assembled context string
[L503] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L504] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L505] query_intent: str # DEFINITION, COMPARISON, etc.
[L506] seed_node_ids: list # Initial seed nodes from vector search
[L507] token_count: int # Estimated tokens in context
[L508]
[L509] class RetrievalEngine:
[L510] def __init__(self, forest, vector_index, cache_manager, config):
[L511] self.forest = forest
[L512] self.vector_index = vector_index
[L513] self.cache = cache_manager # Redis cache integration
[L514] self.config = config
[L515] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L516]
[L517] def retrieve(self, query) -> RetrievalResult:
[L518] # Step 0: Cache check

```

```
[L519] cache_key = hash_query(query)
[L520] cached = self.cache.get_query_result(cache_key)
[L521] if cached:
[L522] return cached
[L523]
[L524] # Step 1: Analyze query
[L525] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L526] complexity = classify_complexity(intent) # Rule-based mapping
[L527]
[L528] # Step 2: Embed and find seed nodes
[L529] query_emb = embed_query(query) # Asymmetric embedding model
[L530] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L531]
[L532] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L533] depth_config = {
[L534] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L535] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L536] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L537] }
[L538] traversal = depth_config[complexity]
[L539]
[L540] # Step 4: Traverse forest from all seeds
[L541] candidate_nodes = set()
[L542] for seed in seed_nodes:
[L543] candidate_nodes |= self.traverse(seed, traversal)
[L544]
[L545] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L546] selected = self.multi_tree_knapsack(
[L547] candidate_nodes,
[L548] budget=self.config.token_budget
[L549] )
[L550]
[L551] # Step 6: Assemble context in semantic order
[L552] context_text = self.assemble_context(selected, intent)
[L553]
[L554] # Build result with traversal path for Layer 8 feedback
[L555] result = RetrievalResult(
[L556] selected_nodes=selected,
[L557] context_text=context_text,
[L558] traversal_path_edges=self._get_path_edges(selected),
[L559] query_complexity=complexity,
[L560] query_intent=intent,
[L561] seed_node_ids=[s.id for s in seed_nodes],
[L562] token_count=estimate_tokens(context_text)
[L563] )
[L564]
[L565] # Cache result
[L566] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L567]
[L568] return result
```

[L569]
[L570] def multi_tree_knapsack(self, candidate_nodes, budget):
[L571] """Apply Tree Knapsack per distinct tree, then merge results.
[L572]
[L573] For multi-seed queries, candidates may span multiple trees.
[L574] We optimize each tree independently, then merge within budget.

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file8

[L575] metadata:
[L576] query_result_index: 8
[L577] file_id: file_000000003218820b918ee3d86ec345ae
[L578] version_id: 1
[L579] name: SHIA_RAG_Final_Report(1).md
[L580] mime_type: text/markdown
[L581] surface: library
[L582] score: 0.01948051948051948
[L583] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--40
[L584] snippet:
[L585] temporal versioning |
[L586] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |
[L587] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |
[L588] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |
[L589]
[L590] ---
[L591]
[L592] ## 13. Conclusion & Future Work
[L593]
[L594] ### 13.1 Summary
[L595]
[L596] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:
[L597]
[L598] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem
[L599] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
[L600] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction
[L601] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation
[L602] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality
[L603]
[L604] The complete system specification covers:
[L605] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
[L606] - **12 defined functions** with complete implementations
[L607] - **12 relation types** across 4 hierarchical and 8 semantic categories
[L608] - **8 schemas** covering all data objects from document blocks to query logs
[L609] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline
[L610] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing
[L611]

[L612] ### 13.2 Key Novelties

[L613]

[L614] | # | Novelty | Level | Why It's New |

[L615] |---|-----|-----|-----|

[L616] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L617] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L618] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L619] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L620] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L621]

[L622] ### 13.3 Next Steps

[L623]

[L624] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L625] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L626] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L627] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L628] 5. **Build Layer 4** (Relation management with ontology validation)

[L629] 6. **Build Layer 5** (SHIA Core with algorithms)

[L630] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L631] 8. **Build Layer 7** (Generation with verification)

[L632] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L633] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L634] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L635] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L636]

[L637] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 01:19:51

Citation Marker: `filecite`turn19file9

[L638] metadata:

[L639] query_result_index: 9

[L640] file_id: file_00000000dcc082119f935683a5e13eed

[L641] version_id: 1

[L642] name: SHIA_RAG_Final_Report(4)(1).md

[L643] mime_type: text/markdown

[L644] surface: conversation

[L645] score: 0.01941886199757264

[L646] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--3

[L647] snippet:

[L648] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L649]

[L650] ### 1.3 Objectives

[L651]

[L652] 1. Design a complete 8-layer architecture for hierarchical RAG

[L653] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L654] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L655] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L656] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L657] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L658]

[L659] ### 1.4 Contributions

[L660]

[L661] This work makes five novel contributions to the RAG literature:

[L662]

[L663] | # | Contribution | Novelty Level |

[L664] |---|-----|-----|

[L665] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L666] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L667] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L668] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L669] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L670]

[L671] ---

[L672]

[L673] ## 2. Literature Review

[L674]

[L675] ### 2.1 Related Work

[L676]

[L677] #### 2.1.1 Flat-Chunk RAG Systems

[L678]

[L679] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Gua et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L680]

[L681] #### 2.1.2 Structured RAG Systems

[L682]

[L683] Recent work has begun exploring structured retrieval:

[L684]

[L685] | System | Approach | Key Limitation |

[L686] |-----|-----|-----|

[L687] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L688] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L689] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L690] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L691]

[L692] #### 2.1.3 Knowledge Graph Construction

[L693]

[L694] Ontology learning and taxonomy induction have a rich history in NLP:

[L695] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L696] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L697] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L698]

[L699] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L700]

[L701] ### 2.2 Research Gap

DEEP RESEARCH 24/08/2026, 01:19:55

Citation Marker: `filecite`turn20file0

[L1] Find: find=0 ref=turn17file3 name=SHIA_RAG_Final_Report(4)(1).md pattern=## 9. Layer 6

[L2] Scope: full file surface=conversation

[L3] Matches: shown=0 total=0

[L4] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 24/08/2026, 01:19:55

Citation Marker: `filecite`turn20file1

[L5] Find: find=1 ref=turn17file3 name=SHIA_RAG_Final_Report(4)(1).md pattern=### 9.

[L6] Scope: full file surface=conversation

[L7] Matches: shown=5 total=5

[L8] [match 0 | match_lines 1668 | shown_lines 1663-1673]

[L9]

[L10] ---

[L11]

[L12] ## 9. Mathematical Framework

[L13]

[L14] ### 9.1 Hierarchy Energy (Global Objective)

[L15]

[L16]
$$J(\mathcal{F}) = \sum_{(P_i, N_i) \in \mathcal{F}} E(P_i, N_i) + \alpha \cdot R(\mathcal{F})$$

[L17]

[L18] **Constraints:**

[L19] - $\lambda_1 + \lambda_2 + \lambda_3 + \lambda_4 = 1$

[L20] [matches 1-4 | match_lines 1686,1695,1705,1713 | shown_lines 1681-1718]

[L21] |-----|-----|-----|

[L22] | CS Education | 0.10 | Moderate depth variance acceptable |

[L23] | Medical/Clinical | 0.20 | Deeper, more balanced hierarchies preferred |

[L24] | Legal | 0.05 | Flat, wide hierarchies are natural |

[L25]

[L26] ### 9.2 Parent Ranking Score (PRE)

[L27]

[L28]
$$\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}(P)}{D_{\max}} + w_4 \cdot S_{\text{evi}}(P, N) + w_5 \cdot \text{Conf}(P)$$

[L29]

[L30] Where:

[L31] - $H(P, N)$ = Hypernym score (Hearst patterns + LLM fallback, returns 0.0 or 1.0)

[L32] - $S_{\text{evi}}(P, N)$ = Evidence support (co-occurrence + citation overlap)

[L33] - $w_1 = 0.30, w_2 = 0.25, w_3 = 0.15, w_4 = 0.20, w_5 = 0.10$

[L34]

[L35] ### 9.3 Tree Knapsack Formulation

[L36]

[L37]
$$\max \sum_{i=1}^n U_i \cdot x_i \quad \text{s.t.} \quad \sum_{i=1}^n C_i \cdot x_i \leq B, \quad x_i \leq 1 \quad \forall i$$

[L38]

[L39] Where:

[L40] - $U_i = \text{utility}(\text{node}_i)$ — quality score incorporating edge weights from self-evolution

[L41] - $C_i = \text{token_cost}(\text{node}_i)$ — estimated tokens for context representation

[L42] - B = token budget (default: 4096)

[L43] - $x_i \in \{0, 1\}$ — selection variable

[L44]

[L45] ### 9.4 Self-Evolution Update

[L46]

[L47] After each query with feedback signal $f \in \{+1, -1\}$:

[L48]

[L49]
$$w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f) \quad \forall e \in \text{traversal_path}$$

[L50]

[L51] Where $\eta = 0.02$. Edge weights are clipped to $[0.1, 10.0]$.

[L52]

[L53] ### 9.5 Answer Confidence

[L54]

[L55]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported_claims}}{\text{total_claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{node_conf}}{\text{cited_nodes}} + 0.1 \cdot \frac{\text{cited_nodes}}{\text{selected_nodes}}$$

[L56]

[L57] ---

DEEP RESEARCH 24/08/2026, 01:19:55

Citation Marker: `[[filecite]]turn20file2`

[L58] Find: find=2 ref=turn17file3 name=SHIA_RAG_Final_Report(4)(1).md pattern=Tree Knapsack

[L59] Scope: full file surface=conversation

[L60] Matches: shown=20 total=28 has_more=true next_match_offset=20

[L61] [matches 0-1 | match_lines 15,19 | shown_lines 10-24]

[L62]

[L63] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L64]

[L65] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L66]

[L67] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated

hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L68]

[L69] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L70]

[L71] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L72]

[L73] ---

[L74]

[L75] ## Table of Contents

[L76] [matches 2-4 | match_lines 75,86,96 | shown_lines 70-101]

[L77]

[L78] The system:

[L79] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L80] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L81] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L82] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L83] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L84] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L85]

[L86] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L87] ### 1.3 Objectives

[L88]

[L89] 1. Design a complete 8-layer architecture for hierarchical RAG

[L90] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L91] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L92] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L93] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L94] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L95]

[L96] ### 1.4 Contributions

[L97]

[L98] This work makes five novel contributions to the RAG literature:

[L99]

[L100] | # | Contribution | Novelty Level |

[L101] |---|-----|-----|

[L102] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L103] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L104] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L105] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L106] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L107] [match 5 | match_lines 155 | shown_lines 150-160]

[L108]

[L109] | Parameter | Details |

[L110] |-----|-----|

[L111] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L112] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L113] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L114] | **Time Range** | 2020–2026 |

[L115] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L116]

[L117] ---

[L118] [match 6 | match_lines 215 | shown_lines 210-220]

[L119] KnowledgeNode objects (canonical, deduplicated)

[L120] ↓ Layer 4: Validate relations against ontology, score confidence

[L121] KnowledgeEdge objects (typed, scored)

[L122] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)

[L123] Knowledge Forest (rooted trees + cross-links)

[L124] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection

[L125] Selected context nodes (within token budget)

[L126] ↓ Layer 7: Build prompt, invoke LLM

[L127] ↓ Layer 7: Verify claims against evidence, bind citations

[L128] Answer with citations + confidence score

[L129] ↓ Layer 8: Log, collect feedback, update forest weights

[L130] [match 7 | match_lines 233 | shown_lines 228-238]

[L131] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L132] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L133] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L134] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L135] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L136] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L137] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L138] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L139]

[L140] ---

[L141] [match 8 | match_lines 1211 | shown_lines 1206-1216]

[L142]

[L143]

```

[L144] @dataclass
[L145] class RetrievalResult:
[L146]     """Output of Layer 6 – input to Layer 7."""
[L147]     selected_nodes: list          # Nodes selected by Tree Knapsack
[L148]     context_text: str             # Assembled context string
[L149]     traversal_path_edges: list    # Edges traversed (for Layer 8 feedback)
[L150]     query_complexity: str         # SIMPLE, MODERATE, COMPLEX
[L151]     query_intent: str             # DEFINITION, COMPARISON, etc.
[L152]     seed_node_ids: list          # Initial seed nodes from vector search
[L153] [match 9 | match_lines 1253 | shown_lines 1248-1258]
[L154]     # Step 4: BFS traversal from all seeds
[L155]     candidate_nodes = set()
[L156]     for seed in seed_nodes:
[L157]         candidate_nodes |= self.traverse(seed, traversal)
[L158]
[L159]     # Step 5: Tree Knapsack selection (per-tree, then merge)
[L160]     selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L161]
[L162]     # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L163]     context_text = self.assemble_context(selected, intent)
[L164]
[L165] [matches 10-11 | match_lines 1268,1274 | shown_lines 1263-1279]
[L166]     seed_node_ids=[s.id for s in seed_nodes],
[L167]     token_count=estimate_tokens(context_text)
[L168] )
[L169]

```

[L170]

[L171] ##### Multi-Tree Knapsack Selection

[L172]

[L173] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L174]

[L175] 1. **Group** candidates by tree root

[L176] 2. **Allocate** token budget proportionally to each tree

[L177] 3. **Run** Tree Knapsack on each tree independently

[L178] 4. **Merge** results and trim to total budget

[L179]

[L180]

```

[L181] def multi_tree_knapsack(self, candidate_nodes, budget):
[L182]     tree_groups = {}
[L183] [match 12 | match_lines 1305 | shown_lines 1300-1310]
[L184]
[L185] ##### Utility Function (Self-Evolution Integrated)
[L186]
[L187]

```

python

[L188] def utility(node, forest):

[L189] """Utility for Tree Knapsack — uses edge weights from self-evolution feedback."""

[L190] if not node.parent:

[L191] return node.confidence * len(node.evidence)

[L192]

[L193] parent_edge = forest.get_edge(node.parent, node.id)

[L194] edge_weight = parent_edge.weight if parent_edge else 1.0

[L195] [matches 13-15 | match_lines 1514,1520,1524 | shown_lines 1509-1529]

[L196] accidentally received positive feedback."""

[L197] if random.random() < EPSILON:

[L198] # EXPLORE: randomly traverse edges, ignoring weights

[L199] return random_tree_traversal(seed_nodes, forest, budget)

[L200] else:

[L201] # EXPLOIT: use Tree Knapsack with utility (weight-based)

[L202] return tree_knapsack_traversal(seed_nodes, forest, budget)

[L203]

[L204]

[L205] ### 7.4 Closing the Self-Evolution Loop

[L206]

[L207] The self-evolution loop is closed through the `utility()` function in Tree Knapsack:

[L208]

[L209]

[L210] User Feedback → edge.weight update → utility() uses edge.weight →

[L211] Tree Knapsack selects differently → future retrieval improves

[L212] ```

[L213]

[L214] This means edges along paths that consistently lead to helpful answers get higher weights, making those paths more likely to be selected in future queries.

[L215]

[L216] ### 7.5 Forest Restructuring

[L217] [matches 16-18 | match_lines 1562,1566,1572 | shown_lines 1557-1577]

[L218]

[L219] ## 8. Novel Contributions

[L220]

[L221] SHIA-RAG introduces five novel contributions to the RAG literature. Each is supported by a systematic literature review (see Section 2.3).

[L222]

[L223] ### 8.1 Novelty 1: Precedence-Constrained Tree Knapsack for Context Selection

[L224]

[L225] **Existing approach:** All current RAG systems use flat top-k selection or simple greedy knapsack for context packing.

[L226]

[L227] **SHIA-RAG approach:** A greedy approximation for the **Tree Knapsack** formulation where selecting a child node forces inclusion of its parent (precedence constraint):

[L228]

[L229]
$$\max \sum_{i=1}^n U_i \cdot x_i \quad \text{s.t.} \quad \sum_{i=1}^n C_i \cdot x_i \leq B, \quad x_i \leq x_{\text{parent}(i)} \quad \forall i$$

[L230]

[L231] The implementation uses a greedy utility-density heuristic rather than exact DP, as exact solutions are computationally prohibitive for large token budgets. The precedence-constrained formulation itself is the novelty.

[L232]

[L233] **Novelty level:** Incremental (Novel Application) — adaptation of the Tree Knapsack formulation from operations research to RAG context selection.

[L234]

[L235] ---

[L236]

[L237] ### 8.2 Novelty 2: Self-Evolving Knowledge Forest via Retrieval Feedback

[L238] [match 19 | match_lines 1658 | shown_lines 1653-1663]

[L239]

[L240] ### 8.6 Novelty Level Assessment Summary

[L241]

[L242] | # | Novelty | Level | Justification |

[L243] |---|-----|-----|-----|
[L244] | 1 | Tree Knapsack for Context Selection | **Incremental (Novel Application)** | Adaptation of existing OR technique to a new domain (RAG context selection) |
[L245] | 2 | Self-Evolving Knowledge Forest | **Moderately Novel** | Novel combination of retrieval feedback + hierarchy restructuring |
[L246] | 3 | Multi-Objective Energy Minimization | **Moderately Novel** | Combined formulation for RAG hierarchy induction is new |
[L247] | 4 | Adaptive Depth Traversal | **Incremental** | Novel application to hierarchical RAG traversal depth |
[L248] | 5 | SHEF Evaluation Framework | **Incremental (Novel Combination)** | Standard metrics adapted as a unified framework for hierarchical RAG |

DEEP RESEARCH 24/08/2026, 01:19:58

Citation Marker: `filecite`turn21file0

[L1] metadata:
[L2] ref_id: turn17file3
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] lines_shown: 1180-1359
[L7] total_file_lines: 2066
[L8] has_more: True
[L9] next_start_line: 1360
[L10] next_read: {"ref_id":"turn17file3","mode":"full","start_line":1360,"max_lines":180,"include_images":false}
[L11] content:
[L12] cross_links.append((neighbor.id, relation_type, sim))
[L13]
[L14] return cross_links
[L15]

```
[L16]  
[L17] ---  
[L18]  
[L19] ## 6. Online Pipeline – Layers 6-7  
[L20]  
[L21] ### 6.1 Layer 6: Retrieval Engine  
[L22]  
[L23] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes with  
[L24]  
[L25] **Input:** User query string → **Output:** `RetrievalResult`  
[L26]  
[L27] #### Query Complexity Classification  
[L28]  
[L29] | Intent | Example | Complexity | Traversal Depth |  
[L30] |-----|-----|-----|-----|  
[L31] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |  
[L32] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |  
[L33] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |  
[L34] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |  
[L35] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |  
[L36]  
[L37] #### Retrieval Pipeline  
[L38]  
[L39]
```

python
[L40] @dataclass

```

[L41] class RetrievalResult:
[L42] """Output of Layer 6 — input to Layer 7."""
[L43] selected_nodes: list # Nodes selected by Tree Knapsack
[L44] context_text: str # Assembled context string
[L45] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L46] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L47] query_intent: str # DEFINITION, COMPARISON, etc.
[L48] seed_node_ids: list # Initial seed nodes from vector search
[L49] token_count: int # Estimated tokens in context
[L50]
[L51] class RetrievalEngine:
[L52] def __init__(self, forest, vector_index, cache_manager, config):
[L53] self.forest = forest
[L54] self.vector_index = vector_index
[L55] self.cache = cache_manager
[L56] self.config = config
[L57]
[L58] def retrieve(self, query) -> RetrievalResult:
[L59] # Step 0: Cache check
[L60] cached = self.cache.get_query_result(hash_query(query))
[L61] if cached:
[L62] return cached
[L63]
[L64] # Step 1: Classify query intent and complexity
[L65] intent = detect_intent(query)
[L66] complexity = classify_complexity(intent)
[L67]
[L68] # Step 2: Embed query and find seed nodes
[L69] query_emb = embed_query(query) # Asymmetric embedding model
[L70] seed_nodes = self.vector_index.search(query_emb, k=5)
[L71]
[L72] # Step 3: Adaptive depth traversal
[L73] depth_config = {
[L74] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L75] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L76] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L77] }
[L78] traversal = depth_config[complexity]
[L79]
[L80] # Step 4: BFS traversal from all seeds
[L81] candidate_nodes = set()
[L82] for seed in seed_nodes:
[L83] candidate_nodes |= self.traverse(seed, traversal)
[L84]
[L85] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L86] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L87]
[L88] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L89] context_text = self.assemble_context(selected, intent)
[L90]

```

```
[L91] return RetrievalResult(
[L92] selected_nodes=selected, context_text=context_text,
[L93] traversal_path_edges=self._get_path_edges(selected),
[L94] query_complexity=complexity, query_intent=intent,
[L95] seed_node_ids=[s.id for s in seed_nodes],
[L96] token_count=estimate_tokens(context_text)
[L97] )
[L98]
```

```
[L99]
[L100] ##### Multi-Tree Knapsack Selection
[L101]
[L102] For multi-seed queries, candidates may span multiple trees. The algorithm:
[L103]
[L104] 1. Group candidates by tree root
[L105] 2. Allocate token budget proportionally to each tree
[L106] 3. Run Tree Knapsack on each tree independently
[L107] 4. Merge results and trim to total budget
[L108]
[L109]
```

python

```
[L110] def multi_tree_knapsack(self, candidate_nodes, budget):
[L111] tree_groups = {}
[L112] for node in candidate_nodes:
[L113] root = self.forest.find_root(node)
[L114] tree_groups.setdefault(root.id, []).append(node)
[L115]
[L116] all_selected = []
[L117] for root_id, nodes in tree_groups.items():
[L118] tree_budget = int(budget len(nodes) / max(len(candidate_nodes), 1))
[L119] _, selected = tree_knapsack(self.forest.get_node(root_id), tree_budget, self.forest)
[L120] all_selected.extend([n for n in selected if n in nodes])
[L121]
[L122] # Final trim to total budget
[L123] all_selected.sort(key=lambda n: utility(n, self.forest), reverse=True)
[L124] final, total_tokens = [], 0
[L125] for node in all_selected:
[L126] cost = token_cost(node)
[L127] if total_tokens + cost <= budget:
[L128] final.append(node)
[L129] total_tokens += cost
[L130] return final
[L131]
```

```
[L132]
[L133] ##### Utility Function (Self-Evolution Integrated)
[L134]
[L135]
```

python

```
[L136] def utility(node, forest):
[L137] """Utility for Tree Knapsack — uses edge weights from self-evolution feedback."""
[L138] if not node.parent:
[L139] return node.confidence len(node.evidence)
```

```
[L140]
[L141] parent_edge = forest.get_edge(node.parent, node.id)
[L142] edge_weight = parent_edge.weight if parent_edge else 1.0
[L143]
[L144] return (
[L145] node.confidence # Node quality
[L146] edge_weight # Self-evolution signal
[L147] len(node.evidence) * # Evidence richness
[L148] (1.0 / max(token_cost(node), 1)) # Cost efficiency
[L149] )
[L150]
```

```
[L151]
[L152] ---
[L153]
[L154] ### 6.2 Layer 7: Generation Engine
[L155]
[L156] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims, verify each claim
[L157]
[L158] **Input:** `RetrievalResult` → **Output:** `GenerationResult`
[L159]
[L160] #### LLM Configuration
[L161]
[L162] | Parameter | Value | Rationale |
[L163] |-----|-----|-----|
[L164] | Model | GPT-4o-mini (primary), Llama 3.1 8B (fallback) | Cost-effective with good quality |
[L165] | Temperature | 0.3 | Low for factual answers |
[L166] | Max output tokens | 2048 | Sufficient for detailed answers |
[L167] | Response format | JSON | Force structured output for claim extraction |
[L168] | Timeout | 30 seconds | With retry on timeout |
[L169]
[L170] #### Prompt Template
[L171]
[L172]
```

python

```
[L173] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L174] RULES:
[L175] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L176] 2. For each claim, cite the source using [NODE_ID] format.
[L177] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L178] 4. If evidence conflicts, present both views with citations.
[L179]
[L180] REQUIRED OUTPUT FORMAT:
[L181] {
[L182] "answer": "Your answer with [NODE_ID] citations inline.",
[L183] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L184] "overall_confidence": 0.0-1.0,
[L185] "insufficient_evidence": false
[L186] }"""
[L187] ```
[L188]
[L189] #### Claim Verification
[L190]
[L191] After the LLM generates an answer, each claim is verified against its cited evidence:
```

DEEP RESEARCH 24/08/2026, 01:20:02

Citation Marker: `[]filecite[]turn22file0[]`

```
[L1] Find: find=0 ref=turn17file3 name=SHIA_RAG_Final_Report(4)(1).md pattern=RBO
[L2] Scope: full file surface=conversation
[L3] Matches: shown=1 total=1
[L4] [match 0 | match_lines 186 | shown_lines 181-191]
[L5] L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L6] L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L7] end
[L8]
[L9] subgraph "Phase 2: Online — Retrieval & Generation"
[L10] Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RBO"]
[L11] KF --> L6
[L12] L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L13] L7 --> A["Answer + Citations"]
[L14] end
[L15]
```

DEEP RESEARCH 24/08/2026, 01:20:02

Citation Marker: `[]filecite[]turn22file1[]`

```
[L16] Find: find=1 ref=turn17file3 name=SHIA_RAG_Final_Report(4)(1).md pattern=Retrieval Budget Optimization
[L17] Scope: full file surface=conversation
[L18] Matches: shown=0 total=0
[L19] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the
wording or location is uncertain, call files.search for ranked semantic retrieval; otherwise retry files.find with a broader
or alternate exact pattern.
```

DEEP RESEARCH 24/08/2026, 01:20:02

Citation Marker: `[]filecite[]turn22file2[]`

```
[L20] Find: find=2 ref=turn17file3 name=SHIA_RAG_Final_Report(4)(1).md pattern=BFS traversal
[L21] Scope: full file surface=conversation
[L22] Matches: shown=1 total=1
[L23] [match 0 | match_lines 1248 | shown_lines 1243-1253]
[L24] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L25] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L26] }
[L27] traversal = depth_config[complexity]
[L28]
[L29] # Step 4: BFS traversal from all seeds
[L30] candidate_nodes = set()
[L31] for seed in seed_nodes:
[L32] candidate_nodes |= self.traverse(seed, traversal)
[L33]
[L34] # Step 5: Tree Knapsack selection (per-tree, then merge)
```

CHATGPT 24/08/2026, 01:20:01

Bilkul. Ab hum **Layer 6 – Retrieval Engine** ko **same detailed style** mein samjhenge.

Is layer ko samajhna bahut important hai, kyunki **Layer 5** tak system ne knowledge ko organize karke Knowledge Forest bana diya tha. Ab Layer 6 ka kaam hai:

User ke question ke according Knowledge Forest ke andar se exactly relevant knowledge ko intelligently retrieve karna.

Layer 6 is the **online retrieval stage**: it analyzes the query, finds seed nodes, traverses the Knowledge Forest with adaptive depth, selects context under a token budget, and sends a `RetrievalResult` to Layer 7. [filecite]turn21file0[L19-L25]

Layer 6 — Retrieval Engine

1. Sabse pehle ek line mein samjho

Layer 5:
Build the Knowledge Forest

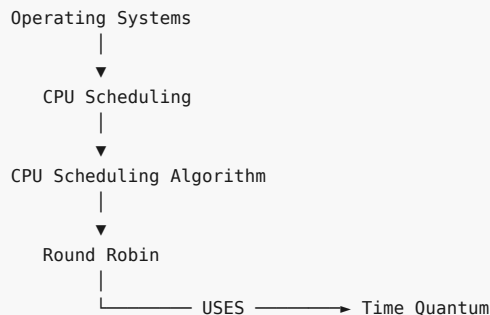
Layer 6:
Search and intelligently navigate the Knowledge Forest according to the user's query

Simple analogy:

Layer 5 builds the library and arranges all books properly. Layer 6 acts like an intelligent librarian who understands your question, finds the right section, explores only the necessary connected knowledge, and gives the most useful information within a limited space.

2. Hum same running example continue karenge

Layer 5 ke baad hamare paas ye Knowledge Forest hai:



Maan lo user query karta hai:

"How does Round Robin work and why is Time Quantum important?"

Ab Layer 6 ko answer generate nahi karna hai.

⚠ **Answer generation Layer 7 karega.**

Layer 6 ka kaam sirf ye hai:

"Is question ka answer dene ke liye Knowledge Forest se kaunsa knowledge retrieve karna chahiye?"

3. Layer 6 ka exact input kya hai?

Primary Input

User Query

Example:

"How does Round Robin work and why is Time Quantum important?"

Lekin Layer 6 indirectly kya access karta hai?

Layer 5 ka output:

Knowledge Forest

Aur retrieval ke liye vector index bhi available hai:

```
Knowledge Forest
  +
Vector Index
  +
Node Embeddings
  +
Evidence
  ↓
Layer 6
```

The source describes Layer 6 as receiving the user query while accessing the Knowledge Forest and vector search infrastructure to retrieve relevant nodes. [filecite\turn21file0\L21-L25](#) [filecite\turn18file6\L425-L455](#)

4. Layer 6 ka complete output kya hota hai?

Output:

RetrievalResult

Ismein important information hoti hai:

```
RetrievalResult
|
├─ Selected Nodes
├─ Context Text
├─ Traversal Path
├─ Query Complexity
├─ Query Intent
├─ Seed Node IDs
└─ Token Count
```

Example:

```
Selected Nodes:
- CPU Scheduling Algorithm
- Round Robin
- Time Quantum

Query Intent:
EXPLANATION / WHY

Query Complexity:
MODERATE or COMPLEX

Context:
Relevant definitions + evidence

Traversal Path:
CPU Scheduling Algorithm
  ↓
Round Robin
  —USES—> Time Quantum
```

Ye complete result **Layer 7 ko diya jata hai**, jahan actual answer generate hota hai. [filecite\turn21file0\L39-L49](#)

5. Layer 6 ka complete pipeline

Sabse important flow yaad rakho:



This is the retrieval sequence specified in the report. [filecite\turn21file0\L58-L97](#)

6. Step 0 — Cache Check

Pehle cache kyu check karte hain?

Maan lo user ne query ki:

```
"What is Round Robin?"
```

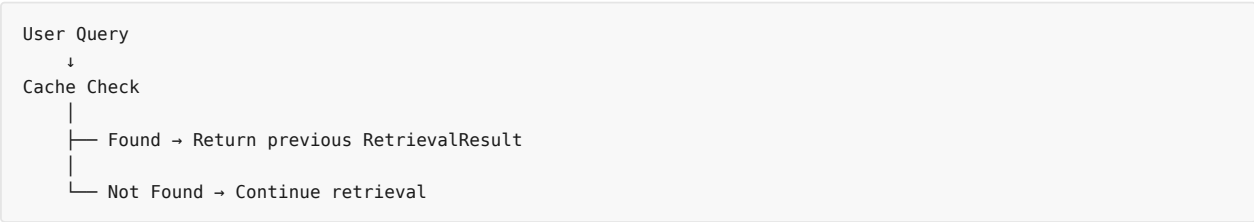
Aur 5 minutes baad koi same query fir poochta hai.

System ko har baar:

```
Query Analysis
→ Embedding
→ Vector Search
→ Forest Traversal
→ Tree Knapsack
```

repeat karna zaroori nahi.

Instead:



Report's retrieval pseudocode performs this cache check before query analysis. `filecite[turn21file0L58-L62]`

Technology

Cache Manager

The broader system specification also mentions Redis integration for caching query results. `filecite[turn17file0L22-L35]`

7. Step 1 — Query Analysis

Ab query hai:

“How does Round Robin work and why is Time Quantum important?”

Layer 6 pehle query ko understand karta hai.

Do important questions:

Question 1:

User kya pooch raha hai?

Question 2:

Answer ke liye kitni deep retrieval chahiye?

8. Query Intent kya hota hai?

Intent means:

User information kis form mein chahta hai?

Source examples include:

DEFINITION
COMPARISON
EXPLANATION
MULTI-HOP
SYNTHESIS

For example:

Query	Intent
What is Round Robin?	DEFINITION
Compare Round Robin and FCFS	COMPARISON
How does Round Robin work?	EXPLANATION
How does X affect Y?	MULTI-HOP
Summarize all scheduling algorithms	SYNTHESIS

The report uses these query categories to guide retrieval. `filecite[turn21file0L27-L35]`

9. Query Complexity Classification

Ye Layer 6 ka important part hai.
Har query ko same amount of context nahi chahiye.

SIMPLE Query

"What is Round Robin?"

Maybe system ko chahiye:

Round Robin
+
its parent

MODERATE Query

"How does Round Robin work?"

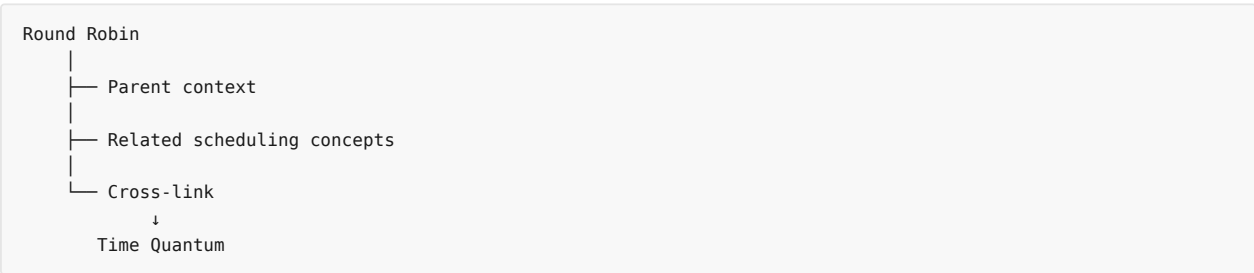
System ko chahiye:

Round Robin
+
Parent Context
+
Relevant Children/related concepts

COMPLEX Query

"How does Round Robin affect performance and why is Time Quantum important?"

Ab answer multiple connected concepts se banega:



10. Adaptive Hierarchical Traversal

Yahan Layer 6 ka important idea aata hai:

Adaptive Depth

System fixed depth use nahi karta.
Query complexity ke according traversal depth change hoti hai.
Source configuration:

Complexity	Max Depth	Parents	Children	Cross-links
SIMPLE	1	Yes	No	No

| MODERATE | 2 | Yes | Yes | No |
| COMPLEX | 3 | Yes | Yes | Yes |
fileciteurn21file0L72-L78

Iski zarurat kyu hai?

Traditional retrieval aksar:

Top 5 chunks

retrieve karta hai.

Chahe question:

What is Round Robin?

ho ya:

How does Round Robin interact with Time Quantum?

Fixed retrieval amount use ho sakta hai.

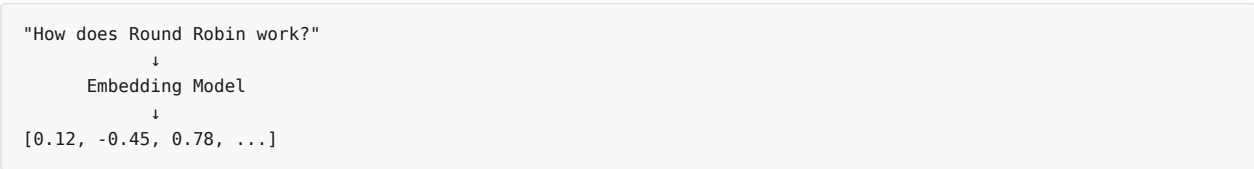
Layer 6 ka approach:

- Question simple hai → less traversal.
- Question complex hai → more connected knowledge retrieve karo.

This adaptive-depth behavior is one of the report's proposed retrieval contributions. fileciteurn18file0L28-L34

11. Step 2 — Query Embedding

Ab system natural-language query ko vector mein convert karta hai.



Is vector ka purpose:

- Query ka semantic meaning represent karna.

Example

Ye dono exact same words nahi hain:

"Explain Round Robin"

and

"How does the Round Robin scheduling algorithm work?"

But semantic meaning similar hai.

Embedding system ko help karta hai:

```

Different Words
  ↓
Similar Meaning
  ↓
Similar Vectors

```

Layer 6's source pipeline embeds the query and uses the vector index to find initial seed nodes. [filecite]turn21file0[L68-L70]

12. Step 3 — Seed Node Retrieval

Ab query vector vector database/index mein search hota hai.

```

Query Embedding
  ↓
Vector Search
  ↓
Top-k Similar KnowledgeNodes

```

Example:

Query:

"How does Round Robin work?"

Possible seed nodes:

```

Seed 1 → Round Robin
Seed 2 → CPU Scheduling Algorithm
Seed 3 → Time Quantum
Seed 4 → CPU Scheduling
Seed 5 → FCFS

```

Source retrieval pseudocode uses `k=5` seed nodes in the shown configuration. [filecite]turn21file0[L68-L70]

Seed Node kya hota hai?

Simple words mein:

Seed node is the starting point from which the system begins exploring the Knowledge Forest.

Example:

```

Operating Systems
  ↓
CPU Scheduling
  ↓
CPU Scheduling Algorithm
  ↓
[ROUND ROBIN]
  ↓
  USES
  ↓
Time Quantum

```

Yahan:

Round Robin

seed ho sakta hai.

13. Vector Search ke baad direct answer kyu nahi?

Ye bahut important point hai.

Suppose vector search returns:

```
Round Robin
```

Agar system sirf isi node ko Layer 7 ko bhej de:

- × Parent context missing ho sakta hai
- × Important related concept missing ho sakta hai
- × Multi-hop question answer nahi hoga
- × Semantic relation ka context missing ho sakta hai

Isliye Layer 6 sirf **search nahi karta**.

It does:

```
Search + Traverse + Select
```

14. Step 4 — Knowledge Forest Traversal

Ab seed node mil gaya:

```
Round Robin
```

System Knowledge Forest mein travel karega.

Source implementation specifies **BFS traversal from all seed nodes**. [filecite\turn21file0\L80-L84](#)

15. SIMPLE Query Traversal

Query:

```
"What is Round Robin?"
```

Complexity:

```
SIMPLE
Depth = 1
```

Traversal:

```
CPU Scheduling Algorithm
|
▼
Round Robin
```

System:

- Seed node retrieve karega ✓
- Parent include karega ✓
- Children explore nahi karega
- Cross-links follow nahi karega

Source configuration follows this pattern for SIMPLE queries. [filecite\turn21file0\L72-L78](#)

16. MODERATE Query Traversal

Query:

"How does Round Robin work?"

Complexity:

MODERATE
Depth = 2

System explore kar sakta hai:

```

CPU Scheduling
  |
  ▼
CPU Scheduling Algorithm
  |
  ▼
Round Robin
  
```

Aur relevant children bhi explore kiye ja sakte hain.

Parents ✓
Children ✓
Cross-links ✗

17. COMPLEX Query Traversal

Query:

"Why is Time Quantum important in Round Robin?"

Complexity:

COMPLEX
Depth = 3

Ab system:

Parents ✓
Children ✓
Cross-links ✓

Explore kar sakta hai.

Example:

```

CPU Scheduling
  |
  ▼
CPU Scheduling Algorithm
  |
  ▼
Round Robin
  |
  └─── USES ───> Time Quantum
  
```

Yahan cross-link follow karna important hai, kyunki:

Round Robin
USES
Time Quantum

Layer 5 ne ye semantic cross-link banaya tha, aur Layer 6 complex queries mein ise use kar sakta hai. `filecite\turn21file0\L72-L84`

18. BFS kya kar raha hai?

BFS = Breadth-First Search

Simple idea:

```
Seed Node
  ↓
First nearby level
  ↓
Second nearby level
  ↓
Third nearby level
```

Example:

```
Level 0:
Round Robin

Level 1:
CPU Scheduling Algorithm
Time Quantum

Level 2:
CPU Scheduling
Related nodes
```

System pehle nearest relevant connections dekhta hai.

Depth limit hone ki wajah se poora forest traverse nahi hota.

19. Traversal ke baad kya milta hai?

Output:

Candidate Nodes

Example:

```
Candidate Nodes

1. Round Robin
2. CPU Scheduling Algorithm
3. CPU Scheduling
4. Time Quantum
5. FCFS
6. Priority Scheduling
...
```

Lekin ab ek problem hai.

20. Problem — Sab candidate nodes LLM ko de dein?

Suppose 50 candidate nodes hain.

Agar har node ka:

- definition
- evidence

- examples
- metadata

LLM ko bhej diya:

```
50 Nodes
↓
Huge Context
↓
Token Budget Exceeded
```

Ya:

```
Too Much Context
↓
Irrelevant Information
↓
LLM Confusion
```

Isliye next step aata hai:

Tree Knapsack

21. Step 5 — Tree Knapsack Selection

Ye Layer 6 ka **sabse important algorithmic part** hai.

Simple question:

“Candidate nodes mein se kaunse nodes select karein so that maximum useful information mile, but token budget exceed na ho?”

Example:

Token Budget:

```
1000 tokens
```

Candidate nodes:

Node	Utility	Token Cost
Round Robin	High	250
CPU Scheduling Algorithm	High	200
Time Quantum	Very High	200
FCFS	Low	250
Operating Systems	Medium	400

Hum sab select nahi kar sakte.

So algorithm choose karega:

```
Round Robin
+
CPU Scheduling Algorithm
+
Time Quantum
```

because these are more useful for the query.

The source formally defines Tree Knapsack as maximizing node utility while staying within token budget and respecting parent-child precedence constraints. [filecite\turn20file1\L35-L43](#)

22. Normal Knapsack aur Tree Knapsack mein difference

Normal Knapsack

Normal rule:

Select item independently

Example:

Laptop ✓
Phone ✓
Tablet ✗

Har item independent hai.

Tree Knapsack

Hierarchy mein nodes independent nahi hote.

Example:

CPU Scheduling Algorithm
↓
Round Robin

Agar:

Round Robin

select karte ho, hierarchy coherence ke liye parent context bhi required ho sakta hai.

Mathematically:

Child selected
↓
Parent must also be selected

Source formula expresses this precedence rule as:

$$x(\text{child}) \leq x(\text{parent})$$

[filecite\turn20file1\L35-L43](#)

23. Ye precedence constraint kyu important hai?

Maan lo Layer 7 ko sirf ye context mila:

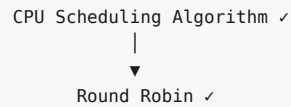
Round Robin:
Uses a time quantum.

But parent context missing hai:

Round Robin is a CPU Scheduling Algorithm.

Answer ka broader context weak ho sakta hai.

Tree Knapsack hierarchy ko preserve karta hai.



Instead of randomly selecting disconnected nodes.

24. Node Utility kaise calculate hoti hai?

Source utility function ke according:

```

Utility
=
Node Confidence
× Edge Weight
× Evidence Richness
× Cost Efficiency
  
```

Conceptually:

1. Node Confidence

High confidence → More useful

2. Edge Weight

Previous retrieval feedback se edge weight influence ho sakta hai.

```

graph TD
    A["Good retrieval path"] --> B["Higher edge weight"]
    B --> C["More likely useful in future"]
  
```

3. Evidence Richness

```

graph TD
    A["More evidence"] --> B["Higher utility"]
  
```

4. Token Cost Efficiency

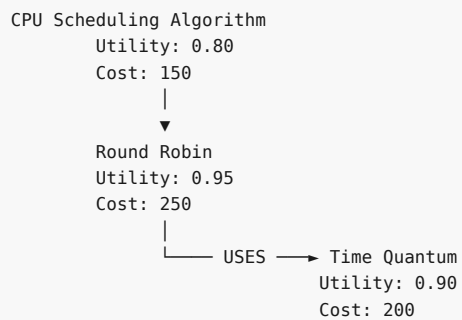
```

graph TD
    A["Useful information"] --> B["÷"]
    B --> C["Token cost"]
  
```

Source utility pseudocode combines these signals directly. [filecite turn21file0 L133-L149](#)

25. Simple Tree Knapsack example

Candidate tree:



Token budget:

600 tokens

Possible selection:

```

CPU Scheduling Algorithm
+
Round Robin
+
Time Quantum
  
```

Total:

150 + 250 + 200
= 600 tokens

Perfect fit.

But if budget:

400 tokens

Then system cannot blindly choose everything.

Tree constraint ke according hierarchy-preserving selection karega.

26. Multi-Tree Knapsack

Ek aur important scenario.

Suppose query:

"Compare Round Robin scheduling and B+ Tree indexing."

Ab seed nodes different Knowledge Trees se aa sakte hain.

```

Tree 1
Operating Systems
  |
  ▼
Round Robin

Tree 2
Data Structures
  |
  ▼
B+ Tree
  
```

Ab system ko dono trees se context chahiye.

Source algorithm:

1. Group candidates by tree root
2. Allocate budget proportionally
3. Run Tree Knapsack separately
4. Merge results
5. Trim to total budget

filecite\turn21file0\L100-L130

27. Hamare example mein Tree Knapsack

Query:

"Why is Time Quantum important in Round Robin?"

Candidate nodes:

Operating Systems
CPU Scheduling
CPU Scheduling Algorithm
Round Robin
Time Quantum
FCFS
Priority Scheduling

Tree Knapsack decide karega:

SELECT ✓

CPU Scheduling Algorithm
Round Robin
Time Quantum

Maybe:

CPU Scheduling

include kare, if parent precedence/context requires it.

But:

FCFS ✗
Priority Scheduling ✗

because query ke liye unnecessary hain.

So final context becomes:

Small
Relevant
Connected
Within Token Budget

28. Step 6 — Context Assembly

Ab selected nodes hain:

CPU Scheduling Algorithm
Round Robin
Time Quantum

Lekin Layer 7 ko raw node objects nahi bhejne.

Context assemble karna padega.

Source configuration says:

Definitions first, then core information, sorted by confidence.

filecite\turn21file0\L85-L97

29. Context Assembly example

Input:

```
Selected Node 1:
CPU Scheduling Algorithm

Selected Node 2:
Round Robin

Selected Node 3:
Time Quantum
```

Context Assembler output:

```
CONTEXT

[CPU Scheduling Algorithm]
Definition:
...

[Round Robin]
Definition:
...
Key properties:
...
Evidence:
...

[Time Quantum]
Definition:
...
Importance:
...
Evidence:
...
```

Source materials describe collecting node definitions, evidence, example sentences, and optionally source snippets during context packing. filecite\turn18file6\L440-L455

30. Semantic order kyu important hai?

Agar context random order mein ho:

```
Time Quantum
Operating Systems
Round Robin
CPU Scheduling
```

LLM ko structure samajhne mein difficulty ho sakti hai.

Better:


```
General Context
↓
Core Concept
↓
Specific Concept
↓
Related Semantic Concept
```

Example:

```
CPU Scheduling Algorithm
↓
Round Robin
↓
Time Quantum
```

Ye logical context flow banata hai.

31. Final RetrievalResult

Ab Layer 6 final object banata hai:

```
RetrievalResult
{
  selected_nodes:
    [
      CPU Scheduling Algorithm,
      Round Robin,
      Time Quantum
    ],

  context_text:
    "assembled relevant context...",

  traversal_path_edges:
    [
      CPU Scheduling Algorithm → Round Robin,
      Round Robin → Time Quantum
    ],

  query_complexity:
    COMPLEX,

  query_intent:
    EXPLANATION,

  seed_node_ids:
    [
      Round Robin,
      ...
    ],

  token_count:
    580
}
```

Source schema defines these fields as the output contract for Layer 6. `filecite\turn21file0\L39-L49`

32. Complete example from start to end

Let's trace our exact query.

User Query

```
"Why is Time Quantum important in Round Robin?"
```

Step 0 — Cache

```
Cache Hit?  
├── Yes → Return old result  
└── No → Continue
```

Step 1 — Analyze Query

Intent:
EXPLANATION / WHY

Complexity:
COMPLEX

Step 2 — Create Query Embedding

```
Query  
↓  
Embedding Model  
↓  
Query Vector
```

Step 3 — Find Seed Nodes

```
Vector Search  
↓  
Seed Nodes  
  
1. Round Robin  
2. Time Quantum  
3. CPU Scheduling Algorithm
```

Step 4 — Adaptive Traversal

Because:

COMPLEX

system explores:

```
Parents ✓  
Children ✓  
Cross-links ✓  
Depth = 3
```

Result:

```
CPU Scheduling  
|  
▼  
CPU Scheduling Algorithm  
|  
▼  
Round Robin  
|  
└── USES ───> Time Quantum
```

Step 5 — Candidate Nodes

```
CPU Scheduling
CPU Scheduling Algorithm
Round Robin
Time Quantum
FCFS
Priority Scheduling
```

Step 6 — Tree Knapsack

Token budget:

```
1000 tokens
```

Selected:

```
CPU Scheduling Algorithm ✓
Round Robin ✓
Time Quantum ✓

FCFS ✗
Priority Scheduling ✗
```

Step 7 — Context Assembly

```
CPU Scheduling Algorithm
  ↓
Round Robin
  ↓
Time Quantum
```

Definitions and evidence are assembled.

Final Output

```
RetrievalResult
  ↓
Layer 7
  ↓
LLM generates final answer
```

33. Layer 6 ke main technologies/algorithms

| Component | Technology / Algorithm | Kaam |

|---|---|---|

| Query Analysis | Intent Detection | User kya pooch raha hai |

| Complexity Classification | Rules / classifier | Retrieval kitni deep hogi |

| Query Embedding | Embedding Model | Query ko semantic vector banana |

| Seed Retrieval | Vector Search / ANN | Starting KnowledgeNodes find karna |

| Traversal | BFS | Forest mein controlled exploration |

| Adaptive Depth | Complexity-based configuration | Query ke according depth change karna |

| Cross-Link Traversal | Semantic edges | Related knowledge retrieve karna |

| Context Selection | Tree Knapsack | Best nodes under token budget |

Multi-Tree Selection	Per-tree optimization + merge	Multiple trees handle karna
Context Assembly	Semantic ordering	Layer 7 ke liye readable context
Cache	Query result caching	Repeated retrieval fast banana

The architecture explicitly identifies **Adaptive Depth** and **Tree Knapsack** as the key algorithms for Layer 6.
filecite\turn17file4\L391-L403

34. Layer 6 mein possible errors

Error 1 — Wrong seed node

Query:

"Explain Round Robin"

But vector search retrieves:

Priority Scheduling

Safeguard

Multiple seed nodes:

Top-k seeds

and semantic search rather than relying on one exact match.

The shown configuration retrieves multiple seed nodes ($k=5$). filecite\turn21file0\L68-L70

Error 2 — Simple query mein too much traversal

"What is Round Robin?"

Agar poora subtree traverse ho:

× Wasted tokens

× Noise

Safeguard

SIMPLE
Depth = 1

Error 3 — Complex query mein insufficient context

"Why is Time Quantum important?"

Agar sirf:

Round Robin

retrieve hua:

× Time Quantum relation missing.

Safeguard

COMPLEX
↓
Cross-links enabled

Error 4 — Token budget exceed

Candidate nodes:

20 nodes

But LLM context budget limited hai.

Safeguard:

Tree Knapsack

Only high-utility nodes select karega within budget. `filecite\turn20file1\L35-L43`

Error 5 — Child without required parent context

Example:

Round Robin ✓
Parent x

Safeguard

Precedence constraint:

Child selected
↓
Parent must be selected

Error 6 — Vector database unavailable

The report includes a fallback pattern:

Vector DB unavailable
↓
Use cached recent results

`filecite\turn17file8\L678-L684`

35. Layer 5 vs Layer 6

Layer 5 asks:

"Where does this knowledge belong?"

Example:

CPU Scheduling Algorithm
↓
Round Robin

Layer 6 asks:

| *"For this user's query, which knowledge should I retrieve?"*

Example query:

Why is Time Quantum important?

Retrieval path:

Round Robin
↓
Cross-link
↓
Time Quantum

36. Traditional RAG vs Layer 6

Traditional RAG

User Query
↓
Vector Search
↓
Top-K Chunks
↓
LLM

SHIA-RAG Layer 6

User Query
↓
Understand Intent
↓
Classify Complexity
↓
Vector Search
↓
Find Seed Nodes
↓
Traverse Hierarchy
↓
Follow Semantic Links if needed
↓
Tree Knapsack
↓
Assemble Structured Context
↓
Layer 7

The core distinction in the source is that SHIA-RAG retrieves by navigating a structured Knowledge Forest rather than simply returning flat top-k chunks. [filecite turn18file0 L19-L36](#)

37. Layer 6 ka most important concept

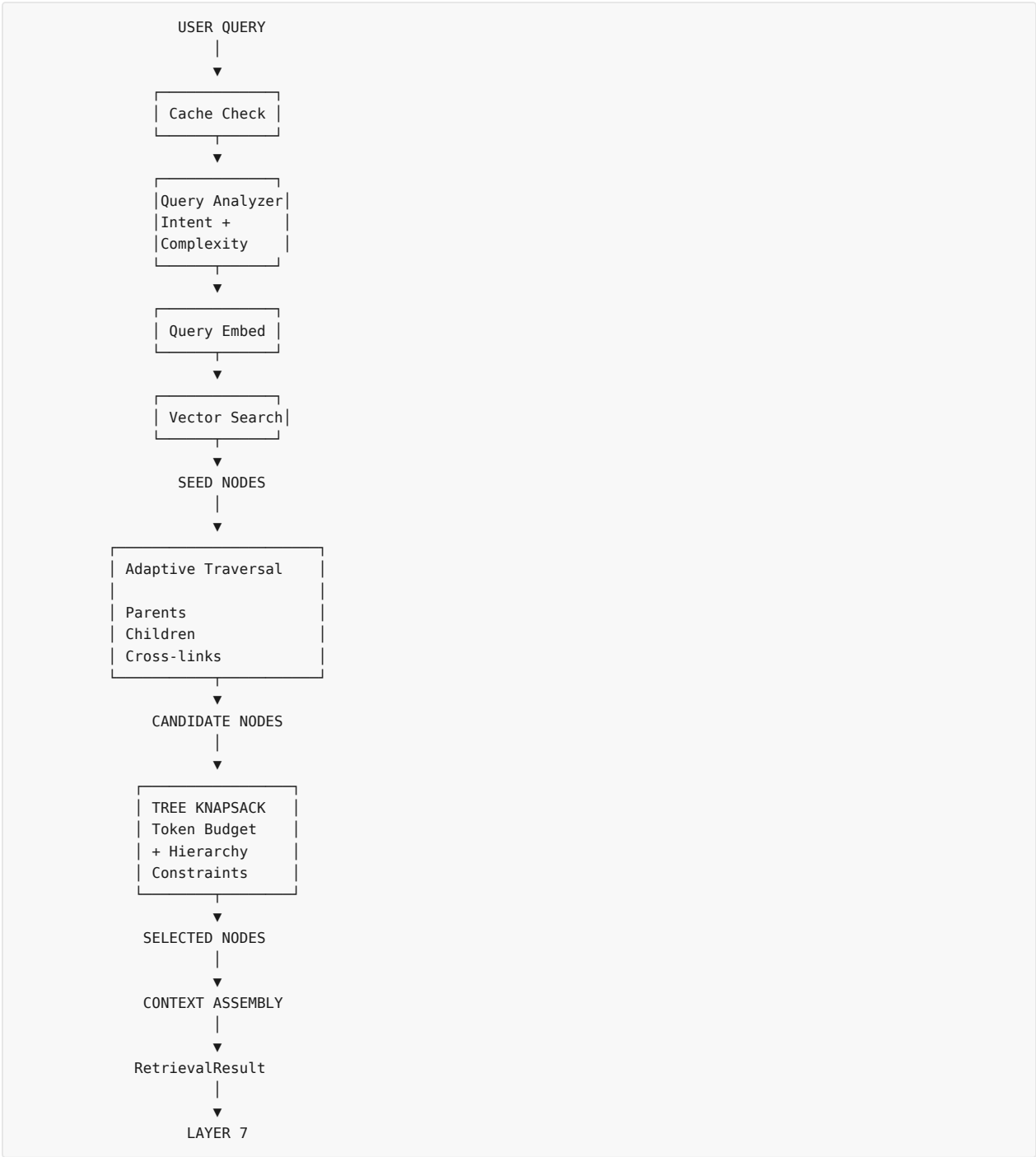
Search alone is not retrieval.

Layer 6 actually does:

```
SEARCH
+
UNDERSTAND QUERY
+
NAVIGATE KNOWLEDGE STRUCTURE
+
SELECT OPTIMAL CONTEXT
```

This is the core idea.

38. Layer 6 ka complete architecture



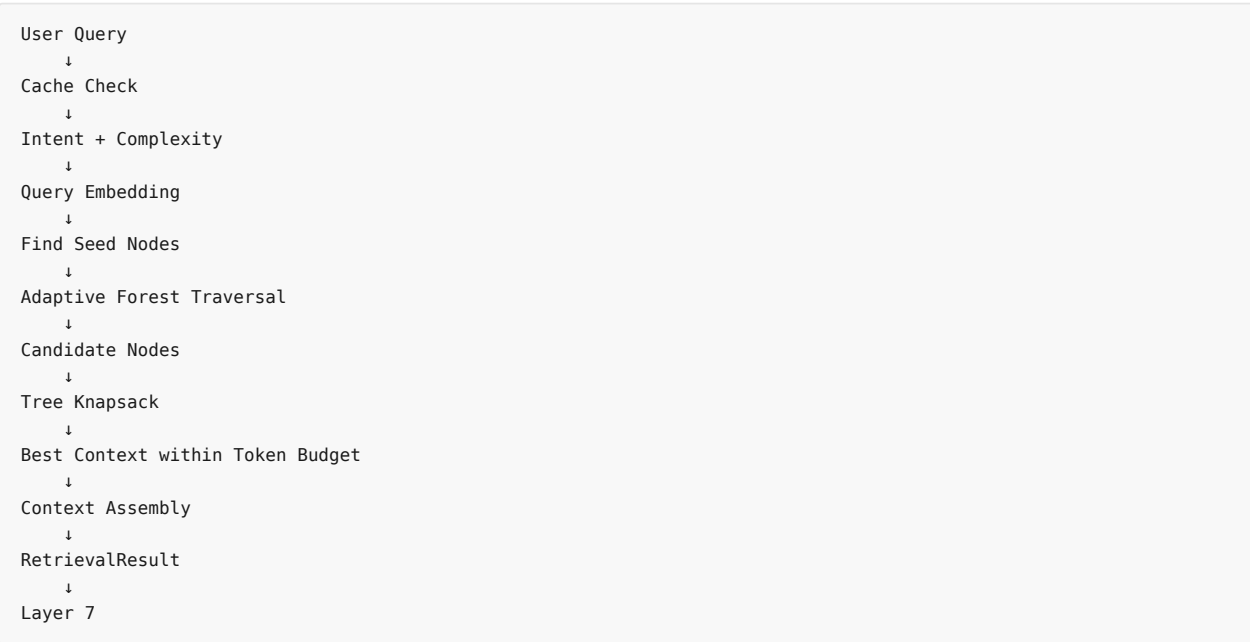
39. Final Summary Table

Point	Layer 6
Name	Retrieval Engine
Input	User Query
Knowledge Source	Knowledge Forest + Vector Index
First Step	Cache Check
Query Processing	Intent + Complexity Classification
Search Method	Query Embedding + Vector Search
Starting Points	Seed Nodes
Traversal	BFS
Adaptive Strategy	Depth depends on query complexity
Simple Query	Parent context, shallow depth
Moderate Query	Parents + children
Complex Query	Parents + children + cross-links
Selection Algorithm	Tree Knapsack
Constraint	Token Budget + Hierarchical Precedence
Final Processing	Context Assembly
Output	RetrievalResult
Next Layer	Layer 7 – Generation Engine

40. Sabse important one-line understanding

“Layer 6 is the intelligent retrieval layer of SHIA-RAG. It understands the user's query, finds relevant seed concepts, adaptively traverses the Knowledge Forest based on query complexity, selects the most useful connected knowledge within the token budget using Tree Knapsack, and sends the structured context to Layer 7 for answer generation.”

Complete flow yaad rakho:



Ek simple analogy

Layer 5 built and organized the library. Layer 6 is the intelligent librarian who understands the question, finds the right starting books, explores only the necessary connected shelves, removes unnecessary information, fits the best knowledge within the available space, and gives that context to Layer 7.

□memcite□

YOU 24/08/2026, 01:32:57

mujhe layer 6 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 6 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 24/08/2026, 01:33:06

{ "size": "1024x1024", "n": 1, "transparent_background": false, "is_style_transfer": false, "prompt": null, "referenced_image_ids": null }

CHATGPT 24/08/2026, 01:33:56

Layer 6 – Retrieval Engine

You can explain it to your guide like this:

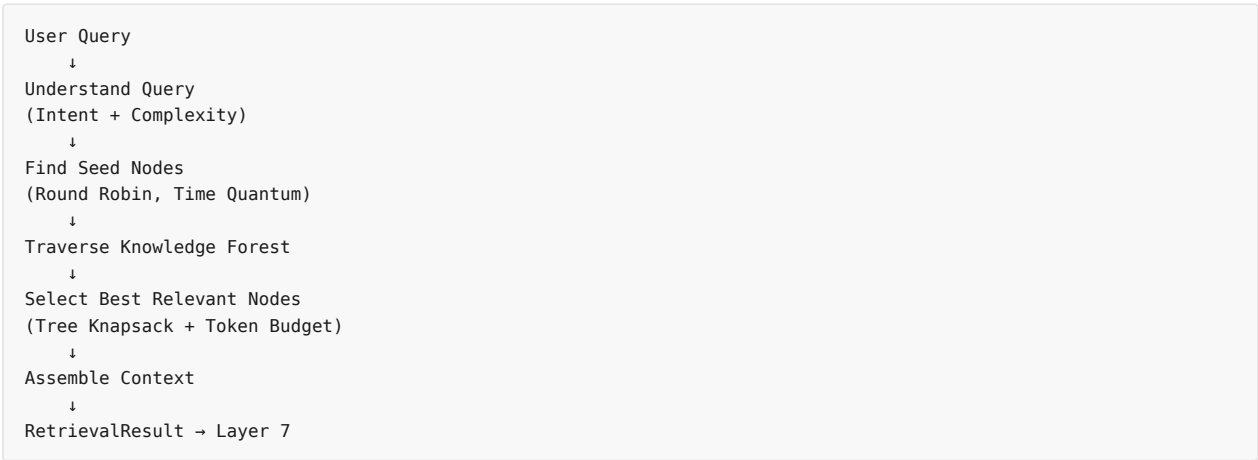
“Layer 6 is the intelligent retrieval layer of SHIA-RAG. It receives the user query and retrieves the most relevant knowledge from the Knowledge Forest created in Layer 5. Instead of simply selecting random top-k documents, it understands the query, finds relevant starting nodes, adaptively traverses the knowledge hierarchy, and selects the best context within the available token budget.”

Simple Example

User asks:

“Why is Time Quantum important in Round Robin?”

Layer 6:



Main Components

- **Query Analyzer** – Understands the user's intent and query complexity.
- **Vector Search** – Finds relevant starting nodes (seed nodes).
- **Adaptive Hierarchical Traversal (AHT)** – Explores the Knowledge Forest based on query complexity.
- **Tree Knapsack** – Selects the most useful knowledge within the token limit.
- **Context Assembly** – Organizes the selected knowledge before sending it to Layer 7.

Output

A structured **RetrievalResult** containing the most relevant nodes and context for answer generation.

In one simple line

"Layer 6 intelligently searches and navigates the Knowledge Forest to retrieve only the most relevant and useful information for answering the user's query."

YOU 24/08/2026, 01:42:26

ab mujhe layer 7 smjhao depth and detailed me jese baki layer smjhayi thi

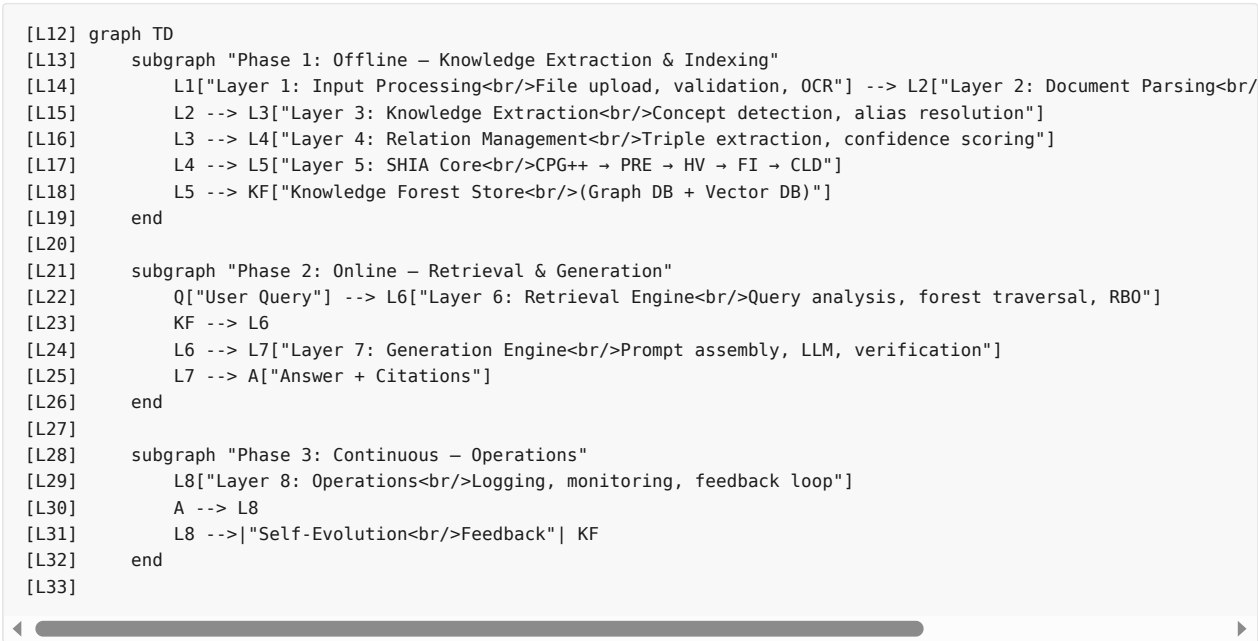
CHATGPT 24/08/2026, 01:42:31

{"paths":["files"],"query":"search"}

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `Filecite`turn23file0

[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.03007688828584351
[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6
[L10] snippet:
[L11]



[L34]
[L35] ### 3.2 Data Flow
[L36]
[L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39]   ↳ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41]   ↳ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43]   ↳ Layer 3: Split sentences, extract concepts/definitions/rerelations
[L44] KnowledgeUnit objects (candidate concepts)
[L45]   ↳ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47]   ↳ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49]   ↳ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51]   ↳ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53]   ↳ Layer 7: Build prompt, invoke LLM
[L54]   ↳ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56]   ↳ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]

[L59] ### 3.3 Layer Summary

[L60]

[L61] | Layer | Name | Input | Output | Key Algorithms |

[L62] |-----|-----|-----|-----|-----|

[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L72]

[L73] ---

[L74]

[L75] ## 4. Data Model — Layer 0

Citation Marker: fileciteturn23file1

[L76] metadata:

[L77] query_result_index: 1

[L78] file_id: file_000000003218820b918ee3d86ec345ae

[L79] version_id: 1

[L80] name: SHIA_RAG_Final_Report(1).md

[L81] mime_type: text/markdown

[L82] surface: library

[L83] score: 0.025165372447512224

[L84] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--6

[L85] snippet:

[L86]

```
[L87] graph TD
[L88]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L89]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L90]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L91]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L92]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L93]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L94]     end
[L95]
[L96]     subgraph "Phase 2: Online – Retrieval & Generation"
[L97]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L98]         KF --> L6
[L99]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L100]        L7 --> A["Answer + Citations"]
[L101]     end
[L102]
[L103]    subgraph "Phase 3: Continuous – Operations"
[L104]        L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L105]        A --> L8
[L106]        L8 -->| "Self-Evolution<br/>Feedback" | KF
[L107]    end
[L108]
```

[L109]

[L110] ### 3.2 Data Flow

[L111]

[L112]

```
[L113] Raw Document (PDF/DOCX/HTML)
[L114]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L115] Normalized Text + Metadata
[L116]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L117] [Page → Region → Block] hierarchy
[L118]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L119] KnowledgeUnit objects (candidate concepts)
[L120]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L121] KnowledgeNode objects (canonical, deduplicated)
[L122]     ↓ Layer 4: Validate relations against ontology, score confidence
[L123] KnowledgeEdge objects (typed, scored)
[L124]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L125] Knowledge Forest (rooted trees + cross-links)
[L126]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L127] Selected context nodes (within token budget)
[L128]     ↓ Layer 7: Build prompt, invoke LLM
[L129]     ↓ Layer 7: Verify claims against evidence, bind citations
[L130] Answer with citations + confidence score
[L131]     ↓ Layer 8: Log, collect feedback, update forest weights
[L132]
```

[L133]

[L134] ### 3.3 Layer Summary

[L135]

[L136] | Layer | Name | Input | Output | Key Algorithms |

[L137] |-----|-----|-----|-----|-----|

[L138] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L139] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L140] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L141] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L142] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L143] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L144] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L145] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L146] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L147]
[L148] ---
[L149]
[L150] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `filecite`turn23file2

[L151] metadata:
[L152] query_result_index: 2
[L153] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L154] version_id: 1
[L155] name: SHIA_RAG_Final_Report(2).md
[L156] mime_type: text/markdown
[L157] surface: library
[L158] score: 0.02490096208262592
[L159] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--6
[L160] snippet:
[L161]

```
[L162] graph TD
[L163]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L164]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L165]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L166]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L167]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L168]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L169]     end
[L170]
[L171]     subgraph "Phase 2: Online – Retrieval & Generation"
[L172]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L173]         KF --> L6
[L174]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L175]         L7 --> A["Answer + Citations"]
[L176]     end
[L177]
[L178]     subgraph "Phase 3: Continuous – Operations"
[L179]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L180]         A --> L8
[L181]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L182]     end
[L183]
```

[L184]
[L185] ### 3.2 Data Flow
[L186]
[L187]

```
[L188] Raw Document (PDF/DOCX/HTML)
[L189]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L190] Normalized Text + Metadata
[L191]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L192] [Page → Region → Block] hierarchy
[L193]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L194] KnowledgeUnit objects (candidate concepts)
[L195]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L196] KnowledgeNode objects (canonical, deduplicated)
[L197]   ↓ Layer 4: Validate relations against ontology, score confidence
[L198] KnowledgeEdge objects (typed, scored)
[L199]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L200] Knowledge Forest (rooted trees + cross-links)
[L201]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L202] Selected context nodes (within token budget)
[L203]   ↓ Layer 7: Build prompt, invoke LLM
[L204]   ↓ Layer 7: Verify claims against evidence, bind citations
[L205] Answer with citations + confidence score
[L206]   ↓ Layer 8: Log, collect feedback, update forest weights
[L207]
```

[L208]

[L209] ### 3.3 Layer Summary

[L210]

[L211] | Layer | Name | Input | Output | Key Algorithms |

[L212] |-----|-----|-----|-----|-----|

[L213] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L214] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L215] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L216] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L217] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L218] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L219] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L220] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L221] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L222]

[L223] ---

[L224]

[L225] ## 4. Data Model — Layer 0

Citation Marker: `[[filecite]]turn23file3[[`

[L226] metadata:

[L227] query_result_index: 3

[L228] file_id: file_00000000fb2c81fd82f14a7a48993554

[L229] version_id: 1

[L230] name: SHIA_RAG_Final_Report.md

[L231] mime_type: text/markdown

[L232] surface: library

[L233] score: 0.024644945697577275

[L234] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--6

[L235] snippet:

[L236]

```
[L237] graph TD
[L238]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L239]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L240]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L241]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L242]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L243]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L244]     end
[L245]
[L246]     subgraph "Phase 2: Online – Retrieval & Generation"
[L247]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L248]         KF --> L6
[L249]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L250]         L7 --> A["Answer + Citations"]
[L251]     end
[L252]
[L253]     subgraph "Phase 3: Continuous – Operations"
[L254]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L255]         A --> L8
[L256]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L257]     end
[L258]
```

[L259]

[L260] ### 3.2 Data Flow

[L261]

[L262]

```
[L263] Raw Document (PDF/DOCX/HTML)
[L264]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L265] Normalized Text + Metadata
[L266]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L267] [Page → Region → Block] hierarchy
[L268]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L269] KnowledgeUnit objects (candidate concepts)
[L270]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L271] KnowledgeNode objects (canonical, deduplicated)
[L272]     ↓ Layer 4: Validate relations against ontology, score confidence
[L273] KnowledgeEdge objects (typed, scored)
[L274]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L275] Knowledge Forest (rooted trees + cross-links)
[L276]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L277] Selected context nodes (within token budget)
[L278]     ↓ Layer 7: Build prompt, invoke LLM
[L279]     ↓ Layer 7: Verify claims against evidence, bind citations
[L280] Answer with citations + confidence score
[L281]     ↓ Layer 8: Log, collect feedback, update forest weights
[L282]
```

[L283]

[L284] ### 3.3 Layer Summary

[L285]

[L286] | Layer | Name | Input | Output | Key Algorithms |

[L287] |-----|-----|-----|-----|-----|

[L288] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L289] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L290] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L291] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L292] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L293] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L294] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L295] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L296] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L297]
[L298] ---
[L299]
[L300] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `filecite`turn23file4

[L301] metadata:
[L302] query_result_index: 4
[L303] file_id: file_00000000f620821186e54e582d59731e
[L304] version_id: 1
[L305] name: SHIA_RAG_Final_Report(3).md
[L306] mime_type: text/markdown
[L307] surface: library
[L308] score: 0.024396929824561403
[L309] document_chunk_id: file_00000000f620821186e54e582d59731e--1--6
[L310] snippet:
[L311]

```
[L312] graph TD
[L313]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L314]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L315]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L316]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L317]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L318]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L319]     end
[L320]
[L321]     subgraph "Phase 2: Online – Retrieval & Generation"
[L322]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L323]         KF --> L6
[L324]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L325]         L7 --> A["Answer + Citations"]
[L326]     end
[L327]
[L328]     subgraph "Phase 3: Continuous – Operations"
[L329]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L330]         A --> L8
[L331]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L332]     end
[L333]
```

[L334]
[L335] ### 3.2 Data Flow
[L336]
[L337]


```
[L338] Raw Document (PDF/DOCX/HTML)
[L339]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L340] Normalized Text + Metadata
[L341]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L342] [Page → Region → Block] hierarchy
[L343]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L344] KnowledgeUnit objects (candidate concepts)
[L345]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L346] KnowledgeNode objects (canonical, deduplicated)
[L347]   ↓ Layer 4: Validate relations against ontology, score confidence
[L348] KnowledgeEdge objects (typed, scored)
[L349]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L350] Knowledge Forest (rooted trees + cross-links)
[L351]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L352] Selected context nodes (within token budget)
[L353]   ↓ Layer 7: Build prompt, invoke LLM
[L354]   ↓ Layer 7: Verify claims against evidence, bind citations
[L355] Answer with citations + confidence score
[L356]   ↓ Layer 8: Log, collect feedback, update forest weights
[L357]
```

[L358]

[L359] ### 3.3 Layer Summary

[L360]

[L361] | Layer | Name | Input | Output | Key Algorithms |

[L362] |-----|-----|-----|-----|-----|

[L363] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L364] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L365] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L366] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L367] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L368] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L369] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L370] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L371] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L372]

[L373] ---

[L374]

[L375] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `[[filecite]]turn23file5`

[L376] metadata:

[L377] query_result_index: 5

[L378] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L379] version_id: 1

[L380] name: SHIA_RAG_Final_Report(4).md

[L381] mime_type: text/markdown

[L382] surface: library

[L383] score: 0.02415654520917679

[L384] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--6

[L385] snippet:

[L386]

```
[L387] graph TD
[L388]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L389]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L390]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L391]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L392]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L393]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L394]     end
[L395]
[L396]     subgraph "Phase 2: Online – Retrieval & Generation"
[L397]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L398]         KF --> L6
[L399]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L400]         L7 --> A["Answer + Citations"]
[L401]     end
[L402]
[L403]     subgraph "Phase 3: Continuous – Operations"
[L404]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L405]         A --> L8
[L406]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L407]     end
[L408]
```

[L409]

[L410] ### 3.2 Data Flow

[L411]

[L412]

```
[L413] Raw Document (PDF/DOCX/HTML)
[L414]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L415] Normalized Text + Metadata
[L416]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L417] [Page → Region → Block] hierarchy
[L418]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L419] KnowledgeUnit objects (candidate concepts)
[L420]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L421] KnowledgeNode objects (canonical, deduplicated)
[L422]     ↓ Layer 4: Validate relations against ontology, score confidence
[L423] KnowledgeEdge objects (typed, scored)
[L424]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L425] Knowledge Forest (rooted trees + cross-links)
[L426]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L427] Selected context nodes (within token budget)
[L428]     ↓ Layer 7: Build prompt, invoke LLM
[L429]     ↓ Layer 7: Verify claims against evidence, bind citations
[L430] Answer with citations + confidence score
[L431]     ↓ Layer 8: Log, collect feedback, update forest weights
[L432]
```

[L433]

[L434] ### 3.3 Layer Summary

[L435]

[L436] | Layer | Name | Input | Output | Key Algorithms |

[L437] |-----|-----|-----|-----|-----|

[L438] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L439] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L440] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L441] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L442] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L443] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L444] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L445] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L446] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L447]
[L448] ---
[L449]
[L450] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `filecite`turn23file6

[L451] metadata:
[L452] query_result_index: 6
[L453] file_id: file_000000001dd481fa9855f2731dbc53d5
[L454] version_id: 1
[L455] name: SHIA_RAG_Complete_Report(6).md
[L456] mime_type: text/markdown
[L457] surface: library
[L458] score: 0.018057336133766127
[L459] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--63
[L460] snippet:
[L461] 5""",
[L462] "down": """"ALTER TABLE knowledge_nodes
[L463] DROP COLUMN abstraction_level,
[L464] DROP COLUMN base_confidence,
[L465] DROP COLUMN extraction_confidence""""
[L466] },
[L467] {
[L468] "version": 3,
[L469] "description": "Migrate properties from string array to JSONB",
[L470] "up": """"ALTER TABLE knowledge_nodes
[L471] ALTER COLUMN data TYPE JSONB USING data::JSONB"""" ,
[L472] "down": "-- Manual rollback required"
[L473] }
[L474]]
[L475]
[L476] def run_migrations(db, target_version=None):
[L477] current = db.get_migration_version()
[L478] for migration in MIGRATIONS:
[L479] if migration["version"] > current:
[L480] if target_version and migration["version"] > target_version:
[L481] break
[L482] db.execute(migration["up"])
[L483] db.set_migration_version(migration["version"])
[L484] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L485]

```
[L486]  
[L487] ---  
[L488]  
[L489] ### 7.13 Error Handling & Retry Logic  
[L490]  
[L491]
```

```
python
[L492] import time
[L493] from functools import wraps
[L494]
[L495] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L496] """Decorator for retrying failed operations with exponential backoff."""
[L497] def decorator(func):
[L498] @wraps(func)
[L499] def wrapper(*args, **kwargs):
[L500] for attempt in range(max_retries):
[L501] try:
[L502] return func(*args, **kwargs)
[L503] except (TimeoutError, ConnectionError) as e:
[L504] if attempt == max_retries - 1:
[L505] raise
[L506] delay = min(base_delay * (2 ** attempt), max_delay)
[L507] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L508] time.sleep(delay)
[L509] return wrapper
[L510] return decorator
[L511]
[L512] class ProcessingPipeline:
[L513] """Error handling for the SHIA-RAG processing pipeline."""
[L514]
[L515] def process_document(self, doc):
[L516] # Layer 1: OCR with fallback
[L517] try:
[L518] text = self.extract_text(doc)
[L519] except OCRFailure:
[L520] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L521]
[L522] # Layer 3: LLM extraction with retry + fallback model
[L523] try:
[L524] concepts = self.llm_extract_concepts(text)
[L525] except LLMLLMTimeout:
[L526] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L527]
[L528] # Layer 6: Vector DB with cache fallback
[L529] try:
[L530] results = self.vector_search(query_embedding)
[L531] except VectorDBUnavailable:
[L532] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L533]
[L534] return {"status": "SUCCESS", "concepts": concepts}
[L535]
```

```
[L536]
[L537] ---
[L538]
[L539] ### 7.14 Caching Strategy
[L540]
[L541]
```

python

```
[L542] import redis
[L543] import hashlib
[L544] import json
[L545]
[L546] class CacheManager:
[L547] """Multi-layer caching for SHIA-RAG."""
[L548]
[L549] def __init__(self, redis_url="redis://localhost:6379"):
[L550] self.redis = redis.from_url(redis_url)
[L551] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L552] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L553] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L554]
[L555] def get_query_result(self, query_text):
[L556] """Check if this exact query has been answered recently."""
```

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `filecite[turn23file7]`

```
[L557] metadata:
[L558] query_result_index: 7
[L559] file_id: file_00000000a0a481fab4c739c73d87551b
[L560] version_id: 1
[L561] name: SHIA_RAG_Complete_Report(7).md
[L562] mime_type: text/markdown
[L563] surface: library
[L564] score: 0.018057336133766127
[L565] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--63
[L566] snippet:
[L567] 5"""
[L568] "down": """ALTER TABLE knowledge_nodes
[L569] DROP COLUMN abstraction_level,
[L570] DROP COLUMN base_confidence,
[L571] DROP COLUMN extraction_confidence"""
[L572] },
[L573] {
[L574] "version": 3,
[L575] "description": "Migrate properties from string array to JSONB",
[L576] "up": """ALTER TABLE knowledge_nodes
[L577] ALTER COLUMN data TYPE JSONB USING data::JSONB"""
[L578] "down": "-- Manual rollback required"
[L579] }
[L580] ]
[L581]
```

```

[L582] def run_migrations(db, target_version=None):
[L583] current = db.get_migration_version()
[L584] for migration in MIGRATIONS:
[L585] if migration["version"] > current:
[L586] if target_version and migration["version"] > target_version:
[L587] break
[L588] db.execute(migration["up"])
[L589] db.set_migration_version(migration["version"])
[L590] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L591]

```

```

[L592]
[L593] ---
[L594]
[L595] ### 7.13 Error Handling & Retry Logic
[L596]
[L597]

```

python

```

[L598] import time
[L599] from functools import wraps
[L600]
[L601] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L602] """Decorator for retrying failed operations with exponential backoff."""
[L603] def decorator(func):
[L604] @wraps(func)
[L605] def wrapper(*args, **kwargs):
[L606] for attempt in range(max_retries):
[L607] try:
[L608] return func(*args, **kwargs)
[L609] except (TimeoutError, ConnectionError) as e:
[L610] if attempt == max_retries - 1:
[L611] raise
[L612] delay = min(base_delay * (2 ** attempt), max_delay)
[L613] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L614] time.sleep(delay)
[L615] return wrapper
[L616] return decorator
[L617]
[L618] class ProcessingPipeline:
[L619] """Error handling for the SHIA-RAG processing pipeline."""
[L620]
[L621] def process_document(self, doc):
[L622] # Layer 1: OCR with fallback
[L623] try:
[L624] text = self.extract_text(doc)
[L625] except OCRFailure:
[L626] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L627]
[L628] # Layer 3: LLM extraction with retry + fallback model
[L629] try:
[L630] concepts = self.llm_extract_concepts(text)

```

```
[L631] except LLMTimeout:
[L632] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L633]
[L634] # Layer 6: Vector DB with cache fallback
[L635] try:
[L636] results = self.vector_search(query_embedding)
[L637] except VectorDBUnavailable:
[L638] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L639]
[L640] return {"status": "SUCCESS", "concepts": concepts}
[L641]
```

```
[L642]
[L643] ---
[L644]
[L645] ### 7.14 Caching Strategy
[L646]
[L647]
```

```
python
[L648] import redis
[L649] import hashlib
[L650] import json
[L651]
[L652] class CacheManager:
[L653] """Multi-layer caching for SHIA-RAG."""
[L654]
[L655] def __init__(self, redis_url="redis://localhost:6379"):
[L656] self.redis = redis.from_url(redis_url)
[L657] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L658] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L659] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L660]
[L661] def get_query_result(self, query_text):
[L662] """Check if this exact query has been answered recently."""
```

DEEP RESEARCH 24/08/2026, 01:42:41

Citation Marker: `filecite[turn23file8]`

```
[L663] metadata:
[L664] query_result_index: 8
[L665] file_id: file_000000003c8081fab658b1752549d64
[L666] version_id: 1
[L667] name: SHIA_RAG_Complete_Report(5).md
[L668] mime_type: text/markdown
[L669] surface: library
[L670] score: 0.018057336133766127
[L671] document_chunk_id: file_000000003c8081fab658b1752549d64--1--63
[L672] snippet:
[L673] 5"""
[L674] "down": """ALTER TABLE knowledge_nodes
[L675] DROP COLUMN abstraction_level,
```

```

[L676] DROP COLUMN base_confidence,
[L677] DROP COLUMN extraction_confidence"""
[L678] },
[L679] {
[L680] "version": 3,
[L681] "description": "Migrate properties from string array to JSONB",
[L682] "up": """ALTER TABLE knowledge_nodes
[L683] ALTER COLUMN data TYPE JSONB USING data::JSONB""",
[L684] "down": "-- Manual rollback required"
[L685] }
[L686] ]
[L687]
[L688] def run_migrations(db, target_version=None):
[L689] current = db.get_migration_version()
[L690] for migration in MIGRATIONS:
[L691] if migration["version"] > current:
[L692] if target_version and migration["version"] > target_version:
[L693] break
[L694] db.execute(migration["up"])
[L695] db.set_migration_version(migration["version"])
[L696] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L697]

```

```

[L698]
[L699] ---
[L700]
[L701] ### 7.13 Error Handling & Retry Logic
[L702]
[L703]

```

python

```

[L704] import time
[L705] from functools import wraps
[L706]
[L707] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L708] """Decorator for retrying failed operations with exponential backoff."""
[L709] def decorator(func):
[L710] @wraps(func)
[L711] def wrapper(*args, **kwargs):
[L712] for attempt in range(max_retries):
[L713] try:
[L714] return func(*args, **kwargs)
[L715] except (TimeoutError, ConnectionError) as e:
[L716] if attempt == max_retries - 1:
[L717] raise
[L718] delay = min(base_delay * (2 ** attempt), max_delay)
[L719] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L720] time.sleep(delay)
[L721] return wrapper
[L722] return decorator
[L723]
[L724] class ProcessingPipeline:

```



```
[L725] """Error handling for the SHIA-RAG processing pipeline."""
[L726]
[L727] def process_document(self, doc):
[L728] # Layer 1: OCR with fallback
[L729] try:
[L730] text = self.extract_text(doc)
[L731] except OCRFailure:
[L732] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L733]
[L734] # Layer 3: LLM extraction with retry + fallback model
[L735] try:
[L736] concepts = self.llm_extract_concepts(text)
[L737] except LLMLLMTimeout:
[L738] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L739]
[L740] # Layer 6: Vector DB with cache fallback
[L741] try:
[L742] results = self.vector_search(query_embedding)
[L743] except VectorDBUnavailable:
[L744] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L745]
[L746] return {"status": "SUCCESS", "concepts": concepts}
[L747]
```

```
[L748]
[L749] ---
[L750]
[L751] ### 7.14 Caching Strategy
[L752]
[L753]
```

```
python
[L754] import redis
[L755] import hashlib
[L756] import json
[L757]
[L758] class CacheManager:
[L759] """Multi-layer caching for SHIA-RAG."""
[L760]
[L761] def __init__(self, redis_url="redis://localhost:6379"):
[L762] self.redis = redis.from_url(redis_url)
[L763] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L764] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L765] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L766]
[L767] def get_query_result(self, query_text):
[L768] """Check if this exact query has been answered recently."""
```

[L769] metadata:

[L770] query_result_index: 9

[L771] file_id: file_00000000383081fa8435ff2a7b4a6f5f

[L772] version_id: 1

[L773] name: Executive Summary (1)(5)(1).pdf

[L774] mime_type: application/pdf

[L775] surface: library

[L776] score: 0.018057336133766127

[L777] document_chunk_id: file_00000000383081fa8435ff2a7b4a6f5f:0--1--18

[L778] snippet:

[L779] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L780] Use knowledge bases: e.g. Wikipedia links.

[L781] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L782] Context Assembler may include one-hop neighbors via CLD).

[L783] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L784] additional edges are "related-to."

[L785] Data Structures and Indexing

[L786] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L787] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L788] embedding vector, and a version/timestamp.

[L789] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L790] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L791] NetworkX, Neo4j).

[L792] •

[L793] •

[L794] •

[L795] •

[L796] •

[L797] •

[L798] •

[L799] •

[L800] •

[L801] •

[L802] •

[L803] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L804] embeddings. This supports CPG++ search and semantic neighbor queries.

[L805] Ancillary Indexes:

[L806] Entity-to-Node map: For quick lookups by name/alias.

[L807] Relation Index: If needed, a mapping from relation type to node pairs.

[L808] Metadata store: (document IDs, source text) for evidence retrieval.

[L809] All indices are updated incrementally as nodes are added.

[L810] Retrieval and Context Assembly

[L811] 10. Query Analyzer

[L812] Input: Natural language user query.

[L813] Output: Query intent and candidate target concepts.

[L814] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L815] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L816] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L817] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L818] 11. Semantic Retrieval Engine

[L819] Input: Query intent, key terms.
[L820] Output: Set of relevant KnowledgeNodes.
[L821] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with
[L822] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).
[L823] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:
[L824] Parent Expansion: If the query is general, include some ancestors of the matched nodes.
[L825] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For
[L826] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related
[L827] knowledge.)
[L828] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's
[L829] semantic paths【29†L37-L44】).
[L830] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example
[L831] sentences, and optionally a context snippet from source documents.
[L832] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a
[L833] subset of these nodes and their evidence that maximizes informational coverage under a token limit.
[L834] This process yields a Context Graph of nodes and edges relevant to the query.

DEEP RESEARCH 24/08/2026, 01:42:44

Citation Marker: □filecite□turn24file0□

[L1] Find: find=0 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=## 10. Layer 7

[L2] Scope: full file surface=conversation

[L3] Matches: shown=0 total=0

[L4] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 24/08/2026, 01:42:44

Citation Marker: □filecite□turn24file1□

[L5] Find: find=1 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=### Layer 7

[L6] Scope: full file surface=conversation

[L7] Matches: shown=0 total=0

[L8] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 24/08/2026, 01:42:44

Citation Marker: □filecite□turn24file2□

[L9] Find: find=2 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=GenerationResult

[L10] Scope: full file surface=conversation

[L11] Matches: shown=4 total=4

[L12] [matches 0-1 | match_lines 234,235 | shown_lines 229-240]

[L13] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L14] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L15] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L16] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L17] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L18] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

```
[L19] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L20]
[L21] ---
[L22]
[L23] ## 4. Data Model — Layer 0
[L24] [match 2 | match_lines 1326 | shown_lines 1321-1331]
[L25]
[L26] ### 6.2 Layer 7: Generation Engine
[L27]
[L28] Responsibility: Build a prompt from retrieved context, invoke the LLM, extract claims, verify each claim against
evidence, bind citations, and compute answer confidence.
[L29]
[L30] Input: RetrievalResult → Output: GenerationResult
[L31]
[L32] ##### LLM Configuration
[L33]
[L34] | Parameter | Value | Rationale |
[L35] |-----|-----|-----|
[L36] [match 3 | match_lines 1408 | shown_lines 1403-1413]
[L37]
[L38] ##### Output Schema
[L39]
[L40] ```python
[L41] @dataclass
[L42] class GenerationResult:
[L43]     """Output of Layer 7 — ready for user and Layer 8."""
[L44]     answer: str # Answer text with citations
[L45]     claims: list # List of VerifiedClaim
[L46]     citations: list # Formatted citation list
[L47]     overall_confidence: float # 0.0-1.0
```

DEEP RESEARCH 24/08/2026, 01:42:44

Citation Marker: `filecite`turn24file3

```
[L48] Find: find=3 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=claim verification
[L49] Scope: full file surface=conversation
[L50] Matches: shown=4 total=4
[L51] [match 0 | match_lines 17 | shown_lines 12-22]
[L52]
[L53] We present SHIA-RAG (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer
RAG system that replaces flat chunks with a dynamically constructed Knowledge Forest — a collection of rooted trees where
each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A,
PART_OF) and semantic (USES, CAUSES) relationships.
[L54]
[L55] SHIA-RAG introduces five novel contributions: (1) Precedence-Constrained Tree Knapsack for context selection under
token budgets, ensuring hierarchical coherence; (2) Self-Evolving Knowledge Forest with a closed-loop feedback mechanism
that modifies edge weights based on retrieval success/failure; (3) Multi-Objective Energy Minimization for automated
hierarchy induction; (4) Adaptive Depth Traversal that adjusts retrieval depth based on query complexity; and (5) SHEF (SHIA-
RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG
quality.
```

[L56]

[L57] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L58]

[L59] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L60]

[L61] ---

[L62] [match 1 | match_lines 234 | shown_lines 229-239]

[L63] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L64] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L65] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L66] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L67] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L68] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L69] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L70]

[L71] ---

[L72]

[L73] ## 4. Data Model — Layer 0

[L74] [match 2 | match_lines 1357 | shown_lines 1352-1362]

[L75] "overall_confidence": 0.0-1.0,

[L76] "insufficient_evidence": false

[L77] } } ""

[L78]

[L79]

[L80] #### Claim Verification

[L81]

[L82] After the LLM generates an answer, each claim is verified against its cited evidence:

[L83]

[L84]

python

[L85] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:

[L86] [match 3 | match_lines 2033 | shown_lines 2028-2038]

[L87] - **DAG Support:** Allow multiple hierarchical parents per node (currently strict single-parent trees)

[L88] - **Multi-Tenancy:** Implement access control and document-level permissions

[L89] - **Exact Tree Knapsack DP:** Implement exact dynamic programming for small token budgets

[L90] - **Cross-Lingual Support:** Extend to multilingual documents and concept hierarchies

[L91] - **Incremental Updates:** Support adding new documents without full forest reconstruction

[L92] - **Advanced Verification:** Replace semantic similarity with NLI (Natural Language Inference) model for claim verification

[L93] - **Exploration in Self-Evolution:** Upgrade from ϵ -greedy ($\epsilon=0.1$, currently implemented) to Thompson sampling for more principled exploration-exploitation trade-off

[L94]

[L95] ---

[L96]

[L97] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0-8) are fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

DEEP RESEARCH 24/08/2026, 01:42:46

Citation Marker: [filecite]turn25file0[

[L1] metadata:
[L2] ref_id: turn23file0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] lines_shown: 1321-1500
[L7] total_file_lines: 2066
[L8] has_more: True
[L9] next_start_line: 1501
[L10] next_read: {"ref_id":"turn23file0","mode":"full","start_line":1501,"max_lines":180}
[L11] content:
[L12]
[L13] ### 6.2 Layer 7: Generation Engine
[L14]
[L15] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims, verify each claim against evidence, bind citations, and compute answer confidence.
[L16]
[L17] **Input:** RetrievalResult → **Output:** GenerationResult
[L18]
[L19] ##### LLM Configuration
[L20]
[L21] | Parameter | Value | Rationale |
[L22] |-----|-----|-----|
[L23] | Model | GPT-4o-mini (primary), Llama 3.1 8B (fallback) | Cost-effective with good quality |
[L24] | Temperature | 0.3 | Low for factual answers |
[L25] | Max output tokens | 2048 | Sufficient for detailed answers |
[L26] | Response format | JSON | Force structured output for claim extraction |
[L27] | Timeout | 30 seconds | With retry on timeout |
[L28]
[L29] ##### Prompt Template
[L30]
[L31]

```
[L32] SYSTEM_PROMPT = ""You are a knowledge-grounded question answering system.  
[L33] RULES:  
[L34] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.  
[L35] 2. For each claim, cite the source using [NODE_ID] format.  
[L36] 3. If evidence is insufficient, say "Insufficient evidence to answer."  
[L37] 4. If evidence conflicts, present both views with citations.  
[L38]  
[L39] REQUIRED OUTPUT FORMAT:  
[L40] {  
[L41]   "answer": "Your answer with [NODE_ID] citations inline.",  
[L42]   "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],  
[L43]   "overall_confidence": 0.0-1.0,  
[L44]   "insufficient_evidence": false  
[L45] }""  
[L46]
```

[L47]
[L48] ##### Claim Verification

[L49]

[L50] After the LLM generates an answer, each claim is verified against its cited evidence:

[L51]

[L52]

```

[L53] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L54]     """Returns (status, confidence).
[L55]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L56]     - PARTIAL:      evidence partially supports (0.4 < confidence ≤ 0.7)
[L57]     - UNSUPPORTED:  no evidence supports (confidence ≤ 0.4)
[L58]     - CONFLICTING:  evidence items contradict each other
[L59]     """
[L60]     scores = []
[L61]     for eid in evidence_ids:
[L62]         node = node_map.get(eid)
[L63]         if node and node.evidence:
[L64]             evidence_text = ' '.join(e.text for e in node.evidence)
[L65]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L66]
[L67]     if not scores:
[L68]         return 'UNSUPPORTED', 0.0
[L69]
[L70]     avg_score = sum(scores) / len(scores)
[L71]
[L72]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L73]         return 'CONFLICTING', avg_score
[L74]     if avg_score > 0.7:
[L75]         return 'SUPPORTED', avg_score
[L76]     elif avg_score > 0.4:
[L77]         return 'PARTIAL', avg_score
[L78]     else:
[L79]         return 'UNSUPPORTED', avg_score
[L80]

```

[L81]

[L82] ##### Unsupported Claim Policy

[L83]

[L84] | Status | Action |

[L85] |-----|-----|

[L86] | SUPPORTED | Keep as-is with citation |

[L87] | PARTIAL | Keep but mark as partially supported |

[L88] | UNSUPPORTED | Remove from answer, note at end |

[L89] | CONFLICTING | Present both views with note |

[L90]

[L91] ##### Answer Confidence Formula

[L92]

[L93]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{node_conf}}{\frac{\text{cited nodes}}{\text{selected nodes}}}$$

[L94]

[L95] ##### Output Schema

[L96]

[L97]

```
[L98] @dataclass
[L99] class GenerationResult:
[L100]     """Output of Layer 7 – ready for user and Layer 8."""
[L101]     answer: str                # Answer text with citations
[L102]     claims: list              # List of VerifiedClaim
[L103]     citations: list          # Formatted citation list
[L104]     overall_confidence: float # 0.0-1.0
[L105]     retrieval_result: object  # Preserved for Layer 8
[L106]     generation_metadata: dict # Model used, claim counts, etc.
[L107]
```

[L108]

[L109] ---

[L110]

[L111] ## 7. Operations Layer — Layer 8

[L112]

[L113] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L114]

[L115] ### 7.1 Logging

[L116]

[L117] Every query processed through the system generates a structured `QueryLog` entry:

[L118]

[L119]

```
[L120] @dataclass
[L121] class QueryLog:
[L122]     event_id: str
[L123]     timestamp: str
[L124]     query: str
[L125]     intent: str                # DEFINITION, COMPARISON, etc.
[L126]     complexity: str          # SIMPLE, MODERATE, COMPLEX
[L127]     seed_nodes: list
[L128]     candidate_count: int
[L129]     selected_count: int
[L130]     traversal_edges: list      # For feedback attribution
[L131]     answer_length: int
[L132]     claim_count: int
[L133]     supported_claims: int
[L134]     unsupported_claims: int
[L135]     overall_confidence: float
[L136]     model_used: str
[L137]     latency_ms: float
[L138]     user_feedback: str = None # HELPFUL / NOT_HELPFUL (filled later)
[L139]
```

[L140]

[L141] ### 7.2 Feedback Acquisition & Attribution

[L142]

[L143] User feedback is collected via API:

[L144]

[L145]

```
[L146] POST /api/feedback { "event_id": "...", "feedback": "HELPFUL" | "NOT_HELPFUL" }
[L147]
```

[L148]

[L149] **Attribution strategy:** Feedback applies ONLY to edges in the recorded `traversal_path`. This isolates retrieval quality

from generation quality.

[L150]

[L151] **Safeguards:**

[L152] - Edge weight updates use a dampened learning rate ($\eta=0.02$, small increments)

[L153] - Layer 7 verification metrics (supported/unsupported claims) are tracked separately

[L154] - Anomaly detection: if `confidence > 0.8 AND feedback == "NOT_HELPFUL"`, log a warning (possible answer format issue, not retrieval issue)

[L155]

[L156] **7.3 Self-Evolution Update Rule**

[L157]

[L158] After each query with feedback signal $f \in \{+1, -1\}$:

[L159]

[L160] $w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f) \quad \forall e \in \text{traversal_path}$

[L161]

[L162] Where $\eta = 0.02$. Edge weights are clipped to $[0.1, 10.0]$.

[L163]

[L164]

```
[L165] import math
[L166] import random
[L167]
[L168] ETA = 0.02
[L169] WEIGHT_MIN, WEIGHT_MAX = 0.1, 10.0
[L170] RESTRUCTURE_THRESHOLD = 0.3
[L171] EPSILON = 0.1 # ε-greedy exploration rate
[L172]
[L173] def update_forest_from_feedback(selected_nodes, user_feedback, forest):
[L174]     f = +1 if user_feedback == "HELPFUL" else -1
[L175]     path_edges = get_traversal_path_edges(selected_nodes, forest)
[L176]
[L177]     for edge in path_edges:
[L178]         edge.weight = edge.weight * math.exp(ETA * f)
[L179]         edge.weight = max(WEIGHT_MIN, min(edge.weight, WEIGHT_MAX))
[L180]         edge.access_count += 1
[L181]
[L182]         if f == -1 and edge.weight < RESTRUCTURE_THRESHOLD:
[L183]             flag_for_restructure(edge, forest)
[L184]
[L185]     if forest.total_queries % 100 == 0:
[L186]         rebalance_forest(forest)
[L187]     forest.total_queries += 1
[L188]
```

[L189]

[L190] **ε-Greedy Exploration:** To prevent the self-evolution loop from reinforcing wrong paths (e.g., an incorrect path gets positive feedback because the LLM produces a convincing-looking answer), retrieval includes ε-greedy exploration:

DEEP RESEARCH 24/08/2026, 01:42:49

Citation Marker: `filecite[turn26file0]`

[L1] Find: `find=0 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=def generate_answer`

[L2] Scope: `full file surface=conversation`

[L3] Matches: `shown=0 total=0`

[L4] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 24/08/2026, 01:42:49

Citation Marker: `filecite`turn26file1

[L5] Find: find=1 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=Prompt assembly
[L6] Scope: full file surface=conversation
[L7] Matches: shown=2 total=2
[L8] [match 0 | match_lines 188 | shown_lines 183-193]
[L9] end
[L10]
[L11] subgraph "Phase 2: Online — Retrieval & Generation"
[L12] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RBO"]
[L13] KF --> L6
[L14] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L15] L7 --> A["Answer + Citations"]
[L16] end
[L17]
[L18] subgraph "Phase 3: Continuous — Operations"
[L19] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L20] [match 1 | match_lines 234 | shown_lines 229-239]
[L21] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L22] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L23] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L24] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L25] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L26] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L27] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L28]
[L29] ---
[L30]
[L31] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:42:49

Citation Marker: `filecite`turn26file2

[L32] Find: find=2 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=bind citations
[L33] Scope: full file surface=conversation
[L34] Matches: shown=2 total=2
[L35] [match 0 | match_lines 218 | shown_lines 213-223]
[L36] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L37] Knowledge Forest (rooted trees + cross-links)
[L38] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L39] Selected context nodes (within token budget)
[L40] ↓ Layer 7: Build prompt, invoke LLM
[L41] ↓ Layer 7: Verify claims against evidence, bind citations
[L42] Answer with citations + confidence score
[L43] ↓ Layer 8: Log, collect feedback, update forest weights
[L44] ``
[L45]
[L46] ### 3.3 Layer Summary
[L47] [match 1 | match_lines 1324 | shown_lines 1319-1329]
[L48]

[L49] ---

[L50]

[L51] ### 6.2 Layer 7: Generation Engine

[L52]

[L53] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims, verify each claim against evidence, bind citations, and compute answer confidence.

[L54]

[L55] **Input:** RetrievalResult → **Output:** GenerationResult`

[L56]

[L57] ##### LLM Configuration

DEEP RESEARCH 24/08/2026, 01:42:49

Citation Marker: `[]filecite[]turn26file3[]`

[L58] Find: find=3 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=retry on timeout

[L59] Scope: full file surface=conversation

[L60] Matches: shown=1 total=1

[L61] [match 0 | match_lines 1336 | shown_lines 1331-1341]

[L62] |-----|-----|-----|

[L63] | Model | GPT-4o-mini (primary), Llama 3.1 8B (fallback) | Cost-effective with good quality |

[L64] | Temperature | 0.3 | Low for factual answers |

[L65] | Max output tokens | 2048 | Sufficient for detailed answers |

[L66] | Response format | JSON | Force structured output for claim extraction |

[L67] | Timeout | 30 seconds | With retry on timeout |

[L68]

[L69] ##### Prompt Template

[L70]

[L71] ```python

[L72] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.

DEEP RESEARCH 24/08/2026, 01:42:49

Citation Marker: `[]filecite[]turn26file4[]`

[L73] Find: find=4 ref=turn23file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=insufficient_evidence

[L74] Scope: full file surface=conversation

[L75] Matches: shown=1 total=1

[L76] [match 0 | match_lines 1353 | shown_lines 1348-1358]

[L77] REQUIRED OUTPUT FORMAT:

[L78] {

[L79] "answer": "Your answer with [NODE_ID] citations inline.",

[L80] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],

[L81] "overall_confidence": 0.0-1.0,

[L82] "insufficient_evidence": false

[L83] }"""

[L84] ```

[L85]

[L86] ##### Claim Verification

CHATGPT 24/08/2026, 01:42:49

Bilkul. Ab hum **Layer 7 – Generation Engine** ko **Layer 0–6** ki tarah **same depth** mein samjhenge.

Hum **same running example** use karenge:

User Query: "Why is Time Quantum important in Round Robin?"

Yaad rakho:

- **Layer 6 ka kaam:** relevant knowledge retrieve karna
- **Layer 7 ka kaam:** us retrieved knowledge se **grounded answer generate karna, claims verify karna, citations bind karna, aur confidence calculate karna**

File ke according Layer 7 ka official flow hai: **Prompt Assembly → LLM → Claim Verification → Citation Binding → Answer Confidence**. [filecite\turn25file0\L13-L17](#)

1. Layer 7 ko ek line mein samjho

*Layer 7 receives the **RetrievalResult** from Layer 6, converts the retrieved knowledge into a structured prompt, generates an answer using an LLM, verifies every factual claim against the original evidence, attaches citations, and calculates the confidence of the final answer.* [filecite\turn25file0\L13-L17](#)

Simple words mein:

```
Layer 6:
"What knowledge is relevant?"
      ↓
Layer 7:
"Use only that knowledge to generate a trustworthy answer."
```

2. Layer 6 aur Layer 7 mein exact difference

Ye pehle clear karna bahut important hai.

| Layer 6 – Retrieval | Layer 7 – Generation |

|---|---|

| Relevant knowledge dhundta hai | Retrieved knowledge se answer banata hai |

| Knowledge Forest traverse karta hai | LLM ko structured context deta hai |

| Tree Knapsack use karta hai | Prompt assembly karta hai |

| Best context select karta hai | Claims verify karta hai |

| **RetrievalResult** output deta hai | **GenerationResult** output deta hai |

```
User Query
      ↓
Layer 6
      ↓
Relevant Knowledge
      ↓
Layer 7
      ↓
Verified Answer + Citations + Confidence
```

Source architecture bhi Layer 6 ke selected context nodes ko Layer 7 mein **prompt building and LLM invocation**, followed by **claim verification and citation binding** ke through answer mein convert karta hai. [filecite\turn23file0\L49-L56](#)

3. Layer 7 ka exact input kya hota hai?

Input: **RetrievalResult**

Layer 7 directly Knowledge Forest mein search start nahi karta.

Usse Layer 6 se already selected information milti hai.

Conceptually:

```
RetrievalResult
├── Selected Nodes
├── Context Text
├── Traversal Path
├── Query Intent
├── Query Complexity
├── Seed Nodes
└── Token Information
```

Layer 7 ka official input `RetrievalResult` aur output `GenerationResult` hai. `filecite\turn25file0\L13-L17`

4. Hamare example mein Layer 6 se kya aayega?

User ne poocha:

"Why is Time Quantum important in Round Robin?"

Layer 6 ne retrieve kiya:

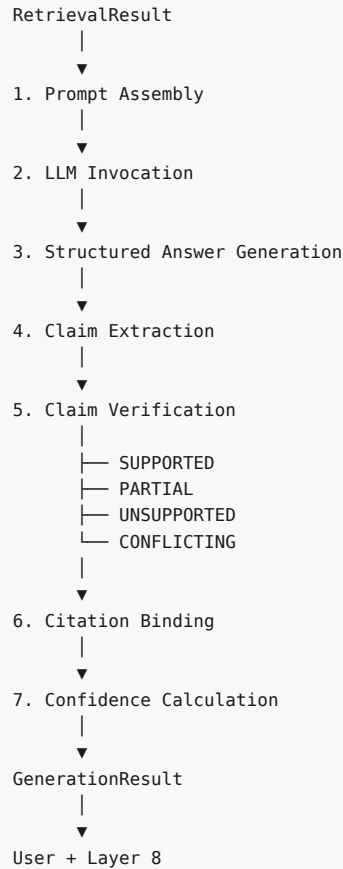
```
Selected Nodes
├── Node 1:
│   └── CPU Scheduling Algorithm
├── Node 2:
│   └── Round Robin
└── Node 3:
    └── Time Quantum
```

Aur context kuch logically aisa hoga:

```
CPU Scheduling Algorithm
    ↓
Round Robin
    ↓
USES
    ↓
Time Quantum
```

Ab Layer 7 ka kaam start hota hai.

5. Layer 7 ka complete pipeline



Yahi Layer 7 ki main responsibility hai: prompt build karna, LLM invoke karna, claims verify karna, citations bind karna, aur answer confidence compute karna. [filecite\turn25file0\L13-L17](#)

6. Step 1 — Prompt Assembly

Ab Layer 7 ko LLM ke liye ek proper prompt banana hai.

Question:

LLM ko directly raw database objects kyu nahi dete?

Suppose Layer 6 se ye objects aaye:

```
Node_ID: N_123
Name: Round Robin
Confidence: 0.92
Evidence: [...]
Relations: [...]
Metadata: {...}
```

Raw object directly LLM ko dena messy ho sakta hai.

Isliye Layer 7 relevant information ko organize karta hai:

USER QUESTION:
Why is Time Quantum important in Round Robin?

RETRIEVED EVIDENCE:

[N1] CPU Scheduling Algorithm
Definition: ...

[N2] Round Robin
Definition: ...
Properties: ...
Evidence: ...

[N3] Time Quantum
Definition: ...
Evidence: ...

Then LLM ko instructions di jaati hain:

Answer only from the provided evidence.
Do not use prior knowledge.
Cite every claim using NODE_ID.
If evidence is insufficient, say so.
If evidence conflicts, present both views.

Ye exact grounding rules source prompt template mein defined hain. `filecite\turn25file0\L29-L46`

7. “Answer ONLY from provided evidence” kyu important hai?

Ye Layer 7 ka **major safety mechanism** hai.

Normally LLM aisa kar sakta hai:

Retrieved Context
+
LLM's Prior Knowledge
↓
Answer

Problem:

LLM apni general knowledge se extra information add kar sakta hai.

Isse **hallucination** ka risk badhta hai.

SHIA-RAG ke Layer 7 mein intended rule:

Retrieved Evidence
↓
Generate Answer ONLY from this
↓
No unsupported external claim

Source explicitly instructs the LLM to answer only from provided evidence and not use prior knowledge.

`filecite\turn25file0\L31-L45`

8. Step 2 — LLM Invocation

Ab prepared prompt LLM ko diya jaata hai.

Report mein configuration:

```
Primary Model:
GPT-4o-mini

Fallback Model:
Llama 3.1 8B

Temperature:
0.3

Max Output Tokens:
2048

Response Format:
JSON

Timeout:
30 seconds with retry
```

filecite\turn25file0\L19-L27

Temperature = 0.3 kyu?

Factual question answering mein:

```
High Temperature
↓
More randomness / creativity
```

But SHIA-RAG ko primarily chahiye:

```
Evidence-grounded
+
Consistent
+
Factual answer
```

Isliye source configuration mein **0.3 low temperature** rakha gaya hai. filecite\turn25file0\L21-L27

9. LLM ka output sirf normal text kyu nahi?

System LLM se structured JSON output expect karta hai.

Conceptually:

```
{
  "answer": "...",
  "claims": [
    {
      "text": "...",
      "evidence_ids": ["N1"],
      "confidence": 0.95
    }
  ],
  "overall_confidence": 0.90,
  "insufficient_evidence": false
}
```

Iska reason:

Layer 7 ko baad mein *har individual claim verify karna hai*.

Agar sirf ek paragraph milta:

```
Round Robin uses a Time Quantum...
```


toh system ko automatically identify karna difficult ho sakta hai ki:

```
Which factual claim?
  ↓
Which evidence supports it?
  ↓
How confident is the system?
```

Structured output claim-level verification ko possible banata hai. Source prompt required output format mein answer, claims, evidence IDs, overall confidence, aur insufficient-evidence flag define karta hai. `[]filecite[]turn25file0[]L39-L46[]`

10. Step 3 — Answer Generation

Ab LLM answer generate karta hai.

Hamare example mein:

Why is Time Quantum important in Round Robin?

Possible generated answer structure:

```
Answer:

Time Quantum is important in Round Robin because
the algorithm allocates a fixed time interval to
each process. The selected evidence connects
Round Robin with the use of Time Quantum.
[N2][N3]
```

⚠ Important:

Ye answer abhi final nahi hai.

Kyunki:

LLM ne answer generate kiya hai, lekin system ko ab verify karna hai ki har factual claim actual evidence se supported hai ya nahi.

11. Step 4 — Claim Extraction

Maan lo LLM ne answer diya:

"Round Robin uses a fixed Time Quantum. The Time Quantum determines how long a process receives CPU time."

Is answer ko Layer 7 claim-level units mein break karta hai.

```
Claim 1:
"Round Robin uses a fixed Time Quantum."

Claim 2:
"Time Quantum determines how long a process
receives CPU time."
```

Phir har claim ke saath:

```
Claim
+
Evidence IDs
+
Claim Confidence
```

store kiya jaata hai.

Source output format har factual claim ke liye `text` , `evidence_ids` , aur confidence define karta hai.

`filecite\turn25file0\L39-L45`

12. Step 5 — Claim Verification

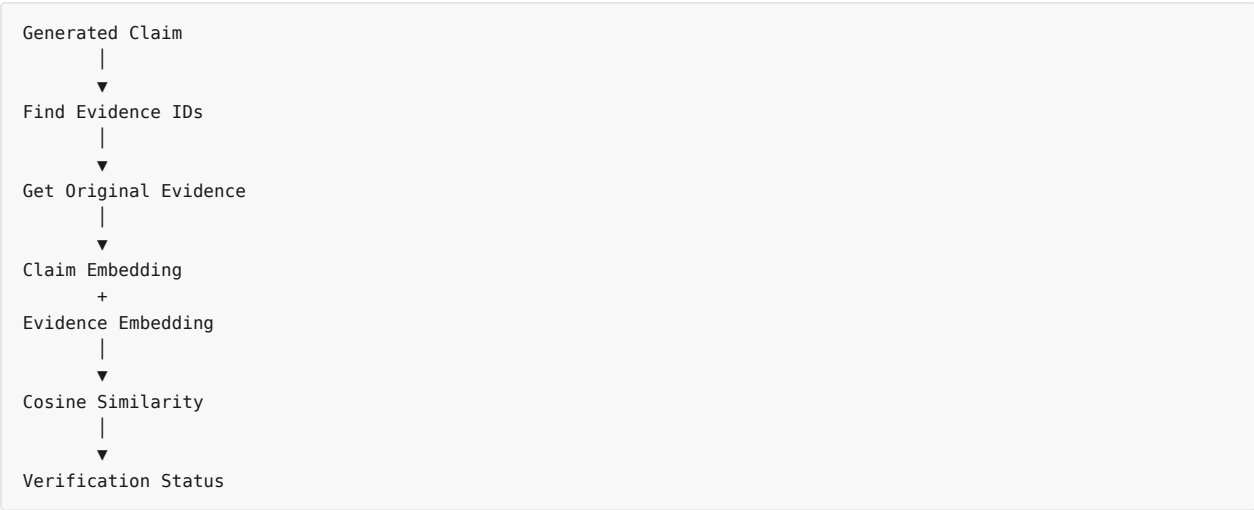
Ye Layer 7 ka most important part hai.

Ab system check karta hai:

“Jo LLM ne bola hai, kya actual retrieved evidence usse support karta hai?”

Verification function claim aur uske evidence nodes ko compare karta hai. Source implementation claim text aur evidence text ke embeddings ka cosine similarity use karta hai. `filecite\turn25file0\L48-L80`

Complete verification flow



Source-defined statuses hain:

SUPPORTED
PARTIAL
UNSUPPORTED
CONFLICTING

`filecite\turn25file0\L48-L80`

13. Verification Status ko deeply samjho

A. SUPPORTED

Claim:
"Round Robin uses a Time Quantum."

Evidence:
"Round Robin uses a fixed time quantum."

Similarity:
High

Result:

SUPPORTED ✓

Source threshold:

Confidence > 0.7

filecite\turn25file0\L53-L79

B. PARTIAL

Suppose LLM bolta hai:

"Time Quantum improves overall CPU efficiency."

But evidence sirf partially related hai.

Result:

PARTIAL

Source configuration mein average verification confidence $0.4 < \text{score} \leq 0.7$ hone par claim PARTIAL hota hai.

filecite\turn25file0\L53-L79

C. UNSUPPORTED

Suppose retrieved evidence mein kahi nahi likha:

"Round Robin always gives the fastest response time."

But LLM ne ye claim bana diya.

System:

Claim
↓
Search cited evidence
↓
No support found
↓
UNSUPPORTED ✕

Source threshold mein ≤ 0.4 unsupported category mein aata hai, aur no usable evidence par confidence 0.0 return hota hai. filecite\turn25file0\L53-L79

D. CONFLICTING

Suppose do evidence items:

Evidence A:
Statement X is true.

Evidence B:
Statement X is false.

System simply ek side choose nahi karta.

Instead:

CONFLICTING

Aur final answer mein:

Both views can be presented with citations.

Source conflict rule evidence-score variation ke basis par conflict detect karta hai, aur conflicting claims ke liye both views present karne ka policy define karta hai. `filecite\turn25file0\L70-L89`

14. Claim verification ke baad kya action hota hai?

| Status | Layer 7 Action |

|---|---|

| **SUPPORTED** | Claim ko citation ke saath keep karo |

| **PARTIAL** | Claim ko partially supported mark karke keep karo |

| **UNSUPPORTED** | Claim remove karo |

| **CONFLICTING** | Dono views present karo |

Ye unsupported-claim policy source mein explicitly defined hai. `filecite\turn25file0\L82-L89`

15. Hamare example ko claim-by-claim verify karte hain

Generated answer:

"Round Robin uses a fixed Time Quantum. The Time Quantum controls the duration for which a process receives CPU execution."

Claim 1

Claim:
Round Robin uses a fixed Time Quantum.

Evidence:
Round Robin node + evidence

Result:
SUPPORTED ✓

Claim 2

Claim:
Time Quantum controls the duration of CPU execution.

Evidence:
Time Quantum node

Result:
SUPPORTED / PARTIAL

Exact result retrieved evidence ki wording aur verification score par depend karega.

Agar LLM ek extra claim add kare:

Claim:
Round Robin is always the best scheduling algorithm.

But evidence nahi hai.

Result:

UNSUPPORTED ✕
↓
Remove from final answer

Yahi Layer 7 ko normal "LLM answer generator" se stronger banata hai.

16. Step 6 — Citation Binding

Ab verified claims ke saath citations bind ki jaati hain.

Before:

```
Round Robin uses a Time Quantum.
```

After:

```
Round Robin uses a Time Quantum. [NODE_45]
```

Final answer ka har factual part ideally traceable hota hai:

```
Answer
├── Claim 1 → Node A
├── Claim 2 → Node B
└── Claim 3 → Node C
```

System ka official output goal **Answer + Citations + Confidence Score** hai. `filecite\turn23file0\L51-L56`

17. Citation binding ki zarurat kyu padi?

Simple answer:

Because we should be able to answer: "System ne ye statement kis evidence ke basis par diya?"

Without citation:

```
LLM says something
↓
We don't know its source
```

With citation:

```
LLM Claim
↓
NODE_ID
↓
Original Evidence
↓
Source Document
```

Isse answer **traceable** aur **auditable** banta hai.

18. Step 7 — Overall Answer Confidence

Ab har claim verify ho chuka hai.

Lekin user ko overall answer ke liye ek confidence chahiye.

Source formula:

```
Overall Confidence
=
0.4 × Supported Claim Ratio
+
0.3 × Average Claim Confidence
+
0.2 × Average Node Confidence
+
0.1 × Citation Coverage
```

fileciteurn25file0L91-L94

19. Is formula ko simple language mein samjho

Factor 1 — Supported Claims

Weight:

40%

Question:

Kitne generated claims actually evidence se supported hain?

Example:

```
Total Claims = 10
Supported = 9

Supported Ratio = 0.9
```

Ye answer reliability ka biggest factor hai.

Factor 2 — Average Claim Confidence

Weight:

30%

Question:

Har individual claim kitni strongly evidence se supported hai?

Factor 3 — Average Node Confidence

Weight:

20%

Question:

Layer 6 ne jin KnowledgeNodes ko retrieve kiya tha, woh khud kitne reliable/confident the?

Yahan ek important cross-layer connection hai:

Layer 4

↓

Node/Edge Confidence

Layer 6

↓

Selects Relevant Nodes

Layer 7

↓

Uses Node Confidence in Answer Confidence

Factor 4 — Citation Coverage

Weight:

10%

Question:

Selected nodes mein se kitne actually final answer ke citations mein used hue?

Source formula cited nodes aur selected nodes ka ratio include karta hai. `filecite\turn25file0\L91-L94`

20. Ek illustrative confidence example

Suppose:

Supported Claim Ratio = 0.90
Average Claim Confidence = 0.85
Average Node Confidence = 0.80
Citation Coverage = 1.00

Then conceptually:

Overall Confidence
=
0.4(0.90)
+
0.3(0.85)
+
0.2(0.80)
+
0.1(1.00)

=
0.36
+
0.255
+
0.16
+
0.10

= 0.875

So:

Overall Confidence = 0.875

Yaani roughly:

87.5%

⚠️ Ye calculation sirf formula samjhane ke liye example hai; actual values actual claim and node verification se aayengi.
Formula source report mein defined hai. `filecite\turn25file0\L91-L94`

21. Final Output — `GenerationResult`

Layer 7 ka final output:

`GenerationResult`

Source schema:

```
GenerationResult
|
├─ answer
├─ claims
├─ citations
├─ overall_confidence
├─ retrieval_result
└─ generation_metadata
```

`GenerationResult` mein answer with citations, verified claims, formatted citations, overall confidence, preserved retrieval result, aur generation metadata defined hain. `filecite\turn25file0\L95-L107`

22. Har field ko samjho

`answer`

User ko dikhne wala final answer.

```
"Time Quantum is important because..."
```

`claims`

Answer ke individual factual claims.

```
Claim 1
Claim 2
Claim 3
```

`citations`

Har claim ka supporting source/evidence.

```
Claim 1 → NODE_12
Claim 2 → NODE_25
```

`overall_confidence`

Answer ki overall reliability.

```
0.0 → Low confidence
1.0 → High confidence
```

`retrieval_result`

Layer 6 ka result preserve kiya jaata hai.

Kyun?

Because Layer 8 ko baad mein samajhna hoga:

```
Which nodes were retrieved?  
Which path was traversed?  
How many claims were generated?  
Which claims were supported?
```

Source schema explicitly `retrieval_result` ko Layer 8 ke liye preserve karta hai. `filecite[turn25file0]L99-L107`

generation_metadata

Generation ke technical details:

```
Model used  
Claim counts  
etc.
```

Source schema is metadata ko explicitly include karta hai. `filecite[turn25file0]L99-L107`

23. Complete example: Start to End

User Query

```
Why is Time Quantum important in Round Robin?
```

Layer 6 Output

```
RetrievalResult  
├── CPU Scheduling Algorithm  
├── Round Robin  
├── Time Quantum  
├── Relevant Evidence  
└── Traversal Path
```

Layer 7 – Step 1

Prompt Assembly

```
QUESTION:  
Why is Time Quantum important in Round Robin?  
  
EVIDENCE:  
[N1] CPU Scheduling Algorithm  
[N2] Round Robin  
[N3] Time Quantum  
  
RULES:  
- Use only provided evidence  
- Cite every claim  
- Do not use unsupported knowledge  
- Report insufficient evidence if needed
```

Step 2

LLM Generation

Draft Answer:

"Round Robin uses a fixed Time Quantum.
The Time Quantum is therefore a relevant
part of how the algorithm operates."

Step 3

Claim Extraction

Claim 1:
Round Robin uses a fixed Time Quantum.

Claim 2:
Time Quantum is part of how
Round Robin operates.

Step 4

Claim Verification

Claim 1
↓
Check evidence
↓
SUPPORTED ✓

Claim 2
↓
Check evidence
↓
SUPPORTED / PARTIAL

Step 5

Citation Binding

Claim 1 → [RoundRobin_Node]

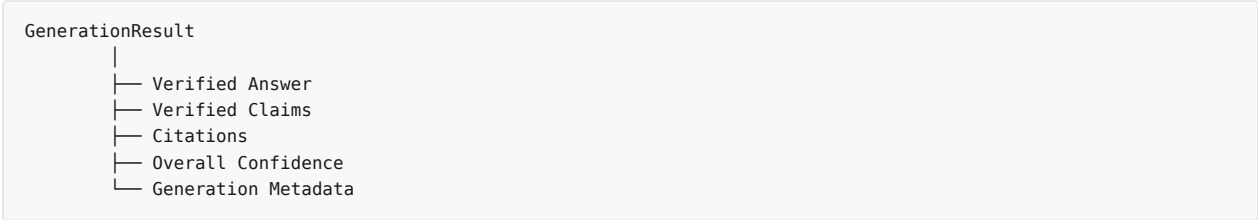
Claim 2 → [TimeQuantum_Node]

Step 6

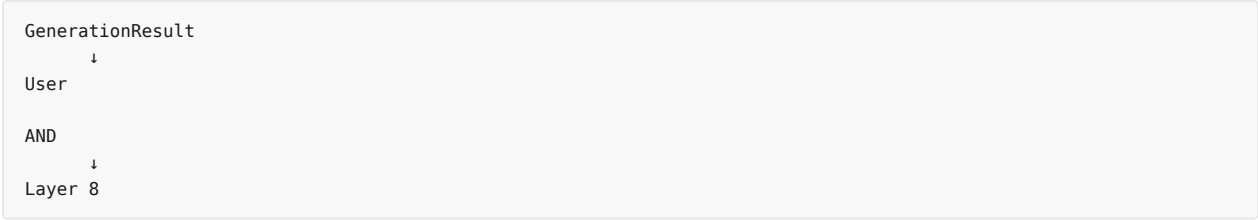
Confidence Calculation

Supported Claims
+
Claim Confidence
+
Node Confidence
+
Citation Coverage
↓
Overall Confidence

Final Output



Then:



Layer 8 later logs generation information and tracks supported/unsupported claims separately from retrieval feedback.

filecite[turn25file0L111-L154]

24. Layer 7 mein technologies kya use ho rahi hain?

- | Component | Technology / Method | Kaam |
- |---|---
- | Prompt Assembly | Prompt Engineering | Retrieved context ko LLM instructions mein organize karna |
- | Generation | Primary LLM | Answer generate karna |
- | Fallback | Llama 3.1 8B | Primary model failure/fallback |
- | Structured Output | JSON | Claims ko machine-readable format mein lena |
- | Claim Verification | Embeddings + Cosine Similarity | Claim aur evidence compare karna |
- | Citation Binding | NODE_ID Mapping | Claim ko evidence se connect karna |
- | Confidence Scoring | Weighted Formula | Overall answer reliability calculate karna |
- | Retry | Timeout retry | LLM timeout handle karna |

Source LLM configuration, structured JSON response, claim-verification algorithm, aur confidence formula specify karta hai.

filecite[turn25file0L19-L27] filecite[turn25file0L48-L107]

25. Layer 7 mein possible errors aur safeguards

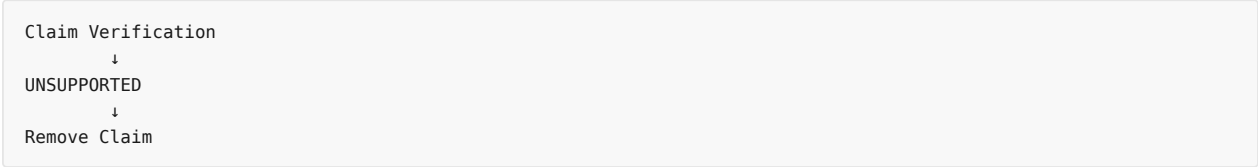
Error 1 — LLM hallucination

LLM bolta hai:

```
"Round Robin is always the fastest algorithm."
```

Evidence mein nahi hai.

Safeguard



filecite[turn25file0L82-L89]

Error 2 — Evidence insufficient hai

Suppose user ka question:

```
"What is the exact performance of Round Robin?"
```

But retrieved evidence mein answer hi nahi hai.

Safeguard

LLM ko instruction hai:

```
"Insufficient evidence to answer."
```

Source prompt explicitly insufficient evidence case define karta hai. `filecite\turn25file0\L31-L45`

Error 3 — Do evidence sources conflict karte hain

```
Evidence A → X  
Evidence B → Not X
```

Safeguard

```
CONFLICTING  
↓  
Present Both Views  
+  
Citations
```

`filecite\turn25file0\L72-L89`

Error 4 — LLM timeout

Source configuration:

```
Timeout = 30 seconds  
↓  
Retry
```

Aur configuration mein fallback model bhi specified hai. `filecite\turn25file0\L19-L27`

Error 5 — Claim ke paas evidence ID nahi hai

```
Claim:  
Something is true.  
  
Evidence IDs:  
[]
```

Verification:

```
No evidence  
↓  
UNSUPPORTED  
↓  
Confidence = 0.0
```

Source verification function exactly no verification scores hone par unsupported status return karta hai.

`filecite\turn25file0\L60-L79`

26. Layer 7 ka sabse important idea

Normal RAG:

```
Retrieved Context
↓
LLM
↓
Answer
```

SHIA-RAG Layer 7:

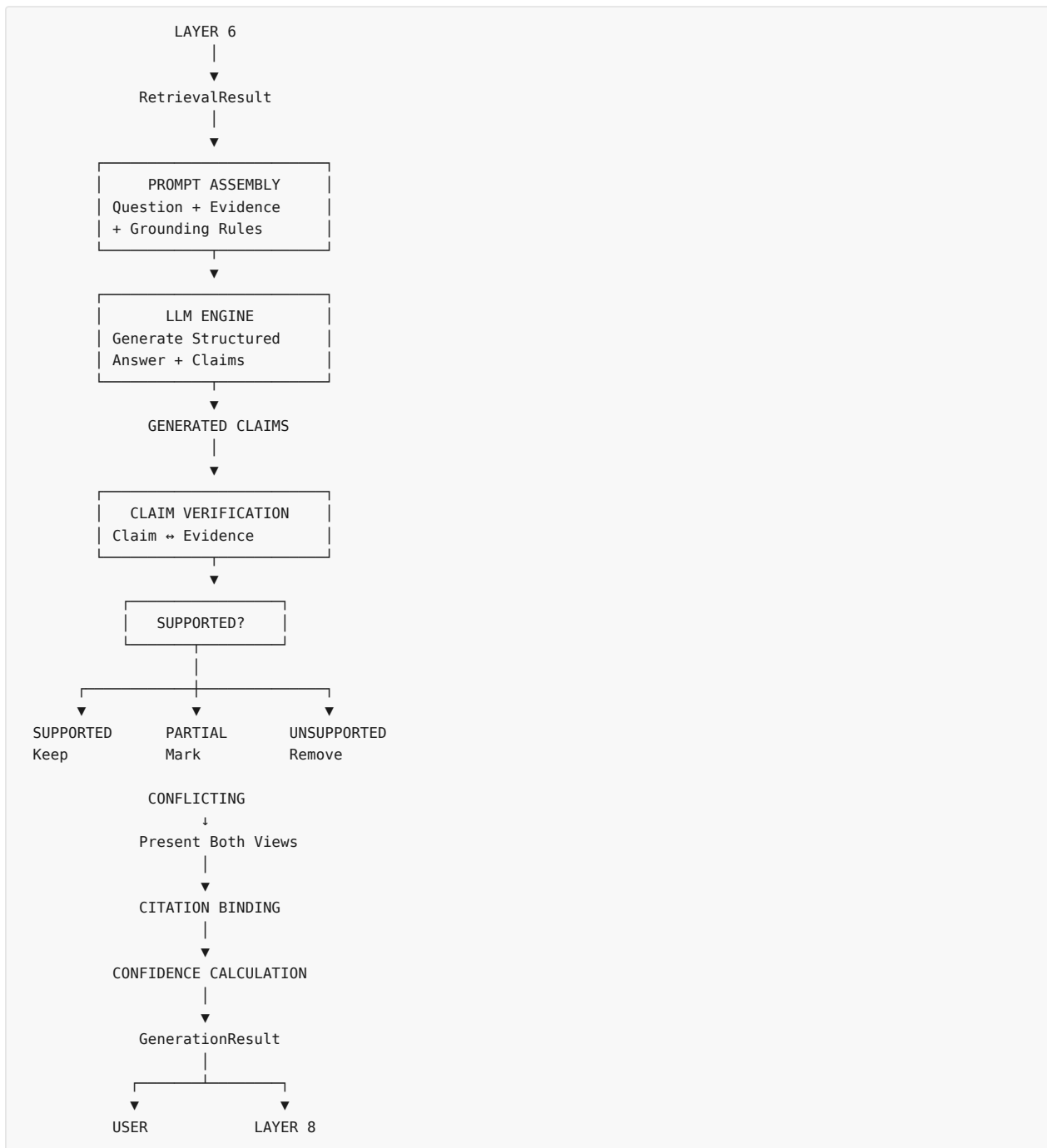
```
Retrieved Context
↓
Structured Prompt
↓
Grounded LLM Generation
↓
Extract Claims
↓
Verify Each Claim
↓
Remove Unsupported Claims
↓
Handle Conflicts
↓
Bind Citations
↓
Calculate Confidence
↓
Verified Answer
```

Main difference ye hai ki Layer 7 generation ke baad answer ko blindly accept nahi karta.

It performs a post-generation verification step.

Ye source report mein Layer 7 ki core responsibility hai. `filecite[turn25file0][L13-L17]`

27. Layer 7 ka complete architecture



28. Layer 7 ka input-output contract

Input

RetrievalResult

Containing:

Relevant Context

+

Selected Nodes

+

Evidence

+

Node IDs

+

Retrieval Information

Processing

Prompt Assembly

↓

LLM Generation

↓

Claim Extraction

↓

Claim Verification

↓

Citation Binding

↓

Confidence Calculation

Output

GenerationResult

Containing:

Verified Answer

+

Verified Claims

+

Citations

+

Overall Confidence

+

Retrieval Result

+

Generation Metadata

Source Layer Summary exactly defines Layer 7 as `RetrievalResult → GenerationResult` , with **Prompt assembly** and **Claim verification** as its key algorithms/processes. `filecite[turn23file0]L59-L71`

29. Sabse simple real-world analogy

Maan lo:

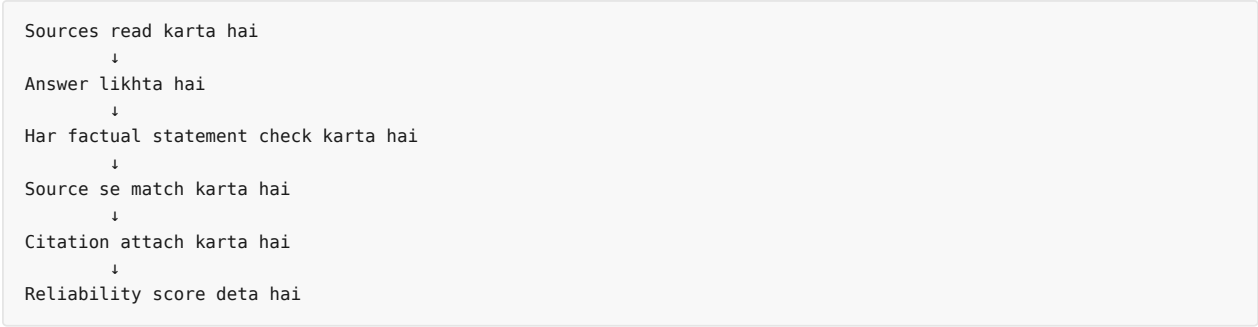
Layer 6 = Research Assistant

Woh tumhare liye books aur relevant pages lekar aata hai.

"Sir, ye 3 sources relevant hain."

Layer 7 = Expert Writer + Fact Checker

Woh:



So:

Layer 6 finds the evidence; Layer 7 turns that evidence into a verified answer.

30. Final Summary Table

Point Layer 7
--- ---
Name Generation Engine
Input RetrievalResult
Main Goal Evidence-grounded answer generate karna
Step 1 Prompt Assembly
Step 2 LLM Invocation
Step 3 Structured Answer + Claim Generation
Step 4 Claim Verification
Verification Method Evidence comparison using embeddings + cosine similarity
Statuses Supported, Partial, Unsupported, Conflicting
Unsupported Claim Remove
Conflicting Evidence Present both views
Citation NODE_ID-based binding
Confidence Weighted answer confidence formula
Output GenerationResult
Next Layer Layer 8 – Operations

31. Guide ko samjhane ke liye sabse important one-line answer

“Layer 7 is the Generation and Verification Engine of SHIA-RAG. It receives the retrieved context from Layer 6, builds a grounded prompt, generates an answer using an LLM, verifies every factual claim against the original evidence, removes unsupported claims, handles conflicting evidence, attaches citations, and calculates the overall confidence before producing the final answer.”

Complete flow yaad rakho:


```
Layer 6 RetrievalResult
  ↓
Build Grounded Prompt
  ↓
LLM Generates Answer
  ↓
Extract Claims
  ↓
Verify Claims with Evidence
  ↓
Remove Unsupported Claims
  ↓
Handle Conflicts
  ↓
Bind Citations
  ↓
Calculate Confidence
  ↓
GenerationResult
  ↓
User + Layer 8
```

Layer 7 ka core philosophy:

“Do not trust the generated answer blindly — generate first, then verify every claim against evidence.”

YOU 24/08/2026, 01:47:11

mujhe layer 7 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 7 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 24/08/2026, 01:47:19

```
{"size":"1024x1024","n":1,"transparent_background":false,"is_style_transfer":false,"prompt":null,"referenced_image_ids":null}
```

YOU 24/08/2026, 01:50:30

mujhe layer 7 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 7 ko english me btao lekin brife me

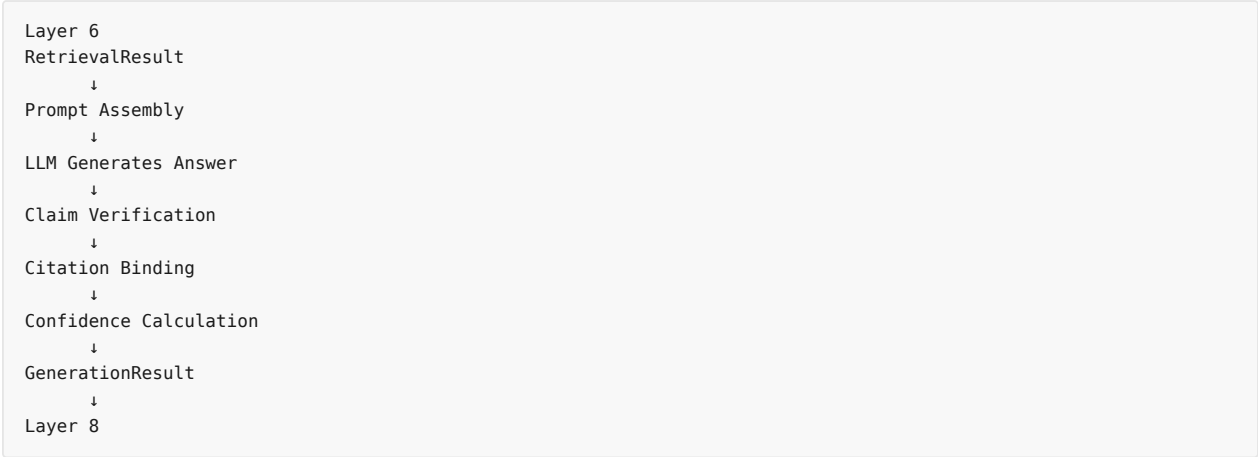
CHATGPT 24/08/2026, 01:50:35

Layer 7 – Generation Engine

You can explain it to your guide like this:

“Layer 7 is responsible for converting the retrieved knowledge from Layer 6 into a trustworthy final answer. It first prepares a structured prompt using the retrieved context and then uses an LLM to generate the answer. After generation, it verifies each factual claim against the original evidence, removes unsupported claims, attaches citations, and calculates an overall confidence score.”

Simple Flow



In One Simple Line

“Layer 6 finds the relevant knowledge, while Layer 7 converts that knowledge into a verified, cited, and reliable answer.”

filecite turn25file0 L13-L17

YOU 24/08/2026, 01:54:55

ab mujhe layer 8 smjhao detailed me jese baki layer smjhayi hai tumne

CHATGPT 24/08/2026, 01:54:57

{"paths":["files"]}

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: filecite turn27file0

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_00000000fb2c81fd82f14a7a48993554
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report.md
- [L6] mime_type: text/markdown
- [L7] surface: library
- [L8] score: 0.031544957774465976
- [L9] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--6
- [L10] snippet:
- [L11]

```
[L12] graph TD
[L13]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L14]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L15]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L16]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L17]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L18]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L19]     end
[L20]
[L21]     subgraph "Phase 2: Online – Retrieval & Generation"
[L22]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L23]         KF --> L6
[L24]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L25]         L7 --> A["Answer + Citations"]
[L26]     end
[L27]
[L28]     subgraph "Phase 3: Continuous – Operations"
[L29]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L30]         A --> L8
[L31]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L32]     end
[L33]
```

[L34]

[L35] ### 3.2 Data Flow

[L36]

[L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43]     ↓ Layer 3: Split sentences, extract concepts/definitions/rerelations
[L44] KnowledgeUnit objects (candidate concepts)
[L45]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47]     ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53]     ↓ Layer 7: Build prompt, invoke LLM
[L54]     ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56]     ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]

[L59] ### 3.3 Layer Summary

[L60]

[L61] | Layer | Name | Input | Output | Key Algorithms |

[L62] |-----|-----|-----|-----|-----|

[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L72]
[L73] ---
[L74]
[L75] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `filecite`turn27file1

[L76] metadata:
[L77] query_result_index: 1
[L78] file_id: file_00000000f620821186e54e582d59731e
[L79] version_id: 1
[L80] name: SHIA_RAG_Final_Report(3).md
[L81] mime_type: text/markdown
[L82] surface: library
[L83] score: 0.031544957774465976
[L84] document_chunk_id: file_00000000f620821186e54e582d59731e--1--6
[L85] snippet:
[L86]

```
[L87] graph TD
[L88]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L89]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L90]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L91]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L92]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L93]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L94]     end
[L95]
[L96]     subgraph "Phase 2: Online – Retrieval & Generation"
[L97]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L98]         KF --> L6
[L99]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L100]        L7 --> A["Answer + Citations"]
[L101]     end
[L102]
[L103]     subgraph "Phase 3: Continuous – Operations"
[L104]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L105]         A --> L8
[L106]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L107]     end
[L108]
```

[L109]
[L110] ### 3.2 Data Flow
[L111]
[L112]

```
[L113] Raw Document (PDF/DOCX/HTML)
[L114]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L115] Normalized Text + Metadata
[L116]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L117] [Page → Region → Block] hierarchy
[L118]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L119] KnowledgeUnit objects (candidate concepts)
[L120]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L121] KnowledgeNode objects (canonical, deduplicated)
[L122]   ↓ Layer 4: Validate relations against ontology, score confidence
[L123] KnowledgeEdge objects (typed, scored)
[L124]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L125] Knowledge Forest (rooted trees + cross-links)
[L126]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L127] Selected context nodes (within token budget)
[L128]   ↓ Layer 7: Build prompt, invoke LLM
[L129]   ↓ Layer 7: Verify claims against evidence, bind citations
[L130] Answer with citations + confidence score
[L131]   ↓ Layer 8: Log, collect feedback, update forest weights
[L132]
```

[L133]

[L134] ### 3.3 Layer Summary

[L135]

[L136] | Layer | Name | Input | Output | Key Algorithms |

[L137] |-----|-----|-----|-----|-----|

[L138] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L139] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L140] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L141] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L142] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L143] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L144] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L145] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L146] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L147]

[L148] ---

[L149]

[L150] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `[[filecite]]turn27file2`

[L151] metadata:

[L152] query_result_index: 2

[L153] file_id: file_00000000dcc082119f935683a5e13eed

[L154] version_id: 1

[L155] name: SHIA_RAG_Final_Report(4)(1).md

[L156] mime_type: text/markdown

[L157] surface: conversation

[L158] score: 0.03125763125763126

[L159] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6

[L160] snippet:

[L161]

```
[L162] graph TD
[L163]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L164]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L165]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L166]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L167]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L168]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L169]     end
[L170]
[L171]     subgraph "Phase 2: Online – Retrieval & Generation"
[L172]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L173]         KF --> L6
[L174]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L175]         L7 --> A["Answer + Citations"]
[L176]     end
[L177]
[L178]     subgraph "Phase 3: Continuous – Operations"
[L179]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L180]         A --> L8
[L181]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L182]     end
[L183]
```

[L184]

[L185] ### 3.2 Data Flow

[L186]

[L187]

```
[L188] Raw Document (PDF/DOCX/HTML)
[L189]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L190] Normalized Text + Metadata
[L191]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L192] [Page → Region → Block] hierarchy
[L193]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L194] KnowledgeUnit objects (candidate concepts)
[L195]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L196] KnowledgeNode objects (canonical, deduplicated)
[L197]     ↓ Layer 4: Validate relations against ontology, score confidence
[L198] KnowledgeEdge objects (typed, scored)
[L199]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L200] Knowledge Forest (rooted trees + cross-links)
[L201]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L202] Selected context nodes (within token budget)
[L203]     ↓ Layer 7: Build prompt, invoke LLM
[L204]     ↓ Layer 7: Verify claims against evidence, bind citations
[L205] Answer with citations + confidence score
[L206]     ↓ Layer 8: Log, collect feedback, update forest weights
[L207]
```

[L208]

[L209] ### 3.3 Layer Summary

[L210]

[L211] | Layer | Name | Input | Output | Key Algorithms |

[L212] |-----|-----|-----|-----|-----|

[L213] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L214] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L215] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L216] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L217] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L218] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L219] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L220] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L221] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L222]
[L223] ---
[L224]
[L225] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `[[filecite]]turn27file3[[`

[L226] metadata:
[L227] query_result_index: 3
[L228] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L229] version_id: 1
[L230] name: SHIA_RAG_Final_Report(4).md
[L231] mime_type: text/markdown
[L232] surface: library
[L233] score: 0.031024531024531024
[L234] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--6
[L235] snippet:
[L236]

```
[L237] graph TD
[L238]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L239]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L240]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L241]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L242]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L243]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L244]     end
[L245]
[L246]     subgraph "Phase 2: Online – Retrieval & Generation"
[L247]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L248]         KF --> L6
[L249]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L250]         L7 --> A["Answer + Citations"]
[L251]     end
[L252]
[L253]     subgraph "Phase 3: Continuous – Operations"
[L254]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L255]         A --> L8
[L256]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L257]     end
[L258]
```

[L259]
[L260] ### 3.2 Data Flow
[L261]
[L262]

```
[L263] Raw Document (PDF/DOCX/HTML)
[L264]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L265] Normalized Text + Metadata
[L266]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L267] [Page → Region → Block] hierarchy
[L268]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L269] KnowledgeUnit objects (candidate concepts)
[L270]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L271] KnowledgeNode objects (canonical, deduplicated)
[L272]   ↓ Layer 4: Validate relations against ontology, score confidence
[L273] KnowledgeEdge objects (typed, scored)
[L274]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L275] Knowledge Forest (rooted trees + cross-links)
[L276]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L277] Selected context nodes (within token budget)
[L278]   ↓ Layer 7: Build prompt, invoke LLM
[L279]   ↓ Layer 7: Verify claims against evidence, bind citations
[L280] Answer with citations + confidence score
[L281]   ↓ Layer 8: Log, collect feedback, update forest weights
[L282]
```

[L283]

[L284] ### 3.3 Layer Summary

[L285]

[L286] | Layer | Name | Input | Output | Key Algorithms |

[L287] |-----|-----|-----|-----|-----|

[L288] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L289] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L290] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L291] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L292] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L293] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L294] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L295] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L296] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L297]

[L298] ---

[L299]

[L300] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `[[filecite]]turn27file4`

[L301] metadata:

[L302] query_result_index: 4

[L303] file_id: file_000000004bc481fdbf9e949ec3b7544b

[L304] version_id: 1

[L305] name: SHIA_RAG_Final_Report(2).md

[L306] mime_type: text/markdown

[L307] surface: library

[L308] score: 0.030536130536130537

[L309] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--6

[L310] snippet:

[L311]


```
[L312] graph TD
[L313]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L314]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L315]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L316]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L317]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L318]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L319]     end
[L320]
[L321]     subgraph "Phase 2: Online – Retrieval & Generation"
[L322]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L323]         KF --> L6
[L324]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L325]         L7 --> A["Answer + Citations"]
[L326]     end
[L327]
[L328]     subgraph "Phase 3: Continuous – Operations"
[L329]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L330]         A --> L8
[L331]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L332]     end
[L333]
```

[L334]

[L335] ### 3.2 Data Flow

[L336]

[L337]

```
[L338] Raw Document (PDF/DOCX/HTML)
[L339]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L340] Normalized Text + Metadata
[L341]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L342] [Page → Region → Block] hierarchy
[L343]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L344] KnowledgeUnit objects (candidate concepts)
[L345]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L346] KnowledgeNode objects (canonical, deduplicated)
[L347]     ↓ Layer 4: Validate relations against ontology, score confidence
[L348] KnowledgeEdge objects (typed, scored)
[L349]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L350] Knowledge Forest (rooted trees + cross-links)
[L351]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L352] Selected context nodes (within token budget)
[L353]     ↓ Layer 7: Build prompt, invoke LLM
[L354]     ↓ Layer 7: Verify claims against evidence, bind citations
[L355] Answer with citations + confidence score
[L356]     ↓ Layer 8: Log, collect feedback, update forest weights
[L357]
```

[L358]

[L359] ### 3.3 Layer Summary

[L360]

[L361] | Layer | Name | Input | Output | Key Algorithms |

[L362] |-----|-----|-----|-----|-----|

[L363] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L364] | 1 | Input Processing | Raw Files | NormalizedDocument | Format detection, OCR fallback |

[L365] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L366] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L367] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L368] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L369] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L370] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L371] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L372]
[L373] ---
[L374]
[L375] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `[[filecite]]turn27file5[[`

[L376] metadata:
[L377] query_result_index: 5
[L378] file_id: file_000000003218820b918ee3d86ec345ae
[L379] version_id: 1
[L380] name: SHIA_RAG_Final_Report(1).md
[L381] mime_type: text/markdown
[L382] surface: library
[L383] score: 0.030303030303030304
[L384] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--6
[L385] snippet:
[L386]

```
[L387] graph TD
[L388]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L389]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L390]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L391]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L392]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L393]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L394]     end
[L395]
[L396]     subgraph "Phase 2: Online – Retrieval & Generation"
[L397]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L398]         KF --> L6
[L399]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L400]         L7 --> A["Answer + Citations"]
[L401]     end
[L402]
[L403]     subgraph "Phase 3: Continuous – Operations"
[L404]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L405]         A --> L8
[L406]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L407]     end
[L408]
```

[L409]
[L410] ### 3.2 Data Flow
[L411]
[L412]

```
[L413] Raw Document (PDF/DOCX/HTML)
[L414]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L415] Normalized Text + Metadata
[L416]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L417] [Page → Region → Block] hierarchy
[L418]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L419] KnowledgeUnit objects (candidate concepts)
[L420]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L421] KnowledgeNode objects (canonical, deduplicated)
[L422]     ↓ Layer 4: Validate relations against ontology, score confidence
[L423] KnowledgeEdge objects (typed, scored)
[L424]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L425] Knowledge Forest (rooted trees + cross-links)
[L426]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L427] Selected context nodes (within token budget)
[L428]     ↓ Layer 7: Build prompt, invoke LLM
[L429]     ↓ Layer 7: Verify claims against evidence, bind citations
[L430] Answer with citations + confidence score
[L431]     ↓ Layer 8: Log, collect feedback, update forest weights
[L432]
```

[L433]

[L434] ### 3.3 Layer Summary

[L435]

[L436] | Layer | Name | Input | Output | Key Algorithms |

[L437] |-----|-----|-----|-----|-----|

[L438] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L439] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L440] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L441] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L442] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L443] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L444] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L445] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L446] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L447]

[L448] ---

[L449]

[L450] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `[[filecite]]turn27file6`

[L451] metadata:

[L452] query_result_index: 6

[L453] file_id: file_00000000fb2c81fd82f14a7a48993554

[L454] version_id: 1

[L455] name: SHIA_RAG_Final_Report.md

[L456] mime_type: text/markdown

[L457] surface: library

[L458] score: 0.01833142711907485

[L459] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--40

[L460] snippet:

[L461] temporal versioning |

[L462] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L463] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L464] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L465]

[L466] ---

[L467]

[L468] ## 13. Conclusion & Future Work

[L469]

[L470] ### 13.1 Summary

[L471]

[L472] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L473]

[L474] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem

[L475] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L476] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction

[L477] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation

[L478] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality

[L479]

[L480] The complete system specification covers:

[L481] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L482] - **12 defined functions** with complete implementations

[L483] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L484] - **8 schemas** covering all data objects from document blocks to query logs

[L485] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L486] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L487]

[L488] ### 13.2 Key Novelties

[L489]

[L490] | # | Novelty | Level | Why It's New |

[L491] |---|-----|-----|-----|

[L492] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L493] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L494] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L495] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L496] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L497]

[L498] ### 13.3 Next Steps

[L499]

[L500] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L501] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L502] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L503] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L504] 5. **Build Layer 4** (Relation management with ontology validation)

- [L505] 6. **Build Layer 5** (SHIA Core with algorithms)
- [L506] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
- [L507] 8. **Build Layer 7** (Generation with verification)
- [L508] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
- [L509] 10. **Run first evaluation** on OS textbook dataset using SHEF
- [L510] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
- [L511] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
- [L512]
- [L513] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `filecite`turn27file7

- [L514] metadata:
- [L515] query_result_index: 7
- [L516] file_id: file_000000003218820b918ee3d86ec345ae
- [L517] version_id: 1
- [L518] name: SHIA_RAG_Final_Report(1).md
- [L519] mime_type: text/markdown
- [L520] surface: library
- [L521] score: 0.018067016754188547
- [L522] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--40
- [L523] snippet:
- [L524] temporal versioning |
- [L525] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |
- [L526] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |
- [L527] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |
- [L528]
- [L529] ---
- [L530]
- [L531] ## 13. Conclusion & Future Work
- [L532]
- [L533] ### 13.1 Summary
- [L534]
- [L535] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:
- [L536]
- [L537] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem
- [L538] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
- [L539] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction
- [L540] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation
- [L541] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality
- [L542]
- [L543] The complete system specification covers:
- [L544] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
- [L545] - **12 defined functions** with complete implementations
- [L546] - **12 relation types** across 4 hierarchical and 8 semantic categories
- [L547] - **8 schemas** covering all data objects from document blocks to query logs

[L548] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L549] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L550]

[L551] **### 13.2 Key Novelties**

[L552]

[L553] | # | Novelty | Level | Why It's New |

[L554] |---|-----|-----|-----|

[L555] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L556] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L557] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L558] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L559] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L560]

[L561] **### 13.3 Next Steps**

[L562]

[L563] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L564] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L565] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L566] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L567] 5. **Build Layer 4** (Relation management with ontology validation)

[L568] 6. **Build Layer 5** (SHIA Core with algorithms)

[L569] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L570] 8. **Build Layer 7** (Generation with verification)

[L571] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L572] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L573] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L574] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L575]

[L576] **### 13.4 Future Work**

DEEP RESEARCH 24/08/2026, 01:55:05

Citation Marker: `filecite`turn27file8

[L577] metadata:

[L578] query_result_index: 8

[L579] file_id: file_000000003c8081fab658b1752549d64

[L580] version_id: 1

[L581] name: SHIA_RAG_Complete_Report(5).md

[L582] mime_type: text/markdown

[L583] surface: library

[L584] score: 0.018057336133766127

[L585] document_chunk_id: file_000000003c8081fab658b1752549d64--1--63

[L586] snippet:

[L587] 5""",

[L588] "down": """"ALTER TABLE knowledge_nodes

```

[L589] DROP COLUMN abstraction_level,
[L590] DROP COLUMN base_confidence,
[L591] DROP COLUMN extraction_confidence""""
[L592] },
[L593] {
[L594] "version": 3,
[L595] "description": "Migrate properties from string array to JSONB",
[L596] "up": """"ALTER TABLE knowledge_nodes
[L597] ALTER COLUMN data TYPE JSONB USING data::JSONB""""",
[L598] "down": "-- Manual rollback required"
[L599] }
[L600] ]
[L601]
[L602] def run_migrations(db, target_version=None):
[L603] current = db.get_migration_version()
[L604] for migration in MIGRATIONS:
[L605] if migration["version"] > current:
[L606] if target_version and migration["version"] > target_version:
[L607] break
[L608] db.execute(migration["up"])
[L609] db.set_migration_version(migration["version"])
[L610] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L611]

```

```

[L612]
[L613] ---
[L614]
[L615] ### 7.13 Error Handling & Retry Logic
[L616]
[L617]

```

python

```

[L618] import time
[L619] from functools import wraps
[L620]
[L621] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L622] """Decorator for retrying failed operations with exponential backoff."""
[L623] def decorator(func):
[L624] @wraps(func)
[L625] def wrapper(*args, **kwargs):
[L626] for attempt in range(max_retries):
[L627] try:
[L628] return func(*args, **kwargs)
[L629] except (TimeoutError, ConnectionError) as e:
[L630] if attempt == max_retries - 1:
[L631] raise
[L632] delay = min(base_delay * (2 ** attempt), max_delay)
[L633] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L634] time.sleep(delay)
[L635] return wrapper
[L636] return decorator
[L637]

```

```
[L638] class ProcessingPipeline:
[L639] """Error handling for the SHIA-RAG processing pipeline."""
[L640]
[L641] def process_document(self, doc):
[L642] # Layer 1: OCR with fallback
[L643] try:
[L644] text = self.extract_text(doc)
[L645] except OCRFailure:
[L646] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L647]
[L648] # Layer 3: LLM extraction with retry + fallback model
[L649] try:
[L650] concepts = self.llm_extract_concepts(text)
[L651] except LLMLTimeout:
[L652] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L653]
[L654] # Layer 6: Vector DB with cache fallback
[L655] try:
[L656] results = self.vector_search(query_embedding)
[L657] except VectorDBUnavailable:
[L658] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L659]
[L660] return {"status": "SUCCESS", "concepts": concepts}
[L661]
```

```
[L662]
[L663] ---
[L664]
[L665] ### 7.14 Caching Strategy
[L666]
[L667]
```

```
python
[L668] import redis
[L669] import hashlib
[L670] import json
[L671]
[L672] class CacheManager:
[L673] """Multi-layer caching for SHIA-RAG."""
[L674]
[L675] def __init__(self, redis_url="redis://localhost:6379"):
[L676] self.redis = redis.from_url(redis_url)
[L677] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L678] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L679] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L680]
[L681] def get_query_result(self, query_text):
[L682] """Check if this exact query has been answered recently."""
```



```

[L683] metadata:
[L684] query_result_index: 9
[L685] file_id: file_00000000a0a481fab4c739c73d87551b
[L686] version_id: 1
[L687] name: SHIA_RAG_Complete_Report(7).md
[L688] mime_type: text/markdown
[L689] surface: library
[L690] score: 0.018057336133766127
[L691] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--63
[L692] snippet:
[L693] 5""",
[L694] "down": """"ALTER TABLE knowledge_nodes
[L695] DROP COLUMN abstraction_level,
[L696] DROP COLUMN base_confidence,
[L697] DROP COLUMN extraction_confidence""""
[L698] },
[L699] {
[L700] "version": 3,
[L701] "description": "Migrate properties from string array to JSONB",
[L702] "up": """"ALTER TABLE knowledge_nodes
[L703] ALTER COLUMN data TYPE JSONB USING data::JSONB"""" ,
[L704] "down": "-- Manual rollback required"
[L705] }
[L706] ]
[L707]
[L708] def run_migrations(db, target_version=None):
[L709] current = db.get_migration_version()
[L710] for migration in MIGRATIONS:
[L711] if migration["version"] > current:
[L712] if target_version and migration["version"] > target_version:
[L713] break
[L714] db.execute(migration["up"])
[L715] db.set_migration_version(migration["version"])
[L716] print(f"Applied migration v{migration['version']}: {migration['description']}")
[L717]

```

```

[L718]
[L719] ---
[L720]
[L721] ### 7.13 Error Handling & Retry Logic
[L722]
[L723]

```

python

```

[L724] import time
[L725] from functools import wraps
[L726]
[L727] def retry_with_backoff(max_retries=3, base_delay=1.0, max_delay=30.0):
[L728] """Decorator for retrying failed operations with exponential backoff."""
[L729] def decorator(func):
[L730] @wraps(func)
[L731] def wrapper(*args, **kwargs):

```

```

[L732] for attempt in range(max_retries):
[L733] try:
[L734] return func(*args, **kwargs)
[L735] except (TimeoutError, ConnectionError) as e:
[L736] if attempt == max_retries - 1:
[L737] raise
[L738] delay = min(base_delay * (2 ** attempt), max_delay)
[L739] print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
[L740] time.sleep(delay)
[L741] return wrapper
[L742] return decorator
[L743]
[L744] class ProcessingPipeline:
[L745] """Error handling for the SHIA-RAG processing pipeline."""
[L746]
[L747] def process_document(self, doc):
[L748] # Layer 1: OCR with fallback
[L749] try:
[L750] text = self.extract_text(doc)
[L751] except OCRFailure:
[L752] return {"status": "MANUAL_REVIEW_REQUIRED", "reason": "OCR_FAILED"}
[L753]
[L754] # Layer 3: LLM extraction with retry + fallback model
[L755] try:
[L756] concepts = self.llm_extract_concepts(text)
[L757] except LLMLLMTimeout:
[L758] concepts = self.smaller_llm_extract_concepts(text) # Fallback to smaller model
[L759]
[L760] # Layer 6: Vector DB with cache fallback
[L761] try:
[L762] results = self.vector_search(query_embedding)
[L763] except VectorDBUnavailable:
[L764] results = self.cache.get_recent_results(query_hash) # Redis cache fallback
[L765]
[L766] return {"status": "SUCCESS", "concepts": concepts}
[L767]

```

```

[L768]
[L769] ---
[L770]
[L771] ### 7.14 Caching Strategy
[L772]
[L773]

```

python

```

[L774] import redis
[L775] import hashlib
[L776] import json
[L777]
[L778] class CacheManager:
[L779] """Multi-layer caching for SHIA-RAG."""
[L780]

```

```
[L781] def __init__(self, redis_url="redis://localhost:6379"):
[L782] self.redis = redis.from_url(redis_url)
[L783] self.QUERY_CACHE_TTL = 3600 # 1 hour for query results
[L784] self.NODE_CACHE_TTL = 86400 # 24 hours for node data
[L785] self.EMBEDDING_CACHE_TTL = 604800 # 7 days for embeddings
[L786]
[L787] def get_query_result(self, query_text):
[L788] """Check if this exact query has been answered recently."""
```

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `filecite`turn28file0

```
[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_00000000dcc082119f935683a5e13eed
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4)(1).md
[L6] mime_type: text/markdown
[L7] surface: conversation
[L8] score: 0.03252247488101534
[L9] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--5
[L10] snippet:
[L11] Gap: No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3)
hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L12]
[L13] ### 2.3 Literature Search Methodology
[L14]
[L15] | Parameter | Details |
[L16] |-----|-----|
[L17] | Databases Searched | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L18] | Conferences/Venues | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L19] | Search Keywords | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval
augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L20] | Time Range | 2020–2026 |
[L21] | Key Papers Reviewed | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk
(LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L22]
[L23] ---
[L24]
[L25] ## 3. System Architecture
[L26]
[L27] ### 3.1 Overview
[L28]
[L29] SHIA-RAG is organized as an 8-layer pipeline divided into three phases:
[L30]
[L31] | Phase | Layers | Mode | Purpose |
[L32] |-----|-----|-----|-----|
[L33] | Phase 1: Offline | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |
[L34] | Phase 2: Online | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |
[L35] | Phase 3: Continuous | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |
```

[L36]

[L37] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L38]

[L39]

```
[L40] graph TD
[L41]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L42]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L43]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L44]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L45]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L46]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L47]     end
[L48]
[L49]     subgraph "Phase 2: Online – Retrieval & Generation"
[L50]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L51]         KF --> L6
[L52]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L53]         L7 --> A["Answer + Citations"]
[L54]     end
[L55]
[L56]     subgraph "Phase 3: Continuous – Operations"
[L57]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L58]         A --> L8
[L59]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L60]     end
[L61] end
```

[L62]

[L63] ### 3.2 Data Flow

[L64]

[L65] ```

[L66] Raw Document (PDF/DOCX/HTML)

[L67] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L68] Normalized Text + Metadata

[L69] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L70] [Page → Region → Block] hierarchy

[L71] ↓ Layer 3: Split sentences,

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `[[filecite]]turn28file1[[`

[L72] metadata:

[L73] query_result_index: 1

[L74] file_id: file_00000000dcc082119f935683a5e13eed

[L75] version_id: 1

[L76] name: SHIA_RAG_Final_Report(4)(1).md

[L77] mime_type: text/markdown

[L78] surface: conversation

[L79] score: 0.03225806451612903

[L80] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--3

[L81] snippet:

[L82] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L83]

[L84] ### 1.3 Objectives

[L85]

[L86] 1. Design a complete 8-layer architecture for hierarchical RAG

[L87] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L88] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L89] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L90] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L91] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L92]

[L93] ### 1.4 Contributions

[L94]

[L95] This work makes five novel contributions to the RAG literature:

[L96]

[L97] | # | Contribution | Novelty Level |

[L98] |---|-----|-----|

[L99] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L100] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L101] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L102] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L103] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L104]

[L105] ---

[L106]

[L107] ## 2. Literature Review

[L108]

[L109] ### 2.1 Related Work

[L110]

[L111] ##### 2.1.1 Flat-Chunk RAG Systems

[L112]

[L113] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L114]

[L115] ##### 2.1.2 Structured RAG Systems

[L116]

[L117] Recent work has begun exploring structured retrieval:

[L118]

[L119] | System | Approach | Key Limitation |

[L120] |-----|-----|-----|

[L121] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L122] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L123] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L124] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L125]

[L126] ##### 2.1.3 Knowledge Graph Construction

[L127]

[L128] Ontology learning and taxonomy induction have a rich history in NLP:
[L129] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text
[L130] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora
[L131] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations
[L132]
[L133] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**
[L134]
[L135] **### 2.2 Research Gap**

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `filecite[turn28file2]`

[L136] metadata:
[L137] query_result_index: 2
[L138] file_id: file_00000000dcc082119f935683a5e13eed
[L139] version_id: 1
[L140] name: SHIA_RAG_Final_Report(4)(1).md
[L141] mime_type: text/markdown
[L142] surface: conversation
[L143] score: 0.031009615384615385
[L144] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--0
[L145] snippet:
[L146] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation
[L147]
[L148] > **Project Type:** Minor Project (M.Tech)
[L149] > **Date:** August 2026
[L150] > **Author:** [Your Name]
[L151]
[L152] ---
[L153]
[L154] **## Abstract**
[L155]

[L156] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L157]
[L158] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L159]
[L160] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L161]
[L162] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation

management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L163]

[L164] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L165]

[L166] ---

[L167]

[L168] ## Table of Contents

[L169]

[L170] 1. [Introduction](#)

[L171] 2. [Literature Review](#)

[L172] 3. [System Architecture](#)

[L173] 4. [Data Model — Layer 0](#)

[L174] 5. [Offline Pipeline — Layers 1-5](#)

[L175] 6. [Online Pipeline — Layers 6-7](#)

[L176] 7. [Operations Layer — Layer 8](#)

[L177] 8. [Novel Contributions](#)

[L178] 9. [Mathematical Framework](#)

[L179] 10. [Implementation](#)

[L180] 11. [Evaluation Framework](#)

[L181] 12. [Risk Analysis](#)

[L182] 13. [Conclusion & Future Work](#)

[L183]

[L184] ---

[L185]

[L186] ## 1. Introduction

[L187]

[L188] ### 1.1 Problem Statement

[L189]

[L190] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L191]

[L192] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L193] 2. Embed each chunk into a vector space

[L194] 3. At query time, retrieve the top-k most similar chunks

[L195] 4. Feed these chunks as context to an LLM for answer generation

[L196]

[L197] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `filecite[turn28file3]`

[L198] metadata:

[L199] query_result_index: 3

[L200] file_id: file_00000000dcc082119f935683a5e13eed

[L201] version_id: 1

[L202] name: SHIA_RAG_Final_Report(4)(1).md

[L203] mime_type: text/markdown

[L204] surface: conversation

[L205] score: 0.030798389007344232

[L206] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--54

[L207] snippet:

[L208] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L209] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L210] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L211] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L212] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L213]

[L214] The complete system specification covers:

[L215] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L216] - **12 defined functions** with complete implementations

[L217] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L218] - **8 schemas** covering all data objects from document blocks to query logs

[L219] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L220] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L221]

[L222] ### 13.2 Key Novelties

[L223]

[L224] | # | Novelty | Level | Why It's New |

[L225] |---|-----|-----|-----|

[L226] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L227] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L228] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L229] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L230] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L231]

[L232] ### 13.3 Next Steps

[L233]

[L234] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L235] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L236] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L237] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L238] 5. **Build Layer 4** (Relation management with ontology validation)

[L239] 6. **Build Layer 5** (SHIA Core with algorithms)

[L240] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L241] 8. **Build Layer 7** (Generation with verification)

[L242] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L243] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L244] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L245] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
[L246]
[L247] ### 13.4 Future Work

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `filecite`turn28file4

[L248] metadata:
[L249] query_result_index: 4
[L250] file_id: file_00000000dcc082119f935683a5e13eed
[L251] version_id: 1
[L252] name: SHIA_RAG_Final_Report(4)(1).md
[L253] mime_type: text/markdown
[L254] surface: conversation
[L255] score: 0.030158730158730156
[L256] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--6
[L257] snippet:
[L258]

```
[L259] graph TD
[L260]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L261]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, tokenization"]
[L262]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L263]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L264]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L265]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L266]     end
[L267]
[L268]     subgraph "Phase 2: Online – Retrieval & Generation"
[L269]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L270]         KF --> L6
[L271]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L272]         L7 --> A["Answer + Citations"]
[L273]     end
[L274]
[L275]     subgraph "Phase 3: Continuous – Operations"
[L276]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L277]         A --> L8
[L278]         L8 --> SE["Self-Evolution<br/>Feedback"] | KF
[L279]     end
[L280]
```

[L281]
[L282] ### 3.2 Data Flow
[L283]
[L284]

```
[L285] Raw Document (PDF/DOCX/HTML)
[L286]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L287] Normalized Text + Metadata
[L288]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L289] [Page → Region → Block] hierarchy
[L290]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L291] KnowledgeUnit objects (candidate concepts)
[L292]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L293] KnowledgeNode objects (canonical, deduplicated)
[L294]   ↓ Layer 4: Validate relations against ontology, score confidence
[L295] KnowledgeEdge objects (typed, scored)
[L296]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L297] Knowledge Forest (rooted trees + cross-links)
[L298]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L299] Selected context nodes (within token budget)
[L300]   ↓ Layer 7: Build prompt, invoke LLM
[L301]   ↓ Layer 7: Verify claims against evidence, bind citations
[L302] Answer with citations + confidence score
[L303]   ↓ Layer 8: Log, collect feedback, update forest weights
[L304]
```

[L305]

[L306] ### 3.3 Layer Summary

[L307]

[L308] | Layer | Name | Input | Output | Key Algorithms |

[L309] |-----|-----|-----|-----|-----|

[L310] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L311] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L312] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L313] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L314] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L315] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L316] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L317] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L318] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L319]

[L320] ---

[L321]

[L322] ## 4. Data Model — Layer 0

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `[[filecite]]turn28file5`

[L323] metadata:

[L324] query_result_index: 5

[L325] file_id: file_00000000dcc082119f935683a5e13eed

[L326] version_id: 1

[L327] name: SHIA_RAG_Final_Report(4)(1).md

[L328] mime_type: text/markdown

[L329] surface: conversation

[L330] score: 0.030158730158730156

[L331] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--1

[L332] snippet:

[L333] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L334]

[L335] ---

[L336]

[L337] ## Table of Contents

[L338]

[L339] 1. Introduction

[L340] 2. Literature Review

[L341] 3. System Architecture

[L342] 4. Data Model — Layer 0

[L343] 5. Offline Pipeline — Layers 1-5

[L344] 6. Online Pipeline — Layers 6-7

[L345] 7. Operations Layer — Layer 8

[L346] 8. Novel Contributions

[L347] 9. Mathematical Framework

[L348] 10. Implementation

[L349] 11. Evaluation Framework

[L350] 12. Risk Analysis

[L351] 13. Conclusion & Future Work

[L352]

[L353] ---

[L354]

[L355] ## 1. Introduction

[L356]

[L357] ### 1.1 Problem Statement

[L358]

[L359] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L360]

[L361] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L362] 2. Embed each chunk into a vector space

[L363] 3. At query time, retrieve the top-k most similar chunks

[L364] 4. Feed these chunks as context to an LLM for answer generation

[L365]

[L366] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L367]

[L368] | Problem | Description | Impact |

[L369] |-----|-----|-----|

[L370] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L371] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L372] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L373] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L374] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L375]

[L376] ### 1.2 Proposed Solution: SHIA-RAG

[L377]

[L378] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L379]

[L380]

[L381]	Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L382]	SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L383]		

[L384]

[L385] The system:

[L386] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L387] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L388] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L389] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L390] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L391] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `[[filecite]]turn28file6`

[L392] metadata:

[L393] query_result_index: 6

[L394] file_id: file_00000000dcc082119f935683a5e13eed

[L395] version_id: 1

[L396] name: SHIA_RAG_Final_Report(4)(1).md

[L397] mime_type: text/markdown

[L398] surface: conversation

[L399] score: 0.02976190476190476

[L400] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--35

[L401] snippet:

[L402] ##### Prompt Template

[L403]

[L404]

```
[L405] SYSTEM_PROMPT = ""You are a knowledge-grounded question answering system.
[L406] RULES:
[L407] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L408] 2. For each claim, cite the source using [NODE_ID] format.
[L409] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L410] 4. If evidence conflicts, present both views with citations.
[L411]
[L412] REQUIRED OUTPUT FORMAT:
[L413] {
[L414]   "answer": "Your answer with [NODE_ID] citations inline.",
[L415]   "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L416]   "overall_confidence": 0.0-1.0,
[L417]   "insufficient_evidence": false
[L418] }""
[L419]
```

[L420]

[L421] ##### Claim Verification

[L422]

[L423] After the LLM generates an answer, each claim is verified against its cited evidence:

[L424]

[L425]

```
[L426] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L427]     """Returns (status, confidence).
[L428]     - SUPPORTED:     evidence entails claim (confidence > 0.7)
[L429]     - PARTIAL:      evidence partially supports (0.4 < confidence ≤ 0.7)
[L430]     - UNSUPPORTED:  no evidence supports (confidence ≤ 0.4)
[L431]     - CONFLICTING:  evidence items contradict each other
[L432]     """
[L433]     scores = []
[L434]     for eid in evidence_ids:
[L435]         node = node_map.get(eid)
[L436]         if node and node.evidence:
[L437]             evidence_text = ' '.join(e.text for e in node.evidence)
[L438]             scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L439]
[L440]     if not scores:
[L441]         return 'UNSUPPORTED', 0.0
[L442]
[L443]     avg_score = sum(scores) / len(scores)
[L444]
[L445]     if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L446]         return 'CONFLICTING', avg_score
[L447]     if avg_score > 0.7:
[L448]         return 'SUPPORTED', avg_score
[L449]     elif avg_score > 0.4:
[L450]         return 'PARTIAL', avg_score
[L451]     else:
[L452]         return 'UNSUPPORTED', avg_score
[L453]
```

[L454]

[L455] ##### Unsupported Claim Policy

[L456]

[L457] | Status | Action |

[L458] |-----|-----|

[L459] | SUPPORTED | Keep as-is with citation |

[L460] | PARTIAL | Keep but mark as partially supported |

[L461] | UNSUPPORTED | Remove from answer, note at end |

[L462] | CONFLICTING | Present both views with note |

[L463]

[L464] ##### Answer Confidence Formula

[L465]

[L466]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{node_conf}}{\text{cited nodes}} + 0.1 \cdot \frac{\text{selected nodes}}{\text{selected nodes}}$$

[L467]

[L468] ##### Output Schema

[L469]

[L470]

```
[L471] @dataclass
[L472] class GenerationResult:
[L473]     """Output of Layer 7 – ready for user and Layer 8."""
[L474]     answer: str                # Answer text with citations
[L475]     claims: list              # List of VerifiedClaim
[L476]     citations: list           # Formatted citation list
[L477]     overall_confidence: float  # 0.0-1.0
[L478]     retrieval_result: object   # Preserved for Layer 8
[L479]     generation_metadata: dict  # Model used, claim counts, etc.
[L480]
```

[L481]

[L482] ---

[L483]

[L484] ## 7. Operations Layer — Layer 8

[L485]

[L486] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L487]

[L488] ### 7.1 Logging

[L489]

[L490] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `filecite`turn28file7

[L491] metadata:

[L492] query_result_index: 7

[L493] file_id: file_00000000dcc082119f935683a5e13eed

[L494] version_id: 1

[L495] name: SHIA_RAG_Final_Report(4)(1).md

[L496] mime_type: text/markdown

[L497] surface: conversation

[L498] score: 0.02946912242686891

[L499] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--2

[L500] snippet:

[L501] | Problem | Description | Impact |

[L502] |-----|-----|-----|

[L503] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L504] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L505] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L506] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L507] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L508]

[L509] ### 1.2 Proposed Solution: SHIA-RAG

[L510]

[L511] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five

limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L512]

[L513]

[L514] Traditional RAG:

Document → [Chunk₁, Chunk₂, ..., Chunk_n] → Top-k → LLM

[L515] SHIA-RAG:

Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM

[L516]

[L517]

[L518] The system:

- [L519] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents
- [L520] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees
- [L521] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth
- [L522] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints
- [L523] 5. **Generates verified answers** — with claim-level evidence verification and citation binding
- [L524] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure
- [L525]

[L526] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L527]

[L528] ### 1.3 Objectives

[L529]

- [L530] 1. Design a complete 8-layer architecture for hierarchical RAG
- [L531] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)
- [L532] 3. Implement a self-evolving feedback loop that improves retrieval quality over time
- [L533] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)
- [L534] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality
- [L535] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L536]

[L537] ### 1.4 Contributions

[L538]

[L539] This work makes five novel contributions to the RAG literature:

[L540]

[L541] | # | Contribution | Novelty Level |

[L542] |---|-----|-----|

- [L543] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |
- [L544] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |
- [L545] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |
- [L546] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |
- [L547] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L548]

[L549] ---

[L550]

[L551] ## 2. Literature Review

[L552]

[L553] ### 2.1 Related Work

[L554]

[L555] #### 2.1.1 Flat-Chunk RAG Systems

Citation Marker: [filecite]turn28file8[

[L556] metadata:

[L557] query_result_index: 8

[L558] file_id: file_00000000dcc082119f935683a5e13eed

[L559] version_id: 1

[L560] name: SHIA_RAG_Final_Report(4)(1).md

[L561] mime_type: text/markdown

[L562] surface: conversation

[L563] score: 0.029418126757516764

[L564] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--51

[L565] snippet:

[L566] ### 11.4 Statistical Significance Testing

[L567]

[L568] | Test | Application | Details |

[L569] |-----|-----|-----|

[L570] | Paired t-test | SHIA-RAG vs each baseline | Per-query metric comparison |

[L571] | Significance level | All tests | $\alpha = 0.05$ |

[L572] | Effect size | All comparisons | Cohen's d (small: 0.2, medium: 0.5, large: 0.8) |

[L573] | Confidence intervals | All reported metrics | 95% CI via bootstrap (1000 resamples) |

[L574] | Multiple comparison correction | When comparing 5+ baselines | Bonferroni or Holm-Bonferroni |

[L575]

[L576] ### 11.5 Error Analysis

[L577]

[L578] | Analysis Type | Method | Details |

[L579] |-----|-----|-----|

[L580] | Query categorization | Classify all test queries | DEFINITION, COMPARISON, MULTI-HOP, SYNTHESIS |

[L581] | Failure pattern analysis | Cluster failed queries | Identify common failure modes |

[L582] | Qualitative analysis | Manual inspection | Review 100 randomly sampled failed queries |

[L583] | Per-category breakdown | Stratified metrics | Report accuracy per query type |

[L584]

[L585] ### 11.6 Computational Cost Analysis

[L586]

[L587] | Metric | How Measured | Report Format |

[L588] |-----|-----|-----|

[L589] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |

[L590] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |

[L591] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |

[L592] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |

[L593] | Memory usage | Peak RAM + VRAM | GB |

[L594] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |

[L595]

[L596] ---

[L597]

[L598] ## 12. Risk Analysis

[L599]

[L600] | Risk | Impact | Probability | Mitigation |

[L601] |-----|-----|-----|-----|

[L602] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L603] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L604] | $O(N^2)$ operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L605] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning, conflict logging |

[L606] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L607] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L608] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L609] | Self-evolution reinforces wrong paths | Medium | Medium | ϵ -greedy exploration ($\epsilon=0.1$), periodic weight normalization, anomaly detection |

[L610]

[L611] ---

[L612]

[L613] ## 12.5 Limitations & Current Status

[L614]

[L615] This section explicitly documents known limitations and the current project status.

[L616]

[L617] ### Current Status

[L618]

[L619] SHIA-RAG is currently at the **specification stage**. This report provides a complete architectural specification with schemas, algorithms, pseudocode, and implementation contracts. However:

[L620]

[L621] - **No implementation exists yet** — the system has not been built or deployed

[L622] - **No experimental results exist** — all evaluation metrics are planned, not measured

[L623] - **Cost/latency numbers are projections** — actual measurements require a running system

[L624]

[L625] The implementation roadmap (Section 10.3) details the path from specification to working system.

[L626]

[L627] ### Known Limitations

DEEP RESEARCH 24/08/2026, 01:55:10

Citation Marker: `filecite turn28file9`

[L628] metadata:

[L629] query_result_index: 9

[L630] file_id: file_00000000dcc082119f935683a5e13eed

[L631] version_id: 1

[L632] name: SHIA_RAG_Final_Report(4)(1).md

[L633] mime_type: text/markdown

[L634] surface: conversation

[L635] score: 0.02889344262295082

[L636] document_chunk_id: file_00000000dcc082119f935683a5e13eed--1--4

[L637] snippet:

[L638] ##### 2.1.2 Structured RAG Systems

[L639]

[L640] Recent work has begun exploring structured retrieval:

[L641]

[L642] | System | Approach | Key Limitation |

[L643] |-----|-----|-----|

[L644] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at

multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L645] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L646] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L647] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L648]

[L649] ##### 2.1.3 Knowledge Graph Construction

[L650]

[L651] Ontology learning and taxonomy induction have a rich history in NLP:

[L652] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L653] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L654] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L655]

[L656] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L657]

[L658] ### 2.2 Research Gap

[L659]

[L660]

[L661] [L662] System	Structured Retrieval?	Concept-Level Extraction?	Self- Evolving?	Hierarchy-Aware Context Selection?
[L663]				
[L664] Traditional RAG	x	x	x	x
[L665] RAPTOR	✓ (summaries)	x	x	x
[L666] GraphRAG	✓ (subgraph)	✓ (partial)	x	x
[L667] TreeRAG	✓ (tree)	x	x	x
[L668] HiChunk	✓ (chunks)	x	x	x
[L669]				
[L670] SHIA-RAG (Ours)	✓ (forest)	✓ (full)	✓	✓
[L671]				

[L672]

[L673] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L674]

[L675] ### 2.3 Literature Search Methodology

[L676]

[L677] | Parameter | Details |

[L678] |-----|-----|

[L679] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L680] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L681] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L682] | **Time Range** | 2020–2026 |

[L683] | **Key Papers Reviewed** | RAPTOR (Sarthi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L684]

[L685] ---

[L686]

[L687] ## 3. System Architecture

[L688]

[L689] ### 3.1 Overview

[L690]

[L691] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

DEEP RESEARCH 24/08/2026, 01:55:14

Citation Marker: `filecite`turn29file0

[L1] metadata:

[L2] ref_id: turn28file6

[L3] file_id: file_00000000dcc082119f935683a5e13eed

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4)(1).md

[L6] lines_shown: 484-983

[L7] total_file_lines: 2066

[L8] has_more: True

[L9] next_start_line: 984

[L10] next_read: {"ref_id": "turn28file6", "mode": "full", "start_line": 984, "max_lines": 500}

[L11] content:

[L12] Signal 1: Text density (text chars / page area)

[L13] Signal 2: Font metadata availability (scanned pages lack font info)

[L14] Signal 3: Image coverage ratio (scanned pages are mostly images)

[L15] """

[L16] text = page.get_text()

[L17] area = page.rect.width *page.rect.height*

[L18]

[L19] # Signal 1: Text density

[L20] *text_density = len(text.strip()) / max(area, 1)*

[L21]

[L22] # Signal 2: Font metadata (scanned PDFs have no embedded fonts)

[L23] *has_fonts = len(page.get_fonts()) > 0*

[L24]

[L25] # Signal 3: Image coverage (scanned pages are large images)

[L26] *images = page.get_images()*

[L27] *image_area = sum(img[2] *img[3]* for img in images) if images else 0*

[L28] *image_coverage = image_area / max(area, 1)*

[L29]

[L30] # Decision: OCR needed if text is sparse AND (no fonts OR high image coverage)

[L31] *if text_density < 0.1 and (not has_fonts or image_coverage > 0.5):*

[L32] *return True*

[L33] *return False*

[L34]

[L35]

[L36] ### OCR Confidence Calculation

[L37]

[L38]

python

[L39] def calculate_ocr_confidence(ocr_result) -> float:

[L40] """Calculate OCR confidence from Tesseract per-word confidence scores.

[L41]

[L42] Tesseract returns per-word confidence (0-100). We:

```

[L43] 1. Get per-word confidences
[L44] 2. Filter out words with confidence < 10 (noise)
[L45] 3. Compute weighted average by word length (longer words = more reliable signal)
[L46] 4. Normalize to [0.0, 1.0]
[L47] """
[L48] word_data = ocr_result.get('words', [])
[L49] if not word_data:
[L50] return 0.0
[L51]
[L52] total_weight, weighted_sum = 0, 0
[L53] for word in word_data:
[L54] if word['confidence'] >= 10: # Filter noise
[L55] weight = len(word['text']) # Weight by word length
[L56] weighted_sum += word['confidence'] * weight
[L57] total_weight += weight
[L58]
[L59] return (weighted_sum / max(total_weight, 1)) / 100.0
[L60]

```

```

[L61]
[L62] ##### Text Cleaning (Safe)
[L63]
[L64]

```

python

```

[L65] def clean_text(text: str, pages: list) -> str:
[L66] """Clean extracted text while preserving legitimate content.
[L67]
[L68] Safeguards against false header/footer removal:
[L69] 1. Identify candidate headers/footers by frequency across pages
[L70] 2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L71] 3. Never remove lines that appear on fewer than 3 pages
[L72] 4. Normalize whitespace but preserve paragraph boundaries
[L73] """
[L74] if len(pages) >= 3:
[L75] # Frequency-based header/footer detection
[L76] first_lines = [p.split("\n")[0].strip() for p in pages if p.strip()]
[L77] last_lines = [p.split("\n")[-1].strip() for p in pages if p.strip()]
[L78]
[L79] from collections import Counter
[L80] header_counts = Counter(first_lines)
[L81] footer_counts = Counter(last_lines)
[L82]
[L83] threshold = max(3, int(len(pages) * 0.6)) # 60% of pages OR at least 3
[L84] headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L85] footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L86]
[L87] lines = text.split("\n")
[L88] lines = [l for l in lines
[L89] if l.strip() not in headers_to_remove
[L90] and l.strip() not in footers_to_remove]

```

```
[L91] text = '\n'.join(lines)
[L92]
[L93] # Normalize whitespace (preserve paragraph breaks)
[L94] import re
[L95] text = re.sub(r'[\t]+' , ' ', text) # Collapse horizontal space
[L96] text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L97] return text.strip()
[L98]
```

```
[L99]
[L100] ##### Language Detection
[L101]
[L102]
```

```
python
[L103] from langdetect import detect
[L104]
[L105] def detect_language(text: str) -> str:
[L106] """Detect document language for OCR configuration.
[L107]
[L108] Used to configure Tesseract's language parameter.
[L109] Falls back to 'eng' if detection fails."""
[L110] try:
[L111] lang = detect(text[:5000]) # Sample first 5000 chars
[L112] # Map langdetect codes to Tesseract codes
[L113] lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L114] return lang_map.get(lang, 'eng')
[L115] except Exception:
[L116] return 'eng'
[L117]
```

```
[L118]
[L119] ##### Partial Extraction Policy
[L120]
[L121] If some pages fail during extraction, the system uses a **best-effort policy**:
[L122]
[L123] | Condition | Action |
[L124] |-----|-----|
[L125] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |
[L126] | <80% pages succeed | Reject entire document, queue for manual review |
[L127] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L128]
[L129]
```

```
python
[L130] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L131] """Layer 1: Input Processing pipeline with all safeguards."""
[L132] # Step 0: Duplicate check
[L133] content_hash = check_duplicate(file_path, doc_registry)
[L134]
[L135] fmt = detect_format(file_path)
[L136] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L137]
[L138] failed_pages = []
[L139] if fmt == 'pdf':
[L140] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
```

```

[L141] # Partial extraction policy
[L142] success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L143] if success_rate < 0.8:
[L144]     raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L145] elif fmt == 'docx':
[L146]     text, pages = extract_docx(file_path), []
[L147]     is_ocr, ocr_conf = False, 1.0
[L148] elif fmt == 'html':
[L149]     text, pages = extract_html(file_path), []
[L150]     is_ocr, ocr_conf = False, 1.0
[L151] else: # txt
[L152]     text = open(file_path, 'r', encoding='utf-8').read()
[L153]     pages, is_ocr, ocr_conf = [], False, 1.0
[L154]
[L155]     language = detect_language(text)
[L156]     text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L157]     metadata = extract_metadata(file_path, fmt)
[L158]
[L159]     doc = NormalizedDocument(
[L160]         doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L161]         format=DocumentFormat(fmt), text=text, pages=pages,
[L162]         metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L163]         char_count=len(text), page_count=len(pages) or 1,
[L164]         content_hash=content_hash, language=language,
[L165]         partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L166]     )
[L167]     doc_registry[content_hash] = doc.doc_id # Register for dedup
[L168]     return doc
[L169]

```

```

[L170]
[L171] ---
[L172]
[L173] ### 5.2 Layer 2: Document Parsing
[L174]
[L175] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with block cl
[L176]
[L177] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L178]
[L179]

```

python

```

[L180] class BlockType(Enum):
[L181]     HEADING = "heading"
[L182]     PARAGRAPH = "paragraph"
[L183]     LIST = "list"
[L184]     TABLE = "table"
[L185]     CODE = "code"
[L186]     ALGORITHM = "algorithm"
[L187]     FIGURE_CAPTION = "figure_caption"
[L188]     EQUATION = "equation"
[L189]     DEFINITION = "definition"
[L190]

```

```
[L191] @dataclass
[L192] class DocumentBlock:
[L193] """Atomic unit of document structure."""
[L194] block_id: str
[L195] doc_id: str
[L196] page_number: int
[L197] region_index: int # Which region on the page
[L198] block_type: BlockType
[L199] text: str
[L200] reading_order: int # Global ordering across the document
[L201] confidence: float = 1.0 # Classification confidence
[L202] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L203]
```

```
[L204]
[L205] The parser uses rule-based heuristics for block classification:
[L206] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase
[L207] - **List:** Starts with ` - `, `* `, `• `, or `1.`
[L208] - **Table:** Contains 3+ pipe characters (`|`) – uses regex `re.match(r'^(def |class |import |from )', line)`
[L209] - **Code:** Uses regex pattern matching (NOT `def` in text, which falsely matches "definitely")
[L210] - **Definition:** Contains phrases like "is defined as", "refers to"
[L211] - **Default:** Paragraph
[L212]
[L213] ##### Table Cell-Level Parsing
[L214]
[L215] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level st
[L216]
[L217]
```

python

```
[L218] def parse_table_cells(page, table_region) -> list:
[L219] """Extract table structure with cell-level detail.
[L220]
[L221] Uses pdfplumber for text-based tables, Camelot for image-based.
[L222] Returns list of rows, each row is a list of cell strings."""
[L223] try:
[L224] # Primary: pdfplumber table extraction
[L225] table = page.extract_table(table_region)
[L226] if table and len(table) > 1: # At least header + 1 data row
[L227] return table
[L228] except Exception:
[L229] pass
[L230]
[L231] # Fallback: Camelot for image-based tables
[L232] try:
[L233] import camelot
[L234] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L235] if tables:
[L236] return tables[0].df.values.tolist()
[L237] except Exception:
[L238] pass
[L239]
[L240] return None # Table structure could not be parsed
[L241]
```

```
[L242]
[L243] ##### Table Detection Fallback
[L244]
[L245] If table detection confidence is low, the block is flagged:
[L246]
[L247]
```

python

```
[L248] def classify_block_with_fallback(text, page) -> tuple:
[L249] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L250] block_type, confidence = classify_block(text)
[L251]
[L252] if block_type == BlockType.TABLE and confidence < 0.6:
[L253] # Attempt cell-level parsing to confirm
[L254] cells = parse_table_cells(page, text)
[L255] if cells is None:
[L256] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L257] else:
[L258] return BlockType.TABLE, 0.8 # Confirmed by parser
[L259]
[L260] return block_type, confidence
[L261]
```

```
[L262]
[L263] ##### Reading Order Validation
[L264]
[L265] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L266]
[L267]
```

python

```
[L268] def assign_reading_order(blocks: list) -> list:
[L269] """Assign global reading order with confidence.
[L270]
[L271] Multi-column layouts can produce incorrect reading order.
[L272] We detect potential issues and flag low confidence."""
[L273] # Sort by page, then y-coordinate, then x-coordinate
[L274] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L275]
[L276] for i, block in enumerate(sorted_blocks):
[L277] block.reading_order = i
[L278]
[L279] # Detect potential multi-column issues
[L280] if i > 0:
[L281] prev = sorted_blocks[i - 1]
[L282] if (prev.page_number == block.page_number and
[L283] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L284] prev.x_pos > block.x_pos): # But previous is to the right
[L285] block.confidence *= 0.7 # Reduce confidence — possible column issue
[L286]
[L287] return sorted_blocks
[L288]
```



```

[L289]
[L290] ---
[L291]
[L292] ### 5.3 Layer 3: Knowledge Extraction
[L293]
[L294] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L295]
[L296] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L297]
[L298] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L299]
[L300] #### Extraction Prompt
[L301]
[L302]

```

[L303] You are a knowledge extraction system. Given a text block, extract:

[L304] 1. Concepts: Named entities, technical terms

[L305] 2. Definitions: If the text defines any concept, extract the definition

[L306] 3. Properties: Key attributes with types and values

[L307] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L308] 5. Domain: Subject area (e.g., "Operating Systems")

[L309]

[L310] Respond in JSON format:

```

[L311] {
[L312] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."]},
[L313] "properties": {"key": {"type": "...", "value": "..."}},
[L314] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "..."},
[L315] "evidence": "..."},
[L316] "domain": "..."}
[L317] }
[L318]

```

```

[L319]
[L320] #### Definition Extraction (Dependency-Parse Based)
[L321]
[L322] Simple regex patterns like `^(.*) is (a|an|the)? (.*)` are too broad – they match non-definitions like "The s
[L323]
[L324]

```

python

[L325] def is_definition(text: str) -> tuple:

[L326] """Detect definitions using dependency parse + pattern matching.

[L327]

[L328] Returns (is_def: bool, concept: str, definition: str).

[L329]

[L330] Strategy:

[L331] 1. Check for explicit definitional phrases (high precision)

[L332] 2. Use dependency parse to find copular definitions (medium precision)

[L333] 3. Fallback to LLM classification (expensive, high recall)

[L334] """

[L335] import re

[L336]

[L337] # Strategy 1: Explicit definitional phrases (high precision)

[L338] explicit_patterns = [

```
[L339] r'^(.+?)\s+is defined as\s+(.+)',
[L340] r'^(.+?)\s+refers to\s+(.+)',
[L341] r'^(.+?):\s+A-Z', # "Concept: A description..."
[L342] r'^(.+?)\s+is a\s+(type|kind|form|category|class)\s+of\s+(.+)',
[L343] ]
[L344] for pattern in explicit_patterns:
[L345] match = re.match(pattern, text, re.IGNORECASE)
[L346] if match:
[L347] return True, match.group(1).strip(), match.group(2).strip()
[L348]
[L349] # Strategy 2: Copular structure via dependency parse
[L350] # (requires spaCy or similar — use LLM as fallback if not available)
[L351]
[L352] # Strategy 3: LLM fallback (only for DEFINITION-classified blocks)
[L353] return False, None, None
[L354]
```

```
[L355]
[L356] ##### Alias Resolution (with Antonym Protection)
[L357]
[L358] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin",
[L359]
[L360] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases
[L361] 2. **Antonym check**: Before merging, verify candidates do not have a CONTRASTS_WITH relation
[L362] 3. Find connected components of near-duplicate units
[L363] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evi
[L364]
[L365]
```

python

```
[L366] def alias_resolution(units: list, relation_cache: dict) -> list:
[L367] """Merge KnowledgeUnits referring to the same concept.
[L368]
[L369] Merge strategy:
[L370] - Canonical name: most frequent name across units
[L371] - Aliases: union of all names and aliases
[L372] - Definitions: union of all definitions
[L373] - Co
```

DEEP RESEARCH 24/08/2026, 01:55:20

Citation Marker: `filecite`turn30file0

```
[L1] Find: find=0 ref=turn28file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=## 7. Operations Layer — Layer 8
[L2] Scope: full file surface=conversation
[L3] Matches: shown=1 total=1
[L4] [match 0 | match_lines 1420 | shown_lines 1415-1425]
[L5] generation_metadata: dict # Model used, claim counts, etc.
[L6] ```
[L7]
[L8] ---
[L9]
[L10] ## 7. Operations Layer — Layer 8
[L11]
```

[L12] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L13]

[L14] ### 7.1 Logging

DEEP RESEARCH 24/08/2026, 01:55:20

Citation Marker: `filecite turn30file1`

[L15] Find: find=1 ref=turn28file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=### 7.1 Logging

[L16] Scope: full file surface=conversation

[L17] Matches: shown=1 total=1

[L18] [match 0 | match_lines 1424 | shown_lines 1419-1429]

[L19]

[L20] ## 7. Operations Layer — Layer 8

[L21]

[L22] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L23]

[L24] ### 7.1 Logging

[L25]

[L26] Every query processed through the system generates a structured `QueryLog` entry:

[L27]

[L28] `python`

[L29] `@dataclass`

DEEP RESEARCH 24/08/2026, 01:55:20

Citation Marker: `filecite turn30file2`

[L30] Find: find=2 ref=turn28file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=### 7.2 Feedback Collection

[L31] Scope: full file surface=conversation

[L32] Matches: shown=0 total=0

[L33] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 24/08/2026, 01:55:20

Citation Marker: `filecite turn30file3`

[L34] Find: find=3 ref=turn28file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=### 7.3 Self-Evolution

[L35] Scope: full file surface=conversation

[L36] Matches: shown=1 total=1

[L37] [match 0 | match_lines 1465 | shown_lines 1460-1470]

[L38] **Safeguards:**

[L39] - Edge weight updates use a dampened learning rate ($\eta=0.02$, small increments)

[L40] - Layer 7 verification metrics (supported/unsupported claims) are tracked separately

[L41] - Anomaly detection: if `confidence > 0.8 AND feedback == "NOT_HELPFUL"`, log a warning (possible answer format issue, not retrieval issue)

[L42]

[L43] ### 7.3 Self-Evolution Update Rule

[L44]

[L45] After each query with feedback signal $f \in \{+1, -1\}$:

[L46]

[L47]
$$w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f) \quad \text{for all } e \in \text{traversal_path}$$

DEEP RESEARCH 24/08/2026, 01:55:20

Citation Marker: `filecite`turn30file4

[L48] Find: find=4 ref=turn28file0 name=SHIA_RAG_Final_Report(4)(1).md pattern=### 7.4 Monitoring

[L49] Scope: full file surface=conversation

[L50] Matches: shown=0 total=0

[L51] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 24/08/2026, 01:55:23

Citation Marker: `filecite`turn31file0

[L1] metadata:

[L2] ref_id: turn28file0

[L3] file_id: file_00000000dcc082119f935683a5e13eed

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4)(1).md

[L6] lines_shown: 1420-1739

[L7] total_file_lines: 2066

[L8] has_more: True

[L9] next_start_line: 1740

[L10] next_read: {"ref_id":"turn28file0","mode":"full","start_line":1740,"max_lines":320}

[L11] content:

[L12] ### 7. Operations Layer — Layer 8

[L13]

[L14] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L15]

[L16] ### 7.1 Logging

[L17]

[L18] Every query processed through the system generates a structured `QueryLog` entry:

[L19]

[L20]

```

[L21] @dataclass
[L22] class QueryLog:
[L23]     event_id: str
[L24]     timestamp: str
[L25]     query: str
[L26]     intent: str          # DEFINITION, COMPARISON, etc.
[L27]     complexity: str     # SIMPLE, MODERATE, COMPLEX
[L28]     seed_nodes: list
[L29]     candidate_count: int
[L30]     selected_count: int
[L31]     traversal_edges: list # For feedback attribution
[L32]     answer_length: int
[L33]     claim_count: int
[L34]     supported_claims: int
[L35]     unsupported_claims: int
[L36]     overall_confidence: float
[L37]     model_used: str
[L38]     latency_ms: float
[L39]     user_feedback: str = None # HELPFUL / NOT_HELPFUL (filled later)
[L40]

```

[L41]

[L42] ### 7.2 Feedback Acquisition & Attribution

[L43]

[L44] User feedback is collected via API:

[L45]

[L46]

```

[L47] POST /api/feedback { "event_id": "...", "feedback": "HELPFUL" | "NOT_HELPFUL" }
[L48]

```

[L49]

[L50] **Attribution strategy:** Feedback applies ONLY to edges in the recorded `traversal_path`. This isolates retrieval quality from generation quality.

[L51]

[L52] **Safeguards:**[L53] - Edge weight updates use a dampened learning rate ($\eta=0.02$, small increments)

[L54] - Layer 7 verification metrics (supported/unsupported claims) are tracked separately

[L55] - Anomaly detection: if `confidence > 0.8` AND `feedback == "NOT_HELPFUL"`, log a warning (possible answer format issue, not retrieval issue)

[L56]

[L57] ### 7.3 Self-Evolution Update Rule

[L58]

[L59] After each query with feedback signal $f \in \{+1, -1\}$:

[L60]

[L61]
$$w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f) \quad \text{for all } e \in \text{traversal_path}$$

[L62]

[L63] Where $\eta = 0.02$. Edge weights are clipped to $[0.1, 10.0]$.

[L64]

[L65]

```

[L66] import math
[L67] import random
[L68]
[L69] ETA = 0.02
[L70] WEIGHT_MIN, WEIGHT_MAX = 0.1, 10.0
[L71] RESTRUCTURE_THRESHOLD = 0.3
[L72] EPSILON = 0.1 #  $\epsilon$ -greedy exploration rate
[L73]
[L74] def update_forest_from_feedback(selected_nodes, user_feedback, forest):
[L75]     f = +1 if user_feedback == "HELPFUL" else -1
[L76]     path_edges = get_traversal_path_edges(selected_nodes, forest)
[L77]
[L78]     for edge in path_edges:
[L79]         edge.weight = edge.weight * math.exp(ETA * f)
[L80]         edge.weight = max(WEIGHT_MIN, min(edge.weight, WEIGHT_MAX))
[L81]         edge.access_count += 1
[L82]
[L83]         if f == -1 and edge.weight < RESTRUCTURE_THRESHOLD:
[L84]             flag_for_restructure(edge, forest)
[L85]
[L86]     if forest.total_queries % 100 == 0:
[L87]         rebalance_forest(forest)
[L88]     forest.total_queries += 1
[L89]

```

[L90]

[L91] **ϵ -Greedy Exploration:** To prevent the self-evolution loop from reinforcing wrong paths (e.g., an incorrect path gets positive feedback because the LLM produces a convincing-looking answer), retrieval includes ϵ -greedy exploration:

[L92]

[L93]

```

[L94] def select_traversal_path(seed_nodes, forest, budget):
[L95]     """Select nodes with  $\epsilon$ -greedy exploration.
[L96]
[L97]     With probability (1- $\epsilon$ ), follow highest-weight edges (exploitation).
[L98]     With probability  $\epsilon$ , follow random edges (exploration).
[L99]
[L100]     This prevents a wrong path from permanently dominating if it
[L101]     accidentally received positive feedback."""
[L102]     if random.random() < EPSILON:
[L103]         # EXPLORE: randomly traverse edges, ignoring weights
[L104]         return random_tree_traversal(seed_nodes, forest, budget)
[L105]     else:
[L106]         # EXPLOIT: use Tree Knapsack with utility (weight-based)
[L107]         return tree_knapsack_traversal(seed_nodes, forest, budget)
[L108]

```

[L109]

[L110] ### 7.4 Closing the Self-Evolution Loop

[L111]

[L112] The self-evolution loop is closed through the `utility()` function in Tree Knapsack:

[L113]

[L114]

```

[L115] User Feedback → edge.weight update → utility() uses edge.weight →
[L116] Tree Knapsack selects differently → future retrieval improves
[L117]

```

[L118]

[L119] This means edges along paths that consistently lead to helpful answers get higher weights, making those paths more likely to be selected in future queries.

[L120]

[L121] ### 7.5 Forest Restructuring

[L122]

[L123] When an edge's weight drops below `RESTRUCTURE_THRESHOLD` (0.3), it is flagged for restructuring. During periodic rebalance cycles (every 100 queries):

[L124]

[L125] 1. Re-run CPG++ to find new candidate parents for the child node

[L126] 2. Re-run PRE to rank candidates (excluding current parent)

[L127] 3. If a better parent is found, re-parent the node via HV validation

[L128] 4. If no better parent exists, reset edge weight to 1.0 (neutral)

[L129] 5. Re-propagate confidence after structural changes

[L130]

[L131] ### 7.6 Edge Weight Dynamics

[L132]

[L133] To prevent long-term drift:

[L134]

[L135] - **Normalization:** After every rebalance cycle, normalize all edge weights to mean=1.0 (preserves relative ordering)[L136] - **Decay:** Daily decay with rate 0.999 ensures old feedback gradually fades

[L137]

[L138] ### 7.7 Monitoring Metrics

[L139]

[L140] | Metric | Threshold | Alert Condition |

[L141] |-----|-----|-----|

[L142] | Average confidence | 0.4 | Drops below |

[L143] | Helpful feedback ratio | 60% | Drops below |

[L144] | Orphan node rate | 10% | Exceeds |

[L145] | P95 latency | 5000ms | Exceeds |

[L146] | Error rate | 5% | Exceeds |

[L147]

[L148] ---

[L149]

[L150] ## 8. Novel Contributions

[L151]

[L152] SHIA-RAG introduces five novel contributions to the RAG literature. Each is supported by a systematic literature review (see Section 2.3).

[L153]

[L154] ### 8.1 Novelty 1: Precedence-Constrained Tree Knapsack for Context Selection

[L155]

[L156] **Existing approach:** All current RAG systems use flat top-k selection or simple greedy knapsack for context packing.

[L157]

[L158] **SHIA-RAG approach:** A greedy approximation for the **Tree Knapsack** formulation where selecting a child node forces inclusion of its parent (precedence constraint):

[L159]

[L160]
$$\max \sum_{i=1}^n U_i \text{ s.t. } \sum_{i=1}^n C_i \leq B, \quad x_i \in \{0,1\} \quad \forall i$$

[L161]

[L162] The implementation uses a greedy utility-density heuristic rather than exact DP, as exact solutions are computationally prohibitive for large token budgets. The precedence-constrained formulation itself is the novelty.

[L163]

[L164] **Novelty level:** Incremental (Novel Application) — adaptation of the Tree Knapsack formulation from operations

research to RAG context selection.

[L165]

[L166] ---

[L167]

[L168] ### 8.2 Novelty 2: Self-Evolving Knowledge Forest via Retrieval Feedback

[L169]

[L170] **Existing approach:** All current Knowledge Graphs and hierarchies are static after construction.

[L171]

[L172] **SHIA-RAG approach:** The forest learns from user queries. When a retrieval succeeds (user satisfied), edge weights along the traversal path are boosted. When retrieval fails, the system flags the path for restructuring.

[L173]

[L174] **Update rule:** $w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f)$ where $\eta=0.02$ and $f \in \{+1, -1\}$

[L175]

[L176] The traversal path includes only edges from seed nodes to selected nodes (nodes included in the final context). Edge weights are clipped to $[0.1, 10.0]$ and periodically normalized.

[L177]

[L178] **Novelty level:** Moderately Novel — to the best of our knowledge, no prior work applies closed-loop learning to modify RAG knowledge structures.

[L179]

[L180] ---

[L181]

[L182] ### 8.3 Novelty 3: Multi-Objective Energy Minimization for Hierarchy Induction

[L183]

[L184] **Existing approach:** Hierarchy construction in NLP typically uses single-objective methods (e.g., maximize hypernym probability).

[L185]

[L186] **SHIA-RAG approach:** A formal energy minimization objective:

[L187]

[L188]
$$J(F) = \sum_{(P_i, N_i) \in \mathcal{F}} E(P_i, N_i) + \alpha \cdot R(F)$$

[L189]

[L190] Where:

[L191]
$$E(P, N) = \lambda_1 (1 - \cos(E_P, E_N)) + \lambda_2 \cdot \frac{\text{depth}(N)}{D_{\max}} + \lambda_3 (1 - \text{Conf}(P, N)) + \lambda_4 \cdot \sigma(\text{type_score}(N) - \text{type_score}(P))$$

[L192]

[L193] And $R(F)$ is a normalized regularization term:

[L194]
$$R(F) = \frac{\text{Var}(\{\text{depth}(n) : n \in \text{leaves}(F)\})}{D_{\max}^2}$$

[L195]

[L196] **Novelty level:** Moderately Novel — the combined formulation for RAG hierarchy induction is new.

[L197]

[L198] ---

[L199]

[L200] ### 8.4 Novelty 4: Adaptive Depth Traversal via Query Complexity Classification

[L201]

[L202] **Existing approach:** All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.

[L203]

[L204] **SHIA-RAG approach:** A query complexity classifier that dynamically adjusts traversal depth:

[L205]

[L206] | Query Type | Example | Traversal Strategy |

[L207] |-----|-----|-----|

[L208] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |

[L209] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |

[L210] | MULTI-HOP | "How does OSPF convergence affect reliability?" | Deep: Full path traversal + cross-links (depth=3+) |

[L211] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Root-level retrieval, expand selectively |

[L212]

[L213] **Novelty level:** Incremental — novel in the specific application to hierarchical RAG traversal depth.

[L214]

[L215] ---

[L216]

[L217] ### 8.5 Novelty 5: SHEF — SHIA-RAG Evaluation Framework

[L218]

[L219] **Existing approach:** Standard RAG benchmarks (MS MARCO, NaturalQuestions) only measure flat retrieval. No benchmark tests hierarchical retrieval quality.

[L220]

[L221] **SHIA-RAG approach:** A novel evaluation framework with 3 metric categories:

[L222]

[L223] ##### A. Retrieval Metrics

[L224] | Metric | Formula | What It Measures |

[L225] |-----|-----|-----|

[L226] | Hop Accuracy | correct multi-hop answers / total multi-hop queries | Multi-hop reasoning quality |

[L227] | Ancestor Recall | retrieved ancestors $\cap$ gold ancestors / gold ancestors | Correct concept chain retrieval |

[L228] | Context Coherence | Avg semantic similarity between consecutive context nodes | Logical coherence of assembled context |

[L229]

[L230] ##### B. Hierarchy Metrics

[L231] | Metric | Formula | What It Measures |

[L232] |-----|-----|-----|

[L233] | Parent Assignment Accuracy | correctly placed nodes / total nodes | Quality of hierarchy induction |

[L234] | Tree Edit Distance | Normalized TED vs gold taxonomy | Structural similarity to human taxonomy |

[L235] | Depth Distribution Score | KL-divergence from ideal depth distribution | Tree balance |

[L236]

[L237] ##### C. Forest Quality Metrics

[L238] | Metric | Formula | What It Measures |

[L239] |-----|-----|-----|

[L240] | Cross-Doc Fusion Accuracy | correctly merged nodes / total mergeable pairs | Entity resolution quality |

[L241] | Orphan Rate | root nodes without children / total root nodes | Isolated concept rate |

[L242] | Forest Density | edges / nodes | Knowledge forest connectivity |

[L243]

[L244] **Novelty level:** Incremental (Novel Combination) — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L245]

[L246] ### 8.6 Novelty Level Assessment Summary

[L247]

[L248] | # | Novelty | Level | Justification |

[L249] |---|-----|-----|-----|

[L250] | 1 | Tree Knapsack for Context Selection | **Incremental (Novel Application)** | Adaptation of existing OR technique to a new domain (RAG context selection) |

[L251] | 2 | Self-Evolving Knowledge Forest | **Moderately Novel** | Novel combination of retrieval feedback + hierarchy restructuring |

[L252] | 3 | Multi-Objective Energy Minimization | **Moderately Novel** | Combined formulation for RAG hierarchy induction is new |

[L253] | 4 | Adaptive Depth Traversal | **Incremental** | Novel application to hierarchical RAG traversal depth |

[L254] | 5 | SHEF Evaluation Framework | **Incremental (Novel Combination)** | Standard metrics adapted as a unified framework for hierarchical RAG |

[L255]

[L256] ---

[L257]

[L258] ## 9. Mathematical Framework

[L259]

[L260] ### 9.1 Hierarchy Energy (Global Objective)

[L261]

[L262] $J(\mathcal{F}) = \sum_{(P_i, N_i) \in \mathcal{F}} E(P_i, N_i) + \alpha \cdot R(\mathcal{F})$

[L263]

[L264] **Constraints:**

[L265] - $\lambda_1 + \lambda_2 + \lambda_3 + \lambda_4 = 1$

[L266] - Defaults: $\lambda_1 = 0.35, \lambda_2 = 0.20, \lambda_3 = 0.25, \lambda_4 = 0.20$

[L267] - $D_{\max}$ = configurable max depth (default: 8)

[L268] - α = regularization weight (default: 0.1)

[L269]

[L270] **Recommended α values by domain:**

[L271]

[L272] | Domain | α | Rationale |

[L273] |-----|-----|-----|

[L274] | CS Education | 0.10 | Moderate depth variance acceptable |

[L275] | Medical/Clinical | 0.20 | Deeper, more balanced hierarchies preferred |

[L276] | Legal | 0.05 | Flat, wide hierarchies are natural |

[L277]

[L278] ### 9.2 Parent Ranking Score (PRE)

[L279]

[L280] $\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}(P)}{D_{\max}} + w_4 \cdot S_{\text{evi}}(P, N) + w_5 \cdot \text{Conf}(P)$

[L281]

[L282] Where:

[L283] - $H(P, N)$ = Hypernym score (Hearst patterns + LLM fallback, returns 0.0 or 1.0)

[L284] - $S_{\text{evi}}(P, N)$ = Evidence support (co-occurrence + citation overlap)

[L285] - $w_1 = 0.30, w_2 = 0.25, w_3 = 0.15, w_4 = 0.20, w_5 = 0.10$

[L286]

[L287] ### 9.3 Tree Knapsack Formulation

[L288]

[L289] $\max \sum_{i=1}^n U_i \cdot x_i \quad \text{s.t.} \quad \sum_{i=1}^n C_i \cdot x_i \leq B, \quad x_i \leq x_{\text{parent}(i)} \quad \forall i$

[L290]

[L291] Where:

[L292] - U_i = `utility(node_i)` — quality score incorporating edge weights from self-evolution

[L293] - C_i = `token_cost(node_i)` — estimated tokens for context representation

[L294] - B = token budget (default: 4096)

[L295] - $x_i \in \{0, 1\}$ — selection variable

[L296]

[L297] ### 9.4 Self-Evolution Update

[L298]

[L299] After each query with feedback signal $F \in \{+1, -1\}$:

[L300]

[L301]
$$w_{e^{(t+1)}} = w_{e^{(t)}} \cdot \exp(\eta \cdot f) \quad \forall e \in \text{traversal_path}$$

[L302]

[L303] Where $\eta = 0.02$. Edge weights are clipped to $[0.1, 10.0]$.

[L304]

[L305] ### 9.5 Answer Confidence

[L306]

[L307]
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim_conf}}{\text{node_conf}} + 0.2 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L308]

[L309] ---

[L310]

[L311] ## 10. Implementation

[L312]

[L313] ### 10.1 Technology Stack

[L314]

[L315] | Component | Technology | Justification |

[L316] |-----|-----|-----|

[L317] | Language | Python 3.11+ | ML ecosystem, rapid prototyping |

[L318] | LLM | Llama 3.1 (8B) or GPT-4o-mini | Cost-effective extraction |

[L319] | Embeddings | `bge-base-en-v1.5` (asymmetric, 768-dim) | Better QA retrieval than symmetric models |

[L320] | Vector DB | FAISS (dev) → Milvus (prod) | Fast ANN search, $O(\log N)$ |

[L321] | Graph DB | Neo4j Community | Cypher queries for traversal |

[L322] | Relational DB | PostgreSQL | Documents, pages, blocks storage |

[L323] | Cache | Redis | Query cache, recent contexts |

[L324] | API Framework | FastAPI | Async, OpenAPI auto-docs |

[L325] | Containerization | Docker + Docker Compose | Easy deployment |

[L326]

[L327] ### 10.2 Project Structure

[L328]

[L329] ```

[L330] shia-rag/

[L331] |—— config/

CHATGPT 24/08/2026, 01:55:23

Bilkul. Ab hum **Layer 8 – Operations Layer** ko same depth mein samjhenge, **same running example** ke saath:

User Query: "Why is Time Quantum important in Round Robin?"

Layer 8 ko samajhne ke liye sabse important baat pehle samjho:

Layer 8 directly answer generate nahi karta aur retrieval bhi nahi karta. Layer 8 system ko observe karta hai, user feedback collect karta hai, aur us feedback se Knowledge Forest ko time ke saath improve karta hai.

Source architecture ke according, Layer 8 **Continuous Phase** ka part hai aur iska kaam logging, monitoring, feedback loop aur Knowledge Forest ko self-evolve karna hai. `filecite\turn28file0\L29-L35`

1. Layer 8 ko ek line mein samjho

Layer 8 = System ka Learning + Monitoring + Improvement Layer

Simple flow:

```

User asks a question
↓
Layer 6 retrieves knowledge
↓
Layer 7 generates and verifies answer
↓
User receives answer
↓
↓
LAYER 8
↓
Logs what happened
Collects user feedback
Checks system performance
Updates retrieval path weights
↓
Knowledge Forest improves over time
  
```

Source ke according Layer 8 ka responsibility hai: **all query events log karna, user feedback collect karna, edge weights update karna, system health monitor karna, aur performance evaluate karna.** [filecite\turn31file0\L12-L18](#)

2. Pehle samjho: Layer 8 ki zarurat kyu padi?

Suppose system ne 1 lakh queries answer ki.

Agar system har query ke baad **kuch learn hi nahi karta**, to kya hoga?

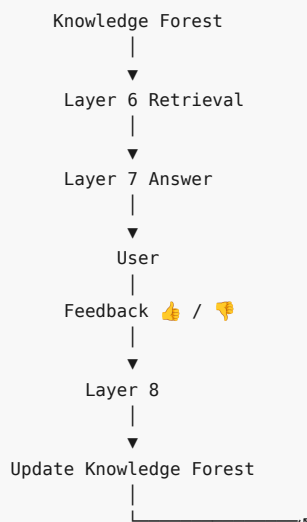
```

Query 1 → Answer → Finish ❌
Query 2 → Answer → Finish ❌
Query 3 → Answer → Finish ❌
  
```

System ko kabhi pata nahi chalega:

- Kaunsa retrieval path useful tha?
- Kaunsa path repeatedly bad answers de raha hai?
- Users kis type ke answers ko helpful bol rahe hain?
- System confidence low ho raha hai ya high?
- Response slow ho raha hai ya fast?
- Knowledge Forest mein kaunse relationships weak hain?

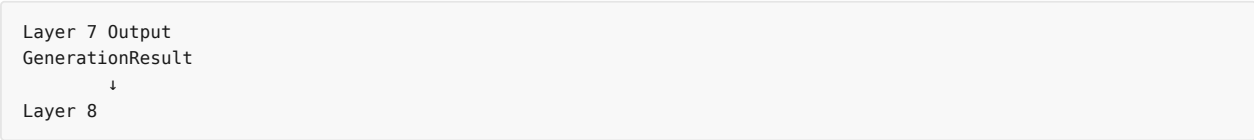
Isliye Layer 8 ek **closed feedback loop** banata hai.



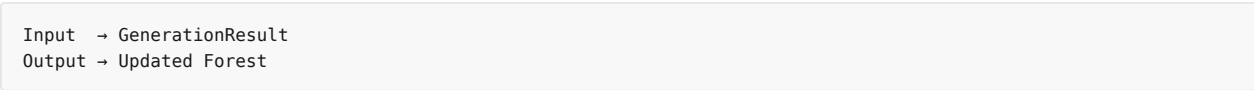
SHIA-RAG ka core idea hi hai ki user interactions ke basis par feedback loop Knowledge Forest ko update kare.

3. Layer 8 ka exact input kya hai?

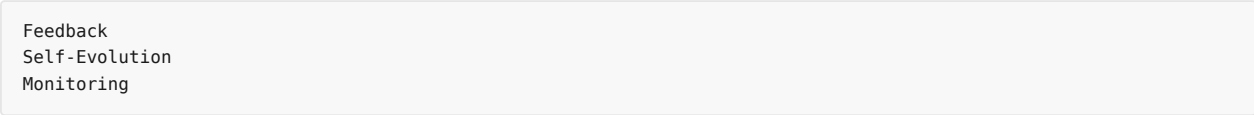
Official layer flow:



Layer summary ke according:



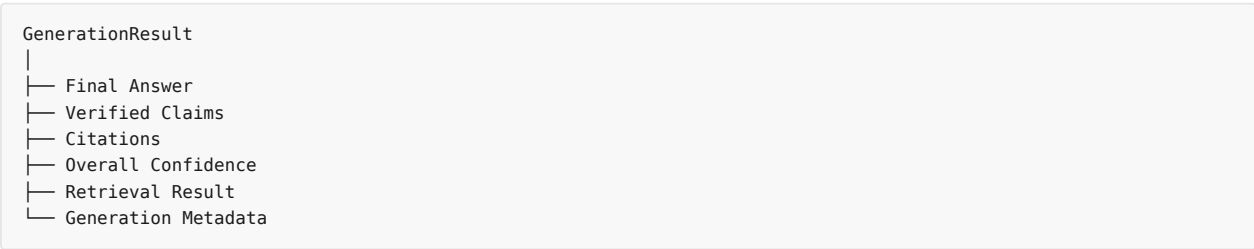
Aur main processes hain:



filecite[turn27file0L59-L71]

Layer 7 se GenerationResult mein kya-kya aata hai?

Conceptually:



Ye important hai kyunki Layer 8 ko sirf final answer nahi chahiye.

Usse **poori history** chahiye ki answer kaise bana.

For example:

Question:
Why is Time Quantum important in Round Robin?

Seed Nodes:
Round Robin
Time Quantum

Traversal Path:
Operating Systems
↓
CPU Scheduling
↓
Round Robin
↓
Time Quantum

Selected Nodes:
N1, N2, N3

Claims:
4

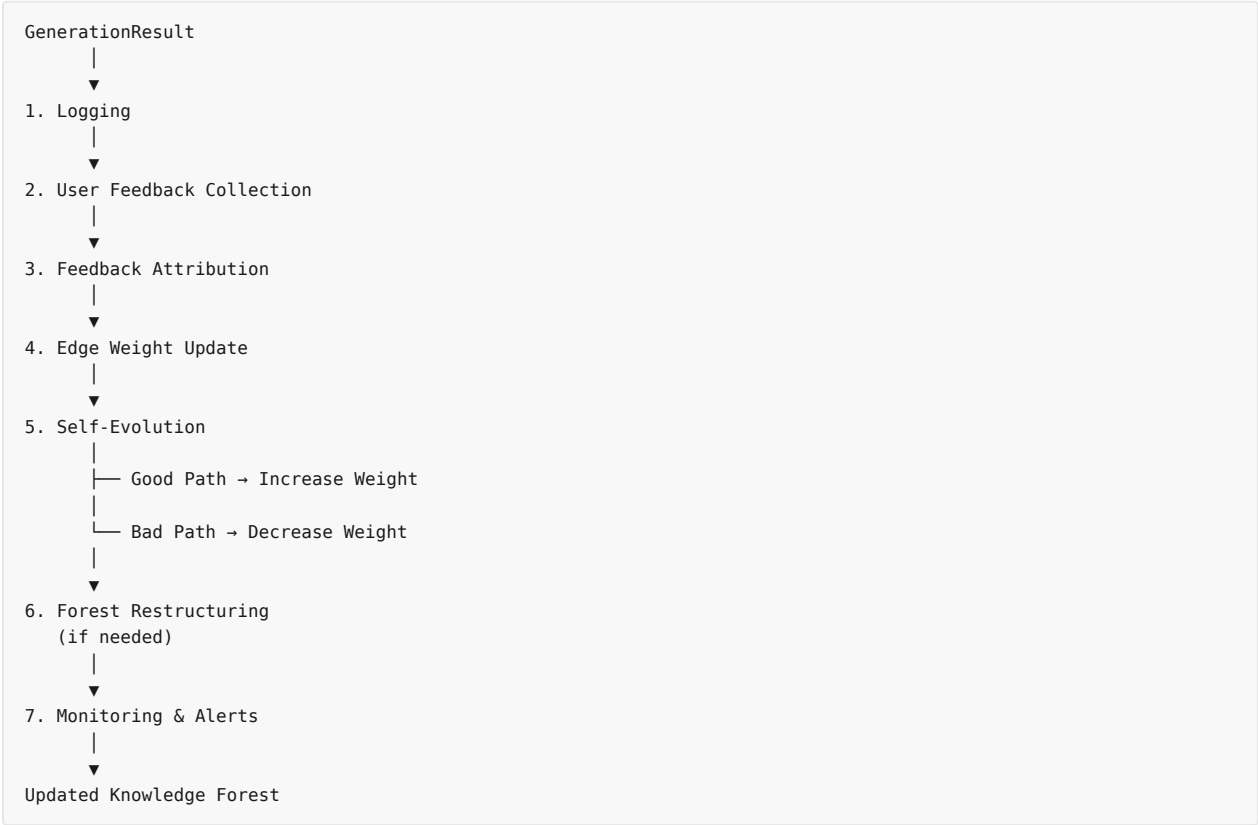
Supported Claims:
4

Unsupported Claims:
0

Overall Confidence:
0.86

Layer 7 ka retrieval_result Layer 8 ke liye preserve kiya jata hai. fileciteurn28file6L468-L480

4. Layer 8 ka complete pipeline



5. Step 1 — Logging

Sabse pehle Layer 8 har query ka record banata hai

Source mein isse **QueryLog** bola gaya hai.

Simple example:

```
QueryLog
|
├─ Event ID
├─ Timestamp
├─ User Query
├─ Query Intent
├─ Query Complexity
├─ Seed Nodes
├─ Candidate Count
├─ Selected Count
├─ Traversal Edges
├─ Claim Count
├─ Supported Claims
├─ Unsupported Claims
├─ Overall Confidence
├─ Model Used
├─ Latency
└─ User Feedback
```

Source **QueryLog** schema exactly query, intent, complexity, seed nodes, selected nodes, traversal edges, claim statistics, confidence, model, latency, aur user feedback store karta hai. `filecite\turn31file0\L16-L40`

Hamare example mein log kaisa dikhega?

User asked:

“Why is Time Quantum important in Round Robin?”

Layer 8 roughly record karega:

Event ID: Q1001

Query:
Why is Time Quantum important in Round Robin?

Intent:
DEFINITION / EXPLANATION

Complexity:
SIMPLE or MODERATE

Seed Nodes:
[Round Robin, Time Quantum]

Traversal Path:
Operating Systems
→ CPU Scheduling
→ Round Robin
→ Time Quantum

Selected Nodes:
3

Claim Count:
4

Supported Claims:
4

Unsupported Claims:
0

Overall Confidence:
0.86

Model Used:
GPT-4o-mini

Latency:
1200 ms

User Feedback:
Initially None

6. Har cheez log karne ki zarurat kyu?

Suppose user bolta hai:

👎 Not Helpful

Ab system ko problem investigate karni hai.

Question:

Problem retrieval mein thi ya answer generation mein?

Agar logging nahi hai:

Bad Answer

↓

We don't know why ❌

Logging ke saath:


```

Bad Answer
↓
Check Traversal Path
↓
Check Selected Nodes
↓
Check Claim Verification
↓
Check Confidence
↓
Find possible failure

```

This is very important.

Layer 8 system ko blind feedback nahi deta.

Woh historical information ke through feedback ko analyze karta hai.

7. Step 2 — User Feedback Collection

User answer ke baad feedback de sakta hai:

👍 HELPFUL

or

👎 NOT_HELPFUL

Source mein feedback API conceptually:

```

POST /api/feedback

{
  "event_id": "...",
  "feedback": "HELPFUL"
}

```

or

NOT_HELPFUL

📄filecite📄turn31file0📄L42-L48📄

Important question: Feedback kisko apply hoga?

Yahan Layer 8 ka ek very important design decision hai.

Feedback **poore Knowledge Forest ko blindly apply nahi hota.**

Sirf un edges ko apply hota hai jo us particular query ke:

Recorded traversal path

mein use hue the.

Source explicitly kehta hai:

Feedback applies only to edges in the recorded traversal path.

Iska purpose retrieval quality ko generation quality se isolate karna hai. 📄filecite📄turn31file0📄L42-L55📄

8. Feedback Attribution ko deeply samjho

Maan lo Knowledge Forest:

```

graph TD
    OS[Operating Systems] --> CS[CPU Scheduling]
    CS --> RR[Round Robin]
    RR --> TQ[Time Quantum]
  
```

User query ke liye Layer 6 ne ye path traverse kiya:

```

E1: OS → CPU Scheduling
E2: CPU Scheduling → Round Robin
E3: Round Robin → Time Quantum
  
```

Ab user feedback deta hai:

👍 HELPFUL

Layer 8 sirf:

```

E1
E2
E3
  
```

ko update karega.

Forest ke baaki unrelated edges ko touch nahi karega.

Why?

Suppose forest mein ye bhi hai:

```

graph TD
    OS[Operating Systems] --> MM[Memory Management]
    OS --> FS[File Systems]
  
```

User ne Round Robin ka question poocha tha.

Toh feedback:

```

Round Robin path → Update ✓
Memory Management path → Don't update ✗
File Systems path → Don't update ✗
  
```

Isse learning **targeted** banti hai.

9. Lekin ek problem hai — User feedback answer ke liye hota hai, retrieval ke liye directly nahi

Ye bahut important point hai.

Suppose:

User says:
NOT_HELPFUL

Iska matlab automatically ye nahi ki:

Retrieval path wrong tha.

Maybe:

- Retrieved context correct tha
- But LLM explanation poor thi
- Answer too short tha
- Answer confusing tha
- User expected a different format

Isliye Layer 8 ek safeguard use karta hai.

Source ke according:

Layer 7 verification metrics
supported / unsupported claims
are tracked separately

Aur agar:

Confidence > 0.8
AND
Feedback = NOT_HELPFUL

toh system warning log karta hai ki issue **answer format/generation ka ho sakta hai, retrieval ka zaruri nahi.**

filecite\turn31file0\L50-L55

10. Is safeguard ko example se samjho

Suppose:

Overall Confidence = 0.92

Supported Claims = 5/5

Citations = Complete

User Feedback = NOT_HELPFUL

Agar Layer 8 simply bole:

Negative feedback
↓
All retrieval edges decrease ❌

toh system galat learning karega.

Kyunki retrieval actually correct thi.

Maybe user chahta tha:

Simple answer

But system ne:

Long technical answer

de diya.

So Layer 8 warning dega:

```
High confidence
+
Negative feedback
  ↓
Possible generation/format issue
  ↓
Do not blindly blame retrieval
```

Ye design retrieval aur generation failure ko separate karne ki koshish karta hai. `filecite\turn31file0\L50-L55`

11. Step 3 — Self-Evolution

Ye Layer 8 ka sabse important aur novel part hai

Normal Knowledge Graph:

```
Build once
  ↓
Remain static
```

SHIA-RAG:

```
Build Forest
  ↓
Retrieve
  ↓
Get Feedback
  ↓
Learn
  ↓
Update Forest
  ↓
Retrieve Better Next Time
```

Source is mechanism ko **Self-Evolving Knowledge Forest via Retrieval Feedback** ke roop mein describe karta hai.

`filecite\turn31file0\L168-L178`

12. Edge weight kya hota hai?

Layer 5 mein nodes ke beech relationships/edges banaye gaye the.

For example:

```
CPU Scheduling
  |
  | weight = 1.0
  ▼
Round Robin
```

Weight represent karta hai ki:

Retrieval ke perspective se ye connection kitna useful/reliable raha hai.

Initially:

```
E1 = 1.0
E2 = 1.0
E3 = 1.0
```

13. Positive feedback par kya hota hai?

User:

👍 HELPFUL

Feedback signal:

$f = +1$

Source update rule:

New Weight
=
Old Weight $\times e^{(\eta \times f)}$

Where:

$\eta = 0.02$

filecite turn31file0 L57-L72

Simple numerical example

Suppose:

Old Edge Weight = 1.0

User feedback:

HELPFUL

Then approximately:

New Weight
=
 $1.0 \times e^{(0.02)}$

 ≈ 1.0202

So:

1.0
↓
1.0202

The increase is intentionally **small**, not huge.

Why?

Because one user's one feedback se system ko dramatically change nahi hona chahiye.

14. Negative feedback par kya hota hai?

Suppose:

Old Weight = 1.0

Feedback:
NOT_HELPFUL

$f = -1$

Then:

New Weight
=
 $1.0 \times e^{(-0.02)}$

 ≈ 0.9802

So:

1.0
↓
0.9802

Again, only a small decrease.

Source dampened learning rate $\eta = 0.02$ use karta hai. [filecite\turn31file0\L52-L63](#)

15. Why small updates?

Imagine agar system:

One negative feedback
↓
Weight: 1.0 → 0.0

kar de.

That would be dangerous.

Maybe:

- User accidentally clicked 🚫
- User disliked writing style
- Answer correct tha but too long tha

So system gradually learns:

Feedback 1 → Small change
Feedback 2 → Small change
Feedback 3 → Small change
...
Repeated pattern → Meaningful learning

This prevents one feedback from destroying the Knowledge Forest.

16. Weight limits kyu hain?

Source edge weights ko:

Minimum = 0.1

Maximum = 10.0

ke beech clip karta hai. `filecite\turn31file0\L69-L84`

So:

$$0.1 \leq \text{Edge Weight} \leq 10.0$$

Why maximum?

Suppose edge gets thousands of positive feedback.

Without limit:

$$\text{Weight} \rightarrow 1000 \rightarrow 10000 \rightarrow 100000$$

Ek path permanently dominate kar sakta hai.

Why minimum?

Suppose edge gets lots of negative feedback.

Without limit:

$$\text{Weight} \rightarrow 0$$

Phir path kabhi explore hi nahi hoga.

Maybe future queries ke liye woh path useful ho.

So bounded learning:

$$0.1 \leftarrow \text{minimum safety limit}$$

$$10.0 \leftarrow \text{maximum safety limit}$$

17. Step 4 — When does the forest restructure?

Suppose edge repeatedly bad feedback receive karta hai.

```
Initial:
weight = 1.0

Repeated negative feedback
  ↓
0.8
  ↓
0.6
  ↓
0.4
  ↓
0.29
```

Source restructuring threshold:

$$0.3$$

Agar:

$$\text{weight} < 0.3$$

then edge:

Flag for Restructuring

filecite\turn31file0\L69-L88

18. Forest restructuring mein kya hota hai?

Suppose current hierarchy hai:

```
Operating Systems
  ↓
CPU Scheduling
  ↓
Round Robin
  ↓
Time Quantum
```

Imagine system repeatedly learn karta hai ki:

Current Parent → Child relation
retrieval ke liye useful nahi hai

Toh periodic rebalance mein system:

Step 1

CPG++ dobara run karega.

Find alternative parent candidates

Step 2

PRE candidates ko rank karega.

Which parent is better?

Step 3

HV validate karega.

Is this new hierarchy structurally valid?

Step 4

Agar better parent mila:

Re-parent Node

Step 5

Agar better parent nahi mila:

Reset weight = 1.0

Step 6

Confidence propagation dobara hoti hai.

Source restructuring process exactly CPG++, PRE, HV validation, neutral reset, aur confidence re-propagation define karta hai.

filecite\turn31file0\L121-L129

19. Example of restructuring

Suppose initially:

```
CPU Scheduling
  ↓
Time Quantum
```

But over time feedback and structure analysis suggest:

```
Round Robin
  ↓
Time Quantum
```

is a more meaningful relationship.

Then Layer 8 can eventually trigger restructuring:

Before:

```
CPU Scheduling
  ↓
Time Quantum
```

After:

```
CPU Scheduling
  ↓
Round Robin
  ↓
Time Quantum
```

⚠ Important: Source ke according restructuring directly arbitrary nahi hoti. Low-weight edge pehle **flag** hota hai, phir periodic rebalance mein hierarchy algorithms dobara run hote hain. `filecite\turn31file0\L121-L129`

20. Rebalance kab hota hai?

Source implementation mein:

```
Every 100 queries
```

forest rebalance kiya jaata hai. `filecite\turn31file0\L74-L88`

So:

```
Query 1-99
  ↓
Small weight updates

Query 100
  ↓
Rebalance Forest
```

21. Bahut important problem — What if the system learns the wrong path?

Suppose ek wrong path:

```
A → B → C
```

accidentally positive feedback le raha hai.

Agar system sirf highest-weight path follow karta raha:

```

Wrong path
↓
Gets selected
↓
Gets more positive feedback
↓
Weight increases
↓
Selected even more
↓
Wrong path dominates ❌

```

This is called a kind of **feedback loop reinforcement problem**.

22. Solution — ϵ -Greedy Exploration

Source:

```
 $\epsilon = 0.1$ 
```

Meaning:

90% of the time

```
EXPLOIT
```

System highest-weight/best known path use karta hai.

10% of the time

```
EXPLORE
```

System alternate/random edges explore karta hai.

📄filecite📄turn31file0📄L91-L108📄

23. Simple analogy

Maan lo tumhe ek city mein restaurant dhundhna hai.

Exploitation:

```

90% time
→ Go to your favourite restaurant

```

Exploration:

```

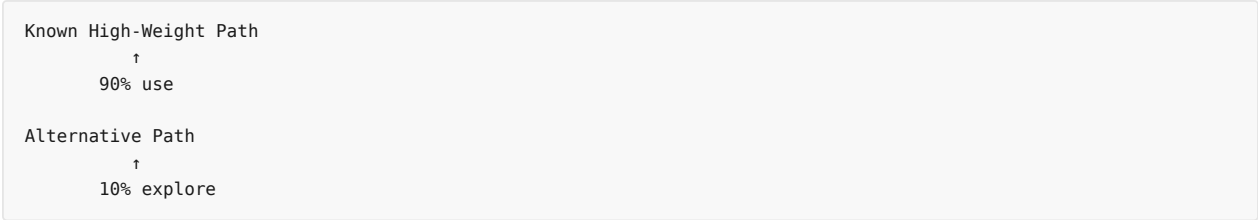
10% time
→ Try a new restaurant

```

Why?

Maybe new restaurant is actually better.

Same:



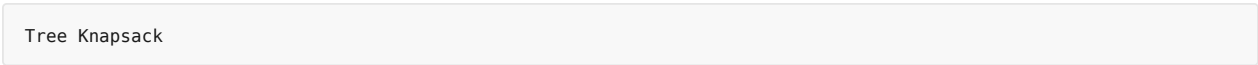
This prevents the Knowledge Forest from becoming permanently biased toward an accidentally successful path.

filecite[turn31file0][L91-L108]

24. Step 5 — How does feedback affect future retrieval?

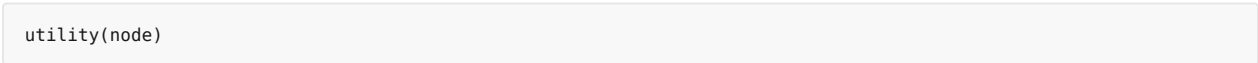
Ye Layer 8 aur Layer 6 ka connection hai.

Remember Layer 6:



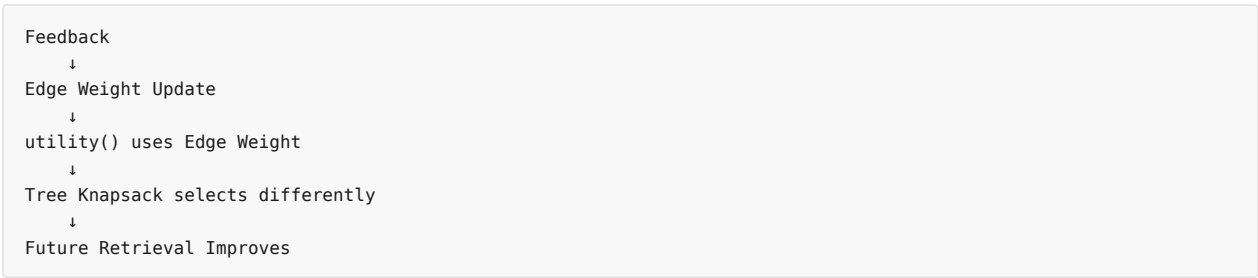
use karta hai.

Tree Knapsack mein:



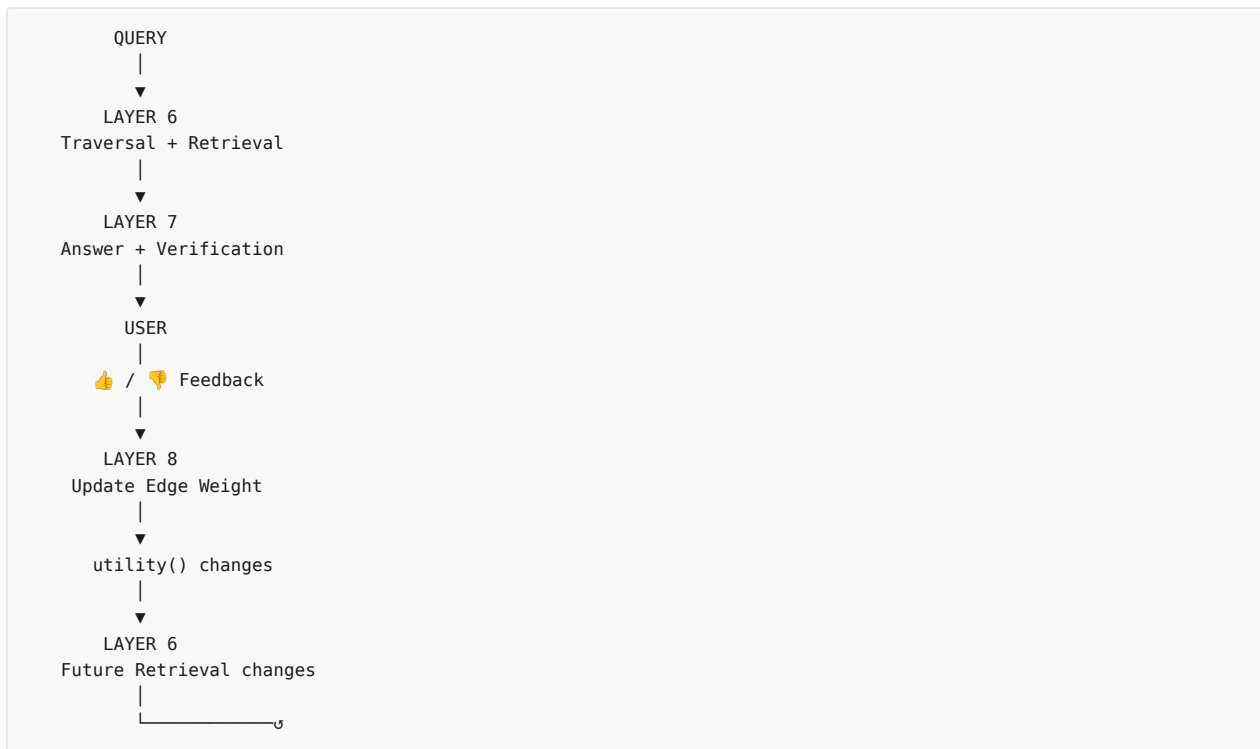
important hota hai.

Source ke according:



filecite[turn31file0][L110-L119]

Complete feedback loop



This is the actual Self-Evolution Loop.

25. Step 6 — Weight normalization

Suppose over months:

```

Some weights = 9.8
Some weights = 0.11
Some weights = 3.4
  
```

Long-term drift ho sakta hai.

Source safeguard:

```

After every rebalance cycle:
Normalize all edge weights
Mean = 1.0
  
```

This preserves relative ordering while preventing uncontrolled drift. [filecite\turn31file0\L131-L136](#)

26. Daily decay

Old feedback forever equally important nahi hona chahiye.

Example:

```

Feedback from 1 year ago
  
```

may not represent current usage patterns.

So source daily decay use karta hai:

```

Decay Rate = 0.999
  
```

Meaning:

Old feedback gradually fades.

fileciteurn31file0L131-L136

27. Step 7 — Monitoring

Layer 8 system health bhi monitor karta hai.

Source-defined metrics:

Metric	Threshold	Alert
Average Confidence	0.4	Below this
Helpful Feedback Ratio	60%	Below this
Orphan Node Rate	10%	Above this
P95 Latency	5000 ms	Above this
Error Rate	5%	Above this

fileciteurn31file0L138-L146

28. Har monitoring metric ka matlab

A. Average Confidence

Question:

System ke answers generally kitne reliable hain?

If:

Average Confidence = 0.35

Threshold:

0.4

Then:

⚠️ Alert

Possible issue:

- Retrieval weak
- Claims unsupported
- Evidence poor

B. Helpful Feedback Ratio

Suppose:

100 users gave feedback
70 → Helpful
30 → Not Helpful

Then:

Helpful Ratio = 70%

Good.

But:

55%

then threshold 60% se below:

⚠ Alert

Source monitoring threshold 60% specify karta hai. `filecite\turn31file0\L138-L146`

C. Orphan Node Rate

Orphan node = aisa node jo Knowledge Forest mein properly connected nahi hai.

Too many orphan nodes:

Knowledge exists
✗
But retrieval path is weak

Threshold:

10%

If higher:

⚠ Forest structure issue

D. P95 Latency

P95 latency ka simple meaning:

95% requests is time ke andar complete honi chahiye.

Threshold:

5000 ms
= 5 seconds

If exceeded:

⚠ System is becoming slow

E. Error Rate

If:

More than 5% queries fail

then:

⚠ System Alert

`filecite\turn31file0\L138-L146`

29. Layer 8 mein kya technologies use ho sakti hain?

Source ke overall implementation stack mein:

Purpose	Technology
Main language	Python 3.11+
Graph / Forest	Neo4j
Vector storage	FAISS → Milvus
Relational logs/data	PostgreSQL
Cache	Redis
API / Feedback endpoint	FastAPI
Deployment	Docker + Docker Compose

filecite\turn31file0\L311-L326

Layer 8 ke context mein simple mapping:

```
graph TD
    UserFeedback[User Feedback] --> FastAPI[FastAPI]
    FastAPI --> QueryLogStorage[QueryLog Storage]
    QueryLogStorage --> PostgreSQL[PostgreSQL]
    PostgreSQL --> UpdateForestEdge[Update Forest Edge]
    UpdateForestEdge --> Neo4j[Neo4j]
```

Aur:

```
graph TD
    Monitoring[Monitoring / Metrics] --> AppMonitoring[Application Monitoring Layer]
```

Note: Report overall technology stack define karta hai; har individual Layer 8 subcomponent ke liye exact separate tool name specify nahi karta. Isliye upar mapping architecture ke stated stack par based hai. filecite\turn31file0\L311-L326

30. Hamare complete example ko start-to-end dekhte hain

User asks:

“Why is Time Quantum important in Round Robin?”

Layer 6

```
Seed Nodes:
Round Robin
Time Quantum

Traversal:
Operating Systems
→ CPU Scheduling
→ Round Robin
→ Time Quantum
```

Layer 7

Generate Answer
↓
Extract 4 Claims
↓
Verify 4 Claims
↓
4 Supported
↓
Confidence = 0.86

User gets answer

Time Quantum is important in Round Robin because...

Layer 8 — Logging

Query: Stored ✓
Path: Stored ✓
Claims: Stored ✓
Confidence: Stored ✓
Latency: Stored ✓

User feedback

👍 HELPFUL

Feedback Attribution

Only these edges:

OS → CPU Scheduling
CPU Scheduling → Round Robin
Round Robin → Time Quantum

get feedback.

Self-Evolution

Edge Weight
1.0
↓
1.02

small positive update.

Future query

Next time similar query aaye:

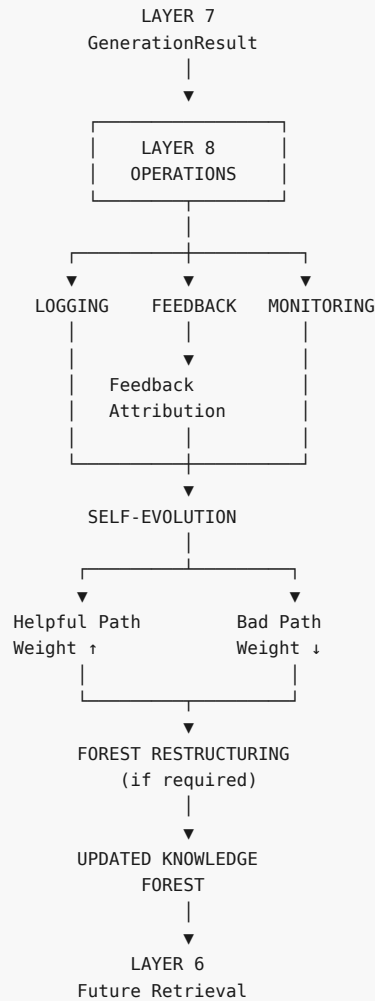
"Explain Time Quantum in Round Robin"

Tree Knapsack utility calculation mein improved edge weights consider honge.

Higher useful path weight
 ↓
 Higher chance of selecting path
 ↓
 Potentially better retrieval

Yahi closed loop hai. [filecite\turn31file0\L110-L119](#)

31. Layer 8 ka complete architecture



32. Layer 8 ka sabse important concept

Layers 1–5

Build Knowledge Forest

Layer 6

Retrieve Knowledge

Layer 7

Generate + Verify Answer

Layer 8

Learn from what happened

So:

Layers 1–5 = Build

Layers 6–7 = Use

Layer 8 = Learn and Improve

33. Layer 8 ka input-output contract

Input

GenerationResult
+
User Feedback

GenerationResult se:

Answer
Claims
Confidence
Retrieval Path
Selected Nodes
Generation Metadata

milta hai.

User se:

HELPFUL
or
NOT_HELPFUL

milta hai.

Internal Processing

1. Create QueryLog
2. Store query history
3. Collect feedback
4. Identify exact traversal edges
5. Separate retrieval issues from generation issues
6. Update edge weights
7. Detect low-performing paths
8. Explore alternative paths
9. Restructure forest if needed
10. Normalize and decay weights
11. Monitor health metrics

Output

Updated Knowledge Forest
+
Updated Edge Weights
+
Query History
+
Monitoring Metrics
+
Alerts / Restructuring Flags

Official layer summary mein Layer 8 ka output **Updated Forest** aur key processes **Feedback, self-evolution, monitoring** hain.
filecite turn27file0 L61-L71

34. Possible errors aur safeguards

Problem 1 — Wrong feedback se wrong learning

Wrong path gets 👍
↓
Weight increases

Safeguard:

ε-Greedy Exploration

10% alternate paths explore hote hain. filecite turn31file0 L91-L108

Problem 2 — One negative feedback destroys a path

Safeguard:

Small learning rate
η = 0.02

Small gradual updates. filecite turn31file0 L52-L63

Problem 3 — Weight becomes too high or too low

Safeguard:

0.1 ≤ Weight ≤ 10.0

filecite turn31file0 L59-L84

Problem 4 — Negative feedback was actually due to bad answer formatting

Safeguard:

Track separately:

Supported Claims
Unsupported Claims
Confidence

High confidence + NOT_HELPFUL feedback triggers warning rather than blindly blaming retrieval. filecite turn31file0 L50-L55

Problem 5 — Old feedback dominates forever

Safeguard:

```
Daily Decay = 0.999
```

Old feedback gradually loses influence.

Problem 6 — Forest becomes structurally poor

Safeguard:

```
Low-weight edge
  ↓
Flag for restructuring
  ↓
Periodic Rebalance
  ↓
CPG++ + PRE + HV
  ↓
Re-parent if better
```

35. Normal RAG vs SHIA-RAG Layer 8

Traditional/static approach

```
Documents
  ↓
Build Index
  ↓
Retrieve
  ↓
Answer
  ↓
Finish
```

× System does not structurally learn from user feedback.

SHIA-RAG

```
Build Knowledge Forest
  ↓
Retrieve
  ↓
Generate Verified Answer
  ↓
User Feedback
  ↓
Update Traversal Path
  ↓
Improve Edge Weights
  ↓
Restructure if necessary
  ↓
Better Future Retrieval
  ↻
```

This closed-loop self-evolution is one of the project's key proposed contributions.

36. Final summary table

Point Layer 8
--- ---
Name Operations Layer
Phase Continuous / Background
Input GenerationResult + User Feedback
Main Goal Monitor, learn, and improve the Knowledge Forest
Step 1 Query Logging
Step 2 Feedback Collection
Step 3 Feedback Attribution
Step 4 Edge Weight Update
Step 5 Self-Evolution
Step 6 Forest Restructuring
Step 7 Monitoring & Alerts
Learning Rate $\eta = 0.02$
Exploration $\epsilon = 0.1$
Weight Range 0.1 to 10.0
Restructure Threshold 0.3
Rebalance Every 100 queries
Output Updated Knowledge Forest
Main Novelty Closed-loop self-evolution

37. Sabse simple real-world analogy

Imagine:

Layer 6 = Librarian

Relevant books find karta hai.

Layer 7 = Writer + Fact Checker

Books se answer banata aur verify karta hai.

Layer 8 = Quality Manager + Learning System

Har interaction ke baad dekhta hai:

Did the user find it helpful?

↓

Which knowledge path was used?

↓

Was the answer well supported?

↓

Which paths are working?

↓

Which paths are failing?

↓

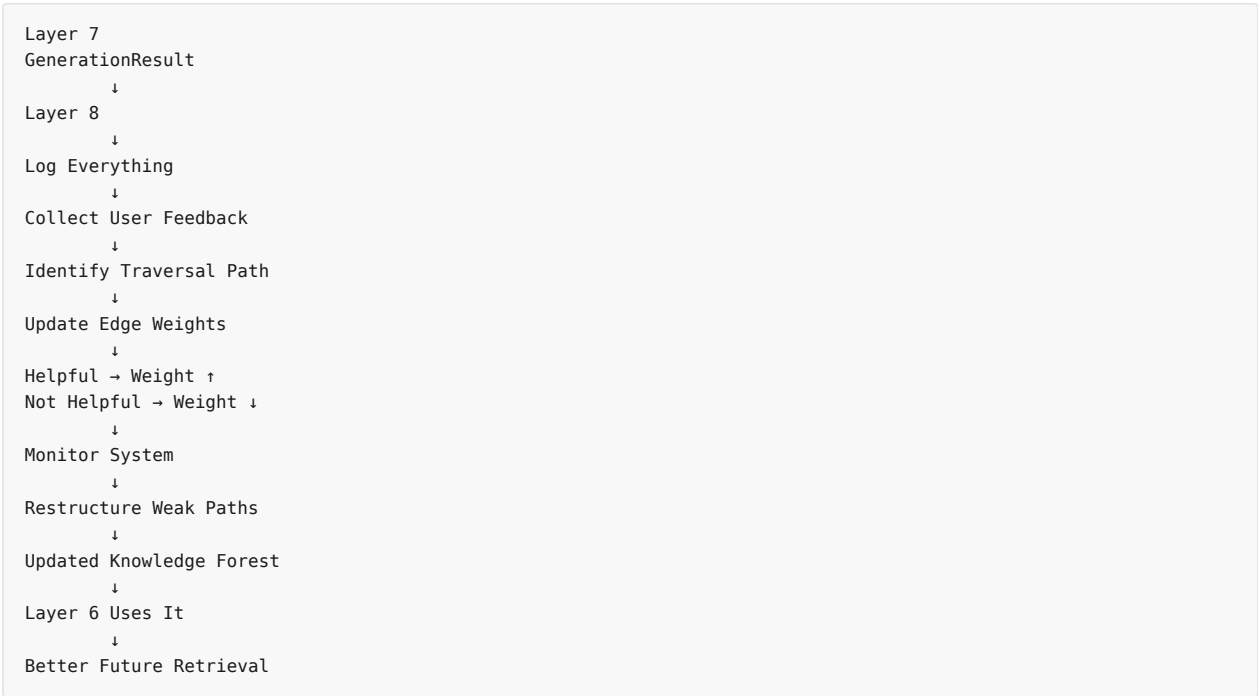
Improve the system for next time

38. Guide ko explain karne ke liye main one-line answer

“Layer 8 is the continuous learning and operations layer of SHIA-RAG. It logs every query and retrieval path, collects user feedback, identifies which knowledge relationships contributed to the answer, updates their weights based on success or

failure, monitors system health, and periodically restructures weak parts of the Knowledge Forest. This creates a closed feedback loop that allows retrieval to improve over time."

39. Sabse important final flow yaad rakho



In one sentence:

Layer 8 closes the loop — it converts user interaction and system performance data into feedback that continuously improves the Knowledge Forest and future retrieval decisions.

YOU 24/08/2026, 02:02:35

mujhe layer 8 mere guide ko smjhana hai mujhe basic understanding deni hai mere sir ko toh tum mujhe layer 8 ko english me btao lekin brife me sath me ek image generate kro taki me aur ache se smjha sku

CHATGPT 24/08/2026, 02:02:41

{"size":"1024x1024","n":1,"transparent_background":false,"is_style_transfer":false}

CHATGPT 24/08/2026, 02:03:24

Layer 8 – Operations Layer

You can explain it to your guide like this:

"Layer 8 is the continuous learning and monitoring layer of SHIA-RAG. It receives the GenerationResult from Layer 7, logs the complete query process, collects user feedback, and monitors system performance. Based on the feedback, it updates the weights of the retrieval paths and restructures weak relationships when necessary. Therefore, the Knowledge Forest can continuously improve over time."

Simple Flow



In one simple line

“Layer 8 closes the feedback loop by learning from user interactions and continuously improving the Knowledge Forest for better future retrieval.”

YOU 24/08/2026, 02:08:55

kya tum images generate kr skte ho jisme bss ek example ho from layer 1 to layer 8
kese ek example layer 1 se layer 8 tk jata jata hai kya changes se gujarata hai
tum ek image me concise krne ki kosis mt krna tum 3-4 image bna skte ho taki ache se smjh me aa ske

CHATGPT 24/08/2026, 02:09:02

{"skipped_mainline":true}

YOU 31/08/2026, 09:47:09

Mujhe hrr mahine ek monthly progress report submit krni padti hai
Mene last month literature survey and problem statement likha tha
Mujhe ab iss mahine kya likhna chahiye
Give me in english and 3-4 points

CHATGPT 31/08/2026, 09:47:11

Since last month you completed **Literature Survey and Problem Statement**, this month's progress should show that you moved from **problem identification** → **proposed solution/architecture design**.

Monthly Progress – This Month

- 1. **Proposed the SHIA-RAG architecture** and defined its overall workflow from document ingestion to answer generation and continuous improvement.
- 1. **Designed and studied the Layer 0–8 architecture**, defining the role, input, processing, and output of each layer.
- 1. **Developed the Knowledge Forest and retrieval framework concept**, including hierarchical knowledge organization, semantic relationships, and adaptive retrieval.
- 1. **Studied the self-evolution mechanism**, including user feedback, retrieval-path weight updates, monitoring, and restructuring of the Knowledge Forest.

These points accurately represent a natural progression after your **Literature Survey + Problem Statement** phase.

YOU 31/08/2026, 09:47:46

Reduce the content

CHATGPT 31/08/2026, 09:47:49

Monthly Progress

- Proposed the **SHIA-RAG architecture**.
- Designed the **Layer 0–8 workflow**.
- Developed the **Knowledge Forest and retrieval framework**.
- Studied the **self-evolution and feedback mechanism**.

YOU 31/08/2026, 09:48:06

Give me in short paragraph

CHATGPT 31/08/2026, 09:48:08

This month, I worked on **designing the SHIA-RAG architecture**, including the Layer 0–8 workflow, Knowledge Forest construction, adaptive retrieval, and the **self-evolution mechanism based on user feedback**.

YOU 31/08/2026, 09:48:37

Increase little bit content

CHATGPT 31/08/2026, 09:48:40

This month, I worked on **designing the SHIA-RAG architecture** and defining the Layer 0–8 workflow. I focused on **Knowledge Forest construction, adaptive retrieval, answer generation, and the self-evolution mechanism** based on user feedback. I also studied the interaction between different layers and how the system can continuously improve its retrieval performance.

YOU 02/09/2026, 09:15:38

mujhe ye btao hum knowledge edge aur knowledge node kese bnate hai

CHATGPT 02/09/2026, 09:15:44

`{"paths":["files"],"query":"search"}`

DEEP RESEARCH 02/09/2026, 09:15:47

Citation Marker: `filecite`turn32file0

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_00000000dcc082119f935683a5e13eed
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report(4)(1).md
- [L6] mime_type: text/markdown
- [L7] surface: conversation

[L8] score: 0.032266458495966696
[L9] snippet:
[L10]

```
[L11] graph TD
[L12]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L13]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L14]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L15]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L16]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L17]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L18]     end
[L19]
[L20]     subgraph "Phase 2: Online – Retrieval & Generation"
[L21]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L22]         KF --> L6
[L23]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L24]         L7 --> A["Answer + Citations"]
[L25]     end
[L26]
[L27]     subgraph "Phase 3: Continuous – Operations"
[L28]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L29]         A --> L8
[L30]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L31]     end
[L32]
```

[L33]
[L34] ### 3.2 Data Flow
[L35]
[L36]

```
[L37] Raw Document (PDF/DOCX/HTML)
[L38]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L39] Normalized Text + Metadata
[L40]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L41] [Page → Region → Block] hierarchy
[L42]     ↓ Layer 3: Split sentences, extract concepts/definitions/rerelations
[L43] KnowledgeUnit objects (candidate concepts)
[L44]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L45] KnowledgeNode objects (canonical, deduplicated)
[L46]     ↓ Layer 4: Validate relations against ontology, score confidence
[L47] KnowledgeEdge objects (typed, scored)
[L48]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L49] Knowledge Forest (rooted trees + cross-links)
[L50]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L51] Selected context nodes (within token budget)
[L52]     ↓ Layer 7: Build prompt, invoke LLM
[L53]     ↓ Layer 7: Verify claims against evidence, bind citations
[L54] Answer with citations + confidence score
[L55]     ↓ Layer 8: Log, collect feedback, update forest weights
[L56]
```

[L57]
[L58] ### 3.3 Layer Summary
[L59]
[L60] | Layer | Name | Input | Output | Key Algorithms |
[L61] |-----|-----|-----|-----|-----|
[L62] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L63] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L64] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L65] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L66] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L67] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L68] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L69] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L70] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L71]

[L72] ---

[L73]

[L74] ## 4. Data Model — Layer 0

DEEP RESEARCH 02/09/2026, 09:15:47

Citation Marker: `filecite`turn32file1

[L75] metadata:

[L76] query_result_index: 1

[L77] file_id: file_00000000dcc082119f935683a5e13eed

[L78] version_id: 1

[L79] name: SHIA_RAG_Final_Report(4)(1).md

[L80] mime_type: text/markdown

[L81] surface: conversation

[L82] score: 0.03177805800756621

[L83] snippet:

[L84] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L85]

[L86] ### 2.3 Literature Search Methodology

[L87]

[L88] | Parameter | Details |

[L89] |-----|-----|

[L90] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L91] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L92] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L93] | **Time Range** | 2020–2026 |

[L94] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L95]

[L96] ---

[L97]

[L98] ## 3. System Architecture

[L99]

[L100] ### 3.1 Overview

[L101]

[L102] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L103]

[L104] | Phase | Layers | Mode | Purpose |

[L105] |-----|-----|-----|-----|

[L106] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L107] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |
[L108] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |
[L109]
[L110] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.
[L111]
[L112]

```
[L113] graph TD
[L114]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L115]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text normalization, tokenization"]
[L116]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L117]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L118]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L119]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L120]     end
[L121]
[L122]     subgraph "Phase 2: Online – Retrieval & Generation"
[L123]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L124]         KF --> L6
[L125]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L126]         L7 --> A["Answer + Citations"]
[L127]     end
[L128]
[L129]     subgraph "Phase 3: Continuous – Operations"
[L130]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L131]         A --> L8
[L132]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L133]     end
[L134]
```

[L135]
[L136] ### 3.2 Data Flow
[L137]
[L138] ```
[L139] Raw Document (PDF/DOCX/HTML)
[L140] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L141] Normalized Text + Metadata
[L142] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L143] [Page → Region → Block] hierarchy
[L144] ↓ Layer 3: Split sentences,

DEEP RESEARCH 02/09/2026, 09:15:47

Citation Marker: `filecite`turn32file2

[L145] metadata:
[L146] query_result_index: 2
[L147] file_id: file_00000000dcc082119f935683a5e13eed
[L148] version_id: 1
[L149] name: SHIA_RAG_Final_Report(4)(1).md
[L150] mime_type: text/markdown
[L151] surface: conversation
[L152] score: 0.030798389007344232
[L153] snippet:
[L154] deduplicated)
[L155] ↓ Layer 4: Validate relations against ontology, score confidence
[L156] KnowledgeEdge objects (typed, scored)

[L157] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L158] Knowledge Forest (rooted trees + cross-links)
[L159] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L160] Selected context nodes (within token budget)
[L161] ↓ Layer 7: Build prompt, invoke LLM
[L162] ↓ Layer 7: Verify claims against evidence, bind citations
[L163] Answer with citations + confidence score
[L164] ↓ Layer 8: Log, collect feedback, update forest weights
[L165]

```
[L166]
[L167] ### 3.3 Layer Summary
[L168]
[L169] | Layer | Name | Input | Output | Key Algorithms |
[L170] |-----|-----|-----|-----|-----|
[L171] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L172] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L173] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L174] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L175] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L176] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L177] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L178] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L179] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L180]
[L181] ---
[L182]
[L183] ## 4. Data Model – Layer 0
[L184]
[L185] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L186]
[L187] ### 4.1 KnowledgeNode Schema
[L188]
[L189] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L190]
[L191]
```

json

```
[L192] {
[L193]   "node_id": "NODE_000789",
[L194]   "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L195]   "canonical_name": "Round Robin",
[L196]   "aliases": ["RR", "Round-Robin Scheduling"],
[L197]   "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L198]   "summaries": ["CPU scheduling method using time slices"],
[L199]   "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L200]   "domain": "Operating Systems",
[L201]   "properties": {
[L202]     "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L203]     "preemptive": {"type": "boolean", "value": true}
[L204]   },
[L205]   "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L206]   "parent": "NODE_000456",
[L207]   "children": ["NODE_001001", "NODE_001002"],
[L208]   "related": ["NODE_000890"],
[L209]   "incoming_edges": ["EDGE_001"],
[L210]   "outgoing_edges": ["EDGE_002", "EDGE_003"],
}
```

```
[L211] "evidence": [
[L212] {
[L213] "evidence_type": "DEFINITION",
[L214] "text": "Round Robin is a preemptive scheduling algorithm...",
[L215] "source_doc": "DOC_000001",
[L216] "source_block": "BLK_000234",
[L217] "page": 42
[L218] }
[L219] ],
```

DEEP RESEARCH 02/09/2026, 09:15:47

Citation Marker: `filecite[turn32file3]`

```
[L220] metadata:
[L221] query_result_index: 3
[L222] file_id: file_00000000dcc082119f935683a5e13eed
[L223] version_id: 1
[L224] name: SHIA_RAG_Final_Report(4)(1).md
[L225] mime_type: text/markdown
[L226] surface: conversation
[L227] score: 0.03055037313432836
[L228] snippet:
[L229] Select the relation_type with highest weighted vote
[L230] 4. Log conflicts for manual review
[L231] ""
[L232] from collections import defaultdict
[L233]
[L234] # Group by node pair
[L235] pair_groups = defaultdict(list)
[L236] for edge in candidate_edges:
[L237] key = (edge.source_node, edge.target_node)
[L238] pair_groups[key].append(edge)
[L239]
[L240] resolved = []
[L241] for (src, tgt), edges in pair_groups.items():
[L242] if len(edges) == 1:
[L243] resolved.append(edges[0])
[L244] continue
[L245]
[L246] # Tally weighted votes per relation type
[L247] votes = defaultdict(float)
[L248] evidence_pool = defaultdict(list)
[L249] for edge in edges:
[L250] weight = edge.confidence * len(edge.evidence)
[L251] votes[edge.relation_type] += weight
[L252] evidence_pool[edge.relation_type].extend(edge.evidence)
[L253]
[L254] # Select winning relation type
[L255] winner = max(votes, key=votes.get)
[L256]
```

```

[L257] # Log conflict if close contest
[L258] sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L259] if len(sorted_votes) > 1:
[L260] margin = sorted_votes[0][1] - sorted_votes[1][1]
[L261] if margin < 0.2: # Close contest
[L262] log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L263]
[L264] winning_edge = edges[0]
[L265] winning_edge.relation_type = winner
[L266] winning_edge.evidence = evidence_pool[winner]
[L267] winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L268] winning_edge.metadata['conflict_resolved'] = True
[L269] winning_edge.metadata['alternative_relations'] = [
[L270] (rt, score) for rt, score in sorted_votes if rt != winner
[L271] ]
[L272] resolved.append(winning_edge)
[L273]
[L274] return resolved
[L275]

```

```

[L276]
[L277] ---
[L278]
[L279] ### 5.5 Layer 5: SHIA Core – Hierarchy Construction
[L280]
[L281] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline
[L282]
[L283] **Input:** `KnowledgeNode[]` + `KnowledgeEdge[]` → **Output:** Knowledge Forest
[L284]
[L285] #### Pipeline Overview
[L286]
[L287]

```

mermaid

```

[L288] graph LR
[L289] N["New Node N"] --> CPG["CPG++<br/>Generate ≤5<br/>parent candidates"]
[L290] CPG --> PRE["PRE<br/>Rank ALL candidates<br/>(sorted list)"]
[L291] PRE --> HV["HV<br/>Validate #1?"]
[L292] HV -->|"Pass"| FI["FI<br/>Insert into Forest"]
[L293] HV -->|"Fail"| HV2["HV<br/>Validate #2?"]
[L294] HV2 -->|"Pass"| FI
[L295] HV2 -->|"Fail"| HV3["HV<br/>Validate #3?"]
[L296] HV3 -->|"Pass"| FI
[L297] HV3 -->|"All Fail"| ROOT["FI<br/>Insert as New Root"]
[L298] FI --> CLD["CLD<br/>Discover Cross-Links"]
[L299] CLD --> DONE["Node Placed ✓"]
[L300]

```

```

[L301]
[L302] #### 5.5.1 CPG++ – Candidate Parent Generator
[L303]
[L304] Generates up to 5 candidate parents for a new node using a multi-stage filter:
[L305]
[L306]

```

```
python
[L307] def cpg_plus_plus(node_q, forest, config):
[L308] """Generate ≤k candidate parents for node q."""
[L309] # Stage 1: Domain Routing (O(1)) — filter to same-domain nodes
[L310] domain = classify_domain(node_q)
[L311] domain_nodes = forest.get_nodes_by_domain(domain)
[L312]
[L313] # Stage 2: ANN Retrieval (O(log N)) — find semantically similar nodes
[L314] if len(domain_nodes) > config.ann_k:
[L315] ann_results = vector_index.search(node_q.embedding, k=config.
```

DEEP RESEARCH 02/09/2026, 09:15:47

Citation Marker: `filecite`turn32file4

```
[L316] metadata:
[L317] query_result_index: 4
[L318] file_id: file_00000000dcc082119f935683a5e13eed
[L319] version_id: 1
[L320] name: SHIA_RAG_Final_Report(4)(1).md
[L321] mime_type: text/markdown
[L322] surface: conversation
[L323] score: 0.02919863597612958
[L324] snippet:
[L325] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation
[L326]
[L327] > Project Type: Minor Project (M.Tech)
[L328] > Date: August 2026
[L329] > Author: [Your Name]
[L330]
[L331] ---
[L332]
[L333] ## Abstract
[L334]
[L335] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.
[L336]
[L337] We present SHIA-RAG (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed Knowledge Forest — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.
[L338]
[L339] SHIA-RAG introduces five novel contributions: (1) Precedence-Constrained Tree Knapsack for context selection under token budgets, ensuring hierarchical coherence; (2) Self-Evolving Knowledge Forest with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) Multi-Objective Energy Minimization for automated hierarchy induction; (4) Adaptive Depth Traversal that adjusts retrieval depth based on query complexity; and (5) SHEF (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.
[L340]
```

[L341] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L342]

[L343] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L344]

[L345] ---

[L346]

[L347] ## Table of Contents

[L348]

[L349] 1. [Introduction](#)

[L350] 2. [Literature Review](#)

[L351] 3. [System Architecture](#)

[L352] 4. [Data Model — Layer 0](#)

[L353] 5. [Offline Pipeline — Layers 1-5](#)

[L354] 6. [Online Pipeline — Layers 6-7](#)

[L355] 7. [Operations Layer — Layer 8](#)

[L356] 8. [Novel Contributions](#)

[L357] 9. [Mathematical Framework](#)

[L358] 10. [Implementation](#)

[L359] 11. [Evaluation Framework](#)

[L360] 12. [Risk Analysis](#)

[L361] 13. [Conclusion & Future Work](#)

[L362]

[L363] ---

[L364]

[L365] ## 1. Introduction

[L366]

[L367] ### 1.1 Problem Statement

[L368]

[L369] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L370]

[L371] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L372] 2. Embed each chunk into a vector space

[L373] 3. At query time, retrieve the top-k most similar chunks

[L374] 4. Feed these chunks as context to an LLM for answer generation

[L375]

[L376] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

CHATGPT 02/09/2026, 09:15:47

Haan. Tumhare **SHIA-RAG report ke according**, Knowledge Node aur Knowledge Edge same time par directly nahi bante. Pehle **Layer 3 mein Knowledge Nodes** create hote hain, phir **Layer 4 mein un Nodes ke beech Knowledge Edges** create/validate hote hain. `filecite\turn32file0\37-L55`

Let's take one simple example.

Example Document

Suppose tumhare paas ek Operating Systems PDF hai:

"Round Robin is a preemptive CPU scheduling algorithm. It assigns a fixed time quantum to each process in circular order. A smaller time quantum improves response time but increases context switching."

1. Layer 1 & 2 ke baad kya mila?

Layer 1 document ko normalize karta hai, aur Layer 2 usse blocks mein divide karta hai.

Eventually Layer 3 ko kuch aisa input milega:

```
DocumentBlock

Block 1:
"Round Robin is a preemptive CPU scheduling algorithm."

Block 2:
"It assigns a fixed time quantum to each process..."

Block 3:
"A smaller time quantum improves response time..."
```

Report ke data flow mein Layer 2 ke baad `DocumentBlock[]` aur Layer 3 ke baad `KnowledgeNode[]` generate hote hain. [filecite\turn32file0\L39-L47](#)

2. Layer 3 — Knowledge Node kaise banta hai?

Layer 3 ka kaam hai document ke text se meaningful concepts identify karna.

Example:

```
"Round Robin is a preemptive CPU scheduling algorithm."
```

System identify karta hai:

```
Concept = Round Robin
```

Similarly:

```
"CPU scheduling"
```

aur

```
"Time Quantum"
```

bhi concepts ho sakte hain.

So Layer 3 output:

```
KnowledgeNode[]
|
├─ Node 1: Round Robin
├─ Node 2: CPU Scheduling
└─ Node 3: Time Quantum
```

Report specifically says Layer 3 performs **concept detection and alias resolution**, producing canonical, deduplicated `KnowledgeNode` objects. [filecite\turn32file0\L42-L47](#)

3. Node sirf naam nahi hota

Ye bahut important hai.

Round Robin ko simply:

"Round Robin"

store nahi karte.

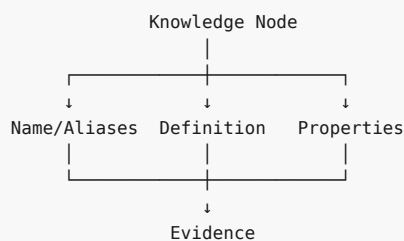
Uska structured Knowledge Node banta hai.

Report ka example:

```
{
  "canonical_name": "Round Robin",
  "aliases": ["RR", "Round-Robin Scheduling"],
  "definitions": [
    "A preemptive CPU scheduling algorithm..."
  ],
  "summaries": [
    "CPU scheduling method using time slices"
  ],
  "keywords": [
    "scheduling",
    "time quantum",
    "preemptive",
    "CPU"
  ],
  "domain": "Operating Systems",
  "properties": {
    "preemptive": true
  },
  "examples": [
    "Process P1 gets 20ms, then P2 gets 20ms..."
  ],
  "evidence": [...]
}
```

filecite\turn32file2\L183-L219

So conceptually:



4. Alias Resolution kyu?

Suppose document mein kahin likha hai:

Round Robin

aur doosri jagah:

RR

aur teesri jagah:

Round-Robin Scheduling

Agar system inhe alag-alag Nodes bana de:

```
Node A = Round Robin
Node B = RR
Node C = Round-Robin Scheduling
```

toh duplication ho jayega.

Instead Layer 3 alias resolution ke through:

```
Round Robin
RR
Round-Robin Scheduling
  ↓
ONE canonical Knowledge Node
```

Report alias resolution ke liye **MinHash LSH** mention karta hai. [filecite\turn32file0\L42-L45](#)

5. Ab Knowledge Edge kaise banta hai?

Ab Layer 4 start hota hai.

Layer 4 ka input:

```
KnowledgeNode[]
```

Aur output:

```
KnowledgeEdge[]
```

Report mein explicitly:

Layer 4 = Relation Management

Input = KnowledgeNode[]

Output = KnowledgeEdge[]

Key methods = ontology validation + confidence scoring. [filecite\turn32file0\L60-L67](#)

6. Edge banane ke liye system kya dekhta hai?

Ab system Nodes ko pair karke dekhta hai:

```
Round Robin
  ?
CPU Scheduling
```

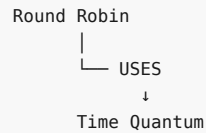
Document mein sentence hai:

Round Robin is a preemptive CPU scheduling algorithm.

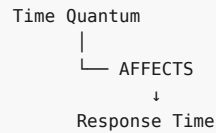
Isse relationship identify ho sakta hai:

```
Round Robin
  |
  └─ IS_A / related-to
        ↓
    CPU Scheduling
```

Similarly:



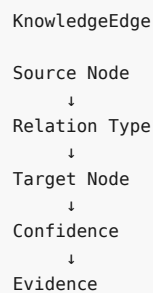
Aur:



Exact relation types source architecture mein hierarchical relationships jaise **IS_A** , **PART_OF** aur semantic relationships jaise **USES** , **CAUSES** diye gaye hain. [filecite\turn32file4\L333-L341](#)

7. Edge ka structure kya hota hai?

Conceptually:



For example:

```

Source:
Round Robin

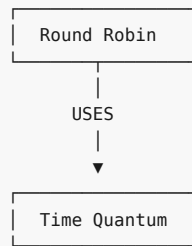
Relation:
USES

Target:
Time Quantum

Confidence:
0.91

Evidence:
"It assigns a fixed time quantum
to each process..."
  
```

So visually:



Confidence = 0.91

8. Confidence score kyu?

Suppose LLM/document se relationship mila:

Round Robin → USES → Time Quantum

System blindly trust nahi karta.

Layer 4:

```

Extract relation
  ↓
Validate relation
  ↓
Calculate confidence
  
```

Report specifically says relations are **validated against an ontology and confidence-scored**. [filecite\turn32file2\L154-L158](#)

9. Agar multiple relations mil jayein?

Suppose ek document mein:

Round Robin → USES → Time Quantum

aur doosre document mein bhi same relation mila.

Then system multiple pieces of evidence collect kar sakta hai.

For example:

```

Document 1:
Round Robin uses Time Quantum
Confidence = 0.9

Document 2:
Round Robin uses a fixed time quantum
Confidence = 0.85

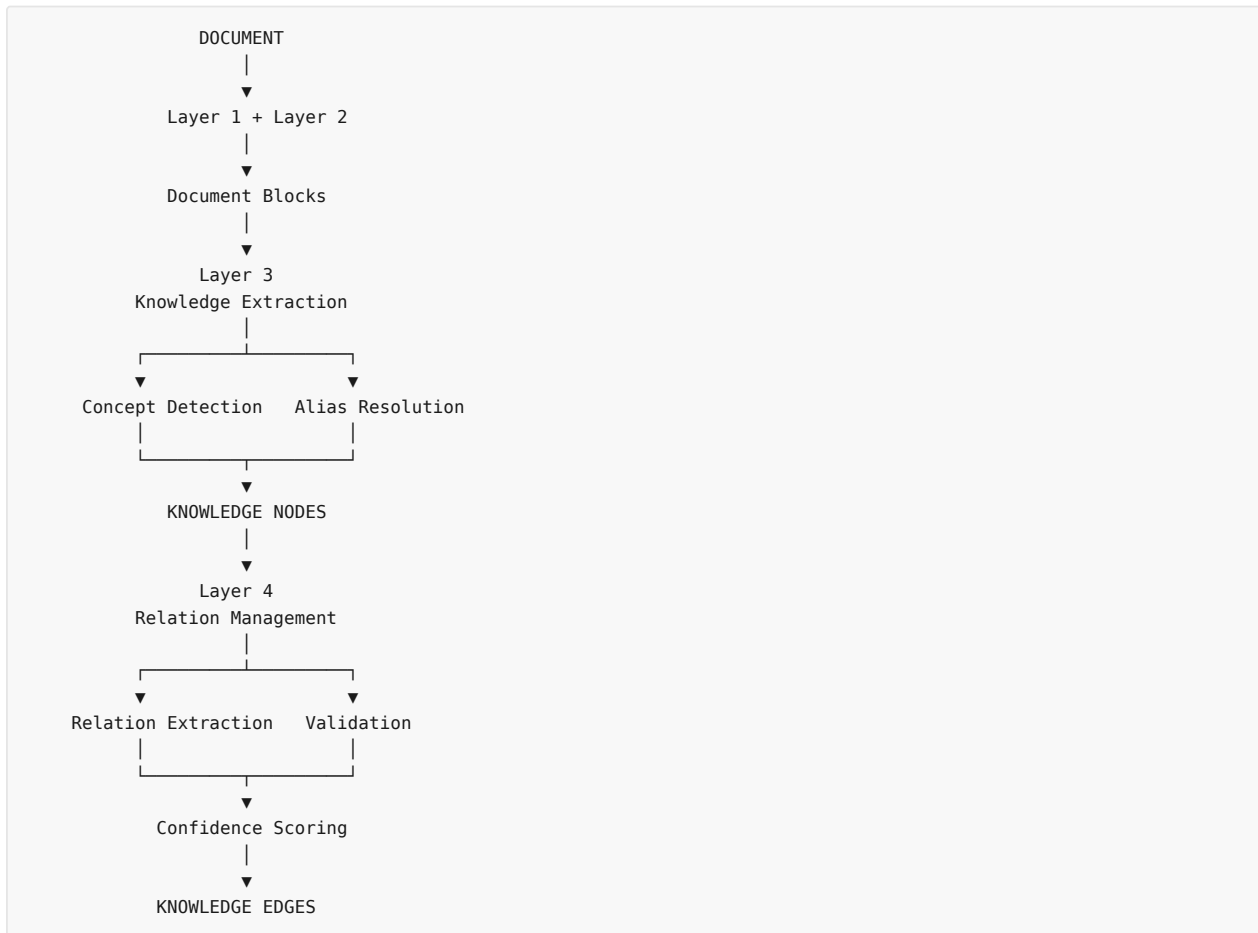
Document 3:
Round Robin assigns time quantum
Confidence = 0.8
  
```

System evidence ko combine karke relation ko stronger confidence de sakta hai.

Report mein conflict-resolution mechanism weighted votes use karta hai, where weight is based on **edge confidence × number of evidence items**. [filecite\turn32file3\L229-L267](#)

10. Toh complete Node → Edge process

Ab poora process dekho:



11. Hamare example mein final result

Nodes

N1 = CPU Scheduling
 N2 = Round Robin
 N3 = Time Quantum
 N4 = Response Time
 N5 = Context Switching

Edges

N1 —IS_A—> N2
 N2 —USES—> N3
 N3 —AFFECTS—> N4
 N3 —AFFECTS—> N5

With confidence:

```
N1 —IS_A [0.94]—> N2
N2 —USES [0.91]—> N3
N3 —AFFECTS [0.82]—> N4
N3 —AFFECTS [0.87]—> N5
```

Then Layer 5 inhi Nodes + Edges ko use karke Knowledge Forest ki hierarchy construct karta hai.

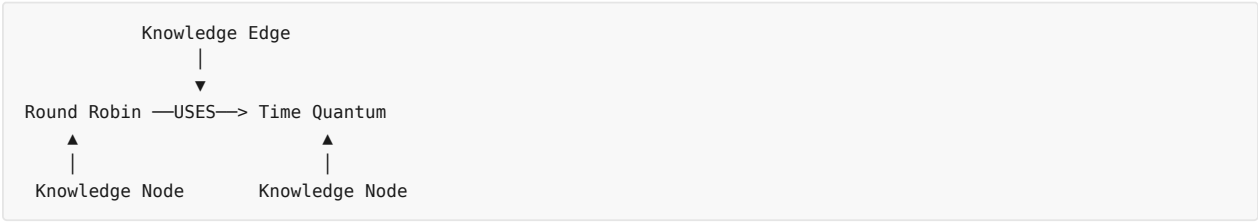
filecite[turn32file3][L279-L300]

Sabse important distinction

Knowledge Node = “What is the concept?”

Knowledge Edge = “How are two concepts related?”

Example:



Aur tumhare architecture mein sequence hai:

Text → Concepts → Knowledge Nodes → Relations → Knowledge Edges → Knowledge Forest. filecite[turn32file0][L37-L55]

YOU 02/09/2026, 09:29:32

is there any code or algorithms exist in the uploaded file for identify knowledge node and knowledge edge

CHATGPT 02/09/2026, 09:29:36

```
{"paths":["files"]}
```

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: filecite[turn33file0]

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_00000000dcc082119f935683a5e13eed
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report(4)(1).md
- [L6] mime_type: text/markdown
- [L7] surface: conversation
- [L8] score: 0.031054405392392875
- [L9] snippet:
- [L10]

```
[L11] graph TD
[L12]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L13]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>"]
[L14]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L15]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L16]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L17]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L18]     end
[L19]
[L20]     subgraph "Phase 2: Online – Retrieval & Generation"
[L21]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L22]         KF --> L6
[L23]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L24]         L7 --> A["Answer + Citations"]
[L25]     end
[L26]
[L27]     subgraph "Phase 3: Continuous – Operations"
[L28]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L29]         A --> L8
[L30]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L31]     end
[L32]
```

[L33]

[L34] ### 3.2 Data Flow

[L35]

[L36]

```
[L37] Raw Document (PDF/DOCX/HTML)
[L38]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L39] Normalized Text + Metadata
[L40]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L41] [Page → Region → Block] hierarchy
[L42]     ↓ Layer 3: Split sentences, extract concepts/definitions/reasons
[L43] KnowledgeUnit objects (candidate concepts)
[L44]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L45] KnowledgeNode objects (canonical, deduplicated)
[L46]     ↓ Layer 4: Validate relations against ontology, score confidence
[L47] KnowledgeEdge objects (typed, scored)
[L48]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L49] Knowledge Forest (rooted trees + cross-links)
[L50]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L51] Selected context nodes (within token budget)
[L52]     ↓ Layer 7: Build prompt, invoke LLM
[L53]     ↓ Layer 7: Verify claims against evidence, bind citations
[L54] Answer with citations + confidence score
[L55]     ↓ Layer 8: Log, collect feedback, update forest weights
[L56]
```

[L57]

[L58] ### 3.3 Layer Summary

[L59]

[L60] | Layer | Name | Input | Output | Key Algorithms |

[L61] |-----|-----|-----|-----|-----|

[L62] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L63] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L64] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L65] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L66] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L67] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L68] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L69] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L70] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L71]
[L72] ---
[L73]
[L74] ## 4. Data Model — Layer 0

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite`turn33file1

[L75] metadata:
[L76] query_result_index: 1
[L77] file_id: file_00000000dcc082119f935683a5e13eed
[L78] version_id: 1
[L79] name: SHIA_RAG_Final_Report(4)(1).md
[L80] mime_type: text/markdown
[L81] surface: conversation
[L82] score: 0.031009615384615385
[L83] snippet:
[L84] ### 11.4 Statistical Significance Testing
[L85]
[L86] | Test | Application | Details |
[L87] |-----|-----|-----|
[L88] | Paired t-test | SHIA-RAG vs each baseline | Per-query metric comparison |
[L89] | Significance level | All tests | $\alpha = 0.05$ |
[L90] | Effect size | All comparisons | Cohen's d (small: 0.2, medium: 0.5, large: 0.8) |
[L91] | Confidence intervals | All reported metrics | 95% CI via bootstrap (1000 resamples) |
[L92] | Multiple comparison correction | When comparing 5+ baselines | Bonferroni or Holm-Bonferroni |
[L93]
[L94] ### 11.5 Error Analysis
[L95]
[L96] | Analysis Type | Method | Details |
[L97] |-----|-----|-----|
[L98] | Query categorization | Classify all test queries | DEFINITION, COMPARISON, MULTI-HOP, SYNTHESIS |
[L99] | Failure pattern analysis | Cluster failed queries | Identify common failure modes |
[L100] | Qualitative analysis | Manual inspection | Review 100 randomly sampled failed queries |
[L101] | Per-category breakdown | Stratified metrics | Report accuracy per query type |
[L102]
[L103] ### 11.6 Computational Cost Analysis
[L104]
[L105] | Metric | How Measured | Report Format |
[L106] |-----|-----|-----|
[L107] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |
[L108] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |
[L109] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |
[L110] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |
[L111] | Memory usage | Peak RAM + VRAM | GB |
[L112] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |
[L113]
[L114] ---

[L115]

[L116] ## 12. Risk Analysis

[L117]

[L118] | Risk | Impact | Probability | Mitigation |

[L119] |-----|-----|-----|-----|

[L120] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L121] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L122] | $O(N^2)$ operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L123] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning, conflict logging |

[L124] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L125] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L126] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L127] | Self-evolution reinforces wrong paths | Medium | Medium | ϵ -greedy exploration ($\epsilon=0.1$), periodic weight normalization, anomaly detection |

[L128]

[L129] ---

[L130]

[L131] ## 12.5 Limitations & Current Status

[L132]

[L133] This section explicitly documents known limitations and the current project status.

[L134]

[L135] ### Current Status

[L136]

[L137] SHIA-RAG is currently at the **specification stage**. This report provides a complete architectural specification with schemas, algorithms, pseudocode, and implementation contracts. However:

[L138]

[L139] - **No implementation exists yet** — the system has not been built or deployed

[L140] - **No experimental results exist** — all evaluation metrics are planned, not measured

[L141] - **Cost/latency numbers are projections** — actual measurements require a running system

[L142]

[L143] The implementation roadmap (Section 10.3) details the path from specification to working system.

[L144]

[L145] ### Known Limitations

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite[turn33file2]`

[L146] metadata:

[L147] query_result_index: 2

[L148] file_id: file_00000000dcc082119f935683a5e13eed

[L149] version_id: 1

[L150] name: SHIA_RAG_Final_Report(4)(1).md

[L151] mime_type: text/markdown

[L152] surface: conversation

[L153] score: 0.03076923076923077

[L154] snippet:

[L155] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3)

hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L156]

[L157] ### 2.3 Literature Search Methodology

[L158]

[L159] | Parameter | Details |

[L160] |-----|-----|

[L161] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L162] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L163] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L164] | **Time Range** | 2020–2026 |

[L165] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L166]

[L167] ---

[L168]

[L169] ## 3. System Architecture

[L170]

[L171] ### 3.1 Overview

[L172]

[L173] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L174]

[L175] | Phase | Layers | Mode | Purpose |

[L176] |-----|-----|-----|-----|

[L177] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L178] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L179] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L180]

[L181] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L182]

[L183]

```
[L184] graph TD
[L185]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L186]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Parsing<br/>Text chunking, embedding"]
[L187]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L188]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L189]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L190]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L191]     end
[L192]
[L193]     subgraph "Phase 2: Online – Retrieval & Generation"
[L194]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L195]         KF --> L6
[L196]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L197]         L7 --> A["Answer + Citations"]
[L198]     end
[L199]
[L200]     subgraph "Phase 3: Continuous – Operations"
[L201]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L202]         A --> L8
[L203]         L8 --> SE["Self-Evolution<br/>Feedback"] | KF
[L204]     end
[L205]
```

[L206]
[L207] ### 3.2 Data Flow
[L208]
[L209] ```
[L210] Raw Document (PDF/DOCX/HTML)
[L211] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L212] Normalized Text + Metadata
[L213] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L214] [Page → Region → Block] hierarchy
[L215] ↓ Layer 3: Split sentences,

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `[[filecite]]turn33file3[[`

[L216] metadata:
[L217] query_result_index: 3
[L218] file_id: file_00000000dcc082119f935683a5e13eed
[L219] version_id: 1
[L220] name: SHIA_RAG_Final_Report(4)(1).md
[L221] mime_type: text/markdown
[L222] surface: conversation
[L223] score: 0.029957522915269395
[L224] snippet:
[L225] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem
[L226] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure
[L227] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence
[L228] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG
[L229] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality
[L230]
[L231] The complete system specification covers:
[L232] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts
[L233] - **12 defined functions** with complete implementations
[L234] - **12 relation types** across 4 hierarchical and 8 semantic categories
[L235] - **8 schemas** covering all data objects from document blocks to query logs
[L236] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline
[L237] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing
[L238]
[L239] ### 13.2 Key Novelties
[L240]
[L241] | # | Novelty | Level | Why It's New |
[L242] |---|-----|-----|-----|
[L243] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |
[L244] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L245] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L246] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L247] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L248]

[L249] ### 13.3 Next Steps

[L250]

[L251] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L252] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L253] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L254] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L255] 5. **Build Layer 4** (Relation management with ontology validation)

[L256] 6. **Build Layer 5** (SHIA Core with algorithms)

[L257] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L258] 8. **Build Layer 7** (Generation with verification)

[L259] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L260] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L261] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L262] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L263]

[L264] ### 13.4 Future Work

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite`turn33file4``

[L265] metadata:

[L266] query_result_index: 4

[L267] file_id: file_00000000dcc082119f935683a5e13eed

[L268] version_id: 1

[L269] name: SHIA_RAG_Final_Report(4)(1).md

[L270] mime_type: text/markdown

[L271] surface: conversation

[L272] score: 0.029906956136464335

[L273] snippet:

[L274] ##### Alias Resolution (with Antonym Protection)

[L275]

[L276] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them — but must **avoid merging antonyms** (e.g., "preemptive" and "non-preemptive" have high similarity but are distinct concepts).

[L277]

[L278] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L279] 2. **Antonym check**: Before merging, verify candidates do not have a CONTRASTS_WITH relation

[L280] 3. Find connected components of near-duplicate units

[L281] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L282]

[L283]

```

[L284] def alias_resolution(units: list, relation_cache: dict) -> list:
[L285]     """Merge KnowledgeUnits referring to the same concept.
[L286]
[L287]     Merge strategy:
[L288]     - Canonical name: most frequent name across units
[L289]     - Aliases: union of all names and aliases
[L290]     - Definitions: union of all definitions
[L291]     - Confidence: average of unit extraction confidences
[L292]     - Properties: merged (latest value wins, conflicts logged)
[L293]
[L294]     Antonym protection:
[L295]     - Before merging, check if candidates have CONTRASTS_WITH relation
[L296]     - Also check for negation prefixes (non-, un-, in-, dis-)
[L297]     """
[L298]     import re
[L299]     NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L300]
[L301]     lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L302]     # ... build LSH index from concept names + aliases ...
[L303]
[L304]     raw_groups = find_connected_components(units, lsh)
[L305]
[L306]     # Antonym protection: split groups containing antonym pairs
[L307]     safe_groups = []
[L308]     for group in raw_groups:
[L309]         names = [u.name.lower() for u in group]
[L310]         has_antonym = False
[L311]         for i, n1 in enumerate(names):
[L312]             for n2 in names[i+1:]:
[L313]                 # Check 1: Negation prefix
[L314]                 if NEGATION_PREFIXES.sub('', n1) == NEGATION_PREFIXES.sub('', n2):
[L315]                     has_antonym = True
[L316]                 # Check 2: Known CONTRASTS_WITH relation
[L317]                 if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L318]                     has_antonym = True
[L319]
[L320]         if has_antonym:
[L321]             safe_groups.extend([u for u in group]) # Don't merge
[L322]         else:
[L323]             safe_groups.append(group)
[L324]
[L325]     return [create_canonical_node(group) for group in safe_groups]
[L326]

```

[L327]

[L328] ##### Embedding Strategy on Merge

[L329]

[L330] When nodes are merged during alias resolution, embeddings are handled as follows:

[L331] - **Do NOT average embeddings** — averaging collapses distinct semantic information

[L332] - **Re-embed the merged node** using the canonical name + merged definitions

[L333] - This ensures the embedding accurately represents the merged concept

[L334]

[L335]

```

[L336] def merge_node_embeddings(merged_node, embed_fn):
[L337]     """Re-compute embedding for merged node instead of averaging.
[L338]
[L339]     Averaging embeddings of 'RR' and 'Round-Robin Scheduling' would
[L340]     produce a vector that represents neither concept well.
[L341]     Instead, embed the canonical representation."""
[L342]     text_to_embed = merged_node.canonical_name
[L343]     if merged_node.definitions:
[L344]         text_to_embed += " " + merged_node.definitions[0]
[L345]     merged_node.embedding = embed_fn(text_to_embed)
[L346]

```

[L347]

[L348] ---

[L349]

[L350] ### 5.4 Layer 4: Relation Management

[L351]

[L352] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve cross-document conflicts, and produce final `KnowledgeEdge` objects.

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite`turn33file5

[L353] metadata:

[L354] query_result_index: 5

[L355] file_id: file_00000000dcc082119f935683a5e13eed

[L356] version_id: 1

[L357] name: SHIA_RAG_Final_Report(4)(1).md

[L358] mime_type: text/markdown

[L359] surface: conversation

[L360] score: 0.029386529386529386

[L361] snippet:

[L362] Respond in JSON format:

[L363] {

[L364] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."],

[L365] "properties": {"key": {"type": "...", "value": "..."}},

[L366] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",

[L367] "evidence": "..."}],

[L368] "domain": "..."

[L369] }

[L370]

[L371]

[L372] ### Definition Extraction (Dependency-Parse Based)

[L373]

[L374] Simple regex patterns like `^(.*) is (a|an|the)? (.*)`` are too broad – they match non-definitions like "The s

[L375]

[L376]

python

[L377] def is_definition(text: str) -> tuple:

[L378] """Detect definitions using dependency parse + pattern matching.

[L379]

[L380] Returns (is_def: bool, concept: str, definition: str).

[L381]

[L382] Strategy:

[L383] 1. Check for explicit definitional phrases (high precision)

[L384] 2. Use dependency parse to find copular definitions (medium precision)

[L385] 3. Fallback to LLM classification (expensive, high recall)

[L386] """

[L387] import re

[L388]

[L389] # Strategy 1: Explicit definitional phrases (high precision)

```
[L390] explicit_patterns = [
[L391] r'^(.+?)\s+is defined as\s+(.+)',
[L392] r'^(.+?)\s+refers to\s+(.+)',
[L393] r'^(.+?):\s+A-Z', # "Concept: A description..."
[L394] r'^(.+?)\s+is a\s+(type|kind|form|category|class)\s+of\s+(.+)',
[L395] ]
[L396] for pattern in explicit_patterns:
[L397] match = re.match(pattern, text, re.IGNORECASE)
[L398] if match:
[L399] return True, match.group(1).strip(), match.group(2).strip()
[L400]
[L401] # Strategy 2: Copular structure via dependency parse
[L402] # (requires spaCy or similar — use LLM as fallback if not available)
[L403]
[L404] # Strategy 3: LLM fallback (only for DEFINITION-classified blocks)
[L405] return False, None, None
[L406]
```

```
[L407]
[L408] ##### Alias Resolution (with Antonym Protection)
[L409]
[L410] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin",
[L411]
[L412] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases
[L413] 2. **Antonym check**: Before merging, verify candidates do not have a CONTRASTS_WITH relation
[L414] 3. Find connected components of near-duplicate units
[L415] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evi
[L416]
[L417]
```

python

```
[L418] def alias_resolution(units: list, relation_cache: dict) -> list:
[L419] """Merge KnowledgeUnits referring to the same concept.
[L420]
[L421] Merge strategy:
[L422] - Canonical name: most frequent name across units
[L423] - Aliases: union of all names and aliases
[L424] - Definitions: union of all definitions
[L425] - Confidence: average of unit extraction confidences
[L426] - Properties: merged (latest value wins, conflicts logged)
[L427]
[L428] Antonym protection:
[L429] - Before merging, check if candidates have CONTRASTS_WITH relation
[L430] - Also check for negation prefixes (non-, un-, in-, dis-)
[L431] """
[L432] import re
[L433] NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L434]
[L435] lsh = MinHashLSH(threshold=0.7,
```


[L436] metadata:

[L437] query_result_index: 6

[L438] file_id: file_00000000dcc082119f935683a5e13eed

[L439] version_id: 1

[L440] name: SHIA_RAG_Final_Report(4)(1).md

[L441] mime_type: text/markdown

[L442] surface: conversation

[L443] score: 0.029386529386529386

[L444] snippet:

[L445] **Novelty level:** Moderately Novel — the combined formulation for RAG hierarchy induction is new.

[L446]

[L447] ---

[L448]

[L449] ### 8.4 Novelty 4: Adaptive Depth Traversal via Query Complexity Classification

[L450]

[L451] **Existing approach:** All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.

[L452]

[L453] **SHIA-RAG approach:** A query complexity classifier that dynamically adjusts traversal depth:

[L454]

[L455] | Query Type | Example | Traversal Strategy |

[L456] |-----|-----|-----|

[L457] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |

[L458] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |

[L459] | MULTI-HOP | "How does OSPF convergence affect reliability?" | Deep: Full path traversal + cross-links (depth=3+) |

[L460] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Root-level retrieval, expand selectively |

[L461]

[L462] **Novelty level:** Incremental — novel in the specific application to hierarchical RAG traversal depth.

[L463]

[L464] ---

[L465]

[L466] ### 8.5 Novelty 5: SHEF — SHIA-RAG Evaluation Framework

[L467]

[L468] **Existing approach:** Standard RAG benchmarks (MS MARCO, NaturalQuestions) only measure flat retrieval. No benchmark tests hierarchical retrieval quality.

[L469]

[L470] **SHIA-RAG approach:** A novel evaluation framework with 3 metric categories:

[L471]

[L472] ##### A. Retrieval Metrics

[L473] | Metric | Formula | What It Measures |

[L474] |-----|-----|-----|

[L475] | Hop Accuracy | correct multi-hop answers / total multi-hop queries | Multi-hop reasoning quality |

[L476] | Ancestor Recall | retrieved ancestors $\cap$ gold ancestors / gold ancestors | Correct concept chain retrieval |

[L477] | Context Coherence | Avg semantic similarity between consecutive context nodes | Logical coherence of assembled context |

[L478]

[L479] ##### B. Hierarchy Metrics

[L480] | Metric | Formula | What It Measures |

[L481] |-----|-----|-----|

[L482] | Parent Assignment Accuracy | correctly placed nodes / total nodes | Quality of hierarchy induction |

[L483] | Tree Edit Distance | Normalized TED vs gold taxonomy | Structural similarity to human taxonomy |

[L484] | Depth Distribution Score | KL-divergence from ideal depth distribution | Tree balance |

[L485]

[L486] ##### C. Forest Quality Metrics

[L487] | Metric | Formula | What It Measures |

[L488] |-----|-----|-----|

[L489] | Cross-Doc Fusion Accuracy | correctly merged nodes / total mergeable pairs | Entity resolution quality |

[L490] | Orphan Rate | root nodes without children / total root nodes | Isolated concept rate |

[L491] | Forest Density | edges / nodes | Knowledge forest connectivity |

[L492]

[L493] **Novelty level:** Incremental (Novel Combination) — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L494]

[L495] ### 8.6 Novelty Level Assessment Summary

[L496]

[L497] | # | Novelty | Level | Justification |

[L498] |---|-----|-----|-----|

[L499] | 1 | Tree Knapsack for Context Selection | **Incremental (Novel Application)** | Adaptation of existing OR technique to a new domain (RAG context selection) |

[L500] | 2 | Self-Evolving Knowledge Forest | **Moderately Novel** | Novel combination of retrieval feedback + hierarchy restructuring |

[L501] | 3 | Multi-Objective Energy Minimization | **Moderately Novel** | Combined formulation for RAG hierarchy induction is new |

[L502] | 4 | Adaptive Depth Traversal | **Incremental** | Novel application to hierarchical RAG traversal depth |

[L503] | 5 | SHEF Evaluation Framework | **Incremental (Novel Combination)** | Standard metrics adapted as a unified framework for hierarchical RAG |

[L504]

[L505] ---

[L506]

[L507] ## 9. Mathematical Framework

[L508]

[L509] ### 9.1 Hierarchy Energy (Global Objective)

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite{turn33file7}`

[L510] metadata:

[L511] query_result_index: 7

[L512] file_id: file_00000000dcc082119f935683a5e13eed

[L513] version_id: 1

[L514] name: SHIA_RAG_Final_Report(4)(1).md

[L515] mime_type: text/markdown

[L516] surface: conversation

[L517] score: 0.02871794871794872

[L518] snippet:

[L519] deduplicated)

[L520] ↓ Layer 4: Validate relations against ontology, score confidence

[L521] KnowledgeEdge objects (typed, scored)

[L522] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)

[L523] Knowledge Forest (rooted trees + cross-links)

[L524] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection

[L525] Selected context nodes (within token budget)
[L526] ↓ Layer 7: Build prompt, invoke LLM
[L527] ↓ Layer 7: Verify claims against evidence, bind citations
[L528] Answer with citations + confidence score
[L529] ↓ Layer 8: Log, collect feedback, update forest weights
[L530]

```
[L531]
[L532] ### 3.3 Layer Summary
[L533]
[L534] | Layer | Name | Input | Output | Key Algorithms |
[L535] |-----|-----|-----|-----|-----|
[L536] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L537] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L538] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L539] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L540] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L541] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L542] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L543] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L544] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L545]
[L546] ---
[L547]
[L548] ## 4. Data Model – Layer 0
[L549]
[L550] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode`
[L551]
[L552] ### 4.1 KnowledgeNode Schema
[L553]
[L554] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L555]
[L556]
```

json

```
[L557] {
[L558] "node_id": "NODE_000789",
[L559] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L560] "canonical_name": "Round Robin",
[L561] "aliases": ["RR", "Round-Robin Scheduling"],
[L562] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L563] "summaries": ["CPU scheduling method using time slices"],
[L564] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L565] "domain": "Operating Systems",
[L566] "properties": {
[L567] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L568] "preemptive": {"type": "boolean", "value": true}
[L569] },
[L570] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L571] "parent": "NODE_000456",
[L572] "children": ["NODE_001001", "NODE_001002"],
[L573] "related": ["NODE_000890"],
[L574] "incoming_edges": ["EDGE_001"],
[L575] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L576] "evidence": [
[L577] {
[L578] "evidence_type": "DEFINITION",
```

```
[L579] "text": "Round Robin is a preemptive scheduling algorithm...",
[L580] "source_doc": "DOC_000001",
[L581] "source_block": "BLK_000234",
[L582] "page": 42
[L583] }
[L584] ],
```

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite`turn33file8

```
[L585] metadata:
[L586] query_result_index: 8
[L587] file_id: file_00000000dcc082119f935683a5e13eed
[L588] version_id: 1
[L589] name: SHIA_RAG_Final_Report(4)(1).md
[L590] mime_type: text/markdown
[L591] surface: conversation
[L592] score: 0.028612012987012988
[L593] snippet:
[L594] Uses pdfplumber for text-based tables, Camelot for image-based.
[L595] Returns list of rows, each row is a list of cell strings. """
[L596] try:
[L597] # Primary: pdfplumber table extraction
[L598] table = page.extract_table(table_region)
[L599] if table and len(table) > 1: # At least header + 1 data row
[L600] return table
[L601] except Exception:
[L602] pass
[L603]
[L604] # Fallback: Camelot for image-based tables
[L605] try:
[L606] import camelot
[L607] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L608] if tables:
[L609] return tables[0].df.values.tolist()
[L610] except Exception:
[L611] pass
[L612]
[L613] return None # Table structure could not be parsed
[L614]
```

```
[L615]
[L616] ##### Table Detection Fallback
[L617]
[L618] If table detection confidence is low, the block is flagged:
[L619]
[L620]
```

python

```
[L621] def classify_block_with_fallback(text, page) -> tuple:
[L622] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L623] block_type, confidence = classify_block(text)
```

```

[L624]
[L625] if block_type == BlockType.TABLE and confidence < 0.6:
[L626] # Attempt cell-level parsing to confirm
[L627] cells = parse_table_cells(page, text)
[L628] if cells is None:
[L629] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L630] else:
[L631] return BlockType.TABLE, 0.8 # Confirmed by parser
[L632]
[L633] return block_type, confidence
[L634]

```

```

[L635]
[L636] ##### Reading Order Validation
[L637]
[L638] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks reliabilit
[L639]
[L640]

```

python

```

[L641] def assign_reading_order(blocks: list) -> list:
[L642] """Assign global reading order with confidence.
[L643]
[L644] Multi-column layouts can produce incorrect reading order.
[L645] We detect potential issues and flag low confidence."""
[L646] # Sort by page, then y-coordinate, then x-coordinate
[L647] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L648]
[L649] for i, block in enumerate(sorted_blocks):
[L650] block.reading_order = i
[L651]
[L652] # Detect potential multi-column issues
[L653] if i > 0:
[L654] prev = sorted_blocks[i - 1]
[L655] if (prev.page_number == block.page_number and
[L656] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L657] prev.x_pos > block.x_pos): # But previous is to the right
[L658] block.confidence *= 0.7 # Reduce confidence — possible column issue
[L659]
[L660] return sorted_blocks
[L661]

```

```
[L662]
[L663] ---
[L664]
[L665] ### 5.3 Layer 3: Knowledge Extraction
[L666]
[L667] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge s
[L668]
[L669] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L670]
[L671] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, definitio
[L672]
[L673] #### Extraction Prompt
[L674]
[L675]
```

[L676] You are a knowledge extraction system. Given a text block, extract:

[L677] 1. Concepts: Named entities, technical terms

[L678] 2. Definitions: If the text defines any concept, extract the definition

[L679] 3. Properties: Key attributes with types and values

[L680] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)

[L681] 5. Domain: Subject area (e.g., "Operating Systems")

DEEP RESEARCH 02/09/2026, 09:29:39

Citation Marker: `filecite`turn33file9

[L682] metadata:

[L683] query_result_index: 9

[L684] file_id: file_00000000dcc082119f935683a5e13eed

[L685] version_id: 1

[L686] name: SHIA_RAG_Final_Report(4)(1).md

[L687] mime_type: text/markdown

[L688] surface: conversation

[L689] score: 0.028594771241830064

[L690] snippet:

[L691] Real knowledge often has multiple valid parents (e.g., "Machine Learning" → both "AI" and "Statistics") | Deliberate design choice | Limits expressiveness | Future work: DAG support (Section 13.4) |

[L692] | L2 | **Greedy Tree Knapsack** — uses greedy approximation instead of exact DP. Exact DP is $O(nB)$ where B = token budget, which is computationally prohibitive | Deliberate trade-off | Near-optimal, not optimal | Future work: Exact DP for small budgets |

[L693] | L3 | **No incremental updates** — adding new documents requires re-running the full offline pipeline (Layers 1-5) | Engineering gap | Limits production scalability | Future work: Incremental forest updates |

[L694] | L4 | **Abstraction-level heuristic** — the multi-factor heuristic for estimating abstraction level has not been empirically validated | Validation needed | May produce incorrect hierarchy | Planned: Empirical validation on domain datasets |

[L695] | L5 | **LLM extraction quality** — knowledge extraction relies on LLM accuracy. Extraction errors propagate through the entire pipeline | Inherent limitation | Garbage-in-garbage-out risk | Mitigated by confidence thresholds and human review |

[L696] | L6 | **Self-evolution bias** — positive feedback on wrong paths can reinforce errors. ϵ -greedy exploration mitigates but does not eliminate this risk | Research challenge | May degrade over time | Mitigated by ϵ -greedy; future: Thompson sampling |

[L697] | L7 | **Novelty claims require verification** — "first" claims are based on systematic literature review but require exhaustive verification before publication | Research maturity | Affects publication readiness | All claims qualified with "to the best of our knowledge" |

[L698]

[L699] ---

[L700]

[L701] ## 13. Conclusion & Future Work

[L702]

[L703] ### 13.1 Summary

[L704]

[L705] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L706]

[L707] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L708] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L709] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L710] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L711] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L712]

[L713] The complete system specification covers:

[L714] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L715] - **12 defined functions** with complete implementations

[L716] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L717] - **8 schemas** covering all data objects from document blocks to query logs

[L718] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L719] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L720]

[L721] ### 13.2 Key Novelties

[L722]

[L723] | # | Novelty | Level | Why It's New |

[L724] |---|-----|-----|-----|

[L725] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L726] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge,

CHATGPT 02/09/2026, 09:29:40

Yes. Your uploaded SHIA-RAG report does contain algorithms and Python/pseudocode for identifying/creating Knowledge Nodes and Knowledge Edges. But there is an important distinction:

The report is currently a specification, not an implemented system. It contains proposed algorithms, schemas, prompts, and pseudocode, but the actual system has not been built yet. `filecite\turn33file1\131-L143`

1. Knowledge Node — YES, algorithm/code exists

The report says **Layer 3** is responsible for extracting concepts and creating `KnowledgeNode[]`. `filecite\turn33file8\1665-L671`

The process is essentially:

Document Block → **LLM Extraction** → **KnowledgeUnit** → **Alias Resolution** → **Canonical KnowledgeNode**

The extraction prompt asks the LLM to identify:

- Concepts / technical terms
- Definitions

- Properties
- Relations
- Domain `filecite[turn33file8]L673-L681`

Then the report provides an actual **Python implementation for alias resolution**.

For example, it uses:

MinHash LSH, threshold = 0.7

to detect concepts that are likely the same:

```
RR
Round Robin
Round-Robin Scheduling
```

Then it groups them and creates one canonical node. `filecite[turn33file4]L274-L325`

It also has **antonym protection**, so similar-looking concepts such as:

```
preemptive
non-preemptive
```

aren't incorrectly merged. `filecite[turn33file4]L294-L325`

There is also code for **definition detection** using:

1. Explicit regex patterns
2. Dependency parsing
3. LLM fallback `filecite[turn33file5]L372-L405`

So for **Knowledge Node creation**, your report has a fairly concrete algorithm.

2. Knowledge Edge — YES, but less concrete

For edges, the report defines:

Layer 4 → Relation Management → KnowledgeEdge[]

The architecture says Layer 4 performs:

- Relation validation
- Ontology validation
- Confidence scoring
- Conflict resolution

and produces typed, scored `KnowledgeEdge` objects. `filecite[turn33file0]L42-L49`

The Layer 3 extraction prompt already asks the LLM to identify relations such as:

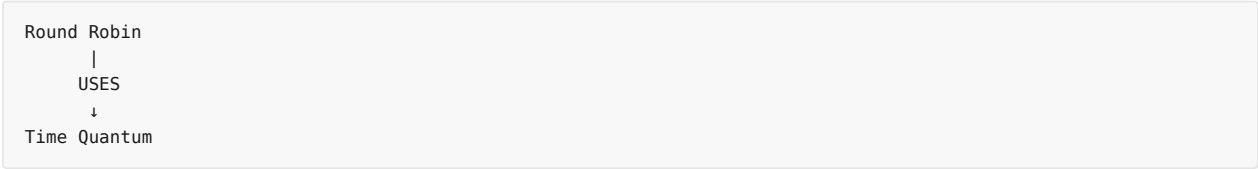
```
IS_A
PART_OF
USES
CAUSES
...
```

with:


```
source
type
target
evidence
```

fileciteurn33file5L362-L369

So conceptually:



becomes a KnowledgeEdge .

The report then says Layer 4 **validates and scores** those candidate relations rather than blindly accepting them.

fileciteurn33file8L665-L671

3. One important thing to tell your guide

Don't say:

"We have implemented the Knowledge Node and Edge extraction."

That would be inaccurate.

Say:

"The report specifies the algorithms and pseudocode for Knowledge Node and Knowledge Edge construction, but implementation is still pending."

This is explicitly supported by the report's current-status section. fileciteurn33file1L135-L143

Overall

Component	In report?	Main approach
Knowledge Node identification	Yes	LLM concept extraction
Definition identification	Yes	Regex + dependency parsing + LLM fallback
Duplicate/alias node merging	Yes	MinHash LSH
Antonym protection	Yes	Negation + CONTRASTS_WITH
Knowledge Edge identification	Yes	LLM relation/triple extraction
Edge validation	Yes	Ontology validation
Edge confidence	Yes	Confidence scoring
Complete production implementation	No	Still specification stage

So yes, your report has a real proposed mechanism for both nodes and edges—and the strongest concrete code is currently on the Knowledge Node side, especially the alias_resolution() implementation.

YOU 02/09/2026, 09:36:24

mujhe complete depth me smjhao ki knowledge node kese bnate hai code bhi do

CHATGPT 02/09/2026, 09:36:27

Bilkul. Main tumhare uploaded SHIA-RAG report ko basis bana kar Knowledge Node creation ko zero se implementation-level depth tak samjhaunga. Sabse important baat: report mein jo actually specified hai usi ko follow karenge; jahan report implementation detail nahi deti, main clearly bataunga.

1. Sabse pehle: Knowledge Node hota kya hai?

SHIA-RAG mein **Knowledge Node** = ek meaningful concept/entity ka structured representation.

For example document mein ye text hai:

"Round Robin is a preemptive CPU scheduling algorithm. It assigns a fixed time quantum to each process in circular order."

Hum simply "Round Robin" ko string ke form mein store nahi karte.

Instead:

```
KnowledgeNode
├── canonical_name: Round Robin
├── aliases:
│   ├── RR
│   └── Round-Robin Scheduling
├── definition:
│   └── A preemptive CPU scheduling algorithm...
├── keywords:
│   ├── scheduling
│   ├── time quantum
│   └── preemptive
├── domain:
│   └── Operating Systems
├── properties:
│   └── preemptive = true
└── evidence:
    ├── source document
    ├── source block
    └── page
```

Your report defines this kind of `KnowledgeNode` schema explicitly. [filecite\turn33file7\L548-L584](#)

2. Knowledge Node exactly kis layer mein banta hai?

Flow:

```
Raw Document
  ↓
Layer 1
Normalization
  ↓
Layer 2
Document Blocks
  ↓
Layer 3
Knowledge Extraction
  ↓
Candidate Knowledge Units
  ↓
Alias Resolution
  ↓
Canonical Knowledge Nodes
  ↓
Layer 4
Knowledge Edges
```

Report explicitly says Layer 3 takes:

```
DocumentBlock[]
```

and produces:

```
KnowledgeNode[]
```

with concept extraction and alias resolution. `filecite{turn33file0}{L60-L70}`

3. Step 1 — Input kya hai?

Layer 3 ko Layer 2 se `DocumentBlock[]` milta hai.

Suppose Layer 2 gives us:

```
Block 1:
"Round Robin is a preemptive CPU scheduling algorithm."

Block 2:
"It assigns a fixed time quantum to each process in circular order."

Block 3:
"A smaller time quantum improves response time but increases
context switching."
```

Ab Layer 3 ka kaam hai:

In blocks se meaningful concepts identify karna.

4. Step 2 — Concept Extraction

Report mein Layer 3 ke extraction prompt mein LLM ko explicitly kaha gaya hai:

- 1. Concepts identify karo
- 2. Definitions identify karo
- 3. Properties identify karo
- 4. Relations identify karo
- 5. Domain identify karo. `filecite{turn33file8}{L673-L681}`

For example:

Input

```
Round Robin is a preemptive CPU scheduling algorithm.
It assigns a fixed time quantum to each process in circular order.
A smaller time quantum improves response time but increases
context switching.
```

LLM extraction

Potential output:

```
{
  "concepts": [
    {
      "name": "Round Robin"
    },
    {
      "name": "CPU Scheduling"
    },
    {
      "name": "Time Quantum"
    },
    {
      "name": "Response Time"
    },
    {
      "name": "Context Switching"
    }
  ],
  "domain": "Operating Systems"
}
```

But **ye abhi final KnowledgeNodes nahi hain.**

Ye candidate concepts / KnowledgeUnit s hain.

This distinction is very important for your guide.

5. KnowledgeUnit vs KnowledgeNode

Think like this:

KnowledgeUnit

"I found something that looks like a concept."

KnowledgeNode

"I have normalized, deduplicated and structured that concept."

So:

```
Text
↓
Concept Extraction
↓
KnowledgeUnit
↓
Alias Resolution
↓
KnowledgeNode
```

Your report describes Layer 3 as candidate extraction followed by alias resolution to create canonical, deduplicated nodes.

filecite\turn33file8\L665-L671

6. Step 3 — Definition Extraction

Ab maan lo LLM ne identify kiya:

Round Robin

Document mein:

Round Robin is a preemptive CPU scheduling algorithm.

Humein identify karna hai:

```
Concept:
Round Robin

Definition:
A preemptive CPU scheduling algorithm
```

Your report actually provides a definition-detection function.

It uses three levels:

- ```
1. Explicit patterns
 ↓
2. Dependency parsing
 ↓
3. LLM fallback
```

filecite\turn33file5\L372-L405

### Explicit patterns

Report gives patterns such as:

```
r'^(.+?)\s+is defined as\s+(.+)'
r'^(.+?)\s+refers to\s+(.+)'
r'^(.+?):\s+[A-Z](.+)'
r'^(.+?)\s+is a\s+(type|kind|form|category|class)\s+of\s+(.+)'
```

filecite\turn33file5\L389-L400

So we can implement:

```
import re

def is_definition(text: str):
 patterns = [
 r'^(.+?)\s+is defined as\s+(.+)',
 r'^(.+?)\s+refers to\s+(.+)',
 r'^(.+?):\s+[A-Z](.+)',
 r'^(.+?)\s+is a\s+(type|kind|form|category|class)\s+of\s+(.+)'
]

 for pattern in patterns:
 match = re.match(pattern, text, re.IGNORECASE)

 if match:
 return {
 "is_definition": True,
 "concept": match.group(1).strip(),
 "definition": match.group(2).strip()
 }

 return {
 "is_definition": False,
 "concept": None,
 "definition": None
 }
```

Example:

```
text = "Round Robin is a type of CPU scheduling algorithm"

print(is_definition(text))
```

Conceptually gives:

```
is_definition = True
concept = Round Robin
definition = CPU scheduling algorithm
```

## 7. But definition detection alone is NOT enough

This is an important point.

Suppose:

The sky is blue today.

A naive regex:

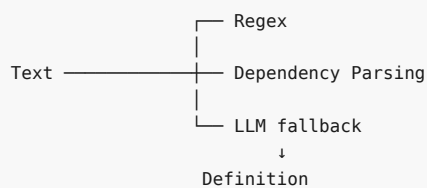
"X is Y"

would incorrectly say:

```
Concept = sky
Definition = blue today
```

Your report specifically points out this problem and therefore proposes dependency parsing + LLM fallback instead of relying only on simple regex. [filecite\[turn33file5L372-L385\]](#)

So the real architecture is:



## 8. Step 4 — Properties extraction

Suppose document says:

Round Robin is preemptive.  
The time quantum is 20 ms.

We don't want only:

Round Robin

We also want structured properties.

For example:

```
{
 "preemptive": {
 "type": "boolean",
 "value": true
 },
 "time_quantum": {
 "type": "integer",
 "value": 20,
 "unit": "ms"
 }
}
```

This structure is actually shown in the report's `KnowledgeNode` example. `filecite[turn33file7][L564-L570]`

## 9. Step 5 — Alias problem

This is one of the **most important parts of your Knowledge Node algorithm**.

Suppose across documents we find:

```
Round Robin
RR
Round-Robin
Round-Robin Scheduling
```

Are these four different concepts?

No.

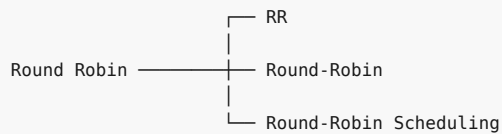
They refer to the same concept.

If we directly create nodes:

```
Node 1 → Round Robin
Node 2 → RR
Node 3 → Round-Robin
Node 4 → Round-Robin Scheduling
```

then our Knowledge Forest becomes polluted with duplicate nodes.

Instead we want:



and finally:

```
ONE KnowledgeNode
```

with:

```
{
 "canonical_name": "Round Robin",
 "aliases": [
 "RR",
 "Round-Robin",
 "Round-Robin Scheduling"
]
}
```

## 10. How does SHIA-RAG solve this?

Your report specifically proposes:

### MinHash + LSH

with:

```
threshold = 0.7
num_perm = 128
```

The report's algorithm is:

1. Build MinHash LSH index
2. Check antonyms
3. Find connected components
4. Select canonical name
5. Merge aliases/definitions/properties/evidence
6. Create canonical node

filecite\turn33file4\L274-L325

## 11. MinHash LSH ko simple language mein samjho

Suppose:

"Round Robin"

and

"Round-Robin Scheduling"

Exact string comparison:

Round Robin == Round-Robin Scheduling

× False.

But semantically they are related.

MinHash LSH helps us efficiently find **near-duplicate strings** without comparing every concept with every other concept.

Instead of:

```
Node 1 ↔ Node 2
Node 1 ↔ Node 3
Node 1 ↔ Node 4
...
```

LSH creates buckets of similar candidates.

## 12. Why LSH?

Suppose you have:

100,000 KnowledgeUnits

Naïvely comparing every pair means approximately:

$O(N^2)$

comparisons.

Your report explicitly identifies LSH as a mitigation for expensive canonicalization at scale. filecite\turn33file1\L118-L127

## 13. The actual alias-resolution code from your report

Your report provides this core function:



```

def alias_resolution(units: list, relation_cache: dict) -> list:
 """Merge KnowledgeUnits referring to the same concept."""

 import re

 NEGATION_PREFIXES = re.compile(
 r'^(non-?|un|in|dis)',
 re.IGNORECASE
)

 lsh = MinHashLSH(
 threshold=0.7,
 num_perm=128
)

 # Build LSH index
 # ...

 raw_groups = find_connected_components(
 units,
 lsh
)

 safe_groups = []

 for group in raw_groups:

 names = [
 u.name.lower()
 for u in group
]

 has_antonym = False

 for i, n1 in enumerate(names):

 for n2 in names[i+1:]:

 if NEGATION_PREFIXES.sub('', n1) == \
 NEGATION_PREFIXES.sub('', n2):

 has_antonym = True

 if relation_cache.get(
 (n1, n2)
) == 'CONTRASTS_WITH':

 has_antonym = True

 if has_antonym:
 safe_groups.extend(
 [[u] for u in group]
)
 else:
 safe_groups.append(group)

 return [
 create_canonical_node(group)
 for group in safe_groups
]

```

This is not something I invented; it is in your uploaded report. [filecite\turn33file4\L283-L325](#)

## 14. Why Antonym Protection?

This is actually a very good safeguard in your architecture.

Consider:

```
preemptive
non-preemptive
```

They are textually similar.

A similarity algorithm might say:

```
Similarity = HIGH
```

and try to merge them.

But:

```
preemptive ≠ non-preemptive
```

They are opposite concepts.

Therefore your report adds two checks:

### Check 1 — Negation prefix

```
non-
un-
in-
dis-
```

### Check 2 — Existing `CONTRASTS_WITH` relation

If:

```
A CONTRASTS_WITH B
```

then:

```
DO NOT MERGE
```

filecite turn33file4 L294-L325

## 15. Step 6 — Connected Components

Suppose LSH finds:

```
Round Robin ↔ RR
Round Robin ↔ Round-Robin
RR ↔ Round-Robin Scheduling
```

We can represent this as:

```
RR
|
├─ Round Robin
└─ Round-Robin
 |
 └─ Round-Robin Scheduling
```

These belong to one connected group.

So:

```
Group 1 =
{
 RR,
 Round Robin,
 Round-Robin,
 Round-Robin Scheduling
}
```

Then we create **one canonical node**.

---

## 16. Step 7 — Choosing canonical name

Your report says:

*Canonical name = most frequent name across units.*

So if documents contain:

```
Round Robin → 15 times
RR → 8 times
Round-Robin Scheduling → 3 times
```

we choose:

```
canonical_name = "Round Robin"
```

and:

```
aliases = [
 "RR",
 "Round-Robin Scheduling"
]
```

The report explicitly specifies this merge strategy. `filecite{turn33file4}{L287-L292}`

---

## 17. Step 8 — Merge information

Now suppose we have:

### Unit 1

```
Name:
Round Robin

Definition:
Preemptive CPU scheduling algorithm

Evidence:
Document A
```

### Unit 2

```
Name:
RR

Definition:
Scheduling algorithm using fixed time slices

Evidence:
Document B
```

After merging:

```
KnowledgeNode
├── canonical_name
│ Round Robin
├── aliases
│ RR
│ Round-Robin Scheduling
├── definitions
│ Definition 1
│ Definition 2
├── properties
│ merged properties
├── confidence
│ average confidence
└── evidence
 Document A
 Document B
```

Your report specifies:

- aliases → union
- definitions → union
- confidence → average
- properties → merged
- conflicts → logged. `filecite\turn33file4\L287-L292`

---

## 18. Step 9 — Embedding of the final node

Your report has another important implementation detail.

It says:

*Do NOT average embeddings when merging nodes.*

Instead:

```
Merged Node
↓
Canonical name + definition
↓
Re-embedding
```

For example:

```
Round Robin
+
Definition
```

becomes the input to the embedding model.

The report provides:

```
def merge_node_embeddings(merged_node, embed_fn):

 text_to_embed = merged_node.canonical_name

 if merged_node.definitions:
 text_to_embed += " " + \
 merged_node.definitions[0]

 merged_node.embedding = embed_fn(
 text_to_embed
)
```

filecite turn33file4 L328-L346

### Why?

Because:

```
Embedding(RR)
+
Embedding(Round Robin)
```

averaging can produce a vector that doesn't represent either concept properly.

So:

```
Merged semantic representation
 ↓
Re-embed
```

is preferred.

## 19. Final KnowledgeNode

After all these steps, we finally create something like:

```

{
 "node_id": "NODE_000789",

 "canonical_name": "Round Robin",

 "aliases": [
 "RR",
 "Round-Robin Scheduling"
],

 "definitions": [
 "A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"
],

 "summaries": [
 "CPU scheduling method using time slices"
],

 "keywords": [
 "scheduling",
 "time quantum",
 "preemptive",
 "CPU"
],

 "domain": "Operating Systems",

 "properties": {
 "time_quantum": {
 "type": "integer",
 "value": 20,
 "unit": "ms"
 },
 "preemptive": {
 "type": "boolean",
 "value": true
 }
 },

 "examples": [
 "Process P1 gets 20ms, then P2 gets 20ms"
],

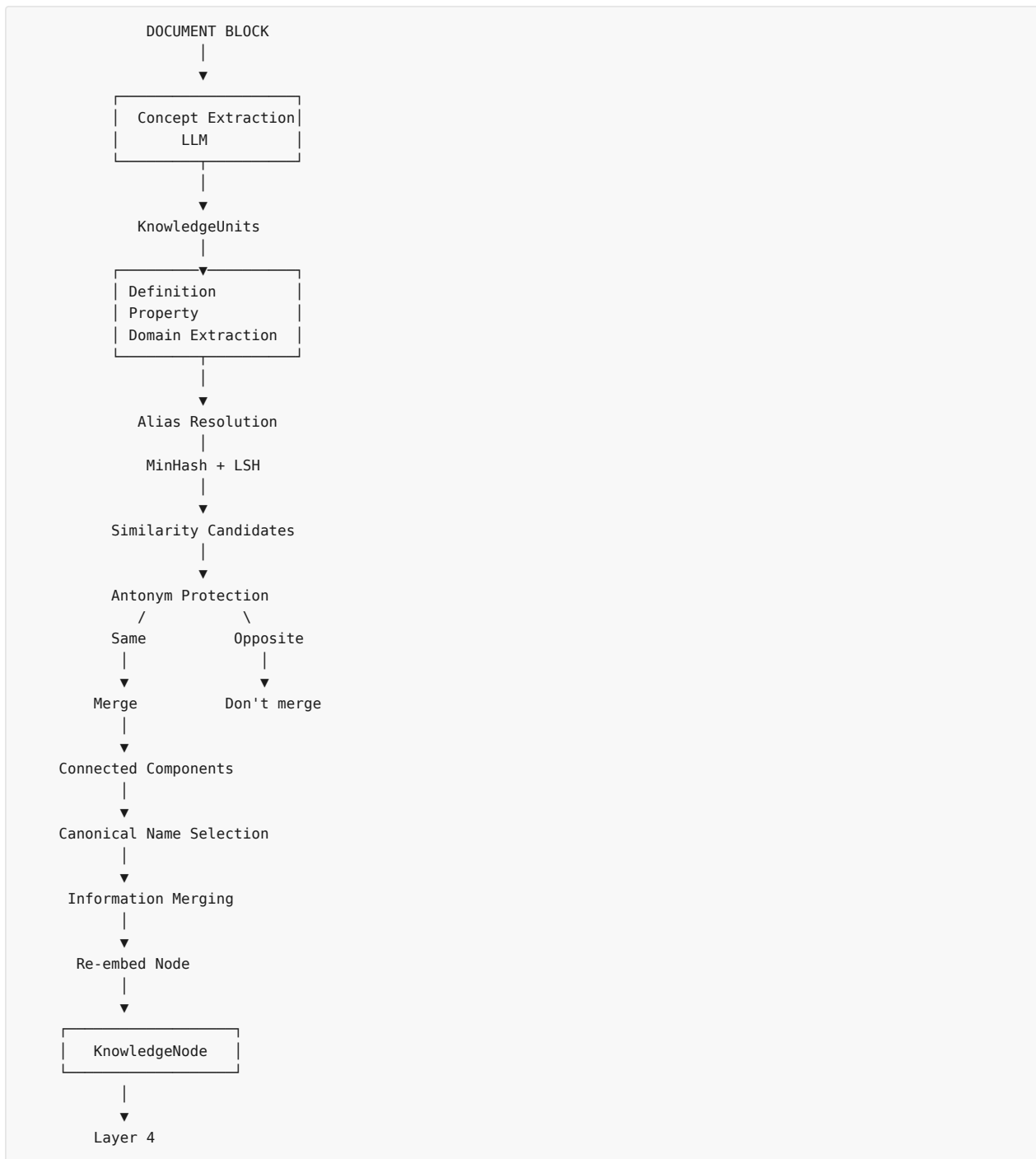
 "evidence": [
 {
 "evidence_type": "DEFINITION",
 "text": "Round Robin is a preemptive scheduling algorithm...",
 "source_doc": "DOC_000001",
 "source_block": "BLK_000234",
 "page": 42
 }
]
}

```

This structure is based directly on the `KnowledgeNode` schema in your report. [filecite\turn33file7\L552-L584](#)

## 20. Complete pipeline in one diagram

This is the **most important thing to remember** for your guide:



## 21. If you want to implement this yourself

A practical implementation can be organized like this:

```

layer3/
├── extraction.py
├── definition_extractor.py
├── alias_resolution.py
├── node_builder.py
├── embedding.py
└── schemas.py

```

And the main pipeline:

```
def build_knowledge_nodes(document_blocks):

 # 1. Extract candidate concepts
 units = extract_knowledge_units(
 document_blocks
)

 # 2. Extract definitions/properties
 units = enrich_units(units)

 # 3. Resolve aliases
 nodes = alias_resolution(
 units,
 relation_cache={}
)

 # 4. Generate embeddings
 for node in nodes:
 node.embedding = generate_embedding(node)

 # 5. Validate nodes
 nodes = validate_nodes(nodes)

 return nodes
```

Conceptually:

```
DocumentBlock[]
 ↓
extract_knowledge_units()
 ↓
KnowledgeUnit[]
 ↓
enrich_units()
 ↓
KnowledgeUnit[]
 ↓
alias_resolution()
 ↓
KnowledgeNode[]
 ↓
generate_embedding()
 ↓
Final KnowledgeNode[]
```

## 22. One correction that's important for your project

Your report says:

*Layer 3 performs candidate extraction; Layer 4 performs relation validation and confidence scoring.*

So don't mix these two.

### Layer 3

```
"What concepts exist?"

"What are their definitions?"

"What properties do they have?"

"What candidate relations appear in the text?"
```

### Layer 4



"Is this relation valid?"

"What type of relation is it?"

"How confident are we?"

This boundary is explicitly stated in the report. [filecite\turn33file8\L665-L671](#)

---

## 23. The simplest way to explain this to your guide

You can say:

*"In SHIA-RAG, Knowledge Nodes are created in Layer 3. First, we use an LLM to extract candidate concepts, their definitions, properties, and domain information from document blocks. These candidates are represented as Knowledge Units. Since the same concept can appear with different names, we perform alias resolution using MinHash-LSH. Similar candidates are grouped, while antonym protection prevents opposite concepts from being merged. We then select a canonical name, merge the associated definitions, properties, aliases, and evidence, and re-generate the embedding for the merged concept. The resulting structured and deduplicated object becomes a Knowledge Node."*

That's the **core story** of your Knowledge Node creation.

And importantly, according to your report, **this is currently a specified architecture with pseudocode, not a completed implementation.** [filecite\turn33file1\L135-L143](#)